System Design Patterns — Vol 2: The Patterns

Modules 1-11: the five-questions method, moving data, storing data, caching, reliability, scaling, consistency, API and edge, operations, batch and streaming, AI-era systems

11 modules · 60+ patterns · every lesson explained, with course diagrams and links to the originals

what it is | problem | how it works | trade-offs | real world | interview | self-check | TL;DR

Personal study notes summarised from the DesignGurus course (Arslan Ahmad). Not for redistribution.

Contents

1. **Module 1 — Thinking in Patterns (Foundations)**
   - Course Introduction
   - Thinking in Patterns
   - The Five Questions
   - Back-of-Envelope Math
2. **Module 2 — Moving Data (Communication)**
   - Module Introduction
   - Request-Response
   - Message Queue
   - Publish-Subscribe
   - Event-Driven Architecture
   - Webhooks
   - Server-Sent Events
   - Bidirectional Streaming
   - Capstone - Designing a Notification System
3. **Module 3 — Storing Data (Storage and Replication)**
   - Module Introduction
   - Primary-Replica
   - Sharding
   - Consistent Hashing
   - Write-Ahead Log
   - Event Sourcing
   - CQRS
   - Capstone - Scaling a Database to 100 Million Users
4. **Module 4 — Serving Data Fast (Caching)**
   - Module Introduction
   - Cache-Aside
   - Read-Through
   - Write-Through
   - Write-Behind
   - Cache Stampede Prevention
   - Capstone - Caching the Product Catalog
5. **Module 5 — Surviving Failure (Reliability)**
   - Module Introduction
   - Timeout
   - Retry with Exponential Backoff
   - Idempotency
   - Circuit Breaker
   - Bulkhead
   - Dead Letter Queue
   - Graceful Degradation
   - Capstone: Hardening a Payment Flow
6. **Module 6 — Growing Under Load (Scaling)**
   - Vertical Scaling
   - Horizontal Scaling
   - Load Balancing
   - Auto-Scaling
   - Database Connection Pooling
   - Multi-Region and Cell-Based Architecture
   - Capstone - From 100 to 1,000,000 Requests per Second
7. **Module 7 — Keeping Data Consistent (Distributed Consistency)**
   - Module Introduction
   - Two-Phase Commit
   - Saga
   - Quorum
   - Vector Clocks
   - Capstone - Order Fulfillment, Built Twice
8. **Module 8 — The Entry Point (API and Edge)**
   - Module Introduction
   - Reverse Proxy
   - CDN
   - API Gateway
   - Backend for Frontend
   - Rate Limiting
   - Cursor-Based Pagination
   - API Versioning
   - Sidecar
   - Service Mesh
   - Authorization Service
   - Capstone - The Entry Point, Assembled
9. **Module 9 — Operating in Production (Observability and Delivery)**
   - Module Introduction
   - Health Check Endpoint
   - Distributed Tracing
   - Blue-Green Deployment
   - Canary Deployment
   - Feature Flags
   - Containers and Kubernetes
   - CI/CD Pipelines and Deployment Strategies
   - Capstone - The Zero-Downtime Migration
10. **Module 10 — Data-Intensive Systems (Batch and Streaming)**
    - Module Introduction
    - MapReduce
    - Stream Processing
    - Lambda and Kappa
    - Change Data Capture
    - Exactly-Once Semantics

- Backpressure
- Partitioned Consumption
- Inside Kafka
- Capstone - Clickstream to Dashboard

11. **Module 11 — AI-Era Systems (LLM and ML Infrastructure)**
    - Module Introduction
    - Feature Store
    - Model Serving (and Batching)
    - GPU Auto-Scaling
    - LLM Gateway
    - Semantic Caching
    - Vector Database Sharding
    - RAG Pipeline
    - Capstone — A Production RAG Chatbot at Scale

Module 1 — Thinking in Patterns (Foundations)

This module sets up the *method* every later module uses: what a pattern is, five fixed questions to ask of any design, and four quick calculations that show whether a design is big enough (or bigger than it needs to be). One example runs through the course: an online store that starts on a single database and grows to 100 million users, with each module tackling its next problem.

Course Introduction

Original lesson: Course Introduction on DesignGurus ↗

What it is

Most large systems are assembled from the same small set of recurring solutions, called **system design patterns**; the course covers more than 60. Once you know them, an unfamiliar architecture diagram stops being a pile of boxes: you can name each part ("a cache, a queue") and build new systems from pieces you understand instead of memorizing whole diagrams.

> **Analogy:** strong chess players don't memorize games; they learn recurring tactics (forks, pins) and spot them in new positions. Patterns are the forks and pins of system design.

Key terms

- **System design pattern:** a reusable solution to a recurring problem, bundled with the price you pay for using it.
- **Pattern vs technology:** a technology is a product; a pattern is the idea it implements. *Redis* (an in-memory data store) is a technology. *Cache-aside* is a pattern: keep a fast temporary copy of frequently used data (a **cache**), look there first, and go to the slower database only when the copy lacks the data (a **cache miss**). Products come and go; patterns keep their value.
- **Server / database / HTTP request / client:** all you need going in. A server answers requests over the network; a database stores data that survives restarts; an HTTP request is the standard message a **client** (browser, app, or another server) sends to a web server. If any feel shaky, see System Design Fundamentals.

Who it's for

Early- to mid-career software engineers, plus data and AI engineers whose work spans many machines and hits the same problems. No **distributed systems** background is assumed (systems of many computers cooperating over a network that must keep working when some fail); every term is defined at first use.

The twelve modules

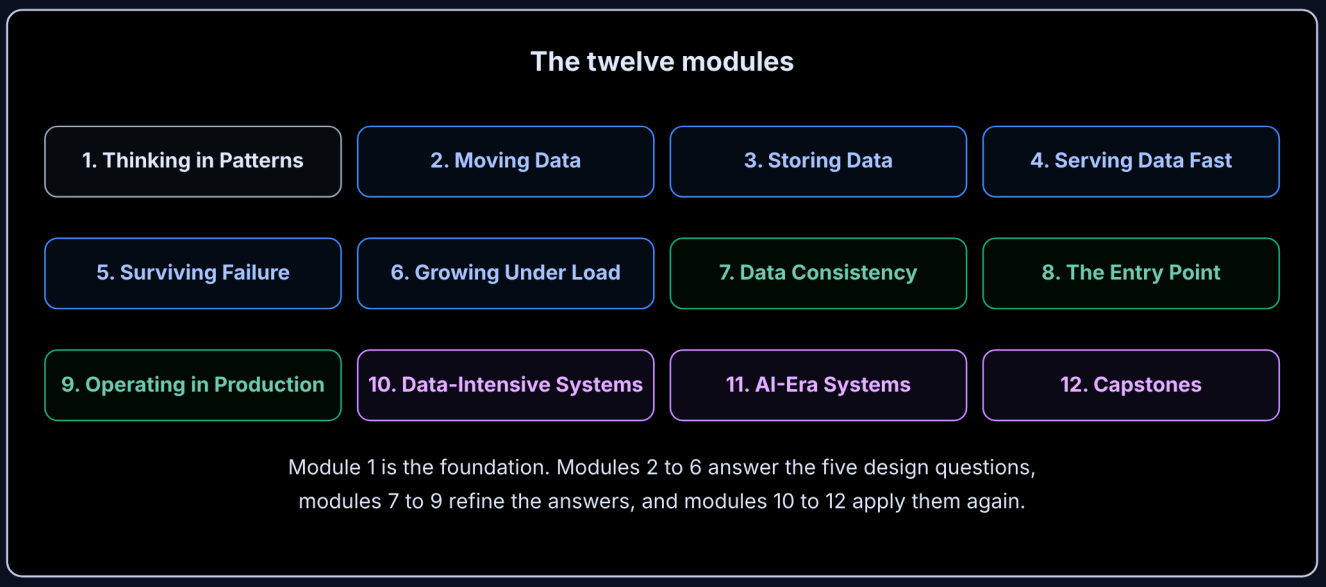


Figure — The twelve modules (course diagram)

Module 1 lays the groundwork, Modules 2–6 each answer one of the five design questions below, 7–9 sharpen those answers, and 10–12 apply everything to larger systems.

Module	What it covers
1. Thinking in Patterns	What a pattern is, plus the simple math used everywhere
2. Moving Data	How services talk: request-response, queues, pub/sub, streams
3. Storing Data	Replication (copies on several machines), sharding (splitting data across machines) and logs
4. Serving Data Fast	Caching and its price
5. Surviving Failure	Calls that hang, fail, or arrive twice
6. Growing Under Load	Scaling, load balancing, connection pooling
7. Keeping Data Consistent	Transactions (all-or-nothing changes), sagas (multi-step workflows with undo steps), quorums (majority votes)
8. The Entry Point	Gateways (front door for requests), rate limits (caps on requests/s), CDNs (servers near users holding copies of content), proxies (middlemen forwarding requests)
9. Operating in Production	Deploying, monitoring, rolling back
10. Data-Intensive Systems	Processing events in batches (large chunks) and streams (one by one as they arrive)
11. AI-Era Systems	The same patterns inside LLM (large language model, like ChatGPT) and ML (machine learning) systems
12. Capstones	Four full designs pulling everything together

Added in these notes (not part of the course): five extra lessons in modules 6, 8, 9 and 10, and six case-study modules — 13 Building Blocks, 14 Messaging and Media, 15 Location and Real-Time, 16 Infrastructure, 17 Money and Reservations and 18 Client and Mobile — applying the same patterns to classic "design X" prompts.

How each lesson works

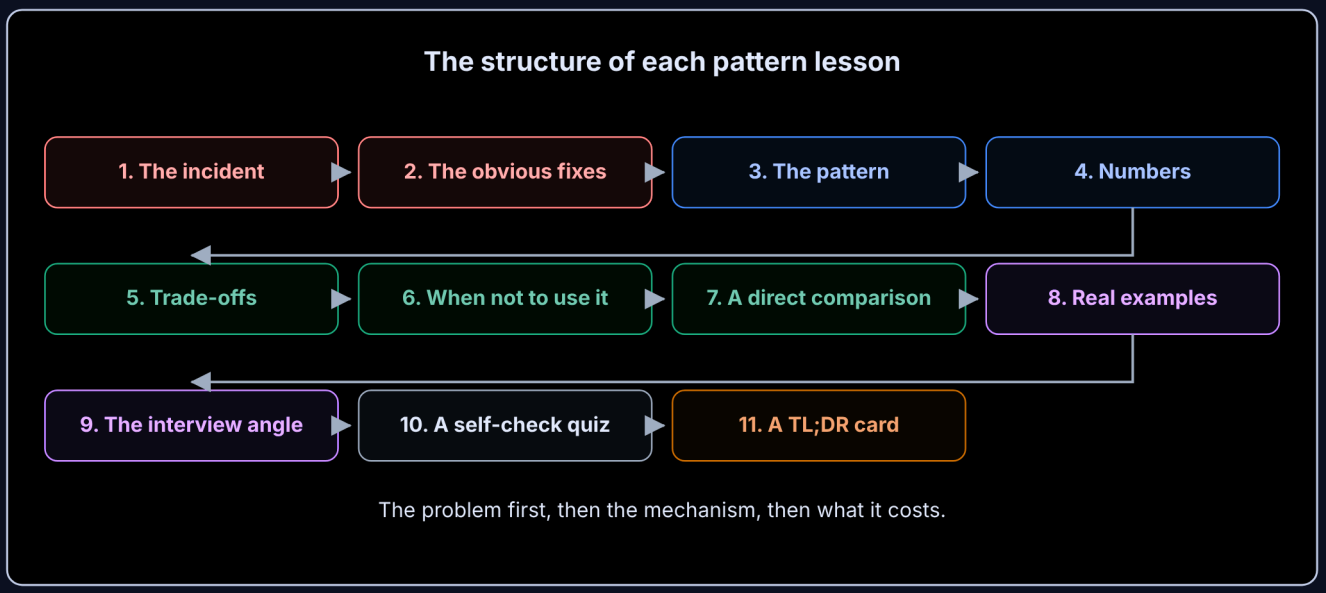


Figure — The structure of each pattern lesson (course diagram)

Every pattern lesson follows the same 11 steps: problem first, then the fix, then its cost.

1. **The incident:** a believable failure in **production** (the live system real users use), with real numbers.
2. **The obvious fixes** engineers try first, and why they fall short.
3. **The pattern**, step by step.
4. **Walkthrough with numbers** on the same incident.
5. **Trade-offs:** the cost, delay or complexity it adds.
6. **When not to use it.**
7. **Direct comparison** with the pattern it's most often confused with.
8. **Real-world examples** from production systems.
9. **Interview angle**, with a rehearsed 30-second answer.
10. **Self-check quiz.**
11. **TL;DR card** ("too long; didn't read"): together the cards form a night-before-the-interview review deck.

Lessons are full or compact (simpler patterns have fewer decisions). Each module ends with a **mini-capstone**: a small design using that module's patterns together.

This module

Three lessons build the method: **Thinking in Patterns** (what a pattern is and why it beats memorized architectures), **The Five Questions** (turning a fuzzy prompt into a plan), and **Back-of-the-Envelope Math** (four reusable calculations). Request-Response, next module, is the first place the method is applied.

Thinking in Patterns

Original lesson: [Thinking in Patterns on DesignGurus ↗]

What it is

A way of reasoning about design: instead of recalling whole architectures, split the system into *problems you already know*, pick the standard solution for each, and state what each costs. **Analogy:** a doctor doesn't memorize patients; they map symptoms to known conditions and know each treatment's side effects.

The problem: two candidates, one question

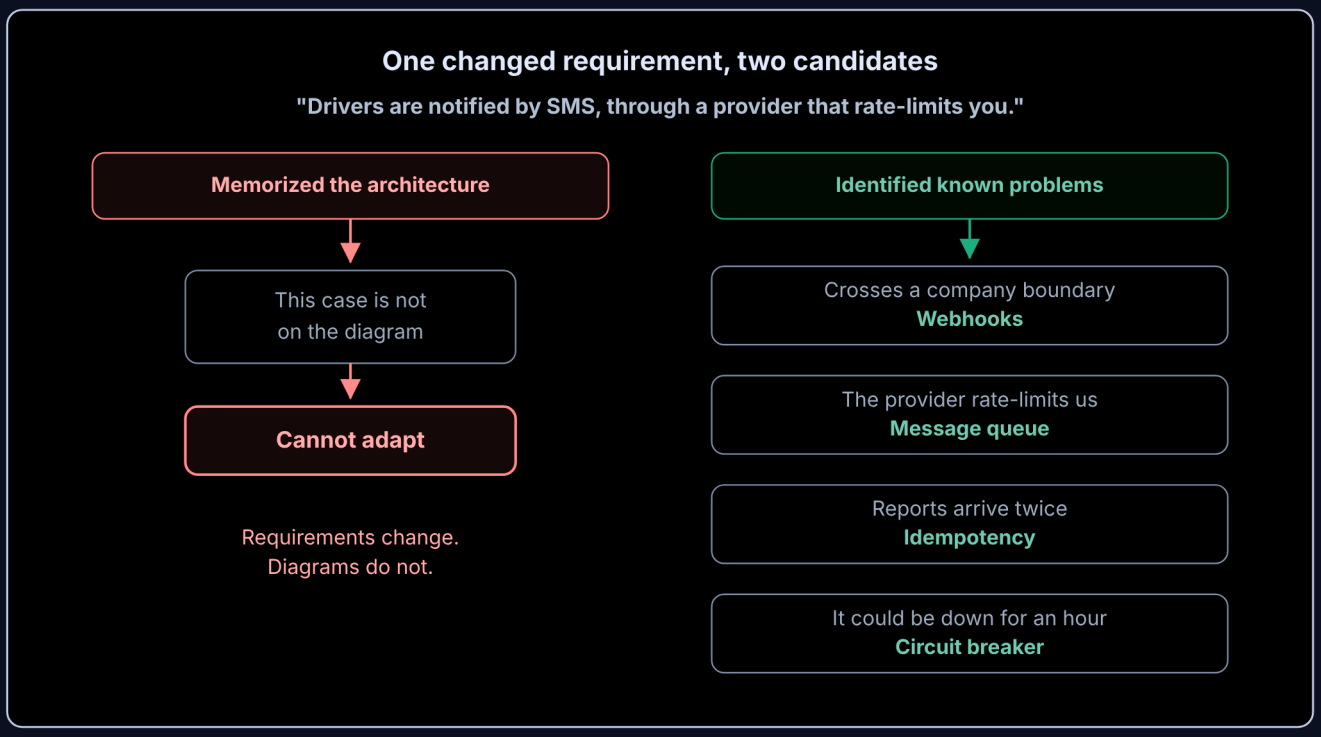


Figure — One changed requirement, two candidates (course diagram)

Two candidates get "design a ride-sharing service." The first redraws Uber's published architecture from memory. Then the interviewer changes one requirement: *driver alerts now go out as SMS through a third-party provider that rate-limits you.* The memorized diagram has no answer, so the first candidate is stuck.

The second splits the change into four known problems:

- **Work crosses into another company.** The provider sends delivery reports back as **webhooks**: HTTP requests the provider makes into *your* server.
- **The provider rate-limits you** (caps requests per second), so messages go on a **message queue** and are sent at a steady, controlled rate.
- **A report may arrive twice**, so handling must be **idempotent**: doing it twice leaves the same result as doing it once.
- **The provider may be down for an hour**, so a **circuit breaker** pauses calls to it and the app alerts users in-app instead.

Four patterns, each with its cost stated; when the next requirement changes, the same reasoning runs again. As the figure says: *requirements change; diagrams do not* — on the job as much as in interviews.

What a pattern is

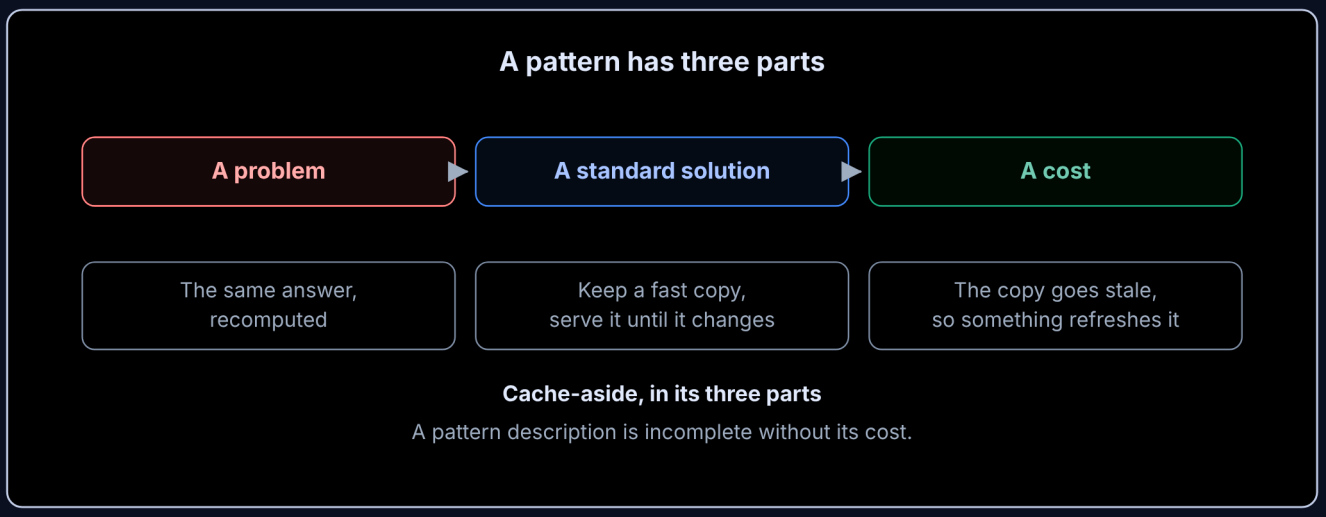


Figure — A pattern has three parts (course diagram)

Every pattern has three parts (shown with cache-aside); missing one means the description is incomplete:

1. **A problem in general terms.** Not "Instagram is slow" but "the same unchanged answer is recomputed thousands of times," which any product can have.
2. **A standard solution.** Keep a fast copy of the answer and serve it until the answer changes.
3. **A cost.** The copy can go **stale** (out of date), so something must refresh it. The cost is part of the pattern, not a detail for later.

Redis, Kafka (a message-log product) and Postgres (a relational database) don't define patterns; they are products that implement them well today.

Patterns predate software. Paper bank ledgers are event sourcing (record every change, derive the balance). Ship bulkheads keep one flooded compartment from sinking the ship. Post offices had a dead letter process for undeliverable mail. About sixty patterns cover so much because every system hits the same limits: networks break, machines have finite capacity, copies drift apart, load arrives unevenly.

Patterns combine into architectures

An architecture is several patterns working together; Netflix is roughly fifteen. Caches cut repeated reads, queues decouple one **service** (a separately running program owning one job, such as payments) from another, circuit breakers guard calls to its **dependencies** (systems it needs to do its job), and replicas keep extra data copies. So "design X" becomes three steps:

1. **Find the problems.** A news feed has **fan-out** (one post must reach many followers) and **hot keys** (one item draws a huge share of traffic); checkout has payment and third-party problems.
2. **Pick a pattern per problem, then check interactions. Retries** (automatically resending a failed request) need idempotency or a retried payment double-charges. Caches need an expiry rule. **Shards** (pieces of a database on different machines) need a **shard key**: the field that decides which piece stores each row.
3. **Name every cost**, and decline a pattern when the problem isn't big enough to justify it.

Each module's capstone runs these steps on a small system; the last module runs them on four full designs.

Every pattern is a trade

The one habit to keep: no pattern makes everything better.

Pattern	What it gives	What it costs
Queue	Absorbs traffic bursts	Adds waiting time; a message may arrive twice
Cache	Fast reads	May serve out-of-date data
Sharding	Room for more writes	Cross-shard queries become hard
Circuit breaker	Fails fast instead of hanging	May "trip" when the dependency could still have replied

Hence every lesson's "When Not to Use It." Weak answer: "We'll use Kafka here." Strong answer: "A queue fits because no one waits on this work. Queues can deliver twice, so the **consumer** (the program reading from the queue) dedupes; they can reorder messages, which this operation tolerates." That covers **problem, pattern and cost**. Juniors know what patterns do; seniors also know each one's price and when to turn it down.

How interviews test patterns

Interviewers rarely ask for definitions. They test:

- **Spotting problems from symptoms.** Every **health check** (an automatic "are you alive?" probe) is green, yet one feature is starved of resources: think "bulkhead" (give each feature its own resource pool) before they say it. That's why lessons open with an incident.
- **Pattern interactions.** "Could a retry charge someone twice?" "What happens when your busiest cache key expires?"
- **Defending choices.** "Why not X everywhere?" means "what does X cost?" Answers without trade-offs sound memorized.

How to study

- **Recognize, don't recite:** at each incident, diagnose before reading the fix.
- **Pair every pattern with its cost** in one sentence (the TL;DR cards help).
- **Practice on apps you use daily** with the five questions (next lesson).
- **Say the cost out loud** every time you name a pattern.

Interview one-liner

"I don't recall architectures. I split the system into problems I already know, choose one pattern per problem, look at how those patterns affect each other, and name the cost of each. A changed requirement just means rerunning the same reasoning."

TL;DR

- **The claim:** ~60 recurring patterns underlie every system design; knowing them beats memorizing architectures, because requirements move and diagrams can't.
- **A pattern is:** problem + standard solution + cost. Technologies implement patterns; they don't replace them.
- **Designing means:** find the problems, map each to a pattern, check interactions, state every cost.
- **The senior habit:** every pattern is a trade-off; interviews reward knowing the price and when to decline.
- **Study by:** diagnosing incidents before reading fixes, and never naming a pattern without its cost.

The Five Questions

Original lesson: [The Five Questions on DesignGurus ↗]

What it is

A checklist of five questions, always in the same order, that you point at any system. Each question surfaces problems; each problem leads to patterns. **Analogy:** a pilot's pre-takeoff checklist — the pilot can fly; the checklist ensures nothing is forgotten under stress.

The problem

"Design a URL shortener" (a service like bit.ly that maps a long URL to a short code and redirects visitors), then silence. Most candidates freeze not from lack of knowledge but lack of procedure. A fixed order of questions means you always know the next step; this lesson shows *where to look* for patterns.

Step 0 — get the numbers first: user count, **RPS** (requests per second), data size, **read:write ratio** (reads per write), and the cost of an outage. A design for 100 users won't suit 100 million (the next lesson shows how to estimate these quickly).

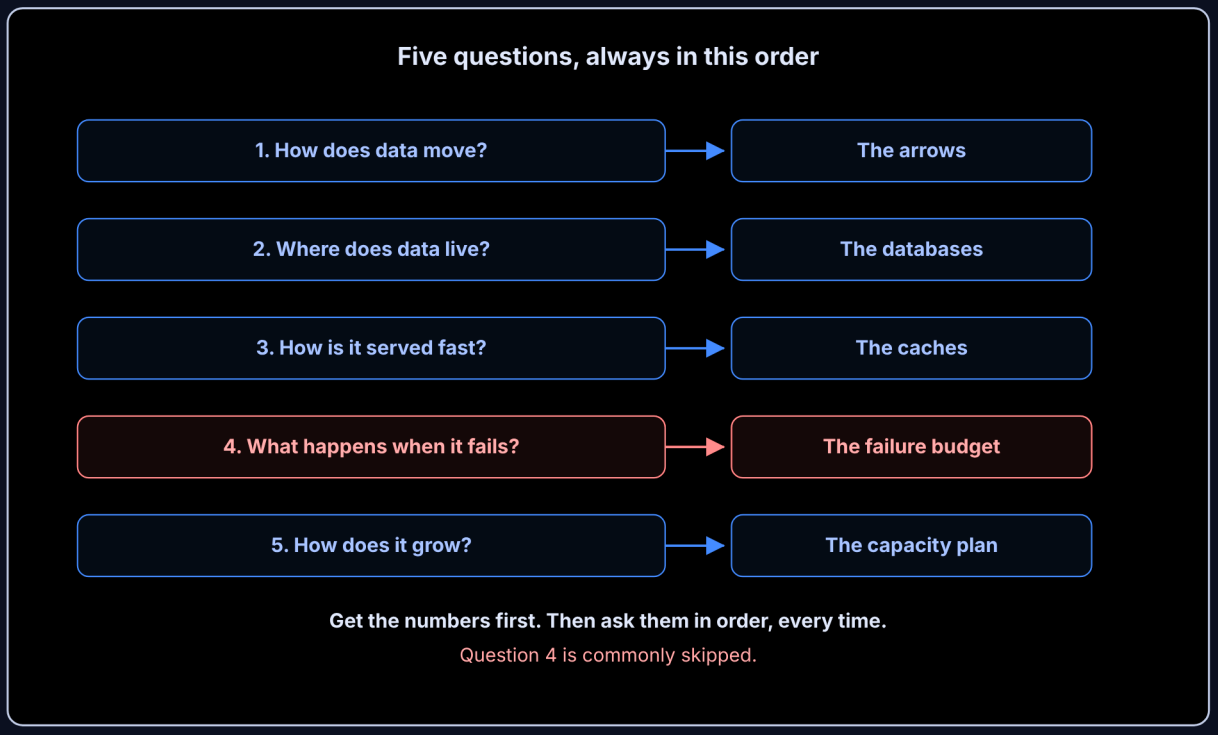


Figure — Five questions, in order (course diagram)

Each question fills in one part of the diagram: arrows, databases, caches, failure budget, capacity plan. Note the figure's warning: question 4 is the most skipped.

The five questions, one by one

1. How does data move? → the arrows. Who talks to whom, and does the caller wait? Must the work happen exactly once? Do several services care about the same fact? Does the call leave your company? Must data flow continuously? Options (Module 2):

- **Request-response** (default): the caller waits for the reply.
- **Message queue**: non-urgent work is held until a worker takes it.
- **Pub/sub** (publish-subscribe): one event delivered to every interested service.
- **Webhooks**: one company's system makes HTTP calls into another's.
- **Streams**: the server pushes data continuously.

Example: "Order confirmation must be immediate, so a direct call. Fulfillment can lag, so a queue. Five teams care about each order event, so we publish it."

2. Where does data live? → the databases. Which store is the **source of truth** (the official copy), how big it grows and how fast, the read:write ratio, and whether you need full history or only current state. Options (Module 3):

- **Replicas**: extra database copies, added when reads outgrow one machine.

- **Sharding**: splitting rows across machines, when writes or total size outgrow one.
- **Consistent hashing**: changing the shard count without moving most data.
- Log patterns when history matters: a **write-ahead log (WAL)** records each change before applying it; **event sourcing** stores every event instead of only the final state; **CQRS** uses separate models for writes and reads.

Example: "Customers dispute charges, so we keep order events, not just current state."

3. How is it served fast? → the caches. Which reads repeat, how stale each data type may be, the **hit ratio** (share of reads the cache answers), and what breaks when a hot entry expires. Start with **cache-aside**, use write-based variants for frequently changing data, and add **stampede prevention** for when many requests miss the same hot entry at once. The caching module's key decision: give each data type a **staleness budget** (how old it may be) and pick a mechanism that honors it.

Example: "Descriptions can be an hour old. Prices a few seconds, if updates **invalidate** the cache (delete or mark the stale copy so the next read fetches fresh). At checkout the price must be exact, so it reads the database."

4. What happens when it fails? → the failure budget. For *every arrow* from question 1: what if the other side never answers, fails once, or is down ten minutes? Could a retry cause a double effect such as a charge? What does the user see? Toolbox (Module 5):

- **Timeout**: stop waiting after a deadline.
- **Retry with backoff**: retry, waiting longer each time.
- **Idempotency**: a repeated request can't repeat the effect.
- **Circuit breaker**: stop calling a service during a long outage.
- **Bulkhead**: keep one broken feature from consuming shared resources.
- **Graceful degradation**: return a smaller result instead of an error.

Example: "If the payment provider slows, we cut the call at twice its normal worst-case **latency** (time one call takes end to end). Retries carry **idempotency keys** (a unique request ID the receiver uses to ignore repeats), so no double charge. If failures persist, the breaker opens, and we save the cart and email the customer."

5. How does it grow? → the capacity plan. At 10x traffic, what breaks first: reads, writes, storage, or one hot key? What scales by adding machines, and what needs a redesign? Module 6 covers **load balancing** (spreading requests across servers), **autoscaling** (adding/removing servers automatically with traffic) and **horizontal scaling** (growing by adding machines).

Example: "Replicas and caching handle reads, but the single **primary** (the one server accepting writes; replicas copy from it) caps writes. Next step is sharding by customer ID, planned before the primary hits its limit."

The questions map to Modules 2–6 in order. Later modules add consistency, the entry point and operations; the data and AI modules run the same questions on **pipelines** (chains of steps moving and transforming data) and ML systems.

Walkthrough: the URL shortener in about three minutes

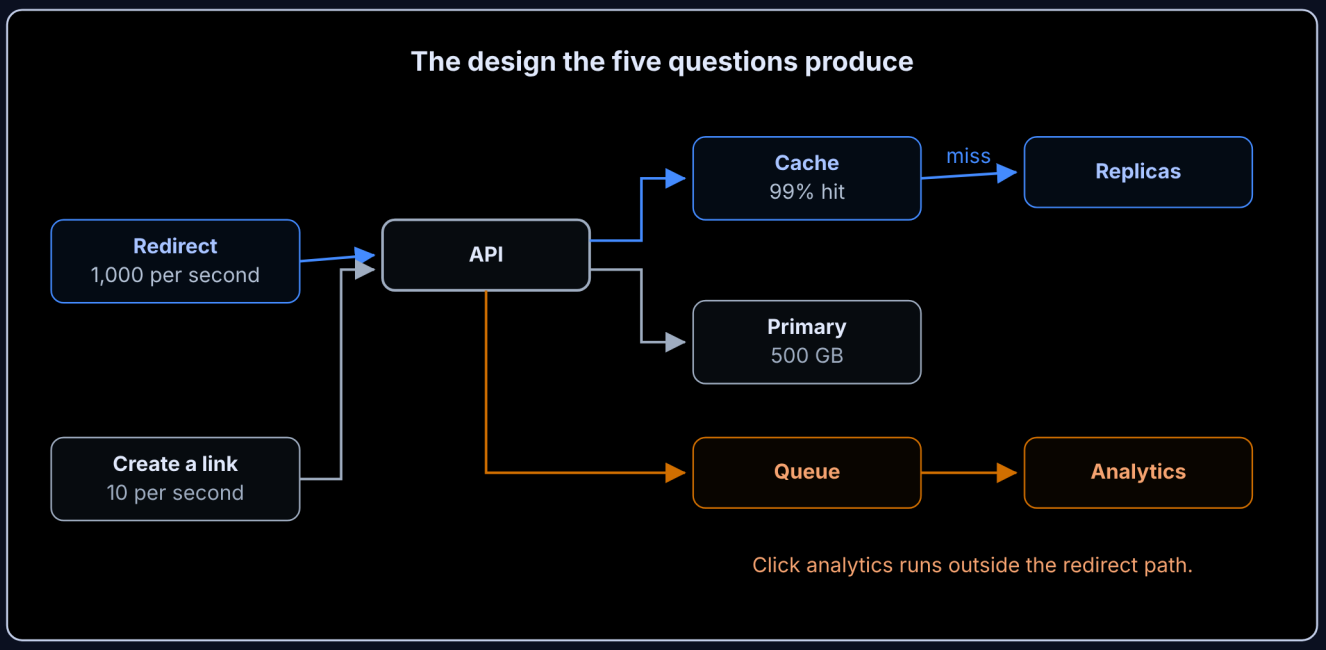


Figure — The design the five questions produce (course diagram)

End state (figure): redirects at 1,000/s hit a cache with a 99% hit ratio; misses go to replicas behind a 500 GB primary; link creation (~10/s) goes to the primary; click analytics flows through a queue *off* the redirect path.

- **Move:** one write path (long URL in, short code out) and one read-heavy redirect path, ~100 reads per write, both direct calls. Click analytics is work nobody waits on, so it's queued and never slows a redirect.
- **Live:** tiny rows (two URLs and a counter), potentially billions. Primary plus replicas for now; at billions of rows, shard by short code, since every lookup includes it. No history needed.
- **Fast:** a code always maps to the same URL, so cache-aside with a long **TTL** (time-to-live: how long an entry stays cached). A few links get most clicks, so hot links get stampede prevention and shouldn't expire in normal operation.
- **Fail:** redirects need very high **availability** (share of time a service answers correctly), so a cache miss reads a replica. Link creation may answer "try again shortly" because its outage hurts less. Each path gets its own reliability target.
- **Grow:** replicas and cache nodes absorb reads; writes are few. The real risk is **key generation**: random codes need **collision** checks (two links getting the same code); counter-based codes need coordination so two servers never issue the same one. Flag it as future work with both options.

Three minutes yields separate read/write paths, storage with a chosen shard key, a caching plan, per-path reliability targets, and one flagged growth risk. When a requirement changes, rerun the questions.

Usage tips

- **Announce the questions as your agenda:** "First how data moves, then storage, caching, failures, growth."
- **Use them as a checklist** when stuck or when time remains.
- **Ask question 4 for every arrow**: it's the most skipped and draws the most follow-ups.
- **Bring them to design reviews at work.** "What if the other service never answers?" is among the most useful questions in distributed systems.

Interview one-liner

"After getting the numbers, I go through five questions in order: how data moves, where it lives, how it's served fast, what happens when it fails, and how it grows. Each surfaces problems that map to patterns, and the fixed order means nothing gets skipped."

TL;DR

- **Procedure:** numbers first, then five fixed questions (move, live, fast, fail, grow).
- **Why it works:** questions surface problems, problems lead to patterns, fixed order guarantees coverage.
- **Structure:** questions map to Modules 2–6; later modules refine the answers.
- **Most-skipped:** Q4, failure handling — where designs become dependable and interviewers push hardest.
- **Use it as:** your spoken agenda, your checklist when stuck, your go-to review question.

Back-of-Envelope Math

Original lesson: Back-of-Envelope Math on DesignGurus ↗

What it is

Fast, deliberately rough calculations that give the *order of magnitude* a system needs: one machine or ten? 600 requests in flight or 60,000? Does the data fit in memory? **Analogy:** "about 5 hours and two tanks of gas" is enough to choose between driving and flying. The fundamentals are covered in Module 0; this lesson keeps the four formulas the course reuses.

Why engineers estimate

A rough answer in 30 seconds shapes today's design; a precise one next week doesn't. You can't answer the five questions without numbers, and interviewers ask "how much storage?" expecting stated assumptions, rounded numbers and visible steps.

Round hard. A day's 86,400 seconds → **100,000**; a month's ~2.5 million → **3 million**. You want **orders of magnitude** (powers of ten), not decimals. When direction matters, round demand up and capacity down to leave headroom.

Numbers to memorize

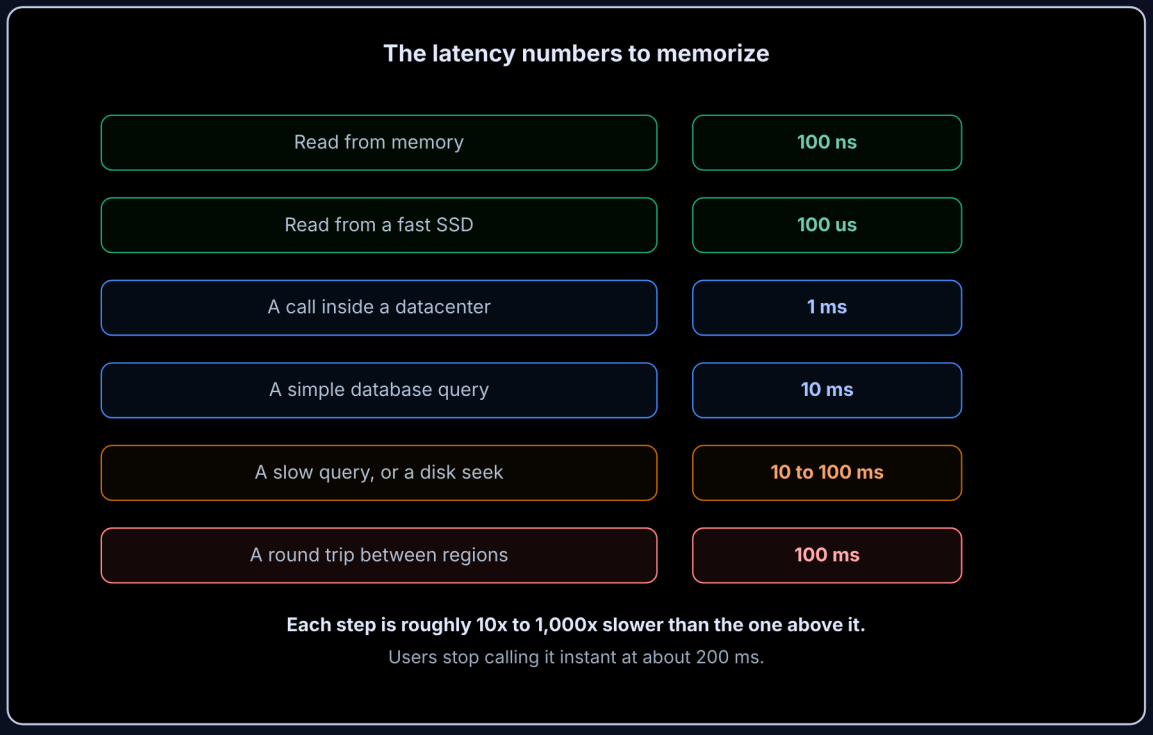


Figure — The latency numbers to memorize (course diagram)

Latency is how long an operation takes. Each row is roughly 10–1,000x slower than the one above (fuller table: Module 0):

Operation	Time
Read from RAM (fast working memory)	~100 ns
Read from fast SSD (solid-state drive, flash storage)	~100 µs
Network call within one datacenter (a building full of servers), e.g. a cache read	~0.5–1 ms
Simple database query	~5–15 ms
Spinning-disk seek, or a slow query	~10–100 ms
Round trip (request there, reply back) between regions (geographic areas with their own datacenters)	~100 ms
Longest delay that still feels "instant"	~100–200 ms

Three design facts follow:

- A cache hit (~1 ms) is ~10–15x faster than a DB query (~10–15 ms): why caching helps.
- Sequential disk writes are much faster than scattered ones: why the write-ahead log works.
- Each *sequential* cross-region round trip adds ~100 ms, so a few consume the whole response budget.

Sizes: a character ≈ 1 byte; an ID ≈ 16–36 bytes; a DB row ≈ 1 KB; an image ≈ 100 KB–1 MB. Each unit is 1,000x the last (KB → MB → GB → TB), so 1 million 1 KB rows ≈ 1 GB and 1 billion ≈ 1 TB.

The four calculations

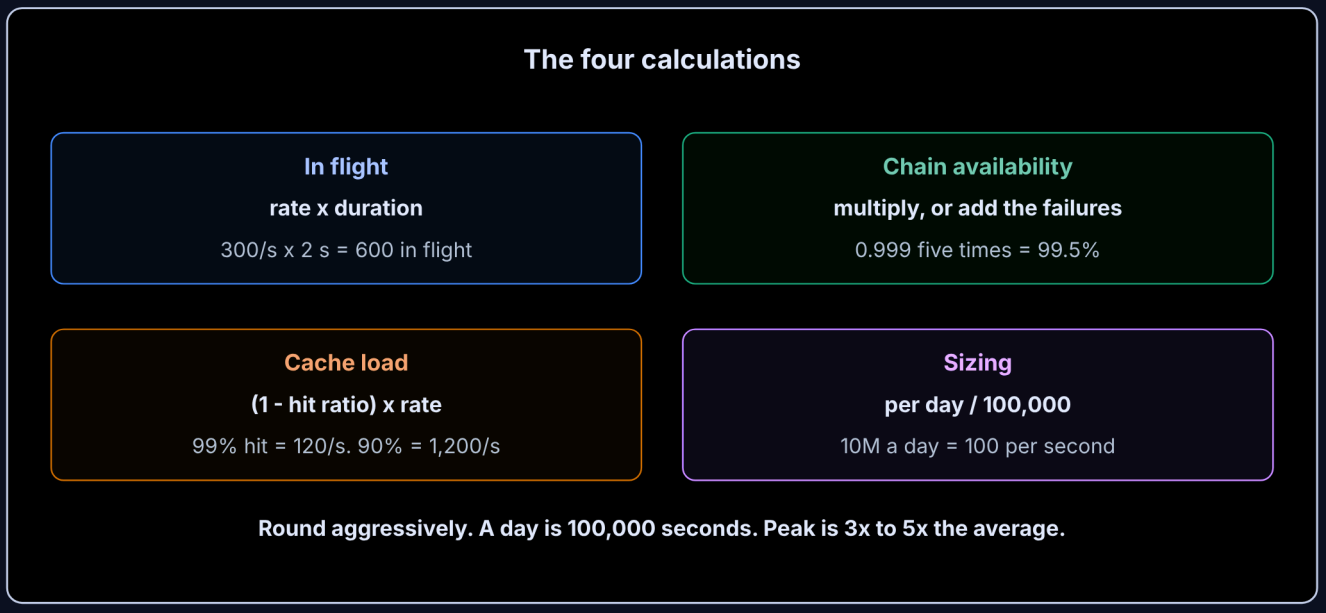


Figure — The four calculations (course diagram)

1. Rate × duration = work in progress (Little's law). `things in flight = arrival rate × duration`. 300 requests/s × 2 s each = **600 active requests**, each holding a **thread** (an independent line of execution; servers run many) or a database connection, so a **connection pool** (a fixed set of reusable open DB connections) of 200 is too small. The same formula gives a queue's **drain time** (time to clear a backlog): 552,000 messages at 400/s ≈ 1,400 s ≈ **23 minutes**. It's the course's most-used calculation, sizing thread pools, connection pools, bulkhead limits and backlog drain; name **Little's law** in interviews (Module 0).

2. Availability drops along a chain. Availability is the share of time a service answers correctly. When a request needs several parts in series, multiply their availabilities; the result is always below the weakest part. Five dependencies at 99.9%: 0.999^5 ≈ **99.5%**. Shortcut for small failure rates: *add* them (5 × 0.1% ≈ 0.5% failure).

Availability	Downtime per year	Per month
99%	~3.7 days	~7 hours
99.9%	~9 hours	~43 minutes
99.99%	~53 minutes	~4 minutes
99.999%	~5 minutes	~26 seconds

Each extra "nine" costs about **10x** the engineering effort of the last, and every required dependency can only *pull the total down*. So the practical fix is **fewer required dependencies**, not perfect ones; graceful degradation builds on this by returning a smaller but useful answer when an optional dependency fails. (More: Module 0.)

3. Cache hit ratio. With hit ratio *h* (share of requests the cache answers):

- `average latency = h × fast + (1 − h) × (fast + slow)` (a miss pays for the cache check *and* the DB)
- `load on the database = (1 − h) × request rate`

At 12,000 RPS, 1 ms hits and 15 ms DB reads:

- h = 99% → ~1.15 ms average; DB gets 1% × 12,000 = **120 QPS** (queries per second).
- h = 90% → ~2.5 ms average; DB gets **1,200 QPS**, ten times the load.

99% → 90% isn't "9% worse": the *miss* rate goes 1% → 10%, so the DB sees 10x traffic. Judge caches by their misses. It also explains a **cache stampede**: when a hot entry expires, its hit ratio briefly drops to ~0 and nearly every request hits the DB at once.

4. Sizing.

- `per second = per day / 100,000`: 10M orders/day ≈ **100/s average**.
- Plan for a **peak of 3–5x average**: 300–500/s.
- `storage = records × size × retention`: 10M/day × 1 KB × 365 ≈ **3.7 TB/year** — fits one ordinary database server, so this estimate *saves* you from sharding prematurely.

Walkthrough: sizing the URL shortener (~60 seconds, five numbers)

State each assumption aloud:

- "Assume **100M redirects/day** ÷ 100,000 = **1,000/s** average; plan for **~4,000/s** peak."
- "Each mapping is two URLs plus **metadata** (creation date, owner): **~1 KB**. **500M links** ≈ **500 GB**. One database handles that today, so sharding waits."
- "~100:1 reads to writes, and a few links get most traffic. Caching the top 1% is **5M rows ≈ 5 GB**; at a 99% hit ratio only **~10 queries/s** reach the DB."
- "At peak, 4,000/s × 2 ms = **8 requests in flight**. So: one strong database, one replica, a cache, careful failure handling."

Every component now has a capacity-based reason, and nothing is built for demand that hasn't arrived.

Traps

- **90% is not close to 99%:** it means 10x the misses.
- **Each extra nine costs ~10x**, and long dependency chains only lower availability, so keep them short.

Using estimation in interviews

- **State assumptions before calculating** ("1 KB per record, 10M records a day"); an interviewer can correct a spoken assumption, not a silent one.
- **Round, multiply, then check units:** rate × seconds = count; bytes × rows = storage. Wrong units, wrong math.
- **Let the numbers remove components.** "At this scale we don't need that yet" shows cost awareness, backed by arithmetic.

Interview one-liner

"I round hard (100k seconds per day, a gigabyte per million 1 KB rows, peak at 3–5x average) and use four formulas: rate × duration for concurrency, multiplied availabilities for chains, hit ratio for cache latency and DB load, and records × size × retention for storage. That tells me what the design needs and what it doesn't need yet."

TL;DR

- **Rule:** round hard; a day = 100,000 s; 1M × 1 KB = 1 GB; peak = 3–5x average.
- **Table:** RAM ~100 ns, SSD ~100 µs, same-datacenter hop ~1 ms, DB query ~10 ms, cross-region ~100 ms, "instant" ends at ~100–200 ms.
- **Four calculations:** in flight = rate × duration; chain availability = multiply (or add failure rates); cache latency and DB load from hit ratio; per second = per day / 100,000 and storage = rows × bytes × retention.
- **Traps:** 90% ≠ 99% (10x the misses); each nine ~10x the cost; chains only lower availability.
- **In interviews:** state assumptions, check units, let arithmetic rule out over-engineering.

Module 2 — Moving Data (Communication)

Module Introduction

Original lesson: Module introduction on DesignGurus ↗

What this module covers

A design diagram is boxes (services such as an **API** — application programming interface, the set of requests a program accepts — a database, an email sender) and arrows (data moving between them). This module is about the **arrows**: each one hides a choice of *how* A reaches B, and that choice decides three behaviors:

- **Waiting:** must someone wait for the answer?
- **Downtime:** what happens if B is offline?
- **Duplicates:** what happens if the message arrives twice?

Wrong choices cause real outages. Two from the coming lessons: a **flash sale** (a short, heavily advertised sale that brings huge simultaneous crowds) took down a store that had enough total capacity, and a "real-time" dashboard spent most of its budget asking "anything new?" to hear "no."

The seven patterns

Situation	Pattern
The caller can't go on until it has the answer	Request-Response
The work can be done later, and no one is waiting on it	Message Queue
Something happened, and more than one service needs to hear about it	Publish-Subscribe
Pack the details into the event so no one needs to call back for them	Event-Driven Architecture
A system at another company needs to be told about something	Webhooks
A steady flow of updates going one way	Server-Sent Events
Both sides keep sending messages to each other	Bidirectional Streaming

Two rules for the whole module

- **Rule 1: request-response is the default.** If someone waits for the answer, make a direct call. The other six patterns are for when waiting is pointless or impossible, and each has costs; switching isn't automatically an improvement. Less experienced engineers often reach for a fancy pattern where a direct call is clearer and safer.
- **Rule 2: anything that retries can deliver twice.** Queues, events and webhooks resend whenever unsure a message got through — deliberately, so nothing is lost. Your code must therefore receive a message twice without applying its effect twice: **idempotency** (running an operation twice gives the same result as once; full lesson in Module 5). Plan for duplicates from day one.

Four terms used throughout

- **Job vs fact.** A *job* is work to do once ("resize this image"); a *fact* records that something happened ("an order was placed"). Jobs go on **queues**; facts get **published**. Mixing them up is the most common communication mistake.
- **At-least-once delivery.** A message is never lost but may be delivered more than once (Rule 2).
- **Push vs pull.** *Pull*: the client keeps asking "anything new?" *Push*: the server sends updates as they happen. Pull costs requests; push costs open connections.
- **Synchronous vs asynchronous.** *Synchronous*: the caller waits for the answer. *Asynchronous*: the caller hands off the message and continues.

How to read this module

One online store drives the story: a flash sale breaks its synchronous design (**queue**); more teams want the data (**pub/sub**); thin events cause callbacks (**event-driven architecture**); a payment provider sends data in (**webhooks**). Read those five in order; **SSE** and **Bidirectional Streaming** cover live streams, and the **capstone** combines all seven.

Short on time? The four most asked: request-response, message queue, pub/sub, webhooks.

Before you start

For a system you've worked on ("I don't know" is a fine first answer):

- Which API call is slowest, and is a person waiting for its result?
- If a queued message is delivered twice, what happens?
- What refreshes on a timer, and out of 100 refreshes, how many find something new?

Request-Response

Original lesson: Request-Response on DesignGurus ↗

What it is

The caller sends a request and **waits**, doing nothing else, until a result or error returns. It's the normal way services communicate; the other six patterns are deliberate alternatives. **Analogy:** asking a shop assistant at the counter — you stay put, and can't finish buying without the answer.

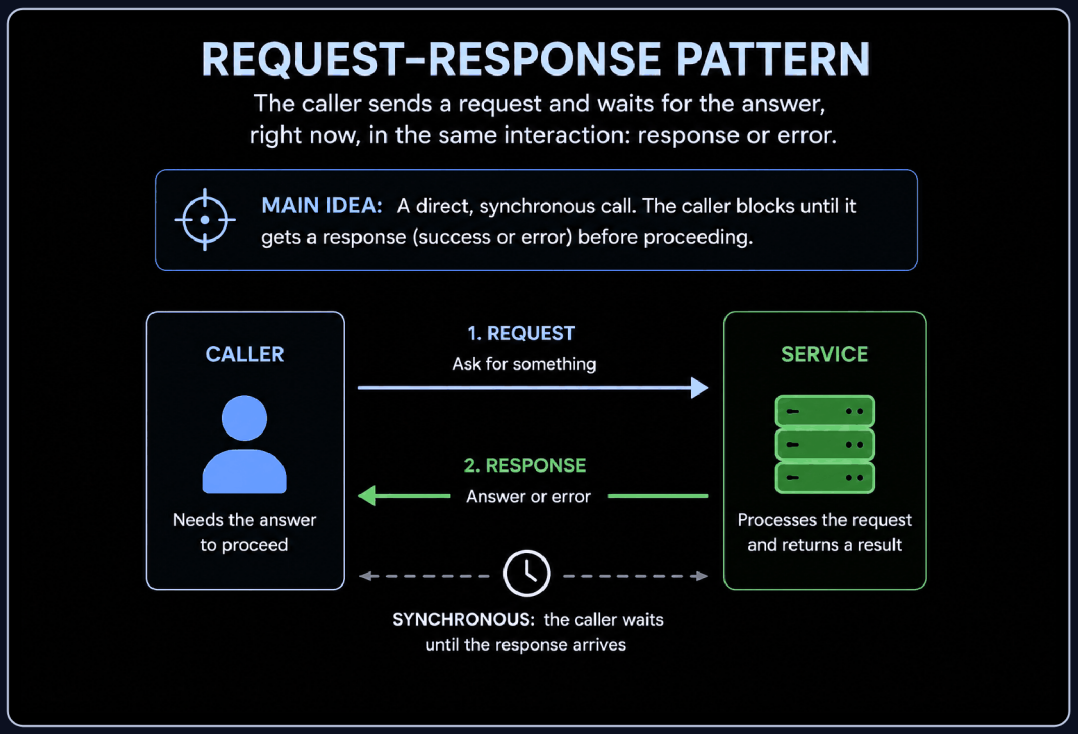


Figure — Request-Response (course diagram)

The caller (which "needs the answer to proceed") sends (1) a request; (2) a response or error returns. The clock between them marks the key property: the call is **synchronous**, so the caller is blocked for the whole round trip.

Why it needs a lesson

Engineers new to queues and events overuse them. A plain synchronous call is right for most traffic — page loads, searches, password checks, price lookups, payment authorization. When a user waits, a queue doesn't remove the wait; it hides it behind machinery.

What it gives you

- **The answer immediately.** The next line of code uses it; no "received, come back later" state to design.
- **Errors where you can handle them.** The caller can retry, show a message or fall back. A failed background job is far harder to trace, since it fails apart from the request that caused it.
- **Easy debugging.** One request, one path, one **trace** (record of the path through services).

Reusing connections matters a lot. A new connection starts with **TCP and TLS handshakes** (TCP gives a reliable, ordered connection; TLS adds the encryption behind `https`; see Module 0), costing 1–3 round trips — ~100 ms between regions. **Connection pooling** (**keep-alive**) keeps connections open for reuse; every production HTTP client supports it, so verify it's on.

What it costs

Each cost is small alone, but a *chain* of calls compounds them:

- **Both sides must be up at once.** If the receiver is down, the call fails immediately: **temporal coupling**.
- **Availability multiplies along a chain.** An **endpoint** (one operation a service exposes, e.g. `POST /checkout`) calling 5 services in series at 99.9% each works only when all do: $0.999^5 \approx$ **99.5%**, roughly **44 hours of failures a year**, and every added dependency lowers it.
- **Latency adds up.** Five sequential 80 ms calls take **400 ms**. Independent calls made **in parallel** cost only the slowest (~80 ms); turning accidental sequencing into parallelism is often the quickest speed-up.
- **Waiting ties up resources.** Each waiting caller holds a thread or connection: 300 requests/s × 2 s = 600 in flight (Little's law, Module 1).

So request-response at scale needs three safeguards, each with its own lesson: a **timeout** (stop waiting after a deadline), **retries** (repeat after a transient failure), and a **circuit breaker** (stop calling an obviously down service).

One question per arrow

Before drawing an arrow, ask: **does the caller's next step need the answer, *and* is someone waiting for it?** Both yes → direct call. Otherwise pick from the seven patterns table. Each alternative has its own cost: a **pending state** (accepted but unfinished work), possible duplicates, or a connection to manage. Switch because an instant answer *isn't needed*, not because the alternative sounds fancier.

When not to use it

- **Long-running work** (more than a few seconds): accept the job, reply "started" immediately, deliver the result later via queue, status endpoint or webhook.
- **Telling many services about one event:** calling five services separately couples them all; publish the event (Pub/Sub).
- **Bursty work** (arriving in spikes) **with loose deadlines:** if no one waits, queue it.
- **Chains that are too long:** by six synchronous hops, availability has dropped and latency climbed. Shorten, parallelize, cache, or queue what can wait.
- **The opposite mistake, async by habit:** search behind Kafka (a high-throughput message log) or event-driven login. The user still waits, now via a **message broker** (a server that holds and forwards messages), adding latency and duplicate risk to a ~200 ms path. Async is right when waiting is optional, not better across the board.

In the interview

Rarely asked by name; more often the interviewer points at a sync arrow: "why is this synchronous?" Name the dependency: "Checkout can't proceed without the payment result and the user is waiting, so it stays synchronous, protected by a timeout, bounded retries and a circuit breaker."

> **30-second answer:** "Request-response is my default whenever the caller's next step needs the answer: auth (authentication, checking who the user is), pricing, payment authorization, interactive reads.

Immediate answers, local error handling, simple tracing. Costs: both sides up at once, availability multiplying down a chain, latency adding up. So I parallelize independent calls, pool connections, and wrap every call in timeout, retry and circuit breaker. Anything nobody waits on goes to a queue or an event."

Likely follow-ups:

- *"Your endpoint calls five services and is slow. Fix it without going async."* Parallelize independent calls (latency drops from *sum* to *slowest*), cache rarely changing answers, drop unneeded calls, confirm connection reuse.
- *"When do you switch a call to async?"* When nothing needs the answer to proceed, or the work outlasts a reasonable open connection. Async adds resilience but also pending states, duplicates and ordering questions: a trade-off, not an upgrade.

Mistake that fails candidates: making a user-facing path async (search through Kafka), or drawing six unprotected synchronous calls. Both show you don't understand what sync calls cost or when their instant answer is worth it.

Self-check

- **Q1: Which two questions decide sync vs async, and what costs does each sync call add to a chain?** → Does the next step need the answer, and is someone waiting? Costs: temporal coupling, multiplying availability, added latency, and a thread held per waiting caller — hence timeout, retry and breaker at scale.
- **Q2: Checkout makes five sequential 80 ms calls to services at 99.9% each. Latency, availability, and how to improve both without async?** → 400 ms and ~99.5% (~44 h/year of failed checkouts). Parallelize independent calls (~80–120 ms), cache rarely changing answers like pricing rules, drop unneeded calls. For availability, let optional page parts degrade instead of failing the page (graceful degradation).
- **Q3: A teammate wants payment authorization async "for resilience" via a `payment.requested` event. Assess it.** → Wrong: checkout needs the result and the user is waiting. It creates a pending state the product must design for, and duplicate delivery could authorize twice. Protect the sync call instead: timeout, retries with an **idempotency key** (unique ID so the provider can ignore a repeat), a circuit breaker, and an outage plan (save the cart, email once payment succeeds). The *receipt email* is what should be async.

Related patterns

Timeout, Retry and Circuit Breaker (the mandatory safeguards) · Message Queue (for work no one waits on) · Load Balancing (one logical service, many servers) · API Gateway (where external calls are authenticated and rate-limited).

TL;DR

- **Problem:** every arrow needs a default; teams err both ways — async where someone waits, or sync chains until latency and fragility snowball.
- **Mechanism:** the caller waits for result or error. Pool connections, parallelize independent calls, wrap every call in timeout, retry and breaker.
- **Costs:** both sides up together, availability multiplies down, sequential latency adds, each waiting caller holds a thread.
- **Skip it when:** no one waits, the work outlasts a connection, many services care about one event, or the chain is too long.

Message Queue

Original lesson: [Message Queue on DesignGurus ↗]

What it is

A **message queue** is a durable to-do list between the service that creates work (the **producer**) and the one that does it (the **consumer** or **worker**). *Durable* means saved to disk, surviving crashes. The producer drops a task in and moves on; workers take tasks at their own pace; the queue keeps each task until a worker says it's done. **Analogy:** a kitchen ticket rail — waiters clip tickets as fast as customers order, cooks take them as fast as they can cook, and a rush just lengthens the rail.

The problem: the flash sale

The store sells 10,000 signed editions at 10:00 AM. Normal traffic is 200 orders/s; at 10:00:01 it jumps to **5,000 orders/s for two minutes**. The pipeline is synchronous: while the buyer waits, the API takes the order, reserves stock, writes the database and schedules fulfillment before saying "success." Its capacity is **400 orders/s**, plenty for years.

5,000 in, 400 done per second leaves **4,600/s** waiting, timing out or failing: **92% of buyers see an error**, and their refreshes add load.

The postmortem (post-incident review) finds the real cause. The pipeline could have processed those 600,000 orders well within a day; buyers needed a *confirmation* immediately, not fulfillment within two minutes. Fusing confirmation and fulfillment into one synchronous call forced all fulfillment work to beat the confirmation deadline. **That coupling caused the outage.**

Why the obvious fixes fail

- **"Scale up for the peak." 12x** the servers all year for a two-minute event, and the database has its own ceiling, so traffic fails there instead. The **bottleneck** (the slowest part, limiting throughput) moves rather than disappears.
- **"Buffer orders in memory."** An unsafe queue: a crash loses paid orders, other API servers can't see it, and monitoring can't see its size until memory runs out.
- **"Reject the excess."** Fine for unimportant traffic, but here it turns away **9 of 10 customers** in the biggest sale of the quarter, for work that could safely run an hour later.

All three treat it as a capacity problem. It's a **coupling** problem: arrivals and processing, which should run on separate schedules, were forced to the same rate every second.

How it works

Four mechanics, each answering a "what if it fails?":

1. **Save before confirming.** The **broker** (the software running the queue) writes each message to disk and *only then* acknowledges the producer. A worker crash, a **deploy** (rolling out a new version, restarting servers) or an hour of downtime can't lose the work.
2. **Workers confirm on finish, not on start.** A worker takes a message, does the job, then sends an **ack** (acknowledgment). If it crashes mid-job, no ack arrives, and after a **visibility timeout** the broker hands the message to another worker. *Consequence:* a worker that finishes but crashes just before acking looks identical to a mid-job crash, so the job runs twice. That's **at-least-once delivery** (one or more times, never zero), so every consumer must be **idempotent**.
3. **Competing consumers.** Many workers pull from one queue; each message goes to exactly one. More **throughput** (messages per second)? Add workers; no coordination code needed.
4. **Queue depth is the health signal. Depth** is tasks waiting (arrivals minus completions). Rising during a brief spike: buffering is working. Rising for a *week*: a capacity problem.

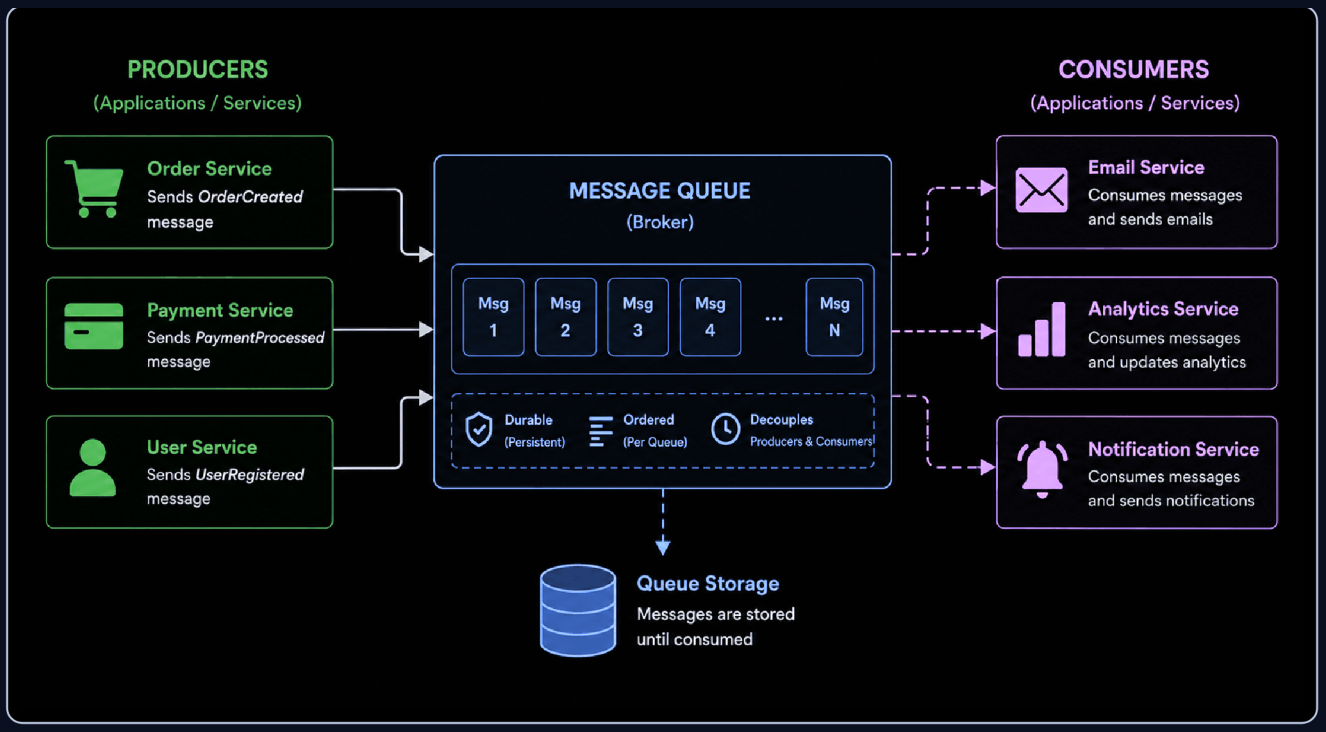


Figure — Message Queue (course diagram)

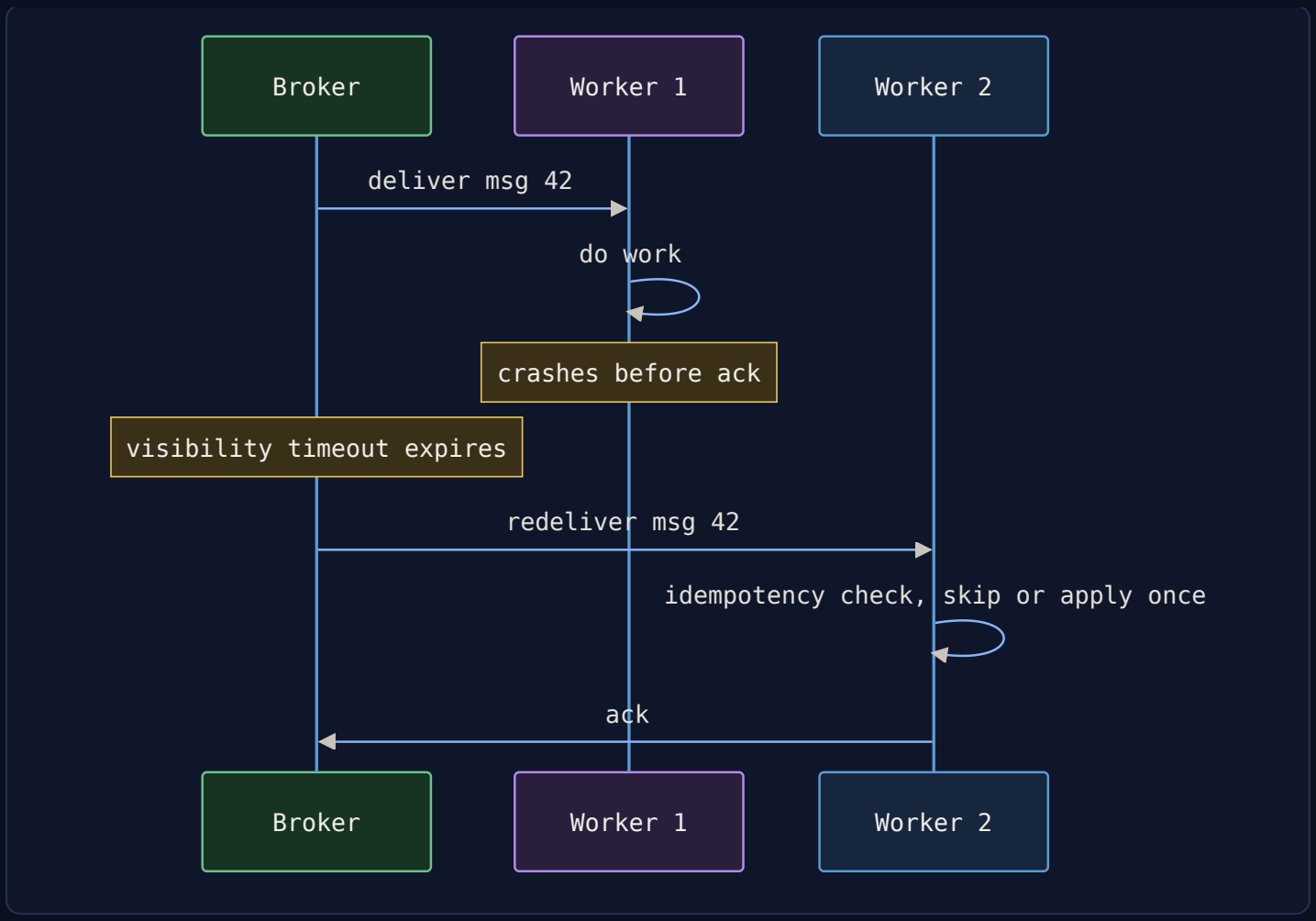
Producers drop messages into the broker, which stores them until consumed and releases them at the consumers' pace. The figure's three properties — **durable, ordered (per queue), decouples producers and consumers** — are the point of the pattern.

Together these separate three concerns the synchronous design fused:

- **Rate:** producers burst to 5,000/s while workers hold 400/s.
- **Time:** workers can be down, mid-deploy, or not yet built; the work waits.
- **Failure:** a worker crash means redelivery, not loss, invisible to the producer.

Where the duplicate comes from:

Ack and redelivery producing at-least-once delivery



Walkthrough with numbers

The flash sale with a queue between API and pipeline:

- **10:00–10:02.** 5,000 orders/s arrive. The API validates, enqueues and replies "Order confirmed" in **~50 ms**. The queue grows 4,600/s; over 120 s that's a **backlog** (accepted, not yet done) of ~**552,000 messages**, cheap on disk.
- **10:02–10:25.** Workers continue at 400/s: 552,000 / 400 = **1,380 s ≈ 23 minutes** to drain. Some orders start fulfillment 20 minutes after purchase, but the confirmation promised "ships today" — a delay the business approved in advance, so a design choice, not a failure.
- **10:07, failure drill.** A worker dies mid-message; after the visibility timeout another takes it, and the idempotency check prevents double application. No order lost.
- **The bill.** A queue service costing pennies per million messages replaces a 12x fleet.

Key point: the queue added **no** processing power. It removed the requirement that processing keep pace with arrivals second by second, which the business never needed.

Trade-offs

You gain:

- Spikes become backlog, not errors; the limit becomes disk space, not your slowest service.
- A worker outage becomes a longer list, not an outage; deploys stop being scary.
- Producers and workers scale independently, and depth is a ready-made **autoscaling** signal (adding/removing workers automatically).
- Redelivery provides retries with no retry code.

You pay:

- **Latency, deliberately.** Work finishes seconds or minutes later, so nothing a user actively waits on belongs here.
- **Duplicates.** Idempotent workers are mandatory.
- **Lost ordering.** With 20 workers, messages can finish out of arrival order; ordering by key (below) helps at some throughput cost.
- **A gap between "accepted" and "done"** that the product must design for.
- **More to run:** a broker, monitoring of depth and message age, a **dead letter queue (DLQ)** for repeatedly failing messages (its own lesson), and debugging across systems instead of one **stack trace** (the call list printed on an error).

Ordering by key: Kafka's **partition key** or SQS FIFO's **message group** (SQS is Amazon's managed queue; FIFO = first in, first out) sends all of one customer's messages to the same **partition** (an independently ordered slice of the queue). Each customer stays in order while customers process in parallel. Costs: one very busy customer is capped at one partition's speed, a stuck message blocks its partition, and moving a failed message to the DLQ can break strict order.

Depth growing for days means arrivals permanently exceed processing (self-check Q2); alert on **oldest-message age**, the delay users actually feel.

When not to use it

1. **The caller needs the answer:** login, price lookups, search. No one wants search results by email.
2. **Tight end-to-end deadlines:** within ~200 ms, a broker hop plus worker wait adds risk with no useful decoupling.
3. **Small, simple systems:** at ~10 requests/s, call the function until scale, spikes or failure boundaries justify a broker.
4. **As a database:** queues move work; data you need to search, modify or keep forever belongs in a database.
5. **Masking a permanent capacity shortfall:** queues absorb bursts; they don't create capacity.

Message Queue vs Publish-Subscribe

Both use a broker and asynchronous delivery, but they answer different questions:

	Message Queue	Publish-Subscribe
A message is	A job: work that should happen once	A fact: something that happened
Delivered to	Exactly one worker	Every subscriber, each on its own
Adding a consumer	Adds more workers for the same job	Adds another use for the same fact
Question it answers	"Who does this task?"	"Who cares that this happened?"

"Resize this image" is one job for one worker: queue. "An order was placed" is a fact that fulfillment, email, analytics and fraud each need: pub/sub. They combine well — publish the fact once and feed a separate work queue per team (AWS **SNS** → **SQS**, SNS being Amazon's managed pub/sub; or Kafka **consumer groups**, each a named set of readers that together receive one full copy of the stream).

Real-world examples

- **AWS SQS:** the pattern in simplest form — visibility timeouts, dead-letter support, near-unlimited depth, pennies per million messages.
- **RabbitMQ:** the classic self-hosted broker, adding rich routing on the same ack-and-redeliver core.
- **Kafka:** strictly a **replicated log** (an append-only record copied across machines), but consumer groups behave like a queue, and **replay** of old messages powers event-driven architecture and stream processing later in the course.
- **Celery, Sidekiq, BullMQ:** the pattern as application libraries; most engineers meet it here.

- **For AI engineers:** GPU work (on the graphics chips that run models) is costly, bursty and rarely urgent — ideal for queues: embedding pipelines (turning text into number lists called **embeddings** that capture meaning), **batch inference** (running a model over many inputs at once), **fine-tuning** (further training a model on your data) and evaluation runs. Depth, or better **oldest-message age**, drives GPU autoscaling. **RAG** (retrieval-augmented generation: fetching relevant documents to give the model with the question) ingestion uses a queue plus DLQ. Anthropic's and OpenAI's **batch APIs** sell exactly this trade: cheaper if it can finish later.

In the interview

Appears in "handle a 10x traffic spike" or "design an order pipeline," or when the interviewer points at a sync arrow: "does the user need to wait for this?" If not, it may belong on a queue.

30-second answer: "A durable queue between API and pipeline: the API validates, enqueues and confirms in milliseconds; workers process at a sustainable rate, so a 5,000/s spike becomes backlog, not errors. The broker persists before confirming and redelivers if a worker dies — at-least-once, so consumers are idempotent and repeated failures go to a monitored DLQ. Depth drives autoscaling and alerts. The trade is latency, acceptable only because the business agreed fulfillment can take minutes; anything the user waits for stays synchronous."

Likely follow-ups:

- *"A worker crashes halfway through a message. Walk me through it."* Visibility timeout expires, another worker gets it, idempotency prevents a double effect; repeated failures go to the DLQ with an alert.
- *"Orders for the same customer must be processed in order. Now what?"* Partition by customer ID (ordering by key, above); a very busy customer is then capped at one partition.

Mistake that fails candidates: a queue on a path the user waits on (search through Kafka), or a queue with no plan for duplicates. The first misses *why* queues exist; the second ignores at-least-once delivery.

Self-check

- **Q1: Name the three ways a queue decouples producer and consumer, and explain step by step where at-least-once delivery comes from.** → Rate (bursts in, steady out), time (work waits while workers are down), failure (crash means redelivery, not loss). Workers ack only after finishing, so the broker must redeliver both an interrupted job and a finished job whose worker died before acking — the latter is the duplicate.
- **Q2: Depth has grown steadily for six days; a teammate proposes raising the storage limit. What is depth telling you, and why is that a trap?** → Arrivals permanently exceed processing; the oldest message is six days late. A bigger limit only delays failure. Add workers or fix the bottleneck, shed or reprioritize work, route some elsewhere, alert on oldest-message age.
- **Q3: Charges and refunds for a customer share a queue with 20 workers; a refund got processed before its charge, leaving a negative balance. Explain and fix without losing parallelism.** → Two workers took the two messages and the refund finished first (many workers lose global order). Partition by customer ID: each customer's messages run in sequence, customers in parallel. Costs: a hot customer is capped at one partition, a stuck message blocks its partition, and DLQ moves can break strict order, so some streams must pause and be repaired instead.

Related patterns

Idempotency (mandatory for every consumer) · Dead Letter Queue (where redelivery gives up) · Retry with Exponential Backoff (redelivery *is* broker-driven retrying; tune attempts and delays there) · Publish-Subscribe (the counterpart for facts) · Auto-Scaling (depth and backlog age are its clearest signals).

TL;DR

- **Problem:** a synchronous pipeline must process work as fast as it arrives, so a spike becomes an outage despite ample total capacity.
- **Mechanism:** a durable task list between producer and consumer: save before confirming, one worker per message, redeliver after a crash, drain at worker pace.
- **Costs:** latency by design, duplicates (idempotency required), no global ordering, an accepted-vs-done gap to own, a broker to run.
- **Skip it when:** the caller needs the answer now, end-to-end deadlines are tight, scale doesn't justify a broker, or it would mask a permanent shortfall.

Publish-Subscribe

Original lesson: [Publish-Subscribe on DesignGurus ↗]

What it is

With **publish-subscribe (pub/sub)**, a producer announces a fact once to a named channel (a **topic**) without knowing who listens. The broker gives every **subscription** its own copy, which each subscriber processes at its own pace with its own failure handling. **Analogy:** a newspaper prints once, every subscriber gets a copy, and a new reader subscribes without changing anything in the newsroom.

The problem: checkout delivering to five teams

After the queue fixed the flash sale, email wanted order data for receipts, then analytics, fraud and loyalty. Each request added a "send to this team's queue" call inside checkout. Eighteen months later checkout writes to **five queues** in sequence, causing three ongoing problems:

1. **Every new consumer needs a checkout deploy.** The company's most important revenue service must change for each team's data request, so its release schedule blocks everyone.
2. **A crash leaves teams with mismatched data.** Checkout crashed after writing 3 of 5 queues: fulfillment shipped, fraud never saw it. Five separate writes can't succeed or fail as one unit.
3. **Other teams' problems become checkout's.** When loyalty's queue was slow for an hour, checkout's sequential writes slowed too.

Then data science wanted *historical* order events for training and had to open a **pull request** (a proposed code change) against checkout and wait for a release. When the producer hand-delivers to every consumer, each new one adds work, risk and coordination; it doesn't scale across teams.

Why the obvious fixes fail

- **"Keep adding queues."** It runs, but checkout owns every connection, retry policy and delivery guarantee, and each extra write raises the odds of a partial crash.
- **"Let teams read the orders database."** Five teams now depend on the table's exact shape; a **schema migration** (changing table structure, e.g. renaming columns) silently breaks dashboards and models checkout doesn't know about, and their constant **polling** (repeatedly asking "anything changed?") loads production. **Change data capture (CDC)**, which turns the database's change log into an event stream, is the controlled way to do this; raw table access has none of its safety.
- **"One shared queue everyone reads."** A queue gives each message to **one** worker, so fraud, email and analytics each get a random slice of orders, with no error anywhere.

The teams don't need five jobs; each needs its own access to one *fact*. Publish it once.

How it works

1. **The topic** is a named stream of facts, e.g. `order.placed`. Checkout publishes once and moves on; the broker makes **one durable write** regardless of listener count, so a crash can't deliver to some and not others.

2. **A subscription** is one team's independent copy — effectively its private queue, with its own backlog, position in the stream (the **offset**), acks, redeliveries and DLQ. If fraud is down an hour, only fraud's backlog grows.
3. **Joining is configuration, not code.** Data science subscribes without touching checkout. Brokers that keep history (e.g. Kafka) let new subscribers **replay** old events — train on months of data, then switch to live.
4. **The event format becomes a public contract** that unknown consumers treat like an API. Publish **facts, not commands** ("order placed, here are the details," not "send a receipt"): the publisher says what happened, each consumer decides what it means. Keep changes **additive** (new optional fields are safe; renames and removals break invisible subscribers). A **schema registry** validates event formats before publishing.

The producer states *what happened*; each consumer decides *what it means*. Checkout needn't know receipts exist.

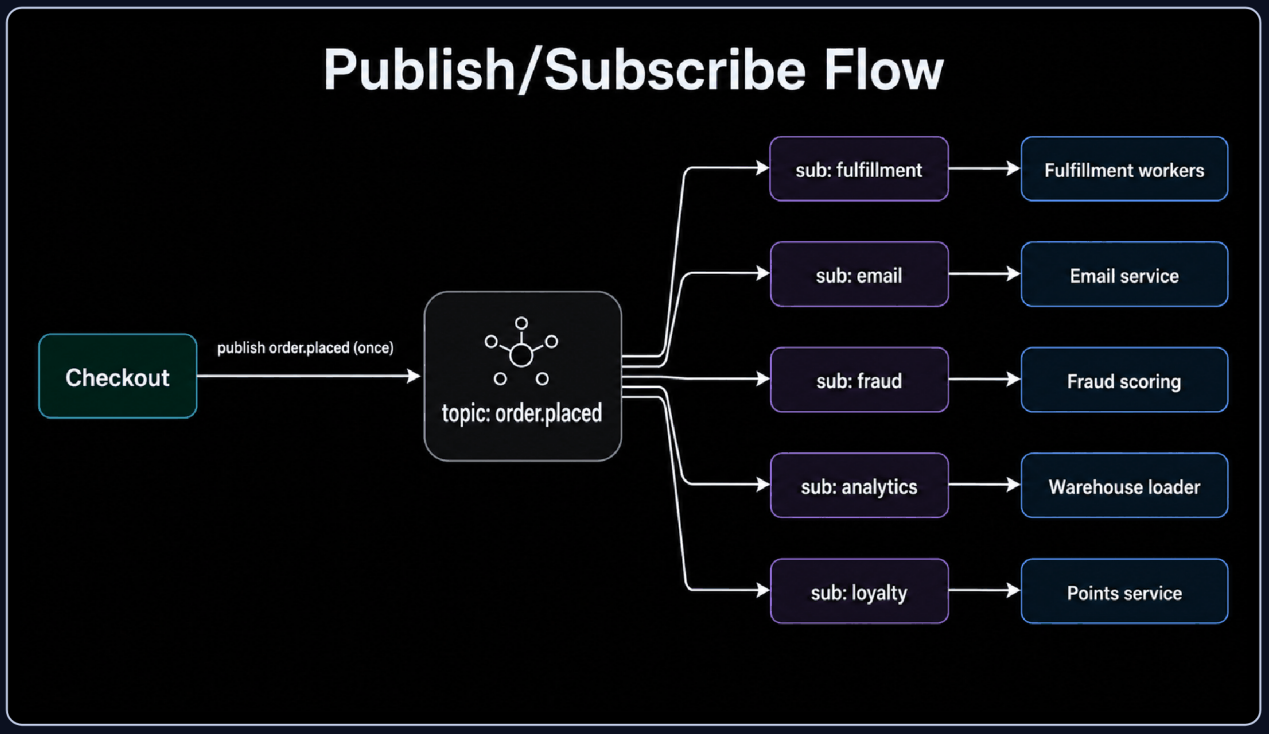


Figure — Publish-Subscribe (course diagram)

Checkout publishes `order.placed` **once**; the broker fans it out to five subscriptions (fulfillment, email, fraud, analytics, loyalty). A sixth team adds one subscription on the right; nothing on the left changes.

Walkthrough with numbers (200 orders/s)

- **Publish cost:** before, five sequential queue writes ≈ **25 ms**, growing per team; after, one durable write ≈ **5 ms**, flat regardless of subscribers.
- **Data science:** subscribed in an afternoon with no checkout change; **30 days** of retained history lets them replay ~**500 million** events before going live. Previously: a PR, a release cycle, and *no* history.
- **Failure drill:** fraud down **40 minutes** → its subscription accumulates 200 × 2,400 s = **480,000 events**, processed on return. Email and checkout unaffected; before, fraud's outage would have slowed checkout or lost 40 minutes of events.
- **Audits pass:** each subscription can say whether order X was processed, is in backlog, or went to the DLQ.

Trade-offs

You gain:

- Producer cost and risk independent of consumer count.

- Teams release independently — the biggest practical benefit.
- Failures contained per subscription; with retained history, new consumers start with the full past.

You pay:

- **The event format is a public API.** A renamed field breaks consumers you've never heard of. Deploy-time coupling becomes *contract discipline*: registry, additive changes, **deprecation windows** (a period when the old version still works).
- **Duplicates, times N.** Each subscription is at-least-once, so every subscriber must be idempotent.
- **Hidden coupling.** Dependencies still exist but vanish from the producer's code. Mature organizations keep a **subscriber catalog** listing each topic's owner, **schema** (agreed field shape), **retention period** (how long events are kept) and **data classification** (sensitivity), plus every subscription's owner — making schema changes, incidents and deletion requests manageable and exposing unmaintained consumers.
- **No "all done" signal.** Each subscription only knows its own progress, so pub/sub can't say "order X is fully processed." When inventory, payment and shipping must all succeed, a coordinator must track them (orchestration, below).
- **Slow consumers can lose data.** Brokers keep history only for a **retention window**; a subscriber further behind loses events permanently. Alert on backlog age well before retention runs out.

Rule of thumb: **broadcast across services, compete within a service.** For a truly breaking schema change, publish v2 alongside v1 for a deprecation window.

When not to use it

1. **It's a job, not a fact:** "resize this image" → queue.
2. **The producer needs the result:** checkout must know if payment was authorized; publishing `payment.requested` doesn't change that. Use request-response or an orchestrated workflow.
3. **Two services, low traffic:** one producer and one consumer just need a queue.
4. **Request-response rebuilt from events:** `user.lookup.requested` + `user.lookup.completed` recreates a sync call with more latency and harder debugging. "Request" in an event name is a warning sign.

Unmanaged pub/sub in a big organization becomes **topic sprawl**: hundreds of topics without owners, schema rules or visible consumers. Removing required coordination means explicit contracts and a catalog must replace it.

Choreography vs Orchestration

Pub/sub enables **choreography**: each service reacts to events independently and the overall process *emerges*. **Orchestration** uses one coordinator that runs the workflow step by step.

	Choreography (pub/sub)	Orchestration (coordinator)
Who's in control	No one in the middle; the behavior emerges	The coordinator, openly
Adding a step	A new subscription, no coordination needed	Change the coordinator's workflow
"Is order X fully processed?"	No single place has the answer	The coordinator knows, by design
Failure handling	Each consumer deals with its own	The coordinator retries, undoes, or escalates
Best for	Reactions that don't depend on each other (analytics, email, indexing)	Steps that depend on each other (inventory → payment → shipping)

Ask whether the steps are truly independent. Email and analytics are: choreography. Reserve inventory → charge → ship → undo on failure is a dependent workflow; the **Saga** lesson covers orchestrating it. Many systems do both: orchestrate the workflow and publish facts from each stage.

Real-world examples

- **AWS SNS → SQS:** SNS is the topic, each team's SQS queue its subscription: broadcast first, then per-team task lists.
- **Kafka:** each consumer group is an independent subscription over a stored log — broadcast, replay and per-team queueing in one system, hence its dominance.
- **Google Cloud Pub/Sub:** the same model, fully managed, with subscriptions as first-class objects.
- **Redis Pub/Sub, the warning:** fire-and-forget; an offline subscriber loses the message forever. Without durable subscriptions, "pub/sub" is a much weaker promise under the same name.
- **For AI engineers:** a `user.interacted` topic often feeds a whole ML platform — the analytics warehouse, a **feature store** (service holding model input features), embedding refresh jobs and real-time recommender features — with replayable history for future models. Agent platforms publish each tool call and decision once so observability, evaluation and audit read them without slowing the agent.

In the interview

Asked directly ("how should five teams get order data?") or sideways ("a sixth service needs this too"). Drawing yet another arrow out of the producer each time signals growing coupling. Interviewers also check you don't choreograph a dependent order→payment→shipping flow that needs orchestration.

30-second answer: "Checkout publishes `order.placed` once to a topic. Each team has its own subscription with its own backlog, retries and DLQ, so a slow or crashed consumer affects nobody else; adding one is configuration, not a checkout deploy, and with Kafka it can replay history. I manage the costs: the schema is a public API (additive, registry-checked), every subscription is idempotent, and I alert on backlog age before retention. A workflow needing completion tracking gets orchestrated instead."

Likely follow-ups:

- *"One subscriber runs at half the speed of the others. What happens?"* Only its backlog grows; past retention, events are lost. Watch backlog age and scale that subscription with competing consumers.
- *"How do you change a schema without breaking consumers you don't know about?"* Additive optional fields only, registry-enforced; for a truly breaking change, v2 beside v1 for a deprecation window; the catalog shows who depends on what.

Mistake that fails candidates: wiring several services to one shared queue and calling it broadcast (each gets a sample), or rebuilding request-response from paired events. Both confuse jobs and facts.

Self-check

- **Q1: Topic vs subscription, and what does adding a sixth consumer cost the producer?** → A topic is one durable stream of facts; a subscription is one consumer's feed with its own position, backlog, acks and DLQ. Cost to the producer: nothing — no code, deploy, latency or awareness. The newcomer owns idempotency and backlog monitoring, and the system gains one more dependency on the schema.
- **Q2: Argue against "pub/sub decouples services."** → Consumers stay coupled to the event schema, so a field change can break systems the producer can't see. Pub/sub moves coupling from the release schedule into a data contract, which needs a registry, additive changes, deprecation windows, a consumer catalog and named subscription owners.
- **Q3: Fraud scores ~20% of orders and email sends ~30% of receipts, yet nothing was lost and publish counts match orders. Diagnose.** → The services share one subscription/queue and have become *competing* consumers, so each event reaches only one. Give each service its own subscription; compete only among one service's workers. *Broadcast across services, compete within a service.*

Related patterns

Message Queue (each subscription essentially is one) · Idempotency (N subscribers, N chances of double application) · Event-Driven Architecture (decides what goes in the events) · Change Data Capture (the controlled "just read my tables") · Webhooks (the same idea between companies over HTTP).

TL;DR

- **Problem:** a producer delivering to each consumer itself means a deploy per new team, slower fan-out, and partial crashes where some teams get the event and some don't.
- **Mechanism:** publish each fact once to a topic; each subscription gets its own copy, backlog, retries and DLQ. Subscribing is configuration; log-based brokers add replay.
- **Costs:** schema becomes a public API, duplicates per subscription, invisible coupling without a catalog, no "done" signal, retention as a hard limit.
- **Skip it when:** it's a job (queue), the producer needs the result (request-response/orchestration), or it's a step-by-step workflow (saga).

Event-Driven Architecture

Original lesson: Event-Driven Architecture on DesignGurus ↗

What it is

Event-driven architecture (EDA) means services communicate mainly through events. Its core question: **how much should each event carry** — enough for consumers to act without calling the producer back? **Analogy:** a ticket saying "Order #7f3a" sends the cook to the front desk to ask what was ordered; "3 items, $142, ships to Austin" lets them start cooking. Pub/sub settled *how* events travel; this lesson settles *what goes inside*.

The problem: the callback storm

Checkout publishes `order.placed` once to five subscribers, but each event is only `{event: "order.placed", order_id: "ord_7f3a"}` — a **notification** that an order exists. Fraud then calls `GET /orders/ord_7f3a` for amount and items, as do email, analytics, loyalty and fulfillment (twice, by accident). **Every published event bounced back as five API calls.**

At the next flash sale, 5,000 orders/s became 5,000 events/s became **25,000 GET requests/s** on checkout's API — during the very spike the queue was meant to absorb. Checkout's read service fell over, consumer calls timed out, retries added load, and fraud scoring fell minutes behind in peak fraud season.

Events said *that* something happened, not *what*: **asynchronous delivery wrapped around a synchronous dependency**, not real event-driven design.

Why the obvious fixes fail

- **"Cache the lookups."** Every consumer asks about a brand-new order that didn't exist a second ago, so all five miss and hit the API anyway — and still need checkout up.
- **"Slow the consumers down."** Rate-limiting protects checkout, but caps every downstream team at checkout's capacity, growing backlogs in exactly the bursts that matter. The dependency is postponed, not removed.
- **"Put the details in the event?"** Yes — that's the pattern. Copying data has real costs (below), but the first two ideas make callbacks cheaper while complete events make them unnecessary.

How it works: three levels of event content

```
Level 1: Event Notification     "Order ord_7f3a was placed." → consumers call back
Level 2: Event-Carried State    "Order ord_7f3a: 3 items, $142, customer c_91,
         Transfer                ships to Austin." → consumers act without calling
Level 3: Event Sourcing         The events ARE the data; state is rebuilt by replay
```

- **Level 1, notification.** Says something happened and points elsewhere for details. Small events, no copied data — and exactly what caused the callback storm.
- **Level 2, event-carried state transfer.** The event includes the facts consumers need. Each consumer keeps a **local view** (a small table of just the fields it uses, updated from events). Fraud reads its order fields locally in **under a millisecond**, even with checkout down. Consumers stop being callers and own *disposable copies*. **This is the sensible default.**
- **Level 3, event sourcing.** The event log *is* the **system of record** (the official data) and tables are rebuilt from it — a storage decision with its own lesson, not needed to fix a callback storm.

Three rules for Level 2:

1. **Events are facts, never commands.** "Order placed," not "send a receipt"; a command ties the publisher to one consumer's job, defeating the topic.
2. **Local views are copies, never the truth.** Any view can be wiped and rebuilt by replay; checkout remains the owner. **Each fact belongs to exactly one owning service.** A consumer that spots a wrong value must not patch its copy (creating a rival truth); it sends a command to the owner, which validates and publishes a new fact so every view corrects the same way.
3. **Freshness is about "when," not "if."** Views always lag the source — milliseconds normally, minutes during a backlog. This is **eventual consistency**: copies agree *once* updates finish propagating. Make the lag visible: store the source version or event timestamp in each view, publish view age as a metric, show "as of" times on screens like support dashboards, and alert when a view exceeds its **freshness SLO** (SLO = service-level objective).

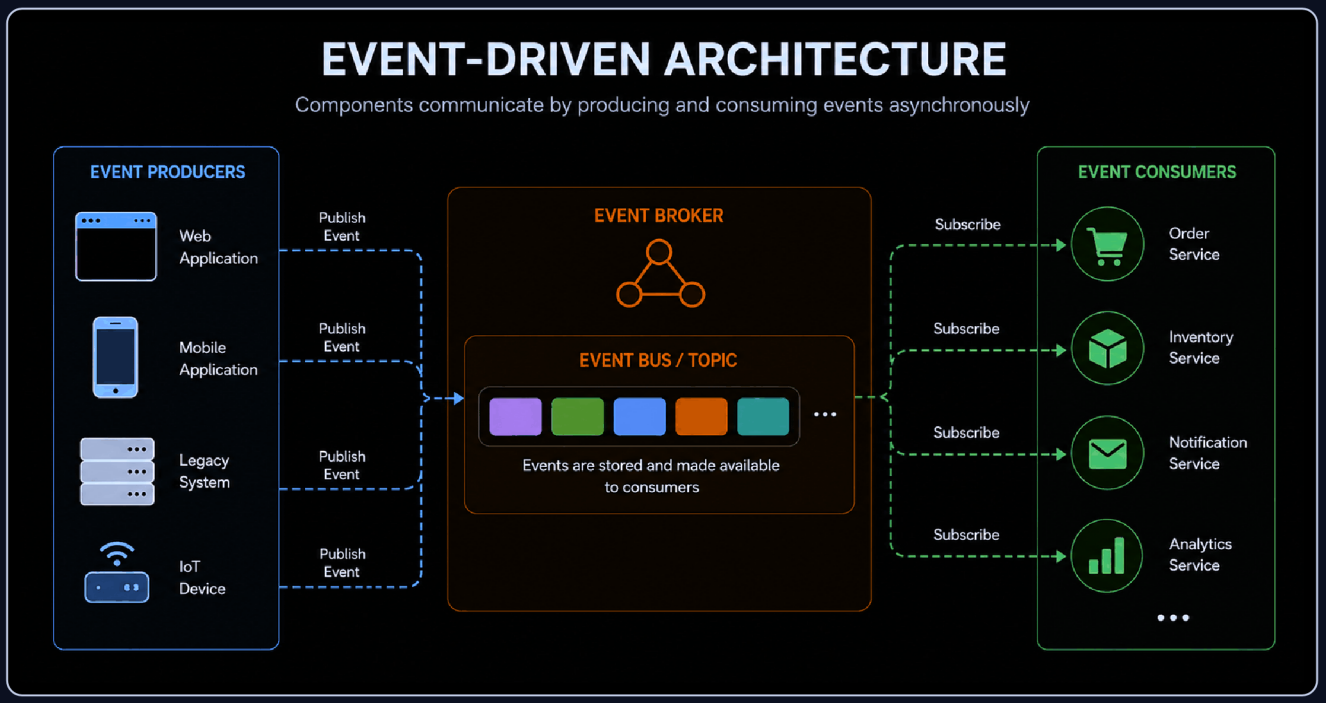
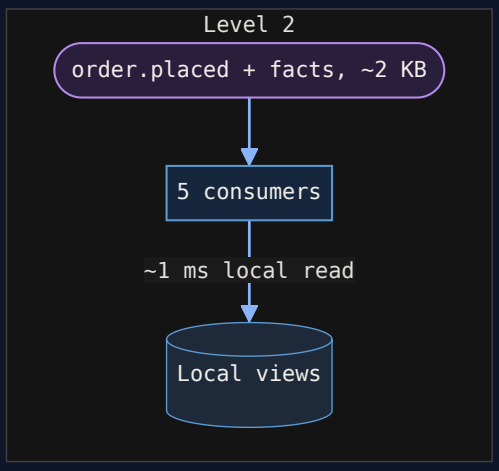


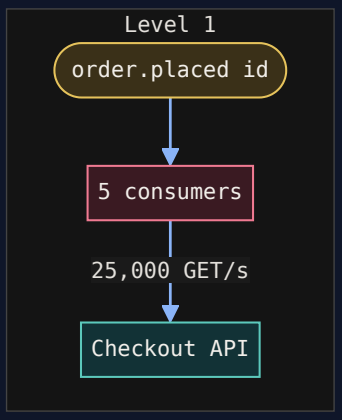
Figure — Event-Driven Architecture (course diagram)

Producers (web, mobile, a **legacy system** — an older system still in use — and an **IoT** device, "Internet of Things" gadgets like sensors) publish to an event bus; consumers subscribe and react asynchronously. With Level 2 events, none calls a producer back.

The two levels you'll actually use:

Callback storm vs local views





Walkthrough with numbers

- **Events grow** from ~100 bytes to ~**2 KB**: at 5,000 events/s the broker moves **10 MB/s**, a small load. In exchange the **25,000 GET/s vanish** — a little bandwidth for independence.
- **Checkout's read traffic stays flat** during the sale, serving real users.
- **Fraud reads locally in ~1 ms** instead of a 50 ms network call with timeout and retry; its speed and availability no longer depend on checkout at peak.
- **A new consumer replays history:** 30 days ≈ **500 million events**, replayed in hours to build its view. Without replay it would **backfill** (load all past data at once) through checkout's API.

Trade-offs

You gain:

- Independence: consumers read locally, survive producer outages, never flood anyone with callbacks.
- Producer load independent of how many consumers there are or what data they want.
- Rebuildable views: recovering a view and onboarding a new one are the same replay.

You pay:

- **Copies everywhere.** Six services may hold order data in six shapes; storage is cheap, but everyone must remember copies are disposable and the owner is authoritative.
- **Constant eventual consistency.** Products must cope with, e.g., a support dashboard 40 seconds behind.
- **Personal data spreads.** A customer email in an event is copied into every subscriber and the retained topic, so deletion touches **N+1** places. Decide *before* publishing: include only widely needed facts, label sensitive fields, record every subscription owner. Encryption protects **data**

at rest (stored, not in transit) but doesn't remove the duty to delete or minimize copies. A convenience field can become a lasting legal obligation.

- **Schema matters more:** bigger events expose more public fields, so registry, additive-only changes and a consumer catalog become mandatory.
- **Harder debugging.** Each service has its own timestamped version of order X, so **correlation IDs** (one ID tagging a business request across events and logs) and **distributed tracing** (recording a request's path and timing across services) are essential.
- **Replay must be safe to rerun.** Rebuilding a view must never resend emails or recharge cards: separate *state-building* consumers from *side-effecting* ones, and test replay on production-sized history before relying on it in an incident.

When not to use it

- **Inside one service:** a **monolith** (one application deployed as a unit) already shares **transactions** (changes that succeed or fail together) and consistency. EDA is for service and team boundaries.
- **When a read must be exactly current:** selling the last item or approving a withdrawal needs the latest value; read synchronously from the owner, and use views for lag-tolerant data.
- **Request-response rebuilt from events:** state-rich events don't help a flow that is still a sync dependency underneath.
- **Before the team is ready:** without schema enforcement, tracing and backlog alerts, EDA is hard to run; fewer services with direct calls may be safer for now.
- **To dodge ownership decisions:** six views with no owner of the fact drift apart with no official answer.

Notification events vs Full-state events

	Notification (Level 1)	State transfer (Level 2)
Event contains	Only an ID ("go look it up")	The facts themselves
When a consumer needs data	It calls the producer	It reads its own local view
Producer read load	Rises with how much consumers ask	Zero
Copies / privacy exposure	Minimal	Every subscriber + the retained history
Freshness	The lookup is always current	Views run slightly behind
Best when	Only a few consumers need details; the data is sensitive	Many consumers need the same data; independence matters

A sensible middle path: decide **field by field**. Widely needed fields go in the event; sensitive or rarely used ones stay with the owner behind an ID lookup.

Real-world examples

- **LinkedIn** built Kafka for this: activity events formed one stream every team read into its own view — the origin of Kafka's "central nervous system" nickname.
- **Uber:** a trip exists as an event stream before it's a database row; pricing, ETA (estimated time of arrival) prediction, fraud and driver payments each consume it independently.
- **Bank ledgers:** an ancient form — immutable transaction facts, with your balance a derived view.
- **Netflix:** viewing events flow through a central stream into recommendations, artwork personalization and capacity planning.
- **For AI engineers:** full-state events keep model features fresh without callbacks. A feature store's online values update from interaction events; a **vector index** (search index over embeddings finding similar meaning) re-embeds content when a `content.updated` event carries it; training pipelines use retained history as a dataset (the same replay that bootstraps a **microservice**, a small service owning one narrow job); a recommender sees a purchase within **200 ms** without calling checkout.

In the interview

"How does B get A's data without calling A?" — or, once you draw a topic, **"what's in the event?"** That choice decides callback load, consumer independence and how far personal data spreads.

30-second answer: "What's in the event is the main decision. Thin notifications make consumers call back, so coupling and load return. For widely consumed facts I carry state in the event and each consumer keeps a local view — replay-rebuildable, fast, available when the producer is down, but eventually consistent, so exactly-current reads go synchronously to the owner. Sensitive fields stay out or are encrypted, the schema is a registry-governed public API, and one service owns each fact."

Likely follow-ups:

- *"A support agent updates an order but doesn't see the change on their dashboard. Why, and what do you do?"* The dashboard's view hasn't caught up with the write. Options: show "as of 12:04:31"; give the writer **read-your-writes** (route their next read to the source); or wait until the view reaches that write's version. Choose per screen.
- *"A user invokes their right to be deleted; their data is in a 30-day retained topic and six local views."* Delete from the owning store and publish a **tombstone** (an event telling every view to wipe that user). If retained history must disappear before natural expiry, use **crypto-shredding**: encrypt personal fields with a per-user key, then destroy the key, making every retained copy, replay and backup unreadable.

Mistake that fails candidates: calling notification events "decoupled" while every consumer calls back, or proposing full-state events without addressing staleness, deletion or ownership.

Self-check

- **Q1: Name the three levels and where the truth lives in each.** → Notification: all truth with the producer. State transfer: producer still authoritative; consumers hold disposable copies. Event sourcing: the log is the truth; tables are rebuilt from it.
- **Q2: Fraud wants `customer.lifetime_value` added to `order.placed` "since it's already flowing." Argue both sides, then decide.** → For: fraud needs it, it's non-breaking, saves a callback. Against: it describes the *customer*, not the order, goes stale fast, and exposes a sensitive business number to every subscriber. Decision: keep `order.placed` about the order; publish lifetime value separately or have fraud read it from the feature store, and join them in fraud's own view.
- **Q3: Eight months in, legal forwards a deletion request. The email exists in checkout's DB, the 30-day topic, and six local views, two with no listed owner. Design the deletion and name the process failure.** → Delete in checkout, publish a tombstone, verify every consumer wipes it; if 30-day expiry misses the legal deadline, crypto-shred. The ownerless views reveal the real failure: subscribing to personal data didn't require a named owner, catalog entry and data-handling approval. Long term, consider removing the email from the event.

Related patterns

Publish-Subscribe (how events travel) · Event Sourcing (Level 3) · Change Data Capture (produces full-state events from a DB change log when the producer can't or won't) · Saga (for multi-step workflows events shouldn't imitate) · Idempotency (view updates arrive at-least-once).

TL;DR

- **Problem:** notification-only events send every consumer back to the producer's API, so coupling and load return, just asynchronously.
- **Mechanism:** put the facts in the event; each consumer keeps a replay-rebuildable local view; one owner per fact, views never the truth.

- **Costs:** data copied to every subscriber, always-lagging views, personal data spread so widely deletion becomes a project, a bigger public schema.
- **Skip it when:** inside one service, the read must be exactly current, it's a sync call or workflow in disguise, or the team lacks schema rules, tracing and alerts.

Webhooks

Original lesson: [Webhooks on DesignGurus ↗]

What it is

A **webhook** is an event that arrives as an **HTTP POST from another organization's system** (POST sends data to a server; GET reads). E.g. your payment provider calls a URL on your server to say "payment succeeded." It's pub/sub across a company boundary over plain HTTP. The receiving rule: **verify it's genuine, save it, answer "received" within milliseconds, and do the real work later behind your own queue. Analogy:** sign for the courier's parcel in seconds and unpack later; keep the courier waiting while you assemble everything and they mark it "failed" and come back with a second copy.

The problem: the handler that did everything

The provider POSTs `payment.succeeded` to `https://api.yourstore.com/webhooks/payments`. The handler marked the order paid, reserved stock, notified the warehouse, sent a receipt, and only *then* returned 200 (HTTP "OK"; status codes are the 3-digit numbers on every HTTP reply, e.g. 401 = "not authenticated", 202 = "accepted, will process later"). Under load that took **~7.5 s**; the provider waits only **5 s**.

So the provider saw **40%** of deliveries fail and **retried** them — but your handler usually finished anyway after the provider hung up, so each retry repeated everything: **two shipments** for real payments. The failure rate crossed the provider's health threshold and, per its published policy, it **disabled your endpoint**.

The failure was silent (no requests, no errors): for **three hours** customers paid while orders stayed "pending," and support tickets found it before monitoring. Unlike earlier lessons, the other side is **another company** whose timeout, retry schedule and disable policy are fixed; you work within them.

Why the obvious fixes fail

- **"Make the handler faster."** 4 s works today, but one slow warehouse call, DB hiccup or spike breaks it again. Business work inside the request still allows delay → retry → duplicate → disable.
- **"Ask for a longer timeout or fewer retries."** Their short timeout protects *them* from your slowness; their retries protect *you* from losing financial events. Removing either shifts risk onto both companies.
- **"Poll their API instead."** Polling every minute burns your **rate limit** (the provider's cap on your API calls), adds up to a minute of delay, and mostly finds nothing. But polling *guarantees* eventual completeness while webhooks give speed — the right answer uses both.

How it works: five receiver rules, in the order a request hits them

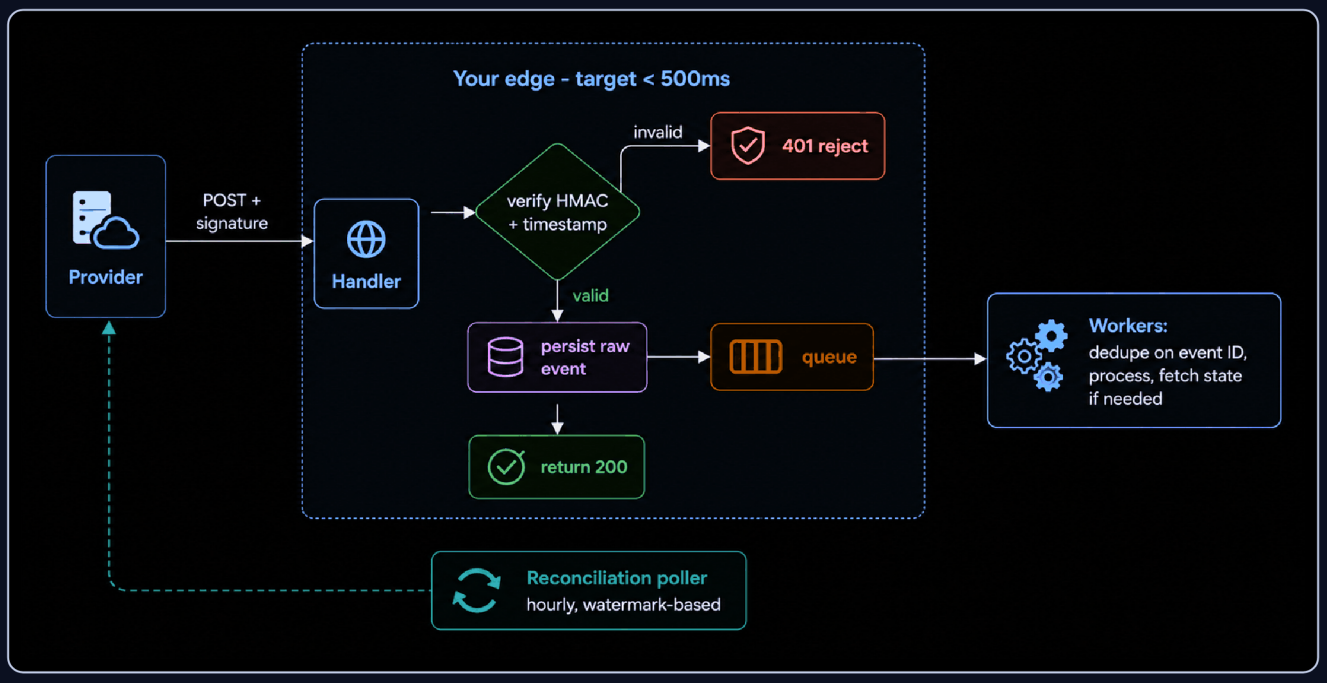


Figure — Webhooks (course diagram)

The figure traces a request through your edge (target under 500 ms): signature check (invalid → 401), raw store, enqueue, 200; workers behind the queue **dedupe** (detect and discard duplicates) and process; an hourly reconciliation poller catches anything missed. The five rules:

1. **Verify before you believe.** The URL is public; anyone can POST `{"type": "payment.succeeded"}`. Providers attach an **HMAC** (a hash of the message keyed with a secret only you and they know). Recompute it over the **raw body** and compare with a **constant-time comparison** (same duration however close the guess, leaking nothing). Check the provider's **timestamp** against an allowed window to block **replay** of old validly signed messages. Reject before parsing. Skip this and anyone can mark unpaid orders paid. (An **IP allowlist** — accepting only the provider's known addresses — is a useful extra layer, not authentication.)
2. **Save and acknowledge; nothing else.** Store the raw event, enqueue it, return 200 in **~15 ms**. **200 means "safely received," never "fully processed."** Inventory, warehouse and receipts happen behind your queue under your own retry and failure rules, so the provider's 5 s deadline no longer touches business processing.
3. **Deduplicate on event ID.** Every provider documents at-least-once delivery. Use each event's unique ID as an idempotency key so repeats take effect once. This prevents double shipments.
4. **Don't trust order or freshness.** "Refund updated" can arrive before "charge succeeded"; a retried payload can be hours old. For state-dependent decisions, treat the webhook as a *heads-up* and **fetch current state from the provider's API**, the source of truth (Stripe's own advice). Skip the fetch for self-contained append-only events, when it would exceed the API rate limit, or when the action is cheap, idempotent and easily corrected.
5. **Reconcile, because silence isn't success.** Events can simply never arrive: endpoint disabled, outage longer than the retry window, a silently broken network path. Periodically ask the provider for "everything changed since my **watermark**" (a saved bookmark of how far you've checked). Webhooks give speed; reconciliation gives completeness. Advance the watermark only *after* recovered events are safely queued (crashing in between creates a new silent gap), record each run's covered time range for audits, and dedupe by event ID across both paths so replaying from an older watermark is harmless.

Also: validate event type and object before enqueueing, and have the worker check each state transition is allowed. A valid signature proves *who* sent it, not that the requested change is permitted.

If you're the sender, these become your duties: sign with **rotatable secrets** (swappable keys), give every event a unique ID, publish your retry/backoff schedule, keep delivery timeouts short, use a durable queue per endpoint, auto-disable persistently failing endpoints, and provide a dashboard with a

delivery log (event ID, destination, attempt number, response code, latency, next retry; no secrets) plus a **redeliver** button, so customers can tell "never attempted" from "your server rejected it."

Walkthrough with numbers

- **Handler time ~15 ms:** ~1 ms signature check, ~10 ms store and enqueue, then 200. No slow dependency in the request, so the provider sees **100% success** through spikes and deploys.
- **The real work** runs behind the queue as in the queue lesson: spikes become backlog, crashes become redeliveries, persistent failures go to the DLQ.
- **A duplicate** arrives 90 s later because the first 200 was lost in transit. The worker sees the event ID already handled and stops: **one shipment** — idempotency across a company boundary.
- **Hourly reconciliation** normally finds nothing; during one provider delivery incident it found **217** missing confirmations and quietly queued them.
- **A silence alarm:** if a normally busy endpoint is quiet for **15 minutes**, **page** someone (urgent alert to the on-call engineer). It catches failures that produce no local errors.

Trade-offs

You gain:

- Near-real-time updates from other companies without constant polling.
- The provider's retry machinery working *for* you, now that you ack fast.
- One consistent entry for external events, feeding your existing queue-and-worker setup.

You pay:

- **You inherit their rules:** each provider's delivery guarantees, ordering, retry schedule, disable policy and payload versioning — a separate contract per integration.
- **A public endpoint to defend.** Signature checks, timestamp windows and secret rotation join your security boundary; a leaked signing secret lets anyone fake "payment succeeded."
- **Cross-company debugging.** "They never sent it" vs "we dropped it" needs their dashboard, your raw-event store and alignable timestamps; keep one **correlation record per event ID** spanning receipt, queue processing, business effects and reconciliation.
- **The polling loop is permanent.** Both run forever in production; monitor reconciliation lag too.

When not to use it

- **Between your own services:** a webhook imitates a broker over HTTP without durable subscriptions, replay or managed redelivery. Use pub/sub when you own both ends.
- **When completeness beats speed:** billing reconciliation, financial close and compliance exports start from polling or bulk-export APIs, with webhooks adding freshness.
- **Very high-frequency streams:** thousands of POSTs per second is wasteful; use the provider's streaming or bulk interface.
- **When someone waits for the answer:** interactive questions need request-response.

The incident's root mistake was using the webhook request to *do* the business work instead of just *accepting* the notification.

Webhooks vs Polling

	Webhooks (push)	Polling (pull)
Freshness	Seconds	As often as you poll
Cost when idle	Close to zero	A steady stream of "anything new?" requests
Completeness	Best effort; gaps can happen	Guaranteed, in the end
Who controls delivery	The sender	You
Worst failure looks like	A quiet day	Stale (but visible) data

The danger is the last row: a failed delivery looks *exactly* like no activity. Hence **webhooks for speed, polling for truth**, with a bookmark so polling catches what push missed. (For continuous streams to browsers or apps you control, persistent connections such as Server-Sent Events replace separate POSTs; webhooks are specifically server-to-server push across company boundaries.)

Real-world examples

- **Stripe:** the model — signed payloads with timestamp checks, event IDs for dedupe, retries spread over days, automatic disabling, and official advice to fetch current state rather than trust payload freshness.
- **GitHub:** signs every delivery with the `X-Hub-Signature-256` header and offers a delivery log with a redeliver button — sender duties done well.
- **Slack:** demands a 200 within **3 seconds**; no business logic fits, deliberately forcing accept-now, work-later.
- **Twilio, Shopify, PayPal:** every integration platform converges on the same terms because they face the same failures.
- **For AI engineers:** async AI services report completion by webhook (batch inference, long fine-tuning jobs); receivers still verify, store, ack fast and queue. Agent products receive webhook triggers from email, calendars and CRMs (customer relationship management tools such as Salesforce), where dedupe matters doubly: a repeated event can rerun an expensive agent.

In the interview

Asked directly ("integrate Stripe") or indirectly ("how does payment settlement reach your system?"). Expect probes on work done before the 200, duplicates, and detecting events that never arrive — the completeness question reveals the most.

30-second answer: "The handler verifies signature and timestamp, saves the raw event, enqueues it, and returns 200 in ~15 ms — 200 means received, not processed. Workers dedupe on event ID (delivery is at-least-once) and fetch current state from the provider before state-dependent decisions. Because webhooks are best-effort and the worst failure is silence, I add hourly reconciliation from a bookmark and an alarm when a busy endpoint goes quiet. Speed from push, truth from poll."

Likely follow-ups:

- *"How do you know you missed one?"* The webhook path alone can't tell — a missing event leaves no trace. Reconciliation eventually fills every gap and the silence alarm catches large gaps fast. Treat "no traffic" as a monitored state.
- *"Secure the endpoint."* Receiver rule 1: HMAC over the raw body, constant-time compare, timestamp window against replays, safe secret rotation, reject before parsing. IP allowlists add a layer; signatures authenticate.

Mistake that fails candidates: processing business logic before replying, or ignoring duplicate delivery. Either design works only while every dependency behaves.

Self-check

- **Q1: List the five receiver rules in order and say exactly what 200 means.** → Verify signature + timestamp; durably save the raw event; ack 200 within milliseconds; process later via a queue with event-ID dedupe (fetching current state where needed); reconcile from a bookmark. 200 means the event is safely yours, *not* that its effects are done.
- **Q2: The payload already has everything; why still fetch current state, and when would you skip it?** → The payload reflects state when that delivery attempt was created; after retries it can be stale or out of order, while the API holds current official state. Skip for self-contained append-only events, when volume would exceed the rate limit, or when the action is idempotent and cheap to fix.
- **Q3: Confirmations quietly stopped; no handler errors for 3 weeks; the dashboard shows the endpoint was auto-disabled 20 days ago after a failure spike during a deploy. Reconstruct and redesign.** → The deploy caused slow responses or errors, retries hit the same problem, the failure rate triggered auto-disable, and deliveries stopped with no local error. Redesign: a 15 ms save-and-ack handler (no business work inline), a silence alarm catching a disable within minutes, hourly reconciliation bounding any gap, and deploy checks confirming endpoint health. Never let completeness depend solely on another company's retry window.

Related patterns

Message Queue (the handler only feeds it) · Idempotency (event-ID dedupe is its cross-company form) · Publish-Subscribe (what a webhook structurally is, with guarantees renegotiated as contract terms) · Retry with Exponential Backoff (runs on the sender's side, for or against you).

TL;DR

- **Problem:** external events arrive as POSTs under the sender's timeout, retry and disable rules; working inline leads to duplicates, a disabled endpoint, and a silent 3-hour gap found by support tickets.
- **Mechanism:** verify signature + timestamp, save raw, 200 within ms, process behind your queue with event-ID dedupe, fetch current state for decisions, reconcile by polling from a bookmark.
- **Costs:** inherited sender rules, a public endpoint to defend, cross-company debugging, a permanent polling check.
- **Skip it when:** you own both ends (pub/sub), completeness is the requirement (poll first), or volume calls for a stream.

Server-Sent Events

Original lesson: Server-Sent Events on DesignGurus ↗

What it is

Server-Sent Events (SSE) is one HTTP response that never finishes: the client connects once, and the server keeps the connection open, sending a small text message whenever there's news. **Analogy:** instead of phoning the pizza place every 3 minutes (polling), you stay on the line and they tell you the moment anything changes.

The problem: polling that mostly says "nothing"

A food-delivery page shows driver location, order status and ETA by **polling** `GET /orders/{id}/status` every 3 seconds — simple, reliable, in production.

On a rainy Friday, **200,000** customers have it open: one request per 3 s each = **~66,000 requests/s**, each a full round trip with TLS, authentication, a DB read and a response, and **99 in 100 say "no change."**

Users still find updates slow: a new status waits **1.5 s on average** for the next poll. Polling every 1 s would mean **200,000 req/s** — for an order that changes maybe **12 times in 40 minutes**. The lesson's key line: **with polling, cost grows with the audience; it should grow with the news.**

Why the obvious fixes fail

- **"Tune the interval."** Shorter is fresher but costlier, longer cheaper but staler — the same trade-off curve. The server *knows* when data changes; it has no way to say so.
- **"Long polling."** The server holds each request until data changes or a timer expires, and the client immediately asks again. That removes most empty responses and staleness (an older form of push), but every update still costs a disconnect and new request, so bursts of updates mean bursts of reconnections, and proxy timeouts need care. Push faked through repeated pulls.
- **"Just use WebSockets."** Works, but gives *two-way* communication to a page that only listens; the unused direction still costs a protocol upgrade, special load-balancer handling and your own reconnect logic (see the table below).

How it works

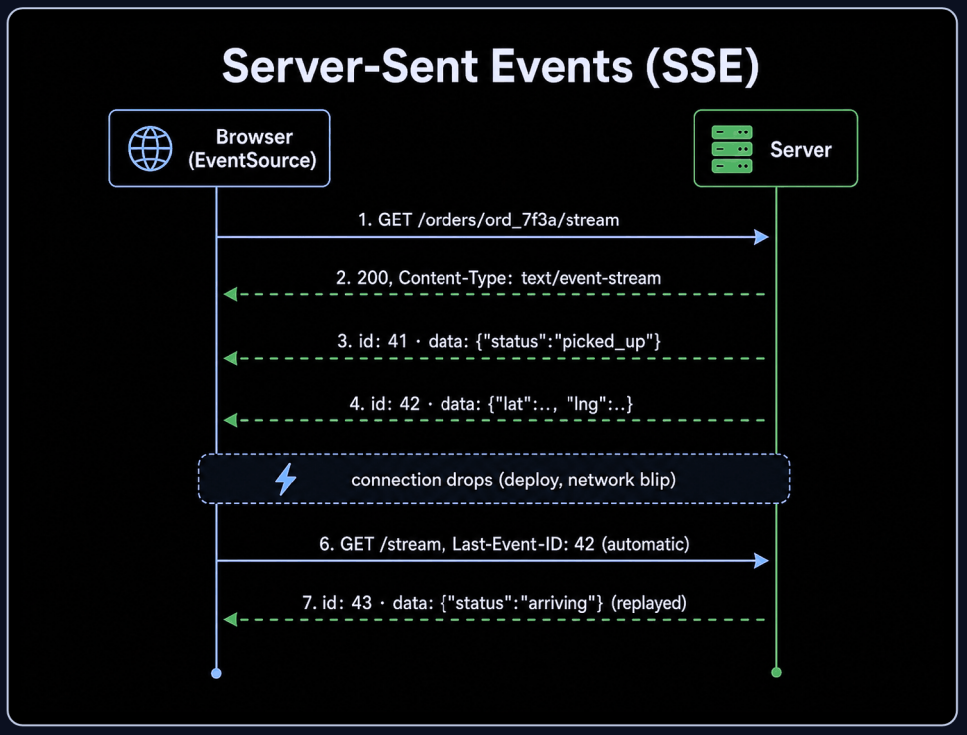


Figure — Server-Sent Events (course diagram)

The whole protocol: `EventSource` opens `GET /orders/ord_7f3a/stream`; the server replies `200` with `Content-Type: text/event-stream`, keeps it open and pushes numbered events (`id: 41`, `id: 42`); after a drop the browser reconnects **automatically** with `Last-Event-ID: 42` and the server replays what was missed (`id: 43`). Four properties make it work:

1. **It's just HTTP.** A normal response held open with content type `text/event-stream` — no upgrade or special handshake — so existing load balancers (servers spreading requests across backends), TLS, authentication **middleware** (shared code running on every request before your handler) and proxies all handle it. Most of its operational simplicity comes from this.
2. **The browser reconnects.** Deploys, tunnels and closed laptops break connections constantly; `EventSource` reconnects on its own with a `Last-Event-ID` header naming the last message received. The server keeps a short **replay buffer** of recent events and resends everything after that ID. A reconnect can deliver a message twice, so **event IDs double as dedupe keys** and the client skips IDs already handled.
3. **One direction, by design.** The server talks, the client listens; client actions (e.g. cancelling an order) go via ordinary request-response. Many real-time UIs have exactly this shape.
4. **Held-open connections change the server math.** 200,000 connections can't each hold a thread. **Async I/O** (Node, Go, Nginx; I/O = input/output such as network reads; *async* = the thread doesn't block waiting) lets one **event-loop** thread watch thousands of idle connections, working only when data arrives. At a few tens of KB of memory per connection, one server holds **tens of thousands of connections**.

The SSE server usually gets updates by **subscribing to internal pub/sub** and forwarding each event to the right open streams. **Keep streams tightly scoped:** each carries only events the logged-in user may see, with routing rechecked after account or membership changes. Piping a broad internal topic to browsers turns a routing bug into a data leak.

Infrastructure gotchas. Some load balancers, **reverse proxies** (servers in front of your app forwarding requests to it) and corporate networks **buffer** responses (collect output before passing it on) — harmless normally, fatal for streams: events arrive in 30–60 s batches. So:

- Disable buffering on stream routes (e.g. header `X-Accel-Buffering: no`, plus `Cache-Control: no-cache`).
- Raise idle timeouts and send a **heartbeat** comment every **~15 s** so middleboxes don't close a seemingly idle connection.
- Use **HTTP/2** (newer HTTP carrying many requests over one connection) at the **edge** (internet-facing servers) so streams share connections, avoiding browsers' old **6 connections per domain** limit.

Authentication must outlast a normal request. Check the session when the stream opens and close it when credentials expire or are revoked, since one connection may stay open for hours; the reconnect re-authenticates normally.

Walkthrough with numbers

- **Nothing happens, nothing is sent** (except heartbeats): 200,000 open connections, almost no requests, so the **66,000 req/s** of "nothing yet" disappear. Work follows events, not audience.
- **News arrives in under a second**, versus up to 3 s staleness with polling.
- **Capacity: ~50,000** connections per async server → **4 machines** for a rainy Friday. Cost shifts from CPU (66k req/s) to memory (held connections).
- **Deploy drill:** restarting a server drops 50,000 connections that all reconnect; arriving at the same instant, that **reconnect stampede** can overwhelm the fleet. The SSE `retry:` field lets the server set the reconnect delay; randomize it (**jitter**) to spread load. Returning clients send `Last-Event-ID` and the buffer fills gaps, so keep **several minutes** of events — longer than a typical deploy.

Trade-offs

You gain:

- Push over plain HTTP: middleware, proxies and tooling work unchanged.
- A built-in browser client with reconnect-and-resume you didn't write.
- Cost that tracks events, not audience size or poll frequency.

You pay:

- **One direction only.** Once the client must stream typing, cursors or voice back, use Bidirectional Streaming.
- **Long-lived connection operations.** Plan capacity by *connections*, not requests. **Drain** servers before deploys (stop accepting new connections; let existing ones move). Use **least-connections** balancing, since similar request rates can hide very different stream and memory counts. Treat active connections, memory per connection, disconnect/reconnect rates and replay-buffer misses as top-level metrics; load-test idle connections and simultaneous broadcasts separately, as they strain different resources.
- **A resume window, not a history.** The buffer covers short disconnects, not yesterday. A client returning after its ID has aged out must detect the gap and refetch current state through the normal API. Make it explicit in the protocol: the server sends a named reset event (or closes with a "refetch" status), and the client replaces local state with refetched data *before* reopening, so old and new never mix. **The stream is for delivery, never for storage.**
- **Text only.** Text and **JSON** (common structured text format) are fine; binary audio/video needs a media protocol.

When not to use it

- **The client talks too** (typing indicators, collaborative editing, multiplayer): Bidirectional Streaming.
- **Rare updates, many viewers:** a once-a-day price doesn't justify a million idle connections; relaxed polling plus a cache is simpler.
- **Latency-critical binary media** (voice, video): **WebRTC**, built for media.
- **Between your own backends:** pub/sub or **gRPC** streams (typed server-to-server streaming).

Anti-patterns: homemade chat from SSE down plus constant POSTs up (a two-way protocol in disguise), and using the stream as storage with no recovery once a client falls outside the replay window.

Four levels of push

Pick the lightest method that meets your direction and freshness needs:

	Polling	Long polling	SSE	WebSocket
Staleness	~Half the interval	~Zero	~Zero	~Zero
Cost when idle	A constant flow of requests	One held request per client	One held connection per client	A held connection + the protocol
Direction	Pull	Pull pretending to be push	Server → client	Both ways
Works with existing proxies/infra	Perfectly	Timeout problems	Yes, it's plain HTTP	Needs special handling
Reconnect handling	Not needed	You write it	Built into the browser	You write it
Best for	Rare changes, relaxed deadlines	Old infrastructure limits	Feeds, tickers, progress, LLM (AI chat model) tokens	Conversations

Ask **who has data to send**: poll when the server speaks occasionally, SSE when it speaks constantly, WebSocket when *both* do. Long polling survives mainly where old infrastructure can't carry SSE reliably.

Real-world examples

- **Every LLM chat app streams over SSE.** An **LLM** (large language model) produces text in small pieces called **tokens** (roughly word fragments); Anthropic's and OpenAI's APIs send them as SSE events, which is why words appear one at a time. A perfect fit: one-way, text, plain HTTP every proxy and **SDK** (software development kit: a ready-made client library) understands.
- **Live dashboards and tickers:** log tails, stock and sports feeds, live news.
- **Build and deploy logs: CI** (continuous integration: servers that build and test every change) output streaming into your browser is nearly always SSE.
- **Notification feeds:** one SSE stream per session for all in-app events (the capstone builds this).
- **For AI engineers:** token streaming makes latency *feel* shorter: a 30 s answer feels snappy if the first token arrives in **300 ms**, so **time-to-first-token** is the metric that matters. A buffering proxy ruins it. Agent UIs send **named event types** (tool started, result arrived, thinking), and an LLM gateway must pass streams through rather than collecting whole responses.

In the interview

Live order tracking, notification feeds and token-by-token AI output all test the same skill: choosing among the four push levels instead of defaulting to WebSockets or sticking with expensive polling.

30-second answer: "For one-way live updates, SSE: one long-lived HTTP response per client, server pushes as things happen, so cost tracks news, not audience — polling here would be hundreds of thousands of mostly empty requests a second. It's plain HTTP, so auth, load balancers and proxies just work. EventSource reconnects with Last-Event-ID against a replay buffer sized to cover a

deploy; async I/O holds tens of thousands of connections per server; jittered reconnects prevent stampedes. WebSockets only once the client must stream back."

Likely follow-ups:

- *"A server holding 50,000 streams restarts. Walk me through it."* Drain, then jittered reconnects with `Last-Event-ID` against a buffer sized to the deploy (the deploy drill above).
- *"Why not WebSockets for everything?"* Two-way needs a protocol upgrade, special infrastructure handling and your own resume logic — all wasted when the client only listens. Move up only when you need upstream streaming.

Mistake that fails candidates: defaulting to WebSockets for a read-only feed; dismissing per-user polling as "cacheable" (each user's status differs); or treating the stream as storage with no refetch path past the replay window.

Self-check

- **Q1: What single transport fact explains most of SSE's operational advantages, and which two mechanisms make a dropped connection invisible?** → It's plain HTTP with no upgrade, so existing TLS, auth, load balancers, HTTP/2 and proxies carry it. Invisible drops come from EventSource's automatic, jittered reconnect plus `Last-Event-ID` against a server replay buffer (event IDs also dedupe).
- **Q2: The replay buffer holds 60 s; deploys take 4 minutes. What happens, to whom, and what are your three options?** → Clients disconnected early in the deploy return after their last event aged out and silently miss events unless the server detects it. Options: enlarge the buffer; have clients refetch current state before reopening (needed regardless); drain servers so disconnections are shorter.
- **Q3: Corporate-network users get "live" updates in 30–60 s bursts; home users are smooth; server logs show immediate writes. Diagnose and fix.** → A corporate proxy is buffering. Disable buffering on the route (`X-Accel-Buffering: no` or equivalent), set `Cache-Control: no-cache`, send heartbeats, prefer end-to-end HTTP/2; where you can't change their infrastructure, detect the batching and fall back to long polling. The SSE path includes *every* proxy between server and browser.

Related patterns

Publish-Subscribe (the news source) · Request-Response (client actions upward; refetch after a resume gap) · Bidirectional Streaming (once the client talks) · Load Balancing (least-connections and draining for held connections).

TL;DR

- **Problem:** polling makes cost grow with audience and staleness with interval; heavy infrastructure mostly answers "nothing yet."
- **Mechanism:** one never-ending HTTP response per client; server pushes text events; EventSource auto-reconnects with Last-Event-ID against a replay buffer; async I/O holds connections; jittered reconnects prevent stampedes.
- **Costs:** one direction, long-lived connection operations (draining, heartbeats, proxy buffering), a resume window not a history, text only.
- **Skip it when:** the client streams too (WebSockets), changes are rare (polling), the payload is media (WebRTC), or both ends are your servers (pub/sub).

Bidirectional Streaming

Original lesson: [Bidirectional Streaming on DesignGurus ↗]

What it is

Bidirectional streaming keeps one connection open over which *both* sides send whenever they like: chat, collaborative editing, multiplayer games, live voice. In browsers it's usually a **WebSocket** (an HTTP connection "upgraded" to a two-way channel); between backends, a **gRPC** bidirectional stream. The lesson's insight: opening the socket is easy; the hard part is that a conversation is **shared state spread across many machines**. **Analogy:** placing a call is simple; running a conference call where participants dialed into *different* switchboards means someone must relay each speaker to every switchboard with a listener.

The problem: messages that silently vanish

A team launches WebSocket chat; it works in the demo and in **staging** (a test copy of production). Production has **three gateway servers** (a **gateway** holds client connections) behind a load balancer. Alice connects to server A, Bob joins the same room via server B. Alice sends "hey"; server A checks its local list for the room, sees only Alice, and Bob gets nothing.

No errors anywhere: same-server users chat fine, cross-server messages vanish or flow one way. Room membership lived only in each server's memory, and staging's single server hid it.

The socket was never the hard part. SSE usually sends outward from one central source; in two-way chat a *client-created* message must reach recipients connected to other machines.

Why the obvious fixes fail

- **"Sticky sessions"** (the load balancer keeps sending a client to the same server). Needed, since a live connection can't move, but it pins *clients*, not *rooms*: Alice and Bob can still land on different servers. Necessary, not sufficient.
- **"Route each room to one server."** Works while a room fits one machine, but big all-hands rooms outgrow it, one server failure takes down all its rooms, and rebalancing busy rooms means moving state. You've accidentally built a stateful, sharded, failover-capable distributed system without designing one.
- **"One big server."** Vertical scaling delays the limit but doesn't remove it, and its failure disconnects everyone.

All three try to solve **message routing** by controlling **where connections land**. The scalable design separates the two.

How it works: two tiers

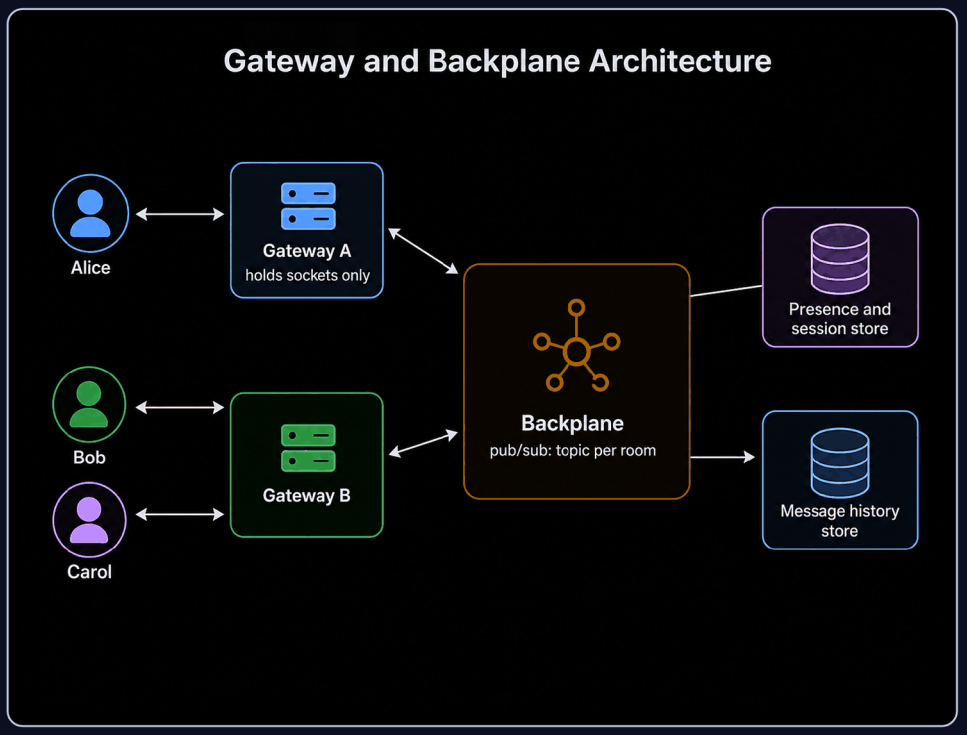


Figure — Bidirectional Streaming (course diagram)

Alice's socket is on Gateway A ("holds sockets only"); Bob and Carol are on Gateway B. Both connect to a **backplane** (pub/sub, one topic per room), alongside a **presence and session store** (who's online) and a **message history store** (the durable record of every message).

The reasoning:

1. A connection is physical: it exists on exactly one machine.
2. A room is people whose connections are on different machines.
3. So machines must relay messages, but having them track "who is on which machine" is fragile.
4. Reuse pub/sub instead: **each room is a topic.** Every gateway subscribes to the topics of rooms its users are in; sending to a room means publishing to its topic; each gateway forwards to its local sockets. **No gateway needs to know any other exists.**

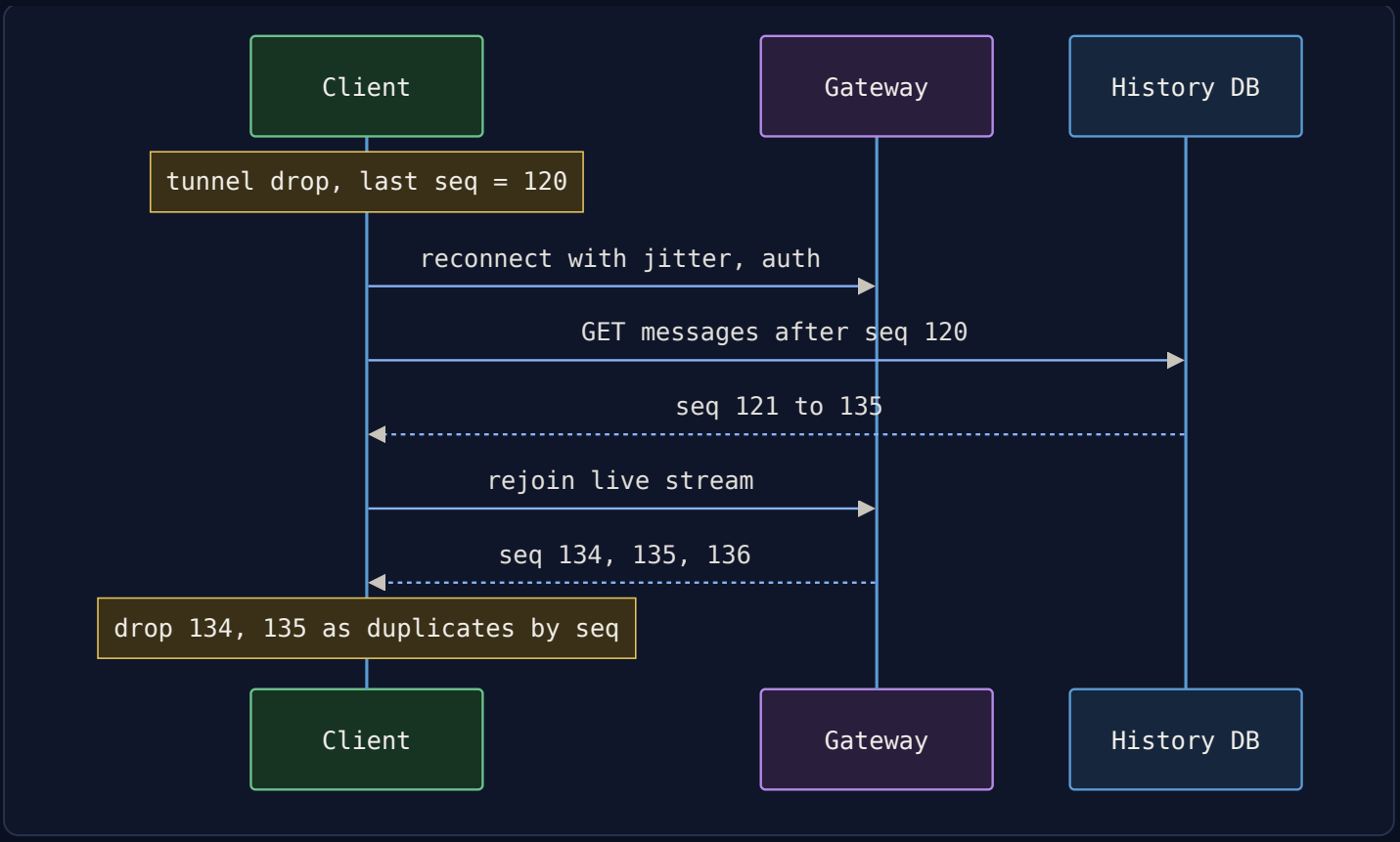
Alice's "hey" goes socket → Gateway A → room topic → every subscribed gateway (A and B) → local sockets, fixing the launch bug by design. **Presence** lives in a shared store and **messages** in a history database: **the stream carries data; the history database holds the truth.**

You now own a protocol. WebSocket or gRPC gives raw transport only; everything `EventSource` gave SSE for free you must design:

1. **Framing and message types, with versioning.** Separate **durable messages** from **ephemeral signals**. A chat message needs ID, sender, room, sequence number, timestamp and version because it's stored and may be replayed; a typing indicator is overwritten by the next, expires on its own and is never stored. The split keeps cheap signals from getting costly durability guarantees.
2. **Sequence numbers** assigned at publish per room, so clients detect duplicates and gaps.
3. **Heartbeats:** regular ping/pong so both ends detect a dead connection instead of sending into the void.
4. **Auth and acks.** Authenticate on socket open and recheck periodically in long sessions (room access or account status can change). Where delivery matters, add application-level **acks**: a successful socket write does *not* mean the client received it.
5. **Reconnect and resume.** The client stores its last sequence number per room; after reconnecting it fetches missed messages from history (request-response), rejoins the live stream and drops overlap by sequence number. Resume makes delivery at-least-once, so sequence numbers are dedupe keys. **History must be written before publishing to the backplane, or in the same atomic step** (*atomic* = fully happens or not at all), or the gap query can miss messages.
6. **Backpressure.** If a client receives slower than the system sends, an unbounded buffer for one weak mobile connection can exhaust gateway memory and take down everyone on it (the shared-resource risk the **bulkhead** pattern addresses). So cap each connection's outbound buffer, **coalesce** superseding updates (typing and cursors need only the latest), and at the cap **disconnect with a "please resync" hint**; normal reconnect-and-resume then restores durable messages safely.

Resume after a dropped connection:

Reconnect and resume via history



The overlap (134, 135) is dropped by sequence number, leaving a gap-free conversation.

Walkthrough with numbers

- **One million connected users** at **~50,000 sockets per async gateway** → **20 gateways**; adding gateways adds connection capacity without changing where rooms live.
- **Message path:** two internal hops (gateway → backplane → gateway), a few tens of ms end to end. A 50-member room costs **one publish**, not 50 sends from Alice's server.
- **Typing indicators** can generate **50x** the traffic of real messages if every keystroke goes to every member. Throttle to **at most 2 updates/s per typer**, latest wins, in the gateway, never stored.
- **Deploy drill:** stop new connections to the gateway (load balancer routes elsewhere), send clients a "going away" notice, let them reconnect elsewhere with jittered delays (avoiding a stampede), resuming by sequence number from history.
- **Slow consumer:** a subway rider's buffer hits its **1 MB cap**; the gateway disconnects with a resync hint instead of hoarding. **~30 s** later the client reconnects and fetches missed messages from history. The stream never served as storage.

Trade-offs

You gain:

- A real conversation: both sides send any time over one connection with minimal per-message overhead.
- The lowest interactive latency in the HTTP family.
- One channel per session for messages, **presence** (who's online) and signals.

You pay:

- **You own the protocol:** framing, sequencing, heartbeats, resume, versioning — including failures HTTP and SSE handled for you.
- **A stateful edge:** gateways need socket limits, planned draining, and connection-aware balancing (least-connections *and* stickiness).

- **HTTP tooling goes blind:** after the upgrade, auth, rate-limiting and tracing middleware can't see individual messages, so you need socket-aware versions.
- **Backpressure must be designed**, since any unbounded buffer eventually eats all memory.
- **The backplane is another system** whose throughput limits **fan-out** (recipients one message can reach). Load-test big rooms (one message to 100,000) separately from total volume (100,000 private messages), and measure backplane delay, gateway write queues and history writes separately to locate the bottleneck.

When not to use it

- **The client rarely sends:** SSE down plus ordinary POSTs up. Sockets pay off for *frequent, latency-sensitive* two-way data (typing, cursors), not a Like button.
- **Request-response over a socket:** `{"method": "getUser"}` and wait rebuilds HTTP without status codes, caching or standard tools. Calls with responses stay calls.
- **Between your own backends:** native gRPC streams bring typing, **multiplexing** (many streams over one connection) and service-mesh support; use a queue when work must survive restarts.
- **Media:** WebRTC is built for low-latency, loss-tolerant audio/video (its setup may use a WebSocket).

Once a socket exists, teams are tempted to route forms, uploads and fetches through it, creating a private protocol without HTTP's tools. Keep the socket for continuous two-way traffic.

WebSocket vs gRPC bidirectional stream

Same communication shape, different homes:

	WebSocket	gRPC bidirectional stream
Natural home	Browsers, mobile apps, the public internet	One backend service talking to another
Message format	You define it (raw frames)	Typed **protobuf** (Google's compact binary message format) messages, with generated code
Through proxies/firewalls	Excellent (it looks like normal HTTPS)	Unreliable on the public internet
Tooling	Needs special handling everywhere	Fully supported in gRPC and service meshes
Best for	Chat, collaboration, multiplayer	Internal control planes, live links between services

Rule: **WebSocket faces users; gRPC streams face services. Envoy's xDS** is a typical gRPC stream, pushing configuration over long-lived typed connections to proxies in a **service mesh** (small proxies beside every service handling networking, retries and security). **WebRTC** is a third option for peer-to-peer media, using these transports only for setup. Failure handling differs: mobile and browser clients switch networks, sleep and sit behind infrastructure you don't control, so WebSocket designs need explicit reconnect and history recovery; backend gRPC clients run on networks you control but still need deadlines, **flow control** (stream-level backpressure) and a clear resume policy after service restarts.

Real-world examples

- **Slack:** this exact two-tier design (WebSocket gateways plus backplane); its write-ups on draining and reconnect stampedes mirror this lesson's walkthrough.
- **Figma:** multiplayer editing is WebSocket transport with **CRDTs** on top (conflict-free replicated data types: structures that merge concurrent edits deterministically). Interview line: *the transport delivers; the data structure resolves concurrent edits.*
- **Discord:** millions of concurrent sockets on a small, fully disposable gateway tier.
- **Envoy xDS and Kubernetes watch APIs:** long-lived machine-to-machine streams pushing **control-plane** updates (configuration and coordination, not user data; Kubernetes is the dominant system for

running containers across machines).

- **For AI engineers:** realtime voice sends audio both ways over one WebSocket, enabling **barge-in** (interrupting the model mid-speech), impossible on a one-way channel since the client must send while the server streams. Same concerns apply: number audio frames, apply backpressure when generation outruns playback, resume when a phone switches networks. Token-only output stays on SSE; interactive conversation justifies the socket.

In the interview

Slack, collaborative editor and multiplayer questions often start with "use WebSockets" and then probe topology: **"how does Alice on server A reach Bob on server B?"** and **"what happens to a slow consumer?"** (unbounded buffers can kill a gateway).

30-second answer: "Two layers. Gateways hold WebSocket connections and nothing authoritative; a pub/sub backplane, one topic per room, routes each message to the gateways holding recipients. Presence is in a shared store and history in a database, so the stream is never the source of truth. My protocol on the socket: sequence numbers, heartbeats, and resume from history with dedupe by sequence. Send buffers are capped, ephemeral signals merged latest-wins, and past the cap I disconnect with a resync hint. Gateways are disposable: drain, deploy, reconnect with jitter."

Likely follow-ups:

- *"Walk Alice's message to Bob on a different server."* Gateway A publishes to the room topic; Gateway B (subscribed because it holds Bob) writes to Bob's socket; A serves its local members. History is written at publish, giving resume one source.
- *"A client receives at 1 KB/s in a room producing 50 KB/s. What happens?"* Coalesce ephemeral updates; if the capped buffer still fills, disconnect with a resync signal and let resume restore history. Gateway memory stays safe.

Mistake that fails candidates: room membership only in gateway memory, no reconnect-and-resume, or WebSockets for a stream where the client never sends.

Self-check

- **Q1: What must you build above a raw socket that SSE's EventSource gave you free, and where does truth live?** → Jittered reconnect, sequence-based resume, framing and versioning, acks where needed, heartbeats, auth at open plus rechecks, and a backpressure policy. The history store owns messages and a shared store owns presence; sockets and backplane only move data.
- **Q2: Why gateways plus a backplane rather than gateways messaging each other directly? Which state lives where?** → Direct routing needs every server to keep an always-correct global table of room members' locations through connects, disconnects and failures — the same fragility as the queue lesson's in-memory buffer. The backplane centralizes subscriptions; each gateway knows only its sockets and topics. Gateways: sockets + capped buffers. Backplane: routing subscriptions. Shared store: presence and sessions. History DB: messages.
- **Q3: Mobile users lose messages after elevators and tunnels; connections do come back, but conversations have gaps. Design the fix end to end.** → Reconnect without resume skips everything published during the outage. Number messages at publish, store them in history (before publishing or atomically with it), have clients save the last sequence per room, and after reconnecting fetch newer messages via request-response, apply them, rejoin live and drop overlapping sequences. It's SSE's `Last-Event-ID` rebuilt for a two-way protocol.

Related patterns

Publish-Subscribe (the backplane is one) · Server-Sent Events (the simpler one-way option whose resume design this borrows) · Load Balancing (stickiness, least-connections, draining) · Idempotency (sequence numbers are the dedupe key).

TL;DR

- **Problem:** a conversation is shared state across connections on different machines; past one server, "send to the room" becomes routing. The socket was never the hard part.
- **Mechanism:** gateways hold only sockets; a pub/sub backplane routes by room; presence in a shared store; truth in a history DB; your own protocol of sequences, heartbeats, resume and capped-buffer backpressure.
- **Costs:** you own the protocol, a stateful edge with draining and stampedes, HTTP middleware blind after upgrade, a backplane limiting fan-out.
- **Skip it when:** the client rarely sends (SSE + POST), it's request-response in disguise, it's service-to-service (gRPC), or it's media (WebRTC).

Capstone - Designing a Notification System

Original lesson: Capstone - Designing a Notification System on DesignGurus ↗

How this capstone works

You **pick one pattern per arrow**, and a notification system uses every direction in this module. At each decision ask: **who has the information, is anyone waiting, is this a job or a fact, and who controls the destination?**

The brief

The store now has **5 million registered users** and needs one notification platform.

- **Triggers:** order updates, shipping events, security alerts, marketing campaigns, and whatever future teams invent.
- **Channels:** in-app feed with unread badge, mobile push, email, SMS, and for enterprise customers, delivery into their own systems.
- **Scale:** a flash sale creates **600,000 order events in 2 minutes**; a campaign sends one message to **5 million** users; **~300,000** users are online at any moment.
- **Constraints:** the email provider allows **10,000 sends/s**; users have preferences and **quiet hours** (times not to be disturbed); regulators have rules on all of it.

Decision 1 — Getting events in: pub/sub + full-state events

The tempting design: a `POST /notifications/send` API that six product teams call inline, e.g. checkout per order.

Why not: the pub/sub incident in reverse — every producer coupled to the service, its latency inside checkout, its API changes forcing producer redeploys.

Chosen design: flip it. The platform is **another subscriber** to existing topics (`order.placed`, `shipment.updated`, `login.suspicious`) carrying **full-state events**, so each event has everything needed to render a notification without a producer callback. The platform reads additive schemas and ignores irrelevant fields, and benefits from sensitive data staying out of general events (fewer stores and queues to clean after a deletion request).

A direct API only for intentions without an event: marketing campaigns and support messages aren't domain events. The API validates and stores the request, then returns `202 Accepted` in milliseconds — request-response confirming *acceptance*, not delivery, since no caller should wait for 5 million emails.

Rule: subscribe where facts already exist; offer an API only for intentions with no event.

Decision 2 — The pipeline: message queue

The work: load preferences and quiet hours, choose channels, render templates, **deduplicate** (three events about one order mustn't become three notifications), optionally group into digests. Arrivals burst (600k in 2 min); workers sustain **~2,000/s**; nobody waits.

Chosen design: an ordinary message queue, one job per notification. The flash sale becomes a **~5-minute backlog** (600,000 / 2,000 = 300 s), not an outage. The queue's rules come along:

- **Dedupe on (event ID, user) before any side effect** (at-least-once delivery).
- **A monitored DLQ** so persistently failing messages don't block healthy ones.
- **Alert on backlog *age*, not only depth:** a notification three hours late can do harm ("your table is ready").
- **Dedupe and preference checks happen once, centrally**, not per channel; separate decisions cause inconsistencies like sending the email but not the push, or contacting a user who opted out of both.
- **Quiet hours are a scheduled state, not retries:** store the rendered notification with its **earliest allowed send time** and enqueue it on the channel queue then.
- **Recheck preferences right before the final send**, since the user may unsubscribe while work waits.

Decision 3 — Delivering out (SES, Twilio, APNs): one queue per channel

The problem: three providers (AWS SES email, Twilio SMS, Apple APNs push) with different speeds, limits and failure modes, fed by one pipeline. How does a Twilio outage not delay email?

Chosen design: each channel gets its own queue and worker pool, so an SMS backlog affects only SMS while email flows at its 10,000/s limit. That isolation is the **bulkhead** pattern, falling out of one queue per dependency. Each provider call is cross-network request-response, wrapped in a **timeout**, **retries** of transient failures carrying a **unique message key** (for provider dedupe), and a **circuit breaker** (stop calling an unavailable provider).

- **Rate limits apply per channel as a whole**, enforced across all workers rather than per process (all email workers together under 10,000/s; SMS and push follow their providers' limits).
- **Per-channel backlog-age thresholds:** a late security alert is urgent; a late marketing message isn't.

Decision 4 — Provider status back in: webhooks (receiver side)

Delivery receipts, bounces, complaints and unsubscribes return as webhooks. Apply the receiver rules: verify signature, save raw, ack in milliseconds, process behind a queue, dedupe by event ID, reconcile via the provider's API. **Hard bounces** (addresses that permanently reject mail) must **update the preference store**; continuing to send damages your **sender reputation** (how mail providers decide whether to trust your email).

Rule: protect outgoing request-response calls and apply all five receiver rules to incoming webhooks; one integration contains both directions.

Decision 5 — In-app feed and badge: SSE over a database of record

The problem: ~300,000 online users whose feed and badge should update without refresh. The server has the news; this part of the client only *reads*.

Why not polling: 300,000 users every 5 s = **60,000 requests/s**, nearly all "no change." **Why not WebSocket:** a two-way protocol for a client with nothing to send.

Chosen design: SSE. One `EventSource` per session; events update the badge within milliseconds.

- **The database is the truth.** Page load fetches the current feed via request-response; SSE carries only live updates. Short disconnects replay via `Last-Event-ID`; longer gaps trigger a full refetch.
- **Multiple devices** hold separate streams for the same user.
- **Mark-as-read** is an ordinary POST that publishes an event updating badges on the user's other devices — still no WebSocket.
- **The badge count comes from the database**, not from summing stream events, or one missed event leaves it wrong forever. Any reconnect or detected replay gap refetches the official unread count.

Decision 6 — Enterprise delivery: webhooks (sender side)

Enterprise customers want order and security events POSTed into their infrastructure, so **you're the provider** with the sender duties: sign with **rotatable secrets**, include unique event IDs for their

dedupe, retry from a **durable queue per endpoint** on a published backoff schedule (one dead customer can't block others), keep timeouts short, auto-disable persistently failing endpoints, and offer a delivery-status dashboard with redeliver. The checklists mirror each other: what you expect from providers in Decision 4, you provide here.

Decision 7 — The escalation question: decline the WebSocket

The request: product wants users to reply to support notifications from the feed and suggests replacing SSE with WebSockets.

Why decline: the client sends only *occasionally*, so each reply is an ordinary POST and SSE updates the user's other devices. Bidirectional streaming is for **frequent, latency-sensitive two-way traffic** (typing, cursors, voice); a reply box needs no sequencing, resume, backpressure (slowing a fast sender to a slow receiver) or stateful gateway tier. If it becomes **live support chat**, give *only that feature* its own gateway-and-backplane system.

The final architecture

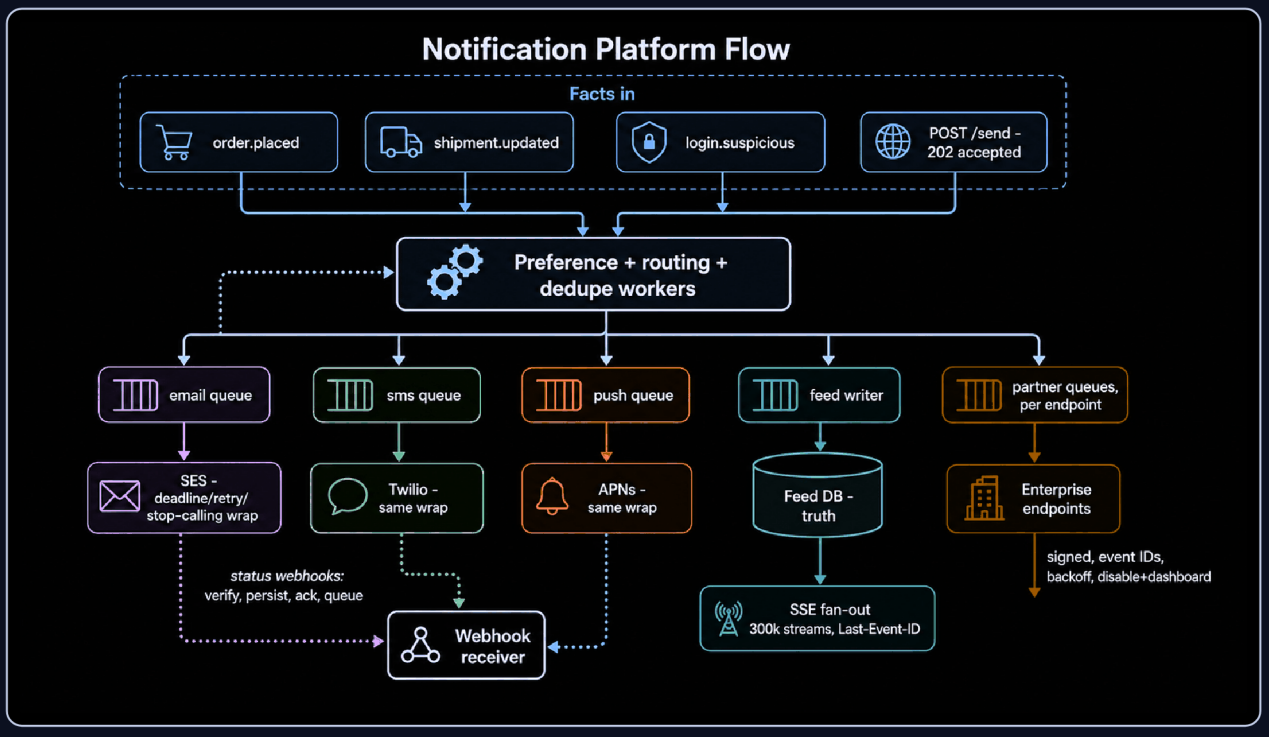


Figure — Notification System Architecture (course diagram)

Subscribed topics and `POST /send → 202` feed the central **preference + routing + dedupe workers**, which fan out to per-channel lanes (**email queue → SES**, **SMS queue → Twilio**, **push queue → APNs**, each protected), a **feed writer → Feed DB (the truth) → SSE fan-out** (300k streams), and **partner queues per endpoint → enterprise endpoints**. Provider **status webhooks** return through a **webhook receiver** into the workers (e.g. hard bounces updating preferences).

Debrief: one question per arrow

Arrow	The routing question answered	Pattern
Product events → platform	Facts that many parties care about; don't make producers call you	Pub/sub + full-state events
Ad-hoc sends → platform	The caller needs acceptance now and delivery later	Request-response (202) into a queue
Events → rendered notifications	Jobs that come in bursts, done once, with nobody waiting	Message queue + dedupe + monitored DLQ
Platform → SES / Twilio / APNs	A call into infrastructure someone else owns	Protected request-response, one queue per channel
Providers → platform (status)	Another company's events arriving at your public URL	Webhooks, receiver rules
Platform → browsers	The server has news; the client only reads	SSE over a database of record
Platform → enterprise systems	Push across the company boundary; your duties	Webhooks, sender rules
"Make it conversational"	The client sends, but rarely	Declined the socket: POST + SSE

The middle column is the module's opening question asked eight times. Result: **no WebSocket, one API, seven asynchronous relationships**, each with a stated reason, none chosen for novelty.

Two sources of truth: the **notification database** owns feed entries, unread state, preferences and delivery status; **external providers** own their delivery records, brought in by reconciliation. Queues, topics, SSE streams and webhook attempts only *carry* information and never replace these records.

Left for later (Module 5): the defensive controls around every arrow — timeouts, bounded retries, idempotent operations, monitored DLQs, per-dependency lanes (bulkheads). This capstone decides *where data moves*; those make each path survive failure.

Extend it yourself

- **Digests** ("bundle my marketing notifications into one daily email"): where does batching sit relative to dedupe? What data model holds a pending digest? What happens to it when a user changes timezone?
- **Multi-region:** users and SSE connections span two continents, but the feed database and topics are in one region. Which arrows change? Which replication patterns (primary-replica, to start) do you need?
- **For AI engineers:** add an **LLM** stage summarizing each user's unread notifications into a morning brief. Route its arrows: batch inference via your own queue, or a provider batch API reporting completion by webhook? Stream the brief token by token in-app with which transport (SSE)? And the **semantic cache** question (reusing an answer when a new request *means* nearly the same thing, not just when identical): should two users with near-identical unread lists share one generation?

Interview one-liner

"I route each arrow on its own. Producers publish facts the platform subscribes to; ad-hoc sends get a 202 into a queue. A deduping queue pipeline feeds per-channel, bulkheaded queues calling providers with timeout, retry and breaker. Provider status returns as verified, reconciled webhooks. Browsers get SSE over a database of record, and enterprises get signed, retried webhooks. There's no WebSocket anywhere, because nothing here streams both ways."

Module 3 — Storing Data (Storage and Replication)

Module Introduction

Original lesson: Module introduction on DesignGurus ↗

What this module is about. Module 2 covered the *arrows*; this one covers the most important box: the database. API servers are *stateless* (they keep no lasting **state**, i.e. data remembered between requests), so you can add ten or lose three unnoticed. The database holds the system's **facts** — payments, ownership, the record of what happened — and you can't simply run a second independent one beside it, because two separate copies of the truth drift apart. So as a product grows, the database is usually the first part to hit its limit and the most dangerous to change.

Analogy. The database is the company's official ledger. Hiring clerks (app servers) is easy; letting each clerk write in whichever of two ledgers is nearer guarantees they soon disagree.

The six patterns and when each applies

The situation	The pattern
Reads are too many for one machine, but every write still has to land in one place	Primary-Replica
There is more data, or more writes per second, than one machine can handle	Sharding
Adding a single extra machine should not make most of the data move	Consistent Hashing
A crash in the middle of a write must never lose or damage data	Write-Ahead Log
You need every change that ever happened, not just today's value	Event Sourcing
Reads and writes each need the data arranged in a different shape	CQRS

Two facts that organize the whole module

1. **Reads and writes scale differently.** Scale reads by *copying* (put the data on several machines, point each reader at one). Copying does nothing for writes, since every copy must apply every write; scale writes by *dividing* (each machine owns one piece). "Copy for reads, split for writes" tells you when the first three patterns fit and answers much of any database design question.
2. **An append-only log sits underneath everything.** A file you only add to, never edit. The Write-Ahead Log uses it for crash recovery and replication; Event Sourcing makes it the app's data model; CQRS builds separate read and write models from it. Module 2 explained *why* services announce facts; this module explains *where they live*: an ordered log from which every other view is built.

How the module is told. The same online store starts with one database doing everything, as most companies do. Reads overload it (→ Primary-Replica); then writes and data size (→ Sharding); changing the shard count hurts (→ Consistent Hashing); a power cut tests durability (→ Write-Ahead Log); finally the order system is rebuilt on the log (→ Event Sourcing, CQRS). The capstone replays the journey at 100 million users. **Short on time?** Primary-Replica, Sharding and Consistent Hashing underpin nearly every "design X at scale" question.

Four terms used throughout

- **Read path vs. write path** — queries that *get* data vs. requests that *change* it; they need different scaling tools, carry different risks, and may end up on different databases.
- **Replication lag** — a copy receives changes from the original after a delay (milliseconds, sometimes more): the database form of **eventual consistency** (copies match *in the end*). Basics:

Module 0 — Replication.

- **Shard key** — the field that decides which machine owns a row; it decides how evenly work spreads, and a poor one causes the capstone's main failure.
- **Durability** — once the database says "saved," the data survives a crash, reboot or power cut. The write-ahead log keeps that promise.

Before you start — three questions about a system you know (the capstone asks them again):

- If your database server died now, how much data would you lose, how long to recover, and has anyone practiced it?
- What is your read/write split? (Measure, don't guess; it decides which patterns fit.)
- With twice the users tomorrow, what runs out first: write capacity, read capacity, or disk?

Primary-Replica

Original lesson: Primary-Replica on DesignGurus ↗

What it is. One database machine, the **primary**, takes every write and streams its changes to one or more **replicas**, each holding a live copy and answering reads. If the primary dies, a replica can be *promoted* to replace it.

Analogy. Only the teacher (primary) writes on the main whiteboard; assistants (replicas) copy each change onto their boards; students read the nearest board; if the teacher is sick, the most up-to-date assistant takes over.

The problem

The store has run for years on one PostgreSQL (open-source relational database) machine. Traffic: **8,000 reads/s** (product pages, search, order history) and **300 writes/s** (orders, carts), on a machine that handles **5,000 queries/s** total. Product pages take **4 seconds**, and checkout slows because reads and writes compete for CPU, memory and disk. **96%** of traffic is reads; that split *is* the problem.

Behind it hides a second one: this machine is the **only copy** of the data — one disk, one power supply, a **single point of failure** (one part whose failure stops everything). Last night's backup can't restore today's orders; a 4 PM failure loses the day. One pattern fixes both.

Why the obvious fixes fail

- **"Buy a bigger machine."** Maybe a year's relief, but hardware tops out, high-end prices climb steeply, upgrades mean downtime, and there's still one copy.
- **"Put a cache in front."** Helps (Module 4), but a cache is a *performance layer*, not a trusted second copy: misses still hit the database, cached data goes stale, and cached pages can't rebuild the orders table.
- **"We copy the database nightly."** A backup isn't a replica: hours old, serves no traffic, takes hours to restore. It prevents total loss but protects neither today's data nor today's load.

You need extra copies that stay *current*, *serve traffic*, and are *ready to take over*.

How it works

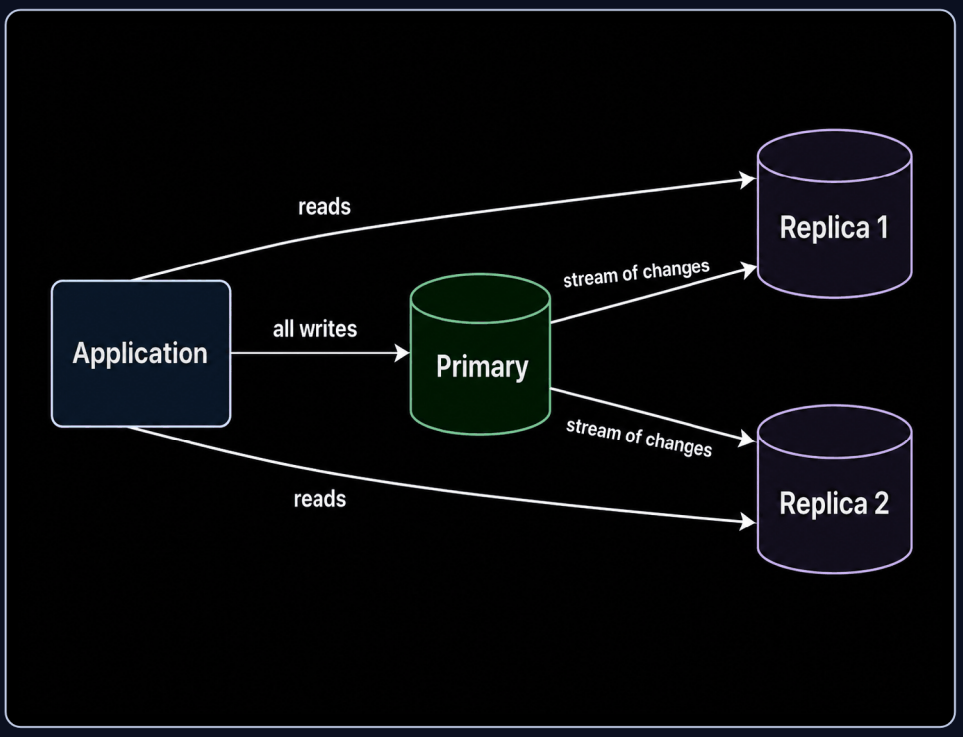


Figure — Primary-Replica (course diagram)

The application sends **all writes** to the primary and **reads** to Replicas 1 and 2; the primary pushes a **stream of changes** to each replica.

1. **Every write goes to the primary.** Two writers could change one row two ways and the copies would disagree; one writer, many readers rules that out *by design*.
2. **The primary writes each change to an ordered log and ships it to replicas** — the same log it uses to survive crashes (Write-Ahead Log). *Replication means shipping the log.*
3. **Each replica replays changes in exactly the same order**, so its copy matches.
4. **Reads are spread across replicas** — the source of the extra capacity.
5. **Replication lag is expected and handled.** Usually milliseconds, seconds under heavy load. Its visible symptom is the **read-your-writes problem**: a user saves (to the primary), reloads (from a lagging replica), and their edit seems to vanish.
6. **Failover when the primary dies.** *Failover* is switching to a backup. Missing **heartbeats** (small regular "I'm alive" messages) reveal the failure; the most up-to-date replica is **promoted** (set to accept writes, with application traffic redirected to it).
7. **Fencing prevents split brain. Split brain** is when the old primary returns, unaware it was replaced, and keeps accepting writes, so two primaries diverge. Safe failover first **fences** the old primary (forcibly blocks its writes), then promotes.

Walkthrough with numbers

- **Two replicas splitting reads:** **~4,000 reads/s** each, within capacity. The primary handles 300 writes/s plus the few reads that must be fresh, dropping from **8,300 queries/s to a few hundred**. Product pages go **4 s → 80 ms**.
- **Measure the lag:** usually **10–50 ms**, unnoticed; during a bulk import it hits **8 seconds** and read-your-writes complaints start. Lag is *a number you monitor*, not a constant.
- **2 AM failover drill:** detection takes **~15 s** of missed heartbeats, promotion **~30 s**; for those **45 seconds** writes (checkout) fail while reads keep working.
- **Hidden cost of asynchronous replication:** the primary says "saved" *before* the change reaches a replica, so failover can lose the last **~200 ms** of confirmed writes (see the comparison).
- **Bonus:** backups and heavy analytics run at full speed on a replica, away from production.

Async vs. sync replication (the comparison)

Every replicated setup must decide: *when does the primary tell the client "saved"?*

	Asynchronous	Synchronous
Primary says "saved"...	Right away, and sends the change afterward	Only once a replica confirms it has the change
Write latency (time until "saved")	Fast (only local work)	Slower (one network round trip at minimum)
If the primary dies	The most recent confirmed writes might be lost	No confirmed write is lost
If a replica dies or slows	No one notices	Writes stall or fail, because the replica now sits on the critical path (the steps a write must wait for)
Fits	Most workloads: feeds, catalogs, profiles	Moving money, committing inventory

- **Async is the usual default:** losing ~200 ms of writes in a rare failover usually costs less than sync's price (a round trip on every write, and stalls whenever a replica struggles).
- **Semi-synchronous** waits for exactly *one* replica: zero-loss failover for modest latency — what serious money-handling systems typically run.
- **Strong answers pick a mode per table or operation**, not one for everything.

Trade-offs

You gain:

- Read capacity that grows with machines — the standard answer to a 96%-read workload.
- Survival: a promoted replica restores service in under a minute instead of a half-day restore.
- A safe place for backups, analytics and experiments.

You pay:

- **Lag and its oddities.** Replica reads are slightly stale, so the app must route reads *needing fresh data* to the primary and the rest to replicas.
- **Zero write scaling.** Every replica applies every write, so replicas add no write capacity (the primary even gains a follower to feed). Writes beyond one machine need *dividing* (Sharding).
- **Untested failover is dangerous.** Split brain, async data loss and never-run promotion scripts each break recovery; an unpracticed failover plan doesn't exist.
- **More to run:** machines, lag dashboards, heartbeats, **runbooks** (step-by-step incident guides).

When not to use it

- **Write-heavy workloads** (logging, **telemetry** — constant measurement streams from devices or servers — frequent updates): replicas multiply work, add no capacity. That's sharding's problem.
- **Small scale with good backups:** one well-sized machine, tested backups and an hour's acceptable recovery is a professional setup. Don't adopt distributed-systems problems early.
- **Every read must be fully fresh** (some trading and inventory cases): routing all reads to the primary cancels the benefit. Pay for sync replication or challenge the requirement.
- **As a promise writes never pause:** failover takes seconds to minutes.
- **Classic misuse:** adding read replicas for a *write* overload, which only adds primary work. Always ask "reads or writes?" first.

Real-world examples

- **PostgreSQL streaming replication** and **MySQL replication** — the textbook form; most companies' first scaling step looks like the walkthrough.
- **MongoDB replica sets** — MongoDB is a document database (JSON-like records instead of rows); replicas hold an *election* and promote a new primary themselves, usually within seconds.
- **AWS RDS and Aurora** — the pattern as checkboxes: "add read replica" and "enable Multi-AZ" (AZ = availability zone, a separate datacenter in a region; keeps a synchronously replicated standby for failover). **Aurora** allows up to **15 replicas** sharing one storage layer.
- **For AI engineers: vector databases** (storing **embeddings** — number lists representing meaning — and finding the closest) use one writer and replicas for **similarity search**. A **feature store**'s online reads (input values for ML models) are exactly replica-friendly traffic. Run training and evaluation jobs on a replica so a 2-hour scan never slows production. Read-your-writes bites ML too: a prediction served from a lagging replica may use yesterday's feature value.

In the interview

Prompts: "Your database is overloaded — what do you do?" (ask first: *"reads or writes?"*), "What happens when the database dies?", "A user can't see their new comment right away."

30-second answer: One primary takes every write and streams its change log to replicas that serve reads, scaling reads horizontally for read-heavy workloads and giving failover: the most caught-up replica is promoted, with fencing so the old primary can't cause split brain. Two costs: replication lag, so reads that must be fresh (like a user's own write) go to the primary; and zero added write capacity, since every replica applies every write. Write bottlenecks need sharding, not more replicas.

Likely follow-ups (with answers):

- *"A user edits their profile and the edit isn't there after reload. Fix it."* — Read-your-writes. Cheapest first: (1) route that user's reads to the primary for ~10 s after a write; (2) record the log position of their latest write and read only from replicas past it; (3) show the change optimistically in the UI while storage catches up.
- *"How automatic should failover be?"* — Automatic detection and promotion (seconds count at 2 AM), but only with fencing in place; two primaries are a far worse day than one dead one. Whatever the mechanism, practice it deliberately.

Mistake that fails candidates: read replicas for a write bottleneck, or describing failover as instant and free with no mention of lag, the data-loss window or split brain.

Self-check

- **Q1. What scales when you add replicas, what doesn't, and why?** → Read capacity scales (each replica serves reads). Write capacity doesn't: a write costs work once *per copy*, and the primary gains another follower. You also gain failover and a home for backups/analytics.
- **Q2. A platform stores social-feed posts and account balance transfers. Which replication mode for each?** → Feed posts: async (users feel latency on every post; loss only in rare failovers). Transfers: synchronous or at least semi-sync — a bank can't lose ~200 ms of confirmed transfers. Choose per table/operation.
- **Q3. After adding two replicas: "I updated my shipping address, refreshed, the old one was back; a minute later it was fine." Diagnose and give two fixes with costs.** → Read-your-writes from replication lag; it "fixed itself" when the replica caught up. Fix 1: route the user's reads to the primary for ~10 s after a write — costs **session state** (remembering per-user facts like "wrote 5 s ago" between requests) and loses the replica benefit for those reads. Fix 2: read only from

replicas past the write's log position — extra data-layer plumbing, but reads stay on replicas. (Cosmetic cases: optimistic UI.) The *wrong* fix is sending every read to the primary.

Related patterns

Sharding (when *writes* outgrow one machine) · Write-Ahead Log (the replicated stream *is* this log) · Load Balancing (spreads reads over replicas) · Cache-Aside (catches the hottest reads before any replica).

TL;DR

- **Problem:** one database machine is a read bottleneck (reads usually ~95% of traffic) and the only copy — a single point of failure.
- **Mechanism:** one primary takes all writes and streams its change log to replicas, which serve reads and stand ready for promotion; fencing prevents split brain.
- **Costs:** lag and read-your-writes surprises, zero extra write capacity, failover that only counts if practiced.
- **Skip it when:** writes are the bottleneck (shard), one machine plus tested backups fits, or every read must be fully fresh.

Sharding

Original lesson: Sharding on DesignGurus ↗

What it is. Sharding divides *the rows of one table* across many machines (**shards**). A rule on one field, the **shard key**, decides where each row goes. Each shard fully owns its piece — writes, reads and **indexes** (sorted lookup structures that make finding rows fast). No machine holds everything.

Analogy. A phone book too big for one volume splits into A–F, G–M, N–S, T–Z: everyone knows the rule, any name is one step away, and four printers update four volumes at once.

The problem

Replicas fixed reads, but eighteen months later the *primary* struggles in a way replicas can't fix. Writes rose from **300 to 2,500/s** (orders, carts, inventory, reviews), and since every replica applies every write, the primary *and every replica* are overloaded; lag grows to **minutes**. A fourth replica would make it worse.

The data is also too big: the orders table is **4 TB**, its indexes no longer fit in memory (so lookups hit disk), backups run all night, a **schema migration** (table-structure change) is a weekend project, and you already rent the cloud's largest machine. Write volume and data volume are the same problem: *too much for one machine, however many copies.*

Why the obvious fixes fail

- **"More replicas."** Copies help reads; this is a write problem, which copies worsen.
- **"An even bigger machine."** You have the biggest. Even if one existed: steep prices, downtime, and the same wall within a year.
- **"Archive old data."** Moving orders older than two years to **cold storage** (cheap, slow storage for rarely read data) is worth doing — maybe 4 TB → 1.5 TB — but buys quarters, not years; the *write rate* doesn't fall.
- **"Split by table"** (a **functional or vertical split**: orders, users, inventory on separate machines). Often a sensible first move, but orders alone holds most writes and bytes; eventually one table outgrows one machine.

These *move* load; you need to *divide* it.

How it works

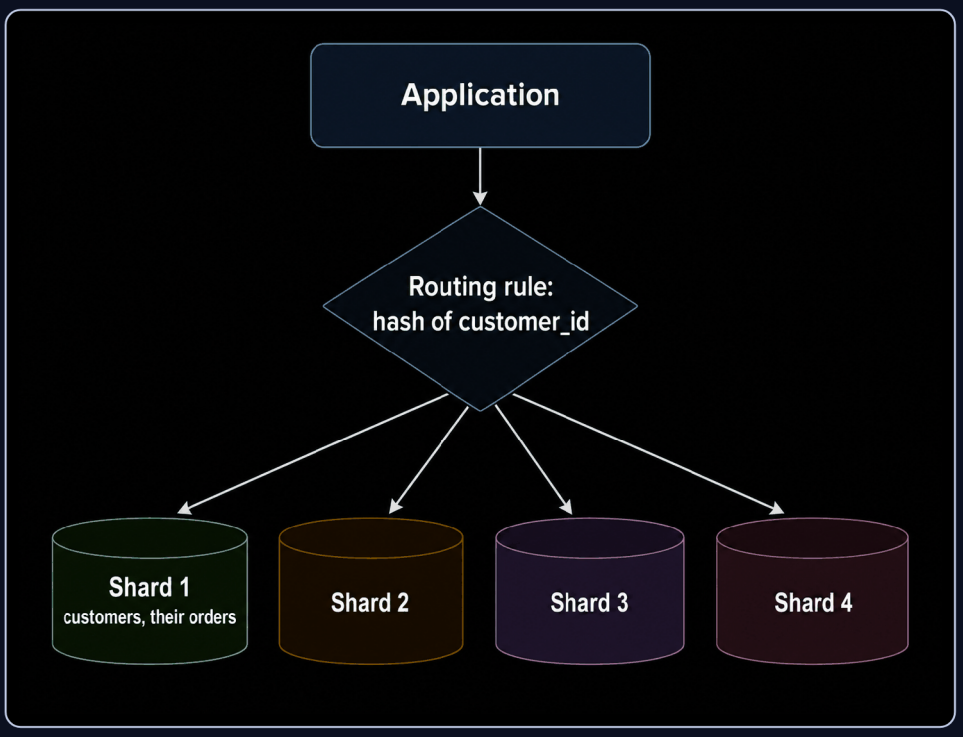


Figure — Sharding (course diagram)

Each request passes a **routing rule** (a hash of `customer_id`) that sends it to exactly one of four shards; Shard 1 holds a group of customers *together with their orders*.

Three decisions determine success:

1. **Pick the shard key — the most important decision in the module.** A good key:
   - **Appears in almost every query**, so each query hits one shard (shard by product and every "my orders" query searches all shards).
   - **Spreads load evenly**, so no shard is the busy one.
   - **Keeps related rows together:** with `customer_id`, a customer's orders, addresses and cart share a shard, so checkout never crosses machines.
2. **Pick the routing rule** (key → shard number).
   - **Hash sharding:** run the key through a **hash function** (turns any value into an evenly spread number) and take the remainder: `hash(customer_id) % 4`. Even spread; the default.
   - **Range sharding:** each shard owns a range (A–F, or a date range). Neighbors stay together, great for range queries, but sharding by date sends 100% of today's writes to one shard.
3. **Decide who routes:** the application, a **proxy** (a middleman forwarding each query to the right shard), or the database itself (MongoDB, Vitess, Citus). All work at scale if every caller applies the identical rule.

The price. Cross-shard questions get harder. "This customer's orders" hits one shard; "top 10 products today" must ask every shard and merge: **scatter-gather**. Cross-shard **joins** (combining rows from two tables on a shared field like `customer_id`) become application work, and a **transaction** spanning two shards (two customers on different shards exchanging money) can no longer be one **atomic** (all-or-nothing) database transaction; the **saga** pattern (later) exists for this. Sharding buys write capacity with query flexibility.

Walkthrough with numbers

Four shards, routed by `hash(customer_id)`:

- **Writes:** 2,500/s ÷ 4 ≈ **625/s per shard**, with headroom. Each shard is its own primary-replica cluster: sharding doesn't replace replication; it multiplies it.
- **Data:** 4 TB ÷ 4 = **1 TB per shard**: indexes fit in memory, backups take hours, migrations become manageable.

- **Queries, easy to hard:** "customer 8412's orders" → hash, one shard, as fast as before. "Top sellers today" → scatter-gather over four shards and merge in the app: slower and clumsy, which is why heavy analytics usually moves to a separate event-fed system (see change data capture).
- **The problem waiting:** in two years you need 8 shards; `hash(id) % 4` → `% 8` changes the remainder (what `%`, **modulo**, keeps) for nearly every customer, so almost all data must move. Next lesson.

Hash vs. range sharding (the comparison)

	Hash sharding	Range sharding
How it routes	`hash(key) % N`	By range: A–F on one shard, G–M on another; Jan on one, Feb on another
Load spread	Excellent, and happens automatically	Only as good as the ranges you pick; time ranges send every new write to one shard
Range queries (dates, prefixes)	Bad: one range ends up spread over every shard	Excellent: neighboring values sit together
Hot key risk	A single huge key still overloads its shard	The same, and hot ranges too
Resharding	Painful with plain modulo (the next lesson fixes this)	You can split a busy range where it sits, which is gentler

Hash by default: for transactional work, even spread beats range queries. **Range** when access really is by range (**time series** — timestamped measurements — leaderboards, alphabetical browsing), managing ranges so the current month doesn't become one overloaded shard.

Trade-offs

You gain:

- Write capacity that grows with machines, which replication never gives.
- Data size that grows the same way; no table must fit one machine.
- A smaller **blast radius** (share of users one failure affects): shard 3 dying hits a quarter of customers, and its own replica failover fixes even that.

You pay:

- **Query flexibility:** single-shard queries stay fast; cross-shard ones become scatter-gather; cross-shard joins and transactions are yours.
- **Hot shard risk:** a **hot shard** gets far more load than the rest. Hashing spreads *keys*, not *load*: one huge key — a celebrity, or a giant **tenant** (one customer company on a shared product) — lands entirely on one shard, and more shards don't help.
- **Resharding pain:** changing shard count moves lots of live data; plain modulo makes it a disaster.
- **More to operate:** 4 shards × (primary + 2 replicas) = **12 databases**: twelve failovers, twelve backup schedules, and a routing layer everyone must understand.

When not to use it

Sharding is a **last resort**, after cheaper options:

- **Vertical scaling still has room** (a bigger machine adds no new failure modes).
- **The real problem is reads** (replication, at ~a tenth of the complexity).
- **The data can shrink** (archiving or expiry can delay sharding by years).
- **A functional split suffices** (no single table outgrows a machine).
- **Queries don't match any split** (heavy cross-entity joins and transactions); rethink the data model first.

Misuses: *sharding for prestige* — four shards at a scale one Postgres handles, paying complexity for years before scale arrives, if ever. And the classic key mistake: **auto-incrementing ID with range routing** (IDs assigned 1, 2, 3…, so new rows always get the highest), sending every new row to the newest shard.

Real-world examples

- **Instagram** sharded Postgres by user ID and embedded shard info in the IDs; a well-chosen key ran billions of photos on ordinary databases.
- **Slack** shards by workspace, a natural tenant key since nearly every query is "inside this workspace"; giant workspaces creating hot shards is the self-check scenario at huge scale.
- **Vitess** (from YouTube, now **open source** — public, free code) and **Citus** add sharding to MySQL and Postgres; **MongoDB** has it built in (choose a key; routing and rebalancing are handled).
- **DynamoDB and Cassandra** (Amazon's managed key-value database; an open-source wide-column database) are built around sharding: your table's *partition key is* the shard key, which is why their docs teach choosing it first.
- **For AI engineers:** feature stores shard by entity ID (user, item); training datasets shard by example ranges for parallel loading; a vector index beyond one machine is sharded with scatter-gather similarity queries (later vector-database sharding lesson).

In the interview

Any "design Twitter / Instagram / Uber at scale" reaches here within minutes. The interviewer wants your **shard key, with reasons**, then the predictable follow-ups: the celebrity, resharding, the cross-shard transaction.

30-second answer: When writes or data size outgrow one machine, I shard rows by a key that appears in almost every query, spreads load evenly, and keeps related rows together — customer ID for commerce, so checkout never crosses machines. Hash routing by default; each shard is its own primary-replica cluster. Accepted prices: cross-shard queries become scatter-gather, cross-shard transactions need application-level patterns, one hot key can still overload a shard, and resharding needs consistent hashing rather than modulo.

Likely follow-ups (with answers):

- *"You sharded by user ID. A celebrity with 40 million followers goes live."* — Their shard overloads while others idle (hashing spreads keys, not load). Escalating fixes: (1) cache the hot entity's reads; (2) give the few extreme keys dedicated shards; (3) split that entity's data further (e.g. reshard their follower list by follower ID). Naming the problem before the interviewer does counts most.
- *"Two users on different shards send each other money."* — First avoid it: a good key keeps most transactions on one shard. Otherwise it can't be one database transaction; it becomes a coordinated series of steps with failure handling — a saga. Seeing the key choice as the first line of defense marks a senior candidate.

Mistake that fails candidates: "we'll shard from day one" with no mention of cost, a key chosen without reasons, or no answer for changing the shard count.

Self-check

- **Q1. What does sharding scale that replication can't, and what are the three properties of a good shard key?** → Write capacity and total data size: replication copies *everything* everywhere and every copy applies every write; sharding gives each machine its own slice. A good key appears in nearly every query, spreads load evenly, and keeps related rows together.

- **Q2. Multi-tenant SaaS (software-as-a-service sold by subscription; multi-tenant means many customer companies share one system; companies sign up, employees use it): shard by `company_id`, `user_id`, or the row's random ID?** → `company_id`: nearly every B2B (business-to-business) query is "inside this company," so it stays on one shard and a company's data stays together for export and deletion. Known risk: very unequal tenant sizes, so plan dedicated shards for giants. `user_id` turns company-wide queries ("all documents in this workspace") into scatter-gather. A random row ID spreads perfectly but no lookup knows where anything lives.
- **Q3. Hashed `user_id` across 8 shards; an influencer's live shopping event (40M followers) overloads shard 5 while the others sit at 15%. Diagnose and give three fixes.** → A hot key: keys are spread evenly, but one carries 40M users' load. (1) Cache storefront reads (nearly identical for all viewers; may suffice alone). (2) Dedicated shards for celebrity accounts. (3) Split the influencer's data by a second key such as follower ID. Bigger lesson: monitor per-shard load, because the business keeps creating unequal keys.

Related patterns

Primary-Replica (each shard is a replicated cluster) · Consistent Hashing (changing shard count without moving almost everything) · Saga (what a cross-shard transaction becomes) · Cache-Aside (first defense against hot keys).

TL;DR

- **Problem:** writes and data size outgrow one machine, and replication can't help (every copy applies every write; no copy is smaller).
- **Mechanism:** split rows across shards by a well-chosen key (in nearly every query, evenly spread, keeps related rows together); hash routing by default; each shard a primary-replica cluster.
- **Costs:** scatter-gather queries, app-handled cross-shard transactions, hot keys, resharding data movement, operations multiplied by shard count.
- **Skip it when:** a bigger server, more replicas, archiving or a functional split still works, or the workload is cross-everything joins and transactions. Last resort.

Consistent Hashing

Original lesson: Consistent Hashing on DesignGurus ↗

What it is. A routing rule for spreading keys over servers so that adding or removing one server moves only a small slice of data (~1/N) instead of nearly all. Servers and keys are hashed onto a circle; each key belongs to the first server clockwise.

Analogy. Houses (keys) on a circular road are covered by the next fire station (server) clockwise. A new station takes over only the houses between it and the previous station; all others keep theirs.

The problem

Four shards routed by `hash(customer_id) % 4`, 625 writes/s each. Growth calls for a **fifth shard**: `% 4` → `% 5`. A key stays put only if `hash % 4 == hash % 5`, true for just **20%** of hashes. One extra shard leaves **80% of the data on the wrong machine**: **~3.2 of 4 TB** must move.

That means copying terabytes over a weekend while orders keep arriving, writing to old and new locations, a tested rollback plan, a slow week — and again at shard six and seven. The `% N` rule is the problem: N is built into every key's address, so changing N breaks nearly every address. A better rule moves only what the new machine should own.

Why the obvious fixes fail

- **"Migrate over a weekend."** 3.2 TB moving while 2,500 writes/s change the source needs double-writes and consistency checks, repeated at every resize; teams put it off and run too few overloaded shards for years.
- **"Only ever double: 4 → 8."** Each key stays or moves to one specific new shard, but **50%** still moves, and needing 25% more capacity means buying 100% more plus a **2 TB** migration.
- **"Keep a lookup table."** A **directory** ("customer 8412 lives on shard 3") is valid (compared below), but one row per key means a hundred million rows answering every query — the most availability-critical part of the system. Grouping keys into ranges lightens it.

How it works

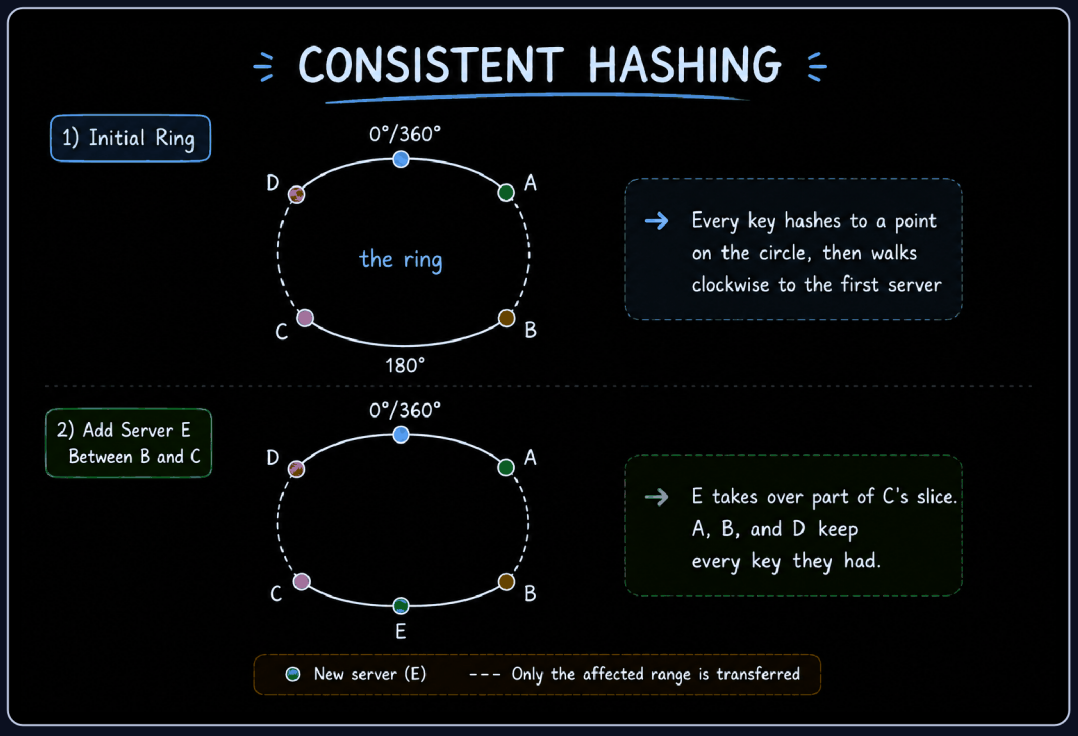


Figure — Consistent Hashing (course diagram)

Panel 1: servers A–D on the ring; each key hashes to a point and walks clockwise to a server. Panel 2: adding E between B and C takes part of C's slice; A, B and D keep every key, and only the dashed range transfers.

1. **The hash space is a circle:** 0 up to a large maximum, wrapping to 0.
2. **Hash every server name and every key onto it.**
3. **Each key belongs to the first server clockwise.** A sorted list of server positions finds it in microseconds.
4. **Adding a server moves one slice.** When E lands between B and C, only keys in E's new slice (all formerly C's) change owner; E copies them from C. With N servers, adding one moves **~1/(N+1)**: **20%** for 4→5, **~2%** for 50→51.
5. **Removing a server is the reverse:** the next server clockwise takes its slice and all its load. Two weaknesses: few servers with random placement give very uneven slices, and a failure dumps everything on one neighbor.
6. **Virtual nodes fix both.** Place each server at **100–200 points** (**virtual nodes, vnodes**). Each server owns many small scattered slices, so randomness evens out; a dead server's ~150 slices go to ~150 different neighbors. Stronger servers can get more vnodes, weighting by capacity.

What it does *not* fix. It handles **membership churn** (servers joining and leaving), not **hot keys** (one key with unusually heavy load). The celebrity still hashes to one point and one server, and still needs caching and special placement.

Walkthrough with numbers

- **The resize again:** adding a fifth shard to 4 TB — modulo moves 3.2 TB from every shard; the ring moves **~0.8 TB (20%)**, only from the new shard's neighbors, copied in the background over days while old owners serve until handoff.
- **The failure again:** without vnodes, shard 3 dying dumps all its load on shard 4, which may also fall — a **cascade**. With **150 vnodes** per server, every survivor takes a manageable ~25% increase.
- **Where it started — caches:** in a 50-node cache cluster (Cache-Aside), modulo routing remaps **~98%** of keys whenever a node is added or removed, so ~98% of lookups miss and hit the database: a **miss storm**. A ring remaps **~2%**. Cache clients have used consistent hashing since the late 1990s because cache clusters resize constantly.

The ring vs. the directory (the comparison)

Built properly, the lookup table becomes a **directory service** mapping *ranges* (not single keys) to servers, with clients caching answers — the ring's real rival.

	Consistent hashing (ring)	Directory (range-to-server map)
Data moved on resize	About 1/N, with no manual work	Whatever you decide; completely flexible
Placement control	None: the hash decides	Complete: you pin, split and move ranges yourself
Hot range/tenant	Can't be treated as a special case	Split the hot range, move it, finished
Extra infrastructure	None: it's just math inside the client	A metadata service (a small service holding the "what lives where" map) that has to stay highly available
Range queries	Spread around	Natural: ranges stay together
Typical home	Caches, clusters that change size, peer-to-peer (machines that talk directly with no central server)	Databases that want operational control

The modern hybrid: hash keys into many small fixed **buckets** and keep a small directory mapping buckets to servers. **Redis Cluster** does this with **16,384 "hash slots"**: the hash gives even spread (the ring's strength), and moving a slot is a one-row directory change (the directory's strength). It's the answer to "what comes after the ring?"

Trade-offs

You gain:

- Minimal data movement on membership change (~1/N per server).
- Growth by exactly one machine instead of doubling.
- Failures spread thinly (with vnodes) instead of cascading.
- No central directory: every client with the server list computes the same answer.

You pay:

- **Virtual nodes are mandatory;** home-made versions often omit them (mature libraries enable them by default).
- **Slightly more complex routing:** a sorted lookup instead of one modulo — microseconds, but every client must implement it identically.
- **Still hashing:** range queries ("all orders from March") stay scattered.
- **Less movement is still movement:** caches shrug off misses, but storage handoffs need ordering: stream the slice → keep serving from the old owner → switch ownership → delete the old copy.

When not to use it

- **Cluster size truly never changes** (a fixed 4-shard setup for five years can keep modulo; be honest about "never").
- **Range queries dominate:** range sharding with a directory; no hashing suits it.
- **You want manual placement:** "pin this giant tenant to its own shard" is clumsy on a ring, easy in a directory.
- **Tiny fixed systems:** two cache nodes don't need ring math.
- **Common misuse:** the ring for database shards but modulo for the cache layer — caches change membership *more* often, each change a potential miss storm, so they may need it first.

Real-world examples

- **Akamai** — consistent hashing was invented there (Karger et al., **1997**) to route web requests over constantly changing cache fleets.
- **Amazon's Dynamo paper (2007)** made "ring plus virtual nodes" the signature technique of distributed databases; **Cassandra** and **Riak** adopted it directly and **DynamoDB** descends from it.
- **Memcached** (a simple in-memory cache server) **clients** have included it for two decades, because of the miss-storm math.
- **Redis Cluster** uses the hybrid: 16,384 hash slots mapped to nodes via a directory.
- **For AI engineers:** in LLM **inference** (running a trained model to produce answers), a conversation's **KV cache** (the model's saved intermediate results for earlier tokens) stays on the GPU that last served it; routing by conversation ID with consistent hashing preserves that as autoscaling changes worker count, moving only ~1/N of conversations per event. The same approach assigns embedding-space partitions to workers in a sharded vector index.

In the interview

Entry points: "design a distributed cache," then "now add a node"; or you shard with modulo and are asked about 4→5. Be ready to derive the 80%.

30-second answer: Modulo routing builds the server count into every key's address, so going 4→5 shards moves 80% of the data. Consistent hashing puts servers and keys on a ring; each key belongs to the next server clockwise, so a new server takes one slice (~1/N) and nothing else moves. Each server appears as 100+ virtual nodes, keeping slices even and spreading a dead server's load. It solves churn, not hot keys. For manual placement or range queries I'd use the hybrid — hash slots plus a small directory, as Redis Cluster does.

Likely follow-ups (with answers):

- *"Why virtual nodes?"* — Both reasons: one point per server gives very uneven slices, and a failure dumps its whole load on one neighbor. 100+ points even out slices and spread failure load.
- *"Does the ring fix your celebrity problem?"* — No, and saying so plainly earns points. One key still hashes to one server. The ring fixes *churn*; *skew* (unequal load per key) needs caching and special handling.

Mistake that fails candidates: modulo routing in any autoscaling design, or saying "consistent hashing" without explaining what moves when a node joins — the whole pattern answers that one question.

Self-check

- **Q1. Going from N to N+1 servers, what fraction of keys move under modulo vs. consistent hashing, and why?** → Modulo: ~N/(N+1) (80% for 4→5), because the divisor is in every address. Ring: ~1/(N+1)

(20% for 4→5), because only the new server's slice changes hands; every other key's "next server clockwise" is unchanged.

- **Q2. A teammate places each server at exactly one point "to keep it simple". What two problems arrive, and when?** → Uneven load (some slices several times larger), immediately; and at the first failure, a whole slice lands on one neighbor, which may fail too. Virtual nodes fix both.
- **Q3. The cache cluster autoscales up every evening; ten minutes later the database spikes and alerts, though nothing is failing. Connect them, fix it, and quantify for 20 nodes.** → The cache client routes by modulo, so each scale-up changes N and remaps most keys; the cluster misses on nearly everything and the misses hit the database together — a self-inflicted stampede visible only as database load. Fix: consistent hashing with vnodes in the cache client. At 20 nodes, adding one remaps ~1/21 ≈ **5%** instead of **~95%**, a **~19×** smaller spike. For caches, resizing is the normal case.

Related patterns

Sharding (the ring is its routing layer done properly) · Cache-Aside (where the pattern was born) · Load Balancing (sticky routing that survives autoscaling is this pattern in another job).

TL;DR

- **Problem:** modulo builds the server count into every address, so one more shard moves ~80% of data and every resize is a painful migration.
- **Mechanism:** hash servers and keys onto a ring; each key goes to the next server clockwise; 100+ vnodes per server keep slices even and spread failures; a resize moves ~1/N.
- **Costs:** vnodes mandatory, identical routing code everywhere, scattered range queries, storage handoffs still need orchestration.
- **Skip it when:** cluster size truly never changes, range queries dominate (directory), or you need manual placement (slot-directory hybrid).

Write-Ahead Log

Original lesson: Write-Ahead Log on DesignGurus ↗

What it is. Before changing any real data file, the database appends one record describing the entire change to an append-only log, forces it onto the physical disk, and only then tells the client "saved." Tables and indexes are updated later in the background; after a crash, the database replays the log to rebuild them.

Analogy. A careful cook pins up every order ticket *before* cooking; after a mid-dish power cut, the tickets show exactly what was promised and what remains.

The problem

Tuesday, 3:07 AM: a power failure takes out the **rack** (cabinet of stacked servers) holding shard 2. It reboots 90 seconds later into one of two bad mornings:

- **Bad morning A — corruption.** The database won't start: "data files corrupted." One `UPDATE` isn't one disk operation; changing an order's status touches the row *and* two indexes — three places on disk. Power died after the first write, so an index points at a row that says something else, and the database rightly trusts none of it.
- **Bad morning B — lost writes.** It starts, but the last two seconds are gone: **40 orders** customers were told were "saved" never happened, confirmed from memory before reaching disk.

The physics look unbeatable: a disk can lose power mid-write, even mid-**4 KB page** (a **torn write**), and data spans thousands of pages that can't all change in one atomic instant. Every serious database's answer: *first write down what you're about to do, in one safe place; then do it.*

Why the obvious fixes fail

- **"Write everything to disk immediately, no memory buffering."** Very slow: one `UPDATE` hits three scattered places, and **random writes** (jumping around the disk) cost far more than one continuous stream — **~100×** on spinning disks. It still doesn't prevent corruption, since power can fail between locations.
- **"Keep everything in memory, snapshot every few minutes."** Fast, but a crash loses *minutes*, and copying a changing dataset is its own hard problem. (Snapshots are half the real answer; see the comparison.)
- **"Buy batteries."** A **UPS** (battery backup) narrows the window but doesn't close it: kernel panics (OS core crashes), hardware faults, the wrong cable unplugged. A durability plan that needs nothing to go wrong isn't one.

The need: record one *logical* change (the whole `UPDATE`) as a single cheap, atomic disk operation. Scattered pages can't; a log can.

How it works

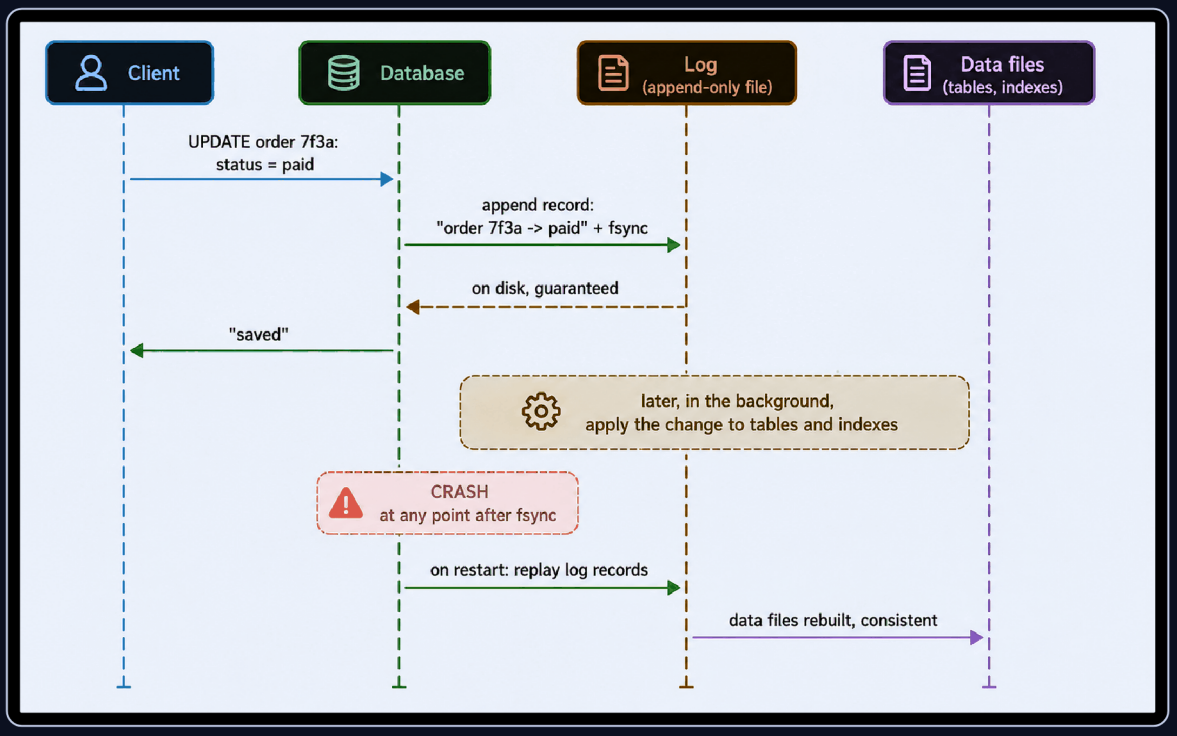


Figure — Write-Ahead Log (course diagram)

The client sends "UPDATE order 7f3a: status = paid"; the database appends a log record, **fsync**s it, gets "on disk, guaranteed," and only then answers "saved." Tables and indexes are updated later; after any crash post-fsync, replay restores consistent data files.

1. **Append one record describing the whole change** (row plus both indexes) to the log's end. One continuous stream to one file is the *easiest* job a disk does. The record carries a **checksum** (a small number computed from its bytes that exposes damage).
2. `fsync` **the log.** Operating systems normally buffer writes and flush when convenient; `fsync` is the system call meaning "don't return until this is physically on disk." This makes the promise real.
3. **Acknowledge the client. The one rule everything depends on: never acknowledge before the log record is fsynced.** Everything confirmed is written down; anything not written down was never confirmed.
4. **Apply to tables and indexes later**, in background batches, off the **critical path** (work the client waits for).
5. **After a crash, replay the log on restart**, re-applying each change so the data files match every promise. Both bad mornings disappear. A torn record at the end (the **tail**) fails its checksum and is discarded — safe, since an unfinished fsync was never confirmed.
6. **Checkpoints keep replay short.** Periodically the database flushes pending changes to the data files and writes a **checkpoint** marker ("everything before this is applied"); recovery replays only what follows, and older log can be archived or deleted.

The connection to replication. The "stream of changes replayed in order" the primary ships to replicas *is* the write-ahead log: a replica is a machine doing continuous crash recovery from another machine's log. One mechanism, two jobs — durability and replication — and the next lesson pushes the log further still.

Walkthrough with numbers

- **Write path timing:** appending an order's ~**100-byte** record and fsyncing takes ~**1 ms**; updating the row and two indexes in place while the client waits would take several times longer and still not be atomic. With a log, the fast path is the safe path.
- **Group commit multiplies throughput** (*throughput* = writes completed per second): **50 writes** arriving in the same millisecond are appended and fsynced *once* — 50 durable commits for one flush. That's how ordinary disks sustain thousands of durable writes per second.
- **The crash drill again:** since the last checkpoint (30 s ago) the log holds ~**75,000 records**, replayed in seconds. All 40 orders survive. The torn tail record fails its checksum and is dropped; its client never heard "saved," retried with an **idempotency key** (unique request ID), and was applied once.
- **The knob:** a 30 s checkpoint means fast recovery but frequent flushing; hourly means less overhead but minutes of replay (see the self-check).

The log vs. the snapshot (the comparison)

Redis makes it visible: **RDB** (periodic snapshots) or **AOF** (an append-only log), your choice.

	Snapshot (RDB-style)	Write-ahead log (AOF-style)
Cost per write	Nothing	One append to the log (plus whatever fsync policy you set)
Crash loses	Everything since the most recent snapshot (minutes)	At most the final moment not yet fsynced (~zero to seconds)
Recovery speed	Fast: load a single compact file	Slower: every change must be replayed, unless a recent snapshot gives you a starting point
Disk footprint	Small and compact	Keeps growing, unless history that's already absorbed can be thrown away

Each is strong where the other is weak, so real systems **use both** via checkpointing: a snapshot plus the log since. Production Redis enables both; PostgreSQL does it with a **base backup + WAL archive**. "Snapshot or log?" → both.

Trade-offs

You gain:

- A durability promise that holds: "acknowledged" always means "on disk."
- Fast writes anyway: one sequential append replaces many scattered updates.
- Replication nearly free: ship the log you already write.
- **Point-in-time recovery (PITR):** old logs plus regular snapshots restore the database to *any chosen moment* — standard for every serious operation.

You pay:

- **Everything is written twice** (log, then data files): **write amplification**, disk bandwidth spent on safety.
- **A latency floor per commit:** no faster than the disk confirms an fsync; group commit trades a little latency for a lot of throughput.
- **Log housekeeping:** checkpoints and retention need tuning, and stale dependencies — a dead replica still "subscribed," a stuck archive job — can block log deletion and silently fill the disk.

- **Recovery time is a deliberate setting:** the checkpoint interval caps restart time.

When not to use it (really: when to turn it down)

Your database chose a WAL long ago; you choose when to *weaken* it:

- **Truly disposable data.** A pure cache refills from the source, so Memcached has no persistence and Redis lets you disable it. Correct, not careless.
- **Re-runnable bulk loads.** Reloading **500 million rows** from a file you still have? Relax durability during the load, then restore it. A normal, honest technique.
- **Know the polite phrasings.** `fsync = off` and "asynchronous commit" both mean "saved" may precede reality — fine for, say, a metrics store that can lose a second, but decide deliberately and document it rather than copying a blog's config.
- **Don't build your own.** Need a durable append-only log? Use a database or a message queue; **Kafka** is this pattern as a product. Almost everyone gets hand-written fsync handling wrong at least once.
- **Benchmark trap:** a storage engine "10× faster than Postgres" often has durability off; drop the persistence guarantee and any database speeds up.

Real-world examples

- **PostgreSQL's WAL** — the reference: crash recovery, replication (streaming WAL to replicas) and point-in-time recovery (archived WAL + base backups) at once.
- **MySQL/InnoDB** keeps a *redo log* for exactly this; **SQLite's WAL mode** puts it on most phones.
- **Kafka** — a system that is *nothing but* the log, with consumers replaying; hence its recurring role in Module 2.
- **DynamoDB, Aurora, Spanner** — every cloud database has it at the base.
- **For AI engineers:** vector databases (**Qdrant, Milvus, Weaviate**) route ingestion through a WAL so confirmed embeddings are never lost. Long training runs use checkpoint-plus-log thinking: periodic model-state snapshots let a crash resume rather than restart. An agent can durably record an intended action *before* a side effect, so after a crash it can recover and reconcile — provided the action is also idempotent.

In the interview

Directly: "How does a database survive a power cut without losing acknowledged writes?" Related: design a **key-value store** (a database mapping keys to values, like a giant dictionary), explain why sequential writes are fast, or describe what a restarted database does before accepting queries.

30-second answer: Before touching any data structure, the database appends one record for the whole change to an append-only log, fsyncs it, and only then acknowledges; tables and indexes update later in the background. On restart it replays from the last checkpoint, discarding a torn tail record by checksum, so every acknowledged write survives. The append is sequential, so the safe path is the fast path, and group commit packs many commits per fsync. The same log gives replication (ship it) and point-in-time recovery (archive it with snapshots).

Likely follow-ups (with answers):

- *"Why is the log faster than just updating the data files?"* — One sequential append to one file vs. scattered page writes; disks, SSDs included, strongly prefer streams. And one record can cover a change spanning many pages, giving atomicity in-place writes physically can't.
- *"What bounds recovery time, and what if the crash tore the last record in half?"* — The checkpoint interval (only newer records replay). The torn tail fails its checksum and is discarded — safe because it can't have been fsynced, so its client never heard "saved." Tying it back to the acknowledgment rule completes the answer.

Mistake that fails candidates: "the database writes it to disk" with no step ordering, or quoting benchmarks with durability off.

Self-check

- **Q1. Put these in order — apply to tables, fsync, acknowledge client, append to log — and state the rule that makes it safe.** → Append → fsync → acknowledge → apply to tables (later, background). Rule: never acknowledge before the record is fsynced; then no acknowledged promise can be lost, whenever the crash comes.
- **Q2. One team checkpoints every 15 s, another every 2 h. Describe their good and bad mornings.** → 15 s: constant cost (frequent flushing, more background I/O, slightly less spare throughput) but recovery in seconds. 2 h: little daily overhead, but up to two hours of replay — minutes to tens of minutes of downtime after the machine returns. Neither is wrong; choose from the **recovery-time objective** (how long a restart may take), not defaults.
- **Q3. A teammate says a new engine is "11× faster on writes" than Postgres; its config says `durability: relaxed`. Explain the 11× and make it fair.** → Most of the 11× is the durability promise handed back: it acknowledges before data is on disk, comparing memory speed with a disk promise ("saved means saved" vs. "probably"). For a fair test, set `synchronous_commit = off` in Postgres or make the new engine fsync every commit. Relaxed durability is fine if the workload can lose a second; the mistake is not knowing you chose it.

Related patterns

Primary-Replica (replication *is* shipping this log) · Event Sourcing (the log becomes the app's data model) · Change Data Capture (reads the WAL to emit change events) · Message Queue (Kafka is this pattern as a product).

TL;DR

- **Problem:** one logical write touches many scattered disk spots and power can fail between any two, so in-place writing can't deliver "saved means saved."
- **Mechanism:** append one checksummed record for the whole change, fsync, then acknowledge; apply later; after a crash replay from the last checkpoint; group commit shares one fsync across many writes.
- **Costs:** double writes, an fsync latency floor, log housekeeping, recovery time capped by checkpoint interval.
- **Skip/relax it when:** data is truly disposable (caches) or a redoable bulk load justifies it temporarily — always deliberately, never by config accident.

Event Sourcing

Original lesson: Event Sourcing on DesignGurus ↗

What it is. Instead of storing *current state* and overwriting it, store *what happened*: each change is appended as an **immutable** (never changed once written) event with business meaning (`OrderPlaced`, `ItemRemoved`, `OrderCancelled`). Current state is the result of replaying an entity's events from the start. The log doesn't record the truth; it *is* the truth.

Analogy. Your bank keeps transactions, and your balance is their running total; it can explain "why $412?" line by line forever, and fixes mistakes with correcting entries, never erasures. Accountants have worked this way for ~500 years.

The problem

Three questions in one week the database can't answer:

- **Monday, a dispute.** A customer says they cancelled order 7f3a before it shipped and wants return postage refunded. Support sees `status = shipped`. Was it ever `cancelled`, maybe for minutes at 14:02 before the shipping job overwrote it? Each `UPDATE` replaced the last value; the history isn't hidden, it's **destroyed**.
- **Wednesday, a bug.** A pricing bug two weeks ago corrupted some totals and was fixed. Finance wants to know which of **~90,000** orders were affected, but the wrong totals were overwritten, some fixed by hand, and the fix erased the evidence.
- **Friday, product.** "How often do customers add then remove an item before checkout?" Cart removals were never stored.

One root cause: the database keeps only the present, and `UPDATE` destroys the past. Beneath its tables the database *already* keeps an append-only log, but privately, for its own survival, and in **row-diffs** ("column X changed to Y"), not business terms. What if the application kept such a log in business language and treated it as the truth?

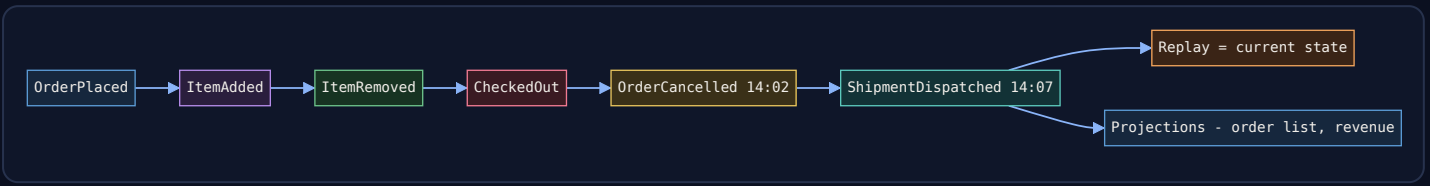
Why the obvious fixes fail

- **"Add an audit table"** (**triggers** — database rules that run automatically on every change — or app code copying changes into `order_history`). Half works, but it's an afterthought copy: some code path forgets it, backfills skip it, and when it disagrees with the orders table, which is right? It's only as reliable as its sloppiest writer, and still records diffs ("status: shipped"), not intent ("customer clicked cancel").
- **"Add `updated_at`, `previous_status`… columns."** Clutters the schema and keeps one step of history; maybe answers Monday, never Wednesday or Friday.
- **"We have point-in-time backups."** True (WAL lesson), but restoring a two-week-old database to answer one question is slow, and a backup shows *that* a row changed, never *why*.

All three bolt history onto a source of truth that forgets. Event sourcing inverts that.

How it works

Order 7f3a's stream: replay reveals the cancel-then-ship race the old design overwrote.



Order 7f3a as six events: placed ($49, item A, 13:41), item B added ($89), item A removed ($40), checked out ($40, 13:51), cancelled at 14:02, dispatched by UPS at 14:07. Replay gives "shipped but cancelled" — a conflict the old design silently overwrote, now in the open. The same stream feeds projections such as an order list and revenue totals.

1. **The event store:** an append-only home organized as **one stream per entity** (all of 7f3a's events, in order). Appending is the *only* write — the write-ahead log again, one level up, with business verbs instead of row-diffs.
2. **Current state = replay:** apply 7f3a's six events in order from nothing. For long streams, a **snapshot** every N events stores computed state so replay covers only the tail — the checkpoint idea, a third time.
3. **Projections handle queries.** "All orders over $100 this month" can't be asked of a log without replaying everything, so you keep **projections**: ordinary read-optimized tables (`current_orders`, daily revenue), each updated from the event stream — the same mechanism as EDA's local views. Projections are **derived, disposable, and rebuildable** from the log.
4. **Mistakes are corrected, never erased.** The pricing fix is a new `OrderTotalCorrected` event with a reason, not an `UPDATE`; error and correction stay visible forever, as finance and auditors want. Immutability is the feature.

Walkthrough with numbers

The week's three questions, from the event store:

- **The dispute:** stream 7f3a shows `OrderCancelled` at **14:02** then `ShipmentDispatched` at **14:07**. The customer is right; the refund goes out in minutes. Bonus: a real **race condition** (two steps running without checking each other) — the shipping job doesn't check for cancellations. The evidence of the bug *is* the data.
- **The pricing bug:** scan two weeks of `CheckedOut` events for replayed totals matching the buggy formula — an exact affected list in one query — then append traceable `OrderTotalCorrected` events. Finance gets a spreadsheet and a complete record.
- **The product question:** count `ItemRemoved` events following an `ItemAdded` for the same item across all streams. Never tracked on purpose, answered after the fact: *history answers questions you haven't asked yet* — the pattern's most underrated strength.
- **The bill:** a typical stream is **~12 events × 300 bytes ≈ 4 KB** vs. one **400-byte** row: **~10–15× the storage**, real but small next to the benefits. Replaying one order takes microseconds, but **rebuilding a projection over 100 million orders takes hours** — the real limit, and why projections are maintained continuously rather than computed on demand.

Event sourcing vs. change data capture (the comparison)

For history alone there's a cheaper option that leaves the app untouched: **change data capture (CDC)** reads the database's own WAL and publishes every row change as an event (full lesson later).

	Event sourcing	CDC-based history
Source of truth	The event log	Your current tables (left as they are)
What is captured	What the business meant: `OrderCancelled`, by customer	Row diffs: status 'active' → 'cancelled'
Adoption cost	You rebuild the write path	You add a connector; the app doesn't change
Can it miss changes?	No, because events *are* the writes	No, because the WAL sees every change (audit tables can miss some)
Answers "why?"	Yes, since intent is stored	Partly: you see what changed and guess why

Routing rule: CDC when you need *history*; event sourcing when the *events are the product*. CDC gives complete, unmissable change history for the cost of a connector — the mainstream choice for analytics, caches and search indexes. Event sourcing earns its cost only where the business meaning of every change must be stored: money, disputes, audits.

Trade-offs

You gain:

- Complete, trustworthy history *by design*, not discipline; audit is the storage model.
- Debugging with full history: the state of anything at any moment and the steps that led there.
- After-the-fact analytics: new questions from old data.
- A natural fit with Module 2: the events to publish already exist, so the log doubles as the **outbox** (store of events waiting to be sent).

You pay:

- **Queries need projections:** the log answers "what happened to this order?" perfectly and "which orders exceed $100?" badly (CQRS addresses this).
- **Events — and their formats — live forever.** Code years from now may replay today's `OrderPlaced`, so event schemas need **versioning from day one**: additive changes only, plus "upcast-on-read" logic upgrading old formats as they're read.
- **Eventual consistency again:** projections trail the log; read-your-writes applies.
- **Model cost:** uniqueness rules ("an email registers once") are awkward across append-only streams; new teams start slowly; and deletion laws clash with an immutable log — answered by **crypto-**

shredding (encrypt each person's personal fields with their own key, destroy the key to make the data unreadable everywhere).

When not to use it

The course's clearest "when not": **the default answer is no.** Most applications manage current state (profiles, settings, content pages) where history doesn't matter; a plain table plus an audit log covers the occasional "who changed this?". Use event sourcing where the domain is a **ledger**:

- **Money** — balances, payments, wallets; the bank statement is the requirement, not an analogy.
- **Inventory and orders** — contested, audited, dispute-prone: the incident's world.
- **Compliance-audited data**, where a complete, provably untampered history is legally required.

Skip it for high-volume latest-value data (sensor readings where only the latest counts), CRUD admin tools (**CRUD** = create, read, update, delete), and teams whose unfamiliarity would cost more than the history is worth. Two misuses: **ideological event sourcing** (converting the whole company because payments loved it), and **CRUD disguised as events** — `OrderUpdated {field: status, new_value: shipped}` is row-diffs with event names; real events are business verbs (`OrderCancelled`, `PaymentCaptured`).

Real-world examples

- **Banking and accounting** — ~500 years old; a double-entry ledger is a hand-kept event log.
- **Git** (standard code version control) — immutable commits are events, the working directory (files you see) is a projection, `git log` the audit trail.
- **Kafka-centric architectures** — the topic is the durable log and every database a projection: event sourcing at company scale.
- **EventStoreDB, Axon, Marten** — purpose-built event stores.
- **For AI engineers:** agents are event-sourced whether planned or not: a conversation is an append-only stream of messages, tool calls and results, and agent state is its replay. That's why you can re-run an **eval** (automated quality test of a model or agent) from history, debug by replaying a session, and audit what an agent did and why — so store traces as real event streams. Feature stores need it for **point-in-time correctness**: training "as of March 1st" means replaying feature history to that moment, possible only if history was kept as events.

In the interview

Classic: "Design a banking ledger or digital wallet" — say transactions are the authority and balances derived. Others: "we need a complete audit history," "how would you find out what happened to one order?"

30-second answer: For true ledgers — money, orders, inventory — I store events, not state: an append-only stream of business facts per entity (OrderPlaced, OrderCancelled), with current state from replay plus snapshots. Queries hit projections — disposable read tables rebuilt from events. Mistakes get compensating events, never edits, so the audit trail is the storage model. Costs I manage: eventually consistent projections, event schemas that only evolve additively, crypto-shredding for deletion. Only where history is the product; for plain history, CDC on a normal database is cheaper.

Likely follow-ups (with answers):

- *"An account has 2 million events — do you replay them all for the balance?"* — No: snapshots every N events, replaying only the tail (like database checkpoints). For hot accounts, a continuously maintained projection means no replay at request time.
- *"Legal says delete this user, but your log is immutable."* — Crypto-shredding: personal fields encrypted with a per-user key from day one; destroying the key makes every copy unreadable —

log, snapshots, projections *and backups*. Immutable structure, erasable content. Mentioning backups unprompted signals seniority.

Mistake that fails candidates: event sourcing for everything, row-diff events with event names (`OrderUpdated`), or no projection plan ("replay on every request" fails at the first long-lived entity).

Self-check

- **Q1. Where does truth live, how is current state obtained, and what happens when a past event was wrong?** → Truth is the append-only event log; everything else (tables, dashboards, caches) is a projection. State = replaying the entity's events, shortened by snapshots. A wrong event is never edited; a compensating event (e.g. `OrderTotalCorrected` with a reason) is appended, keeping mistake and fix visible.
- **Q2. A digital wallet, user profile settings, warehouse inventory, a marketing CMS — which two deserve event sourcing?** → Wallet and inventory: ledgers where "how did we reach this number?" is business-critical. Settings and the CMS are current-state domains: plain tables plus an audit log or CDC if "who changed this?" matters.
- **Q3. The revenue dashboard (a projection) disagrees with the event store's total. Which is right, what happened, what's the fix?** → The event store (the authority). The projection's consumer likely crashed and missed events, applied a redelivered event twice without **idempotency** (safe repeat handling), or has a bug. Fix the consumer, then rebuild the table by replaying the log — hours for a big store, but derived data can always be regenerated rather than staying corrupt.

Related patterns

Write-Ahead Log (the same technique one level down) · Publish-Subscribe (the log doubles as the outbox) · CQRS (events on the write side, different storage for reads) · Change Data Capture (cheaper for plain history).

TL;DR

- **Problem:** `UPDATE` wipes history, so disputes can't be settled, bug impact can't be reconstructed, and future questions can't be answered.
- **Mechanism:** append immutable business events (one stream per entity) as the source of truth; state by replay (with snapshots); queries from rebuildable projections; mistakes fixed by compensating events.
- **Costs:** projections and lag, ~10–15× storage, additive-only schemas forever, crypto-shredding for deletion, awkward uniqueness, a learning curve.
- **Skip it when:** the domain is current-state CRUD (most). Default no; CDC or an audit log for plain history; event sourcing for ledgers — money, orders, inventory, compliance.

CQRS

Original lesson: CQRS on DesignGurus ↗

What it is. CQRS (Command Query Responsibility Segregation) splits one data model in two. **Commands** (writes, e.g. "place order 7f3a") go to a **write model** built for correctness: **normalized** (each fact stored once), validated, transactional. **Queries** (reads, e.g. "orders for customer 8412") go to one or more **read models**, each shaped for its question: pre-joined tables, search indexes, **aggregates** (pre-computed totals and counts). A change stream from the write side keeps every read model current.

Analogy. At a bank, one desk takes deposits and a separate system prints statements; each is fast because it has one job.

The problem

One orders database serves every team, each wanting something different:

- **Checkout** writes orders and wants a normalized schema, strict validation and transactions.
- **The orders page** joins six tables (order, items, customer, addresses, shipments, payments) for one screen: **300 ms**.
- **Support** wants search by customer name and product — text search, which the database does badly.
- **Analytics** wants daily aggregates.

Each read problem was "fixed" with another **index** — really a re-sorted partial copy of the table, updated on *every* insert. Orders now has **14 indexes**, so each new order does **15 writes**; write latency crept from **5 ms to 40 ms**, and support's latest index made checkout a measured **15%** slower. Writes want a lean schema; reads want it pre-joined, duplicated and indexed sixteen ways; sharing **one model**, every improvement for one side hurts the other. Replicas fixed reads and writes that differed in *volume*; these differ in **shape**, and more copies of the same shape can't fix shape.

Why the obvious fixes fail

- **"Add the index anyway."** Each speeds one query and slows every write, permanently; at 14, out of road.
- **"Denormalize the main table."** **Denormalizing** copies a fact into several places to avoid a join; flattening orders into one wide table speeds reads, but every write must keep duplicates in step — a customer renaming means updating a million rows or living with wrong data. Read model blended into write model, both worse.
- **"Read replicas."** Same schema, so the six-way join is 300 ms there too.
- **"Cache the hot queries."** Helps (Module 4), but the shape mismatch remains and you add invalidation. A workaround.

Fix: stop asking one model to do two jobs.

How it works

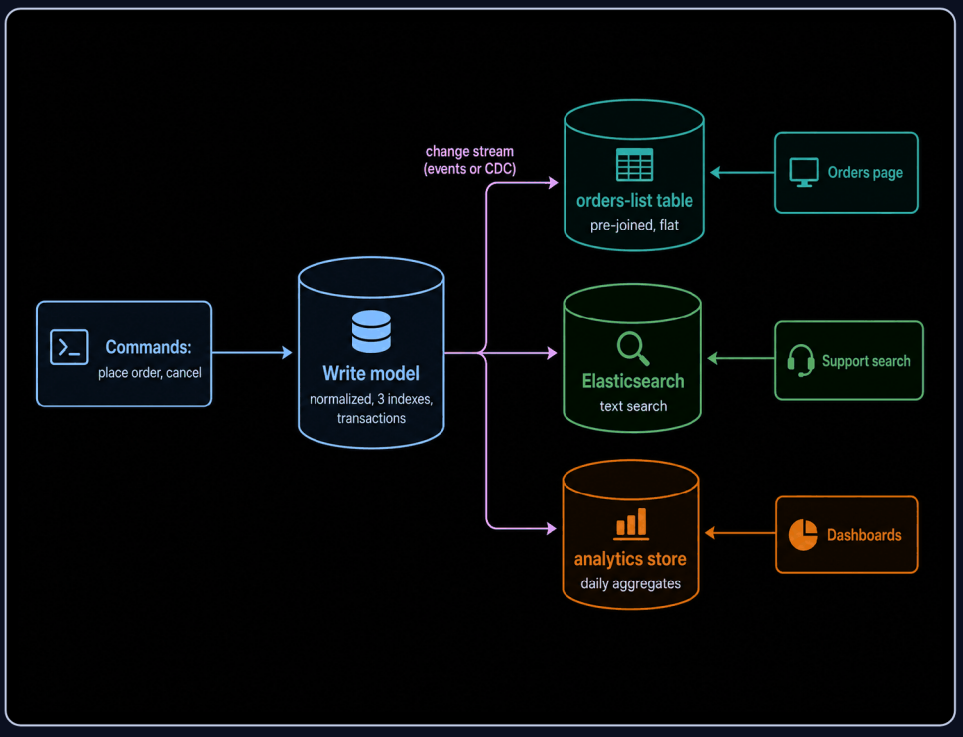


Figure — CQRS (course diagram)

Commands (place order, cancel) enter a write model — normalized, 3 indexes, transactional. A **change stream** (events or CDC) feeds three read models: a flat pre-joined `orders-list` table for the orders page, Elasticsearch (a dedicated search engine) for support, and a daily-aggregates analytics store for dashboards.

1. **Commands and queries take separate routes.** Commands go to the write model (validated, committed); queries go to the right-shaped read model and **never touch the write model**.
2. **Read models are projections** — the derived, disposable, rebuildable copies of Event Sourcing (and EDA's local views). CQRS names the split; the machinery is familiar.
3. **Sync from the log, never with dual writes.** The first universal mistake: app code writes Postgres and *then* **Elasticsearch** — a **dual write**. Not one transaction, so a crash between them leaves the models disagreeing forever, unnoticed. Instead read one stream of truth: consume the write model's **domain events** (business facts like `OrderPlaced`), or tap the database's change log with **change data capture (CDC)**, which reads each committed change from the WAL. Projection updates must be **idempotent** (applying twice = applying once), making redeliveries harmless. With event sourcing, your projections already are CQRS: one writer, one log, many followers.
4. **CQRS is a spectrum.** The second universal mistake is assuming "two databases." Three levels:
   - **Level 1:** same database, separate code paths and query models for reads and writes. Cheap, often enough.
   - **Level 2:** read models as separate tables or replicas with a different shape (e.g. a flat orders-list table).
   - **Level 3:** separate systems, e.g. Elasticsearch for search and a **data warehouse** (analytics database) for reports. Move only as far as your measured shape conflict forces.

CQRS and event sourcing are related, not the same. Event sourcing without CQRS exists (a log with one state view); CQRS without event sourcing is everywhere (CDC-fed read models over a plain **relational** write model — ordinary SQL tables linked by keys). Either can be adopted alone.

Walkthrough with numbers

- **The write model drops 11 of 14 indexes**, keeping the 3 needed for writing and validation: insert latency **40 ms → 8 ms**.
- **The orders page** reads a flat `orders_list` projection: the **300 ms** six-way join becomes an **8 ms** single-table read. The join happens once, at write time, in the projection updater.
- **Support search** uses Elasticsearch fed from the same stream (**~40 ms** full-text queries); **analytics** reads a warehouse fed the same way, off production.
- **The lag:** read models trail by **~200 ms**, so a just-placed order "isn't there" on the orders page — read-your-writes for the third time (after replica lag and EDA view lag), with the same remedies: briefly route that read to the write side, or show the order optimistically.
- **New features:** six months later the mobile team's compact order summary is just another projection on the stream; the write side sees no change or risk, like adding a pub/sub subscriber.

CQRS vs. read replicas (the comparison)

	Read replicas	CQRS read models
Solves	How *many* reads	What *shape* reads need
The copy is	The same schema, identical byte for byte	Shaped around each query: flattened, indexed and pre-summed
A 300 ms six-way join becomes	Still 300 ms, just on another machine	8 ms, because it was pre-joined at write time
Cost	Low: one checkbox on managed databases	A pipeline, projections, and managing lag
Lags the primary?	Yes	Yes; this trade-off never goes away

Diagnostic: would more copies of the same shape help? Too many reads → replicas (cheaper, faster). Wrong-shaped reads (read-time joins, text search, aggregates) → reshape with CQRS. Real systems combine them: a CQRS read model, itself replicated for volume.

Trade-offs

You gain:

- Each side tuned without hurting the other: lean writes, purpose-shaped reads.
- Per-model read scaling: each read model cached, replicated or sharded for its own load.
- New read needs met by adding projections, not risky core-schema edits.
- The right tool per question — full-text search in a search engine, aggregates in a warehouse — from one source of truth.

You pay:

- **Eventual consistency between sides;** read-your-writes must be handled. Non-negotiable; plan for it.
- **A pipeline to run:** change stream, consumers, lag metrics, alerts — the price of avoiding dual writes.
- **Duplicated data:** storage, plus the discipline that the write model is always the truth.
- **Complexity needing justification.** One of the industry's most over-used patterns — whole CRUD apps split because a talk sounded exciting. Worth it only for a measured shape conflict.

When not to use it

1. **Most applications.** A moderate-scale CRUD app whose one schema serves both sides well enough needs indexes and maybe a replica, not two models and a pipeline. The incident's 14-index conflict was *measured*; wait for yours.
2. **Reads must always be instantly consistent:** the lag rules out the split, or you route so many reads to the write side that it achieves nothing.
3. **The team can't run the pipeline:** without lag monitoring, backfills and projection rebuilds, read models drift silently, and users trusting stale results is worse than correct-but-slow.
4. **Level 1 is enough:** often "CQRS done right" just means not letting the **ORM**'s single model (the library mapping tables to code objects) shape both paths, and writing focused query code.

Misuses: CQRS by default in every microservice template, and dual-write implementations — all the operating cost, none of the correctness.

Real-world examples

- **Every large e-commerce (online shopping) product page** — normalized write side for catalog (product listings) and inventory; denormalized, pre-joined, heavily cached product-view read model; search index alongside.
- **Twitter timelines** — the famous extreme: tweets on the write side; each user's home timeline pre-built at tweet time (**fan-out**) so reading is fetching a list. The "tweets from everyone I follow" join runs once at write time, not billions of times at read time.
- **YouTube, LinkedIn, Uber** — view counts, profiles and trip histories read from purpose-shaped projections, not the transactional core.
- **For AI engineers:** a **feature store** is CQRS: a compute/write path builds features, a separate low-latency online path serves them, pipelines keep them in sync. Every **RAG** system (retrieval-augmented generation: the model answers using documents fetched for the question) is CQRS too: documents live in a source-of-truth store (write model), and the **vector index is a read model** shaped for "what's similar to this?", fed by the document change stream and rebuilt when the embedding model changes — Event Sourcing's projection-rebuild property.

In the interview

Everyday: "this page joins six tables and it's slow — fix it." Famous: "design Twitter's timeline" (write-time precomputation is a CQRS read model, acronym or not). Also: "add full-text search over orders" (CQRS with an Elasticsearch projection).

30-second answer: Writes want normalized, validated data; reads want data pre-joined and indexed per question; one model slows both. So commands go to a lean write model, and queries go to read models shaped per question — a flat orders table, a search index, an aggregates store — each synced from the write side's change stream (events or CDC), never dual writes from app code, which drift apart. Read models lag slightly, so a user's own just-written data reads from the write side or updates optimistically. It's a spectrum — separate query code, reshaped tables, separate systems — and I go only as far as the measured conflict requires.

Likely follow-ups (with answers):

- *"How exactly do the read models stay in sync?"* — From one authoritative change stream: the event log with event sourcing, otherwise CDC on the WAL. Raise the dual-write trap unprompted — app code writing two stores can crash in between, and log-based sync makes that impossible. Idempotent projection updates make redeliveries safe.
- *"A user places an order and their order list doesn't show it."* — Read-your-writes, third appearance: the list is a lagging projection. Route that user's list read to the write side for a few seconds, or show the order optimistically. The interviewer is scoring whether you see it as replica lag again, with the same remedies.

Mistake that fails candidates: separate stores synced by app dual writes, CQRS for a settings page, or claiming CQRS requires event sourcing.

Self-check

- **Q1. What does the write model optimize for, what do read models optimize for, and what single rule governs sync?** → Write model: correctness (normalized, validated, transactional, minimal indexes). Each read model: one question's shape (pre-joined, aggregated, text-indexed). Rule: every read model is fed from **one authoritative change stream** (events or CDC), never app writes to several stores, which can partially fail and drift silently.
- **Q2. Team A: checkout and orders page are fine, but the new free-form order search is painfully slow. Team B: dashboards are fast but write latency tripled as indexes piled up. Prescribe a level for each.** → Team A: a one-feature shape problem (full-text search is poor in a relational store however many indexes) → **Level 3 for that path only**: a CDC-fed Elasticsearch projection, everything else unchanged. Team B: the incident's problem (read indexes slowing writes) → **Level 2**: move dashboard queries to a reshaped read table (or replica + projection) and drop the extra write-side indexes. Neither needs event sourcing or a rewrite; the spectrum lets you stop early.
- **Q3. After a deploy, some orders in Postgres are missing from search. The app writes to Postgres, then calls Elasticsearch, and some ES calls timed out during the deploy. Root flaw, immediate remediation, permanent fix?** → Root flaw: the **dual write** — two calls, no shared transaction, so a failure between them leaves permanent, invisible divergence. Immediately: backfill by comparing recent Postgres orders with the index and re-indexing the missing ones. Permanently: remove the second write and feed Elasticsearch from the change stream (CDC, or the outbox/event stream) with idempotent updates. Sync failures then become *lag* (visible, monitored, self-healing) instead of *loss* (silent, permanent). Read models follow the log, not the application.

Related patterns

Event Sourcing (its projections are ready-made read models; its log the ideal change stream) · Change Data Capture (the mainstream route to CQRS over relational write models) · Primary-Replica (read volume vs. CQRS's read shape; often stacked) · Cache-Aside (a cache is a small, short-lived read model).

TL;DR

- **Problem:** one model serving both sides makes both worse: each read index slows every write, while reads still pay for joins and searches the write schema wasn't built for.
- **Mechanism:** commands to a lean, validated write model; queries to purpose-shaped read models, all fed from one change stream (events or CDC), never app dual writes.
- **Costs:** eventual consistency and read-your-writes handling, a sync pipeline, duplicated data, complexity only a measured shape conflict justifies.
- **Skip it when:** one model serves both well enough (most apps), reads must be instantly consistent, or the team can't run the pipeline. Go only as far along the spectrum as needed.

Capstone - Scaling a Database to 100 Million Users

Original lesson: Capstone - Scaling a Database to 100 Million Users on DesignGurus ↗

What this capstone is. The promised story: **one database grown to 100 million users, one pattern at a time.** The first half is *scaling* (each stage a volume limit a pattern removes); the second is *quality* (limits of trust, history and shape). At each stage, name the pattern and its key decision before reading on. The order matters: real companies often hit exactly this sequence, and interviewers love "which limit comes next?". Each stage summarizes its lesson; follow the link for the full mechanics.

The brief

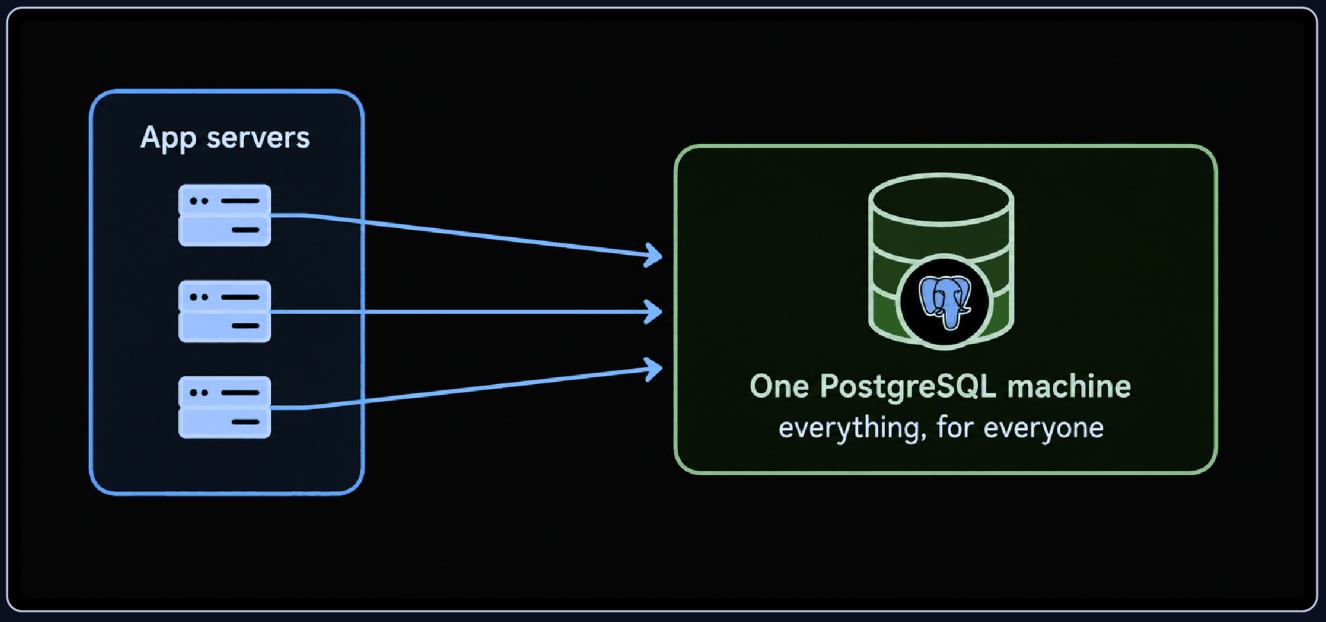


Figure — Scaling the DB: the brief (course diagram)

A few app servers point at **one PostgreSQL machine** doing "everything, for everyone": a commerce startup with **50,000 users** and a few hundred queries per second. One machine handles orders, customers, inventory, search and dashboards; transactions are free and any query can join any table. It won't last.

Stage 1 — Running out of read capacity (500,000 users)

Incident. 10× growth: **8,000 reads/s + 300 writes/s** against a **5,000 queries/s** machine; product pages take four seconds. The review also notes this machine is the *only copy*, with last night's backup as the whole disaster plan.

Pause and decide: 96% reads. Which pattern? Where may writes go? What problem do copies bring?

Decision: Primary-Replica. All writes to the primary; two replicas serve reads (**~4,000 reads/s each**). Chosen over a bigger machine or cache because it fixes read capacity *and* the single point of failure.

Failover is **rehearsed**, not just configured: a practice promotion teaches the **fencing** procedure before a real 2 AM failure.

New problem: replication lag (an edited address reappears from a replica). **Fresh-data reads go to the primary**; the rest tolerate milliseconds of delay.

What this sets up: writes become the problem, and copies can't help.

Stage 2 — Running out of write capacity (5 million users)

Incident. **2,500 writes/s** max out the primary and every replica; lag stretches to **minutes**; orders is **4 TB**, indexes don't fit in memory, backups run all night, no bigger machine exists.

Pause and decide: the data must be divided. Which pattern, and what is its most important decision?

Decision: Sharding on `customer_id` across **4 shards**. The key wins on all three criteria (in almost every query, even spread, keeps a customer's orders, cart and addresses together so checkout never crosses machines). **Hash routing**; each shard a primary-replica cluster (**4 × 3 = 12 databases**). Accepted costs: "top products today" becomes **scatter-gather**; cross-shard transactions are *avoided by the key choice*; a viral influencer storefront (hashing spreads keys, not load) gets **a cache for that hot key's reads**.

What this sets up: routing is `hash(customer_id) % 4`, and that `% 4` will bite in two years.

Stage 3 — Changing the shard count (20 million users)

Incident. Four shards must become five; `% 4` → `% 5` re-addresses **~80% of all rows** (a key stays only when both remainders match, **one time in five**) — terabytes moving under 2,500 writes/s, repeated at every resize.

Pause and decide: what replaces modulo, and what share *should* a resize move?

Decision: Consistent Hashing. Shards and keys on a ring with **~150 virtual nodes** per shard; shard five moves **~1/5** of data, streamed in the background while old owners serve until handoff. **Bonus fix:** the cache layer's nightly autoscaling had been causing **miss storms** (bursts of misses hitting the database together) that nobody had linked to modulo routing.

What this sets up: volume is solved (shards for writes, replicas for reads, ring for growth); next come limits of **trust**.

Stage 4 — Crash and corruption (a bad Tuesday)

Incident — two scares in one quarter:

1. *Power loss.* A rack loses power mid-write; shard 2 restarts. Did the **40 orders** confirmed in the last seconds survive? They did, but nobody can explain why.
2. *Bad deploy.* Corrupted totals are written to thousands of orders for **two hours** — committed and **replicated** (replicas faithfully copied the bad data). Restoring last night's backup would lose a day.

Pause and decide: one mechanism explains both survival and rescue. What is it, and what two abilities beyond crash safety does it give?

Decision: understand (and use) the Write-Ahead Log.

- *Why the 40 orders survived:* a log record is **fsynced** (forced to disk) *before* any confirmation; replay rebuilt every promise.
- *How the corruption is fixed:* **point-in-time recovery** — restore the latest base backup, replay archived WAL to **13:59** (a minute before the bad deploy), then re-apply legitimate orders from the corrupted window. Replicas copy bad writes; only history helps.
- *Operational follow-up:* checkpoint intervals set from recovery-time targets, WAL archiving verified, and durability settings (e.g. for benchmarks) decided in writing.

What this sets up: the database keeps perfect history for itself, in row-diffs, while support handles disputes with a present-only table.

Stage 5 — History destroyed by UPDATE (60 million users)

Incident. "I cancelled before it shipped," but the table shows `status = shipped`; finance can't find which orders a two-week-old pricing bug hit (the fix overwrote the error); compliance wants a provable payment audit trail. One cause: `UPDATE` **destroys history.**

Pause and decide: why not an audit table? Which pattern, where, and where not?

Decision: Event Sourcing, applied selectively. Orders and payments — the ledger-shaped domains — move to append-only event streams (`OrderPlaced` , `ItemRemoved` , `OrderCancelled`), state by replay with **snapshots**. The dispute is settled in minutes (`OrderCancelled` 14:02, then `ShipmentDispatched` 14:07 — the customer was right), exposing a race-condition bug. Corrections become **compensating events** with reasons. **Where not:** profiles, sessions and the catalog stay plain tables. **Why not an audit table:** a copy beside the truth drifts when code forgets it; here the log *is* the truth.

What this sets up: the orders write side is now events — great for history, useless for "all orders over $100" — so queries need projections, and the rest of the system has a related problem.

Stage 6 — One model, two jobs (100 million users)

Incident. The orders page takes **300 ms** (six-table join); support wants full-text search; dashboards run heavy aggregates on production; the write path carries **14 indexes** (fifteen writes per insert), and support's newest index measurably slowed checkout.

Pause and decide: why won't replicas help? Which pattern, what feeds it, which trap to refuse?

Decision: CQRS. Replicas copy the same shape (300 ms everywhere), so split the model: the write side keeps **3 indexes** and validation; reads move to purpose-shaped projections — flat `orders_list` (**300 ms → 8 ms**), Elasticsearch for search, a warehouse for analytics — all fed from **one change stream** (event streams where they exist, **CDC** on the WAL elsewhere). **Dual writes from app code are refused**: lag (visible, self-healing) beats loss (silent, permanent). Read-your-writes returns once more with the usual remedies, and the mobile team's order summary ships as one more projection.

The final architecture

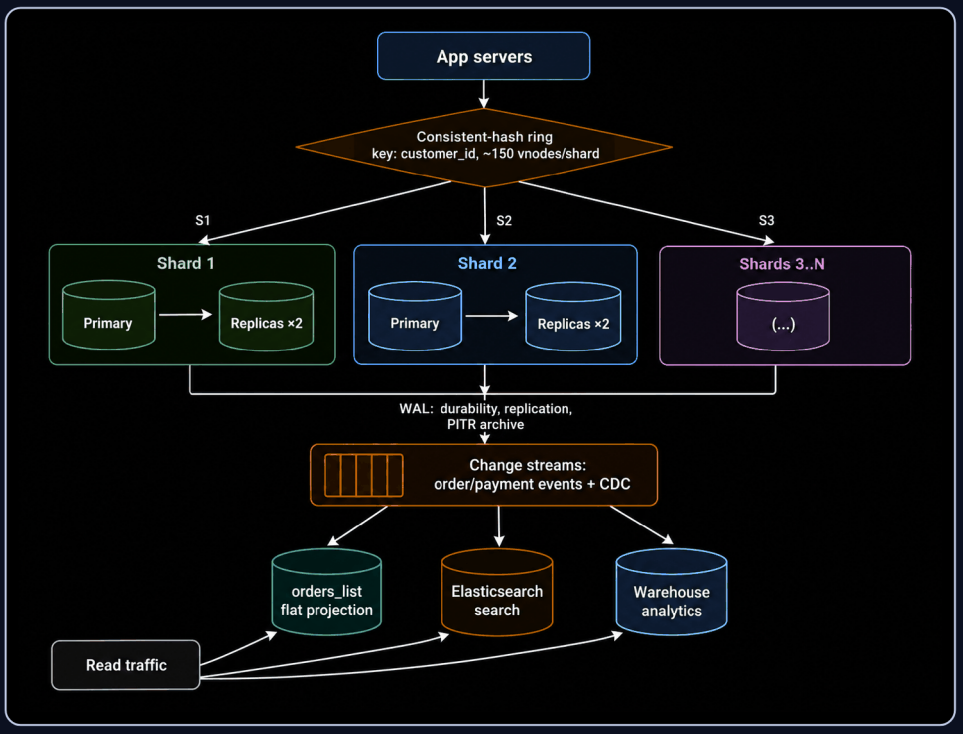


Figure — Scaling the DB: final architecture (course diagram)

App servers route through a **consistent-hash ring** (key `customer_id` , ~150 vnodes per shard) to Shards 1..N, each a primary with two replicas. Their **WAL** provides durability, replication and a PITR archive, and feeds the **change streams** (order/payment events + CDC) that fill three read models — `orders_list` , Elasticsearch, the warehouse — which serve all reads. The append-only log does four jobs here:

durability, replication, point-in-time recovery, *and* feeding every read model. The introduction called the log the key idea; here it *is* the architecture.

Debrief: the limits, in order

Stage	The limit	Pattern	The decision that mattered
1	Too many reads for one machine	Primary-replica	Send reads that need fresh data to the primary; practice failover
2	Too many writes and too much data for one machine	Sharding	Shard key = customer_id, with reasons given
3	Changing how many shards there are	Consistent hashing	Ring + virtual nodes; a resize moves ~1/N
4	Crash and corruption	Write-ahead log	Understood fsync-before-ack; PITR undid the bad deploy
5	History wiped out by UPDATE	Event sourcing	Used only for ledger-shaped domains
6	One model doing two jobs	CQRS	Projections fed from the log; dual writes rejected

- The "limit" column — **reads → writes → growth → trust → history → shape** — is the honest answer to "how does a database scale?". Each pattern answered a *measured* limit, never fashion.
- Stages 4–6 are the append-only log in three roles: the database's WAL, the application's event streams, the change streams feeding projections. Six patterns, one idea: **copies for reads, splits for writes, and a log underneath everything.**

Extend it yourself (open questions to practice)

- **Multi-region:** users on two continents want low local latency, and regulators want EU data kept in the EU. Which patterns stretch across regions cleanly (the ring? replicas?), where does the primary live, and what happens to cross-region writes?
- **The hot tenant:** your largest enterprise customer is **8% of all traffic** on a ring keyed by `customer_id`. Design an alternative with this module's tools (e.g. a dedicated shard via a directory/slot override), and say what breaks first if you don't.
- **For AI engineers:** add semantic search over orders ("find orders like this complaint"). Design the vector index as a CQRS read model: what feeds it, what happens when you switch embedding models, and why does "projections are disposable and rebuildable" become the property you care about most?

Interview one-liner

Scale a database in the order its limits show up: replicas for reads, sharding with a well-argued key for writes and size, consistent hashing for resizing, the WAL for durability and point-in-time recovery, event sourcing only for ledgers, and CQRS projections fed from the log once read and write shapes diverge.

Module 4 — Serving Data Fast (Caching)

Module Introduction

Original lesson: Module introduction on DesignGurus ↗

What this module is about. Module 3 ended by noting that *a cache is a small, short-lived read model*. Caches are among the most used — and misused — pieces of infrastructure, and among the most asked-about in interviews. A **cache** is a small, fast copy of data whose authoritative version lives in a slower system such as a database. (Basics: Module 0 — Caching basics.)

Analogy. Keeping the few books you use daily on your desk instead of walking to the library: quick, but if the library's copy is revised, yours is out of date unless someone tells you.

The key metric: hit ratio. A **hit** finds the data in the cache; a **miss** doesn't, so the request goes to the source. The **hit ratio** is the share of hits. With a 1 ms hit and a miss costing 1 ms *plus* a 100 ms database call, **99%** hits average **~2 ms**; **50%** averages **~51 ms** — and you're running an extra system (the formula). So the question isn't "should I cache?" but **how does data get into the cache, and how does it stay correct?** Five patterns, five answers:

The situation	The pattern
On a miss, the application fetches the data and saves a copy for next time	Cache-Aside
The same idea, except the cache does the fetching itself	Read-Through
Update the copy and the original together before confirming	Write-Through
Update the copy now and write the original later, in batches	Write-Behind
A popular entry expires, and only one request should fetch it again	Cache Stampede Prevention

Two facts that organize the module

1. **A cache buys speed with some accuracy.** Any cached copy can disagree with its source; you decide how much staleness is tolerable and which mechanism keeps the copy correct — expiry timers, write-time updates, or deliberate deletion. (*"There are only two hard things in computer science: cache invalidation and naming things."* Invalidation is half this module.)
2. **Caches fail as crowds, not one request at a time.** One miss is one short query. Serious failures come when a hot key expires and ten thousand requests hit the database at once, or a cache cluster restarts empty at peak. The stampede lesson covers these — the difference between *using* a cache and *running* one reliably.

How the module is told. One product page atop Module 3's database runs through every lesson: Cache-Aside (the default) has the app fetch misses, Read-Through moves fetching into the cache, Write-Through and Write-Behind are the write-side pair (safety vs. speed), Stampede Prevention handles the returning flash sale, and the capstone picks a strategy per data type. **Short on time?** Cache-Aside and Stampede Prevention matter most; the write pair usually appears as a comparison question.

Four terms used throughout

- **Hit / miss / hit ratio** — above; know it for every cache you run.
- **TTL (time to live)** — a per-entry expiry timer ("trust this for 5 minutes"), the simplest, most common correctness tool.
- **Eviction** — a cache is deliberately smaller than the data, so when full it discards something, usually by **LRU (least recently used)**: drop what's gone untouched longest, assuming recent use predicts future use.

- **Invalidation** — actively removing or updating an entry when its source changes; more precise than a TTL, harder to get right.

Before you start — three questions about a system you know:

- What's your cache's hit ratio today? (If nobody knows, its value is a guess.)
- When source data changes, how does the cache learn: timer, update, delete, or not at all?
- At peak, what happens the instant your hottest key expires — who goes to the database, and how many?

Cache-Aside

Original lesson: Cache-Aside on DesignGurus ↗

What it is. The application checks the cache first. **Hit:** return the copy. **Miss:** read the database, save a copy with a TTL, return it, so the next reader is fast. **Write:** update the database, then *delete* the cached copy so the next reader fetches fresh data. The cache sits "to the side" and never talks to the database itself.

Analogy. A receptionist with sticky notes of common answers checks them first; if missing, calls the department, writes a fresh note ("throw away after 1 hour") and replies; when a department announces a change, rips up that note.

The problem

Module 3's database works, but mostly repeats itself. Bestseller **item 8412** is read **4,000 times per minute**; its row hasn't changed in **three weeks**, yet each read recomputes the same **15 ms** answer. Catalog reads peak at **12,000/s**, over **99%** identical to the previous request's answer, on **eight** replica machines; **latency** (time per request) exceeds 15 ms when requests queue at peak, and every campaign triggers a capacity meeting. Keep the last answer in fast storage and serve it until the data changes.

Why the obvious fixes fail

- **"More replicas."** They scale read volume, but cost grows as fast — full queries just to reproduce unchanged answers. The problem is *repetition*, not database capacity.
- **"A dictionary in each app server's memory."** Cheap and tempting, but with **40 app servers** you get 40 private copies: 40 cold misses per key, 40 copies going stale independently, and every **deploy** (rollout of new code, restarting servers) wiping them all and dumping full load on the database. Fine only for tiny, constantly used, staleness-immune data.
- **"Cache it forever."** Excellent hit ratio, but Friday's price change never appears. A cache with no way to stay correct gives *fast old data*.

You need one shared fast copy that fills on demand, plus a plan for keeping it correct.

How it works

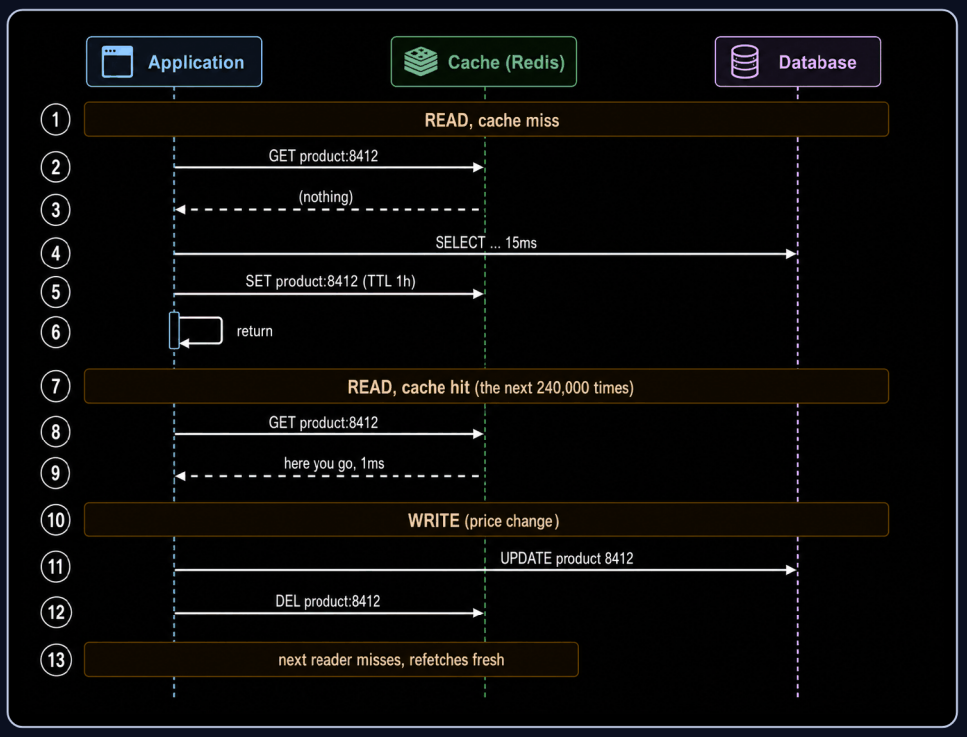


Figure — Cache-Aside (course diagram)

The figure's three phases: a miss (`GET product:8412` empty → `SELECT` , 15 ms → `SET` with a 1-hour TTL; `SELECT` reads rows in SQL, the relational query language, and `UPDATE` changes them), ~240,000 hits at 1 ms, and a write (`UPDATE product 8412` , then `DEL product:8412`).

The cache is deliberately simple. **Redis** and **Memcached** (in-memory key-value servers) just answer "here it is" or "not here"; every decision lives in application code — the pattern's flexibility and its maintenance cost.

1. **Read: check the cache;** on a hit, return.
2. **Read miss: fetch from the database.**
3. **Save a copy with a TTL, then return.** This is **lazy loading**: only requested data is cached. 8412 is cached *because* it's read 4,000 times a minute, while the **long tail** (many rarely read items) uses no memory, so the cache fills naturally with **hot data** (most-read data). Downsides: every item's first reader takes the slow path, and many simultaneous readers of the same missing item cause a stampede (later lesson).
4. **Write: update the database, then delete the key.** Don't update the copy in place: deleting is safe and lazy (the next reader fetches truth), while updating opens races where an older value overwrites a newer one (comparison below).
5. **TTL on everything, even with invalidation.** Delete-on-write works only if *every* code path remembers; eventually a bulk import, manual fix or rushed admin tool won't. The TTL caps how long such a mistake lasts. **Invalidation gives freshness; the TTL gives insurance.** Important data never relies on TTL alone; no data relies on invalidation alone.
6. **Eviction handles overflow:** LRU drops the least recently used item when full; an evicted key just becomes a harmless later miss.

Walkthrough with numbers

Redis cache-aside, 1-hour TTL, delete on every write:

- **First read of 8412 misses:** 1 ms check + 15 ms database + 1 ms SET = **17 ms** — slightly *worse* than no cache, the honest cost of a miss.
- **Next ~240,000 reads that hour: 1 ms** each. Catalog hit ratio settles at **~99.2%**: average = 0.992 × 1 + 0.008 × 17 ≈ **1.1 ms** (from 15 ms).
- **Database read load: 12,000/s → ~150/s** (the real misses); **six of eight replicas** are retired by quarter's end. RAM (fast working memory, where caches keep data) is cheap; database machines

aren't.

- **A price change on 8412:** `UPDATE`, then `DEL`; the next reader refills. Staleness lasts **milliseconds**, versus up to an **hour** with TTL alone.
- **Forgotten-delete drill:** three months later a bulk import updates **2,000 products** without invalidating; they're stale for up to one TTL, then self-heal. Annoying, bounded, survivable — exactly the TTL's job.

Delete vs. update on write (the comparison)

You changed a price in the database; what happens to the cached entry? A frequent interview question.

	DEL the key	SET the new value
Next read	One extra miss (fetch again from the DB)	An immediate hit that is already fresh
Race safety	Safe: the true value is fetched after all writes have settled	Risky: two writes at once can SET in the wrong order and leave the older value cached until the TTL
Wasted work	None, if nobody reads it again	Stores values that might never be read
Complexity	One line that's hard to get wrong	You must serialize (force one-at-a-time) or version updates to stay correct

The race: write A sets $45 and write B sets $40 almost simultaneously, and A commits *last*, so the truth is $45 — but the cache updates arrive reversed, B's `SET 40` after A's `SET 45`. Cache says $40, database $45, wrong for up to a full TTL. With `DEL`, the worst ordering is two deletes and the next reader fetches $45. One extra miss eliminates a whole class of wrong answers.

DEL has one rare race left: a slow reader that fetched the *old* value before the write can put it back right after the delete. The short TTL bounds that too — another reason to keep it.

Trade-offs

You gain:

- Big read relief: **~two orders of magnitude** (~100×) less database read load for one small service and a few GB of RAM.
- Felt latency: **15 ms → 1 ms** on the hottest pages.
- Full control: your code chooses per data type what to cache (rows, pages, computed results), how to key it, how long to trust it.
- Resilience: if Redis dies every read misses — slow but *correct*, since the database is still the truth (whether the database survives the sudden load is the stampede lesson's topic).

You pay:

- **Managed staleness:** between a change and its delete or expiry, readers see the old value — bounded, but chosen per data type.
- **The consistency chore:** every current *and future* write path must invalidate; one that forgets gives instant admin edits but stale bulk imports (self-check Q3).
- **Boilerplate:** check → fetch → set → return in every data-access function, so teams extract helpers — which the next lesson turns into a pattern.
- **Costlier misses:** cache check *plus* database. Fine at 99% hits; at **~60%** you add a hop and save little.
- **Cold starts and hot expiries** send crowds to the database (stampede lesson).

When not to use it

- **Little repetition:** data read about once per write can't produce useful hits (per-user notifications, one-time tokens).
- **Zero staleness tolerance:** the last unit of stock at checkout, a trade's execution price. Read the source of truth (browsing beforehand can usually tolerate seconds).
- **The database's memory already has it:** small data sets often live in the database's **buffer cache**; a 15 ms query on in-memory data is probably missing an **index**. Fix the query first — a cache in front of a bad query still pays full cost on every miss.
- **Data that is state, not a copy:** sessions and carts need a durable primary home, or eviction makes carts vanish.
- **Common misuse:** caching everything by habit with a 24-hour TTL, no invalidation and no hit-ratio monitoring — stale and mediocre, with nobody measuring.

Real-world examples

- **Redis and Memcached** — the industry standard; "**look-aside caching**" is another name for the pattern.
- **Facebook's memcache paper** — cache-aside at huge scale, including fixes for this module's races and surges.
- **Framework helpers** — Rails' `Rails.cache.fetch`, Django's cache API and Spring's `@Cacheable` (popular Ruby, Python and Java web frameworks) implement cache-aside for you.
- **Your own hardware** — the database buffer pool, the OS **page cache** (recently read disk blocks in RAM) and CPU **L1/L2/L3** caches (tiny, very fast on-processor memories) all check a fast copy, fetch from slower storage on a miss, and keep the result. One of computing's most common patterns.
- **For AI engineers:** exact-match caching suits embeddings of recurring strings (search queries, product titles), **moderation** decisions (harmful-content checks) and serving-time **feature vectors** (a model's input numbers; feature stores' online layers often use it); each cached embedding saves a paid API call. LLM answers are harder — users rarely repeat questions byte for byte — so a **semantic cache** (later) matches by meaning.

In the interview

Slow product page or database buried in reads: cache-aside is usually your first move. The real conversation is where its correctness ends — consistency, TTL choice, behavior after a restart.

30-second answer: The app checks Redis and serves hits in a millisecond; on a miss it reads the database and fills the cache with a TTL. On writes I update the database and *delete* the key rather than updating it, because concurrent updates can race and strand an older value, while a delete makes the next reader fetch truth. Every entry keeps a TTL as insurance against write paths that forget to invalidate, sized per data type — an hour for descriptions, minutes for prices, seconds or no cache for inventory. And I track hit ratio; below the high nineties the cache barely earns its keep.

Likely follow-ups (with answers):

- *"Why delete instead of updating the cache with the new value?"* — Near-simultaneous writes can reach the cache in a different order than they committed, stranding the older value until expiry. Deleting reduces the worst case to one extra miss; the one rare remaining race (a reader re-inserting an old value) is bounded by the TTL.
- *"What TTL would you pick?"* — No global value; it depends on each type's cost of staleness. Descriptions ~1 hour; prices minutes, with invalidation doing the real work; inventory seconds or uncached, with the purchase checking the database.

Mistake that fails candidates: updating the cache on write without knowing the race; caching without *both* a TTL and an invalidation plan; or caching a **900 ms** query instead of finding the missing index.

Self-check

- **Q1. Walk the miss path and the write path in order, and say who does each step.** → Miss: the *application* checks the cache, queries the database, SETs with a TTL, returns. Write: the *application* updates the database, then deletes the key; the next reader takes the miss path. The cache never talks to the database — that's "aside."
- **Q2. Pick TTLs (and invalidation) for product descriptions, prices and live inventory, justified by the cost of being wrong.** → Descriptions: 1-hour TTL + invalidate on edit (staleness is cheap). Prices: invalidation as the main mechanism + 5-minute TTL backstop (wrong prices cost revenue and trust). Displayed inventory: 5–30 s TTL, no invalidation, but the purchase reads the database. Each TTL reflects the cost of being wrong.
- **Q3. Admin-panel edits show up immediately, but nightly bulk-import changes take "up to an hour, randomly". No errors. Diagnose, fix, and name what prevented worse.** → The admin panel deletes keys; the import writes the database without invalidating, so each stale copy lives until its own TTL ("random, up to an hour"). Fix: have the import delete affected keys (or bump a **version** in the keys); better, route all writes through one data-access layer so no future path can forget. The TTL kept 2,000 products from staying stale forever.

Related patterns

Read-Through (fetch-and-fill moved into the cache) · Cache Stampede Prevention (many misses on one hot key) · Consistent Hashing (spreads keys, prevents resize miss storms) · CQRS (a cache is a small, short-lived read model).

TL;DR

- **Problem:** the database answers the same unchanged questions thousands of times a minute, paying database cost for repetition.
- **Mechanism:** check the cache; on a miss read the DB and store with a TTL; on a write update the DB and delete the key; TTLs everywhere as insurance; LRU for overflow.
- **Costs:** bounded staleness, a permanent invalidation chore on every write path, boilerplate, double-cost misses, crowds on cold starts and hot expiries.
- **Skip it when:** data rarely repeats, staleness is intolerable (read the truth), or the slow query just needs an index.

Read-Through

Original lesson: Read-Through on DesignGurus ↗

What it is. The application only calls `get(key)` on the cache. On a miss, the **cache itself** fetches from the database via a configured **loader** (a function that knows how to fetch, e.g. a product by key), saves a copy and returns it. Cache-aside with the fetching moved out of your code into the cache layer.

Analogy. Instead of every employee phoning the warehouse when the shelf is empty, one shelf-stocker fetches on request — and if twenty people ask at once, makes just *one* trip.

The problem it solves

Cache-aside repeats check → fetch → set → return in every data-access function. Across many functions and services, some skip a step (the SET), some behave slightly differently, some use mismatched keys.

And because each caller runs its own miss logic, callers can't coordinate: on a hot missing key, each goes to the database separately.

How it works

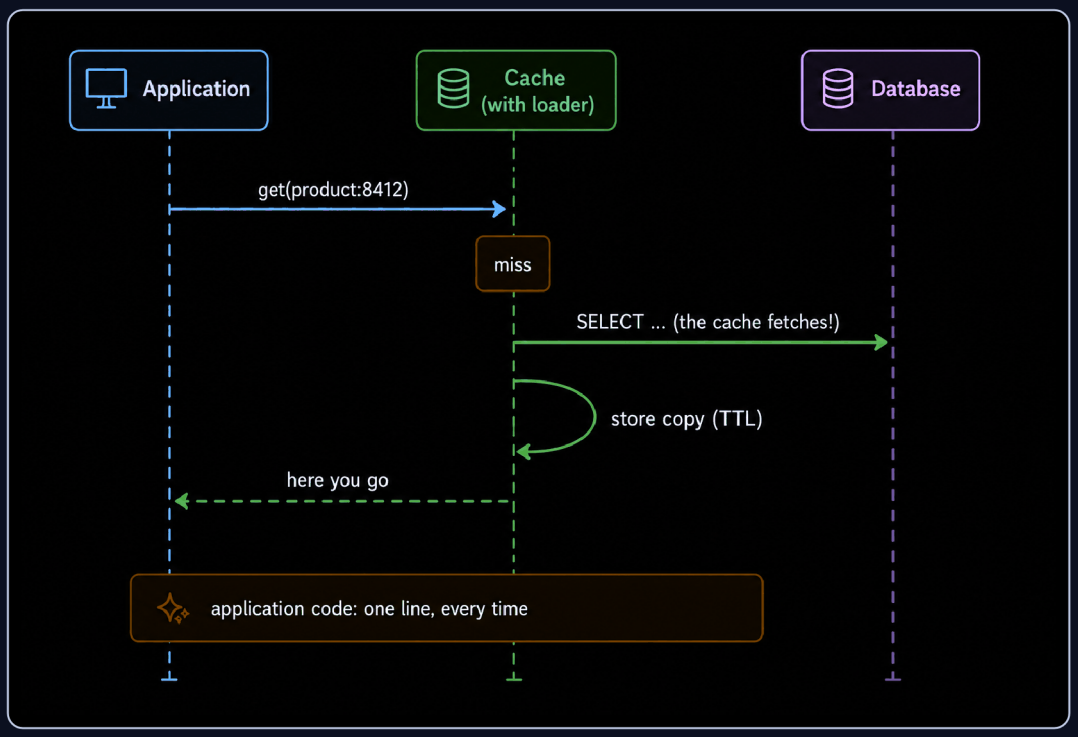


Figure — Read-Through (course diagram)

Application code is **one line, every time**: `get(product:8412)`; on a miss the cache's loader runs the `SELECT` itself.

1. **The app calls** `get(key)`; no cache-checking or database logic in app code.
2. **Hit:** the cache returns the value.
3. **Miss:** the cache runs its loader — fetch, save with TTL, return. Same miss steps as cache-aside, different owner.
4. **Request coalescing comes free**, since the cache sees every reader. Under cache-aside, **5,000** simultaneous misses on one key are 5,000 database queries; a loading cache lets the *first* caller fetch while the other **4,999** wait briefly for the same answer — **one** query. This is the first defense against "caches fail as crowds," developed fully in the stampede lesson.
5. **Writes aren't handled.** Changes still need delete-on-write plus TTLs, or pairing with write-through (next).

Where you meet it: loading-cache libraries like **Caffeine** and **Guava's** `LoadingCache` (Java), **ORM** second-level caches like **Hibernate**'s (an ORM, object-relational mapper, maps database tables to code objects), and managed front ends like **DynamoDB DAX**, a caching service that knows how to reach the table behind it.

Read-through vs. cache-aside (the comparison)

	Cache-Aside	Read-Through
Who fetches on a miss	Your application code	The cache layer, using its loader
Miss logic lives	Inside every data-access function	In one configured place
Concurrent misses on one key	N separate database queries	Can be merged into 1 (the main benefit)
Can cache computed, multi-source results	Yes, since the app can cache anything	Awkward, since a loader fetches one kind of thing by key
Cache infrastructure	Any simple cache works (Memcached)	Needs a smart cache: a library, an ORM layer, or a service
Consistency across teams/services	Every team rebuilds the procedure	One implementation, used the same way everywhere

Trade-offs

You gain:

- One miss-path implementation: no forgotten SETs, no forty variants.
- Request coalescing: simultaneous misses on a **hot key** (one many readers want at once) become one query.
- Cleaner code: data access states *what* (`get`), not *how*.

You pay:

- **Flexibility:** a loader fetches one object type by key; a dashboard or fragment from three sources doesn't fit (cache-aside can cache anything the app computes).
- **Smarter infrastructure:** the cache must run fetch logic with database access — an **in-process** library (running inside your application), an ORM or a caching service. Plain Memcached can't.
- **Coupling:** queries and access rules now live in the cache layer, so **schema** changes (database table-structure changes) can spill into it.
- **Writes are still yours:** same invalidation duties and TTL insurance as cache-aside.

When not to use it

- **Computed and composite results:** the app knows how to combine sources; a per-key loader doesn't. Use cache-aside.
- **Your cache is meant to be a simple shared store:** a shared Memcached fleet has nowhere to run a loader, and adding that may cost more than the boilerplate saved.
- **One small codebase:** a well-built `fetch_with_cache()` helper gets most of the benefit.
- **Where it shines:** many services, teams and languages reading the same data, where separate implementations routinely produce forgotten SETs, mismatched TTLs and misaligned keys. One shared read-through layer prevents all of it by design.

In the interview

Often asked directly: "What's the difference between cache-aside and read-through?" Explain *who owns what* and what follows, not just two diagrams.

> **30-second answer:** Same miss path, different owner. In cache-aside my app checks the cache and does fetch-and-fill itself — maximally flexible, but repeated everywhere, and someone forgets a step. In read-through the app talks only to the cache, which fetches through a configured loader: one implementation, and because it sees every reader, it can merge simultaneous misses

on a hot key into one query. Trade-off: loaders fetch one thing by key, so computed results stay with cache-aside, and either way writes need their own invalidation plan.

Likely follow-ups (with answers):

- *"Which for a product catalog read by forty **microservices** (small, separately deployed services)?"* — Read-through as a shared caching service or layer: one loader, one TTL policy, uniform behavior and coalescing, instead of forty slightly different procedures.
- *"Does read-through fix the stale-after-bulk-import bug from cache-aside?"* — No; that failure is on the *write* path, and read-through controls only reads. It still needs invalidation or a write-side pattern.

Mistake that fails candidates: treating them as rivals with different results, or claiming read-through solves invalidation. Their successful read behavior is identical; only ownership and its consequences differ.

Self-check

- **Q1. Who fetches after a miss in each pattern, and what new capability does centralizing it enable?** → Cache-aside: the application. Read-through: the loader in the cache layer. Centralizing enables coordination (merging simultaneous requests for one key into one query) and one uniform implementation.
- **Q2. Cache (a) product records read by 40 services in 6 languages, and (b) a per-seller dashboard computed from three databases. Which pattern for each?** → (a) Read-through as a shared layer: one loader, one policy, coalescing protecting the shared database. (b) Cache-aside: the app must combine three sources before caching the result with a TTL and invalidation.
- **Q3. A popular key expires at peak and 5,000 requests miss within 50 ms under plain cache-aside. Database load? With coalescing? What lesson does this lead to?** → Cache-aside: **5,000 identical queries**. Coalescing: **one**, with 4,999 brief waits — a spike becomes one normal query. Caches fail as crowds, which Cache Stampede Prevention handles at larger scale.

Related patterns

Cache-Aside (same behavior, different ownership; its TTL, delete-on-write and hit-ratio rules still hold) · Write-Through (the write-side partner) · Cache Stampede Prevention (extends coalescing).

TL;DR

- **Problem:** cache-aside's miss steps are repeated in every function and service, causing forgotten steps, inconsistency, and uncoordinated readers stampeding shared misses.
- **Mechanism:** the app only calls the cache; on a miss the cache fetches via a configured loader, saves and returns — one implementation that can merge simultaneous misses into one fetch.
- **Costs:** per-key loaders (computed results stay with cache-aside), a smart cache with database access, coupling, writes still needing invalidation.
- **Skip it when:** caching composite/computed results, your cache is a simple shared store, or a helper in one small codebase suffices.

Write-Through

Original lesson: Write-Through on DesignGurus ↗

What it is. Writes pass *through* the cache: the app sends a write to the cache layer, which writes the database **synchronously** (the caller waits), then updates its own copy; the caller hears "done" only

after both are updated. For data written this way the cache is never stale, because it took part in the write.

> **Analogy.** A secretary records your dictated change in the official register, then updates the front-desk reference card, and only then says "done."

The problem it solves

Cache-aside and read-through rely on every write path remembering to delete keys (the forgotten-delete bug) and cost an extra miss exactly when a user reads back what they saved. Write-through makes upkeep **structural**: the cache *is* the write path, so it can't forget. Paired with read-through, the cache becomes the single entry point to the data rather than a copy maintained beside the app; both need the same smart infrastructure (a loading cache that can write, an ORM layer, or a DAX-like service).

How it works

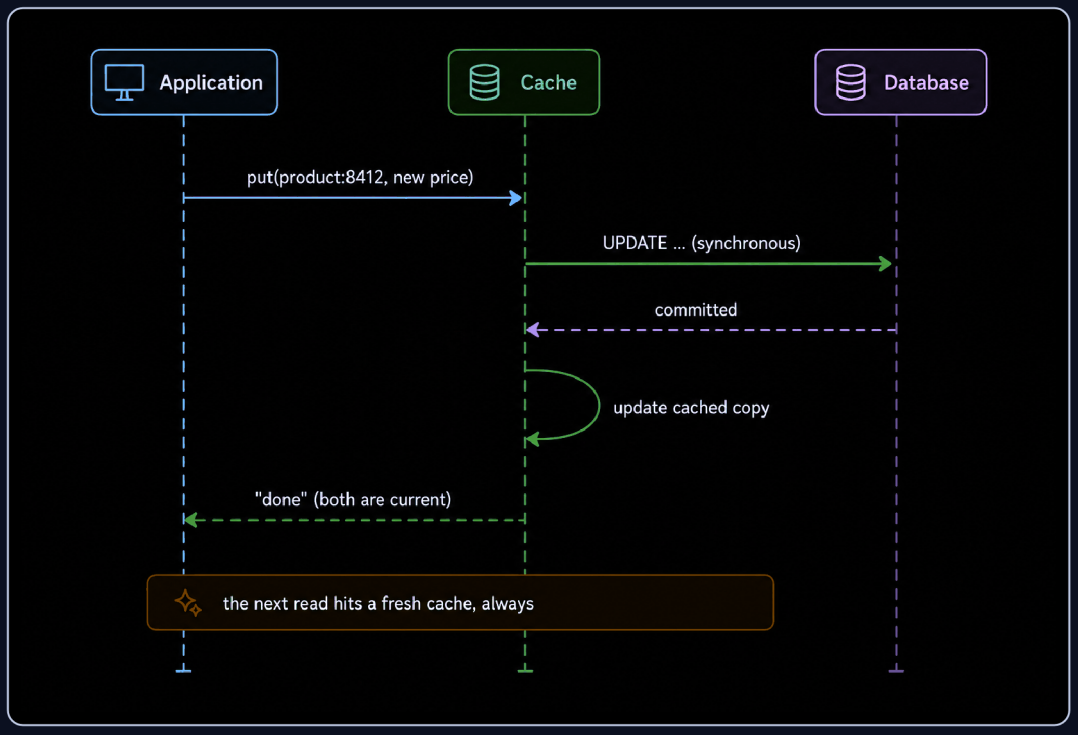


Figure — Write-Through Cache (course diagram)

1. **The app sends the write to the cache layer**, not the database.
2. **The cache writes the database first, synchronously.** The database is the truth, so it must succeed before the copy claims anything.
3. **Database write fails → cache stores nothing and returns the error.** The copy never shows a value the original lacks.
4. **Database write succeeds → cache updates its copy, then confirms.**
5. **Database succeeds but cache update fails → invalidate the entry and still report success.** The commit is safe; the next read misses and fetches truth — one slower, correct read.
6. **Fresh and warm:** unlike cache-aside's delete, write-through *keeps* the new value, so the read right after a user's edit hits.
7. **The guarantee has a boundary.** Only writes *through* the cache stay fresh; a direct database write from another service, a bulk import or a manual fix bypasses it and brings back the forgotten-delete problem. Either every writer uses the cache, or every entry keeps TTL insurance.

Write-through vs. delete-on-write (the comparison)

	Cache-aside + DEL	Write-Through
After a write, the cache is	Empty for that key (the next read fills it)	Already fresh
Read-after-write	One extra miss	An immediate hit
Write latency	Only the database write	Database write + cache update (a little slower)
Caches data nobody reads?	Never	Yes, since every write goes into the cache whether it's read or not
Requires	Any simple cache	A smart cache layer sitting in the write path

Trade-offs

You gain:

- **Structural freshness:** no invalidation chore or forgotten-DEL bugs for writes through the cache.
- **Read-after-write hits:** save-then-view (most user edits) is served from cache immediately.
- With read-through, **one coherent data layer** instead of a cache maintained alongside the code.

You pay:

- **Write latency:** two sequential updates before confirming; the extra millisecond or two matters on latency-critical paths.
- **Cache churn:** every written key enters the cache, including values nobody reads; in write-heavy workloads they push truly hot entries out of limited memory.
- **Boundary discipline:** writes that bypass the cache silently bring staleness back.
- **Smart infrastructure:** a basic key-value store can't run the sequence.

When not to use it

- **Write-heavy, rarely read data** — logs, metrics, audit records, clicks. Caching every record gains nothing and evicts useful entries; write straight to storage.
- **Cache-bypassing writers are a fact of life** — if three other systems update the tables directly, the freshness promise is fiction; cache-aside + TTL states the real guarantee more honestly.
- **Latency-critical writes** that can't afford the extra synchronous step (write-behind may fit, after weighing its durability cost).
- **Common misuse:** layering write-through on top of old invalidation code and removing every TTL, so nobody knows which mechanism actually keeps data correct. Pick **one** main mechanism, document it, keep TTLs as insurance.

Real-world examples

The course gives no separate list; implementations are the same smart-cache infrastructure as read-through: a **loading cache with a writer**, an **ORM layer**, or a **DAX-style** caching service in front of the database.

In the interview

Usually within a cache-aside / write-through / write-behind comparison, or "users edit a profile and must immediately see the new value" — read-after-write is its biggest strength.

> **30-second answer:** Writes go through the cache, which updates the database synchronously, then its own copy, then confirms — so it's never stale for anything written through it, and the read after a user's edit hits a warm, fresh cache. Costs: writes pay for both updates, and every

write lands in the cache whether read or not. The guarantee covers only writes through the cache, so bypassing writers still need TTL insurance. It pairs naturally with read-through, making the cache the single entry point to the data layer.

Likely follow-ups (with answers):

- *"The database write succeeds but the cache update fails. Now what?"* — Invalidate the entry and return success: the commit is safe and the next read fetches truth. One slower read beats a wrong copy.
- *"Why not always use write-through instead of cache-aside's delete?"* — It needs a smart cache in the write path, caches values that may never be read, and loses its guarantee whenever a writer bypasses it. Delete-on-write is one simple operation on basic infrastructure: stronger structure vs. simplicity and flexibility.

Mistake that fails candidates: claiming write-through removes the need for TTLs despite bypassing writers, or not knowing what happens when either update fails.

Self-check

- **Q1. Describe the write sequence including both failure cases, and state exactly which writes get the freshness guarantee.** → App → cache → database updated synchronously → copy updated → confirm. DB fails: cache nothing, return the error. DB commits but cache update fails: invalidate and confirm; the next read fetches truth. Only writes through this whole sequence are guaranteed fresh.
- **Q2. Choose behavior for (a) users editing profiles they view right away, and (b) an activity log written on every click but almost never read.** → (a) Write-through (with read-through): structural freshness and a warm read-back. (b) Don't cache the write; unread records just waste memory. If recent logs become popular, cache-aside can add them on request.
- **Q3. Six months after adopting write-through and removing TTLs, customers see stale prices only for products changed by a new bulk-pricing tool. Diagnose and fix.** → The tool writes the database directly, bypassing the cache, so those copies are never refreshed, and with TTLs gone they stay stale until evicted. Fix: route the tool through the cache's write path *and* restore TTLs. Every correctness guarantee has a boundary, and forgotten writers live outside it.

Related patterns

Read-Through (the natural partner) · Write-Behind (same entry point, delayed database update: durability traded for speed) · Cache-Aside (the simpler option for basic infrastructure and independent writers).

TL;DR

- **Problem:** delete-on-write relies on every write path remembering, and costs a miss exactly when users read back what they saved.
- **Mechanism:** writes go through the cache — database synchronously first, then the copy, then confirmation; on partial failure, invalidate and stay correct.
- **Costs:** extra write latency, churn from unread writes, smart infrastructure in the write path, a guarantee ending at the cache's boundary.
- **Skip it when:** writes far exceed reads, cache-bypassing writers are unavoidable, or write latency is precious.

Write-Behind

Original lesson: Write-Behind on DesignGurus ↗

What it is. Writes land in the cache and the caller gets an **immediate acknowledgment**; the database finds out *later*, when a background **flusher** sweeps changed entries every few seconds and writes them in efficient batches, folding many small changes into a few big ones. Also called *write-back*.

> **Analogy.** A shop's tally counter clicks for every customer; every couple of seconds someone sends head office one message, "+24,000". If the power fails between messages, the clicks since the last one are lost.

The problem

Product wants "12,483 people viewed this today" on every page. Naively, `UPDATE products SET views = views + 1` per view: at peak the bestseller gets **12,000 views/s**, so **12,000 UPDATEs per second on one row**. A database guards a row being changed with a **lock**, so updates queue behind each other; latency passes **one second**, waiting connections eat capacity, and checkout slows because view counting hogs the database.

What's really required? Twelve thousand times a second the system durably records "+1" before confirming. Nobody needs each increment on disk immediately — the number changes constantly and a 2-second delay is invisible — yet each +1 pays for a lock, a write and a **WAL** (write-ahead log) entry. One summary fact every two seconds would do: "+24,000."

Why the obvious fixes fail

- **"Write-through."** Fixes staleness, but still sends *every* write synchronously to the database: 12,000 row locks/s plus a cache update. This incident is about write *volume*.
- **"Sample every 10th view."** Fine for rough analytics, but an estimate: small counts come out wrong (7 might show as 0 or 10), unacceptable once badges, rankings or payouts depend on it.
- **"Queue the writes."** A queue with a batching worker is a valid Module 2 answer and the closest alternative, but the page must *show* the current count and a queue can't serve reads. Add a cache beside it and you've built most of this pattern (comparison below).

How it works

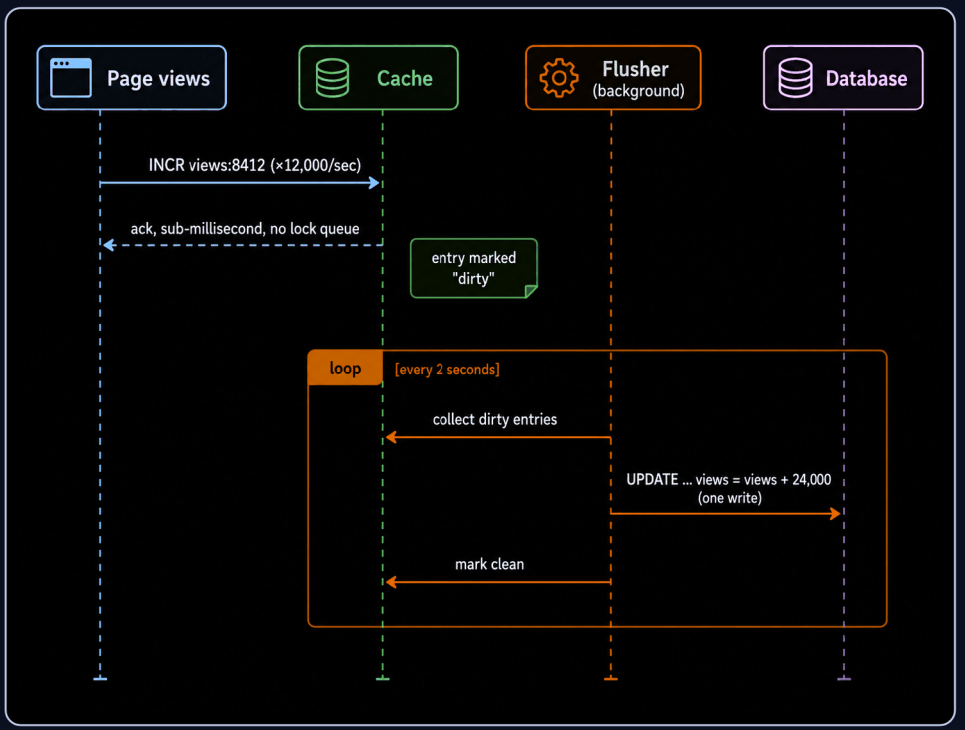


Figure — Write-Behind Cache (course diagram)

1. **The write lands in the cache and is confirmed immediately.**
2. **Dirty tracking.** Entries with changes not yet in the database are **dirty**, and must **never be evicted** before flushing — eviction would delete confirmed data, not just free memory. Implementations **pin** dirty entries (mark them non-evictable) until clean.

3. **The flusher coalesces. Coalescing** merges many changes to one key into one database write: increments are summed (12,000 × "+1" → "+24,000" per flush); repeated field updates keep only the last (five product edits in two seconds → one write of the final state). Database work grows with **distinct keys changed**, not total write rate.
4. **Flush policy:** every N seconds *or* every N dirty entries, whichever first. Shorter intervals risk less data but batch less; longer is more efficient but riskier.
5. **Flush failures need a plan.** If the database is briefly down, the flusher keeps the dirty batch and retries; an entry failing repeatedly goes to **quarantine** (a side area like a dead-letter queue) so one bad record can't block every later flush.

The cost, plainly — the most dangerous pattern in the module:

- **"Acknowledged" no longer means "saved."** Until the next flush, confirmed changes live only in cache memory. The WAL promise (fsync — force to disk — *before* acknowledging) is deliberately given up for speed and lower database load; a cache node failure can lose a flush interval of dirty changes. Every use must complete, in writing: **"if we lose the last N seconds of these writes, then ___"**, with a truly acceptable answer.
- **The truth flips.** During the dirty interval the *cache* holds the current value and the *database* an older one. Anyone needing the live value must read the cache; reports or services reading the database see the past. For that stretch the cache is the authority, not a disposable copy.

Walkthrough with numbers

The view counter with a **2-second** flush:

- **Write path:** `INCR views:8412` in Redis takes **under 1 ms** — no lock, no queue. The page reads the same key, so the count is always current; 12,000 in-memory increments/s is easy for Redis.
- **Database load:** one `UPDATE … + 24,000` every 2 s for the hot key: **12,000 writes/s → 0.5/s**, a **24,000×** reduction. Catalog-wide, each flush folds thousands of dirty keys into a handful of bulk statements, returning capacity to checkout.
- **Crash drill:** a node dies **1.8 s** after its last flush, losing **~21,600** views. The loss sentence ("if we lose 2 seconds of view counts, the numbers are imperceptibly lower") was written and accepted *before* launch, so this is fine.
- **Shrinking the window if needed:** a **Redis replica** survives one node failure; Redis **AOF** (append-only file — essentially the cache's own write-ahead log) persists to disk; a shorter interval approaches write-through. Each buys durability with speed.
- **The proposal to reject in review:** this design for payment records. "If we lose the last 2 seconds of payments, then…" has no safe ending. Money keeps the WAL's promise.

Write-behind vs. queue + batching worker (the comparison)

	Write-Behind	Queue + batching worker
Ack comes from	Cache memory (fast, but volatile: lost if power or the process dies)	The broker, once it has persisted the message
Durability of acknowledged writes	Whatever is in the flush window is at risk	Durable in the broker: nothing confirmed is lost
Coalescing (+1 merging)	Happens naturally, in place	Can be done in the worker, but it's clumsier
Live reads of the current value	The cache serves them, which is the whole point	Not possible; readers see the database's lag
Machinery	Dirty tracking inside the cache	A topic, a consumer and offsets (each reader's saved position in the stream; the usual Module 2 kit)

Deciding question: does any reader need the hot, current value? The product page shows the live count, so write-behind wins: the cache is both write buffer and read layer. For pure ingestion nobody reads immediately (analytics events, telemetry), use a queue and worker — the broker keeps every confirmed

write. Adding a read cache to the queue, or a persistent log to the cache, pushes each design toward the other; choose by the main workload's built-in strengths.

Trade-offs

You gain:

- Cache-speed write confirmations; row **contention** (many writers fighting over one row) vanishes at its source.
- Orders of magnitude fewer database writes via batching and coalescing; load grows with distinct keys, not write rate.
- Natural burst absorption: spikes accumulate as dirty entries and drain at the database's pace (the queue lesson's shape).

You pay:

- **The durability window:** acknowledged writes sit in memory until flushed and a crash loses them — the main cost, with its consequence written down per deployment.
- **Truth inversion:** direct database readers silently see older values during the dirty interval.
- **Real complexity:** dirty tracking, eviction pinning, retries, quarantine and per-key ordering — the module's heaviest machinery.
- **Harder debugging:** "the value at 14:02" has two answers (cache and database) to reconcile.

When not to use it

- **Money, ledgers, payments, orders, compliance data:** any loss window rules it out regardless of **throughput** (operations per second). Individually important writes keep the WAL's promise.
- **Other systems read the database directly** and need current values: truth inversion breaks them *without any error*.
- **Writes that don't coalesce:** unique records (new orders, messages) batch but don't get a hot key's huge reduction; a durable queue batches them without risking confirmed data.
- **Modest volume:** a counter at **50 updates/s** doesn't justify dirty tracking and flushers.
- **Judgment calls:** carts → a durable store; inventory → buffer only the displayed badge; token metering → buffer dashboards, bill from a durable stream (see self-check Q2, the inventory follow-up, and the AI example).
- **Accidental adoption:** some cache products and ORMs expose write-behind as a switch; enabling it for speed without a documented loss window just postpones the decision until a failure makes it.

Real-world examples

- **CPU write-back caches** — the name comes from hardware; processors have made this cache-vs-RAM trade for decades.
- **Social counters at scale** — views, likes and impressions buffered in caches and flushed as totals; a row lock per reaction would be absurd.
- **Rate limiters and leaderboards** — Redis counters serving live reads, persisted later.
- **EhCache, Hazelcast** (Java caching products) — built-in write-behind; the loss-window rule still applies despite the easy switch.
- **For AI engineers:** training metrics and telemetry fit well (losing ~2 s of **loss-curve** points — training error plotted over time — is fine), and online feature stores can buffer updates the same way. Token metering is counter-shaped but is billing: buffer dashboards aggressively, compute invoices from a durable, reconciled event stream. Dashboards can lose a moment; invoices can't.

In the interview

Typical prompt: a like button or view counter at scale — diagnose **row contention** first, then propose this. Then: "what happens when a cache node dies?"

30-second answer: Twelve thousand increments a second on one row is lock contention, so I buffer in the cache: increments land in Redis in under a millisecond, the page reads the same key so it's always current, and a background flusher merges them into one bulk update every couple of seconds, cutting database writes by orders of magnitude. The price: confirmed writes live in cache memory until the flush, so a crash loses that window — acceptable and documented for view counts, never for money — and during the window the cache is the truth, so direct database readers see the past. If nobody needed the live value, I'd use a durable queue with a batching worker.

Likely follow-ups (with answers):

- *"The cache node dies. What exactly is lost, and how would you shrink it?"* — Up to one flush interval of confirmed dirty changes. Shrink with a cache replica (survives one node), AOF persistence (with its own fsync policy), or a shorter interval — each trading performance for durability, moving toward write-through.
- *"Would you use this for inventory during the flash sale?"* — Separate display from decision: "only 3 left" can be seconds stale or buffered, but the decrement accepting a sale needs a durable, authoritative path.

Mistake that fails candidates: write-behind for payments or orders; ignoring dirty-entry eviction risk; or claiming Redis persistence removes all loss, when AOF's default fsync policy `everysec` still allows ~**one second** of loss.

Self-check

- **Q1. Describe the full write path, define "dirty", and say exactly what the acknowledgment promises.** → The cache accepts and confirms immediately, marks the entry dirty (changed but not yet in the database) and pins it against eviction; the flusher periodically merges dirty changes, writes them, marks them clean. The ack promises only that the value is in cache memory and *should* reach the database — not that it's durably saved.
- **Q2. For view counts, cart contents, payment records and a "trending now" score: write-behind or not, and complete the loss sentence.** → View counts: yes, "…the numbers are imperceptibly lower." Trending score: yes — approximate, derived, recomputable. Carts: no, durable store; losing choices is visible harm and volume rarely warrants it. Payments: never; no loss window is acceptable. Fit depends on what losing the dirty window would cause.
- **Q3. A nightly analytics job reading the database reports view counts ~2% below the cache-backed dashboard, and counts dip slightly after every deploy. Explain and fix proportionately.** → The 2% gap is truth inversion working as designed (the database trails dirty counts): have analytics read post-flush data labeled "as of" a time, or accept and label the gap. Post-deploy dips come from restarting nodes dropping unflushed changes: **flush dirty entries on graceful shutdown** (a planned stop that finishes work before exiting), plus a replica if the rest matters. Persisting every write would be overkill — essentially rebuilding write-through.

Related patterns

Write-Through (the safe alternative) · Message Queue (the durable alternative for unread write streams) · Write-Ahead Log (the promise write-behind weakens; AOF partly rebuilds it) · Read-Through (with write-behind, the cache becomes the single entry point).

TL;DR

- **Problem:** frequent writes to hot keys (counters, scores) turn rows into lock queues — thousands of individually unimportant facts per second, each paying for a durable write.

- **Mechanism:** writes land in the cache and are confirmed instantly; dirty entries are pinned and flushed in merged batches every few seconds, so load tracks distinct keys, not write rate.
- **Costs:** a durability window (acknowledged ≠ saved), truth inversion, the module's heaviest machinery.
- **Skip it when:** writes involve money or matter individually, others read the DB directly, writes don't coalesce, or volume is modest. Always finish "if we lose the last N seconds, then ___", and mean it.

Cache Stampede Prevention

Original lesson: [Cache Stampede Prevention on DesignGurus ↗]

What it is. Techniques for when a popular ("hot") entry must be rebuilt, ensuring **exactly one request rebuilds it** while others wait briefly or get the slightly old copy. Without them, thousands of simultaneous misses hit the database: a **cache stampede** (or **dog-piling**).

Analogy. When the office coffee pot runs dry, one person brews; everyone else waits a minute or drinks the last lukewarm cup — not forty people brewing at once.

The problem

In a flash sale the bestseller's page serves **12,000 requests/s** from cache at **99.9%** hits; the database sees **150 queries/s**. At **8:00:00 PM** the entry hits its **one-hour TTL** — routine. The next 50 milliseconds:

1. Rebuilding takes one ~**50 ms** query.
2. Every request in those 50 ms also misses: **12,000 × 0.05 = 600 simultaneous misses**.
3. Under plain cache-aside each runs its own miss path: **600 identical queries** on a database used to 150/s.
4. The burst slows the query to **500 ms**, *stretching* the miss window while traffic continues at 12,000/s: now **6,000** requests are missing.
5. The database's **connection pool** (its limited set of open connections) fills, queries queue, checkout (sharing the database) stops. By **8:00:03** one ordinary expiry has taken the site down.

Damage = **request rate × rebuild time**, and overload lengthens the rebuild, enlarging the crowd: a **feedback loop** — the module's second fact in action. A cache cluster restarting **empty** at peak runs this math on *every* key at once.

Why the obvious fixes fail

- **"Longer TTLs."** A 6-hour TTL makes stampedes rarer, not smaller, and keeps data stale longer.
- **"Size the database for the burst."** Provisioning for 6,000 simultaneous queries is a huge permanent cost for a **50 ms** event a few times a day, and the burst grows with traffic.
- **"Never expire hot keys — refresh everything from a background job."** Works for the hottest keys, but recomputing the whole catalog wastes resources on millions of unread entries. **Refresh-ahead** pays only for the small set whose traffic justifies it.

Misses aren't the problem; **600 requests repeating the same work** is. The fix is coordination — read-through's **request coalescing** (many identical requests → one operation), fully developed.

How it works

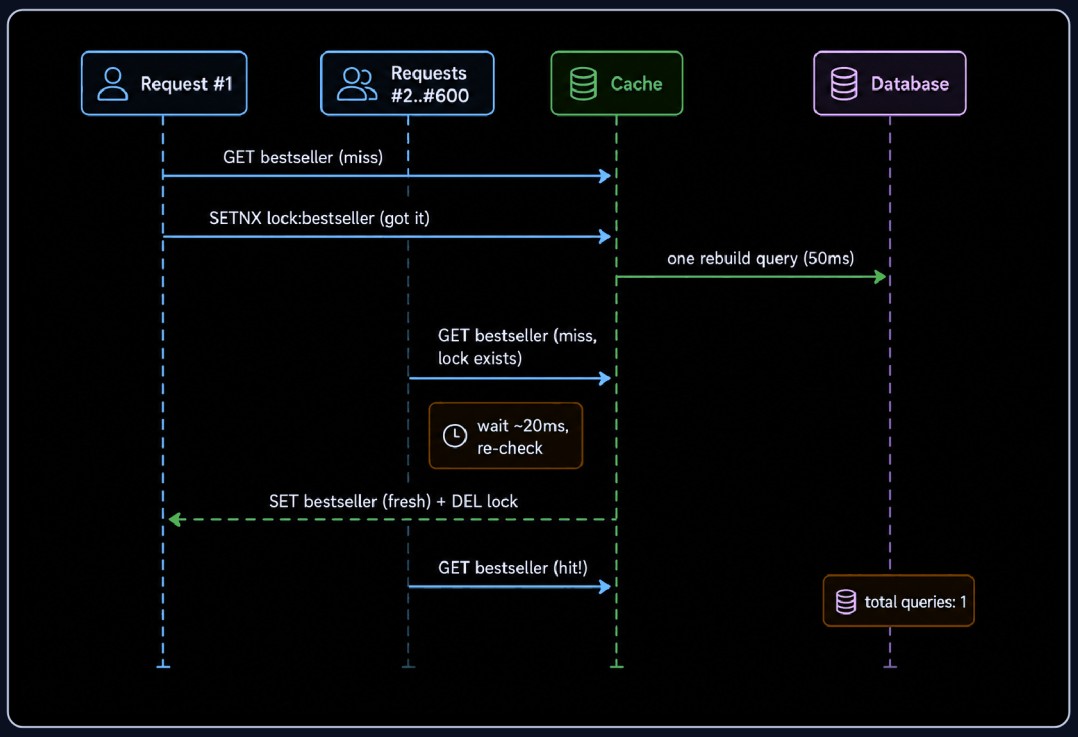


Figure — Cache Stampede Prevention (course diagram)

The figure shows the lock tool: request #1 takes `SETNX lock:bestseller` and rebuilds; requests #2..#600 wait ~20 ms and re-check. Database queries: **1**.

Three tools protect a single hot key; two protect the fleet. Production combines them.

Tool 1 — The lock ("single flight"). One rebuilder per key at a time.

1. The first missing request takes a short per-key lock, atomically in Redis with `SETNX` ("set if not exists"), so only one caller wins.
2. It fetches from the database and fills the cache.
3. Others see the lock, wait briefly, and re-check the cache once the value is there. Result: 600 misses → **1 query + 599 short waits**. The lock needs a **short TTL of its own**, since its holder might crash; a never-expiring lock is an outage by itself.

Tool 2 — Serve stale, refresh behind. If slightly old data is fine, nobody waits: keep entries a little past their *logical* expiry, serve the stale copy immediately, and let **one** lock-protected background refresh fetch the new value. Users get seconds-old data at full speed; the database gets one query. (A product page 50 ms stale looks identical to a fresh one.) HTTP standardizes this with the **`stale-while-revalidate` header** (a name-value metadata line on an HTTP request or response).

Tool 3 — Probabilistic early refresh. Keep the hottest keys from ever expiring: near the TTL, each request has a small random chance of refreshing early, rising as expiry nears. A key with thousands of readers almost always refreshes in time; a cold key just expires quietly. The randomness automatically aims refresh work at keys whose traffic justifies it.

Fleet tool 1 — TTL jitter. Mass expiry is a stampede spread over *many* keys: a deploy warms **10,000** entries within a minute, all TTL **3600**, so all expire together an hour later. Randomize, e.g. **3600 ± 10%**, to spread expirations over many minutes. That randomness is **jitter**.

Fleet tool 2 — A cold-start plan. An empty cluster at peak means 100% misses at full traffic. Before admitting traffic:

1. **Warm** (pre-load) the most popular keys from an **access log** (record of requested keys) or a snapshot (fewer immediate misses).
2. **Ramp traffic up slowly** (remaining load stays within database capacity).
3. Rely on **single flight** to cap database work at **one query per distinct key** — a measurable upper limit instead of an uncontrolled crowd. Also route the cache cluster with **consistent hashing** so a normal resize doesn't cause mass misses.

Walkthrough with numbers

8:00:00 PM again, with serve-stale + a per-key lock:

- The bestseller hits logical expiry; the next request finds it stale, takes the lock, starts the rebuild and serves the stale copy in **1 ms**, as do the other **599** requests in the 50 ms window. Nobody waits.
- **Database queries: 1** (not 600 spiraling to 6,000). **Page latency: flat 1 ms** (not seconds). **Staleness served: 50 ms**, unnoticeable.
- At **8:00:00.050** the fresh value arrives; the next hour runs like the last.
- **Cold-start drill:** a bad deploy restarts the cluster at **7:50 PM**. Warming restores the top **10,000** keys in ~**40 s**, the load balancer ramps traffic over **2 minutes**, and single flight caps work at one query per key. The database briefly peaks under **2,000 queries/s** — unpleasant, survivable, recovered within minutes, where the unplanned version snowballs into an outage.

The three hot-key tools compared

	Lock (single flight)	Serve stale + refresh	Probabilistic early refresh
During rebuild, others get	A few ms of waiting	The stale copy, right away	Nothing, because expiry rarely comes
Staleness served	None: everyone gets fresh data	One rebuild's worth	Practically none
Complexity	Lock TTLs and logic for waiters	Tracking the stale window	Probability math in the client
Wasted work	None	None	A few refreshes that happen early
Best for	Keys where staleness isn't allowed	The default for data that tolerates staleness	The very hottest keys at the largest scale

Big systems often run **all five**: serve stale for tolerant data, lock every rebuild, refresh known hottest keys early, jitter all TTLs, and keep a written cold-start runbook. Each covers a different gap; none replaces another.

Trade-offs

You gain:

- **Flat latency through expiries:** the **p99** (latency 99% of requests beat) stops spiking each TTL cycle.
- **Database load that tracks distinct keys needing rebuild**, never request rate; the crowd is capped at 1 by design.
- **Survivable cold starts and deploys** — the difference between a helpful cache and one your architecture depends on.

You pay:

- **Deliberate, bounded staleness:** serve-stale returns old data for one rebuild interval — negligible for many types, but a per-key decision.
- **Lock complexity and its own bugs:** expiry timers, crashed holders, waiter fallback. Use proven code (Go's `singleflight` package from the Go standard libraries; documented Redis lock recipes), not your own.
- **Waiting requests hold resources:** single flight *without* serve-stale makes requests wait a few ms, which matters at extreme concurrency.
- **Some wasted work:** early refresh sometimes refreshes entries that could have expired harmlessly, and the client gets more complex.

- **Justify with measurements:** simultaneous misses per key, database queries per rebuild, lock wait time, stale responses served — showing whether protections cut real load and which keys deserve early refresh.

When not to use it

- **Cold and long-tail keys:** a key read once a minute can't have many simultaneous misses; measure first.
- **Low-traffic systems:** at **50 requests/s** the largest possible crowd is invisible to the database; **distributed locking** (a lock shared across servers, usually in Redis) at **~100 req/s** guards against a crowd that never comes. (The opposite mistake — ignoring simultaneous expiry until a flash sale — is more common.)
- **Staleness truly not allowed:** no serve-stale; use the single-flight lock so callers wait briefly for fresh data, or don't cache the key (read critical values from the source, as in cache-aside).

Real-world examples

- **Facebook's memcache "leases"** (the cache-aside paper): the first missing request gets a token permitting the rebuild while others wait — single flight at huge scale.
- `stale-while-revalidate` — standardized in **RFC 5861** (RFCs are numbered internet-standard documents); **CDNs** (content delivery networks: worldwide servers caching copies near users) and browsers use it to answer immediately while refreshing in the background.
- **Go's** `singleflight` **package** — single flight within one process as a small standard building block.
- **The XFetch paper**, "Optimal Probabilistic Cache Stampede Prevention" — the math behind probabilistic early refresh.
- **For AI engineers:** a stampede costs more when the rebuild is a **billable, multi-second LLM generation** instead of a 50 ms query; one popular expiring answer can trigger dozens of identical paid generations. Single flight for identical prompts is among the most valuable inference-stack protections, alongside the later semantic cache (which also catches *similar* requests). Inference cold starts likewise need **model weights** (the learned numbers making up a trained model) and caches warmed before traffic is admitted gradually.

In the interview

After you propose caching, expect "What happens when your most popular key expires at peak?" Do the **concurrency** math (requests in flight at once) aloud, then show how coordinated rebuilds cap it.

30-second answer: A hot key expiring under load stampedes: at 12,000 requests a second with a 50 ms rebuild, 600 requests miss together and send identical queries, and the slowdown feeds itself. I stack defenses: a lock on every rebuild so exactly one request does the work; serve-stale with one background refresh for tolerant keys so latency stays flat; probabilistic early refresh so the hottest keys never expire; TTL jitter so keys warmed together don't expire together; and a cold-start plan — warm top keys, ramp traffic, let single flight cap the database at one query per distinct key.

Likely follow-ups (with answers):

- *"The request holding the rebuild lock crashes. Now what?"* — The lock's short TTL releases it automatically; waiters serve stale or retry once. A non-expiring lock would make the protection itself the outage.
- *"Your whole cache cluster restarts empty at peak. Walk me through it."* — Avoid cold restarts where possible (replicas, consistent hashing). During recovery: warm popular keys from an access log, ramp traffic slowly, and use single flight so the database sees at most one query per distinct key — the key operational guarantee.

Mistake that fails candidates: "on expiry it just goes back to the database" with no thought for simultaneous misses; proposing client **retries** (they grow the crowd); or locking every key without a measured traffic threshold.

Self-check

- **Q1. Do the incident's math: why can one key expiring at 12,000 req/s endanger the whole database, and what creates the feedback loop?** → The 50 ms rebuild means 12,000 × 0.05 = 600 identical queries; the burst slows the database and stretches the rebuild to 500 ms, making 6,000 simultaneous queries, slowing it further until shared connections run out. Damage = request rate × rebuild time, and overload lengthens rebuild time.
- **Q2. Pick a tool (or none) for a news site's headline block, a stock-price display, and a retailer's top-100 product pages.** → Headlines: serve stale while refreshing (seconds-old news is harmless, latency flat). Stock price: staleness makes it wrong — lock-guarded refresh with a short wait, if cached at all. Top 100 products: probabilistic (or scheduled) early refresh, so expiry almost never happens. The choice follows each key's traffic and staleness tolerance.
- **Q3. Every day at exactly 4:00 AM the database has a ten-minute load spike; the team deploys daily at 4:00 PM. No errors. Diagnose and fix with one line.** → The 4 PM deploy warms thousands of entries with the same **12-hour** TTL, so they all expire together at 4 AM and rebuild simultaneously. Fix: TTL jitter, `ttl = 43200 ± rand(10%)`, spreading expirations over ~an hour. Moving the deploy only moves the spike; jitter removes the synchronization.

Related patterns

Cache-Aside (whose simultaneous-miss problem this solves) · Read-Through (its coalescing is single flight in miniature) · Consistent Hashing (no mass misses on resize) · Graceful Degradation (serving stale is a reduced answer instead of failure).

TL;DR

- **Problem:** a hot key expiring under load sends every simultaneous request to rebuild it (request rate × rebuild time identical queries) in a self-feeding spiral; a cold cache does this to every key at once.
- **Mechanism:** one rebuilder per key (lock / single flight); serve stale while refreshing; probabilistic early refresh for the hottest keys; jittered TTLs; warm cold caches and ramp traffic.
- **Costs:** deliberate bounded staleness, lock TTLs and waiter logic (use libraries), brief waits, some wasted early refreshes.
- **Skip it when:** keys are cold or traffic is low. Measure simultaneous misses before adding machinery.

Capstone - Caching the Product Catalog

Original lesson: Capstone - Caching the Product Catalog on DesignGurus ↗

What this capstone is. A product page *looks* like one resource, but behind its URL are **five kinds of data**, each with its own read/write pattern and cost of being wrong, so each gets its own caching decision; then a flash sale stress-tests them together. For each decision name the **pattern** and its **correctness mechanism** (TTL, invalidation, or both), and finish **"if this data is N seconds old, then ___."** Answering that per value is the whole module in one exercise.

The brief

- **Traffic:** 12,000 requests/s at peak, **99%+ reads**.
- **Five kinds of data:** the **description** (name, images, specs), the **price**, live **inventory** ("only 3 left!"), the **view counter** ("12,483 viewed today"), and **"customers also bought"** (computed

recommendations).

- **Existing infrastructure:** Module 3's sharded, replicated database; a Redis cluster routed by consistent hashing; and a merchandising **bulk-import tool** writing straight to the database (already behind two incidents in this course).

Decision 1 — The description

The data: constantly read, edited every few weeks; staleness costs almost nothing.

Decision: cache-aside, the default. Check Redis, fetch misses, save with a **1-hour TTL**; an edit updates the database then **deletes** the key (not update — the concurrent-write race). **Why keep the TTL?** The bulk importer writes without invalidating; the TTL caps its staleness at an hour instead of forever.

Staleness sentence: "If the description is an hour old, a typo fix took an hour to appear." Acceptable.

Decision 2 — The price

The data: read like the description, but a wrong price has financial, legal and trust costs.

Decision: same pattern, stricter correctness. Cache-aside still fits the *read profile*, but **invalidation on every write becomes the main freshness mechanism** (a price change deletes its key within milliseconds), and the backup TTL shrinks from 1 hour to **5 minutes**, so the bulk-pricing tool can leave a price wrong for five minutes at most. **Checkout reads the truth:** the price saved on an order comes from the database; only *displayed* prices may be briefly stale.

The module's central principle: **the read profile chooses the cache pattern; the cost of being wrong chooses the correctness rules.**

Decision 3 — Live inventory

The data: "Only 3 left!" changes with every sale, possibly hundreds of times a second, shows on every view, and is close to money.

Decision: split the reads; keep writes on the truth.

- **The display badge** is a marketing number: **10-second TTL**, **no per-sale invalidation** (hundreds of invalidations per second would recreate the database load the cache removes, for no user benefit).
- **The checkout decision** reserving the last unit always runs a **conditional database decrement** (succeeds only if stock remains), never relying on the cache.

Staleness sentences: badge — "if 10 seconds old, a customer sometimes sees 'only 3 left' when it's 2; urgency marketing survives." Checkout — no staleness, so no cache. One number, two designs, because the reads have different consequences.

Decision 4 — The view counter

The data: 12,000 increments/s on the bestseller — exactly the Write-Behind incident, the one data type where *writes* are the problem.

Decision: write-behind. Each view is a Redis `INCR` (under 1 ms, no row locks) and the page reads the same key; every **2 seconds** the flusher merges a batch into one `+24,000`. The loss sentence is written: "if we lose the last 2 seconds of view counts, the numbers are imperceptibly lower." **Flush-on-shutdown** in the deploy process protects routine restarts. **Truth inversion** applies: database-reading analytics trail by ~2 s and must label results "as of" a time.

Decision 5 — "Customers also bought"

The data: ~400 ms to compute, combining order history from several tables; refreshed by a **nightly model run**; yesterday's result is as good as this morning's.

Decision: cache-aside, definitely *not* read-through — the Read-Through lesson's limit: a loader fetches one object by key, but this strip needs app logic combining sources. The app builds it, caches it with a **12-hour TTL**, and **invalidates affected entries when the nightly model publishes**. Warning for the next stage: a 400 ms rebuild makes this key especially exposed to simultaneous misses.

Decision 6 — The flash sale (every decision stress-tested)

The event: at 8 PM traffic **triples**, and every stampede-lesson failure appears: hot keys expiring under 12,000 req/s, co-warmed keys expiring together, and the cache cluster resizing mid-sale.

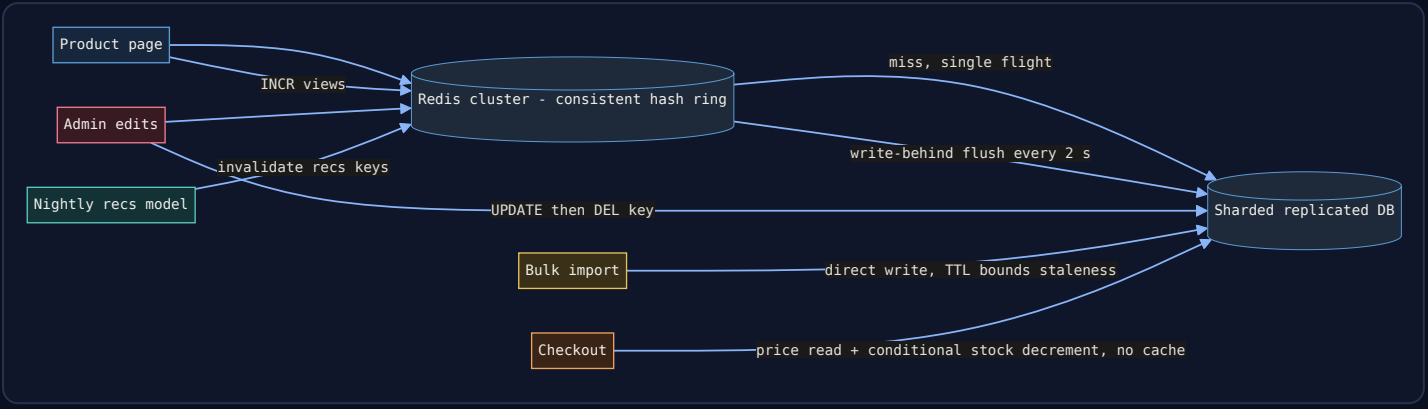
Layered defenses, each matched to its failure:

1. **Serve-stale by default** (hot-key expiry): descriptions, display prices and recommendations return stale values in 1 ms while one protected refresh runs; the 400 ms recommendation rebuild runs **once instead of 600 times**. (A display price is stale only for the rebuild, not its whole TTL; checkout still reads the database.)
2. **Single flight on every rebuild** (crowd cap): short-TTL locks; others get stale data or wait briefly; the database sees **at most one query per distinct expired key** — the limit that makes capacity planning possible.
3. **Early refresh for the top 100 products** (hottest keys): they almost always refresh *before* expiring, so the bestseller's 8 PM stampede never happens.
4. **Jitter on every TTL** (mass expiry): **±10%**, so values warmed by the **4 PM deploy** expire over several minutes, not all at 5.
5. **Consistent hashing for the Redis fleet** (cluster change): adding a node remaps **~1/N** of keys instead of **~95%**; no miss storm.
6. **A written cold-start runbook** (empty cache): warm popular keys, ramp traffic, single flight caps the database at one query per key — documented *before* it's needed.

Result: database reads stay **under 400 queries/s** despite tripled traffic, because caching was a set of separate correctness and load decisions, not one global habit.

The final architecture

Final architecture: each data type gets its own path, with checkout decisions always on the source of truth.



Data on the page	Staleness budget	Pattern + correctness	Stampede treatment
Description	~1 hour	Cache-aside; DEL on edit + 1 h TTL (insurance against bypassing writers)	Serve-stale + single flight
Price (display)	Seconds	Cache-aside; DEL on write + 5 min TTL	Serve-stale (limited to the rebuild) + single flight
Price (on the order)	Zero	No cache; read the source	n/a
Inventory badge	~10 seconds	Cache-aside; TTL only (writes happen too often to invalidate each one)	TTL jitter
Inventory decision	Zero	No cache; conditional write on the source	n/a
View counter	Live from cache	Write-behind; 2 s flush, loss sentence written down, flush-on-shutdown	n/a (it never expires; while dirty, it is the source)
Recommendations	Hours	Cache-aside (computed, so a loader can't build it); nightly invalidation	Early refresh (top 100) + single flight

How to read it: go down the "staleness budget" column — every decision starts from how old the data may be, and the patterns enforce those budgets. **Hit ratio** is the financial reason to cache; **concurrent misses** are the reliability risk. Those three questions sum up the module.

Extend it yourself (open questions to practice)

- **Multi-region:** two continents, a Redis cluster each; a price change in one region must stop being wrong in the other within seconds. Design cross-region invalidation: what's sent (deletes or values?), over what channel (hint: a Module 2 pub/sub-style channel), and what breaks when it lags (the other region serves stale prices)?
- **Personalized prices:** enterprise customers see negotiated prices, so keys become `price:{product}:{customer}` and the **key space** (number of possible distinct keys) jumps from **100K to 500 million**. What survives of the price design? Consider long-tail hit ratios (rarely read, rarely hit), memory budgets, and whether the price calculation's *inputs* are easier to cache than its result.
- **For AI engineers:** add semantic product search ("cozy reading chair") via an LLM-embedded vector query costing **~200 ms** and real money per call. Exact-match caching misses constantly because nobody phrases a query the same way twice. Sketch a design, then compare with the semantic cache lesson (merging *similar* requests is its own pattern).

Interview one-liner

Don't cache "the page". Cache each data type according to its staleness budget: cache-aside with TTL insurance for descriptions and recommendations, invalidation first for prices, a TTL-only badge plus an uncached purchase decision for inventory, write-behind for counters, and layered stampede protection so the database sees at most one query per expired key during the sale.

Module 5 — Surviving Failure (Reliability)

Module Introduction

Original lesson: Module introduction on DesignGurus ↗

What this module is about. An arrow from service A to service B can break four ways in production: B never replies (a *hang*), replies with an error, does only part of the job, or replies too late to be useful. Seven well-known patterns let you handle these outcomes deliberately instead of discovering them by accident.

Why "rare" failures are not rare. A checkout processing 100,000 orders a day where 1 call in 200 fails sees **500 failures every day**. Reliable designs plan for that measured number — and it's why interviewers keep asking, of every arrow you draw, "and what if B doesn't answer?"

Pattern picker — name the failure, then choose the tool:

What the failure looks like	Pattern
Silence; waiting callers keep stacking up	Timeout
Breaks right now, then works again a few seconds later	Retry with Backoff
You can't tell whether your payment actually went through	Idempotency
Stays broken for minutes, not just moments	Circuit Breaker
No errors anywhere, yet one workload is swallowing all the capacity	Bulkhead
A single bad message holds up every message behind it	Dead Letter Queue
All the mechanisms did their job, yet the user still landed on an error page	Graceful Degradation

Two kinds of pattern. Six are **mechanisms** deciding *when to stop* — waiting, retrying, calling, redelivering. Graceful Degradation is a **policy** deciding *what the user gets* while a part stays broken. Mechanisms alone just produce errors faster; a policy alone has nothing enforcing it.

Why the order matters. Each lesson solves what the previous one causes or reveals — timeouts expose failures, retries risk double writes, idempotency makes them safe, breakers handle sustained outages of a **dependency** (a service or system yours calls), bulkheads handle success-shaped floods, DLQs catch messages that never succeed, and degradation decides what users see (the full chain is in the capstone debrief). Short on time? Timeout, Retry, Idempotency and Circuit Breaker are the "critical four"; they build on each other, so study them together.

Four terms used everywhere in this module

- **Transient vs. sustained failure.** *Transient* clears on its own within seconds (e.g. a lost network **packet**, one small chunk of data in transit); *sustained* persists because the service is truly down or swamped. Retries fit transient, breakers sustained; the wrong tool piles load onto an outage.
- **Fail open vs. fail closed.** When a check can't run, *failing open* lets the operation proceed; *failing closed* blocks it. Nice-to-haves may fail open; security and money always fail closed — a down login service must never let everyone in.
- **At-least-once delivery.** Queues and retry loops never lose work but may deliver an item more than once; receivers must handle repeats safely.
- **Little's law.** *Work in progress = arrival rate × time per item.* 300 requests/s × 2 s = 600 in flight, so 600 threads (a thread is the unit of execution handling one request) if each holds one. Three lessons use it to predict exactly when a server runs out (Module 0).

Before you start — for a system you know ("I don't know" is a fine start; the capstone repeats these):

1. If something your service calls goes silent — no error, nothing — how long does it wait, and who chose that number?
2. If your service retries a failed payment or write, what prevents a double charge?
3. When your most important page's *least* important dependency fails, what exactly does the user see?

Timeout

Original lesson: Timeout on DesignGurus ↗

What it is. A deadline on every network call: if no reply arrives by then, the caller stops waiting, marks the call failed, and releases what it held. *Analogy:* after ten unanswered rings you hang up and call another restaurant instead of holding the phone all evening.

The problem: waiting is not free. Many clients have no deadline until you set one: Python's widely used `requests` library waits forever by default, and a raw TCP connection (the basic connection most protocols run on; see Module 0) to a silently dead machine can stay open for minutes before the OS gives up.

A waiting call holds a thread, a connection, often a lock (a marker keeping other work off the same data). At 300 requests/s with a dependency hanging 30 s per call, keeping up needs 300 × 30 = **9,000 threads**; a normal service has a few hundred, so it freezes in about **one second**. *Waiting*, not errors, killed it.

Timeouts come first for a second reason: a hung call is invisible (no error, no metric, no signal), while a timed-out call is an *event* that retries can retry and breakers can count. Without timeouts, the later patterns have nothing to see.

The figure: a timer (e.g. 300 ms) starts with the request and, on expiry, the client deliberately fails the call; a normal call returns in time. Nothing hangs forever, resources are released, and "slow" becomes a countable failure — the foundation for retries, breakers and SLOs (service-level objectives: the **latency** — response time — and **availability** — share of time working — targets a team commits to).

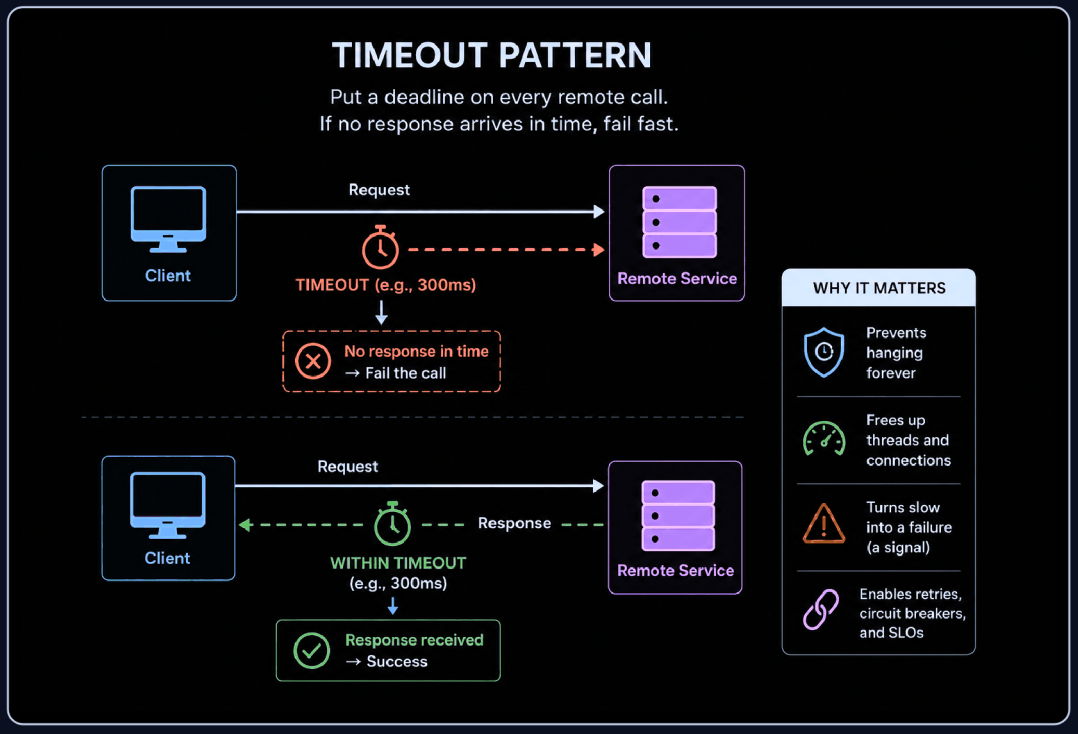


Figure — Timeout Pattern (course diagram)

How it works — three rules

1. **Measure; don't copy-paste.** The industry's most common timeout is whatever was copied — usually 30 seconds — which protects nothing, since no user is still waiting at second 29. Measure the dependency while *healthy*, typically its **p99** (the latency 99 of 100 calls beat). Healthy p99 of 120 ms → timeout **~2–3×, roughly 300–400 ms**: room for slow-but-normal calls, fast cutoff for real hangs. **One value per dependency** — an 8 ms database and a 900 ms ML model can't share a number.
2. **Connection timeout much tighter than request timeout.** "Can I reach you at all?" reveals an unreachable host within milliseconds; "can you answer?" involves real, variable work.
3. **Pass the leftover time down the chain (deadline propagation).** An API promising 2 s that makes three sequential calls, each with a fresh 2 s timeout, has a worst case of 3 × 2 = **6 seconds**. Each hop should inherit only what's *left*:
   - The **API gateway** (front-door server receiving every outside request) starts with 2,000 ms.
   - Auth (checks who the user is) spends 100 ms → 1,900 ms remain.
   - The database spends 400 ms → 1,500 ms remain.
   - Search's timeout = min(its own limit, 1,500 ms).

   gRPC (a widely used remote-call framework) carries the caller's deadline with the request automatically; plain HTTP mostly doesn't, so deep services keep working on requests that already failed upstream.

Trade-offs

- *You gain:* bounded damage (a failing dependency holds threads only for the timeout), failures as signals for retries, breakers, alerts and dashboards, and a predictable worst-case response time — the basis of any latency promise.
- *You pay:*
  - **False alarms.** The timer sometimes fires on a call about to succeed; for a *write*, that leaves real uncertainty — did it happen? Hence the Idempotency lesson.
  - **Ongoing tuning.** Too long protects nothing; too short creates errors on ordinary slow days. Recheck when a dependency's latency changes.
  - **The work doesn't stop.** A timeout ends *your waiting*, not *the server's working*; the database may keep crunching an abandoned query, burning capacity under heavy load on answers nobody reads. gRPC propagates *cancellation*; plain HTTP mostly doesn't.

When not to use it. Never go without timeouts; every misuse is a wrong *value*:

- **Genuinely long jobs** (reports, **transcoding** — converting video formats or sizes — large LLM jobs, an LLM being a large language model like the one behind a chat assistant) shouldn't get a 10-minute live-call timeout. Make them async: accept, reply immediately, deliver the result via a message queue, **polling** (client asks "done yet?" periodically) or a webhook (the server calls you back when finished).
- **One global number** is right for no dependency.
- **Library defaults** range from 30 seconds to infinity; both are wrong. Check every client.

In the interview. Rarely asked directly, yet every arrow silently asks "how long do you wait if this hangs?" — answer before asked. When it comes up: "what happens if this service just… doesn't respond?"

> **30-second answer:** Every network call gets a deadline at 2–3× the dependency's measured healthy p99, with connection timeouts much tighter than request timeouts. Timeouts turn silent hangs into countable failures for my retries and breakers to react to. Along a call chain I pass the

remaining budget down, as gRPC does, so inner calls never outlive the request that started them.

- *Follow-up: "How do you pick the number?"* — Measure: healthy p99 × a small factor, per dependency, revisited when its latency shifts. An unreasoned "30 seconds" fails both ways: useless in incidents, wasteful in recovery.
- *Follow-up: "The timeout fired, but the write actually went through — now what?"* — Unavoidable uncertainty; make the write safe to repeat — the whole topic of Idempotency.
- *Mistake that fails candidates:* a multi-service design with no word on hanging calls, or "30 seconds" with no reasoning — the mark of someone who has only seen systems on good days.

Self-check:

- Q1: Why should connection timeouts be tighter than request timeouts, and where should the request timeout come from? → Unreachable hosts reveal themselves in milliseconds, so extra waiting gains nothing. Request timeout = measured healthy p99 × 2–3, per dependency.
- Q2: What does a timeout *not* stop, and why does that matter most during overload? → Server-side work: abandoned queries keep consuming a struggling server's capacity, deepening overload. Propagated cancellation (as in gRPC) fixes it where available.
- Q3: An API promises 2 s, makes three sequential calls each with a 2 s timeout, and users see 6-second errors. Why, and how do you fix it? → Each hop got a fresh budget: 3 × 2 = 6 s. Use deadline budgeting — timeout = min(own limit, time remaining) — so the request always fails by 2 s.

Related patterns: Retry (reacts to the failures timeouts create); Circuit Breaker (can't notice hangs without them); Idempotency (settles "did my write go through?"); Bulkhead (limits threads stuck waiting within the timeout window).

TL;DR

- **Problem:** calls without deadlines can hang forever, silently holding threads and connections until one slow dependency consumes the whole caller.
- **Mechanism:** a deadline on every call at 2–3× healthy p99, a tighter connection timeout, and the remaining budget passed down the chain.
- **Costs:** occasional false failures (write uncertainty), ongoing tuning, abandoned server work unless cancellation propagates.
- **Skip it when:** never — move long work to async and replace copied values with measured ones.

Retry with Exponential Backoff

Original lesson: Retry with Exponential Backoff on DesignGurus ↗

What it is. When a call fails for a self-correcting reason, try again — waiting longer after each failure, randomizing each wait so thousands of clients don't retry together, capping attempts, and retrying only errors that could turn out differently. *Analogy:* on a busy line you don't redial every second forever; you wait a little, then longer, then give up — and if the whole town redialed in sync, the line would stay jammed.

The problem. A checkout calls a healthy payment provider, both sides correctly coded, yet ~1 call in 200 fails: networks drop packets, reset connections, occasionally route to a bad server. With 100,000 orders/day, **500 customers a day** get an error on a request that would likely have worked 100 ms later. Such brief, self-clearing failures are **transient**; treating each as final throws away work a second try would finish.

Why the obvious fix fails. "Retry immediately, up to 3 times" goes wrong at scale:

- **Wrong moment.** An overloaded provider is still overloaded a microsecond later. At 20,000 requests/s, a one-second failure with three immediate attempts can land **60,000 requests/s** on the provider when it has least room — a self-inflicted **retry storm**.
- **Everyone at once.** Requests that failed together retry together on a fixed schedule, producing synchronized spikes — a **thundering herd** that can knock a recovering service back down.
- A third problem (next lesson): if the payment *succeeded* but the response was lost, a retry can charge twice.

How it works. The figure follows one payment call: attempt 1 times out; wait random 0–100 ms; attempt 2 gets a 500 (HTTP "internal server error"); wait random 0–200 ms; attempt 3 succeeds, unnoticed by the customer. Wait windows double (0–100, 0–200, 0–400, 0–800 ms…, capped at 5 s).

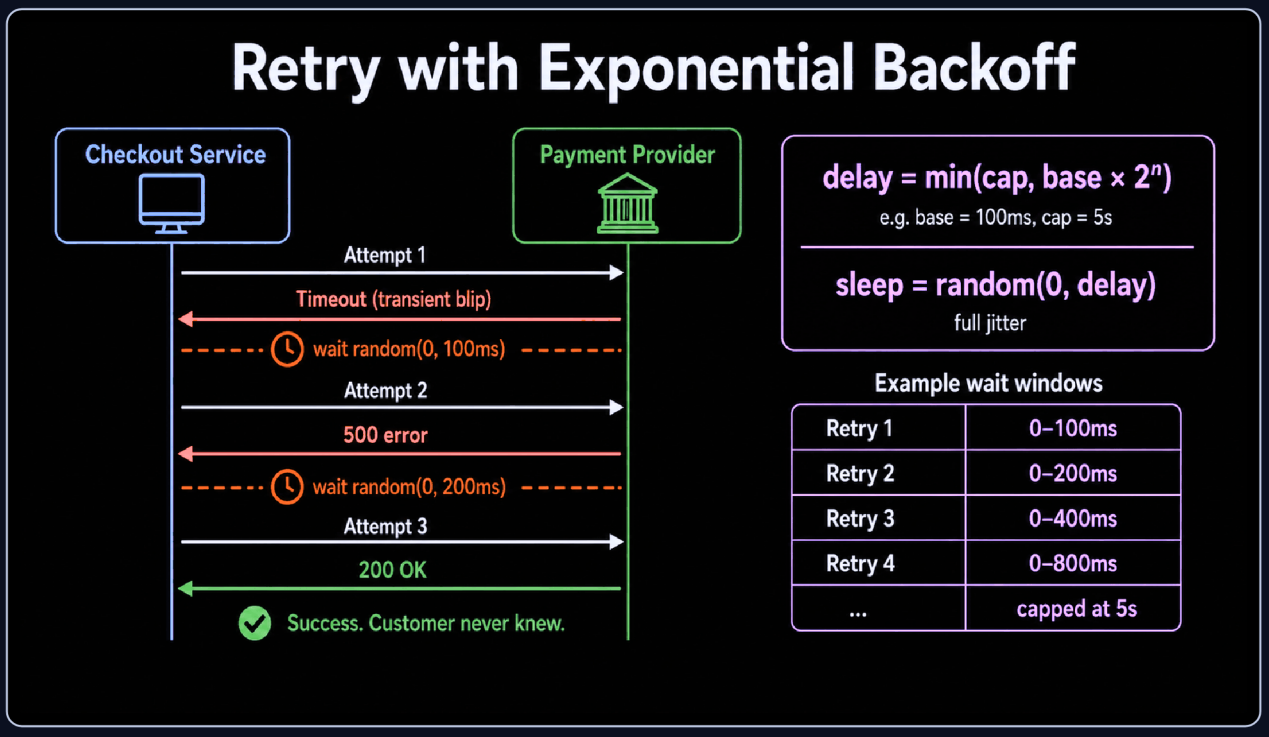


Figure — Retry with Exponential Backoff (course diagram)

The formula ("full jitter"):

```
window = base × 2^attempt          (e.g. base 100 ms → 100, 200, 400 ms…)
wait   = random(0, min(cap, window))   (cap e.g. 5 s)
```

Each part has one job:

1. **Base delay (~100 ms):** lets the most frequent transient problems (lost packets, resets) clear.
2. **Doubling ("exponential backoff"):** each extra failure suggests a bigger problem, so back off harder — the difference between a patient client and one that worsens the outage.
3. **Randomness (jitter) — not optional:** scatters the herd across the window. Backoff *without* jitter just reschedules the same synchronized wave to 100, 200, 400 ms.
4. **Caps and budgets:** a maximum wait (never minutes), a maximum attempt count (3–5 for anything a user waits on), and ideally a service-wide **retry budget** ("retries may add at most 10% extra traffic"); once spent, stop — the failures clearly aren't transient.
5. **Retry only errors that can change.** Retry: timeouts, connection failures, 500 (server error), 503 (service unavailable), 429 (too many requests — obey its `Retry-After` header). Never: 400 (bad request), 401 (bad credentials), 404 (not found); they fail identically every time.

Walkthrough with numbers

- *One request:* a 200 ms network blip; attempt 1 fails, wait ~60 ms; attempt 2 fails, wait ~150 ms; attempt 3 succeeds. Checkout takes ~**400 ms longer** but completes — times 500 customers a day.
- *The fleet:* the provider struggles for one second at 20,000 requests/s. Immediate retries fire ~20,000 × 3 attempts into the outage, then the backlog retries at the instant it ends: ~**3×**

normal load at the weakest moment. With backoff and jitter, retry rounds spread over 100, 200, 400 ms and the provider sees **a small bump a few percent above normal**. Same per-request rule, completely different system behavior: spikes become smooth load.

Trade-offs

- *You gain:* transient failures vanish from screens, dashboards and support queues; recovering services get smooth traffic; fewer 2 AM manual re-runs of **batch jobs** (scheduled programs processing a large pile of work).
- *You pay:*
  - **Slower worst cases:** each retry adds its wait (≈400 ms above); your slowest requests now contain your retry policy.
  - **Extra load in real outages:** retries against a dead dependency are waste; backoff softens it, but only a circuit breaker or retry budget stops it.
  - **Multiplication across layers:** app 3× × gateway 3× × service 3× = **27 requests** per tap at the bottom. Retry in one deliberately chosen layer (two at most), usually nearest the failure.
  - **Hidden chronic problems:** silent retries make a 5%-failing dependency look healthy while adding latency and load; record retry counts — a rising rate is an early warning.
  - **A hard prerequisite:** retrying a *write* is safe only if doing it twice is harmless (idempotency, next lesson). Until then, don't retry it.

When not to use it

- **Writes not safe to repeat:** add an idempotency key first.
- **Errors that can't change** (validation, permission, not-found): classify errors before looping.
- **When the user has waited enough:** a fast failure plus a Retry button often beats four silent attempts over 3 seconds. User-facing paths get small budgets (1–2 retries); background jobs generous ones.
- **Sustained failures:** the circuit breaker's job.
- *Common misuse:* "retry 3 times" at every layer, even on 400s, no jitter, no metrics — painfully obvious at the next partial outage.

Retry now vs. queue for later. Retry is the optimistic tool, the breaker the pessimistic one, and real systems stack them. A message queue is the less obvious alternative; both answer "this call failed — now what?"

Situation	Reach for
The caller needs the answer during this request (payment authorization, a user is waiting)	Synchronous retry with backoff
The caller needs the work to happen, but not the answer right now (send a receipt, sync the warehouse)	Put it on a queue; the consumer retries over minutes or hours
The retry has to survive a process crash or a long outage	Queue, with a dead letter queue as the final safety net
The entire retry window fits inside one request's timeout	Synchronous retry

Ask aloud: *does anyone need this result right now?* If not, the sturdiest retry is a durable one: queued attempts can span hours, survive restarts, and be kept safely when exhausted.

Real-world examples

- **AWS SDKs** use exponential backoff with jitter by default; AWS's "Exponential Backoff and Jitter" post, born of real thundering herds on DynamoDB (Amazon's managed key-value database), is the classic explanation.

- **gRPC and Envoy** (a popular network proxy) support retry budgets capping retries at a percentage of live traffic.
- **Stripe** pairs retries with idempotency keys on every money-moving endpoint — what makes retrying a charge safe.
- **Kafka producers** (programs writing to Kafka, a widely used message log) retry sends with backoff; enabling idempotence adds the next pattern in the client.
- **For AI engineers:** LLM APIs shed load with 429 and "overloaded" errors, and the official Anthropic and OpenAI **SDKs** (ready-made client libraries) include backoff-with-jitter by default. In production: obey `Retry-After` exactly, keep a per-provider retry budget, and when one is exhausted, switch models at your **LLM gateway** (a proxy your apps call instead of providers directly) rather than retrying into a sustained outage.

In the interview. "What happens if this call fails?" is the most common follow-up; answer "retry" and the trap springs: "Could your retries make an outage worse?" Raise retries unprompted, with idempotency in the same breath.

30-second answer: I retry with exponential backoff and jitter — about 100, 200, 400 ms, each randomized so clients don't retry in sync and flatten a recovering service — with capped attempts, and only for errors that can change, like timeouts and 500s. Writes are made idempotent first so a retry can't apply twice. AWS SDKs do exactly this by default. When failure is sustained rather than transient, I stop retrying and let a circuit breaker take over.

- *Follow-up: "Why jitter? What breaks without it?"* — Clients that failed together retry together; backoff alone reschedules the herd to 100, 200, 400 ms. Randomized waits spread them, turning spikes into a smooth plateau.
- *Follow-up: "How do you stop retries from making an outage worse?"* — Weakest to strongest: backoff spaces attempts; a retry budget caps them at a traffic fraction; a circuit breaker drops them to zero once failure is clearly sustained. Structurally: retry at one layer.
- *Mistake that fails candidates:* "I'd just retry 3 times" — no backoff, jitter, idempotency or error classification: four follow-ups you'll miss.

Self-check:

- Q1: Write the full-jitter formula and say what the cap protects you from. → `wait = random(0, min(cap, base × 2^attempt))` . The cap bounds worst-case waits so a persistently failing call can't back off into minutes, blowing response budgets and holding resources, and keeps timing predictable for capacity planning.
- Q2: App retries 3×, gateway 3×, backend 3× on DB calls; the DB has a 10-second brownout (slow but not fully down). What does it experience and where should retries live? → Up to 27 attempts per tap — self-inflicted overload, every layer's queue full of duplicate work. Retry in one layer, usually nearest the failure (the backend's DB call, with a budget); outer layers fail fast and show a Retry button.
- Q3: A dependency recovers from a 30-second outage, collapses 100 ms later, recovers, collapses again, in perfect rhythm; your config is 3 attempts, fixed 100 ms delay, no jitter. → Your clients are a scheduled thundering herd: the whole backlog lands in one wave at each recovery. Fix with doubling delays, jitter and a circuit breaker so sustained failure stops generating retry pressure. The other team's service is fine.

Related patterns: Idempotency (required before retrying writes); Timeout (defines "failed"); Circuit Breaker (takes over for sustained failure); Dead Letter Queue (where exhausted queue retries land); Rate Limiting (the other side — good retriers obey 429 and `Retry-After`, good limiters send them).

TL;DR

- **Problem:** network calls fail briefly all the time, and naive instant retries turn that into synchronized spikes against an already struggling dependency.
- **Mechanism:** longer waits after each failure (100, 200, 400 ms…), randomized, capped attempts and delays, only changeable errors, within a service-wide retry budget.
- **Costs:** slower worst cases, extra load in real outages, 27× multiplication across careless layers, hidden chronic failures, mandatory idempotency for writes.
- **Skip it when:** the write isn't safe to repeat, the error can't change (400s), the user's patience is spent, or the failure is sustained.

Idempotency

Original lesson: Idempotency on DesignGurus ↗

What it is. An operation is **idempotent** if running it many times has exactly the effect of running it once. For operations that aren't (like "charge this card"), the client attaches a unique **idempotency key**; the server remembers each handled key with its result and answers repeats with the saved result instead of executing again. *Analogy:* pressing an elevator call button five times doesn't bring five elevators; the key gives a card charge that behavior.

The problem. A customer clicks Pay on a **$499** course bundle; the provider charges the card but the success response is lost; ~100 ms later the retry sends an identical request, which the provider treats as new and charges again. The customer sees **$998**.

Root cause: a timeout means "no response came back," *not* "the operation failed" — the request may never have arrived, or finished with only the reply lost. Repeating reads is harmless; repeating an uncertain *write* can apply it twice. Duplicate writes come from retry policies, double-clicks, and at-least-once message queues.

The numbers: ~500 uncertain timeouts a day; if a third had already charged → **~165 double charges per day**, each costing a refund, support time and a **~$15 chargeback fee** (the customer's bank reverses the charge, marking your record with the processor): **~$4,000 a day**, plus customers watching you bill them twice.

Why the obvious fixes fail

- **"Check before you write"** ("is there already a recent charge for this order?"). Between check and write, two concurrent retries can both find nothing and both charge — a *race condition* (outcome depends on uncontrollable timing), made far more frequent by retry bursts than tests suggest.
- **"Don't retry writes."** Brings back 500 failed checkouts a day.
- **"Detect duplicates by similarity"** (equal charges, same user, within 60 s). Blocks someone legitimately buying two $499 gifts, and misses a real duplicate at 61 seconds.
- Common flaw: they lack information *only the client has*. To the server, a retry and a genuinely new identical order look the same; only the client knows both came from one click, so the client must send that identity.

How it works. Replaying the $998 incident with idempotency: checkout sends "charge $499, Idempotency-Key: ord_7f3a"; the provider atomically claims the key ("first time seen"), charges, stores

the result under the key; the response is lost; the retry carries the *same* key, the claim finds it taken, and the stored result returns — 200 OK, no second charge.

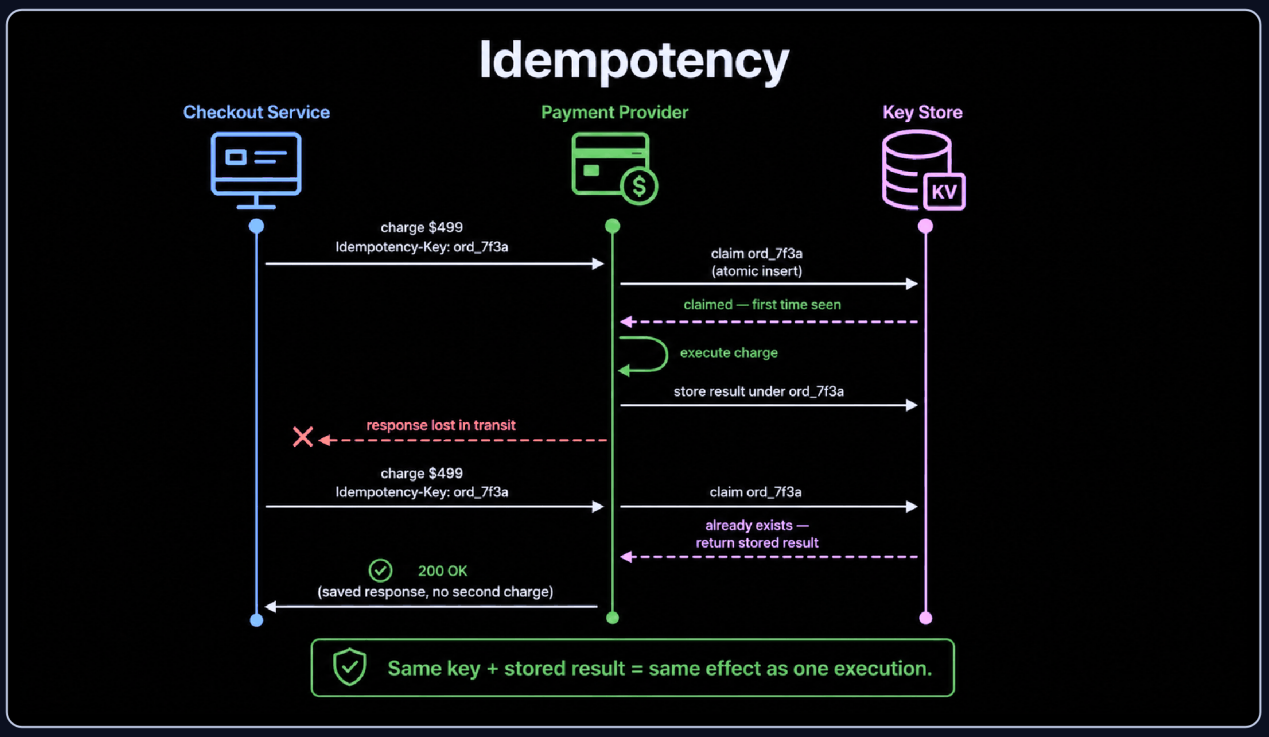


Figure — Idempotency (course diagram)

1. **The client creates the key before the first attempt**, one per *logical* operation (`ord_7f3a` for this order's payment), sent with every retry. Create it at the Pay click and store it with the pending order, so even after an app crash and reload the retry carries the same key. A client inventing a new key per retry only *looks* idempotent.
2. **Claiming is atomic** (one indivisible step, no gap): insert the key under a uniqueness rule — insert if missing, fail if present. The database guarantees one winner, which truly closes the check-then-write race: of two simultaneous requests, exactly one executes; the other gets the winner's result.
3. **Save the result, not just the key.** A retrying caller needs the original answer (charge ID, status), not an error; a duplicate arriving mid-execution gets "in progress — wait" (or HTTP 409 Conflict), not a second run.
4. **Keys expire.** Stripe keeps them **24 hours**; keep them longer than your longest realistic retry window. This expiry timer is a **TTL (time to live)**.

The cheaper option first: naturally idempotent operations. "Set status to shipped" can run ten times; "increment login count" can't. Prefer *set* semantics ("balance = 90") over *increment* ("subtract 10"), **conditional writes** ("update only if version is still 6"), and **upserts** (insert if missing, else update). These never need a key store.

Walkthrough with numbers (100,000 orders/day, 1 in 200 payment calls times out, retries on)

- *Without idempotency:* ~500 uncertain outcomes/day → ~165 double charges → ~$4,000/day plus reputational damage.
- *With keys:* the retry's `ord_7f3a` is recognized in **~5 ms** and the saved response returned. Double charges: **zero** — by design, not "rare."
- *Cost:* one key lookup per write (**~1 ms**) and 100,000 keys/day × ~1 KB saved responses ≈ **100 MB** at a 24-hour TTL — the cheapest protection in the course against your scariest failure.

Trade-offs

- *You gain:* safe retries, redeliveries and double-clicks for writes, by design; **the effect of exactly-once on top of at-least-once delivery** (the only "exactly once" that exists — below); calmer incidents, since "did the retry double anything?" stops being a question.

- *You pay:*
  - **A key store on your busiest write path:** uniqueness guarantee, result storage, expiry — modest but real infrastructure.
  - **Client discipline:** keys made before attempt one and surviving crashes — an agreement with every client team that needs testing.
  - **The in-progress case:** a small *state machine* (states plus transition rules): claimed → in progress → completed.
  - **The boundary problem** (trickiest): the key protects only the claim-and-execute step. Side effects outside it — confirmation email, warehouse webhook, third-party call — can still happen twice and need their own keys (e.g. key the email on the order ID).

When not to use it

- **Already naturally idempotent operations** — reads, deletes, full-value sets; key machinery is pure overhead.
- **When you can redesign instead:** the best key is the one you never need — "set balance to 90" or a conditional write over "subtract 10."
- **Streams where duplicates are acceptable** — high-volume analytics deduplicated later, or where a 0.5% duplicate rate changes no decision.
- **As business-level duplicate detection.** A key means "the same operation, retried," not "the user probably didn't mean to do this twice." Two deliberate identical orders are two operations with two keys; blocking them is a product decision.
- *Common misuse:* key stores on every endpoint while the operations stay unsafe increments and appends — more complexity, no more clarity.

Idempotency vs. "exactly-once delivery." Interviewers raise this deliberately: exactly-once *delivery* is impossible over an unreliable network because any acknowledgment ("got it") can be lost, forcing the sender to resend and risk a duplicate (**at-least-once**) or not and risk loss (**at-most-once**). No third option exists. Systems *can* achieve exactly-once **processing**: at-least-once delivery plus an idempotent handler.

You need	Build
No lost work	At-least-once delivery (retries, queues with acknowledgments)
No doubled effects	An idempotent consumer (keys, or natural idempotency)
"Exactly once"	Both of the rows above — there's no third ingredient

Kafka's "exactly-once semantics" is exactly this: producer IDs plus sequence numbers make the broker's (Kafka server's) writes idempotent, combined with transactional consumers. "I'd use exactly-once delivery" names the problem; "at-least-once delivery plus an idempotent consumer" solves it.

Real-world examples

- **Stripe:** an `Idempotency-Key` header on every money-moving endpoint, saved responses replayed for 24 hours; its docs made the term industry vocabulary.
- **Kafka's idempotent producer:** a producer ID and sequence number per batch let the broker discard duplicates — idempotency keys by another name.
- **AWS SQS FIFO** dedupes by message ID within a **5-minute** window; **DynamoDB conditional writes** give the atomic claim with no extra infrastructure.
- **Webhook consumers:** providers openly deliver at-least-once, so every handler dedupes on event ID.
- **For AI engineers:** agent frameworks may retry a timed-out `send_email` or `create_ticket` tool call, or the agent loop may request it again; every tool with external effects needs an idempotency key derived from the tool-call ID. Usage-recording writes for billing need the same, or retries silently overcharge.

In the interview. Rarely "define idempotency"; it arrives as a follow-up once you say "retry" or draw a queue: "what if the payment succeeded but the response was lost?" or "your consumer crashes before acknowledging — what happens on redelivery?" Answer it in the same sentence as retries.

> **30-second answer:** A timeout is uncertain — the write may have succeeded with only the response lost — so a blind retry can apply it twice. The client creates one unique key per logical operation and sends it with every retry; the server atomically claims it, executes once, saves the result, and replays that result for duplicates. That's Stripe's Idempotency-Key model: an exactly-once effect on at-least-once delivery, the only exactly-once there is.

- *Follow-up: "Two requests with the same key arrive at the same moment — walk me through it."* — The claim is an atomic insert-if-absent: one wins and runs; the other sees it claimed, waits or gets "in progress," then receives the saved result. An answer involving check-then-write has just revealed your race.
- *Follow-up: "Why can't the server generate the key?"* — Only the client knows two physical requests are one logical operation; a server-made key differs per request, defeating the purpose. The client makes it before attempt one and keeps it across retries.
- *Mistake that fails candidates:* "check if it exists, then write" as the defense, or claiming exactly-once queue delivery handles it.

Self-check:

- Q1: Define idempotent; why must the client generate the key, and when? → N executions = one execution's effect. Only the client knows two requests are one intent; the key must exist before the first attempt and survive crashes (stored with the pending order).
- Q2: With standard idempotency keys on checkout, what duplicates can still happen? → Side effects outside the claim-and-execute boundary (emails, webhooks, third-party calls); intentional business duplicates (two keys); buggy clients making new keys per retry; duplicates arriving after the key's TTL.
- Q3: After enabling retries, customers get duplicate welcome emails, yet orders have no duplicate rows and the key store correctly replays results. Diagnose. → The email is sent outside the idempotent boundary: attempt one charged, emailed, lost its response; the retry correctly replayed the result, but the email code ran again. Move the email inside the boundary or, better, key it on the order ID (or send via a deduplicating queue). Each side effect needs its own idempotency plan.

Related patterns: Retry (why this lesson exists); Message Queue (at-least-once makes every consumer a retry target); Webhooks (dedupe on event ID); Saga (every step and undo step must be idempotent, since the coordinator retries them); Exactly-Once Semantics (how Kafka assembles exactly-once processing from these parts).

TL;DR

- **Problem:** timeouts are uncertain, so retries, redeliveries and double-clicks can apply writes twice — double charges, duplicate orders.
- **Mechanism:** the client makes one key per logical operation and reuses it; the server atomically claims it, executes once, saves and replays the result. Prefer naturally idempotent (set-style) operations.
- **Costs:** a key store on the busy write path, client key discipline, an in-progress state, side effects outside the boundary needing their own keys.
- **Skip it when:** the operation is already safe to repeat, duplicates are harmless downstream, or the "duplicate" is a business-rule question.

Circuit Breaker

Original lesson: Circuit Breaker on DesignGurus ↗

What it is. A wrapper around every call to a dependency that tallies recent failures; past a threshold it "trips" and for a while rejects every call immediately without trying the dependency; after a cooldown it lets a few test calls through to check recovery. *Analogy:* your home's breaker panel cuts power before an overloaded wire catches fire — no in-the-moment decision needed.

The problem. Black Friday, 2:15 PM. Checkout calls a recommendations service for an *optional* "customers also bought" widget. The recommendations database starts failing: 20 ms queries now take 30 seconds or never return. Checkout goes down.

Why: checkout has **200 threads** and **300 requests/second**, each now waiting up to 30 s; Little's law wants 300 × 30 = **9,000 threads**, so all 200 are stuck on the widget within about a second. Checkout already had short timeouts and retries, but even a 2 s timeout needs 300 × 2 = **600 threads** — three times the pool — and retries triple calls into the dying database (a *retry storm*).

The deeper issue: timeouts and retries both assume *a later call might work*. During a sustained outage that assumption does harm; the overloaded dependency needs **zero** new calls, and nothing so far delivers that.

Why the obvious fixes fail

- **"Tighten the timeout."** 2 s → 500 ms reduces checkout's thread use, but every request still hits the failing database, spending its remaining capacity on 300 hopeless queries/s. Protects the caller, not the dependency's recovery.
- **"Retry harder."** Retries are for transient failures; this is sustained, so they add load.
- **"Have someone flip a switch."** A flag like `recommendations_enabled: false` lets **on-call** (the engineer currently responding to alerts) stop calls — the right policy at human speed: outage at 2:15:00, alert at 2:19, flag at 2:26 — **11 minutes** of broken checkout versus about a second automatically.

How it works. The breaker sits between Service A (caller) and Service B (dependency). Closed (and successful half-open tests): requests pass normally. Open: the request is never sent; the caller gets a fast failure or fallback (e.g. a cached response) in ~1 ms.

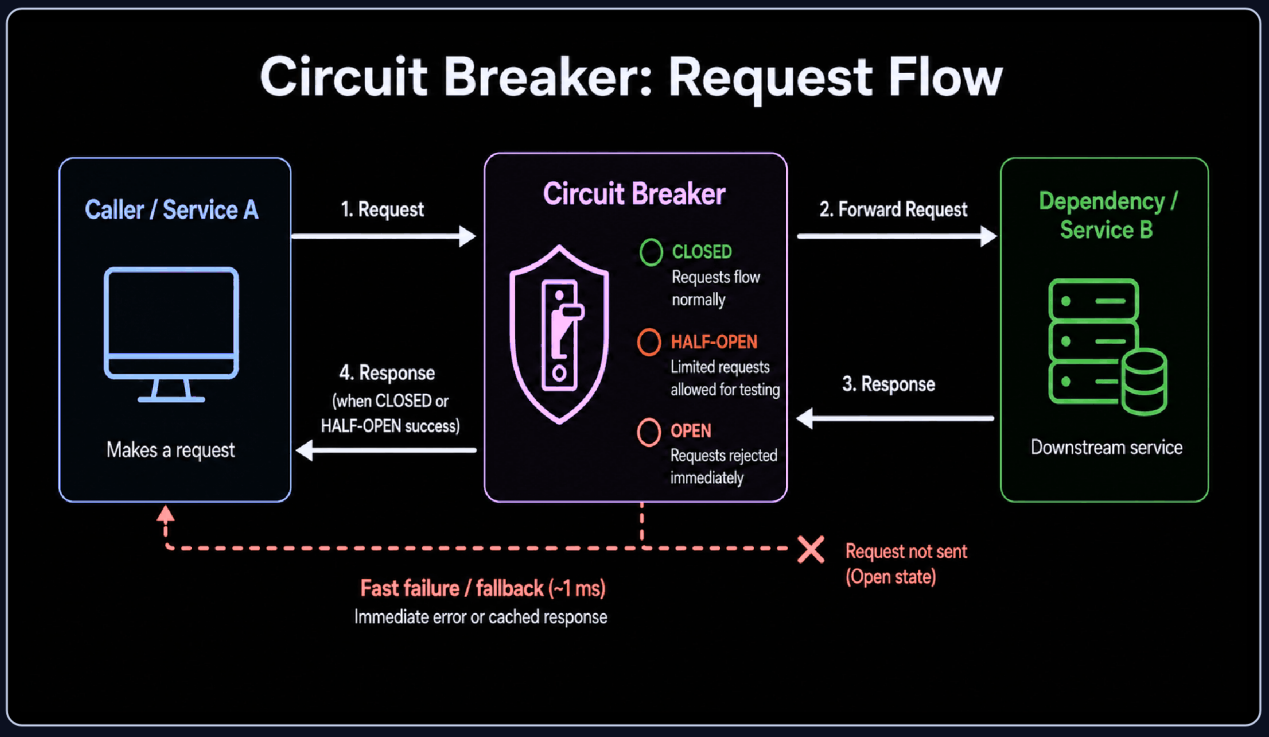


Figure — Circuit Breaker: request flow (course diagram)

It's a three-state **state machine**. Example transitions: Closed → Open when >50% of the last 20 calls fail; Open → Half-Open after a 30-second cooldown; Half-Open → Closed if all 5 test calls succeed, back

to Open if any fails. Open never jumps straight to Closed — recovery must be *proved* by test calls.

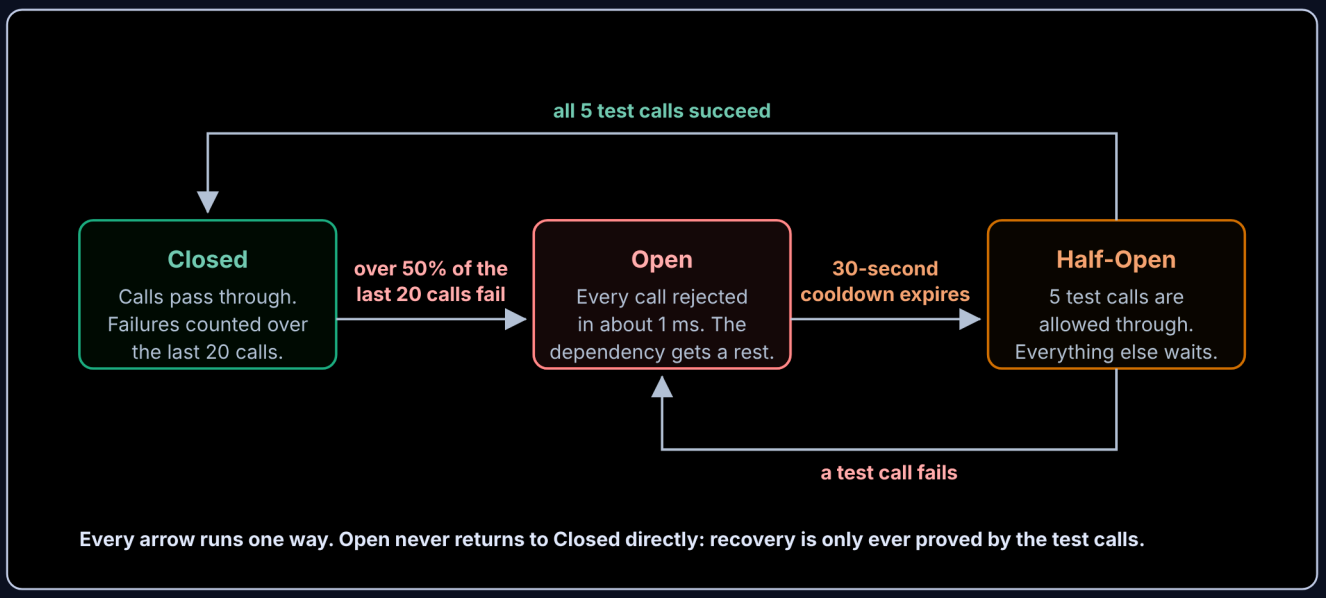


Figure — Circuit Breaker: state transitions (course diagram)

1. **Closed (normal).** Calls pass; failures (errors *and timeouts*) are counted over a recent window, e.g. the last 20 calls. Without timeouts, hung calls never count — hence Timeout first.
2. **Open (tripped).** Past the threshold, every call is rejected in ~1 ms without touching the dependency: your threads stop draining, and the failing service gets what it most needs — a break from traffic.
3. **Half-open (testing).** After the cooldown, a few trial calls go through: success closes the breaker; failure reopens it for another cooldown. Recovery is detected automatically without dumping full traffic on a just-recovered service.

To remember: *a breaker swaps slow, costly failures for fast, cheap ones, and gives the dependency room to recover.*

Walkthrough with numbers (window 20 calls, trip at 50%, 2 s timeout, 30 s cooldown, 5 half-open test calls)

- **2:15:00** — the recommendations DB starts failing; calls time out at 2 s.
- **2:15:01** — ~10 of the last 20 calls failed; the breaker trips. Damage: ten calls held threads 2 s each.
- **2:15:01 onward** — all 300 req/s get an instant "breaker open" in ~1 ms: 300 × 0.001 s = **0.3 threads** busy on average, versus 9,000 unprotected. The page renders without the widget; checkout is normal.
- **2:15:31** — cooldown ends; 5 test calls fail; the breaker reopens. Cost of checking: five calls.
- **2:19:31** — a few cycles later tests succeed; it closes and recommendations return automatically.
- Result: ~4 minutes of dependency failure, zero change in checkout availability — an optional feature traded for a few minutes to protect everything else.

Trade-offs

- *You gain:* failure isolation (one bad dependency can't swallow threads, connections or memory); fast failure (a millisecond, not a full timeout); automatic recovery without full-traffic bursts or 2 AM flag-flipping; and real relief for the dependency — the fastest help for an overloaded service is to stop calling it.
- *You pay:*
  - **False trips:** a short blip — a 5-second *garbage-collection pause* (the runtime freezing to reclaim memory) or a *database failover* (switching to a standby copy) — can trip it and reject good traffic for a whole cooldown. Threshold tuning is ongoing.

- **More settings:** window, threshold, cooldown, test-call count. Bad values give a breaker that *flaps* (repeatedly opens and closes) or never trips.
- **Per-server decisions:** state usually lives per *instance* (running copy); 50 instances each spend ~10 failed calls discovering the same outage. Shared state (e.g. in Redis) fixes it but adds a failure point; most teams accept the per-instance cost.
- **It only decides *whether* to call,** not what the user sees; an open breaker without a fallback is just a faster error page (Graceful Degradation).

When not to use it

- **Local in-process calls:** breakers guard remote things that can hang or fail long; wrapping a function call is cargo-culting.
- **Calls where failing fast is worse than waiting**, like **capturing** a payment (actually taking the money) or sending a legal notice: wait, retry carefully, or queue.
- **Very low-traffic paths:** at 4 calls an hour, "half of the last 20" spans days; it never trips or trips on noise. A timeout suffices.
- **As a stand-in for capacity:** a DB breaker opening daily at peak means insufficient database capacity; don't hide it.
- *Common misuse:* breakers everywhere with defaults, no fallbacks, no dashboards — healthy traffic rejected and nobody knows why. Every breaker needs an **owner, a fallback, and an alert** when it opens.

Circuit breaker vs. retry — opposite diagnoses; interviewers love making you choose:

Situation	Reach for
Transient failures: a dropped packet, one bad server, a brief failover	Retry with backoff
Sustained failures: the dependency is down or overloaded	Circuit breaker
Another try would probably succeed	Retry
Another try only piles load onto a struggling service	Circuit breaker

In production, combine them with the **breaker outside the retries**, counting each request's *final* outcome after retries: retries absorb blips, and the breaker cuts their extra load once failure persists. (A breaker without retries rejects blips that would have recovered quickly.) The Bulkhead answers a different question: the breaker decides *whether to call* (avoidance); the bulkhead limits *how much capacity* a dependency may occupy (containment). They stack.

Real-world examples

- **Netflix** built **Hystrix**, which made the pattern famous, after slow dependencies kept *cascading* (failure spreading service to service) into full streaming outages; its successor **Resilience4j** is now standard on the **JVM** (Java Virtual Machine, the runtime for Java and similar languages).
- **Envoy and Istio** (a *service mesh*: proxies beside every service) do it as "outlier detection," ejecting persistently failing servers from the pool with no application code.
- **Polly** (.NET, Microsoft's platform) and **PyBreaker** (Python) implement the same three states; **AWS SDKs** include adaptive versions for *throttling* (a service capping accepted calls).
- **For AI engineers:** model providers routinely rate-limit and slow under load; a breaker per provider lets an LLM gateway switch models in milliseconds instead of hanging on a failing endpoint.

In the interview. Rarely by name; instead "what happens to your service if this dependency goes down?", "how do you prevent cascading failures?", "what if this search service hangs?" The signal sought: **a slow dependency is more dangerous than a dead one.**

> **30-second answer:** A circuit breaker wraps the call and tracks failures over a rolling window; past a threshold it opens and rejects calls instantly instead of letting them pile up on timeouts, protecting my thread pool and relieving the struggling service. After a cooldown it sends a few test calls and closes once they succeed. Netflix built Hystrix for exactly this after slow dependencies cascaded into full outages.

- *Follow-up: "How do you pick the threshold and cooldown?"* — Measure, as with timeouts: a window with enough traffic to mean something (20+ calls), a threshold well above the normal error rate, a cooldown near the dependency's typical recovery time; then tune from production data and alert on every open.
- *Follow-up: "The breaker is open — what does the user see?"* — The breaker is half the pattern; the other half is a fallback: cached data, a default, or hiding the optional widget (Graceful Degradation).
- *Mistake that fails candidates:* proposing retries for a failing dependency with nothing that ever *stops* retrying.

Self-check:

- Q1: Name the three states and what causes each transition. → Closed (failures counted over a rolling window) → Open when the failure rate passes the threshold; Open (all calls rejected instantly) → Half-open when the cooldown ends; Half-open (a few test calls) → Closed on success, Open on failure.
- Q2: Your team adds a breaker to primary-database calls with library defaults. What new failure modes did you introduce? → False trips (an ordinary 10 s failover rejecting good queries for a whole cooldown, turning a blip into minutes of partial outage); flapping on quiet endpoints; and with no fallback, instant errors on primary data — likely a case where waiting beats failing fast. It needs tuned thresholds, a fallback (cached reads or read-only mode) and an alert.
- Q3: Worst-case latency spikes to exactly 10 s (your timeout) whenever optional search degrades; soon after the whole API errors. Explain and fix. → Every request holds a thread for the full 10 s; requests/s × 10 s exceeds the pool, so healthy endpoints starve — a cascading failure by resource exhaustion. Fix with the whole module: a timeout just above search's healthy p99, a breaker so sustained failure costs 1 ms, a bulkhead capping search's threads, and rendering the page without search results.

Related patterns: Timeout (the breaker's evidence); Retry (handles transient failures inside; the breaker prevents storms); Graceful Degradation (runs while open); Bulkhead (limits occupancy while evidence accumulates); Health Check Endpoint (a URL a **load balancer** — the server spreading requests across service copies — polls for health; the same idea at machine level, removing failing servers).

TL;DR

- **Problem:** a slow or failing dependency eats your threads through timeout-length waits and drags your healthy service into its outage; retries worsen it, and nothing says "stop calling."
- **Mechanism:** a three-state machine — count failures (closed), reject instantly past a threshold (open), probe with a few calls after a cooldown (half-open).
- **Costs:** threshold tuning, false trips on blips, per-instance state, and you still owe users a fallback.
- **Skip it when:** the call is local, traffic is too thin for a signal, or failing fast costs more than waiting.

Bulkhead

Original lesson: [Bulkhead on DesignGurus ↗]

What it is. Divide a shared resource (threads, connections, machines) into walled pools — one per feature, dependency or **tenant** (a customer organization sharing the system) — so one full pool can't starve the rest. *Analogy:* a hull's watertight compartments (bulkheads) mean a hole floods one compartment, not the ship. Total capacity is unchanged; no workload can grab all of it.

The problem. One API **deployment** (a running installation, often many identical copies) serves checkout, search and order history, sharing **200 threads** and **100 database connections** (a *pool* is a fixed set of workers or connections requests borrow briefly). Marketing emails **2 million** people a link to order history, whose traffic leaps from **50 to 1,500 requests/second**, and it runs the service's most expensive query, **~500 ms**.

Little's law: 1,500 × 0.5 = **750 threads** wanted, 200 available; order history takes all of them within ~a fifth of a second, and checkout, with zero threads, stops selling.

Yet *nothing fails*: every query succeeds, the database is up, error rate **0.0%**, dashboards green, circuit breaker closed (no failures to count). The problem is **occupancy** — how much shared capacity one workload holds — not outcomes. Same shape: one big tenant starving 500 others, a batch job holding every connection, one expensive endpoint eating the service.

Why the obvious fixes fail

- **"Add more threads."** 1,000 threads moves the limit without controlling who gets them; the next surge wants 1,200, and meanwhile 1,000 threads fight over 100 DB connections. Growing everything for one temporary workload is costly and still not isolation.
- **"The circuit breaker will handle it."** Calls complete before timeout, so the breaker sees 0% failures. You need something that watches occupancy.
- **"Rate-limit the users."** Useful at the edge for individual clients, but thousands of legitimate users each send one request and none exceeds a per-user limit. You need a cap on the *feature's share* of the pool.

How it works. The figure contrasts one shared 200-thread pool (order history needs 1,500 × 500 ms = 750 threads, takes all 200, others get 0, error rate 0.0%) with walled pools — checkout 80, search 60, order history 40, reserve 20 — each sized by rate × duration and triggering on *fullness*, not errors.

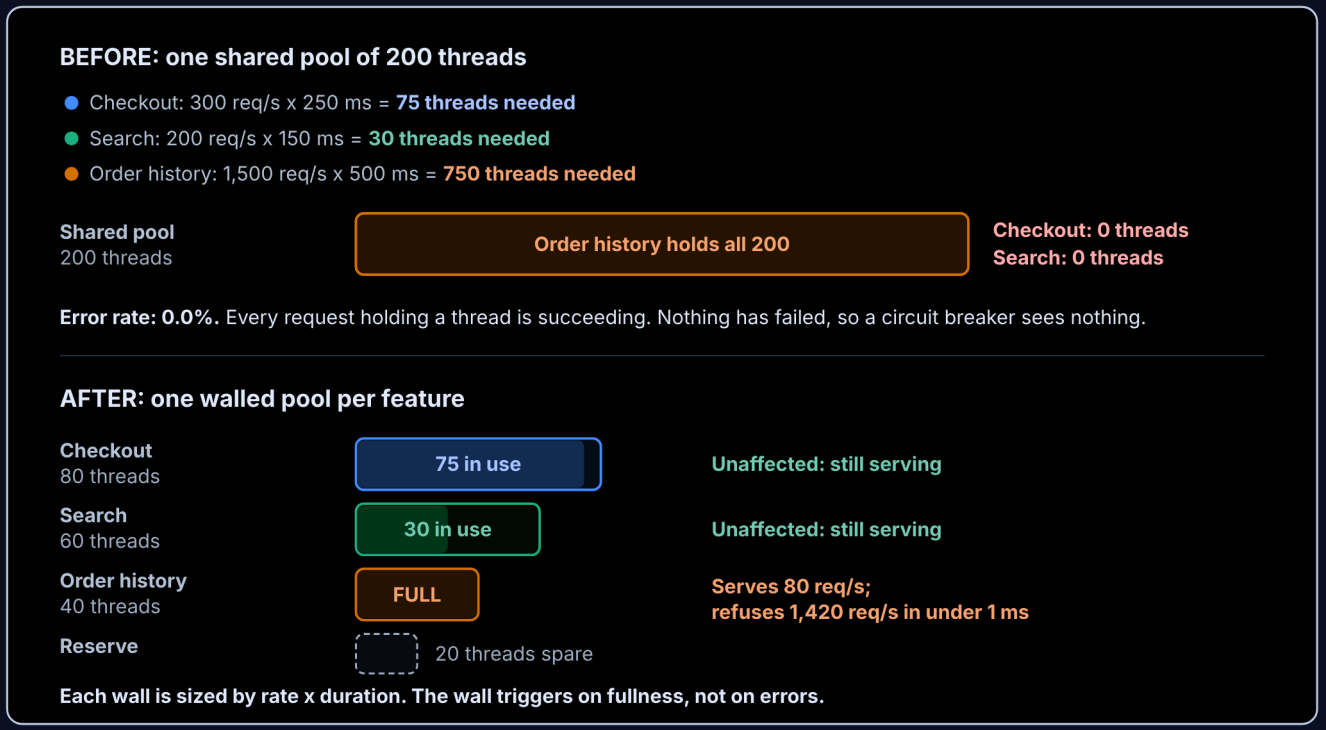


Figure — Bulkhead (course diagram)

Three levels of wall, each costlier:

1. **Counters (semaphores):** a *semaphore* allows at most N concurrent things ("at most 40 order-history requests at once"); the 41st is rejected immediately. Cheapest; Resilience4j's default.
2. **Separate pools:** each feature or dependency gets its own thread or connection pool; more overhead, but even a fully stuck compartment can't touch others' threads. Hystrix's signature.
3. **Physical separation:** different processes, **containers** (lightweight isolated packages running an app with its own CPU and memory slice) or machines per workload or customer tier. Heaviest, and the only level also isolating CPU, memory and crashes.

At every level a full compartment **rejects new requests quickly** (or queues briefly) and never borrows from another. A rejection means the wall works; what the rejected user gets is Graceful Degradation's job.

Sizing: run the incident's arithmetic backwards — *needed* = *rate* × *duration* plus *headroom* (burst room). Checkout: 300 req/s × 0.25 s = 75 → **80**.

Walkthrough with numbers (checkout 80, search 60, order history 40, reserve 20)

- **The flood:** order history's 40 threads at 500 ms serve 40 / 0.5 = **80 req/s**; the other **1,420 req/s** get "busy, try again shortly" in ~1 ms.
- **Checkout meanwhile** needs 75 of its own 80; latency and **conversion** (share of visitors completing a purchase) are unchanged.
- **The database is protected too:** at most 40 concurrent order-history queries instead of 200 — an upstream cap is a downstream ceiling.
- **Total damage:** one feature degrades honestly under load it wasn't sized for, versus the whole product down in a fifth of a second with green dashboards.
- The promise shifts from "survives if *total* demand fits" to "**each feature survives if its own demand fits**" — several keepable promises instead of one fragile one.

Trade-offs

- *You gain:* protection when *nothing is failing* (the module's only occupancy-based tool); a *blast radius* (how far one failure spreads) chosen in advance; honest per-feature capacity numbers (pool fullness shows who needs scaling); a ceiling protecting downstream systems.
- *You pay:*
  - **Idle capacity you can't lend:** overnight checkout's 80 threads may idle while order history rejects next door; every compartment needs its own spare room.
  - **Sizing goes stale:** as traffic mix drifts, an outdated split rejects in one full pool while capacity idles nearby — a self-inflicted outage invisible in totals. Re-split on a schedule.
  - **More to watch:** N pools, N fullness metrics, N rejection alerts, N configs. Alert on **per-compartment fullness**; error-rate alerts can't see this failure.
  - **One more concept** for every on-call engineer.

When not to use it

- **Single-purpose services**, already isolated at the service boundary.
- **Tiny pools:** 20 threads split four ways rejects on ordinary variation; below a few dozen threads, isolation may cost more than it protects.
- **Uniform, elastic workloads:** similar cost and priority, with servers added in seconds (*elastic*), may use capacity better in a shared **autoscaled** pool (growing and shrinking automatically). Bulkheads pay off when workloads differ in cost, importance or trust.
- **As a stand-in for capacity:** every compartment near full daily means you need more total capacity.

- *Common misuse:* a compartment per endpoint and client, each too small for normal variation — a hundred undersized pools give rejections, not resilience.

Bulkhead vs. circuit breaker

	Circuit Breaker	Bulkhead
Watches	Failures (errors, timeouts)	Occupancy (who is holding how much)
Triggers when	The dependency is failing	Something is hogging the capacity
Action	Stop calling completely; check again later	Limit the share; reject the overflow
Sees "everything succeeds, slowly, everywhere"?	No — there's nothing to count	Yes — fullness shows up
Sees "dependency is hard down"?	Yes — drops calls to zero quickly	Partly — limits the stuck threads

Breaker when the dependency is failing; bulkhead when sharing is the danger. Stack them: the bulkhead limits threads stuck in a bad dependency while the breaker gathers evidence, and once it trips the compartment drains — why Hystrix shipped both. Versus rate limiting: rate limiting controls *outside clients by identity* at the front door; bulkheads control *internal sharing by compartment*.

Real-world examples

- **Netflix Hystrix** made per-dependency thread pools famous — a hung dependency could trap only its own small pool; **Resilience4j** continues it, defaulting to lighter counters.
- **Kubernetes** builds bulkheads into the platform: CPU/memory limits per *pod* (a unit of deployed containers), separate node pools for batch vs. interactive work, per-team quotas.
- **Envoy** caps connections per upstream inside the mesh — bulkheads without application code.
- **Database connection pools split by service** (via PgBouncer, a PostgreSQL connection-pooling proxy, or per-deployment limits) keep one busy service from taking every connection.
- **For AI engineers:** multi-tenant GPU serving is today's biggest *noisy-neighbor* problem (one tenant's load hurting everyone on shared hardware) — one customer's batch embedding job can starve every interactive chat. Separate interactive and batch GPU pools (or limited batch quota slots), give each pricing tier a compartment so free-tier load can't push out paying customers, and cap per-tenant concurrency at the LLM gateway. Same rate × duration math, in GPU slots.

In the interview. "One endpoint suddenly gets 100× traffic — what happens to the others?", "noisy neighbor in a multi-tenant system?", "what does one slow feature do to a shared thread pool?"

30-second answer: I split thread, connection or node pools by feature or tenant, each sized by rate × duration plus headroom, and a full compartment rejects only its own overflow. That protects critical work even when the competing requests all *succeed* — invisible to a circuit breaker, which counts failures. Hystrix's per-dependency thread pools are the best-known implementation.

- *Follow-up: "How do you size the compartments?"* — Rate × duration per compartment plus burst headroom; then treat "one pool keeps rejecting while total capacity idles" as the signal to re-split. Admitting the split drifts shows you've operated one.
- *Follow-up: "Bulkhead or circuit breaker here?"* — Diagnose aloud: errors or timeouts from the dependency → breaker; legitimate successful load taking over shared capacity → bulkhead. Production stacks both.

- *Mistake that fails candidates:* prescribing a breaker for hogging, or treating green error dashboards as proof critical features have capacity.

Self-check:

- Q1: What does a bulkhead partition, what triggers it, and how does that differ from a breaker? → It walls shared capacity (threads, connections, memory, machines, GPU slots) and reacts to *fullness*; a breaker reacts to *failure* evidence. So bulkheads catch success-shaped hogging that breakers can't see.
- Q2: Overnight, checkout's pool is 90% idle while order history rejects; a teammate wants to merge pools. What's the real choice? → Efficiency vs. worst-case protection: merging recovers idle capacity but restores the original failure mode. Right only if workloads are equal in importance and shape, capacity autoscales fast, or the pool is too small to split. With revenue-critical checkout involved, idle threads are the price of protection.
- Q3: A multi-tenant **SaaS** (software-as-a-service: one hosted product shared by many paying customers) with 500 customers; one ships a buggy integration opening 2,000 concurrent requests; everyone slows down; error rate near zero. Diagnose and fix at two layers. → Success-shaped hogging starving 499 tenants. Front: per-tenant rate limits and concurrency caps answering excess with 429. Inside: bulkhead shared pools by tenant tier (or per tenant for the largest), sized by rate × duration. Alert on per-compartment fullness, not just errors.

Related patterns: Circuit Breaker (stack them); Timeout (bounds how long each slot is held — without it compartments fill and stay full); Rate Limiting (limits clients by identity at the boundary); Graceful Degradation (what a rejected request becomes).

TL;DR

- **Problem:** shared pools let any feature, tenant or dependency legally take everything, starving the rest at zero error rate.
- **Mechanism:** walled pools per consumer (counters, separate pools, physical separation), sized by rate × duration; a full compartment rejects its own overflow quickly.
- **Costs:** idle capacity that can't be lent, stale sizing, more pools to monitor.
- **Skip it when:** the service does one thing, the pool is too small to split, or workloads are uniform and elastically autoscaled.

Dead Letter Queue

Original lesson: Dead Letter Queue on DesignGurus ↗

What it is. A **dead letter queue (DLQ)** is a separate queue for messages that keep failing: after a set number of failed attempts, the queue system pulls the message from the main line, with notes on how it failed. The main line keeps flowing, and the message is kept safe for inspection, fixing and replay. *Analogy:* the postal "dead letter office" holds undeliverable mail instead of discarding it or jamming the sorting line.

Terms first. A *message queue* stores a *producer*'s messages until a *consumer* processes them; on finishing, the consumer sends an *acknowledgment (ack)* and the queue deletes the message. The *broker* is the queue system (SQS, RabbitMQ, Kafka). Under at-least-once delivery, unacked messages are delivered again. (Full lesson: Message Queue.)

The problem. An order-fulfillment consumer reserves inventory and notifies the warehouse at **200 messages/second**, flawlessly for months. At **2:47 AM** a partner publishes an order with a **null** shipping address; the consumer throws an **exception** (the error signal raised when code can't continue), doesn't ack, and the broker redelivers it forever. This isn't transient: malformed input fails

deterministically — attempt 4,000 fails like attempt 1 — a **poison message**. With ordered processing, everything behind it waits: **720,000 stuck orders per hour** from one null field.

A teammate's emergency fix — catch, ack, move on — restores flow but makes things worse: the next malformed message is simply deleted with no replayable record, and the following weekend **40 paid orders vanish** — never shipped, no payload to investigate. A poison message forces a choice between blocking ordered processing forever and silently deleting business data. The DLQ is the third option.

Why the obvious fixes fail

- **"Retry harder."** Backoff helps failures that might clear; an unparseable message fails every time, and each retry steals capacity from healthy messages.
- **"Catch, log, and skip."** Better than silent deletion, but logs may be sampled, rotated, truncated or barred from storing full payloads, and can't be replayed through the pipeline after a fix. The order is still lost; you just recorded losing it.
- **"Validate everything at the producer."** Worth doing, but can't stop consumer bugs, valid-but-unusual edge cases or downstream rejections. Some messages will always fail in processing; the design must say where they go.

How it works. The producer writes to the main queue; the consumer processes. Success → ack. Failure with attempts < 5 → back to the main queue for redelivery. 5th failure → DLQ. DLQ depth > 0 → alert; an engineer inspects payload and error; after a fix, the message is redriven to the main queue.

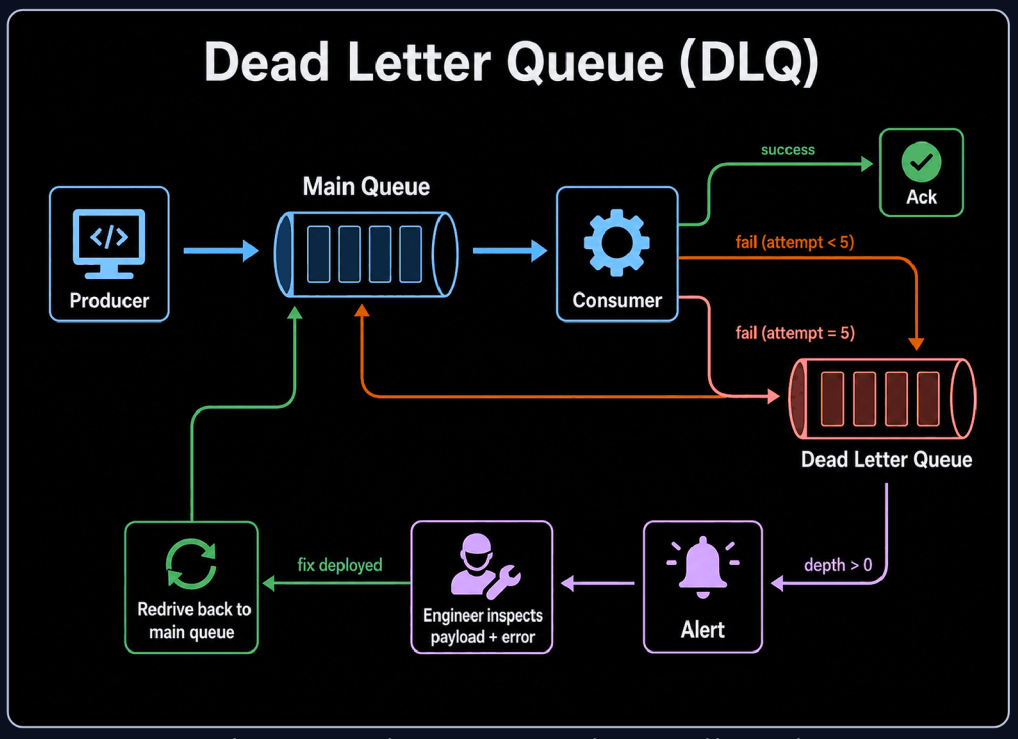


Figure — Dead Letter Queue (course diagram)

Four pieces (only the first is really code):

1. **A broker-enforced attempt limit.** The broker counts deliveries and at the limit (SQS's `maxReceiveCount`, usually **3–5**) moves the message itself. It must be the broker: the failure may be the consumer crashing on this very message, and code that can't survive a payload can't be trusted to route it.
2. **Failure notes travel with it:** source queue, exception, attempt count, first and last failure times. Without context a DLQ entry is a puzzle; with it, a bug report.
3. **An alarm on the DLQ.** An unwatched DLQ only *postpones* deletion: retention is limited (**14 days on SQS**), after which messages are auto-deleted. Alert the moment an important DLQ is non-empty.
4. **A way back (redrive).** After the fix ships, messages are *redriven* to the main queue for real processing — impossible with logs. A redrive is a redelivery, so consumers must be idempotent.

The DLQ separates two jobs the emergency fix conflated: keeping healthy messages moving in milliseconds, and giving a person hours to understand strange input.

Walkthrough with numbers (5 attempts with backoff, then dead-letter)

- **2:47:00** — the null-address event fails attempt 1; attempts 2–5 fail over ~30 seconds.
- **2:47:30** — the broker moves it to the DLQ with exception and history; the consumer moves on.
- **Backlog math:** 30 s × 200 msg/s ≈ **6,000 messages** queue behind it and clear in well under a minute — versus 720,000 per hour without end.
- **2:47:31** — the DLQ alarm fires; on-call reads the **payload** (message contents) and **stack trace** (function calls active at the error), finds the partner's null field; the fix (edge validation plus a consumer patch) ships in the morning.
- **9:15 AM** — one-click redrive; the order ships hours late instead of never.
- **Total:** one order delayed, zero lost, a 30-second slowdown instead of an all-night outage, for one extra queue and one alarm.

Trade-offs

- *You gain:* poison no longer blocks processing and stays available; a structured, searchable record (payload, error, history) logs can't give; replay — fix, redrive, and the work really happens; and DLQ depth, among a pipeline's most honest health signals.
- *You pay:*
  - **One more queue to own:** unwatched, it turns visible loss into delayed silent deletion.
  - **Broken ordering for the parked message:** message 47 runs after 48 through 6,000 — fine for independent events (orders, emails), not where order carries meaning.
  - **A judgment the counter can't make:** an attempt count can't distinguish bad input from a healthy message caught in a ten-minute downstream outage; a bad threshold can dump thousands of valid messages in the DLQ (self-check Q2).
  - **Redrive tooling and a data-retention question:** payloads with personal data (PII) now live in a second place.

When not to use it

- **Strictly ordered streams** — per-customer sequences where event 48 must never precede 47 (account ledgers, state machines). Skipping corrupts data; instead **pause that stream** (stop its **partition**, the ordered queue slice holding that key's messages), alert loudly, and **fix forward** (fix the code and resume without skipping). Other streams continue. A DLQ here is a data-corruption feature.
- **Throwaway data** — high-volume telemetry where one lost reading changes nothing: drop and count it.
- **Before retries are tuned:** dead-lettering on the first failure turns every blip into a ticket; the DLQ is for *after* retries run out.
- **As a business-logic route:** "payment declined" is an expected outcome with its own flow. Keep the DLQ for the unexpected so its alarm always means something.

DLQ vs. retry-queue tiers. In-place redelivery with backoff suits short gaps, but holding a consumer slot through an hour-long downstream outage is wasteful. Some systems (especially around Kafka) use **retry tiers**: a failed message goes to `retry-5m` (whose consumer waits five minutes, then retries), then `retry-1h`, and only then the DLQ.

Situation	Reach for
Failures clear within seconds (transient, brief contention)	In-place redelivery with backoff, then DLQ
Failures need minutes to hours (downstream outages, rate limits)	Retry tiers, with the DLQ as the last stop
Deterministic failure (malformed payload, code bug)	Straight to the DLQ — waiting won't help
Ordering per key must be kept	Neither one can skip safely — pause the stream

These are steps in one sequence — quick redelivery → spaced tiers → DLQ. Most systems need the first and last; add the middle only if outage-length failures are common.

Real-world examples

- **AWS SQS** (Amazon's managed queue) mainstreamed it: per-queue redrive policy, broker-driven moves, a redrive button in the console (AWS's web control panel).
- **RabbitMQ** generalizes it as *dead letter exchanges*; with per-queue delays, that's also how it builds retry tiers.
- **Kafka** has no built-in DLQ; its ecosystem (Kafka Connect, Spring Kafka) provides dead-letter topics, and the strict-ordering warning applies fully.
- **Google Cloud Pub/Sub** includes dead letter topics with built-in delivery-attempt tracking.
- **For AI engineers:** LLM pipelines constantly produce poison — invalid **JSON** (common structured text format), embedding-breaking documents, moderation rejects — each failing deterministically. In RAG ingestion (loading documents for retrieval-augmented generation), one corrupt PDF in a 100,000-document crawl goes to a DLQ for human review while 99,999 get indexed; the DLQ doubles as the review queue.

In the interview. Draw a queue and expect "what happens when a message can't be processed?" or "a malformed message is crashing your consumer — walk me through the next ten minutes." Saying *poison message* instantly marks you as someone who has run a queue.

30-second answer: Under at-least-once delivery a malformed message is redelivered forever and blocks everything behind it, while dropping it is silent data loss. So after a few failed attempts the broker moves it, with error details, to a dead letter queue: the main line flows, nothing is lost, an alarm fires as soon as the DLQ is non-empty, and after the fix we redrive. SQS's redrive policy is the standard example. Rule to remember: a DLQ nobody watches is data loss on a timer.

- *Follow-up: "What's your process for what lands in the DLQ?"* — Alarm at depth ≥ 1 for critical flows; inspect payload and exception; fix; redrive. Any steady inflow is a defect, never normal. Mention retention — "parked" isn't "safe forever."
- *Follow-up: "Your events are ordered per customer — does dead-lettering still work?"* — No; skipping breaks the sequence. Pause the affected stream and fix forward; dead-letter only independent events.
- *Mistake that fails candidates:* "retry until it succeeds" (deterministic failures never do), or an elaborate DLQ with no one watching it.

Self-check:

- Q1: What moves a message to the DLQ, why must the broker do it, and what should travel with it? → The delivery-attempt count reaching its limit (say 5). The consumer may be what's crashing, so it can't route. Carry source queue, exception, attempt count, first/last failure times.

- Q2: A downstream dependency is down for ten minutes; the queue is set to 5 attempts over 30 s, then DLQ. What happens and how do you fix it? → Every message arriving in the outage exhausts its attempts in 30 s and is dead-lettered: thousands of healthy messages flood the DLQ, hiding real poison and forcing a bulk redrive. Fix: stretch the retry window beyond realistic outages (retry tiers) and put a circuit breaker on the consumer's downstream call so consumption pauses during hard outages.
- Q3: A payments DLQ set up eight months ago: 12,000 messages passed through, 300 remain, no incident was ever raised. What happened? → Nobody watched it: 14-day retention silently deleted ~11,700 payment events, found by an auditor instead of an alarm. Missing: an alarm on DLQ depth/inflow, a named owner with an inspect-fix-redrive *runbook* (written steps), and retention long enough for the response process.

Related patterns: Message Queue (at-least-once delivery makes endless redelivery, hence DLQs, necessary); Retry (handles passing failures; the attempt limit is where they meet); Idempotency (redrive is redelivery); Circuit Breaker (on the consumer's downstream calls, keeps outages from looking like thousands of poison messages).

TL;DR

- **Problem:** under at-least-once delivery, a poison message forces a choice: redeliver forever and block everything, or drop it and silently lose data.
- **Mechanism:** after N failed attempts the broker moves it, with failure details, to a separate queue; an alarm fires when that queue is non-empty; messages are redriven after the fix.
- **Costs:** another queue to own, broken ordering for parked messages, an attempt counter that can't tell poison from an outage, and the trap that an unwatched DLQ is delayed loss.
- **Skip it when:** ordering is strict (pause instead), data is throwaway (drop and count), or the "failure" is a normal business outcome.

Graceful Degradation

Original lesson: Graceful Degradation on DesignGurus ↗

What it is. When part of a system fails, return a *lesser* answer instead of an error: optional features disappear or show older data, while the few critical operations that must never be wrong refuse honestly. You get there by ranking features by criticality and giving each a pre-planned sequence of fallbacks, so failures cost *quality* rather than *availability*. *Analogy:* when the grill breaks, a good kitchen pulls grilled dishes and keeps serving the rest — but never serves undercooked chicken to fill an order.

The problem. A homepage makes **12 service calls** (catalog, cart badge, recommendations, reviews, pricing, promotions, recently viewed…), each guarded by a timeout, controlled retries and a circuit breaker. Recommendations fails; its breaker opens within a second and rejects instantly, as designed. The homepage still errors: the caller has no handling for an "open breaker" result, so the exception kills the whole page **render** (assembling the final page for the browser). The breaker worked; the application had no alternative answer.

The math: each dependency is **99.9%** available, but the page renders only if all twelve succeed: $0.999^{12} \approx$ **98.8%**, about **104 hours of homepage downtime a year**, mostly from optional widgets (availability in chains). Rule: *every dependency without a fallback becomes a required one*, and page availability is all of theirs multiplied. Earlier patterns decide when to stop calling; none decides what the user gets afterward.

Why the obvious fixes fail

- **"Make every dependency five-nines"** (99.999% uptime). Twelve services at 99.99% still multiply down, and one bad deploy wrecks the estimate; better targets don't change "all twelve must succeed."

- **"Wrap every call in try/catch and return null"** (*try/catch* intercepts an exception; *null* means "no value"). Treats optional reviews and the essential cart identically, tells the user nothing, and can hide critical failures — say, checkout silently skipping inventory reservation and leaving inconsistent data. It dodges the pattern's central decision: which features are required?
- **"Cache everything."** A cache can be one fallback step, but it serves reads, not writes, needs explicit *staleness limits* (how old an answer may be), and a miss during an outage still needs another response.
- What's needed: engineering **and product** rank each feature and choose its fallback *before* the incident.

How it works — two parts, and the first isn't code

1. **Criticality tiers, decided in a meeting with product.** *Critical* — the page is meaningless without it (catalog, checkout, login). *Important* — clearly worse without it (pricing detail, search filters). *Optional* — nobody would file a ticket (recommendations, recently viewed). That meeting *is* the pattern; code enforces it.
2. **A fallback sequence per feature** — an ordered ladder of progressively lesser answers (figure): (1) fresh, full quality → (2) a cached copy seconds to minutes old → (3) a static default ("popular this week," the list price) → (4) hide the feature → (5) critical features only: refuse clearly and loudly ("nothing was charged, nothing was faked"). Optional features may fall to "hidden," important ones stop at a default, critical ones (checkout, login, payment capture) skip straight to refusing. How far each may drop is agreed with product *beforehand*, not decided by an exception handler mid-incident.

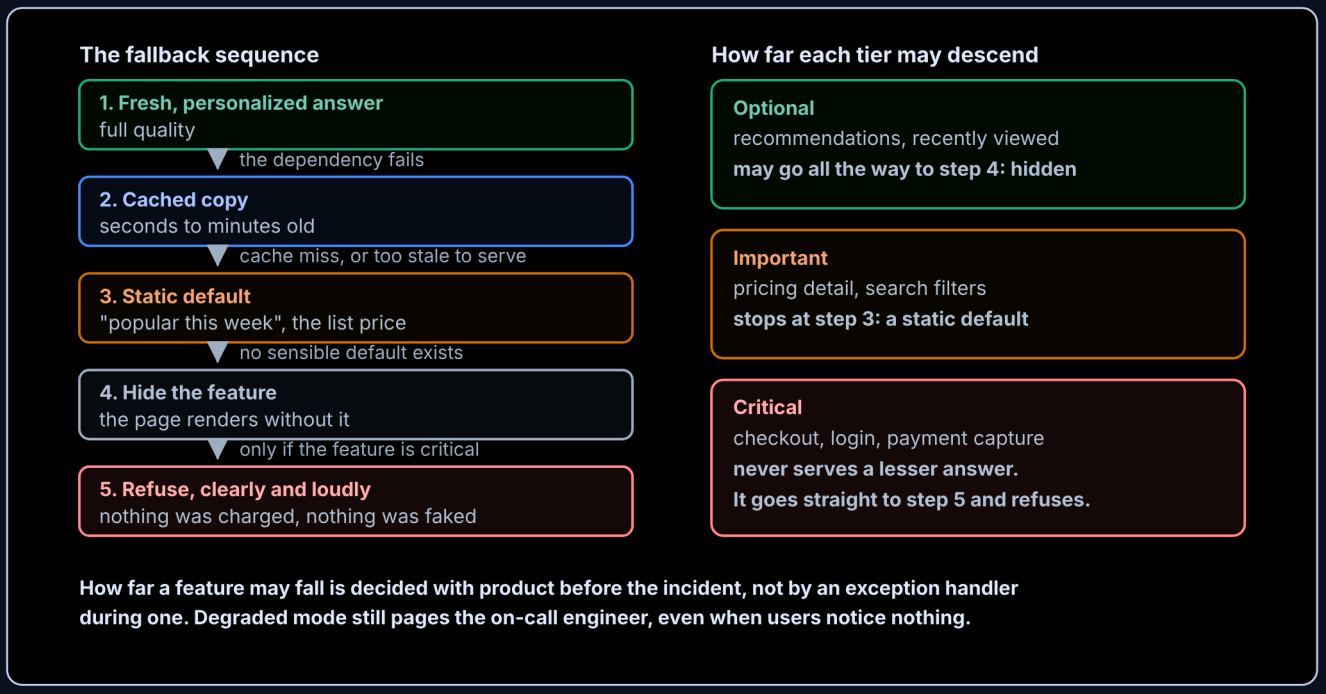


Figure — Graceful Degradation (course diagram)

Examples: *recommendations* → personalized list, else precomputed "popular this week," else hide the row. *Pricing* → live price, else a cached price within a business-approved staleness limit, else fail — a wrong price isn't cosmetic.

Whole-system versions of the same idea

- **Read-only mode:** browsing works; writes are on hold.
- **Queue-writes-for-later:** accept the action, queue it durably, confirm by email once done.
- **Priority shedding:** under overload, drop optional traffic first so critical traffic survives.

Two operating rules that separate real implementations from decorative ones

- **Alarm inside, degrade smoothly outside.** Degraded mode must page on-call even when users see an acceptable page, or the root cause never gets fixed and the "temporary" fallback becomes permanent.
- **Fallbacks rot unless exercised.** An else-branch that never runs is broken when needed. Netflix's **Chaos Monkey** (deliberately breaking things in production) makes fallbacks run for real; **game days** (scheduled practice outages) are the gentler version.

Walkthrough with numbers (catalog critical, 99.95% available alone; the other 11 fragments each get a ladder ending in "hide")

- **The recommendations outage again:** the breaker opens and the fallback serves the hourly "popular this week" list in **~2 ms**; the page looks complete; a `degraded_mode` metric alerts on-call; conversion drops a measured **~1%** instead of to zero.
- **Availability rebuilt:** the page hard-fails only if catalog fails: 99.95% ≈ **4.4 hours/year**, down from **104**. No dependency got more reliable; 11 just lost the power to take the page down — twelve multiplied numbers became one.
- **The write side:** during a 20-minute payments-database incident, checkout switches to queue-writes mode: orders accepted, given idempotency keys, queued durably (surviving a crash), confirmed by email minutes later. Revenue flows through what used to be 20 minutes of zero sales.

Trade-offs

- *You gain:* system availability independent of optional dependencies; gentler incidents (gradual quality loss buys responders time); an honest written map of what matters, sharpening later capacity and spending decisions.
- *You pay:*
  - **Fallback code is real code:** written, reviewed, and above all exercised — or a second bug behind the first.
  - **Staleness needs a named owner:** cached prices and yesterday's inventory each need a business owner approving how wrong they may be.
  - **The masking hazard:** a fallback can hide failure so well nobody investigates — "popular this week" for a whole quarter — unless degraded mode alerts internally.
  - **Many states to test:** N features × M fallback steps, requiring deliberate *fault injection* (causing controlled failures and checking each response).

When not to use it — things that must fail closed (an honest, immediate "no," never a lesser answer)

- **Correctness-critical writes:** payment capture, inventory commits, dosage calculations, trades. A wrong or half-done answer is worse than refusal. Queuing *without executing* can be fine; pretending it ran never is.
- **Authentication and authorization:** an auth outage never justifies "let them through" or "trust the old answer" — degraded security is called a breach. Auth only degrades toward refusing.
- **Compliance flows:** audit trails, consent checks, regulatory reports.
- **When stale costs more than down:** cached prices in a fast market can lose more than an honest maintenance page.
- *Common misuse:* degrading with no metric, alert, owner or exit plan, so the fallback quietly becomes permanent product behavior.

Degradation vs. failover. *Failover* switches to a standby copy of the failing dependency; both answer "the dependency is failing — now what?", in opposite ways:

	Failover	Graceful Degradation
The move	Swap out the dependency (replica, standby, another region)	Swap out the answer (cached, default, hidden)
Quality afterward	Full	Lower, on purpose
Cost paid in advance	Extra capacity sitting idle	Fallback code and product decisions
Fails when	The whole class is down (a bug in every replica, the provider itself)	No sensible lesser answer exists

They're steps in one sequence: fail over first (users never notice), degrade when failover runs out (users barely notice). A primary-replica database promoting a follower is failover (Primary-Replica); this lesson plans for when promotion isn't enough.

Real-world examples

- **Netflix:** failed personalization shows popular non-personalized titles instead of an error; Chaos Monkey triggers those fallbacks in production to verify them.
- **Amazon** degrades recommendations, reviews, even search, rather than let anything touch checkout; "add to cart must always work" has shaped its architecture for two decades.
- **Cloudflare Always Online** serves cached pages when **origin servers** (the site's own servers behind the CDN) are down — degradation as a product.
- **Facebook** has gone read-only during several major incidents.
- **For AI engineers:** an LLM gateway can fall back from the main model to a smaller or older one, then to a **semantic cache** (matching cached answers by meaning, not exact text), before "try again shortly." A RAG system with retrieval down can answer from the base model with a visible "sources unavailable" label — the label matters, since silently dropping sources degrades *correctness*, possibly a fail-closed matter. Under GPU pressure, shed quality first (smaller **token** limits — tokens being the word-pieces models read and write — and trimmed **context**, the text sent with each request) before dropping requests.

In the interview. Directly: "Service X is down — what does the user see?", usually after you mention a breaker or bulkhead. Most candidates stop at "fail fast" without asking *into what?* The seniority signal: **"Which parts of your design are allowed to fail?"** With tiers you have an answer; without them you discover live that you made everything critical.

30-second answer: With product, I rank features by criticality and give each dependency a fallback sequence — fresh, else cached, else a static default, else hide the feature — so only truly critical things fail, and loudly. The system trades quality for availability: recommendations fall back to popular items while checkout stays untouchable, as Netflix does. Degraded mode alarms internally while looking smooth outside, and game days exercise the fallbacks, because an else-branch that never runs is already broken. Auth and payment capture fail closed: a lesser answer there is a breach or a double charge.

- *Follow-up: "Who decides what's critical, and how?"* — Product and engineering together, before the incident: revenue-path and correctness-bound features are critical; enhancements get ladders. The meeting is the pattern, not the try/catch.
- *Follow-up: "How do you know your fallbacks still work?"* — Scheduled fault injection — game days, chaos tooling, or continuously routing a small share of real traffic down fallback paths — plus a metric and alert per degraded state, so degradation is tracked and has an exit.
- *Mistake that fails candidates:* a design where every dependency is secretly critical, or the reverse — degrading something that must fail closed ("if auth is slow we can skip the check" has ended interviews).

Self-check:

- Q1: Name the fallback steps in order; who assigns criticality, and why not engineering alone? → Fresh → cached/stale → static default → hide → (critical only) honest, loud failure. Product and engineering together, because criticality is a business judgment about revenue, trust and legal weight; engineering alone tends to rank by build difficulty or make everything critical.
- Q2: Degradation works so well that a recommendations outage causes zero complaints and no visible revenue dip. Why is that dangerous and what keeps it safe? → It hides the failure, so the root cause can linger for a quarter. Keep it safe with a degraded-mode metric that alerts internally, plus an owner and exit plan for every degraded state.
- Q3: During an auth brownout a teammate proposes caching each user's last auth answer for an hour and letting known session cookies through if auth stays down — "like recommendations." Evaluate. → The halves fall on opposite sides of the fail-closed line. Briefly caching auth answers (including the "no"s) under a security-owned limit is defensible; letting requests through while auth is down is failing open on a security boundary — a breach. The real mistake is treating auth like recommendations: for some features the worst case is "missing," for others "wrong." Auth only degrades toward refusing.

Related patterns: Circuit Breaker (decides when to stop calling; the fallback runs while it's open); Bulkhead (rejected overflow lands on a fallback step); Cache-Aside (the stale-copy step, promoted from performance trick to availability mechanism); Message Queue (drives queue-writes-for-later).

TL;DR

- **Problem:** without fallbacks every optional widget quietly becomes load-bearing, and availability becomes a dozen teams' numbers multiplied.
- **Mechanism:** rank features with product, pre-plan a fallback sequence per feature (fresh, cached, default, hidden, fail), alarm internally while degrading externally.
- **Costs:** fallbacks that rot unless exercised, staleness someone must own, a masking hazard, many states to test.
- **Skip it when:** the operation must fail closed — payment capture, auth, compliance, or anywhere a lesser answer is worse than none.

Capstone: Hardening a Payment Flow

Original lesson: Capstone: Hardening a Payment Flow on DesignGurus ↗

What this capstone is. Seven patterns, one payment system, seven incidents in the order teams usually hit them, each fix causing or uncovering the next — reliability patterns interact as a system, not a checklist. At each stage, decide the pattern and its key numbers before reading on; that's what interviews test. Each stage summarizes its lesson; follow the link for the full mechanics.

The brief. You've just inherited checkout at a mid-size commerce company, a week before Black Friday. The user clicks Pay; the Checkout API (**200-thread pool**) synchronously calls, in order and while the user waits, the Payment Provider (external, healthy **p99 = 400 ms**), the Orders DB (**100-connection pool**), the Email Service and the Warehouse API. Traffic: **100,000 orders/day**, peaking at **300 checkout**

requests/second in promotions. Promise: answer within **2 seconds, 99.9% of the time**. Reliability tooling: none — every call uses its HTTP client's defaults.

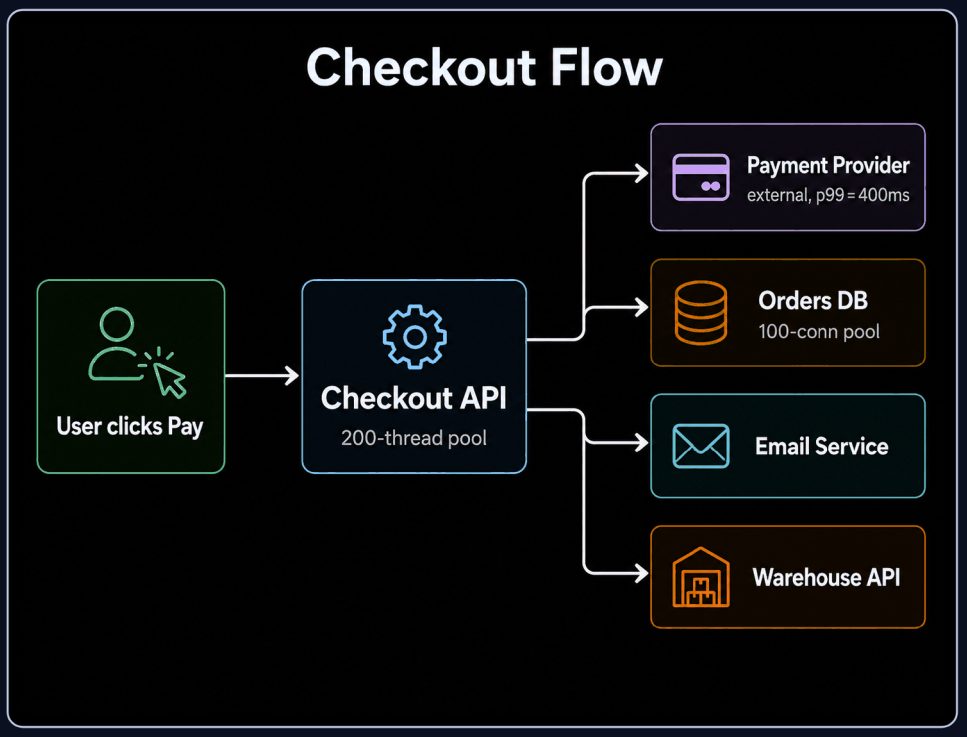


Figure — Payment flow: the brief (course diagram)

Stage 1 — The hang → Timeout

- **Incident.** Monday 11:40 AM: a bad deploy on one provider server returns *nothing*, connections just open. Within **90 seconds** all 200 checkout threads are tied up, health checks fail, and the load balancer pulls your **instances** (running copies of your service) out of rotation. Provider error rate: **0.4%**. Your outage: **total**.
- **Pause and decide:** how did a 0.4% problem become a 100% outage? Which pattern, what number?
- **Why it happened.** The client default waited forever; at 300 req/s, hung calls hold threads indefinitely, draining a 200-thread pool in under a second of hung traffic.
- **Decision (Timeout).** Measured deadlines: provider request timeout **1,200 ms** (3× its 400 ms healthy p99) with a **200 ms** connection timeout; database **250 ms**; email and warehouse **500 ms** each — cutting real hangs fast while leaving room for slow-but-normal calls.
- **Budget check — the first design problem.** 1,200 + 250 + 500 + 500 = **2,450 ms**, over the 2 s promise. Honest conclusion: email and warehouse don't belong in the synchronous path (Stage 6); for now cut them to **300 ms** each so everything fits.
- **What it creates.** Hangs become visible failures: ~1 in 200 payment calls now fails transiently where 100 ms later it would have worked — an invisible problem swapped for a visible one, a good trade.

Stage 2 — Transient failures → Retry with backoff

- **Incident.** **~500 checkouts/day** fail from dropped packets, slow provider servers or brief resets. Tickets: "I clicked Pay, got an error, tried again and it worked."
- **Pause and decide:** failures are transient and random — which pattern, and which three configuration properties?
- **Decision (Retry).** Retry payment calls **up to 3 times** with waits doubling from **~100 ms** plus **jitter**; only changeable errors (timeouts, connection failures, 5xx — server errors in the 500s — and 429 obeying `Retry-After`). A declined card is an *answer*, never retried. Retries live in **exactly one layer** — the Checkout API — so the app and gateway can't multiply them (avoiding 3 × 3 × 3).
- **What it creates.** A timeout is uncertain. Three days after launch, finance forwards a chargeback: a customer charged **$499 twice**.

Stage 3 — The double charge → Idempotency

- **Incident.** About a third of payment timeouts occur *after* the provider took the money; each retry of those is a double charge — dozens a day, each costing a refund, support time and a **$15** chargeback fee. Retries work exactly as designed; that *is* the problem.
- **Pause and decide:** the retry can't know whether attempt one landed. Who can, and what must travel with the request?
- **Decision (Idempotency).** The Checkout API creates **one idempotency key when the user clicks Pay** — one per logical payment, stored with the pending order, so even a crashed-and-reloaded client retries with the same key — sent on every attempt. The provider atomically claims it, charges once, saves and replays the result, so a retry after a lost response gets the original confirmation. Your own writes too: the **order insert is keyed**, so redelivered work can't create duplicate rows. Client-side because only the client knows two requests are one intent.
- **What it creates.** Nothing bad, for once — idempotency is the rare fix adding no new failure mode. But one case remains that retries can't handle.

Stage 4 — The sustained outage → Circuit Breaker

- **Incident.** Black Friday 2:15 PM: the provider enters a sustained *brownout* (up, but too slow to be useful). Every call times out at 1,200 ms and retries faithfully triple traffic against it; each checkout thread spends **~4 seconds** failing through a full retry sequence. Your API isn't down, isn't working, and has become part of the provider's problem.
- **Pause and decide:** retries assume transience; this is sustained. Which pattern, its three states, and what exactly should it count?
- **Decision (Circuit Breaker).** A breaker around the provider call: rolling window of **20 calls**, **50%** threshold, **30 s** cooldown, **5 half-open test calls**, counting **only final outcomes after retries** (breaker outside, retries inside) so neither skews the other's signal.
- **Result.** **~90 seconds** in, the breaker opens: provider calls are rejected in **1 ms instead of 4 s**, payment-thread usage nears zero, and the provider gets no new traffic. About **10 minutes** later successful test calls close it automatically.
- **What it creates.** Two unavoidable questions: (1) while open, users got an *instant* error — still an error (→ Stage 7); (2) the **postmortem** (written incident review) notes everything slowed during the brownout, even provider-free order-status lookups, because all features share thread and connection pools (→ Stage 5).

Stage 5 — The success-shaped flood → Bulkhead

- **Incident.** Cyber Monday: finance's end-of-day reconciliation batch hits the refunds endpoint in the same service at **800 req/s, ~600 ms each**, all succeeding. Error rate **0.0%**, breakers correctly closed, checkout starving: 800 × 0.6 = **480 threads** wanted from **200** shared. Every dashboard green while sales drop.
- **Pause and decide:** nothing fails, so Stages 1–4 are blind. Which pattern, sized by what arithmetic?
- **Decision (Bulkhead).** Split pools by feature and priority: **Checkout 100** (300 req/s × 250 ms = 75 needed, plus headroom); **Refunds 40**, capping reconciliation at **~65 req/s** so the batch takes **3 hours instead of 1** — a trade *agreed with finance*, not discovered by them; **Order-status 40**; **reserve 20**. Same split for DB connections. Reconciliation's overflow waits; checkout notices nothing. New alert: **per-compartment fullness**, since error alerts can't see success-shaped floods.
- **What it creates.** With heavy work capped, slow synchronous work can't be ignored: email (300 ms) and warehouse (300 ms) still sit inside the user's 2 s and can fail a checkout *after* the card was charged.

Stage 6 — The poison message → Dead Letter Queue

- **Change.** Post-payment work goes async: after the charge succeeds and the order commits, the API publishes `order.paid`; consumers send the receipt, notify the warehouse, award loyalty points. The synchronous path shrinks to **charge + order insert** — fast, and a failing email can no longer kill a paid order.

- **Incident.** Three weeks later a partner order has a **null shipping address**; the warehouse consumer throws and the queue redelivers it forever. Every order behind it waits, while checkout keeps selling, unaware fulfillment has stopped company-wide.
- **Pause and decide:** the failure is deterministic — redelivery can't fix it, dropping it loses a paid order. Which pattern, and which *non-code* part makes it real?
- **Decision (Dead Letter Queue).** Every consumer queue gets a redrive policy: **5 attempts with backoff**, then the broker moves the message to a DLQ with exception and attempt history; the line unblocks in **~30 seconds** instead of never. The non-code part: **a DLQ alarm routed to the owning team, with a runbook** (inspect, fix, redrive) — otherwise it's data loss on a **14-day** retention timer. Consumers are idempotent via Stage 3's keys (the receipt keyed on order ID), so redrives can't double-send or double-ship.
- **What it creates.** A changed business rule: an order can be paid but unreported to the warehouse for minutes, which product and engineering must explicitly accept — and the final stage turns such decisions into a system.

Stage 7 — The question nobody answered → Graceful Degradation

- **Incident.** The provider is down **20 minutes** and every mechanism works perfectly — timeouts expire, retries stop, the breaker opens, bulkheads hold, queues drain — yet every Pay click returns an instant error page, losing **$40,000** of purchase intent. The review asks what users *should* have seen; nobody had decided.
- **Pause and decide:** no mechanism is missing. What is, which parts of checkout may degrade, and which must fail closed?
- **Decision (Graceful Degradation, made with product).** Payment capture is **fail-closed** — never fake a charge or assume it worked — but failing closed needn't be rude or total:
  - **The rest of the site stays up:** cart, browsing and order history never touch the provider; bulkheads and fallbacks keep them running.
  - **The cart is protected:** Pay becomes "Payments are briefly delayed; your cart is saved, and we'll email you when we're back," with cart state kept server-side and one-click notification sign-up.
  - **Where the business accepts the risk, capture is postponed:** trusted repeat customers under a spending threshold get **authorize-later** (*authorizing* asks the bank to approve and hold the amount now; capture comes later). The order is accepted, queued durably with its idempotency key, captured when the breaker closes, shown as "payment processing," with an email either way. Finance approved that risk budget *before* the incident — that meeting was the pattern.
  - **Alarm internally, degrade smoothly externally:** a degraded-mode metric pages on-call immediately; the state has an owner and an exit plan.
- **Result.** In the next 20-minute outage, **~70% of would-be buyers convert within the hour**, via saved carts and postponed capture.

The final architecture. Every stage's decision in one picture: bulkheaded pools, a fully wrapped provider call with a degraded-mode path, a protected Orders DB, and an `order.paid` queue whose consumers each have a DLQ with a depth alarm.

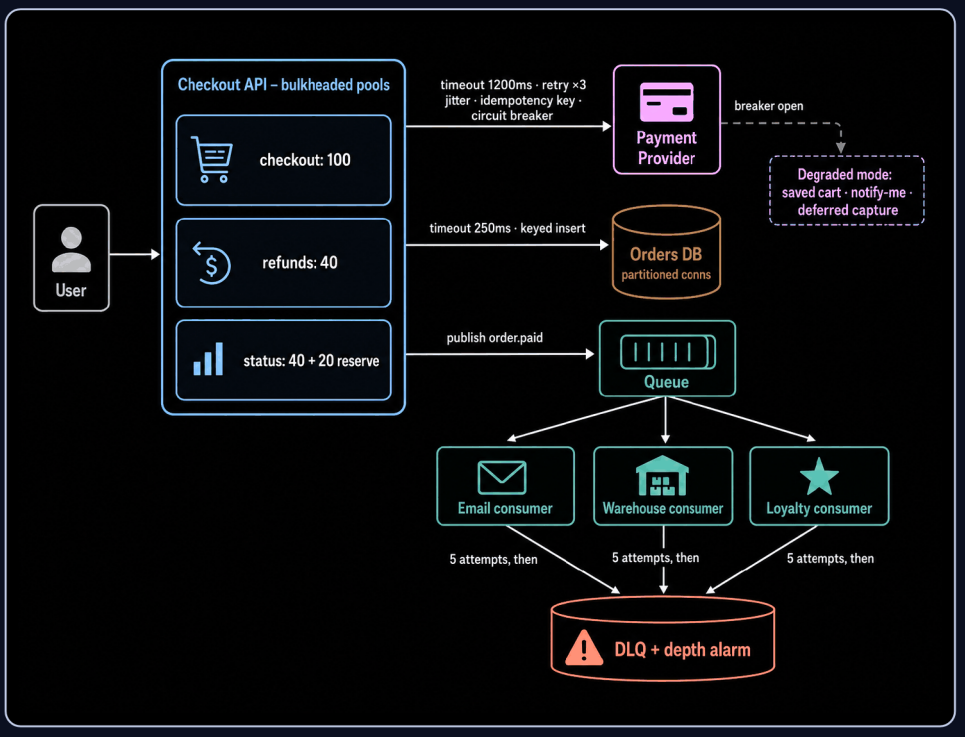


Figure — Payment flow: final architecture (course diagram)

Debrief: the chain, named

Stage	Symptom	Pattern	The number or decision that mattered
1	Hangs eat up every thread	Timeout	3× healthy p99, budgeted along the chain
2	Transient failures kill real checkouts	Retry + backoff	Jitter; only retryable errors; one layer
3	Retries charge twice	Idempotency	Key made on the client, before attempt one
4	A sustained outage wastes threads on hopeless calls	Circuit breaker	Counts outcomes after retries; 30 s cooldown
5	Green dashboards, starving checkout	Bulkhead	Rate × duration per compartment; fullness alerts
6	A poison message halts fulfillment	Dead letter queue	The alarm and runbook, with an owner
7	Flawless machinery, error page anyway	Graceful degradation	Fail-closed capture; tiers signed off by product

The Pattern column, top to bottom, is your answer to "how do you make a payment system reliable?" — and each row exists because the one above shipped: timeouts created visible failures → retries created duplicates → idempotency made retries safe → sustained failure beat retries, hence the breaker → the breaker couldn't see a success-shaped flood, hence the bulkhead → the bulkhead pushed slow work onto queues → queues brought poison, hence the DLQ → and with every mechanism working, one human question remained: *what should the user see?* Don't stop at "what happens when it fails?" Ask **"what happens instead?"**

Interview one-liner: Timeouts make hangs visible, jittered retries soak up blips, idempotency keys make those retries safe, a breaker stops storms caused by sustained failure, bulkheads contain

floods that look like success, async work gets a DLQ that someone watches, and a degradation policy agreed with product decides what users see — with payment capture failing closed.

Extend it yourself

- **Multi-provider:** add a second payment provider. When should the breaker trigger a provider switch, and what does idempotency mean across two providers with no shared key store?
- **The webhook side:** the provider confirms captures by webhook, at-least-once and out of order. Harden that inbound path with this module's patterns.
- **For AI engineers:** swap the provider for an LLM API inside an agent-powered checkout assistant. Which stage changes most? Hint: Stage 7's fallback sequence gains a semantic cache, and Stage 4's breaker can switch models at the LLM gateway.

Module 6 — Growing Under Load (Scaling)

Original lesson: Module introduction on DesignGurus ↗

What this module is about. Earlier modules scaled storage and lifted reads off it (caching). This module scales everything above the data: the application servers, how traffic is shared among them, how many run at 4 AM versus peak, and how to keep a bigger fleet from crushing the database. It finishes the course's fifth question — *how does it grow?* — as four decisions: choose a direction to grow, make servers interchangeable, spread requests across them, and protect what sits behind the app tier.

Pattern picker

The situation	The pattern
The machine isn't big enough	Vertical Scaling
No single machine could ever be big enough	Horizontal Scaling
Traffic has to be steered across many servers	Load Balancing
Demand rises and falls hour by hour	Auto-Scaling
Opening each database connection is costly	Connection Pooling
One region, one bad deploy or one noisy customer can take everyone down	Multi-Region and Cell-Based Architecture *(added lesson)*

Two facts that organize the module

1. **Horizontal scaling has a prerequisite — statelessness — and that is what this module is really about.** Extra servers help only if *any* server can handle *any* request, so no server may remember a client between requests (no sessions in local memory, no files on local disk). Every lasting value needs a shared home (database, cache, object storage). The result is **"cattle, not pets"**: numbered, identical, replaceable servers. Much "scaling work" is pulling state out of app servers.
2. **Scaling one tier multiplies load on the next.** Double the app servers and you may double their DB connections, queries and downstream traffic. So capacity work *follows the bottleneck* — the part currently limiting **throughput** (work completed per second) — and widening it moves the limit elsewhere. The database's connection limit is often the first thing a fast-growing app tier breaks (hence Connection Pooling).

How to read it. Vertical Scaling (no new failure modes, hard ceiling) → Horizontal Scaling (the central lesson: the statelessness changes) → Load Balancing (many servers, one address) → Auto-Scaling (server count tracking demand: signals, flapping, startup delay) → Connection Pooling (how app growth multiplies DB connections until Postgres turns clients away) → **Multi-Region and Cell-Based Architecture** (added lesson: surviving loss of a **region**, and cells so a bad deploy or noisy customer hurts only a slice; plus a cloud building-block map) → the capstone, one service from 100 to 1,000,000 req/s, naming at each stage what to scale *and what to leave alone*. Short on time? Read Horizontal Scaling and Load Balancing — "How would you scale this?" is the question you will hear.

Four terms used throughout (basics such as load balancers, DNS, TLS and latency are in Module 0; see its Vertical and horizontal scaling, Elasticity and Stateless and stateful services)

- **Stateless vs. stateful.** A *stateless* server keeps nothing between requests — everything comes with the request or lives in shared storage. A *stateful* server holds something that makes that machine important (a session, an open socket, a local file). Stateless tiers grow by adding identical servers; stateful ones need the storage patterns.
- **Bottleneck.** The one component currently limiting throughput. Scaling anything else adds nothing, so measure first.

- **Elasticity.** Capacity tracking demand in *both* directions: growing protects users, shrinking saves money.
- **Headroom.** The gap between normal load and capacity; it absorbs surprises while auto-scaling catches up. The course plans peak at **3–5× the average**.

Before you start — think about a system you run:

1. Could *any* app server handle *any* request, or is one special (a session, a file, an in-memory cache with no fallback)?
2. If you doubled your app servers this afternoon, what breaks first downstream?
3. What's your peak-to-average ratio — and who pays for the gap at 4 AM: idle servers, or an auto-scaler?

Vertical Scaling

Original lesson: Vertical Scaling on DesignGurus ↗

What it is. "Scaling up": more CPU cores, RAM and faster disks for the same machine, with the architecture unchanged. *Analogy:* when the kitchen can't keep up, you first buy a bigger stove, not a second restaurant.

Why it matters. It is the least fashionable pattern and deserves more respect. Teams leap to fleets and **microservices** (an app split into many small, separately deployed services calling each other over the network) when one bigger machine would have lasted years. The problem it solves is "the machine is too small"; skipping this fix burns years on distributed complexity nobody needed yet.

How it works, and why it's valuable

1. **No new ways to fail** — unique among scaling patterns. No statelessness work, no load balancer, no cross-machine cache consistency, no distributed coordination; transactions, joins and in-memory state keep working. Every hard problem in the next four lessons starts when a *second* machine appears.
2. **More room than people think.** Cloud instances run from about **2 cores / 8 GB RAM** to **128+ cores and several TB of RAM** — roughly **64× capacity** from one setting. *Stack Overflow* famously served a huge share of the internet's programming questions from a handful of large machines while the industry recommended hundreds of nodes.

Its two hard limits

- **There is a biggest machine.** Catalogs end, and price per unit of capacity rises steeply near the top. Software may stop benefiting first: a database held back by *lock contention* (queries waiting to update the same rows) gains almost nothing from core #96, and single-threaded code can't use extra cores.
- **It is still one machine.** No redundancy; more work on one host *raises* the damage when it fails (a bigger *blast radius*). It remains a *single point of failure* (SPOF), and resizing usually needs a maintenance window.

Combining directions. Real fleets tune size *and* count (24 small vs. 12 medium vs. 6 large), and after sharding every shard still needs its own vertical-size decision.

Trade-offs

You gain	You pay
Capacity with no architecture change — often an afternoon plus a maintenance window	A ceiling, a steep price curve toward it, shrinking software gains near the top
Single-machine guarantees (transactions, consistency, locality) that distributed designs spend modules winning back	No availability gain: one bigger machine is one bigger outage; redundancy needs at least two
Time — a year to do the statelessness work properly rather than in a panic	No elasticity: the big box sits idle and expensive at 4 AM
	Downtime to resize; long waits for top-of-catalog hardware

When not to use it

- **Availability is the need.** Two medium machines keep half their capacity after one fails; one large machine keeps none.
- **Spiky load.** With peak 5× average, a peak-sized machine sits roughly **80% idle** most of the day.
- **Already near the top of the catalog.** The highest unit price for the last 2× only postpones a visible wall; spend it moving to horizontal.
- **Software can't use the hardware** (single-threaded, lock contention). Profile before you buy.
- *The more common misuse is the reverse:* sharding and running a fleet at **50 requests/second** because big companies blog about it.

In the interview. Rarely asked by name, but interviewers watch whether you consider it before going distributed: measure the bottleneck, check whether a bigger instance buys enough time, and use that time to strip state from the app so horizontal scaling is ready before the ceiling.

30-second answer: The bigger machine: more cores and RAM, no architecture change, no new failure modes — transactions and single-machine simplicity keep working, with about 64× of room in one setting. Limits: a biggest machine with a steep price curve before it, and no help for availability or elasticity — still one machine, now with a bigger blast radius. I scale up to buy time and use it to make the service stateless.

- *Follow-up: "Why not keep buying bigger forever?"* — Physical limits (the catalog ends), economic (unit prices climb while each extra core helps less), architectural (no redundancy or hour-by-hour elasticity).
- *Follow-up: "One big machine or three small ones, same capacity?"* — For a stateless service, three: one failure costs a third, not everything. One large machine when the workload needs single-machine guarantees or memory locality, like a primary database. Equal capacity ≠ equal availability.
- *Mistake that fails candidates:* distributed infrastructure for a one-machine problem — or a bigger machine after an *availability* incident.

Self-check:

- Q1: What does vertical scaling preserve that horizontal designs must rebuild, and what are its two hard limits? → Transactions, strong consistency, joins, locality, simple in-memory state, no distributed coordination. Limits: finite hardware (rising prices, falling software benefit at the top); still one machine (no redundancy, no elasticity, downtime to resize).
- Q2: A primary DB runs at 80% CPU growing ~5%/month; half the team wants to shard now. → Scale up one or two sizes first: 2–4× headroom within a week, lasting years at 5%/month. Use the time to

choose a shard key, fix queries that won't survive partitioning, and set a migration trigger (e.g., sustained 70% CPU on the largest reasonable instance) — sharding as a last resort.

- Q3: A payments service on one large instance at 30% utilization lost its host last month (40 min down); leadership wants a bigger instance. → Wrong fix: capacity was fine, there was only one machine. Use 2+ stateless instances behind a load balancer (after the statelessness checklist) and a primary-replica failover plan for stateful parts. Vertical scaling fixes too little capacity; this was too few *machines*.

Related patterns: Horizontal Scaling (scale up to buy the time to scale out); Sharding (each shard still needs a vertical size); Primary-Replica (the answer to availability incidents); Auto-Scaling (elasticity vertical can't offer).

TL;DR

- **Problem:** the machine is too small — and skipping the simple fix wastes years on distributed complexity.
- **Mechanism:** a bigger machine: ~64× of room, zero architecture change, single-machine guarantees intact.
- **Costs:** a ceiling with a steep price curve, zero redundancy (a bigger SPOF), no elasticity, downtime to resize.
- **Skip it when:** availability is the need, load is spiky, or you're near the top of the catalog.

Horizontal Scaling

Original lesson: Horizontal Scaling on DesignGurus ↗

What it is. "Scaling out": many identical copies of the app server, adding copies as load grows. It works only when every server is **stateless**, so any server can handle any request. *Analogy:* more checkout lanes help at rush hour only because any cashier can serve any customer — not if your groceries are locked in lane 3's back room.

The problem. The store's single, ever-bigger server hits the last lesson's limits: the next size costs too much, and a host failure stopped checkout for **40 minutes**. The fix — **two servers** behind a simple splitter sending half the requests to each — goes live Tuesday morning. By afternoon:

- **Random logouts.** Alice's *session* lives in server A's memory; her next request lands on B, which has never heard of her.
- **Uploaded images 404 half the time.** Photos were saved to the local disk of whichever server received the upload.
- **The UI flips back and forth.** Each server caches *feature flags* and refreshes them on its own schedule.

Two servers aren't hard; the code is full of hidden assumptions of exactly *one* machine — one process holding sessions, one disk holding files, one process running the nightly job. Horizontal scaling means hunting down every one.

Why the obvious fixes fail

- **"Sticky sessions."** The splitter uses a cookie or IP address to keep each user on one server. Logouts stop, and it's fine as a *migration bridge*. But if A dies its pinned users are logged out, heavy users overload one server, deploys wait for pinned users to leave, and it does nothing for local images or flag caches.
- **"Synchronize state between servers."** Copying sessions and caches between A and B builds an accidental distributed database in the app tier, re-creating the storage module's hardest consistency problems. Session clustering and multicast sync were once common; shared stores give clearer ownership.

- **"Deploy carefully and hope."** Random session loss is architectural; careful releases can't fix it.
- All three manage *disagreement between stateful servers*. The pattern removes the cause: app servers own no lasting state.

How it works. A load balancer fronts Server 1…N, and every server uses the same shared back-ends — sessions in Redis, files in object storage (e.g., S3), a shared cache, the database. No server holds anything the others lack.

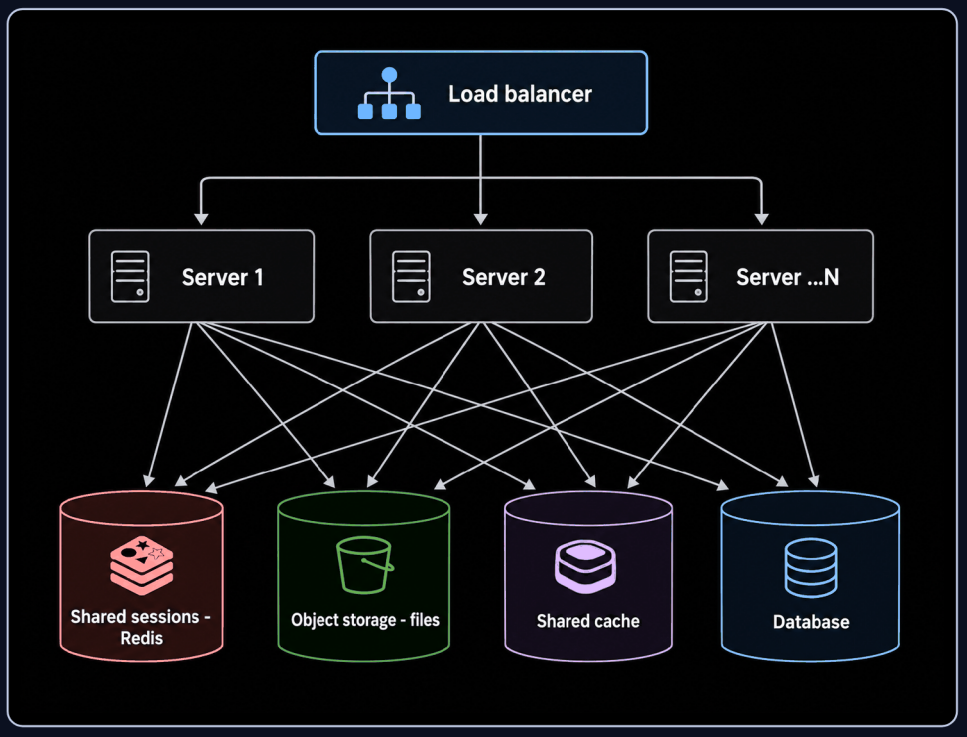


Figure — Horizontal Scaling (course diagram)

Step 1 — the statelessness checklist. Move every value an app server holds to a shared home:

State hiding in the server	Where it moves	Why
Sessions ("who is logged in")	A shared store (Redis), or a signed token the browser carries	Any server recognizes the user
Uploaded files	Object storage (S3 and similar) — never local disk	Every server reads every file
In-process caches	A shared cache (Module 4), or a local one with a short TTL where brief disagreement is harmless	Servers stop disagreeing
Scheduled jobs ("the nightly report")	A job queue or a *distributed lock* (a lock all servers honor)	Otherwise every server runs the cron — N times
Counters, rate limits, locks	A shared store	A per-server counter sees only 1/N of reality

Step 2 — collect the three payoffs.

1. **Capacity by addition** — more servers, with the load balancer spreading requests.
2. **Availability by arithmetic** — run **N+1** (enough for peak plus one spare), so a single failure uses the spare instead of causing an outage.
3. **Uneventful deploys** — a *rolling deploy* swaps servers one at a time with no downtime.

Step 3 — expect the load to multiply. Each server adds downstream connections and requests; databases and caches now carry N servers' load (Connection Pooling and the storage module handle it).

Walkthrough with numbers

- **Sizing.** One server handles **800 req/s**; peak is **4,000 req/s** → 5 needed → run **6** (N+1). The peak estimate already includes traffic headroom; the sixth server is headroom for *failure*.
- **Cost of external state.** Fetching the session from Redis adds about **1 ms** per request — the price of any-server-serves-any-user.
- **A server dies at peak.** Health checks remove it within seconds; the other 5 carry 4,000 req/s. In-flight requests fail once and their retries land on healthy servers — safe because writes are idempotent.
- **Growth.** 10× traffic over two years → about **50 servers**, same architecture, near-linear cost. Capacity is bought, not redesigned.
- **The multiplication arrives.** 50 servers × 20 DB connections = **1,000 connections**, and the database starts refusing clients — the bottleneck moved downstream (→ Connection Pooling).

Horizontal vs. vertical

Figure — Vertical vs. Horizontal Scaling (course diagram)

	Vertical (bigger machine)	Horizontal (more machines)
Ceiling	The catalog runs out (~64× of room)	None in the app tier; it moves downstream
Cost curve	Sharply steeper at the top	Roughly linear (10× traffic, 10× servers)
Availability	None — one bigger SPOF	N+1 — losing a server is routine
Elasticity	No — can't resize hourly	Yes — the fleet grows and shrinks
Prerequisite	None — works on any code	The statelessness overhaul, *finished* (not mostly)
Preserves	Single-machine guarantees, simplicity	Nothing automatically — everything shared is explicit

Horizontal's cost is the up-front overhaul plus ~1 ms per request; six servers for 50 req/s is imitation, not engineering — two servers and a failover often suffice. The two work together: scale up to buy time, remove state, scale out when measurements justify it, and keep tuning size and count (6 large vs. 24 small is a tuning choice, not a principle).

Trade-offs

- *You gain:* capacity by addition (the ceiling moves downstream); availability through redundancy; elasticity as an option; zero-downtime rolling deploys.
- *You pay:*
  - **The statelessness overhaul** — code and testing for sessions, files, caches, scheduled work and counters, paid *before* the first extra server works correctly. The main cost.
  - **~1 ms per request** to fetch external state.
  - **Shared stores take on the criticality.** One Redis holding forty servers' sessions is a possible SPOF needing replication. State is *relocated*, not eliminated.
  - **Coordination problems** — work that ran once on one machine (crons, global counters) needs explicit coordination.
  - **Multiplied downstream load**, every time.

When not to use it

- **Workloads stateful by nature.** A database's job *is* owning state (use storage patterns). WebSocket gateways hold live connections and need a two-tier design (local sockets + global routing). Stateless copies are for compute.
- **Small scale.** Two servers with failover may suffice — though doing the checklist early is cheapest.
- **Bottleneck downstream.** More app servers just pile connections and queries onto a struggling database. Measure first.
- **Microsecond latency budgets** that can't afford a hop to external state may deliberately keep state in-process.
- *The classic failure:* a half-done migration — one forgotten cron, local counter or in-memory rate limiter that runs N times or sees 1/N of reality.

Real-world examples

- **The Twelve-Factor App** (Heroku's manifesto): factor VI, "execute the app as stateless processes," is this pattern in one line.
- **Kubernetes ReplicaSets and AWS Auto Scaling Groups** make it a setting: `replicas: 50` assumes every *pod* (deployed unit) is interchangeable.
- **Every modern web tier** looks like this; the interesting decisions are where sessions, files and other state ended up.
- **For AI engineers:** inference services run identical model copies behind a load balancer, with two twists. Each copy may load tens of GB of *weights* before serving (minutes of startup, which changes auto-scaling). And a conversation's *KV cache* (the model's working memory of earlier tokens) works best on the same copy, so requests aren't fully interchangeable — *consistent-hash* routing by session keeps a conversation on its cache. Batch embeddings and evaluations fit the simple pattern: independent parallel workers.

In the interview. After "add servers," expect "what stops you from doing that today?" List every form of server-local state instead of assuming more copies means more capacity.

30-second answer: Identical, stateless servers behind a load balancer, so any request can land anywhere. The real work is the checklist: sessions to Redis or signed tokens, files to object storage, shared caches, crons behind a lock or queue, counters in a shared store — state makes a server special, and special servers can't be swapped. Then capacity is addition, availability is N+1, deploys are rolling. Two costs up front: shared stores take on the criticality (the

session Redis needs replication), and each server multiplies database load, which pooling and caching absorb.

- *Follow-up: "A server dies mid-request — what do users see?"* — Health checks pull it within seconds; in-flight requests fail once and idempotent retries reach healthy servers; N+1 sizing carries the full load.
- *Follow-up: "You doubled the fleet and latency got worse. Why?"* — The app tier wasn't the bottleneck; more servers multiplied DB connections, queries and lock contention. Fix the data path (storage, caching, a pooler).
- *Mistake that fails candidates:* "add servers" without checking for state, sticky sessions as the permanent design, or scaling a tier that wasn't the limit.

Self-check:

- Q1: The five kinds of state commonly left in app servers and where each goes? → Sessions → Redis or signed tokens; files → object storage; in-process caches → shared cache (or short local TTL); scheduled jobs → queue or distributed lock; counters/rate limits/locks → shared store.
- Q2: For and against sticky sessions; when to remove them? → For: a cheap bridge that stops random logouts while shared sessions are built, and genuinely needed for stateful connections (WebSocket gateways). Against: a dead server drops its pinned users, heavy users unbalance load, deploys drain slowly, new servers start empty. Remove them once sessions live in shared storage or tokens — before more code relies on pinning.
- Q3: 6 → 30 servers; the DB refuses connections at peak and customers get the nightly summary email several times. → 5× servers multiplied DB connections past the limit: add a pooler and shrink per-server pools. The email cron runs on every server: move it to a single-consumer queue or behind a distributed lock, idempotent on customer + date. Statelessness means finishing *every* checklist item.

Related patterns: Vertical Scaling (buys time; stays a sizing option); Load Balancing; Auto-Scaling; Connection Pooling (caps N servers' DB connections); Idempotency (safe retries after a server dies).

TL;DR

- **Problem:** one machine has a ceiling and is a SPOF — but adding a second exposes every hidden one-machine assumption (sessions, files, caches, crons, counters).
- **Mechanism:** the statelessness checklist moves server memory to shared homes; identical servers behind a balancer then give capacity by addition, N+1 availability, rolling deploys.
- **Costs:** the overhaul, ~1 ms per request, shared stores that become critical, new coordination problems, multiplied downstream load.
- **Skip it when:** the state *is* the job (databases, socket gateways), the scale is small, or the bottleneck is downstream.

Load Balancing

Original lesson: Load Balancing on DesignGurus ↗

What it is. A dedicated front-door layer that gives clients **one stable address** for a changing fleet, checks which servers are alive (**health checks**), spreads requests with an algorithm that reflects real load, and **drains** servers (lets current work finish) before replacing them. *Analogy:* a restaurant host who knows which tables are free, which waiter is swamped and which section is closing. Basics (L4/L7, algorithms) are in Module 0 — Load balancers; this lesson is the pattern.

The problem. Six stateless servers sit behind Tuesday's **round-robin** splitter (request 1 → server 1, request 2 → server 2, …). At **2:14 AM** a runaway log fills server 4's disk. It doesn't crash — it slows to **20 seconds** per request before failing. The health-blind splitter keeps sending it **one-sixth** of

traffic for **11 minutes**, until an engineer pulls it by hand. Two weeks later, replacing server 2 in a deploy kills **several hundred in-flight requests** because the splitter can't stop new work while letting current work finish — a predictable error spike every deploy. The flaw: the splitter treats servers as interchangeable *and permanent*, but real servers slow down, die and get replaced, and the front must notice within seconds.

Why the obvious fixes fail

- **"DNS round-robin."** Clients and resolvers cache answers for minutes or hours, so a removed server's address keeps being used. DNS publishes addresses; it can't steer individual requests, measure health, or revoke a cached choice.
- **"Let clients pick a server."** Service meshes legitimately do a capable version internally, but public browsers and apps would need health checks, retries and list management you can't reliably update.
- **"A runbook: on-call removes bad servers."** That *is* the 11-minute response — the right decision at human speed (like the Circuit Breaker lesson's manual kill switch). This pattern makes it at machine speed.

How it works — four jobs. The figure shows a balancer mid-work: one address for clients, health checks every 2 s, least-connections routing, drain before replace. Servers 1, 3, 5, 6 serve (41, 38, 44, 39 in flight); server 2 is *draining* for a deploy (6 finishing); server 4 was *removed* after 3 failed probes; the next request goes to server 3 (fewest in flight).

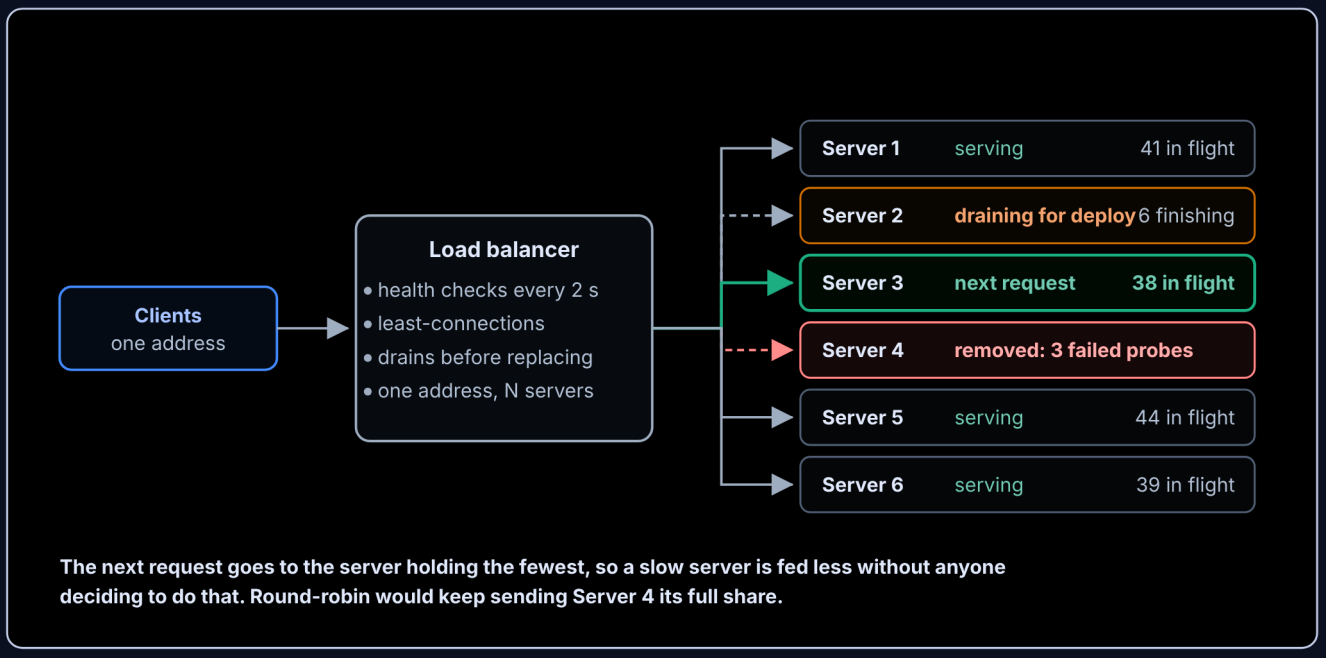


Figure — Load Balancing (course diagram)

1. **Health checks — noticing failure.** *Active* checks call each server's health endpoint every few seconds (the Operations module designs it); several consecutive failures remove a server, several successes restore it. *Passive* checks watch real traffic and react to error jumps between probes. Together: the 11-minute manual response becomes about **6 seconds**.
2. **The algorithm — spreading traffic.**
   - *Round-robin* — fine when requests and servers are alike.
   - *Least-connections* — the server with the fewest in-flight requests gets the next one. The better default: a struggling server accumulates in-flight work and automatically gets less (fixing 2:14 AM).
   - *Weighted* — servers of different sizes.
   - *Hash-based* (consistent hashing, "the ring") — same user to same server when *affinity* matters (warm per-user caches, long-lived connections).

3. **Draining — the deploy story.** Stop new requests to the leaving server, let current ones finish (seconds for API calls; held-open streams get a notice before disconnect), admit the replacement only after it passes health checks. The deploy error spike disappears.
4. **One address.** A stable endpoint while the fleet changes size and membership (what makes auto-scaling possible). The balancer often *terminates TLS*. **L4** balancers route by IP and port (connection level); **L7** read HTTP paths, headers and cookies (request level), enabling path routing and request-aware retries. Public entry points usually use L7; internal tiers may prefer simpler L4.

Who balances the balancer? It's on every request's path, so it can't be a SPOF: a **redundant pair sharing a floating IP** (the address moves to the survivor), or at internet scale **anycast** — many balancers announce one IP and the network sends each client to a nearby one. That ends the chain after one level.

Walkthrough with numbers

- **2:14 AM, replayed.** Probes fail at 0, 2 and 4 s; server 4 is removed at ~6 s (passive monitoring may be faster). One-sixth of traffic sees a short blip; no human involved.
- **The GC pause.** Server 5 freezes in garbage collection — slow, not failed. Round-robin keeps feeding it; least-connections sees in-flight rise from **40 to 300** and routes elsewhere until it recovers.
- **The deploy.** Six servers replaced one at a time: drain **~30 s**, start the replacement, admit on passing probes. The SSE (long-lived streaming) fleet uses longer drains plus *jittered reconnects* (random per-client delays so clients don't stampede back). Zero deploy errors.
- **The math check.** The balancer adds **well under 1 ms** and handles **4,000 req/s** easily. Its value is replacing slow human detection, not speed.

Trade-offs

- *You gain:* machine-speed failure handling; distribution matching reality (least-connections self-corrects for slow servers and uneven requests); zero-downtime deploys via draining; one stable address hiding fleet churn.
- *You pay:*
  - **A critical hop on everything.** Sub-millisecond, but *all* traffic depends on it, so its availability matters more than any server's — hence pairs or distributed deployments.
  - **Health checks wrong in both directions.** A *shallow* check (process running?) may pass a server that can't reach its database. A *deep* check on a shared dependency is worse: when that dependency blips, *every* server reports unhealthy and the balancer removes the whole fleet. Rule: a health check answers "should *this server* receive traffic?", not "is the architecture OK?"
  - **Small settings, big consequences** — probe intervals, thresholds, drain times, algorithms.
  - **L7 costs** — inspecting every request is more work; at very large scale the balancer tier needs its own horizontal capacity.

When not to use it

- **One server** — latency and config with no redundancy (a proxy may still do TLS or routing; see the API & Edge module).
- **Service-to-service traffic inside a mesh** — sidecar proxies balance client-side with rich health data: the same jobs, decentralized.
- **Geographic routing** — happens at DNS/anycast *before* any regional balancer; a complement, not a replacement.
- *Common misuse:* a balancer product left on round-robin defaults, shallow checks and no drain integration — the old splitter's failures plus infrastructure. The pattern is four active jobs, not a box.

Round-robin vs. least-connections

	Round-robin	Least-connections
Assumes	All requests and servers are alike	Nothing — it measures
A slow/failing server gets	Its full equal share (the 2:14 AM failure)	Less and less as its backlog grows
Mixed light/heavy requests	Some servers idle, others overloaded	Corrects itself
Long-lived connections (SSE, sockets)	Bad — balances connection *starts*, not load	The right default — counts what's held
Cost	Trivial	Trivial (one counter per server)

Round-robin still suits short, uniform requests and DNS (which can't see active work); hash-based routing is the third option for cache locality or affinity. In one line: *round-robin assumes traffic is uniform; least-connections measures it.*

Real-world examples

- **nginx and HAProxy:** long-established software balancers with L4/L7 modes and all the algorithms above.
- **AWS: ALB** is L7 (paths, headers, WebSocket upgrades); **NLB** is L4, for millions of connections with microsecond overhead. Other clouds have similar pairs.
- **Envoy:** a small L7 proxy next to every service (a service mesh), giving every caller request-aware balancing and local health data.
- **Anycast at the edge:** Cloudflare and other CDNs announce one IP worldwide; internet routing does the first layer of balancing.
- **For AI engineers:** connection counts misread inference load — two LLM requests can differ **100×** (10 vs. 2,000 tokens), so a GPU with three long generations can be busier than one with ten short ones. Inference routers balance on **queue depth and outstanding tokens** per replica, plus consistent-hash routing by conversation ID to reach the replica holding its KV cache. Still "measure the work," by size rather than count.

In the interview. Everyone draws a load balancer; interviewers check whether you can explain how it chooses, detects failure, handles deploys, and whether it's a SPOF itself.

30-second answer: Four jobs: health checks (active probes plus real errors) remove a failing server in seconds; least-connections distribution self-corrects for slow servers where round-robin keeps feeding them; draining lets in-flight work finish during deploys; one stable address for a changing fleet, usually terminating TLS at L7. It runs as a redundant pair with a floating IP, or anycast at scale. Health checks stay deliberately shallow — a deep check on a shared dependency can pull the whole fleet — so each server reports only its own health and degrades rather than dies on dependency failures.

- *Follow-up: "What should the health check test?"* — The process, the listening port and critical *local* resources. Not every downstream dependency; handle those in-service with breakers and degraded responses.
- *Follow-up: "Who balances the balancer?"* — A redundant pair on a floating address failing over in seconds; at larger scale, anycast.
- *Mistake that fails candidates:* round-robin without thinking about slow servers, restarts without draining, or a balancer that "can't fail."

Self-check:

- Q1: The four jobs and the two ways a health check can be wrong? → Health checking, load-aware distribution, draining, one stable endpoint (often L7 TLS termination). Shallow: keeps a server that responds but can't work. Deep: removes healthy servers when a shared dependency fails — the whole fleet over an outside problem.
- Q2: Algorithm for (a) a uniform internal API under 10 ms, (b) a public API with wildly varying request cost, (c) an SSE fleet holding 50k streams per server? → (a) Round-robin. (b) Least-connections, steering away from servers piling up expensive requests. (c) Least-connections when placing streams (held connections *are* the load), plus long drains and randomized reconnect delays.
- Q3: At 3:07 AM the site goes down; all six servers show low CPU and no errors; the balancer marks all unhealthy at once and restores them two minutes later. → The health endpoint tests a shared dependency (Redis or the DB); a two-minute blip failed every probe. Restrict the check to process, port and local conditions; handle shared-dependency failures in-app with breakers, fallbacks, reduced responses.

Related patterns: Horizontal Scaling (statelessness lets any request go anywhere); Auto-Scaling (adds/removes servers behind this entry point); Consistent Hashing (affinity routing); Health Check Endpoint (what the probe hits); Graceful Degradation (reduced answers instead of dropping out of rotation).

TL;DR

- **Problem:** a simple splitter treats servers as interchangeable and permanent — feeding a failing machine its full share for minutes, killing in-flight work every deploy.
- **Mechanism:** four jobs — active + passive health checks, least-connections, draining, one stable (L4/L7, TLS-terminating) address — itself made redundant with pairs or anycast.
- **Costs:** a critical hop on all traffic, health checks that mislead both ways (the deep-check trap removes the fleet), small settings with big consequences.
- **Skip it when:** one server, traffic inside a mesh (sidecars balance it), or the decision is geographic (DNS/anycast).

Auto-Scaling

Original lesson: Auto-Scaling on DesignGurus ↗

What it is. A **control loop**: measure a load signal, compare it with a target, add or remove servers to close the gap, with the load balancer admitting new servers and draining leaving ones. Fleet size becomes a continuously corrected answer instead of a one-off guess. *Analogy:* a thermostat, not a January guess at the whole year's heating.

The problem. A fixed fleet of **8 servers** is sized for a **4,000 req/s** evening peak. One month, two incidents:

- **The 4 AM bill.** Average traffic is about **900 req/s** (evenings 4–5× that), so five or six servers idle most hours at full price — roughly **60% of the app tier's spend pays for idle time**.
- **The Tuesday the TV mentioned you.** Traffic leaps from 900 to **12,000 req/s in twenty minutes** — 3× the planned peak. All eight max out; by the time an engineer wakes, diagnoses, provisions and boots, much of the spike is over.

Fixed sizing is a design-time estimate of something that changes hourly, so it's wrong both ways.

Why the obvious fixes fail

- **"Size for the worst case."** 24 servers always on handles the TV spike but triples the 4 AM bill — and a growing business makes bigger surprises.
- **"On-call scales by hand."** Notice-diagnose-act-boot ate most of a 20-minute spike, and nobody gets up at 4 AM to *remove* servers. People are slow going up and absent going down.
- **"Schedule it."** More at 6 PM, fewer at midnight is useful (it'll be part of the design) but can't see a TV segment or a quiet evening coming.

How it works. Measure every ~1 minute → compare with a target (e.g., 65%) → over: **add fast**; under: **remove slowly, draining first** → measure again.

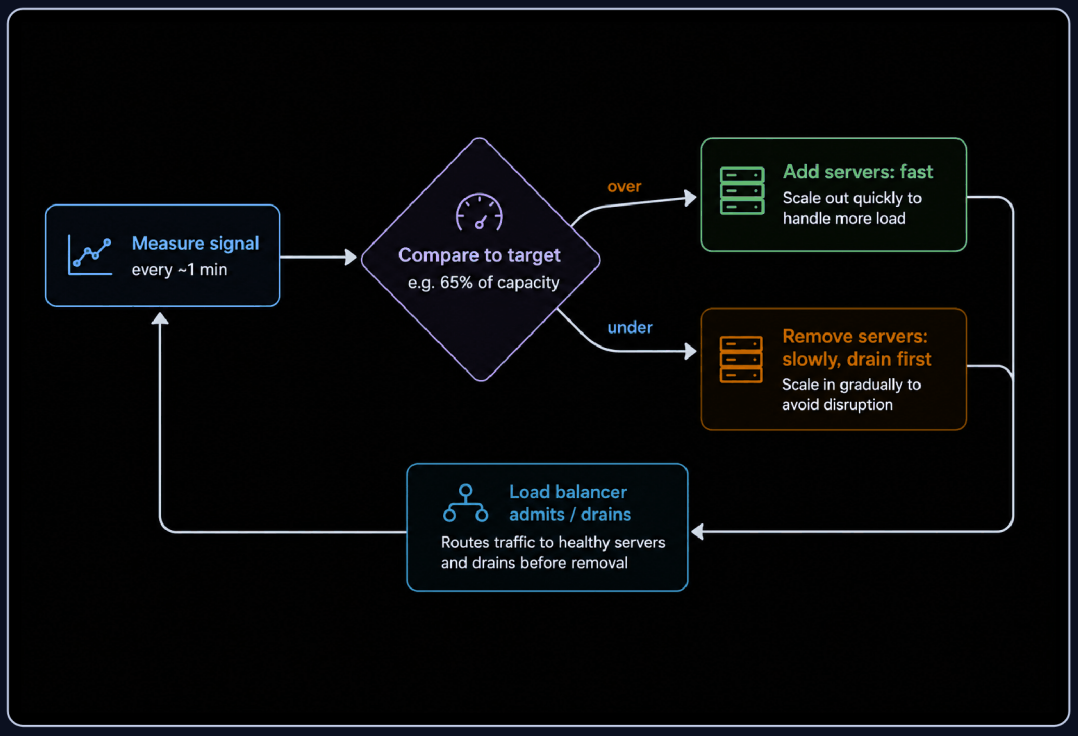


Figure — Auto-Scaling (course diagram)

1. **Pick the signal — *the* design decision.** CPU works for *compute-bound* services. It misleads for *IO-bound* ones (requests mostly waiting on disks, networks, databases): hundreds of blocked threads hurt users at **15%** CPU and a CPU scaler does nothing. Better: **requests per instance** against tested capacity, or **latency** (e.g., p95 — the time within which 95% of requests finish; see percentiles) against target. For queue workers, **backlog age** — if the oldest job has waited **90 seconds**, you have too few workers.
2. **Target with deliberate headroom — about 60–70%, not 90%.** The unused slice covers climbing traffic until new servers are ready.
3. **Respect reaction lag — it's physics.** ~1 minute of metric averaging, then boot, app start, cache warm-up: **3–7 minutes with no new capacity** after a spike starts. Headroom covers it; a fleet at 90% is overloaded before help arrives.
4. **Scale up fast, down slowly.** Add several at once (an unneeded addition is cheap); remove one at a time after a *cooldown* (wait after any action) and a drain. Aggressive removal triggers instant re-adds — **flapping**, rebuilding caches and DB connections each time. The cure: asymmetry plus **hysteresis** (separate up and down thresholds) and cooldowns.
5. **Set a minimum and a maximum. Min = N+1**, so cost-cutting never removes failure headroom. **Max = measured downstream capacity**, not an arbitrary big number — an uncapped scaler adds connections and queries until it flattens the database. The max is a **bulkhead** (a hard limit stopping one part's trouble flooding another) protecting downstream; traffic beyond it is deliberately shed, queued or rate-limited at the edge.
6. **Combine triggers.** *Scheduled* for known cycles (start before the 6 PM ramp — zero lag), *reactive* for surprises, *predictive* (forecast from history) for large fleets or slow boots. Calendar for what you

know, measurement for what you don't.

Walkthrough with numbers (800 req/s per server; a 65% target → **~520** effective; min 3, max 20)

- **4 AM, 900 req/s:** 900 / 520 ≈ 2, raised to the N+1 floor → **3 servers**.
- **6 PM, scheduled: 8 servers** ready when 4,000 req/s arrives; zero reaction lag.
- **TV Tuesday, replayed (900 → 12,000 in 20 min):** headroom absorbs the first **~90 seconds**; the reactive loop adds waves (~60 s to notice, ~3 min to boot); **16** servers by minute six, capped at **20** — the database (with replicas and cache) is known to cope with 20 servers' traffic and no more. Overflow is shed at the edge. Bad minutes: about **four**. Human actions: **zero**.
- **The bill:** average fleet drops from a fixed 8 to about **4.2** — roughly **half the compute spend**. Scaling up protected users; scaling down paid for the project.

Trade-offs

- *You gain:* a bill tracking demand; surprises absorbed at machine speed (limited by boot time and max); free self-healing (the loop replaces crashed instances to keep the minimum); far fewer 2 AM capacity pages.
- *You pay:*
  - **Reaction lag, always** — 3–7 minutes; scheduling and headroom soften it but can't remove boot time. Spikes shorter than the lag need traffic controls at the front.
  - **Wrong-signal disasters.** CPU misses IO-bound incidents; a memory-based policy may keep adding servers to hide a memory leak. Choosing a signal is a diagnosis; automating the wrong one amplifies it.
  - **Flapping and churn** — close thresholds, no cooldowns: nonstop changes, cold caches, connection bursts.
  - **An automated flood downstream** — every scale-up adds unreviewed downstream traffic; the max, pooling and caching must protect the database from its own scaler.
  - **Policies are code running overnight** — they need review, observable metrics and load tests that actually trigger them.

When not to use it

- **Flat, predictable traffic** (an internal service at a steady 200 req/s): a fixed N+1 fleet; an unneeded loop is one more failure mode.
- **Stateful tiers.** Databases and socket gateways aren't interchangeable compute; DB capacity needs careful storage changes (managed DB auto-scaling does this slowly, within explicit limits).
- **Downstream can't take it.** If the DB is the bottleneck, auto-adding app servers worsens the overload.
- **Boot times longer than the spike.** A 15-minute boot can't answer a 10-minute event; AI serving needs scheduled/predictive capacity and warm pools.
- *Common misuse:* auto-scaling instead of measuring capacity — unknown per-instance throughput, no max, CPU only. Fine until an unusual day, when the fleet changing size mid-incident also hampers diagnosis.

Reactive vs. scheduled vs. predictive

	Reactive	Scheduled	Predictive
Trigger	The signal, right now	The calendar	A model's forecast
Known daily cycle	Late by the lag, every time	Perfect — capacity before load	Well
The TV moment	Yes — limited by lag + headroom	No — not on the calendar	Only if it resembles history
Complexity	Moderate	Trivial	High — a model you must trust
Failure mode	Wrong signal, flapping	Wrong guess, quiet drift	Wrong with confidence

Stack them: schedule the known, react to the actual, add prediction when boot delay or fleet size justifies it.

Real-world examples

- **AWS Auto Scaling Groups** with *target tracking*: signal, target, min, max — the whole loop.
- **Kubernetes HPA** (Horizontal Pod Autoscaler) scales pods on custom signals; a separate **cluster autoscaler** adds/removes the machines under them — two stacked loops.
- **Queue-driven workers:** SQS queue depth drives AWS Lambda concurrency — backlog as signal, near-zero startup, zero workers when idle.
- **Netflix Scryer:** a predictive scaler provisioning ahead of forecast demand, because reaction lag is very costly at Netflix scale.
- **For AI engineers:** the signal is queue depth and outstanding tokens, not CPU. Pulling a container and loading tens of GB of weights takes minutes, so purely reactive scaling fails: use warm pools, predictive changes and a realistic scale-to-zero decision (idle GPUs are expensive; cold starts make users wait). The max may be a **provider quota**, so front-end shedding and queueing are a must.

In the interview. Typical setup: predictable evening peaks plus occasional 10× marketing events. Schedule the cycle, react to surprises, keep headroom for boot time, derive the max from downstream limits. After "it auto-scales," expect "on what signal?"

30-second answer: A control loop keeping a signal near ~65%: add fast, remove slowly through the balancer's drain. The signal is the real decision — CPU for compute-bound work; requests per instance or latency for IO-bound work, where blocked threads make CPU misleading; backlog age for queue workers. Headroom covers 3–7 minutes of reaction lag; schedules put capacity ahead of the 6 PM ramp. Min is N+1, max is what the database survives — an uncapped scaler is an automated stampede — and beyond the max the front sheds load.

- *Follow-up: "IO-bound API — what signal, and what if you'd chosen CPU?"* — Requests per instance against tested capacity, or p95 latency. With CPU, a slow database piles up blocked threads at ~15% CPU and the scaler does nothing while users suffer.
- *Follow-up: "Flash sale: the scaler hits max and the site is still slow."* — Find the active bottleneck before touching the cap. If the database is saturated the max is doing its job; improve caching, replicas and pooling, reject or delay excess at the edge, and raise the max only after measuring more downstream capacity.
- *Mistake that fails candidates:* CPU for everything, no max, ignoring lag and headroom, or treating a database as an interchangeable stateless tier.

Self-check:

- Q1: Parts of the loop, and why different up/down policies? → Measure, compare with ~65%, add/remove, balancer admits/drains, repeat. Up fast because surplus is cheap versus an outage; down slowly (cooldowns, one at a time) because aggressive cuts cause flapping, cold caches and connection bursts.
- Q2: Signal for (a) an IO-bound web API, (b) a queue-consuming image processor, (c) a GPU inference fleet? → (a) Requests per instance or p95 latency. (b) Backlog age. (c) Queue depth or outstanding tokens per replica, plus warm/predictive capacity for multi-minute startup.
- Q3: Min 3, max 30, CPU target 65%, scale-down at 60%, no cooldown; the fleet bounces between 6 and 9 all evening, then *shrinks* during a database slowdown. → Thresholds 5 points apart with no cooldown: noise keeps undoing the last action. Widen (e.g., up at 70%, down at 40%), add cooldowns, remove one at a time. During the DB slowdown threads waited instead of burning CPU, so the signal read "spare capacity" — switch to requests per instance or latency.

Related patterns: Horizontal Scaling (only stateless servers can be added/removed automatically); Load Balancing (admit and drain); Connection Pooling (absorbs each scale-up's connection surge); Message Queue (backlog age, the clearest signal); Bulkhead (the fleet max).

TL;DR

- **Problem:** fixed fleets pay for idle servers at 4 AM and drown when the TV mentions you.
- **Mechanism:** a control loop keeping a signal near ~65%, add fast / remove slowly with draining; scheduled for known cycles, reactive for surprises; min N+1, max from downstream capacity.
- **Costs:** 3–7 min reaction lag (headroom bridges it), wrong-signal disasters (CPU misleads for IO-bound work), flapping without hysteresis/cooldowns, an automated downstream flood only the max holds back.
- **Skip it when:** traffic is flat, the tier is stateful, downstream is already the bottleneck, or boot time dwarfs the spikes (schedule, predict, pre-warm).

Database Connection Pooling

Original lesson: Database Connection Pooling on DesignGurus ↗

What it is. Opening a DB connection is expensive; querying over an open one is cheap. A pool opens a few connections once, keeps them alive, and lets requests *borrow* one per query and *return* it. At fleet scale a shared **pooler** in front of the database funnels many servers' connections into a few real ones. *Analogy:* a company car pool instead of everyone buying a car for a ten-minute errand. (Basics: Module 0 — Connection pooling, ORMs and N+1 queries.)

The problem — part 1: setup cost. A new connection needs a TCP handshake, TLS negotiation and authentication — about **20–50 ms** before the first query — and the database reserves real resources per connection (Postgres runs a dedicated *process* with its own memory for each). That's ten times the work of a **5 ms** query.

The problem — part 2: multiplication. Frameworks usually solve part 1 with an app-side pool. The less obvious problem appears as the fleet grows: a sensible 20-connection pool × 50 servers = **1,000 connections** against a database built for a few hundred (Postgres's default limit is around **100**), each process eating memory and scheduling time. When the auto-scaler adds five servers in a spike, they immediately request another 100. The app tier gains capacity while the unchanged database turns clients away — "scaling one tier multiplies load on the next" in its most common form.

How it works. The figure shows the app-side pool: (1) the app asks the in-memory pool for a connection from the *idle* set; (2) it moves to the *in-use* set; (3) the query runs on the database server

(MySQL, PostgreSQL, SQL Server, Oracle…); (4) the connection returns to the idle set. Nothing is opened or closed per query.

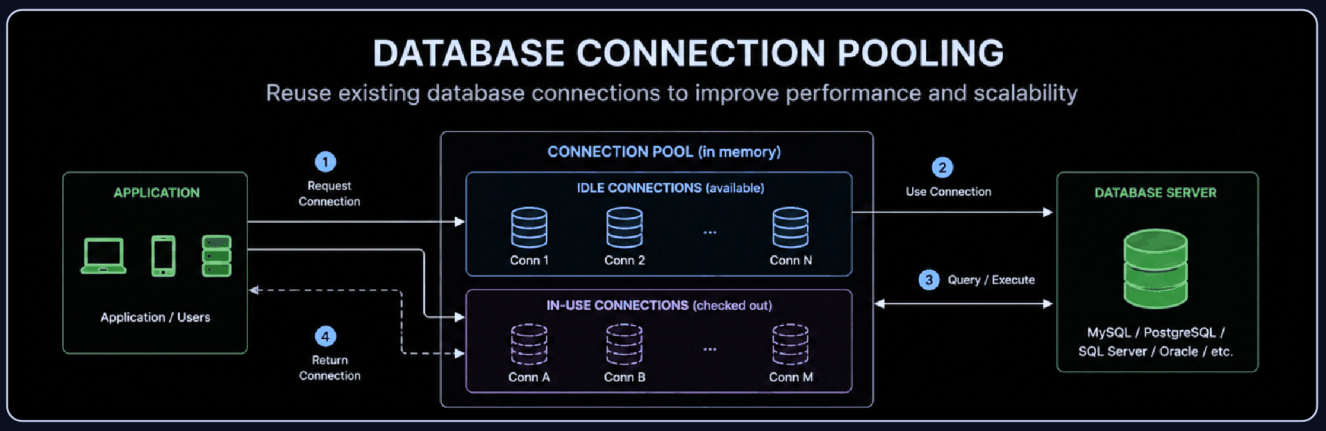


Figure — Database Connection Pooling (course diagram)

1. **Layer 1 — an app-side pool per server.** Each server opens perhaps **10–20** connections at startup; queries borrow and return within microseconds, so the 20–50 ms handshake is paid once per connection *lifetime*.
2. **Layer 2 — a shared pooler.** A proxy such as **PgBouncer** (or managed **AWS RDS Proxy**) accepts, say, 1,000 cheap *client* connections and **multiplexes** them (many clients taking turns on few real connections) over about **50 real database connections**, lending one to whichever client is in a transaction now.

```
Without pooler:  50 app servers × 20 conns ──▶ 1,000 conns ──▶ DB (overloaded)
With pooler:     50 app servers × 20 conns ──▶ PgBouncer ──▶ ~50 conns ──▶ DB (calm)
                 (cheap client connections)   (multiplexes)   (sized to cores)
```

3. **Why so few suffice (Little's law — Module 0).** Active connections = query rate × query duration: **400 queries/s** × **5 ms** = **2** busy at any instant. A thousand *logical* connections, about two doing anything.
4. **Small pools beat big ones.** A database runs roughly one query per CPU core at a time; connections beyond that add no processors — their queries wait while adding context switches and lock contention, so throughput peaks at a modest size and can *fall* beyond it. HikariCP's (a popular Java pool) guidance: about **cores × 2** — tens, not thousands. If that feels too few, fix slow queries or add a cache or read replicas, not connections.
5. **Transaction-mode fine print.** Aggressive poolers multiplex *per transaction*, so features assuming a private, lasting session — **session variables, temporary tables, some prepared-statement setups** — can break. Check compatibility notes before migrating.

Trade-offs

- *You gain:* the handshake paid once per lifetime; a database seeing tens of warm connections instead of a fleet-sized flood (scale-ups stop being connection storms); a **built-in bulkhead** — the pool caps concurrent queries, giving *backpressure* (push-back that slows callers when downstream is full) by design.
- *You pay:*
  - **Pool exhaustion — a new way to be down.** If connections stay borrowed (slow queries, or code that *leaks* them — borrows without returning), requests queue for the pool; the queue moved from the database into the app or proxy. Guardrails: a **borrow timeout** (fail fast), alerts on **pool utilization and wait time**, **leak detection**.
  - **Housekeeping** — detect and replace dead connections with *validation queries* and a *maximum connection lifetime*; a shared pooler is another critical service, so keep it simple and redundant.
  - **The transaction-mode limits** above.

When not to use it — pooling is nearly universal; the questions are *where* and *how big*:

- **Serverless functions** (e.g., AWS Lambda) can't keep reliable local pools — instances are short-lived and thousands may start at once. Pool in a shared proxy outside them; **RDS Proxy** exists to absorb Lambda-style bursts.
- **Small scripts and one-off jobs** with one connection gain nothing.
- **Not a valid exception:** "the pool keeps running out, so make it 500." That moves the queue into the database, where contention *lowers* throughput.
- Related habit: **separate pools for the primary (writes) and replicas (reads)**, so report traffic can't eat checkout's connections.

In the interview. Usually a diagnosis: "the app tier grew from 10 to 50 servers and the database refuses connections — why?" Or "how big should the pool be?", testing whether you reach for a huge number.

30-second answer: Each connection costs a 20–50 ms handshake and real DB memory, so servers keep warm pools. The trap is multiplication — 50 servers × 20 is 1,000 — so at fleet scale a pooler like PgBouncer multiplexes down to tens of real connections, safe because rate × duration shows only a handful are busy. Pools should be small, near the database's core count; extra connections just queue inside the DB and add contention. I alert on pool wait time, because exhaustion is its own outage mode.

- *Follow-up: "Why does a bigger pool make it slower?"* — About one query per core; extra connections add context switches and lock contention, not processors.
- *Follow-up: "Where should the pool live with auto-scaling?"* — Both levels: local pools for reuse, plus a shared proxy capping the *total* however many servers the scaler adds.
- *Mistake that fails candidates:* raising the DB limit to 5,000 as if connections were capacity, or no borrow timeout (slow queries become hung threads fleet-wide).

Self-check:

- Q1: Why is opening a connection expensive, and which two layers does a scaled fleet need? → TCP + TLS + auth ≈ 20–50 ms plus a dedicated DB process and memory. Layer 1: a local pool per server. Layer 2: a shared proxy (PgBouncer) multiplexing onto core-sized real connections, so sane per-server settings can't multiply into an unsafe total.
- Q2: 400 queries/s at 5 ms; a teammate wants a pool of 200 "for safety." → 400 × 0.005 = **2** active; 10–20 leaves ample burst room. 200 wastes DB processes and memory when idle and, under load, lets 200 queries fight over a few dozen cores. Safer: small pool, short borrow timeout, monitored wait time.
- Q3: After a deploy, p99 latency jumps; requests log "pool borrow timeout" while the DB is nearly idle; rollback fixes it. → A connection **leak**: some path (early return or exception — e.g., release missing from a `finally` block) never returns its connection. Guardrails: leak detection reporting connections held past a threshold with the borrower's stack trace; alerts on pool utilization and wait time before exhaustion.

Related patterns: Horizontal Scaling (N sane pools still multiply); Auto-Scaling (each new server is a connection burst unless a proxy caps the total); Bulkhead (bounded concurrency and backpressure); Primary-Replica (separate read/write pools).

TL;DR

- **Problem:** each connection costs a 20–50 ms handshake and a process's worth of DB memory, and per-server pools multiply into a fleet-sized flood (50 × 20 = 1,000) that auto-scaling grows by itself.

- **Mechanism:** warm app-side pools plus a shared pooler (PgBouncer / RDS Proxy) multiplexing onto core-sized real connections — rate × duration shows only a handful are busy.
- **Costs:** pool exhaustion (borrow timeouts, wait-time alerts, leak detection), housekeeping, transaction-mode limits.
- **Skip it when:** practically never — decide where (a shared proxy for serverless and big fleets) and how big (small, near the cores; never "500 for safety").

Multi-Region and Cell-Based Architecture

Added lesson (not part of the DesignGurus course). Written from primary sources listed under "Sources" at the end of the lesson.

What it is. Two ways to cut one system into pieces that fail separately. **Multi-region** runs it in two or more geographic **regions** (a cloud location such as "US East", made of several **availability zones** — AZs, separate datacenters a few kilometres apart). **Cell-based architecture** splits each region's fleet into **cells**: complete, independent copies of the stack (app servers, cache, DB shard, queues), each serving a fixed slice of customers, with a thin **cell router** sending each customer to their cell. *Analogy:* watertight compartments (cells) and a shipping company with ships in two oceans (regions).

The problem. The capstone grows the store to ~1M req/s in **one region, three AZs** — which survives a datacenter fire but not these, all within a year:

- **The regional dependency.** A provider problem in the region's shared control systems (identity, DNS, the load-balancer API) breaks all three AZs: **4 hours** down.
- **The global config push.** A malformed rate-limit rule reaches every server in **90 seconds**, as designed; checkout fails for **100% of customers**. The fault came from *inside*.
- **The poison tenant.** One business customer's export script saturates the shared order DB; **20 million** others get slow pages for an hour.
- Sydney customers wait **~200 ms per round trip** to the US region on every write.

Every fault's **blast radius** (share of users one failure can hurt) was the whole company, because everything was one shared system.

Why the obvious fixes fail

- **"Add a fourth AZ."** AZs share the regional control plane (the systems that create, configure and route resources, versus the data plane serving user traffic) — and *your* code and config.
- **"Back up to another region."** Necessary, but restoring 50 TB onto an untested stack takes **a day or more** — recovery from losing a region forever, not a 4-hour outage.
- **"Every region active, one globally consistent database."** Every commit would wait **~75 ms** (US East ↔ EU West) to **~200 ms** (US ↔ Sydney) — the cost of linearizability (every reader sees the latest write, as if one copy existed) from DDIA Chapter 9. Right for a few datasets, far too costly for all.
- **"Test config changes better."** Some faults appear only under production traffic. What's missing is a way to hurt *5%* of customers before *100%*.

How it works — failure domains first. A **failure domain** is the set of things that fail together:

Failure domain	Typical cause	What contains it
Process / host	Crash, disk full, GC pause	Horizontal Scaling + Load Balancing (N+1)
Availability zone	Power, cooling, network in one building	Every tier spread across 3 AZs
Region	Regional control plane, large network event	**A second (and third) region**
Your own change	Bad deploy, bad config, schema mistake	**Cells**, deployed one cell at a time
One noisy customer	Pathological workload	**Cells** (or shuffle sharding, below)

Regions answer "what if a location goes away?"; cells answer "what if *we* break something, or one customer does?"

Step 1 — pick a multi-region strategy by RPO and RTO. **RPO** (recovery point objective): how much recent data you can lose. **RTO** (recovery time objective): how long you can be down. (Defined in the SQL book's High availability section.) The cloud providers' four strategies, cheapest to strongest:

Strategy	What runs in the second region	RPO	RTO	Extra cost
Backup and restore	Only backups	Hours (last backup)	Hours to a day	Lowest
Pilot light	Data replicated continuously; servers off or minimal	Seconds to minutes	Tens of minutes (start and scale servers)	Low
Warm standby	A small working copy of everything, taking no users	Seconds	Minutes (scale up, shift traffic)	Medium
Active-active (multi-site)	Full stack serving users all the time	Seconds, or zero for synchronously replicated data	Near zero (traffic moves)	Highest

Choose per system, not per company: checkout may need active-active while internal reporting is fine with backup and restore.

Step 2 — decide where each dataset lives. Servers copy easily between regions; data doesn't. Three kinds:

1. **Home-region data** — each customer's orders, cart and profile live in one **home region** (nearest, or where law requires: EU data stays in the EU). Writes go there and are copied asynchronously elsewhere only for disaster recovery (DR). Local *and* conflict-free, since each record has one region allowed to change it. Most data should be this kind.
2. **Read-mostly shared data** — catalog, prices, feature flags. One region writes; every region holds a read-only copy lagging a second or so — leader-follower replication across continents.
3. **Truly global, strongly consistent data** — unique usernames, the last unit of a limited item, balances spendable anywhere. One global home region, or a cross-region consensus database (Spanner, CockroachDB — replicas vote so a majority agrees on every write) paying a cross-region round trip per write. Keep it small. If two regions both accept writes to one record, you're in multi-leader replication and need a conflict rule (last-write-wins; CRDTs, data types that merge concurrent edits automatically; or the Vector Clocks lesson's siblings).

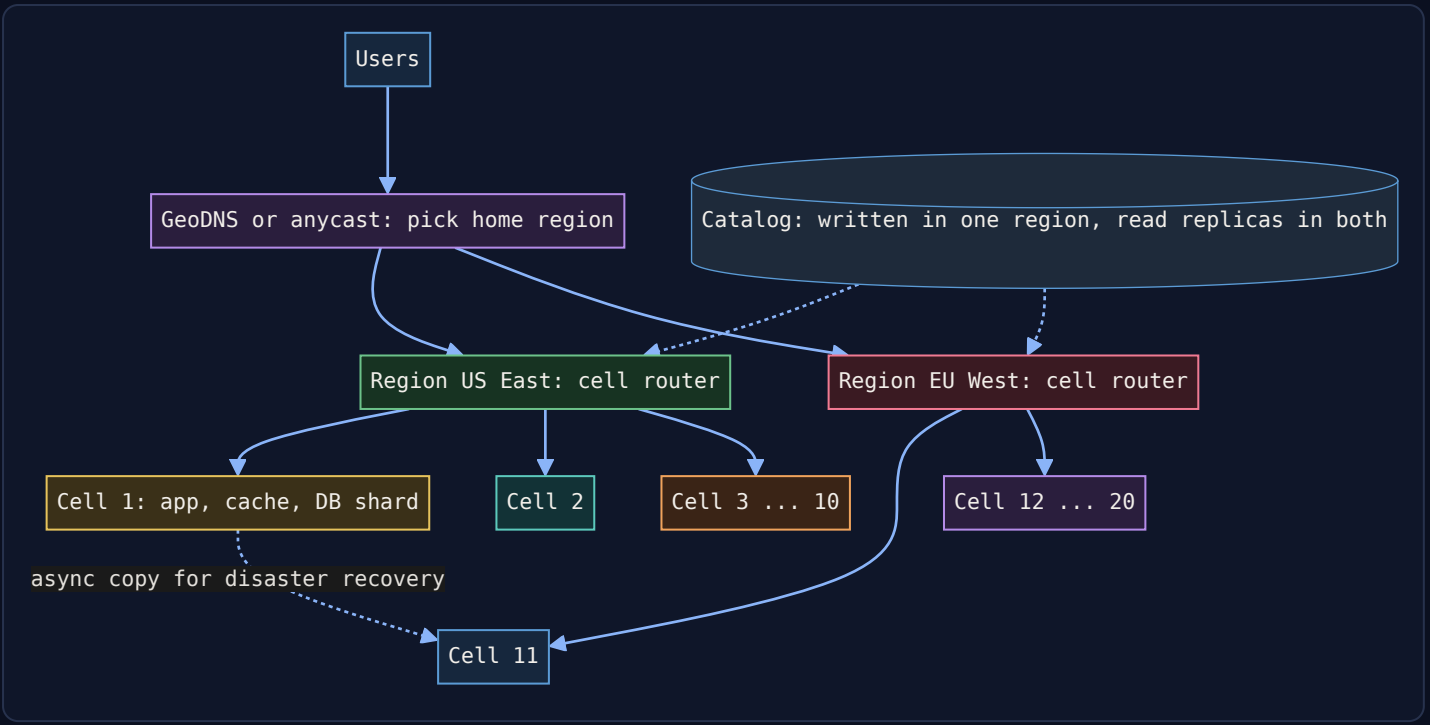
Step 3 — route users to a region. Above the regional load balancer: **GeoDNS** (answer depends on user location) or **anycast**, plus a **global health check** that removes a failed region. DNS failover takes about *TTL + detection* (e.g., 60 s + 30 s); anycast reroutes without waiting for caches.

Step 4 — cells inside each region. A cell is the whole stack at a fixed maximum size, serving a fixed set of customers:

- **Fixed size, grown by count.** Pick a maximum (say 1 million customers or 10,000 req/s), load-test it once, grow by adding cells — "cattle, not pets" one level up.
- **The partition key.** Every request carries a key (customer, tenant or account ID); the router looks it up in a **mapping table**, not `hash % N`, so one customer can move without reshuffling everyone (the modulo trap from Consistent Hashing).
- **A router as dumb as possible.** The one shared component: a cached lookup and a forward, no business logic, *fail-static* (keeps routing with the last known table if the mapping store is down).
- **Deploy in waves.** Code and config go to **one cell first** (a canary *cell*), then a few, then the rest, with bake time (watching metrics before continuing) and automatic checks between waves — Canary Deployment for whole stacks. The global config push becomes a one-cell incident.
- **No cross-cell calls on the request path**, or cell 3's failure hurts cell 7's customers. What must be global (sign-in, billing) is a separate, small, carefully built global service.

Shuffle sharding — cells for shared workers. When a private cell per customer is too expensive, give each customer a small random *combination* of shared workers. With 8 workers and 2 per customer there are 8 × 7 / 2 = **28** pairs. A poison customer takes down its own 2; another customer is fully cut off only if it drew *exactly the same pair* — 1 in 28 (≈ 3.6%) — and one sharing a single worker still gets answers from the other (clients retry). Route 53 (Amazon's DNS) does this at scale: 2,048 virtual name servers, 4 per customer domain, about **730 billion** combinations, assigned so no two domains share more than two name servers.

Two regions, cells inside each, customers routed by home region and then by cell



Cells never call each other; dotted lines are background replication (each cell to a DR partner, the catalog to both regions).

Walkthrough with numbers. **20 million customers**; about **200,000 req/s** reach the app tier at peak (the CDN absorbs the rest — the capstone's Stage 4).

- **Cells.** Tested at **10,000 req/s** and **1 million customers**: 200,000 / 10,000 = **20 cells**, 10 per region. The poison tenant now slows **1 cell = 5%** of customers — and can be moved to a cell of its own.
- **Config push, replayed.** One cell first; its error rate jumps within a minute; the pipeline halts and rolls back automatically. Impact: **5% of customers for ~5 minutes**.
- **Static stability.** If US East is lost, EU West must carry all 200,000 req/s *with capacity already running* — scaling mid-disaster bets on scaling APIs that may be part of the outage. Two regions each sized for full peak run at **~50%**: **2.0×** peak paid. With **three**, any two carry peak, so each needs 100,000 req/s and runs at ~67%: **1.5×**. A third region is often *cheaper* than a second.
- **Latency.** Sydney gets an Asia-Pacific home region: writes drop from **~200 ms** to **~10 ms**. Only the small truly-global category still crosses oceans.
- **Recovery targets.** **~1 s** replication lag → RPO ≈ 1 s (unless synchronous). Failover ≈ detection (~30 s) + DNS TTL (60 s) + replica promotion (~1 min) → **RTO of a few minutes**.

Trade-offs

- *You gain:* survival of region loss; a bounded blast radius for your mistakes (1/N per wave) and noisy customers; lower latency for distant users; growth by copying a tested unit.
- *You pay:* **money** (static stability = 1.5–2× compute, plus billed cross-region transfer); **data complexity** (every dataset needs a home, replication mode and conflict story; global uniqueness

and counters get hard); **operational weight** (N databases to migrate, N dashboards, wave-aware deploys); **a new shared failure domain** in the router and global services, which must stay tiny and fail-static; and **drills** — untested failover doesn't work.

When not to use it

- **Small scale or low stakes:** one region, three AZs and a tested restore beat a design that roughly doubles cost.
- **No clean partition key:** a social graph where everyone interacts with everyone (partition what you can; keep the rest global).
- **Active-active by default:** many systems need only pilot light or warm standby — decide from RPO/RTO.
- *Common misuse:* a "second region" depending on the first (a shared global database, config store or sign-in service).

Real-world examples

- **AWS** recommends cell-based architecture to limit blast radius and describes Route 53's shuffle sharding.
- **Slack** moved to a cellular design where each AZ is a cell that can be drained in minutes, after partial-AZ network failures caused incidents.
- **Netflix** runs active-active across several AWS regions and practises evacuating a region.
- **Google Spanner** productises the truly-global option: Paxos (a classic consensus algorithm) across regions plus tightly bounded clocks (TrueTime), paying the round trip on writes.
- **For AI engineers:** GPU capacity differs by region, so inference goes multi-region for *capacity* as much as failure; prompts and documents may have data-residency rules (laws on which country stores the data); roll models out cell by cell.

Cloud building blocks (a short map).

Need	AWS	Azure	Google Cloud
Virtual machines	EC2	Virtual Machines	Compute Engine
Managed Kubernetes	EKS	AKS	GKE
Functions (serverless)	Lambda	Azure Functions	Cloud Run functions
Object storage	S3	Blob Storage	Cloud Storage
Managed relational DB	RDS, Aurora	Azure SQL Database	Cloud SQL, Spanner
Key-value / document DB	DynamoDB	Cosmos DB	Firestore, Bigtable
Queue / stream	SQS, Kinesis	Service Bus, Event Hubs	Pub/Sub
CDN / global entry	CloudFront	Front Door	Cloud CDN
DNS	Route 53	Azure DNS	Cloud DNS

- **How serverless functions run.** AWS Lambda runs each instance in a **Firecracker microVM** — a minimal VM booting in a fraction of a second, isolating tenants more strongly than a container. A **cold start** (no warm instance) creates or restores a microVM, starts the runtime and runs your init code: ~100 ms to several seconds, worst for large JVM apps. Remedies: small packages, less start-up work, provisioned concurrency (pre-warmed instances), snapshot restore (Lambda SnapStart).
- **Cost levers, biggest first:** delete idle resources; right-size to measured use; scale down at night; commit baseline usage for a discount (savings plans, committed-use discounts); run interruptible batch work on **spot** capacity (much cheaper, reclaimable at short notice); tier cold data with

lifecycle rules; watch **data transfer** — cross-AZ, cross-region and internet egress are billed and surprise new multi-region teams.

In the interview. "How would you survive a region outage?" and "how do you limit the blast radius of a bad deploy?" — with follow-ups like "what happens to writes during failover?"

30-second answer: Three AZs survive a datacenter, not a regional control-plane failure or our own bad config. I pick each region strategy from RPO/RTO and give each dataset a home: customer data in a home region copied async for DR; read-mostly data replicated one-way; only a small truly-global set pays for cross-region consensus. Regions are statically stable — each absorbs a failed one without scaling. Inside each region, fixed-size cells behind a tiny fail-static router, deployed one cell at a time, so a bad change or noisy customer hurts at most one cell.

- *Follow-up: "A region dies mid-write — what is lost?"* — With async replication, writes in the lag window (~1 s) are lost or delayed until the region returns; replicate synchronously only where that's unacceptable (payments).
- *Follow-up: "Why not grow cells as you grow?"* — A cell's value is its *tested* size; past it, it's an untested system with a bigger blast radius. Add cells.
- *Mistake that fails candidates:* two regions sharing one global database, config store or login service; or "active-active" without saying how write conflicts are avoided.

Self-check:

- Q1: Failure domains from process to "your own change", and what contains each? → Host: N+1 behind a balancer. AZ: every tier across three AZs. Region: a second/third region chosen by RPO/RTO. Your own change and noisy customers: cells deployed in waves (or shuffle sharding).
- Q2: Two regions or three for a 300,000 req/s peak with static stability? → Two: each carries 300k → 600k (2×). Three: any two carry 300k → 150k each → 450k (1.5×). Three is cheaper, at the cost of one more region to operate.
- Q3: "We're active-active": both regions serve traffic, but all writes go to a primary in region A, and sign-in lives only in A. → Active-active for reads only: losing A stops all writes and sign-ins. Fix: per-customer home-region writes (or tested automatic promotion in B), sign-in in each region, regular drills evacuating A.

Related patterns: Horizontal Scaling (cells are cattle one level up); Load Balancing (DNS/anycast above the regional balancer); Consistent Hashing (why a mapping table, not `% N`); Bulkhead (a cell bulkheads a whole stack); Quorum (majorities for globally consistent data); Canary Deployment (one cell at a time).

TL;DR

- **Problem:** three AZs don't survive a regional failure, and a single shared system can't contain your own bad change or one noisy customer — every blast radius is 100%.
- **Mechanism:** regions (strategy by RPO/RTO; data as home-region, read-mostly or truly global; statically stable capacity) plus cells (fixed-size stacks, tiny fail-static router keyed by customer, deploys in waves; shuffle sharding for shared workers).
- **Costs:** 1.5–2× capacity, cross-region data and conflict design, N stacks to operate, a router and global services that must stay tiny, failover that works only if drilled.
- **Skip it when:** scale or stakes are small (one region, three AZs, tested backups), or the data has no clean partition key.

Sources: AWS Well-Architected Framework, *Reliability Pillar* and the whitepaper *Disaster Recovery of Workloads on AWS* (the four DR strategies); AWS whitepaper *Reducing the Scope of Impact with Cell-Based Architecture* (2023); C. MacCárthaigh, "Workload isolation using shuffle-sharding", *Amazon Builders' Library*; B. Weiss and M. Furr, "Static stability using Availability Zones", *Amazon Builders' Library*;

Slack Engineering, "Slack's Migration to a Cellular Architecture" (2023); Netflix Tech Blog, "Active-Active for Multi-Regional Resiliency" (2013); J. Corbett et al., "Spanner: Google's Globally-Distributed Database", OSDI 2012; A. Agache et al., "Firecracker: Lightweight Virtualization for Serverless Applications", NSDI 2020; Kleppmann, *DDIA* ch. 5 and 9 (Replication, Consistency and Consensus).

Capstone - From 100 to 1,000,000 Requests per Second

Original lesson: Capstone - From 100 to 1,000,000 Requests per Second on DesignGurus ↗

What this capstone is. Every pattern here shares one habit: **capacity follows the bottleneck**. One service grows across four orders of magnitude (100 → 1,000,000 req/s), asking at every stage:

1. **What is limiting throughput right now?**
2. **What is the cheapest tool that moves that limit?**
3. **What should stay unchanged?**

At each "pause and decide," name the constraint *before* choosing a pattern. Interviewers run exactly this ("now traffic is 10× — what breaks next?"); strong answers track the moving bottleneck rather than drawing planet-scale architecture on day one.

The brief. The store's public API (browsing, search, product details):

- **One modest 4-core server** running the app, with sessions in memory, files on local disk and a midnight cron job.
- **100 requests/second** on average; one database primary (storage patterns not yet applied).
- Business grows about **3× a year**; marketing has big plans.

Stage 1 — 100 → 1,000 req/s: the single point of failure

- **Incident.** Peaks of several hundred req/s push the server past **70% CPU**; moving up **two instance sizes** fixes it in an afternoon — correctly (Vertical Scaling). Eight months later, near 1,000 req/s, that host dies at lunchtime: **40 minutes of total outage**. The problem is now that there's only *one* server.
- **Pause and decide:** the postmortem says "single point of failure." What's the fix, and what hidden work does it uncover?
- **Decision: at least 2 servers behind a load balancer with health checks — plus the statelessness checklist.** The second server surfaces local state at once (lost in-memory sessions, images split across disks, the midnight job running twice): sessions → Redis, files → object storage, cron → behind a distributed lock. Sticky sessions serve as a **two-week migration bridge**, removed once shared state works.
- **Why not an even bigger box:** same failure mode. The win is that *any* server takes *any* request, so losing one lowers capacity instead of wiping sessions or stopping service — making every later fleet change routine.
- **Not scaled:** the database — mostly idle. At ~800 req/s per server, two (1,600 req/s) cover the 1,000 req/s peak. Precisely: this removes the SPOF but isn't strict N+1 — if one dies, the survivor's 800 req/s is below peak, so peak degrades. Strict N+1 would be 3 servers (2 for peak + 1 spare).

Stage 2 — 1,000 → 10,000 req/s: the fleet, and the first downstream flood

- **Incident.** Both servers pass 80% at every peak; adding a third, then a fifth, is routine because they're stateless. At **~10 servers** the limit appears downstream: **database read latency doubles at peak**, since the bigger app tier multiplied queries about fivefold.
- **Pause and decide:** the app tier is healthy and the DB is drowning in reads. Which earlier modules own this, and what app-tier discipline goes with them?
- **Decision. 13 servers (12 for peak + 1 spare)** (12 × 800 = 9,600 req/s covers a peak just under 10,000; a full 10,000 req/s peak would need 13 + 1 spare) behind **least-connections**, with rolling

deploys and drain hooks. **Cache-aside** (check the cache; on a miss read the DB and fill the cache) reaches a **>98% hit ratio** on product reads, cutting DB reads about **40×**; **read replicas** take the remaining misses. Audit app-side pools: 13 × 20 = **260 connections**, already near a common database limit.

- **Why not a bigger database:** reads that never reach the DB are cheaper than a bigger DB answering them.
- **Not scaled:** no auto-scaler yet — traffic is predictable enough for fixed N+1, and attention is better spent on cache correctness.

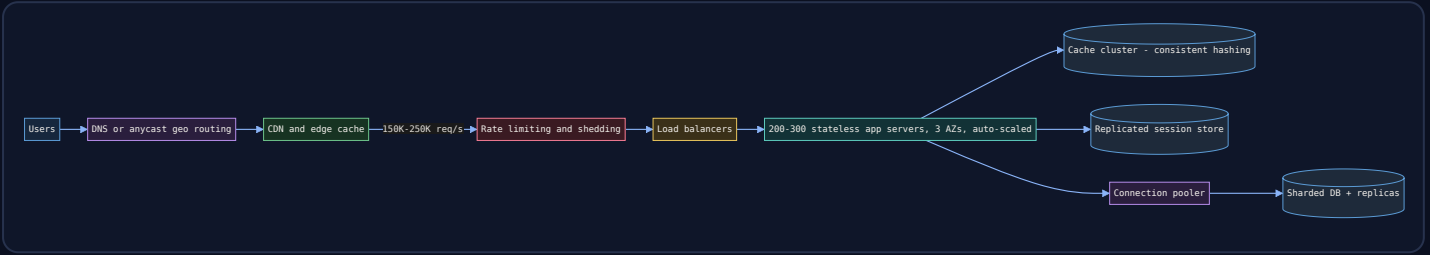
Stage 3 — 10,000 → 100,000 req/s: the fleet becomes elastic

- **Incident. ~65 servers** sized for peak sit **below 30% utilization 18 hours a day**; then a campaign brings **3× normal peak within 20 minutes** and swamps the fixed fleet — wrong both ways, the Auto-Scaling incident again.
- **Pause and decide:** design the loop — signal, target, min, max. Where does the max come from, and what does each scale-up do downstream?
- **Decision: auto-scaling, with the homework done.**
  - **Signal:** requests per instance against measured capacity for the IO-bound API; **backlog age** for async workers.
  - **Scheduled + reactive:** capacity ahead of the 6 PM ramp; a reactive **65%** target; up fast, down slowly, cooldowns against flapping.
  - **Min 8, max 120:** the max from measured database, cache and replica capacity — a **bulkhead**, not a hopeful guess.
  - **The pooler:** up to 120 × 20 = **2,400 clients**; **PgBouncer** multiplexes them onto about **60 real connections**, so scale-ups stop causing connection storms.
- **Not scaled — avoided instead:** images and JavaScript move to a **CDN** (API & Edge module); about **one-third of requests** never reach the app tier. Removing traffic beats serving it efficiently.

Stage 4 — 100,000 → 1,000,000 req/s: putting it all together

- **No single incident** — earlier stages removed or contained the failure modes. The question is how a nominal million req/s spreads across the architecture.
- **Pause and decide:** what share ever reaches an app server, and which tier is hard now?
- **The honest anatomy of a million:**
  - **Most requests never reach your servers.** CDNs and edge caches serve static and public cacheable content; the cache tier absorbs most remaining reads. Perhaps **150,000–250,000 req/s** reach the app fleet.
  - **The app fleet is big and boring: 200–300 stateless servers across three availability zones**, auto-scaled, drained, rolled. Boring is the *outcome* of this module's patterns.
  - **The hard tiers hold state**, each with a specialized design: a **sharded** database, a cache cluster using **consistent hashing**, a **replicated session store**.
  - **The front door gets a module of its own:** rate limiting, shedding beyond the fleet max and geographic routing become first-class — where the next module starts.

Architecture at 1M req/s: most traffic is absorbed before the app fleet; state lives in specialized tiers.



Debrief: the bottleneck's journey

Stage	The bottleneck was	The move	Where it went next
100 → 1K	One machine's capacity, then the fact that there was only one	Bigger machine, then 2+ servers + statelessness + LB	Into growing the fleet
1K → 10K	App capacity (routine), then database reads	Fleet + N+1; cache + replicas	Toward connections and cost
10K → 100K	Cost at 4 AM, surprises at 7 PM, connections all the time	Auto-scaling (signal, max-as-bulkhead) + pooler + CDN	Toward the tiers that hold state
100K → 1M	The tiers that hold state, and the front of the system	Work from earlier modules + the next module's patterns	Where it always ends up: the data

Takeaways

- Every stage *moves* the bottleneck into the tier with the best tools for it. Scaling = the **cheapest pattern that's enough for the current constraint**, leaving unrelated tiers alone.
- **Order matters.** A later-stage pattern too early (an auto-scaler at 1K req/s, sharding at 100 req/s) adds cost without moving the current limit; deferring a needed earlier one (statelessness, pooling) makes the next 10× unsafe.
- Many answers come from other modules — caches, replicas, shards, idempotent retries — which is why scaling questions test the whole course.
- **Beyond a million:** the next bottleneck is *blast radius* — one region, deploy or tenant can still hurt everyone. Regions and cells (the previous lesson) cap it.

Interview one-liner: Follow the bottleneck: a bigger box, then stateless N+1 behind a load balancer; then cache and replicas for DB reads; then auto-scaling capped by downstream capacity, plus a pooler and a CDN; at a million req/s most traffic never reaches the app tier — the hard problems are the stateful tiers and the front door.

Extend it yourself

- **Multi-region (active-active, two regions):** which tiers replicate cleanly (the stateless fleet), which become hard (the database — the Consistency module), and where does balancing move (up to DNS/anycast)? (Worked through in Multi-Region and Cell-Based Architecture.)
- **The stateful edge:** add **500,000 concurrent WebSocket users**. Which assumptions still hold — any-request-anywhere, drain-in-30-seconds, least-connections — and which need the WebSocket lesson's two-tier redesign?
- **For AI engineers:** scale an inference endpoint from **10 to 10,000 req/s**. "Servers" cost **~100×** more and take minutes to boot, the cloud may refuse more (quotas), and work per request varies **~100×**. Which stages carry over, and which must be rethought, starting with the scaling signal?

Module 7 — Keeping Data Consistent (Distributed Consistency)

Module Introduction

Original lesson: Module introduction on DesignGurus ↗

What this module is about. One database moving "$50 from checking to savings" does both halves or neither — **atomicity** — almost for free via its **write-ahead log (WAL)**: it durably records what it intends to do before doing it, and after a crash it reads the log to finish or reverse the work. Split the data across shards or services (each owning its database) and no single WAL sees both halves; a change on shard 2 and its partner on shard 5 are tied together by nothing. This module rebuilds all-or-nothing across machines and shows what each approach costs in speed, availability or complexity. (Basics: Module 0 — Transactions and ACID, Durability: WAL, fsync…, Distributed transactions.)

> **Analogy.** One database is one bank teller with one ledger; a distributed system is two banks, each updating its own ledger for the same transfer. Someone must coordinate them, and every method has a price.

Only four patterns — the smallest module, and the toughest material.

The four patterns

The situation	The pattern
Several parties must either all commit (make changes permanent) or all back out	Two-Phase Commit (2PC)
A workflow has many steps, and any one might fail midway	Saga
Several copies of the same data must agree, with no single leader (main copy taking all writes)	Quorum
We need to know who saw which update, and in what order	Vector Clocks

Two facts that organize the module

Fact 1 — when the network splits, you have to pick a side. During a **network partition** (groups of machines running but unable to reach each other) a system either (a) stops serving until the link returns — **consistent** but not **available** — or (b) keeps serving on both sides and reconciles later — available but briefly inconsistent. This is the **CAP trade-off** (Consistency, Availability, Partition tolerance; see Module 0 — CAP theorem and PACELC). No engineering removes it; it comes from how networks behave. Each pattern here makes its own choice, and interviewers ask about CAP directly.

Fact 2 — the cheapest distributed transaction is one you never need. (A *distributed transaction* is one all-or-nothing change spanning machines or services.) Try cheaper tools first:

- **Shard-key discipline** (sharding lesson): choose the shard key so one customer's checkout lives on one shard, and a normal local transaction still works.
- **Idempotent operations** (repeating has the same effect as once) and **commutative operations** (any order gives the same result) shrink what truly needs agreement.

Few flows truly need distributed-transaction machinery — typically moving money between owners or promising stock across services.

How to read the module

The store's order workflow now touches inventory, payment and shipping, and needs all-or-nothing again.

- **Two-Phase Commit** — a coordinator gathers a unanimous "yes"; every participant waits on its decision.

- **Saga** — the usual answer between services: run steps in order and, on failure, run "undo" steps (compensations) for finished ones. Reuses pub/sub ideas.
- **Quorum** — leaderless replicas: choose write confirmations (W) and read fan-out (R) so **W + R > N**, guaranteeing overlap.
- **Vector Clocks** — track which updates knew about which, separating concurrent edits from sequential ones.
- The **capstone** builds order fulfillment with 2PC and with a saga, and compares how each breaks.

> Short on time? Start with **Saga** ("how do services do transactions?") and **Quorum** ("how do replicas agree?") — together most of the interview questions here.

Four terms used throughout

- **Atomicity** — all-or-nothing; free on one machine via the WAL, costly across machines.
- **Partition** — a network split; both sides alive but unreachable to each other. Partitions *will* happen, forcing Fact 1's choice.
- **Strong vs. eventual consistency** — *strong*: every read returns the latest write, as if one machine existed. *Eventual*: copies converge, but a read may briefly see old data (the **replication lag** and stale views seen earlier). See Module 0 — Consistency models.
- **Compensation** — an "undo" performed as a *new* action (cancel, refund, release). Unlike a rollback, nothing is erased — how businesses have corrected mistakes on paper for centuries. Sagas are built from it.

Before you start — questions to ask about your own system

1. Do you write to two databases (or services) and trust both writes succeed? What does your data look like the day one fails?
2. When reading a replicated store, how many copies back the answer? Do you *know*?
3. Two users change the same record in the same second. Which change survives — and does the loser ever learn?

Two-Phase Commit

Original lesson: Two-Phase Commit on DesignGurus ↗

What it is. Two-Phase Commit (2PC) gives true all-or-nothing to one operation spread over several machines. The **coordinator** asks each **participant** (a database involved), "Can you commit this?"; each answers a binding yes or no; only if all say yes does it tell everyone to commit. No one commits until all can.

> **Analogy.** A wedding: both partners say "I do" before the officiant declares them married; after "I do" neither can leave on their own — and if the officiant faints, both are stuck at the altar.

The problem: $50 that vanished

Customers can now send gift-card balances: A (shard 2) sends $50 to B (shard 5) — one logical operation, two databases, no spanning transaction. Version one does two sequential writes: subtract on shard 2, add on shard 5. Thousands succeed. At **2:37 AM** the application server crashes after the subtraction committed and before the addition was sent. No retry (the process that knew shard 5 needed its addition is gone), no error logged; the nightly reconciliation job later finds $50 missing. (Reverse the order and the same crash *invents* $50.) Each shard behaved perfectly; the *operation as a whole* was never atomic.

Why the obvious fixes fail

- **"Write A, then B, and retry B until it works."** Idempotency keys (a unique ID per operation, so repeats apply once) make retries safe, but two holes remain: if shard 5 is down an hour, money is missing for an hour; and if the app dies between writes, its in-memory retry loop dies too. The plan to finish must be durable outside the process (the saga lesson builds on this).
- **"Do both writes in parallel."** Either can still fail alone — now two possible half-done states. Faster isn't atomic.
- **"Let the nightly reconciliation fix it."** Reconciliation *spots* mismatches after users may have seen them; it doesn't *prevent* them. Fine only if hours of inconsistency are acceptable.

How it works

The coordinator talks directly to Shard 2 (subtract $50) and Shard 5 (add $50); the shards never talk to each other — which is why a silent coordinator freezes both.

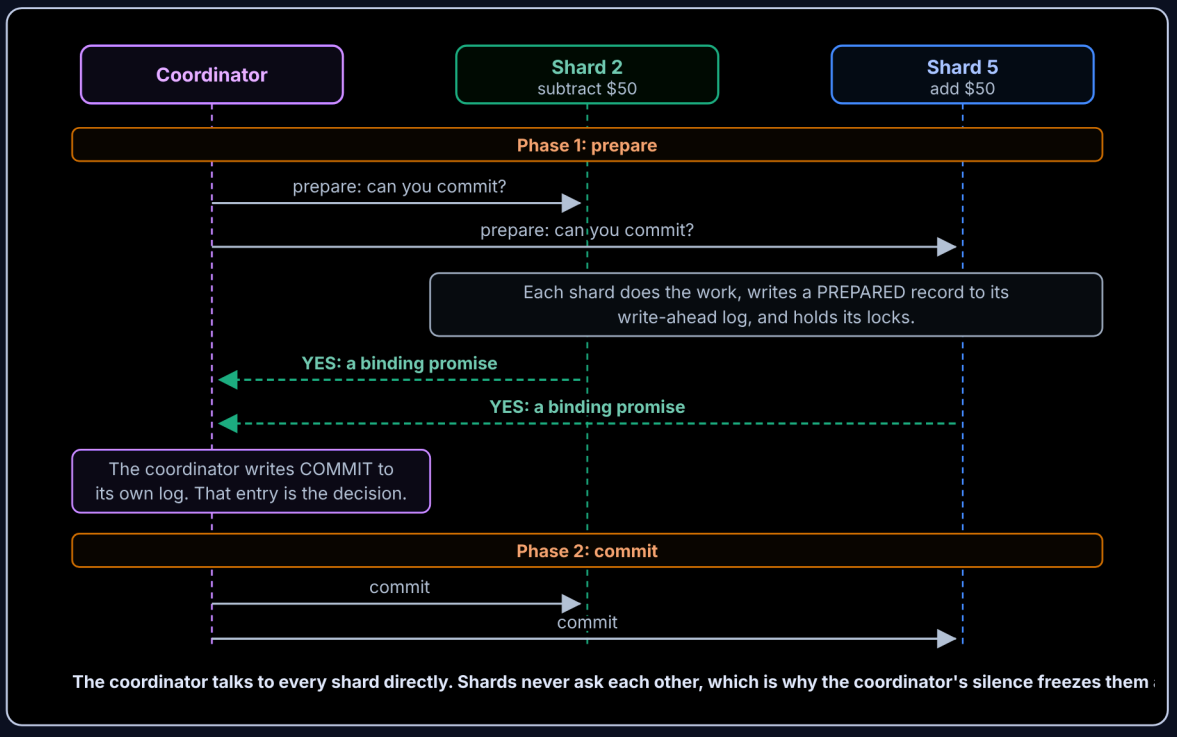


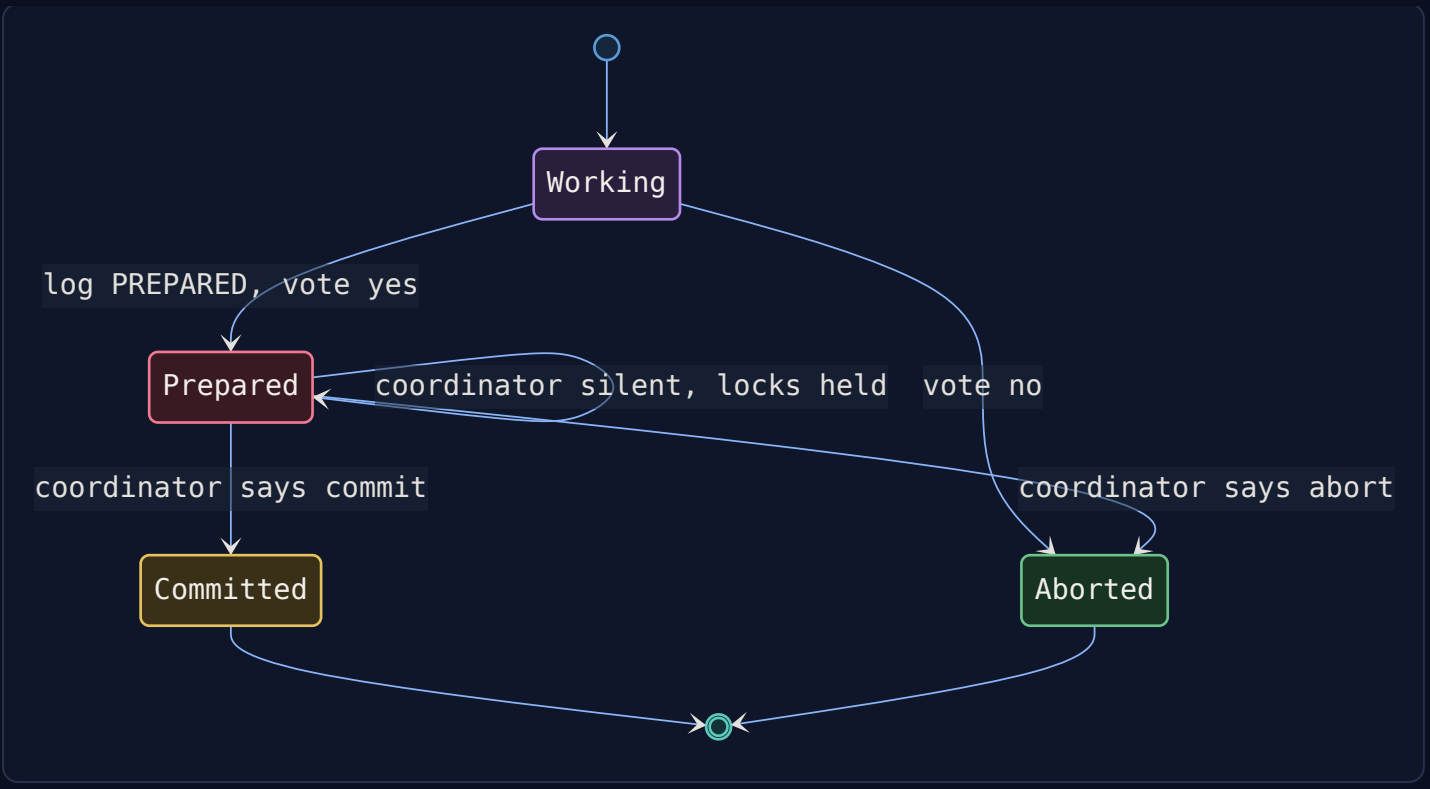
Figure — Two-Phase Commit (course diagram)

1. **Phase 1 — Prepare.** Each participant does *all* the work, writes a durable **PREPARED** entry to its own WAL, **keeps its row locks** (blocking other transactions from changing — sometimes reading — those rows), then votes. *Why:* with work done and logged, it can still commit after a crash and restart.
2. **A yes vote is a binding promise:** "The work is on disk; I can commit even if I crash; I'll hold my locks until you tell me the result."
3. **The coordinator logs its decision** — COMMIT if all voted yes; ABORT if any voted no or stayed silent. That log entry *is* the transaction's outcome, so recovery can always look it up.
4. **Phase 2 — Commit or abort.** The coordinator sends the decision; each participant applies it and releases its locks.

Ordering rules that make recovery safe. A participant logs PREPARED *before* voting yes; the coordinator logs its decision *before* announcing it. A restarting participant reads its log and asks the coordinator for the decision — it never re-runs business logic.

The in-doubt state (the pattern's big risk). After voting yes a participant is **in doubt**: it can't commit alone (another may have voted no) or abort alone (commit may already be decided). It waits, holding locks. If the coordinator crashes right after collecting yes votes, every participant is in doubt and every other transaction needing those rows blocks until it returns. 2PC gets atomicity by centralizing the decision — and inherits that spot's weakness.

Participant states: after voting yes, only the coordinator can move it



Walkthrough with numbers (the $50 transfer under 2PC)

- **Cost when all goes well.** Two network round trips (prepare, commit) plus a durable disk write (**fsync** — forcing data to disk before moving on) at every participant and the coordinator: about **4x** a single local-shard write. Noticeable, often acceptable.
- **The hidden cost: lock time.** Row locks span prepare to commit, stretching from roughly **1 ms to 10–20 ms**. By *rate x duration = concurrency*, locks held **10x longer** mean roughly **10x more collisions** on busy rows — 2PC slows its neighbours too.
- **Failure drill.** The coordinator crashes **5 ms** after both votes. Both shards stay in doubt with locks held until a replacement coordinator replays the log and sends commit **40 seconds** later; meanwhile any transaction touching either customer's rows waits or fails.
- **Replicating the coordinator** makes recovery faster (the decision survives one machine) but doesn't remove prepare-time locks, extra round trips, or the need for every participant to be online.
- **Availability math.** Coordinator and both shards must all be up: three components at **99.9%** each give about **99.7%**, lower with every participant.

Trade-offs

You gain:

- **True atomicity across machines** — the "vanished $50" bug can't happen; no reconciliation needed.
- **Strong consistency** — nobody sees money that left A but hasn't reached B.
- **Looks normal to app code** — with driver support it feels like an ordinary transaction. The **XA standard** is a common 2PC interface; Java's **JTA** builds on it.

You pay:

- **Blocking — its trademark cost.** In-doubt participants keep locks while the coordinator may be unreachable; for *progress*, the coordinator is a SPOF. Stuck prepared transactions are a real DBA emergency.
- **Latency and throughput** — two round trips, several fsyncs, longer lock holds, contention on popular rows.
- **Availability shrinks as you multiply** — no commit unless every participant is up.

- **It can't cross trust boundaries.** No outside company (beyond your **trust boundary**) lets your coordinator hold locks in its database.

When not to use it

- **Between microservices.** Cross-service locks tie every participant's speed and uptime to the slowest; a checkout can take seconds or minutes — far too long to hold locks — and the coordinator becomes a technical and organizational bottleneck. Sagas exist for this.
- **Hot rows (rows taking a large share of traffic), cross-company operations, or flows where "fixed by morning" reconciliation is good enough** — atomicity costs more than the inconsistency it prevents.

Where it *does* fit: short, internal, rare, high-value operations — above all **inside distributed databases**. Google Spanner runs 2PC between shard groups where each participant and the coordinator is a **quorum-replicated group** (a majority of copies must agree), so individual machine deaths don't block it. 2PC isn't obsolete, and isn't for everything.

Operational safeguards (even there): strict transaction deadlines; visible prepared-transaction age; alerts *before* held locks slow unrelated work; recovery tools showing the coordinator's decision and each participant's state — human guessing during recovery causes the very inconsistency 2PC prevents.

2PC vs. Saga (preview)

	Two-Phase Commit	Saga
Atomicity	Genuine: all-or-nothing, guaranteed by locks	Imitated: each step commits separately; compensations undo earlier steps on failure
Intermediate states	Hidden from everyone	Visible — "payment taken, shipping pending"
Blocking	Yes — in-doubt participants keep locks	No — every step is an ordinary local transaction
Coordinator dies	Participants are frozen	The workflow resumes from its durable log
Fits	Short, internal, rare work (or inside a database)	Business workflows spanning services

In one line: **2PC hides the in-between state by holding locks; a saga lets that state show and manages it.**

Real-world examples

- **XA standard and Java's JTA** — 2PC plumbing for committing databases and message brokers together; banks and airlines still run on them.
- **PostgreSQL** `PREPARE TRANSACTION` (finished by `COMMIT PREPARED` or `ROLLBACK PREPARED`) — the participant role directly. The docs warn against casual use: an abandoned prepared transaction holds its locks until an operator intervenes.
- **Google Spanner** — 2PC across shard groups replicated with **Paxos** (a consensus protocol letting copies settle on one value); replication absorbs machine failures at the cost of consensus overhead.
- **Kafka transactions** — a 2PC-shaped protocol for atomic consume-process-produce, with one broker as coordinator.
- **For data engineers:** exactly-once writes (each record's effect lands once) into external systems often use this pattern. **Flink** transactional sinks treat a checkpoint (saved snapshot of job progress) as "prepare" — every operator durably records readiness — and completing it as "commit" into Kafka or a database.

In the interview

"How can two services update their databases atomically?" Walk the full reasoning: 2PC gives real atomicity, blocks on failure, lives mostly inside distributed databases today; service workflows use sagas. "2PC everywhere" and "2PC is dead" both show you don't know its limits.

30-second answer: "2PC gives genuine atomicity across machines: a coordinator gathers binding yes votes — each participant does the work, logs it durably and keeps its locks — then tells all to commit. The downside is blocking: a participant that voted yes can't commit or abort alone, so a coordinator dying at the wrong moment freezes everyone with locks held. Add two round trips and availability that drops per participant. So I use sagas between microservices, and respect where 2PC lives today: inside databases like Spanner, where every party is quorum-replicated so no single crash blocks it."

Likely follow-ups (with answers):

- *"The coordinator crashes just after both vote yes — walk me through it."* Both are in doubt: bound to obey a decision they can't make. They hold locks until the coordinator recovers and replays its log; with weak tooling a human resolves the prepared transactions. The word is **blocking**.
- *"Why is 2PC fine inside Spanner but wrong between your services?"* Inside: replicated-group participants, millisecond transactions, one trust boundary, one ops team. Between microservices: single-machine participants, multi-second workflows, separate teams — every weakness at full force, no protections.

Mistake that fails candidates: 2PC for a multi-minute checkout; ignoring coordinator failure; calling 2PC obsolete without knowing databases use it internally.

Self-check

- **Q1: Name the two phases, state what a yes vote promises, and how that leads to blocking.** → Prepare (do the work, log PREPARED durably, keep locks, vote), then commit/abort (all yes → commit, else abort). Yes means "I can commit even after a crash and will hold state and locks until you decide." It can't commit alone (someone may have voted no) or abort alone (commit may be decided), so a silent coordinator blocks it indefinitely.
- **Q2: 2PC for (a) the gift-card transfer — two shards, same cluster, ~50 ms — and (b) checkout across inventory, payment and shipping services, 2–30 s?** → (a) Defensible: internal, rare, one trust domain, short — but first ask whether both balances could share a shard (Fact 2). (b) A saga: seconds-to-minutes of cross-service locks would destroy throughput and chain three teams' availability.
- **Q3: 3 AM page: queries on orders hang for four minutes; the DBA finds hours-old transactions in `PREPARED` holding locks.** → Participants voted yes; the coordinator never delivered a decision (crashed, lost its log, or failed between phases). PostgreSQL holds the locks until someone finds the decision and runs `COMMIT PREPARED` or `ROLLBACK PREPARED`; if no decision survived, the business decides by hand. Postmortem: does this flow truly need 2PC, and if so why wasn't the coordinator replicated? Fix: a saga or a replicated coordinator.

Related patterns

- **Write-Ahead Log** — holds PREPARED records and the decision; the promise is only as durable as this log.
- **Saga** — the replacement for cross-service workflows.
- **Sharding** — why 2PC became necessary; a good shard key is still the cheapest way to avoid it.
- **Quorum** — how modern systems make participants and coordinators survive crashes (Spanner).

TL;DR

Problem	One logical operation over two machines has a gap where a crash leaves it half-committed (money disappears or doubles). Cross-machine all-or-nothing must be built.
Mechanism	Prepare: each participant does the work, logs readiness, keeps locks, votes (a binding promise). Commit: all yes → all commit, else all abort. The coordinator's log holds the decision.
Costs	Blocking (in-doubt participants freeze with locks if the coordinator goes quiet), two round trips plus fsyncs, lock contention, availability dropping per participant.
Skip it when	Between microservices, across companies, on hot rows, or for long flows (sagas). Fits short internal transactions and quorum-replicated databases.

Saga

Original lesson: Saga on DesignGurus ↗

What it is. A saga breaks a cross-service workflow into a chain of **local transactions**, each committing immediately in its own service with no cross-service locks. Progress is written to a durable log before each step; if a later step fails, **compensations** — new actions cancelling a finished step's business effect — run in reverse order.

> **Analogy.** Booking flight, hotel, then car from different companies: if the hotel falls through you can't turn back time; you *cancel the flight*. Cancellation policies are compensations. (The name comes from a database paper on long-running transactions; travel booking is where the idea started.)

The problem: 340 orders stuck in limbo

Checkout calls three services in order: reserve inventory, charge the card via an **external payment provider**, create a shipment. A bad deploy takes shipping offline for **20 busy minutes**. Reservations and charges still succeed; the third call fails, and the catch block just logs an error. Result: **340 orders** paid and reserved but not shipped. Each service did its part, so none noticed; support learned from angry emails, and **three engineers spent a weekend** refunding and releasing by hand from a spreadsheet.

2PC was already out: an external provider won't hold locks for twenty seconds, let alone twenty minutes. The workflow crosses a trust boundary, takes real time, and can fail after earlier steps committed. It needs (1) a durable record of each order's progress and (2) a plan for reversing finished steps — neither of which locks can safely provide.

Why the obvious fixes fail

- **"Put the undo logic in the catch block."** Better, still unsafe: a crash mid-checkout or mid-undo loses the in-memory plan, and undo calls can fail or time out with no durable retries or escalation. Recovery must outlive the process that started the work.
- **"Just use two-phase commit."** Locks across three services for seconds to hours — and the external provider can't participate.
- **"One service, one database."** If every step *can* share a transaction, this is the module's cheapest fix (Fact 2). But payment is external, and inventory and shipping may be separate for good reasons.

How it works

The forward path is T1 Reserve → T2 Charge → T3 Ship → Order complete, each a local transaction. When T3 still fails after retries, compensations run in *reverse*: C2 Refund, then C1 Release, ending at "Order failed — customer notified, nothing owed".

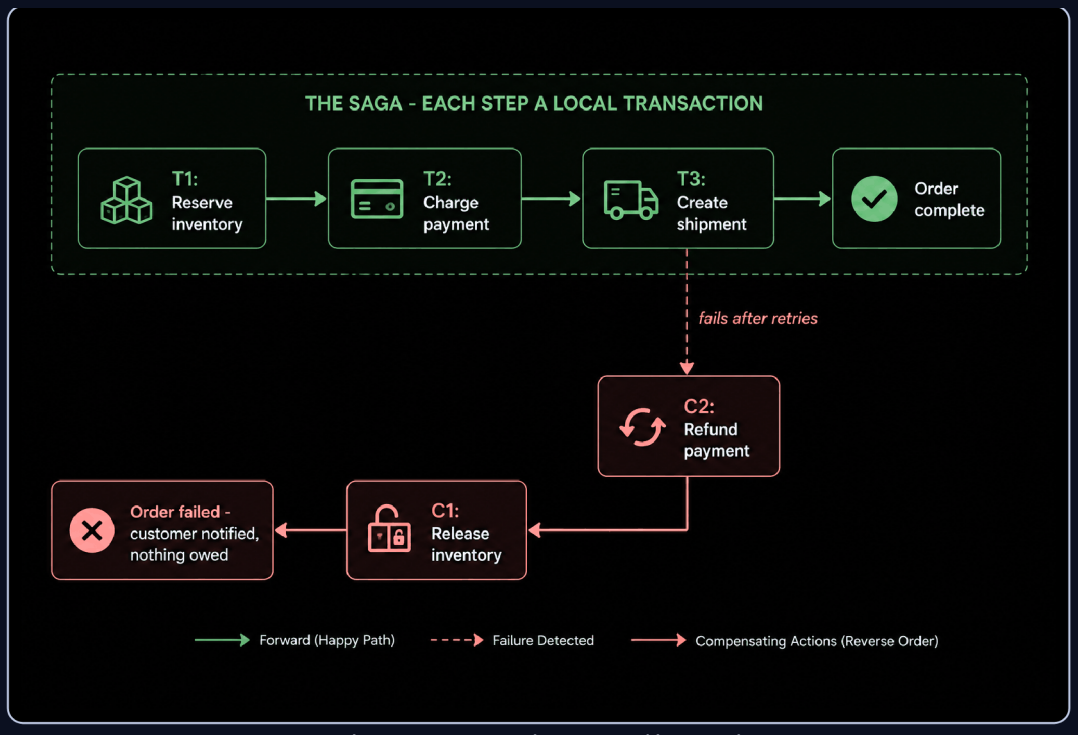


Figure — Saga (course diagram)

1. **Break the workflow into local transactions.** *Why:* no service holds locks waiting for another; hot rows stay free and a slow service can't freeze the rest.
2. **Record intent durably before every step — the saga log.** E.g., "order 7f3a: T1 done, starting T2" — the WAL rule applied to a workflow. The **orchestrator** (the process driving the workflow) may crash; another instance reads the log and continues. Record: saga ID, current state, completed steps, attempt counts, idempotency keys, deadlines, and the next allowed action.
3. **Make each state change conditional** — a lease (a lock that expires automatically) or a **compare-and-set** update ("write only if the state is still what I expect"), so two recovering orchestrators never both advance the same order.
4. **On persistent failure, compensate in reverse** — refund first, then release — undoing changes in the opposite order they were made.
5. **Make every step and compensation idempotent.** Timeouts trigger retries, so any request may arrive twice; each step has a stable **idempotency key**, and "refund order 7f3a" must be safe twice.
6. **Hand off what can't be fixed automatically.** A refund failing for ten minutes stays a durable *pending obligation* (the saga isn't done). After bounded retries the orchestrator opens a human task with the order, provider references, attempts and safe next steps; the unresolved state stays visible.

Compensations are a business undo, not a rollback. A committed transaction can't be erased, so each step gets a visible opposite action (refund, release, cancel) — like event sourcing's corrective entries (storing every change as an append-only event). Consequences:

- Compensations are *new, visible actions*, not erasures.
- **Business owners must define them** — not everything reverses perfectly: you can't un-send a confirmation email; its "compensation" is an apologetic follow-up.
- Some effects can't be compensated: a physically shipped package needs **forward recovery** (return label, support case). That point of no return is the **pivot**; place pivot steps *last* and recover forward after them.

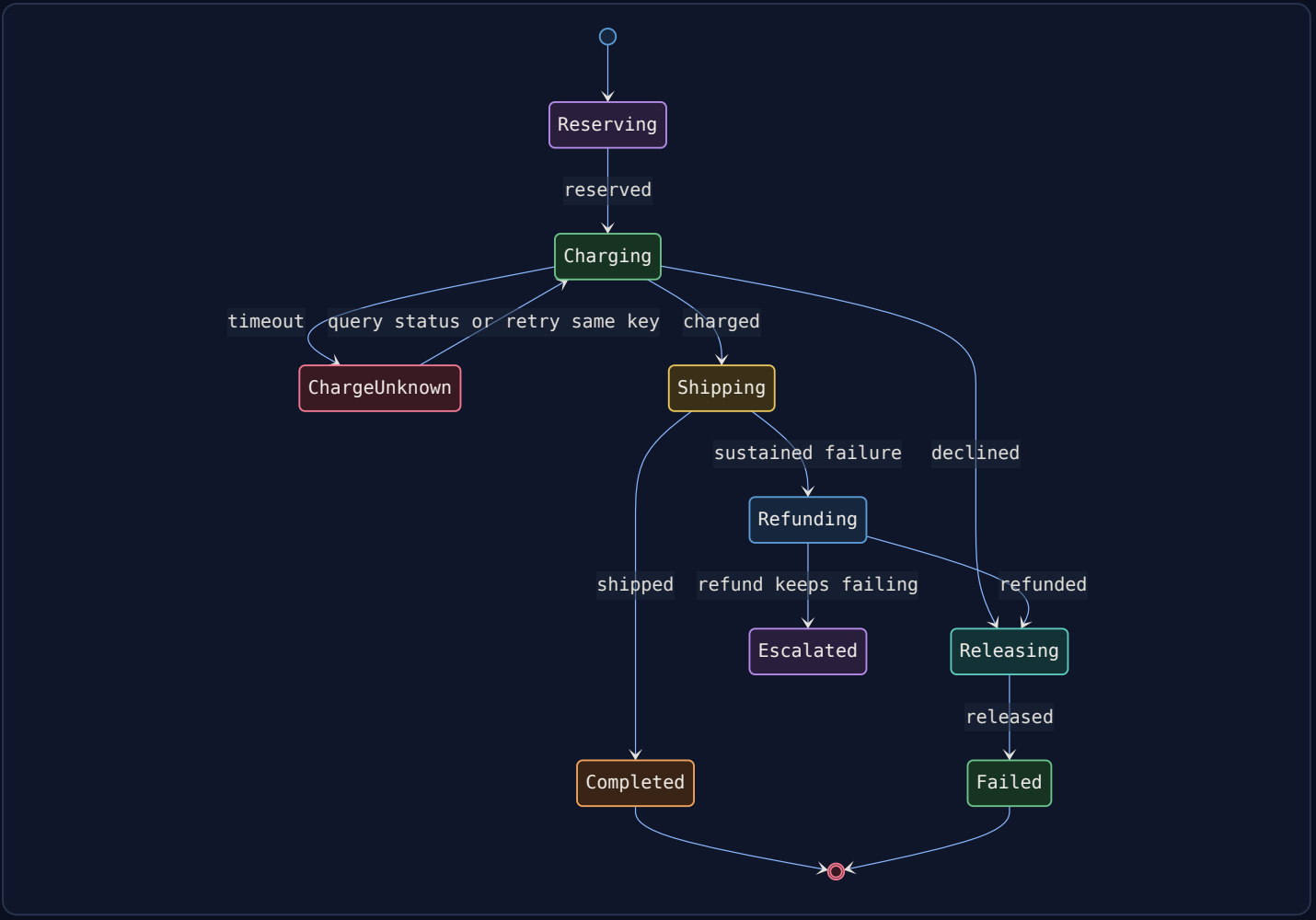
Who drives it: orchestration. Ordered, high-stakes workflows need an **orchestrator** — an explicit **state machine** (possible states and allowed moves) owning the saga log, so "where is order 7f3a?" is one query. (**Choreography** is compared below.)

The honest price: no isolation. Isolation hides a transaction's half-finished state. 2PC hides "charged, not shipped"; a saga exposes it between T2 and T3, and others can read and act on it. Product must *define* in-between states deliberately — "processing" is real workflow state. Some systems add

semantic locks: a field like `order_status=PROCESSING` tells other workflows the order has temporary limits (e.g., it can't be cancelled and shipped at once), without any cross-service database lock.

In the diagram, `ChargeUnknown` covers a timeout where you can't tell whether the card was charged (resolved by a status check or retry with the same key); a persistently failing refund goes to `Escalated` instead of quietly ending.

Orchestrator persists state before every step so a crash resumes instead of stranding



Walkthrough with numbers (reserve 50 ms, charge 200 ms, ship 100 ms)

- **Happy path.** About **400 ms** total, no locks between steps; each local transaction lasts about a millisecond, so hot inventory rows stay open.
- **Shipping outage, replayed.** T3 fails; the orchestrator retries with exponential backoff for about **30 seconds**. Still down → *sustained*, not a blip → refund, then release, both idempotent. Within about **90 seconds** the customer learns the order failed and the charge was reversed. No limbo, no spreadsheet.
- **Crash drill.** The orchestrator dies just after T2 commits; another instance reads "T2 done, T3 not started" and creates the shipment. Even a crash *during* T2 can't double-charge: the retry reuses the idempotency key and the provider returns the original result.
- **Ambiguity drill.** T2 times out — charged or not? Don't guess and don't refund a charge that may not exist: **check the charge status, or retry with the same key**. Log "outcome unknown" *before* checking, so a replacement orchestrator repeats the *status check* (same identifiers) instead of starting a fresh charge.

Trade-offs

You gain:

- **Cross-service and cross-company workflows with no distributed locks.**

- **Crash-surviving progress** — workflows *resume* from the saga log.
- **Long-running flows for free** — a saga can wait hours (e.g., fraud review) without locking anything.
- **Designed-in failure handling** — automatic compensations with retries and escalation.

You pay:

- **No isolation** — visible in-between states and the *anomalies* others cause by acting on them; product must define every state.
- **Compensation code** — a tested, idempotent undo per step, some needing business decisions.
- **Workflow machinery** — an orchestrator with durable state (or choreography's poor visibility).
- **Eventual consistency** — orders really are "in progress" for a while, affecting reads, reports and expectations.

Running it: dashboards of workflows by state, age, last error and next retry; alert on any workflow stuck in a state longer than its **SLO** (service-level objective). Total throughput can look fine while a few valuable orders are stuck.

When not to use it

- **A single database transaction would work** (Fact 2).
- **In-between states must never be seen** — some financial flows can't show "charged but not shipped"; that's 2PC's one real advantage (or a sign to rethink the design).
- **Simple two-step internal flows** — a retry queue plus idempotency suffices.
- **Nobody will run the machinery** — an unwatched workflow engine is the 340-order incident with more config.

Common misuses: sagas around simple CRUD operations; **incomplete compensations** — a refund that forgets loyalty points, a release that forgets a flash-sale counter. Engineering and business owners should sign off on the full reversal together.

Orchestration vs. choreography

Orchestration: one coordinator chooses the next step. **Choreography**: each service reacts to events others publish; nobody is in charge.

	Orchestration	Choreography
Who decides the next step	One coordinator running a state machine	Every service, reacting to events
"Where is order 7f3a?"	One query to the orchestrator	A hunt through N services' logs
The saga log	Explicit, central, resumable	Spread across services, if it exists
Adding a step	Change the workflow definition	New subscriptions — hidden new coupling
Failure handling	Coordinator retries, compensates, escalates	Every service builds its own part
Fits	Workflows where order and stakes matter	Independent reactions (analytics, notifications)

Rule: orchestrate the workflow; use events only for independent reactions. The order saga is orchestrated, and its final `order.completed` fact goes out via pub/sub to analytics, email and others. Choreographing reserve → charge → ship → compensate purely through events spreads control so thin nobody can say what state an order is in.

Real-world examples

- **Travel industry** — the origin: flight, hotel, car booked in sequence, with cancellation policies as compensations.

- **Temporal** (and its predecessor **Cadence**) — turn the saga log into **durable execution** infrastructure: workflow code resumes automatically after a crash. **AWS Step Functions** provides a managed state machine.
- **Every serious order or payment system** — "processing" is an intentional saga in-between state.
- **For AI engineers:** an agent whose tools have side effects *is* a saga (book a meeting, email attendees, update a CRM). If a later call fails you need designed reversals; a durable workflow engine lets an interrupted agent resume instead of charging or messaging twice. Idempotency keys on tool calls plus compensations separate a production agent from a demo.

In the interview

Checkout across inventory, payment and shipping tests saga thinking: note that one local transaction can't cover it, rule out 2PC (blocking; the external provider can't join), then lay out steps, durable state, compensations, idempotency and orchestration.

30-second answer: "Checkout is a saga: reserve, charge, ship — each a local transaction, no cross-service locks. Progress goes to a durable saga log before every step, so a crashed orchestrator resumes. If a step still fails after retries, idempotent compensations run in reverse — refund, then release. In-between states like 'charged, not shipped' are visible on purpose, so product defines them and every reader handles them. Irreversible steps like shipping go last; failures after that pivot recover forward, e.g., with a return label. I orchestrate the workflow and use events only for reactions like analytics and email; 'where is this order?' is a single query."

Likely follow-ups (with answers):

- *"The payment call timed out — do you compensate?"* No. First learn the truth: retry with the **same idempotency key** (the provider returns the original result) or check charge status; then continue or compensate.
- *"What if the refund itself fails?"* Compensations must be as reliable as forward steps: retry the idempotent refund with backoff; if it keeps failing, escalate via a **dead-letter** queue (a side queue parking repeatedly failing messages) that alerts a person with full context.

Mistake that fails candidates: suggesting 2PC; saying "compensations" without explaining them; keeping orchestration state only in memory.

Self-check

- **Q1: In one sentence, what does a saga trade away versus 2PC, and which two mechanisms replace locks and coordinator reliance?** → It gives up isolation (visible in-between states) to avoid locks and blocking; replaced by a **durable saga log** (resume after crash) and **designed, idempotent compensations** (reverse finished effects).
- **Q2: Compensations for reserve inventory, charge card, send confirmation email, hand the package to the courier — which has none?** → Release; refund; an approved explanatory follow-up email (imperfect — so send the confirmation *late*). Courier handoff is the **pivot**: recover forward (return label, support case, redelivery). Compensable steps first, irreversible last.
- **Q3: A customer gets "Thanks for your purchase!" but the order is refunded 90 s later because shipping failed; the email service says it did nothing wrong.** → A no-isolation anomaly: it correctly reacted to the in-between "payment succeeded" state. Shallow fix: confirm only from the final `order.completed` state. Deeper: every consumer treats in-between states as provisional, and outward actions based on them need their own compensation. Tell customers only what the saga can no longer undo.

Related patterns

- **Idempotency** — on every step and compensation; retries make duplicates normal.
- **Two-Phase Commit** — the defining contrast: locking and hiding versus showing and compensating.
- **Message Queue** — steps often travel through queues, inheriting Module 2's delivery rules.
- **Publish-Subscribe** — broadcasting final events to interested parties.
- **Dead Letter Queue** — escalating persistently failing compensations to people, with context.

TL;DR

Problem	Cross-service workflows can't use locks (2PC is out); a mid-flow failure leaves half-finished business state — charged, not shipped — cleaned up by spreadsheet.
Mechanism	Local transactions with progress logged durably before each step; idempotent compensations in reverse on failure; irreversible steps (the pivot) last, recovering forward; an orchestrator owns the state machine; events broadcast only from final states.
Costs	No isolation (visible in-between states and anomalies), a tested undo per step, workflow machinery, eventual consistency while in progress.
Skip it when	One DB transaction suffices, visible in-between states are unacceptable (2PC's one win), or the flow is too simple for a state machine.

Quorum

Original lesson: Quorum on DesignGurus ↗

What it is. How a **leaderless** replicated store (any copy takes reads and writes; no primary) keeps reads reliable. With **N** replicas, a write is done when **W** confirm it and a read asks **R**. With **W + R > N**, every read group overlaps every write group, so at least one replica asked holds the newest completed write. (Basics: Module 0 — Quorums.)

> **Analogy.** Three friends keep copies of a shopping list. Tell at least two about each addition; ask at least two when checking. Since 2 + 2 > 3, one you ask is one you told.

The problem: the flickering cart

The previous lessons were *different services doing different jobs*; this one and the next are *copies of the same data* agreeing on a value.

The cart ran primary-replica, and **failover** (promoting a replica when the primary dies) took about **30 seconds** during which nobody could add items. Many datasets tolerate that; the cart feeds revenue directly, so the business wants writes through any single machine failure. The team moves to a **leaderless** store: **3 replicas**, no primary, so no promotion ever.

Version one writes to one replica and reads from one. A customer adds sneakers via replica A, reloads, and the read hits replica C, which hasn't got the update: the sneakers **disappear**, then **reappear** on a later read via A. All replicas are healthy; they just briefly disagree. Writing anywhere removed failover but also predictable reads.

Why the obvious fixes fail

- **"Write to all three, read from any one."** Reads see every finished write, but one slow or dead replica now holds up *every write* — the availability problem you left the leader to avoid.
- **"Write to one, read all three, pick the newest."** The mirror image: every read depends on every replica.
- **"Always send each user to the same replica" (sticky routing).** Works until that replica fails or the device uses another entry point — then the cart vanishes during exactly the failure you planned for. A pinned replica is a leader with weaker promises.

All three need *every* replica on one side. What's needed is far less: guaranteed **overlap** between the replicas that took a write and those a later read asks.

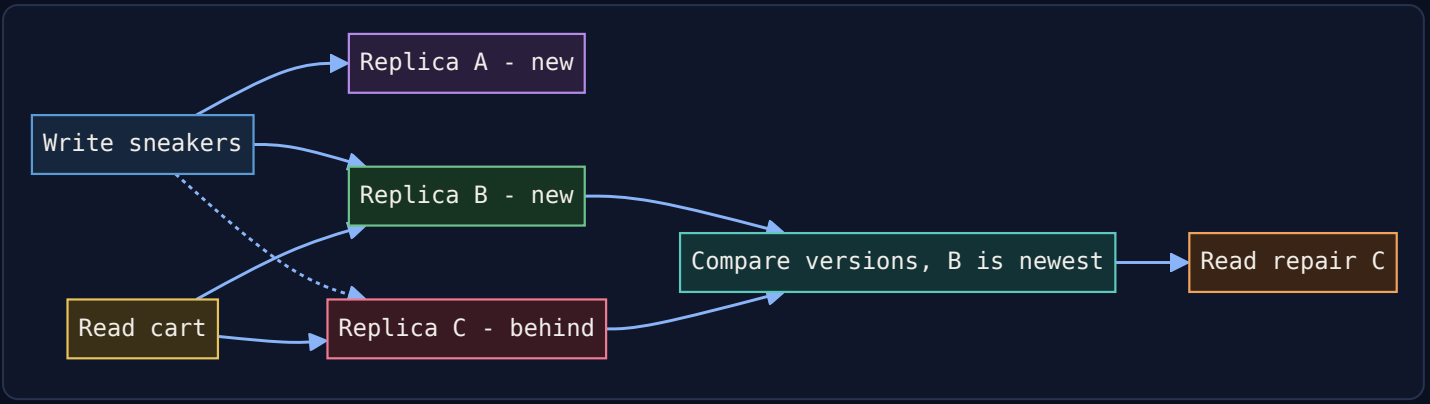
How it works

With N = 3, W = 2, R = 2 (W + R = 4 > 3):

```
WRITE "sneakers" ──▶ A ✓   B ✓   C (behind)
READ the cart    ──▶       B ✓   C ✓
                           ▲
            B is in both groups — it must be, because 2 + 2 > 3.
            The read compares versions; B's is newer, sneakers stay.
```

Two groups of 2 from 3 replicas need 4 "seats", so at least one replica sits in both — the guarantee is just counting.

N=3, W=2, R=2: the read and write groups must share a replica



Supporting mechanisms:

1. **Versions break ties.** The R answers may differ; each value carries a version and the reader picks the newest *by version*, never the fastest reply (speed says nothing about freshness). Timestamps work for simple systems; the next lesson shows why richer versions are often needed. The coordinator returns the chosen version to the client so its next write can state which history it saw — otherwise "replace what I just read" is indistinguishable from an unrelated concurrent edit.
2. **Read repair.** When a read sees C returned an old value, it writes the newer value back to C.
3. **Anti-entropy handles the rest.** A background process compares and syncs replicas, fixing rarely-read keys that read repair never reaches. Monitor version drift, repair backlog and the oldest unrepaired difference — requests can keep succeeding while one replica quietly stops catching up.
4. **W and R are per-operation settings.** W=3/R=1: slow writes, instant fresh reads (read-heavy configuration). W=1/R=3: the opposite. W=1/R=1: fast with **no overlap guarantee** (eventual consistency) — fine for like counts, and exactly the cart bug when chosen by accident.
5. **Tolerating stragglers.** With W=2 of 3 the coordinator proceeds after the two fastest replies, so the slowest replica (the **straggler**) doesn't set your **p99** latency (the time within which 99% of requests finish).

Three qualifications on the guarantee:

- **Sloppy quorums and hinted handoff.** Some Dynamo-style systems (modeled on Amazon's Dynamo, e.g., Cassandra and Riak) let temporary stand-in nodes accept writes when intended replicas are unreachable during a partition; **hinted handoff** later returns the data to the proper replicas. More availability, but read and write groups may not overlap meanwhile — reads can miss writes even with W + R > N.
- **Partial writes.** A coordinator may get fewer than W acks and report failure though one replica saved the value; a later read or retry may still see it. Writes need stable IDs or merge behavior even after "failed".
- **Quorum promises visibility, not ordering.** Two devices updating one cart at once can each reach W=2 via *different* pairs, leaving two "newest" versions — **siblings**. Vector clocks (next lesson) tell

concurrent from sequential; a single ordered winner needs **consensus** (**Raft**, **Paxos**), which adds stricter rules on top of majority counting.

Walkthrough with numbers (N=3, W=2, R=2)

- **Write.** Send "add sneakers" to all three; succeed after **2** acks — about **5 ms**, even if C is slow.
- **Read.** Ask all three, use the first **2**, compare versions, return the newest; overlap guarantees one has the sneakers. Read repair fixes C.
- **Failure drill (the point).** Replica B dies; A and C still meet W=2 and R=2. **Zero failover, zero pause, zero promotion**; B's replacement catches up via anti-entropy.
- **The honest cost — amplification.** Each logical operation takes 2–3 network calls: **100,000 logical reads/s** may mean **200,000–300,000 replica requests**, before repair, anti-entropy and retries. Size for replica traffic, not client traffic. Concurrent edits can split versions, so conflict resolution is mandatory.

Trade-offs

You gain:

- **No leader, no failover** — reads and writes continue through one replica's loss; the revenue-critical cart stays writable.
- **Per-operation tunable consistency** — one cluster can serve carts, telemetry (automatically collected metrics and events) and configuration, each with its own W/R.
- **Straggler tolerance** — the slowest replica no longer sets p99.

You pay:

- **Concurrent writes conflict** — quorum doesn't order them, so you need version tracking and resolution (a leader avoids this by serializing writes).
- **Amplification** — R or W calls per operation plus background repair traffic invisible in client counts.
- **Weakened-guarantee edge cases** — sloppy quorums and partial writes can return old data despite W + R > N; know what your store promises.
- **One key at a time** — quorum settles a single key's value; it won't make a stock decrement and a card charge atomic (2PC or sagas). Values needing one consistency decision may need to share a key — if it stays small and doesn't become a hot partition (a key or shard taking a disproportionate share of traffic).

Client guarantees also need thought. A user can read stale data right after their own write if the next request uses a weaker level. Sessions needing **read-your-writes** should carry the version they saw and reject older responses. Make consistency levels part of API design and service ownership, not buried database defaults — e.g., a checkout read at quorum while a recommendation counter on the same cluster reads one replica. Track strict and sloppy responses separately; an overall success rate hides requests that quietly got weaker guarantees during a partition.

When not to use it

- **Exactly one winner needed** — unique usernames, leader election, "sell the last item exactly once": use consensus (Raft/Paxos) or a single leader. Quorum ensures the winner is *seen*, not that there's only one.
- **Multi-key transactions** — 2PC or sagas.
- **Mostly-read data that tolerates failover** — primary-replica is simpler; a 30-second failover is fine for a product catalog. The cart's always-writable need was specific.
- **Small systems** — quorum math for a **50 requests/s** service is needless complexity.

Common failure: choosing a Dynamo-style store "for scale" without checking its W/R defaults, then learning the real guarantees from flickering production data. W and R should be deliberate product

decisions.

Leaderless quorum vs. primary-replica

	Primary-Replica (leader)	Leaderless Quorum
Write conflicts	Can't happen — all writes go through one leader	Can happen — needs versioning plus resolution
Failover	Real: seconds of write pause, plus promotion and fencing (blocking the old leader from writing)	None: one replica dying goes unnoticed
Consistency story	Easy to reason about (apart from read-your-writes)	Adjustable, subtle, full of fine print
Latency profile	Your p99 is the leader's p99	Tolerates stragglers (waits for the fastest W)
Complexity lives in	The failover machinery	The version and conflict machinery
Fits	Most data: catalogs, orders, accounts	Keys that must always accept writes: carts, sessions, presence (online/offline status)

A leader is simpler but pauses on failover; leaderless stays available but exposes conflicts. Mature systems choose per dataset.

Real-world examples

- **Amazon's Dynamo paper** — introduced the pattern for an always-writable shopping cart, and defined sloppy quorums and hinted handoff.
- **Cassandra** (open-source, Dynamo-inspired wide-column database) — per-query consistency levels `ONE`, `QUORUM`, `ALL` make W and R API parameters.
- **DynamoDB and Riak** — Dynamo descendants.
- **etcd and ZooKeeper** (small, highly reliable configuration/coordination stores) — use *consensus* (Raft and the Paxos family), which is why Kubernetes trusts their ordered control-plane data.
- **For data engineers:** feature stores (serving ML model inputs) and time-series systems on Cassandra use `QUORUM` reads where stale features could change model behavior and `ONE` for high-volume telemetry — chosen per query.

In the interview

Classic: "With three replicas, how many write acks and read responses guarantee you see the newest write?" Follow-ups probe W=1/R=1 or concurrent writes; show what quorum does and doesn't promise.

30-second answer: "With N replicas I confirm writes at W and read R, keeping W + R > N, so every read group overlaps every write group and versioning picks out the newest. N=3, W=2, R=2 is standard and survives any one replica dying with zero failover — the point of going leaderless. But quorum promises visibility, not ordering: two concurrent writes can each reach W=2 and split, so you need vector clocks or last-write-wins. Sloppy quorums trade the overlap guarantee for availability during partitions, and exactly-one-winner problems need consensus like Raft."

Likely follow-ups (with answers):

- *"W=2, R=1 on N=3?"* W + R = 3, *not* > 3: a read can hit the replica the write skipped. Faster reads, eventual consistency — acceptable only when chosen on purpose.

- *"Is W=2/R=2 strong consistency, then?"* It guarantees seeing *completed* writes, but concurrent writes can split, sloppy quorums weaken overlap, and partial writes cause edge cases. Call it "overlap visibility plus conflict resolution."

Mistake that fails candidates: treating quorum as a payment-plus-inventory transaction; forgetting the inequality; running on unexamined storage defaults.

Self-check

- **Q1: The formula and why it works, in one sentence.** → W + R > N: since the total exceeds the replica count, every write group and read group share a replica holding the newest completed write.
- **Q2: One cluster, three datasets — (a) feature flags, read constantly, written weekly; (b) the cart, always writable; (c) raw clickstream (every click and page view), write-heavy, batch-read later. Choose W and R.** → (a) W=3/R=1: rare writes can wait for all; constant reads need one reply and stay current. (b) W=2/R=2: overlap holds through one failure. (c) W=1, let the batch reader choose R; a few late events cost less than slower writes. Set per dataset or query.
- **Q3: The flickering cart returns — only during last week's inter-datacenter partition, only for some users. N=3/W=2/R=2; the docs mention "hinted handoff."** → Coordinators that couldn't reach intended replicas B and C used a **sloppy quorum**, counting stand-ins toward W=2 with hints for later; reads via A+B or A+C missed those writes. The real choice: **strict quorum** (reject writes without W true replicas — consistent, unavailable) vs. **sloppy quorum** (keep accepting writes, temporarily miss data). For a cart, availability plus later reconciliation may be right — as a deliberate CAP decision, with vector clocks reconciling versions.

Related patterns

- **Primary-Replica** — the leader-based alternative; most systems use both, per dataset.
- **Vector Clocks** — the version machinery separating split concurrent writes from sequential ones.
- **Consistent Hashing** — chooses *which* N replicas store each key.
- **Two-Phase Commit** — what databases layer over quorum-replicated groups for cross-shard transactions (Spanner).

TL;DR

Problem	Leaderless replicas always accept writes but lag each other; reading a random one makes the cart flicker, and requiring every replica on one side brings back the fragility you left the leader to escape.
Mechanism	N replicas; writes confirmed at W, reads ask R, W + R > N forces overlap; versions pick the newest; read repair and anti-entropy fix laggards; W/R tuned per dataset.
Costs	Amplified operations; concurrent writes that split and need version machinery; fine-print edge cases (sloppy quorums, partial writes); single-key only — not a transaction.
Skip it when	You need one ordered winner (consensus or a leader), atomic multi-key changes (2PC/saga), or a simple mostly-read dataset where a failover pause is fine.

Vector Clocks

Original lesson: [Vector Clocks on DesignGurus ↗]

What it is. A small version label on each key: **one counter per writer**, which each writer increments when it writes. Comparing two labels shows either that one version *knew about* the other (and replaces

it) or that they were written **concurrently** — neither saw the other, a real conflict to keep and merge, not silently discard. (Basics: Module 0 — Clocks and ordering.)

> **Analogy.** Git: branches are concurrent edits, the commit graph records who built on whose work, and a "merge conflict" means neither branch descends from the other. A vector clock is that idea squeezed to one number per writer.

The problem: which "newest" version wins?

Quorum ended with two devices updating one cart at once, each reaching W=2 via a different pair — two "newest" versions, no clear winner. "Take the later timestamp" breaks two ways:

1. **Clocks don't agree.** Machine clocks differ by milliseconds normally and by seconds when **NTP** (Network Time Protocol, which syncs clocks) breaks.
2. **"Which came later?" is the wrong question.** Alice's phone writes {sneakers}; three seconds later her *synced* laptop writes {sneakers, socks} — it saw and built on the phone's write, so it correctly replaces it. But an *offline* laptop writing {hat} never saw the sneakers: two independent real changes, and picking either loses the other regardless of timestamps.

The real question is **causality** — did one write know about the other? That is what vector clocks record.

Why the obvious fix fails

- **"Keep the clocks tightly in sync."** Even perfect clocks answer the wrong question: writes 5 ms apart are concurrent if neither saw the other, and "later wins" discards a valid change. (Spanner's precise clocks order transactions *inside a leader-based consensus system* — a different problem.)

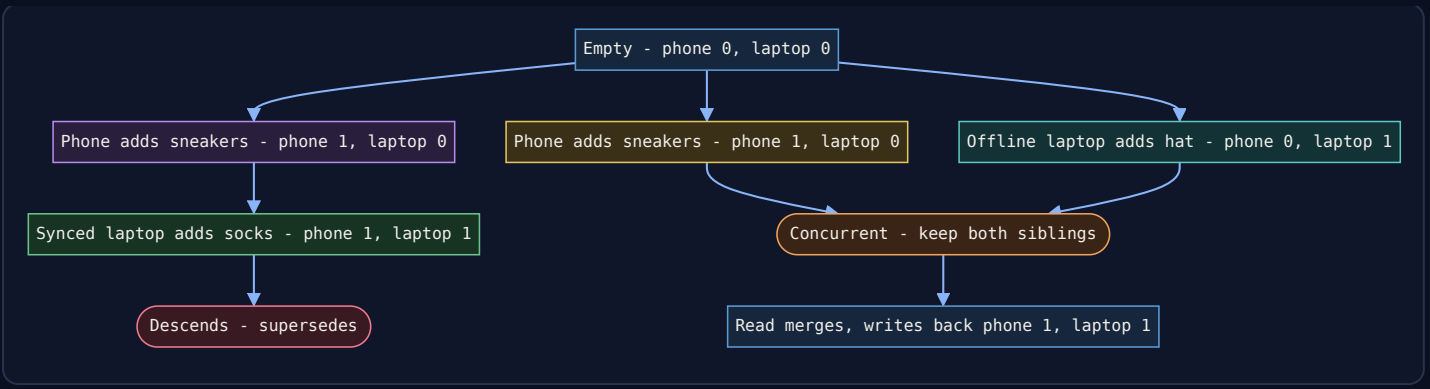
How it works

Two writers, phone and laptop:

```
cart starts empty                          {phone:0, laptop:0}
phone adds sneakers                        {phone:1, laptop:0}
laptop (synced, saw sneakers) adds socks   {phone:1, laptop:1}
  compare {1,1} vs {1,0}: >= in every slot → DESCENDS: supersedes. One story.

...or instead, from the empty cart:
phone adds sneakers                        {phone:1, laptop:0}
laptop (offline!) adds hat                 {phone:0, laptop:1}
  compare: each is ahead in one slot → CONCURRENT: a true conflict.
  Keep BOTH as siblings; resolve at the next read.
```

1. **Each write bumps the writer's own counter** — counting "how many of my writes this version includes."
2. **Compare slot by slot.** If A ≥ B in *every* slot, A **descends** from B (knew everything B knew) and replaces it.
3. **If each leads somewhere, they're concurrent.** The store keeps **both** as **siblings**.
4. **Vector clocks detect conflicts; they don't settle them.** On the next read the application merges siblings by:
   - **Merge by meaning** — for carts, the union {sneakers} ∪ {hat} (Amazon Dynamo's original approach; it has a trap — see self-check Q3).
   - **CRDTs (conflict-free replicated data types)** — counters, sets and documents whose concurrent edits combine automatically; the technology behind Figma-style live collaboration.
   - **Ask the user** — show both, like a git merge conflict.
5. **Write back one merged version whose clock descends from every sibling** — the per-slot maximum, like a git merge commit. Otherwise another replica would see it as concurrent again and the conflict would restart.

Sequential vs. concurrent histories from an empty cart



Tombstones. Union merging makes deletes risky: if the phone removes sneakers while an offline laptop's concurrent version still has them, the union restores them. Store deletions as **tombstones** — explicit records like "sneakers: removed, version X" — that take part in the merge and override the earlier "present" state; garbage-collect them later. A deletion is a *fact*, not missing data.

Trade-offs

You gain:

- **Nothing lost silently** — concurrent writes survive as siblings until a merge policy handles them.
- **Immunity to clock skew** (the gap between machines' clocks) — causality uses counters, not wall time.
- **Precise language** — "descended" vs. "concurrent" makes leaderless updates reasonable.

You pay:

- **You must resolve conflicts** — every data type needs an owned, tested merge policy.
- **Space and growth** — a counter per writer per key grows with many clients; real systems usually key counters by **replica/server**, not end-user device.
- **Pruning is risky** — dropping a retired writer's entry is safe only if it provably won't return (old replica, offline client, backup, delayed message); otherwise causal history is lost and a true descendant looks like a fake conflict.
- **Mental effort** — apps compare versions, handle siblings and maintain merge code instead of accepting whatever the database chose.

When not to use it

- **Anywhere there's a leader** — writes already flow through one leader in order, so causality is free. This pattern belongs to the leaderless world.
- **When losing a concurrent write truly doesn't matter** — presence indicators, metric samples, "last seen" fields. **Last-write-wins (LWW)** — the later timestamp overwrites — is a valid *intentional* choice; the mistake is defaulting to it and discovering the data loss in an incident.
- **When a CRDT fits** — build the resolution into the type instead of handling siblings by hand.

Vector clocks vs. last-write-wins

	Last-write-wins (timestamps)	Vector clocks
Concurrent writes	One is quietly thrown away	Both kept as siblings
Clock skew	Picks the winner (incorrectly)	Doesn't matter
Application burden	None — why people like it	A merge policy per data type
Honest description	A data-loss policy you sign up for	A conflict-detection service you maintain

Cassandra uses LWW by default, deliberately, for simplicity; **Dynamo** and **Riak** keep siblings with vector clocks. Decide by the cost of losing a real write.

In the interview

Usually after a quorum design: "Two clients update the same key at once — who wins?" Timestamps alone ignore skew, concurrency and merge ownership.

30-second answer: "Timestamps can't tell 'B overwrote A on purpose' from 'B never saw A', and skew makes 'later' unreliable. Vector clocks track causality: one counter per writer, bumped per write. Two versions either descend — one cleanly replaces the other — or are concurrent, kept as siblings for the application to merge: union for carts, CRDTs where the type merges itself, or a git-style conflict. They detect, not resolve. And last-write-wins is a data-loss policy — fine only when chosen on purpose."

Likely follow-ups (with answers):

- *"Why not just synchronize clocks tightly?"* Perfect clocks still answer the wrong question: writes 5 ms apart can be concurrent. Spanner's clocks serve a leader-based consensus system.
- *"Who resolves siblings, and when?"* The next reader receives them all, applies the type's merge policy, and writes back one version whose clock descends from all — like a git merge commit.

Mistake that fails candidates: answering with timestamps and never mentioning causality or skew; saying vector clocks *pick* a winner.

Self-check

- **Q1: Compare {phone:2, laptop:1} with (a) {phone:1, laptop:1}, (b) {phone:2, laptop:2}, (c) {phone:1, laptop:2}.** → (a) {2,1} ≥ {1,1} in every slot: ours descends and replaces it. (b) {2,2} ≥ {2,1}: theirs replaces ours. (c) Each leads in one slot: concurrent siblings needing a merge.
- **Q2: LWW, vector clocks or a CRDT for (a) "user last active at", (b) the shopping cart, (c) a shared document's text?** → (a) LWW — rough recency; losing a sample costs nothing. (b) Vector clocks + union + tombstones — a lost added item matters. (c) A CRDT — the type must combine complex concurrent edits automatically.
- **Q3: A Dynamo-style cart uses union merge. The customer removes sneakers on her phone, then adds a hat on her offline laptop. After syncing, the sneakers are back. Why, and the fix?** → Union treats "missing" as "never there", and the laptop's concurrent version still has sneakers; the deletion left no trace. Store deletions as **tombstones** ("sneakers: removed, version X") that participate in the merge, override "present", and are cleaned up later.

Related patterns

- **Quorum** — produces the siblings; vector clocks are its version machinery.
- **Event Sourcing** — causality as first-class history; an event stream is a fully ordered relative.
- **Bidirectional Streaming** — collaborative-editing CRDTs merge by design: vector-clock thinking in data types.

TL;DR

Problem	In a leaderless store, concurrent writes split a key; timestamps "fix" it by silently dropping a real write — often the wrong one, due to skew.
Mechanism	One counter per writer, bumped per write; versions compare as "descends" (replace) or "concurrent" (keep siblings); reads merge by policy (union, CRDT, ask the user) and write back a version descending from all.
Costs	Detection only (merging is yours), per-writer storage with risky pruning, real mental overhead vs. "the database picks."
Skip it when	A leader orders writes, data is short-lived enough for intentional LWW, or a CRDT handles the merge.

Capstone - Order Fulfillment, Built Twice

Original lesson: Capstone - Order Fulfillment, Built Twice on DesignGurus ↗

What this capstone does. Build the same order-fulfillment system with two-phase commit, then with a saga; hit both with the same three failures and compare the damage. The question isn't which pattern is "better" but **which way of failing the business can live with**. At each "pause and decide," predict the damage before reading on.

> **Analogy.** A relay runner stumbles. Either *every* runner freezes until the referee rules (2PC — fails by freezing), or the race continues, the crowd sees the stumble, and a rehearsed recovery plan runs (saga — fails by showing).

The brief

- **Three steps:** reserve inventory → charge payment (**external provider**) → create shipment. Rule: never *charged while nothing ships*, never *shipped without being charged*.
- **Load: 200 orders/s**, spiking on popular releases; one **SKU** (stock-keeping unit — a product code) appears in **30%** of active checkouts — the **hot SKU**.
- **Latency:** checkout confirms in about **2 s**; fulfillment may trail by minutes.
- **The cart:** from the quorum lesson, on a leaderless store because it must always accept writes.

Build One: Two-Phase Commit — fails by freezing

Design. One coordinator; inventory, payment and shipping as prepare/commit participants.

A flaw before anything breaks: the payment provider has no prepare phase (2PC doesn't cross trust boundaries). An internal **payment-intent record** can stand in as participant, but the *actual* charge happens outside the transaction:

- Commit the intent, then the charge fails → success reported, no money moved.
- Charge first, then the internal commit fails → money moved, nothing recorded.

The workaround swaps an explicit distributed workflow for an *undocumented* one that still needs durable retries and compensation — a hidden saga.

Drill 1 — the coordinator crashes after gathering yes votes. *Pause: inventory voted yes on the hot SKU. What can it do, and what happens to everyone else wanting that SKU?*

1. Inventory is in doubt — it can't commit or abort without the coordinator.
2. It holds hot-SKU row locks until the coordinator returns about **40 s** later.
3. For those 40 s, the **30%** of checkouts including the hot SKU queue behind the locks or time out.
4. One failed coordinator turns a small transaction into a broad revenue slowdown.

Drill 2 — shipping is slow, not down. During its deploy, shipping's p99 rises from **100 ms to 4 s**. *Pause: who feels it?*

1. Every transaction waits for shipping's prepare reply.
2. Inventory and payment-intent locks are held for shipping's p99.
3. Rate x duration: **200 orders/s x 4 s = 800 transactions' worth of locks** in flight, many on the same popular rows.
4. One participant's speed sets contention in all three services.

Drill 3 — a normal Tuesday (nothing broken).

1. Every order pays two round trips and several fsyncs.
2. Hot-SKU locks last tens of milliseconds instead of ~1 ms, forcing single file exactly at release-day peak.
3. At 200 orders/s, slightly longer locks multiply contention → queueing → longer locks → more queueing.
4. Capacity can collapse before any service is officially "down".

Verdict: 2PC fails by freezing — it hides partial state by *halting progress* under uncertainty, and unrelated checkouts sharing a row feel the stall.

Build Two: Saga — fails by showing

Design, and the reason behind each choice:

- **Orchestrator + durable saga log** — crashes resume, not strand.
- **Order: reserve → charge → ship** — compensable steps first; the **pivot** (irreversible shipping) last.
- **Compensations: release inventory, refund payment** — business-approved.
- **Every step and compensation idempotent** — retries are safe.
- **Confirmation email only at a final state** — customers hear only irreversible facts.
- **Short local transactions per service** — no distributed locks.

The saga log is now **consistency-critical infrastructure**: each state change durable *before* its external effect; conditional updates so only one orchestrator advances an order; search by state, age and last failure to find workflows retrying without progress.

Drill 1 — the orchestrator crashes after the charge succeeds. *Pause: same moment as 2PC's Drill 1 — money committed, coordinator dead. What's frozen?*

1. Nothing is locked.
2. The log reads "order 7f3a: reserved, charged, shipping not started"; another orchestrator resumes at shipping.
3. The stable payment idempotency key prevents a second charge after an ambiguous crash.
4. Inventory holds a *committed reservation*, not an open lock; other orders keep buying remaining stock. A would-be sitewide slowdown becomes a resumable workflow record.

Drill 2 — shipping down 20 minutes with 340 orders mid-saga. *Pause: trace them and what the customer sees.*

1. Shipping fails; the orchestrator retries with backoff for **~30 s** to separate a hiccup from an outage.
2. Sustained → idempotent **refund**, then **release**, then an approved "we couldn't complete your order" email.
3. About **90 s** per order; no spreadsheet.
4. Cost: 340 visible temporary charges and 340 ledger refunds — bounded and automatic. (2PC would have *rejected* new orders, since every participant must be up, and frozen in-flight ones with locks held.)
5. Communication improves: "Processing" once safely recorded, "Completed" only after shipping; if compensation starts, the state machine explains exactly what happened, so support needn't reconstruct it from three services' logs.

Drill 3 — the anomaly cost (nothing down). *Pause: what's quietly off on the analytics dashboard, and who else acts on it?*

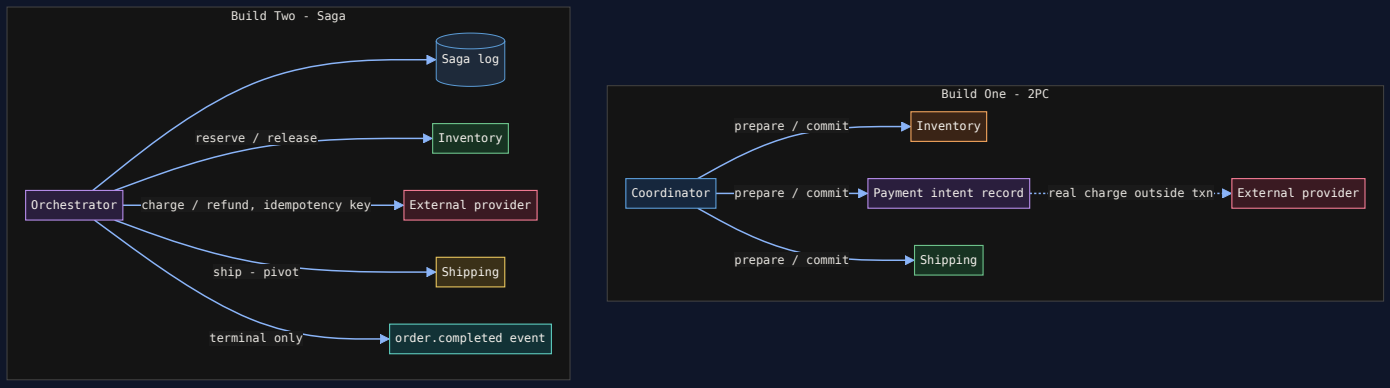
1. A revenue dashboard counting "charged" orders includes some refunded minutes later, so it reads slightly above the final finance ledger.

2. A marketing system confirming purchases from the same provisional state would cause customer-visible errors.
3. Rule: outward messages come from final states unless they have their own compensation; every consumer knows which states are provisional.

Verdict: sagas fail by showing — they reveal in-between truth (refunds, provisional states) and handle it deliberately instead of hiding it behind locks.

In Build One the real charge (dotted line) happens *outside* the transaction; in Build Two the orchestrator writes the saga log, uses idempotency keys with the provider, ships last as the pivot, and emits `order.completed` only at the final state.

Two builds of the same checkout



The comparison

Failure injected	2PC damage	Saga damage
Coordinator/orchestrator crash	In-doubt participants frozen with locks; hot SKU slows for the whole recovery	Workflow resumes from the log; nothing held, nobody waits
One service slow	Everyone's locks held for the slowest p99; a contention storm	Only that step retries; other orders unaffected; steps still take milliseconds
One service down 20 min	New orders rejected (every participant required); in-flight ones frozen	Orders compensate: 340 visible, automatic refunds
Nothing failing at all	Locks force hot rows into single file; 2 round trips + fsyncs per order	Full speed — plus provisional states every consumer must handle
The half-state ("charged, not shipped")	Nobody ever sees it — 2PC's one clean win	Visible for seconds to minutes: designed, labeled, compensated

2PC fails by freezing; sagas fail by showing — pick the failure the business can live with. Customer-facing cross-service workflows usually pick sagas.

The **gift-card transfer** remains a fair 2PC use: two internal shards, ~50 ms, one trust domain, no partial state allowed. In Spanner-class databases every participant is quorum-replicated, so one crash can't block. 2PC for short internal database work; sagas for cross-service workflows.

Postscript: the cart's own consistency

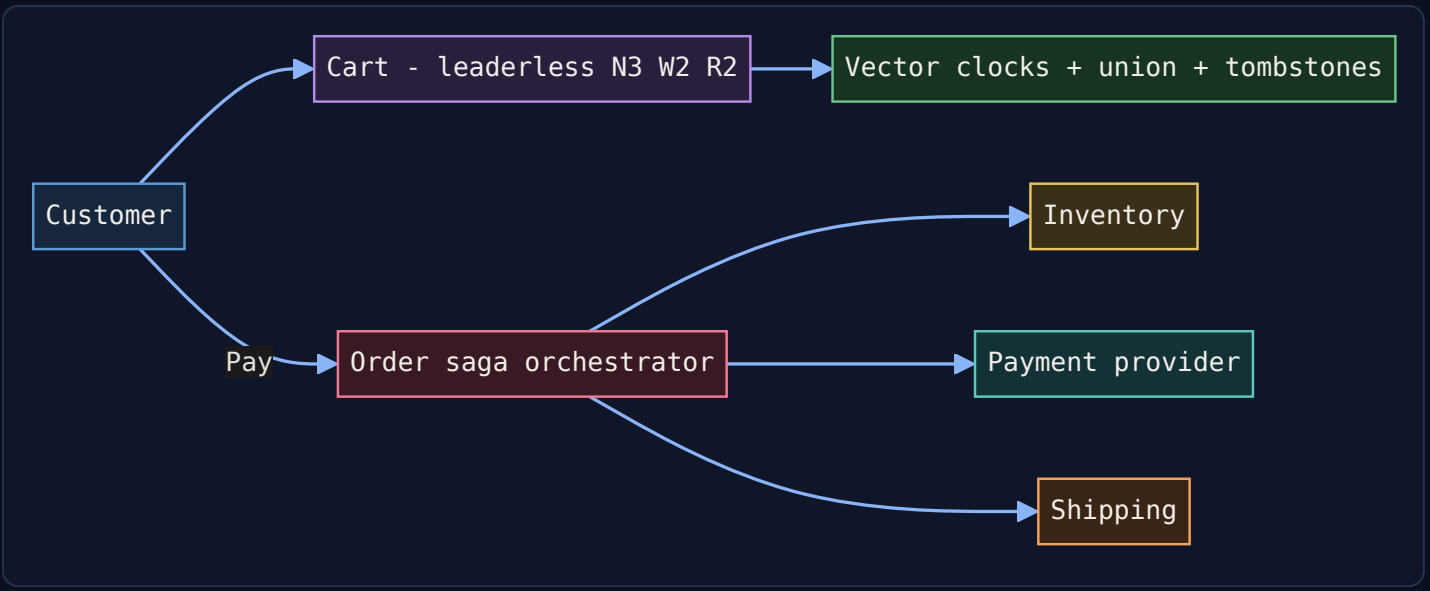
One checkout contains **two separate consistency problems**:

1. **Replica agreement for one key (the cart).** Leaderless N=3, W=2, R=2. A phone and laptop may update the cart concurrently via different replica pairs; vector clocks see neither descends, the read merges siblings by union, and tombstones keep an intentional sneaker deletion from being undone. The customer sees one correct cart.

2. **A cross-service workflow for one order (fulfillment).** Once the customer taps Pay, the saga's per-order log and compensations take over.

Good design starts by assigning each failure to *its* consistency problem.

Where each consistency pattern applies



Extend it yourself

- **Checkout across multiple regions.** The orchestrator, its log and three services span two regions. What replicates easily? Where should the saga log — the new consistency-critical state — live, and with what W and R? What happens to a mid-flight saga when the regions partition? (Fact 1 applied to the machinery itself.)
- **The provider's webhook.** The provider confirms charges later via **webhooks** (HTTP callbacks), delivered at-least-once (every message arrives, some more than once) and possibly out of order. Connect that path to the saga: which step becomes "wait for external confirmation"? What timeout rule? How do duplicate confirmations interact with the saga log?
- **For AI engineers.** An agent on a durable-execution engine books a venue, emails attendees and updates the CRM. Design the compensations, place the pivot, and decide which tool calls may run before the workflow reaches an irreversible state.

Interview summary

Built with 2PC, checkout can't even include the external payment provider, and a coordinator crash or one slow participant freezes hot-SKU locks for everyone. Built as a saga, crashes resume from the log and outages become automatic, visible refunds. 2PC fails by freezing; sagas fail by showing. For customer workflows across services, showing is the failure the business can live with.

Module 8 — The Entry Point (API and Edge)

Module Introduction

Original lesson: Module introduction on DesignGurus ↗

What this module is about. The scaling capstone ended at the **system boundary**, where clients — browsers, phones, partner systems — first reach your system. This module decides there which requests are let in, which are turned away, and which can be answered without bothering an application server. Most patterns accept, control and shape incoming traffic; the last ones apply the same ideas to calls *between* your own services.

Analogy. An office building's security desk, reception, turnstiles and signs — plus an assistant per office inside: all "stand between caller and called," in different spots with different rules.

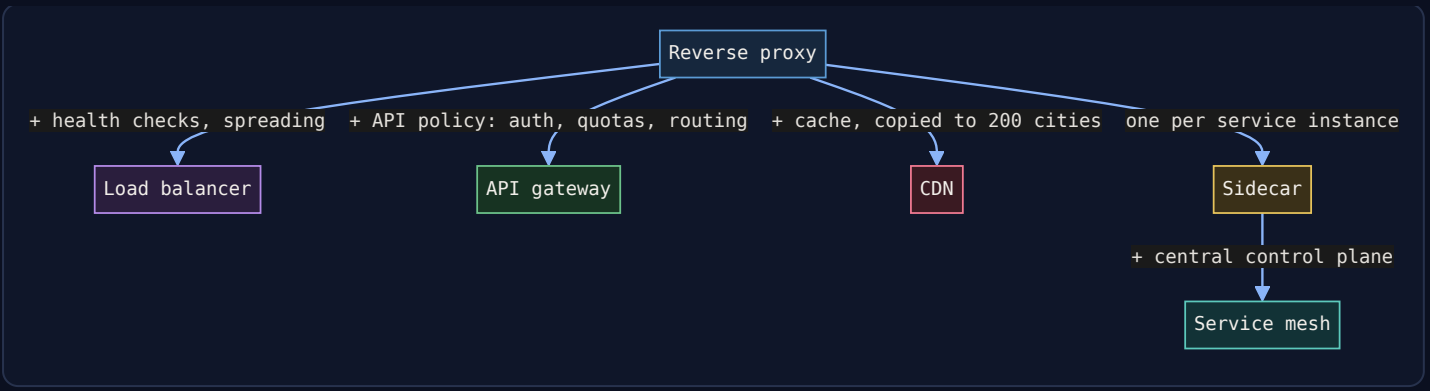
The nine patterns

The situation	The pattern
No one outside should talk to internal servers directly	Reverse Proxy
Popular content should sit close to the people reading it	CDN
A single entrance that authenticates (checks who the caller is), routes, and enforces policy	API Gateway
Different clients need the API shaped differently	Backend for Frontend
Fair use, enforced right at the entrance	Rate Limiting
Pick up a listing exactly where you stopped	Cursor Pagination
Change the API without breaking clients that already use it	API Versioning
Every service gets the same helper for shared chores	Sidecar
All those helpers, configured from one central place	Service Mesh
Every service must answer "may this user do this to this object?" the same way	Authorization Service *(added lesson)*

Two facts that organize the module

Fact 1 — nearly everything here is one building block playing different roles. The **primitive** (basic building block) is the **proxy**: a server that receives requests and passes them on. In front of your servers it's a **reverse proxy**; add health checks and traffic spreading and it's a **load balancer**; add API policies (who may call, how often) and it's an **API gateway**; copy it with a cache into 200 cities and it's a **CDN**; put a small one next to every service and wire them together and you get a **sidecar**, then a **service mesh**. (Basics: Module 0 — Proxies and API gateways, CDNs and anycast.)

One primitive, many roles



Fact 2 — the edge is where you're cheapest and strongest. The cheapest request never reaches the app tier: rejecting bad logins or excess usage at the edge costs no service capacity, and serving cached pages or static files (identical for every user — images, scripts, stylesheets) skips app work entirely. The edge is also the *single* place every request passes, so authentication, rate limits and version policies are applied once there instead of rebuilt, slightly differently, in every service.

North-south vs. east-west traffic

- **North-south** traffic crosses the system boundary (outside clients, browsers, partners) — governed by gateways, CDNs, rate limiters and version policies.
- **East-west** traffic flows *between* internal services (checkout → inventory → pricing) and needs service-to-service trust, retries and tracing (recording one request's path and timing across hops) — supplied by sidecars and meshes.

Both use the proxy primitive but trust different things; that split divides the module and comes up often in interviews.

How to read the module

Outside-in: **Reverse Proxy** (defines the primitive) → **CDN** → **API Gateway** → **BFF** → three gateway policies (**Rate Limiting**, **Cursor Pagination**, **API Versioning**) → east-west **Sidecar** and **Service Mesh** → **Authorization Service** (added lesson: the gateway authenticates, but whether *this user* may touch *this object* belongs to a shared decision service — roles, attributes and Zanzibar-style relationship graphs, Zanzibar being Google's permission system) → the **capstone**, a public API layer for web and mobile.

> Short on time? Read **API Gateway**, **Rate Limiting** and **CDN** first — every "design X" interview draws this boundary, and follow-ups dig into those three.

Terms used throughout

- **Edge vs. origin** — the *edge* is near the client, where traffic enters; the *origin* holds the authoritative servers and data. Answer from the edge when you can.
- **Forward vs. reverse proxy** — a *forward* proxy acts for clients and hides *who is asking* (e.g., a corporate web filter); a *reverse* proxy acts for servers and hides *who is answering*. Everything here grows from the reverse kind.
- **Policy enforcement point** — one place applying a rule instead of N copies; authentication, usage limits and quotas usually live here.

Before you start — questions about your own system

1. How many places in your code check authentication? (Usually surprisingly many — that number is the case for a gateway.)
2. What share of requests could be served without reaching your servers — and are they?
3. When your API's response shape changes, what happens to the year-old mobile app you can't force-update?

Reverse Proxy

Original lesson: Reverse Proxy on DesignGurus ↗

What it is. A server in front of your application servers: every request lands on it first, it handles the "boundary chores," then forwards to an app server. Clients never talk to app servers directly.

> **Analogy.** A restaurant host: guests never enter the kitchen; the host greets them, handles slow or chatty ones, hands out menus (static content) and passes orders in — so the kitchen can cook, and even rearrange, unseen.

The problem it solves

An internet-facing app server must do jobs it's bad at:

- Handle **TLS certificates** (TLS is the encryption behind HTTPS; the certificate proves identity).
- Serve images and JavaScript alongside business logic.
- Expose its address and internal layout (**topology**).
- Least obvious: **cope with slow clients** — a phone dribbling one request over **4 seconds** holds an app thread the whole time.

App servers excel at fast, predictable internal traffic; a reverse proxy absorbs the slow, hostile, messy internet and hands work inward in a tidy burst.

How it works — the six jobs

Clients reach the proxy — the one public entry point — which routes and load-balances across App Server 1–3, keeps them hidden, and returns responses.

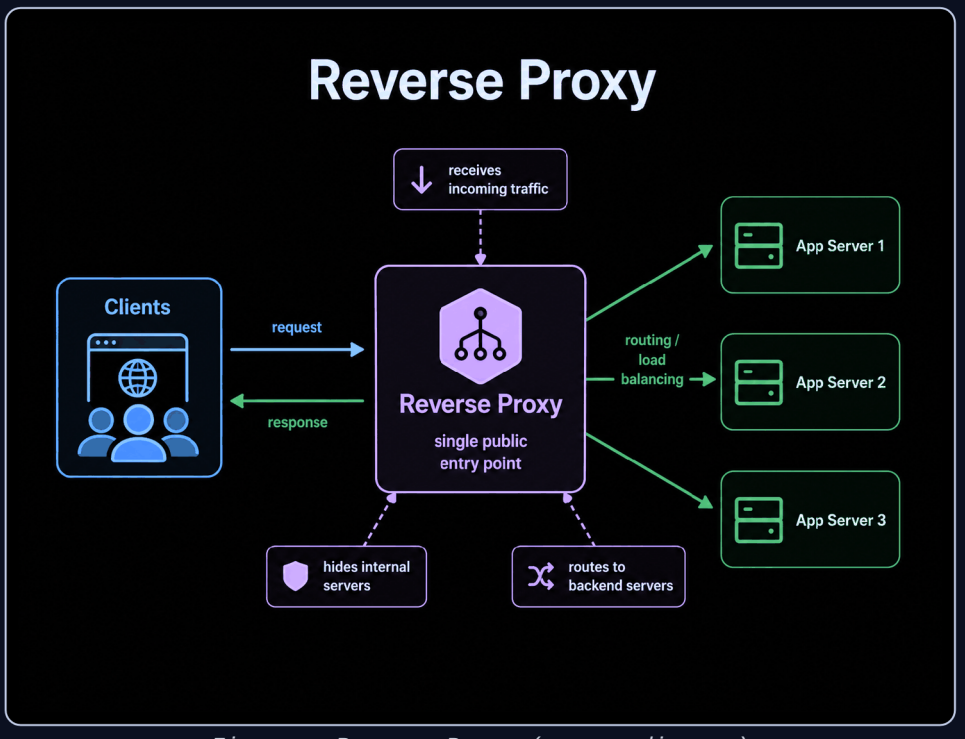


Figure — Reverse Proxy (course diagram)

1. **Terminates TLS.** Certificates, renewals and protocol settings live in one layer — one place to configure and patch.
2. **Buffers slow clients.** An **event-loop proxy** juggles many connections on one thread (the async I/O model from the SSE lesson), holding thousands of idle connections for a few KB each. It waits until a slow upload is *completely* received, then sends one fast request inward. App threads are costly; a slow phone shouldn't tie one up.
3. **Serves static files itself.** Images, JavaScript and CSS never reach app code.

4. **Reuses connections. Keep-alive** lets one client connection carry many requests; **connection pooling** keeps warm connections to the backends. Handshakes are paid once.
5. **Hides the topology.** One public address; internal servers can be renumbered, swapped or added freely.
6. **Serves as the enforcement point.** Every request passes here — the natural home for authentication, limits and the other policies in this module.

The family. + health checks and spreading → load balancer (Module 6); + auth, quotas, routing → API gateway; cached copies in 200 cities → CDN; a small proxy per service → sidecar; connected → mesh. One primitive, five names.

Walkthrough with numbers (buffering)

- **300 requests/s**, each taking **3 s** to arrive from slow clients. Unbuffered, app threads wait for the whole transfer: 300 x 3 = **900 threads**.
- Buffered, the app gets complete requests processed in **50 ms**: 300 x 0.05 = **15 threads** — about **98%** of the pool freed.
- Rule: *threads in use = request rate x how long each request holds a thread* (Little's law). Buffering shrinks "how long" from transfer time to processing time.

Trade-offs

You gain: the internet's messiness (slow clients, TLS, static files, junk requests) absorbed before business logic; one address, one certificate location, one enforcement point, with a freely changing fleet behind; low cost — a modern proxy forwards in **well under 1 ms** and handles enormous connection counts on modest hardware.

You pay:

- **An extra hop** — sub-millisecond, plus one more system to patch and watch. Everything depends on it, so it must be **redundant** across failure domains (groups that fail together, e.g., a rack or datacenter): paired instances, or **anycast** (one IP advertised from several places; the network picks a nearby working one).
- **A trust boundary to configure correctly.** The app now sees the *proxy's* IP, so the proxy writes the real client address into `X-Forwarded-For` — a header clients can also send and forge. Rules: the proxy **overwrites** any client-sent value (never appends), and the app trusts the header **only** from its own proxy's address.
- **A false sense of safety** — a proxy isn't a security control; it's where you *put* controls.

When not to use it

Almost everyone uses one; the question is whether you need *another* layer:

- **Your platform already provides one** — a cloud load balancer or hosted front end; nginx behind it adds delay and another header-trust boundary with no separate owner or purpose.
- **Small, purely internal services on a trusted network** — until they need encryption, retries or **observability** (logs, metrics, traces); then the sidecar brings the primitive east-west.
- **Don't stack proxies thoughtlessly.** CDN + cloud LB + nginx + app = four sets of timeouts, body-size limits and header-trust rules to check per incident; a rejected **12 MB** upload may mean digging through all four.

In the interview

Usually a box you're expected to draw, not a question. Expect "why not expose the app servers directly?" or an `X-Forwarded-For` bug scenario.

30-second answer: "A reverse proxy fronts the fleet: it terminates TLS, buffers slow clients so a 4-second mobile upload doesn't hold an app thread, serves static files, pools connections and hides the topology behind one address. It's the primitive under half of system design — add

health checks and spreading for a load balancer, API policy for a gateway, geography plus cache for a CDN. The config detail I always check: the app trusts client-address headers only from the proxy, and the proxy overwrites whatever outsiders send — otherwise rate limits and audit logs run on forgeable data."

Likely follow-ups (with answers):

- *"What does buffering really save?"* Busy threads drop from rate x *transfer* time to rate x *processing* time; the proxy holds slow sockets cheaply on its event loop.
- *"Forward vs. reverse?"* See Terms above (and self-check Q1).

Mistake that fails candidates: trusting `X-Forwarded-For` without checking its source; not knowing why the app sees the proxy's IP; stacking proxies without clear jobs.

Self-check

- **Q1: Forward vs. reverse in one sentence, then the reverse proxy's four main jobs.** → Forward acts for clients, hiding who's asking; reverse acts for servers, hiding who's answering. Jobs: TLS termination, buffering slow clients, serving static files, reusing connections — plus the strategic job of single enforcement point.
- **Q2: Phones upload 200 photos/s, ~2 MB each, ~3 s per upload. App threads busy with and without buffering (0.1 s processing)?** → Without: 200 x 3 = **600**. With: 200 x 0.1 = **20**. A **30x** reduction.
- **Q3: After launch the rate limiter says 99% of traffic comes from one client and blocks it, taking the site down; the audit log blames `10.0.0.5` — the proxy — for a suspicious admin action.** → Both used the proxy's address instead of a trusted client address: every user looked like `10.0.0.5`. Fix: the proxy sets `X-Forwarded-For` from the real connection source, overwriting incoming values; the app trusts it only from approved proxy addresses; everything using client identity (limits, logs, geo rules, bans) reads that same verified value.

Related patterns

- **Load Balancing** — health checks and distribution on the primitive.
- **API Gateway** — API authentication, quotas, routing.
- **CDN** — cached proxies across geography.
- **Sidecar** — a small reverse proxy per service instance, east-west.

TL;DR

Problem	Directly exposed app servers take on the internet's problems: TLS in app code, threads stuck on slow clients, visible topology, scattered policy.
Mechanism	A front server terminates TLS, buffers slow clients, serves static files, pools backend connections and forwards complete requests: one address outside, an invisible fleet inside, one enforcement point between.
Costs	A sub-millisecond hop, one more component that must be redundant, and header-trust discipline (client address set by exactly one party, trusted from exactly one source).
Skip it when	Your platform already provides one (don't stack), or the service is small and internal — until the sidecar brings the primitive east-west.

CDN

Original lesson: [CDN on DesignGurus ↗]

What it is. A **Content Delivery Network (CDN)** copies the reverse proxy into 200+ cities, each copy with a cache. Each user reaches the nearest **edge** location, which answers from cache and, only on a miss, fetches from your **origin** — once per edge, not once per user.

> **Analogy.** Instead of every Australian reader mailing a library in Virginia, the publisher stocks popular books at a Sydney branch, which orders from Virginia only for titles it lacks.

The problem: six-second pages in Sydney

The store opens in Australia. Sydney sees **6-second** page loads; Virginia sees **400 ms**. The origin is healthy — **20 ms** replies, no errors. The time goes to *distance*:

- One Sydney↔Virginia **round trip** is around **200 ms**.
- The **TLS handshake** needs **2–3 round trips**: **400–600 ms** before any content.
- The page then loads **60** images, scripts and styles, whose dependencies add more round trips even with connection reuse and parallel fetches.

More Virginia servers can't speed up light; the content must already be near the reader.

Why the obvious fixes fail

- **"Optimize the backend."** The origin's 20 ms is ~**3%** of the delay; ~97% is geography. Split server time from network time first.
- **"Open a datacenter in Sydney."** A full region fixes distance eventually, but means running the whole stack there and replicating the database across regions — the consistency module's hardest trade-offs, just to move identical images and JavaScript.
- **"Use HTTP/2 and buy more bandwidth."** **Multiplexing** (several requests over one connection at once) reduces connections and improves parallelism; bandwidth moves more bytes at once. Neither removes the round-trip wait before bytes start arriving.

Most slow content is identical, popular and static: a **distribution** problem, not a computation problem.

How it works

The Sydney user reaches the Sydney edge in 15 ms (a London user their edge in 12 ms). On a miss, an edge asks the **origin shield** — a single mid-tier cache — which fetches from the Virginia origin **once** and caches it.

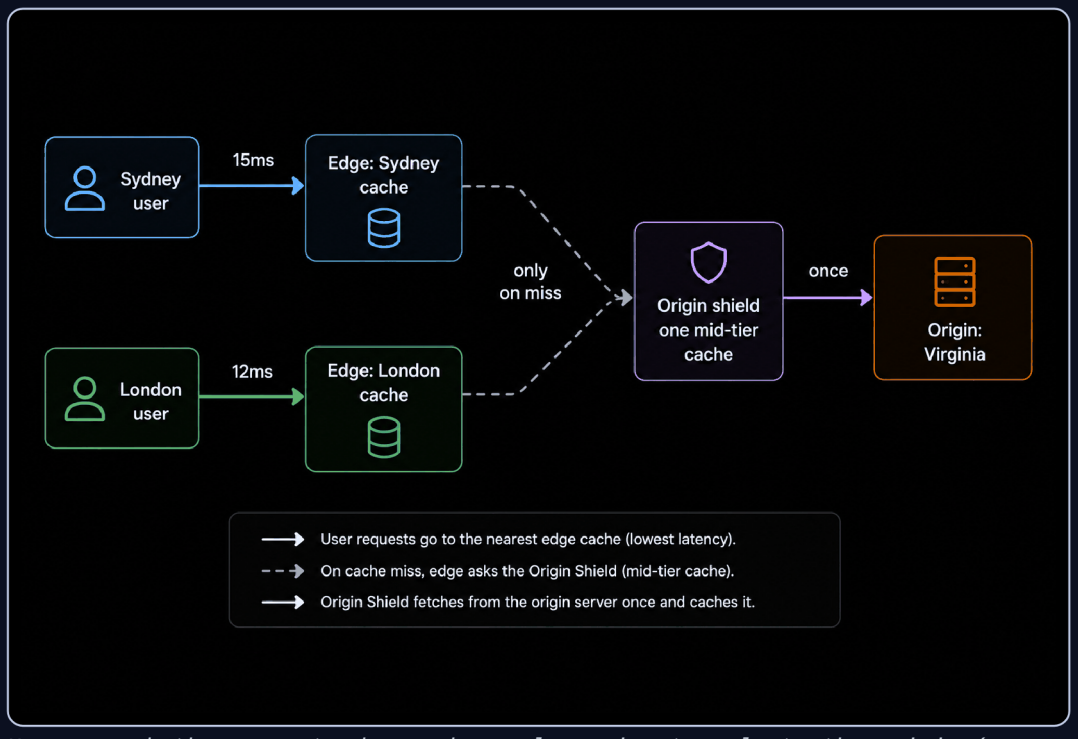


Figure — Users reach the nearest edge cache; only a miss travels to the origin (course diagram)

1. **Route each user to the nearest edge** via DNS or **anycast**. Handshake and cached bytes are ~15 ms away instead of ~200.
2. **The rules travel in a header.** `Cache-Control` tells 200+ edges what to store and for how long:
   - `max-age` — lifetime in seconds; browsers obey it too.
   - `s-maxage` — lifetime in **shared caches** (CDNs) only.
   - `stale-while-revalidate` — serve the old copy while one background refresh runs.
   - `stale-if-error` — serve the old copy while the origin fails. Module 4's rules all apply: **hit ratio** (share answered from cache), **TTL** (validity period — your staleness budget), simultaneous misses.
3. **Beat invalidation by never invalidating — fingerprinted URLs. Invalidation** (removing stale copies) takes seconds to minutes across 200 cities. Instead the build tool puts a content hash in the filename (`app.3f9a2c.js`) and marks it cacheable forever with `immutable`; changed content gets a *new* URL, so a stale response can never live at the new address.
4. **Get the cache key right.** The **cache key** is the URL plus declared headers. Cookies are the classic trap: varying on every cookie makes every request unique (hit ratio → zero); caching a `Set-Cookie` response (often a login session) serves private content to others.
5. **Add an origin shield.** Without one, a popular file missing at 200 edges triggers 200 origin requests — a global **cache stampede**. The shield **coalesces** identical misses into one origin fetch.
6. **Use the edge for more than caching.** TLS ends nearby, so a 3-round-trip handshake costs **45 ms** instead of ~600 ms. Warm **edge-to-origin connections** reuse handshakes, speeding even *uncacheable* dynamic requests. The provider's huge network absorbs **DDoS** floods. `stale-if-error` gives **graceful degradation** during origin outages (e.g., Cloudflare "Always Online").

Walkthrough with numbers (Sydney, edge 15 ms away)

- **First byte.** TLS: 3 x 15 ms = **45 ms** instead of ~**600 ms**, for cacheable *and* dynamic requests.
- **The 60 assets.** With warm caches ~**95%** come from the edge in about **20 ms** each, mostly in parallel: the page loads in roughly **1.2 s** instead of 6 s. Dynamic content still crosses the ocean, but the site stops feeling broken.
- **Origin load.** **70%+** of requests stop at the edge (the scaling capstone's "most traffic shouldn't reach app servers"). **CDN egress** (bytes the CDN sends) is usually cheaper per GB than origin traffic.
- **Flash-sale drill.** A hero image goes from **cold** (in no cache) to globally popular in a minute: ~**200** edges miss at once, the shield makes **one** origin fetch, and the edges serve the next **10 million** requests.
- **Deploy drill.** New JavaScript ships as `app.8c1d4e.js`; fresh HTML (short `s-maxage`) points to it. No purge, no cross-city coordination; the old file sits unused until evicted.

Trade-offs

You gain: latency that works with physics (content and handshakes near users); origin load down **70–95%** for content-heavy sites, plus cheaper bandwidth; DDoS absorption and `stale-if-error` availability during origin outages.

You pay:

- **Staleness across 200 cities.** Purges propagate, not instantly; fingerprinting fixes assets, but HTML and API responses need explicit TTL budgets.
- **Header rules are security.** One loose header can put private content in a shared cache; review cache directives like auth code.
- **Harder debugging** — which edge, hit or miss, stored when? Check `cf-cache-status` and `x-cache` first.
- **Per-GB fees and a third party on the critical path** — very large sites use several CDNs.

When not to use it

- **Purely internal apps** — two offices don't need 200 cities.
- **Single-region products with a nearby origin** — weaker latency case (DDoS protection and offload may still justify it).
- **Every response unique** (heavily personalized APIs) — shared caching doesn't help, though edge TLS and warm connections do; API-only products sometimes use CDN *acceleration* without caching bodies.
- **Hiding a slow dynamic origin** — the edge can't erase a **2-second** server render. Fix the query (Module 4's rule).

Dangerous misuse: caching private data. An API response with `Set-Cookie`, or an account page missing `private`, can be handed by a shared cache to another user. The CDN obeyed the headers; the headers were wrong.

Fingerprinted URLs vs. purging

	Fingerprinted URLs (app.3f9a2c.js)	Purge ("delete /banner.jpg everywhere")
Propagation	Instant — a new URL, nothing to invalidate	Seconds to minutes depending on provider; you must trust it
Can stale content exist?	Not by design	Yes, while the purge spreads
Cache lifetime	Forever + immutable — best hit ratio	Whatever TTL you dared set
Works for	Anything linked from elsewhere (assets referenced from HTML)	The linking documents, and emergencies
Cost	Build-tool setup (standard in bundlers)	An API call — plus trust

Standard setup: fingerprint referenced assets and cache them forever; give the referencing HTML a short `s-maxage` plus `stale-while-revalidate`; purge only for emergencies and legal takedowns.

Real-world examples

- **Akamai, Cloudflare, Fastly, CloudFront** — the pattern as an industry; Cloudflare alone carries a large share of web traffic.
- **Netflix Open Connect** — cache servers *inside* ISPs; for video-heavy traffic, skipping the ocean beats speeding it up.
- **RFC 5861** — standardizes `stale-while-revalidate` and `stale-if-error` (Module 4's stale serving and Module 5's fallbacks as HTTP headers).
- **Edge compute** — Cloudflare Workers and Lambda@Edge run small pieces of your code (auth, redirects, experiment selection) on CDN servers near users.
- **For AI engineers:** model weights are huge static files delivered via CDNs, and GPU inference **cold starts** pull them through the same network. Token streams are per-request and uncacheable but still gain from edge TLS and warm origin connections, cutting **time to first token** for distant users.

In the interview

Classic prompt: "Users in Australia see slow pages, but the Virginia servers look healthy." Spot distance before blaming capacity, then cover invalidation and personalization.

> **30-second answer:** "It's distance: 200 ms per round trip, three of them for TLS, and no origin tuning fixes that. I'd put a CDN in front: fingerprinted assets cached forever, HTML on a short shared-cache TTL with stale-while-revalidate, TLS ending at the edge so the handshake runs over 15 ms, an origin shield merging global misses, warm edge-to-origin connections for dynamic requests. And I review Cache-Control like auth code, because one wrong header serves private

data from a shared cache. Seventy-plus percent of requests never cross the ocean; the rest cross once, warm."

Likely follow-ups (with answers):

- *"Marketing needs the banner changed now, but assets are cached forever."* The new image gets a new fingerprinted URL; only the referencing HTML updates (short TTL or one emergency purge) — why the policy has two layers.
- *"Can API responses go on the CDN?"* Same-for-everyone responses yes (`s-maxage` , correct cache key); personalized bodies never in a shared cache, though they gain edge TLS and warm connections.

Mistake that fails candidates: tuning backend code for geographic delay; relying only on purges; caching a personalized API without defining its cache key.

Self-check

- **Q1: Why can't origin tuning fix Sydney, and which two latency costs does the edge remove?** → ~200 ms of travel per round trip, not 20 ms of compute; TLS needs 2–3 trips, then dependent asset fetches. The edge removes the **content journey** (cached bytes 15 ms away) and the **handshake journey** (~600 ms → 45 ms).
- **Q2: Header policies for (a) a product image, (b) product-page HTML, (c) cart JSON.** → (a) Fingerprinted URL with `Cache-Control: public, max-age=31536000, immutable` . (b) `s-maxage=60, stale-while-revalidate=300` — one minute in shared caches, stale during refresh. (c) `Cache-Control: private, no-store` .
- **Q3: After a deploy touching only caching headers, users see other customers' account dashboards.** → Personalized responses became shared-cacheable (added `public` / `s-maxage` or removed `private` / `no-store` ; caching a `Set-Cookie` response does the same), so the CDN stored one user's dashboard under a shared URL. Now: purge affected entries and restore `private, no-store` on authenticated responses. Prevention: treat cache directives as security-sensitive — `private, no-store` by default on authenticated routes, automated checks, auth-level review for anything loosening them.

Related patterns

- **Reverse Proxy** — spread across cities, plus caching.
- **Cache-Aside** — browser → CDN → app cache → database, each layer handling the previous one's misses.
- **Cache Stampede Prevention** — the shield merges global misses; `stale-while-revalidate` standardizes stale serving.
- **Graceful Degradation** — `stale-if-error` as a cached fallback.

TL;DR

Problem	Distance — ~200 ms per ocean round trip, tripled by TLS and compounded by asset fetches — makes healthy origins feel broken far away; backend tuning can't touch travel time.
Mechanism	The reverse proxy in 200 cities with caches: fingerprinted assets forever, short-TTL HTML with stale-while-revalidate, TLS at the edge, an origin shield merging misses, warm connections for the uncacheable.
Costs	Global staleness management, security-relevant headers, harder debugging, per-GB fees, a third party on the critical path.
Skip it when	Apps are internal, users are in one region, or every response is unique — and never to hide a slow dynamic origin.

API Gateway

Original lesson: API Gateway on DesignGurus ↗

What it is. The one entrance for all **north-south** traffic. It routes each request by path to the right service, enforces shared rules once — authentication, rate limits, TLS, validation — and passes the verified identity inward. Nothing reaches a service unchecked, and no client needs to know your internal layout.

> **Analogy.** A hotel front desk: check in once, show ID once, get a key card that room service and the gym trust. Guests don't need to know who works where. But the desk doesn't cook or run the spa — each department does its own work.

The problem: eleven front doors

The store runs **11 services** (catalog, search, checkout, orders, accounts, reviews, …), and web, iOS and Android call each directly:

- **Authentication is built 11 times.** Every service checks tokens (signed strings issued at login proving who the client is) with whatever library version it last shipped. An audit finds **3 implementations in 2 languages**; reviews' is **14 months old** and ignores **token revocation** (cancelling a token before expiry). After a stolen account's tokens were revoked, they could *still post reviews* — a security incident.
- **The mobile app hardcodes 9 hostnames** (like `orders.shop.com`) — each a promise to clients and an attack surface, unchangeable without an app-store release.
- **Internal refactors break customers.** Splitting orders in two breaks partner integrations; the internal **topology** has become public API.
- **Rate limits, log formats and TLS settings** differ slightly across all 11; none is reliably "the standard".

Root cause: **cross-cutting policy** (rules for every request) is built per service, changing at 11 deploy speeds in 3 languages. It *will* drift. Discipline can't fix structure — a single shared entrance can.

Why the obvious fixes fail

- **"Use a shared auth library."** Consistent only if all 11 adopt and keep upgrading — reviews fell 14 months behind precisely because upgrading was optional. Several languages need parallel versions, and hostnames and topology stay exposed.
- **"Put nginx in front for routing."** One address and hidden topology, but authentication still lives in 11 places.
- **"Put it in the engineering standards doc."** A document can't overrule independent release schedules; drift follows structure.

The reverse proxy must become a real **enforcement point**.

How it works — five responsibilities

Web, Mobile and Partners all enter through one API Gateway (authN · rate limits · routing · TLS · validation · request IDs), which routes `/catalog/*` (with verified identity), `/orders/*` and `/reviews/*`; other services stay invisible.

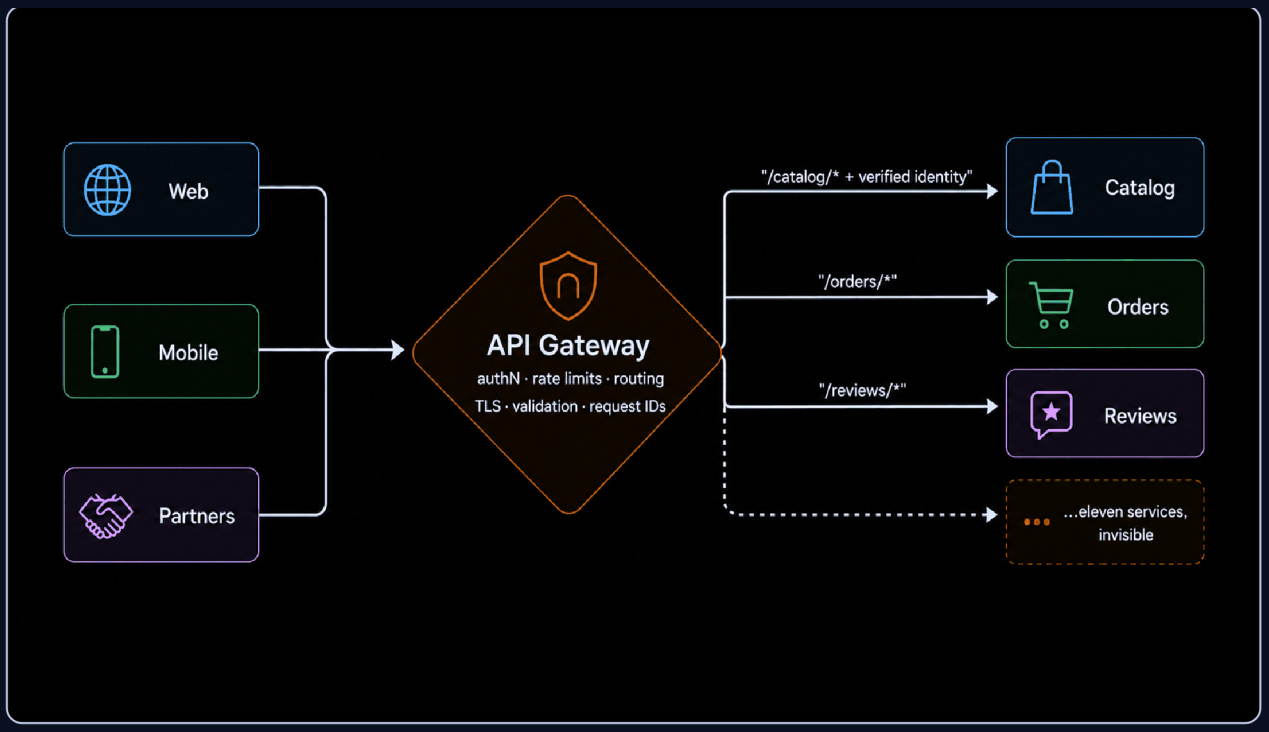


Figure — API Gateway (course diagram)

1. **Routing — topology private again.** Paths map to services (`/orders/*` → orders) behind one hostname; splitting orders into order + fulfillment is an invisible routing change.
2. **Authentication, once.** The gateway checks the token's signature (a cryptographic seal proving issuer and integrity), expiry and revocation, then passes verified identity inward as **signed headers or trusted claims** (facts in a token such as user ID and roles). Services never parse raw external tokens; one revocation fix protects all, in one deploy.
3. **AuthN at the gateway, AuthZ in the services. Authentication (AuthN)** — "who are you?" — applies everywhere. **Authorization (AuthZ)** — "may you do *this*?" (edit this review) — is domain logic, kept with each service's rules (see Authorization Service).
4. **The policy set.** Rate limits and quotas, request-size limits, **schema validation** (request has the expected shape), TLS — applied uniformly at the one layer seeing every external request.
5. **The root of observability.** It generates the **request ID** that distributed tracing is built around, so every later **span** (a timed step in a trace) links to the request's arrival, and writes one uniform audit record per external request.

The rule that keeps it stable: policy yes, product no. Everything passes through it, so teams want response transformations, multi-service orchestration, discount checks… each pushing it toward a hidden monolith needing one team's sign-off per feature and risking *all* incoming traffic on every deploy. Older architectures called this central integration layer an **ESB (enterprise service bus)**, and it was painful. **Test:** can you state the rule without domain words? If not, it belongs in a service. A thin, rarely changed gateway is a stable one.

North-south only. Internal (**east-west**) calls must never exit through the gateway and come back in — added latency and needless coupling. Internal traffic has the sidecar and mesh.

Walkthrough with numbers (one gateway for 11 services)

- **Auth:** one implementation checks each token in about **1 ms** with cached signing keys; the revocation fix ships in **1 deploy instead of 11**.
- **Mobile app: 1** hostname instead of 9; internal address changes need no app-store release.
- **Refactor drill:** orders splits into order + fulfillment via two routes; clients notice nothing.
- **The bill: 1–2 ms** per request, and one component whose failure hits all incoming traffic — so it runs redundantly with load-balancer machinery (health checks, paired instances, anycast at scale) and stays thin so it's rarely deployed.
- **Audit:** one consistent record per external request (identity, route, request ID), independent of 11 services agreeing on a log format.

Trade-offs

You gain: policy in one place (working revocation, limits, validation — drift impossible by design); a private topology; one observability root (request IDs, consistent logs, every trace's first span); simpler clients (one hostname, one auth flow, one error vocabulary).

You pay:

- **A deliberate SPOF** — softened by redundancy and thinness (fewer changes = fewer outages).
- **An extra hop on everything** — 1–2 ms per external request, plus capacity for 100% of incoming traffic.
- **An organizational bottleneck risk** — if one central team approves every route, everyone slows. Use routing-as-code, team-owned config and automated review so routine changes are self-service.
- **A permanent disguised-monolith risk** — the urge to add domain logic never ends; keep enforcing the boundary.

When not to use it

- **A single service** — a monolith already has one entrance; a reverse proxy covers boundary work.
- **East-west traffic** — latency, gateway capacity and false dependencies; that's the mesh's job.
- **Very small scale** — two services and one client may not justify it; the case begins when a *second* independent auth implementation appears.

Watch out for the ESB coming back. Transformations, orchestration and business rules each seem reasonable and together form a distributed monolith centered on the gateway. If product features routinely need gateway changes, the boundary has broken. Client-specific shaping belongs in a BFF (next lesson).

API Gateway vs. Service Mesh

	API Gateway	Service Mesh
Direction	North-south: outside world → your services	East-west: your services → one another
Identity question	"Which user or client is this?" (tokens, API keys — secret strings identifying a calling program)	"Which service is this?" (mutual TLS, workload identity)
Policy flavor	Auth, quotas, versioning — product-facing rules	Retries, timeouts, mutual TLS, traffic splits — infrastructure rules
Topology	One (redundant) entrance	A sidecar per service — hundreds of small entrances
Failure blast radius	All incoming traffic	One service at a time

They complement each other; using either in the wrong direction causes exposure or needless detours.

Real-world examples

- **Kong, Apigee, AWS API Gateway** — open-source, commercial and managed versions.
- **Netflix Zuul** — a well-known large-scale gateway whose history shows product logic piling up until BFFs were needed to pull client shaping back out.
- **Envoy-based gateways (Emissary, Gloo)** — the same proxy code at the edge and inside meshes; role depends on placement and config.
- **For AI engineers:** an **LLM gateway** is this pattern aimed at model providers: authentication becomes API-key custody, rate limits become token budgets, path routing becomes model selection — plus usage metering (who used how much, for billing), provider fallback and semantic caching (reusing answers to equivalent questions).

In the interview

After drawing microservices, expect "how do outside clients reach them?", then probes on responsibilities, availability and avoiding a central monolith.

30-second answer: "All north-south traffic enters one gateway: path routing keeps topology private; cross-cutting rules run once — token validation with revocation, rate limits, size caps, TLS; verified identity flows inward. The gateway authenticates, services authorize. It stays thin — policy yes, product no — or it becomes a monolith with an approval queue. It runs redundantly, creates the request ID, and east-west calls never detour through it."

Likely follow-ups (with answers):

- *"Isn't it a single point of failure?"* Deliberately shared, run like the load balancer (redundant instances, health checks, anycast at scale) and kept thin so it rarely changes.
- *"What goes in the gateway vs. the services?"* The no-domain-words test: "reject expired tokens," "100 requests per minute" → gateway; "only the author can edit a review," fraud rules → services.

Mistake that fails candidates: business logic in the gateway; ignoring logic creep; routing internal calls through it; claiming it can't fail without explaining redundancy.

Self-check

- **Q1: The gateway's five responsibilities, and the AuthN/AuthZ split in one sentence.** → Routing that keeps topology private; authentication with trusted identity passed inward; shared limits, validation and TLS; the observability root; a simpler client contract. The gateway works out *who* the caller is (AuthN); services decide *whether that identity may perform a domain action* (AuthZ).
- **Q2: Gateway or service? (a) token signature validation (b) "may this user edit this document?" (c) per-API-key quotas (d) computing an order total (e) rejecting requests over 10 MB (f) applying a loyalty discount.** → Gateway: (a), (c), (e) — no domain words. Services: (b), (d), (f) — product concepts stay with their domain.
- **Q3: Two years in, nearly every feature needs a gateway-team change; daily deploys everyone dreads; a trace shows checkout's call to inventory leaving the datacenter and coming back.** → (1) ESB creep: move business rules back to services and client shaping to BFFs, ban domain vocabulary in gateway review, make routine routing team-owned and self-service. (2) East-west traffic through the north-south gateway: route internal calls directly or via the mesh. Both treat the entrance as the system's center.

Related patterns

- **Reverse Proxy** — the primitive the gateway adds API policy to.
- **Rate Limiting** — a core gateway policy.
- **BFF** — takes client-specific shaping out of the gateway.
- **Service Mesh** — the east-west counterpart on the same primitive.
- **Load Balancing** — health checks and redundancy so one logical entrance isn't one machine.

TL;DR

Problem	Eleven services, eleven entrances: auth built eleven times (one without revocation), topology as public API, policy drift built into the structure.
Mechanism	One north-south entrance: routes by path, authenticates once and passes verified identity inward, enforces limits/validation/TLS, creates the request ID. Thin: policy yes, product no; AuthN at the entrance, AuthZ in services.
Costs	An intentional SPOF (redundant and thin), 1–2 ms per request, an organizational bottleneck unless config is self-serve, a perpetual temptation to add logic.
Skip it when	One service (a reverse proxy suffices), east-west traffic (the mesh), or the "gateway" is becoming an ESB.

Backend for Frontend

Original lesson: Backend for Frontend on DesignGurus ↗

What it is. A **Backend for Frontend (BFF)** is a thin backend for *one* client experience — web, mobile or TV — owned by that client's team. Behind the gateway, it calls the needed domain services in parallel and returns exactly what that client's screens show. Services stay the same; each client gets its own API shape.

> **Analogy.** A personal shopper who knows your size and taste visits six shops quickly and brings back one bag of only what you'll wear — without setting prices or store policy.

The problem: one API fits every client equally badly

A year after the gateway, the mobile team struggles:

- The **product screen needs data from 6 services — 6 cellular round trips of ~300 ms** each.
- Each response carries **40 fields**; the phone shows **6** — wasted bytes, battery and parsing.
- Every screen change waits on a backend team's ticket.

One client-agnostic API can't suit a rich web page, a constrained phone *and* a bare TV screen. The gateway can't help: client-specific shaping is **product behavior**, and the gateway's rule is "policy yes, product no." The BFF gives shaping its own home.

How it works — four rules

Mobile and web each make **1 call** through the gateway's policy layer. The Mobile BFF (mobile team) fans out **6 parallel calls, ~5 ms each** to Catalog, Reviews, Pricing, Inventory, Recs and Shipping; the Web BFF (web team) gathers what the web UI needs.

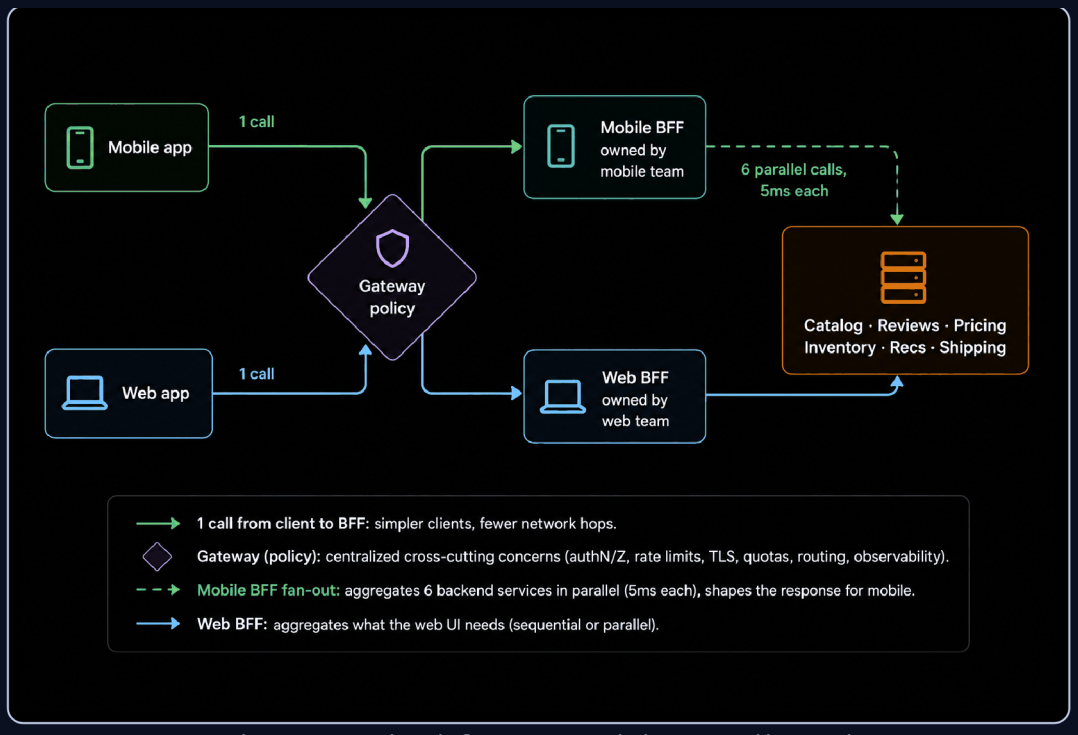


Figure — Backend for Frontend (course diagram)

1. **A BFF gathers and shapes — nothing else.** It **fans out** (parallel calls over fast datacenter links) and returns exactly one screen's fields. Pricing, permissions, totals and other domain rules stay in services; a BFF computing discounts is a chunk of monolith hiding in the presentation layer.
2. **The client team owns it** — changing it on the client's release schedule with no backend ticket. This ownership shift is the real point.
3. **One BFF per experience, not per screen.** Web, mobile and TV may each get one; **iOS and Android usually share one mobile BFF** until they truly diverge. A **partner API** sits in a similar spot but is different: a stable, documented, versioned public product.
4. **BFFs never call one another.** Client → gateway → BFF → domain services, one direction only; BFF-to-BFF calls create an accidental service layer of hidden dependencies.

Walkthrough with numbers (the mobile product screen)

- **Before:** 6 cellular round trips at ~300 ms (mostly sequential), returning a general-purpose ~**45 KB**.
- **After:** **1** cellular round trip, saving at least **1.8 s**; inside the datacenter, **6 parallel calls of ~5 ms** finish gathering in about **20 ms**; the response is **6 KB** of on-screen data.
- **Latency model:** move the *chatty* exchanges onto the fast network and cross the slow one once.

Trade-offs

You gain: per-client latency and payload (chatty gathering on fast links); team independence (no backend ticket queue); a permanently thin gateway.

You pay:

- **N more deployables** — each BFF needs a build pipeline, monitoring and an on-call owner.
- **Intentional duplication** — web and mobile BFFs assemble data differently because their screens differ; a shared "BFF framework" to erase it can bring back the coupling you escaped.
- **Leak risk** — BFFs sit near features, so domain rules drift in; two BFFs applying a rule differently make the product inconsistent.

When not to use it

- **One client experience** — a web-only product's API already serves its presentation; another backend adds latency and ops for nothing.

- **Small teams** — three engineers can't usefully own four backends; independence pays only with separate owning teams.
- **When flexibility belongs on the client** — see GraphQL below.

BFF vs. GraphQL

Both fix **over-fetching** (unused fields) and **under-fetching** (several calls per screen). **GraphQL** lets the *client* query exactly the shape it wants.

	BFF	GraphQL
Who shapes the response	The client's server team, per experience	The client's query, per request
Flexibility	Fixed shapes — changes need a BFF deploy	Whatever the schema (typed description of available data) permits, no deploy
Server-side complexity	N thin services	One graph, with resolver (per-field fetch function) efficiency to manage (the **N+1 query** problem — one query per item instead of one batch), query cost limits, caching complexity
HTTP caching	Easy (fixed endpoints)	Harder (queries usually sent as POST, which caches skip)

They can be mixed (a BFF per experience built *with* GraphQL). Rule of thumb: BFFs for a few stable experiences; GraphQL when many clients' data needs change faster than you can deploy — budgeting server-side defenses against expensive queries.

In the interview

Typical prompt: a slow mobile screen makes six calls with big payloads — fix it without touching web. Or: you've drawn a gateway; where does mobile-specific shaping go?

30-second answer: "Each client experience gets a backend-for-frontend: a thin aggregation layer behind the gateway, owned by that client's team. Mobile's six-call screen becomes one call; the BFF fans out to six services in parallel over datacenter links and returns exactly the screen's fields, crossing the slow network once instead of six times. Rules: aggregate and shape but no domain logic, no BFF-to-BFF calls, iOS and Android share one until they really diverge. GraphQL is the alternative when data needs change faster than BFF deploys — the same problem solved client-side, at a server-side cost."

Likely follow-ups (with answers):

- *"Can the BFF handle checkout? It already talks to every service."* It can gather reads and *start* the operation, but checkout is a write with money and inventory rules, so a **domain-owned saga** (multi-step workflow with an undo per step) coordinates it; the BFF may kick it off and format its status.
- *"One BFF or two for iOS and Android?"* One, usually the same mobile experience; split only when a real difference (a platform-only feature, a tablet redesign) justifies another deployable and owner.

Mistake that fails candidates: business logic in a BFF; BFFs calling each other; a backend per screen with no sustainable ownership.

Self-check

- **Q1: Gateway, BFF and services all sit between client and data. Each layer's one-line job and owner?** → Gateway (platform team): shared auth, limits, routing. BFF (each client team): gathering and shaping. Services (domain teams): authorization, business rules, writes, authoritative data. Policy at the entrance, presentation in the middle, domain truth in services.
- **Q2: Web, iOS, Android, TV and partner integrations — how many BFFs, and what's different about partners?** → Three BFFs (web; one shared mobile; TV) plus a **public API product** for partners, BFF-shaped but stable, documented and versioned because partners can't update in lockstep with you.
- **Q3: A customer sees different order totals on phone and laptop; the mobile BFF applies a loyalty discount locally to save time, the web BFF doesn't.** → A domain rule leaked into presentation. Move loyalty discounts into the pricing or orders service and have both BFFs show the authoritative total. *If two BFFs could disagree about a value, it was never presentation logic.*

Related patterns

- **API Gateway** — BFFs sit behind it and keep shaping out of it.
- **Request-Response** — parallelize over fast internal links; cross the slow client network once.
- **Graceful Degradation** — a BFF can render a partial screen when a dependency is down.

TL;DR

Problem	One uniform API fits every client equally badly: mobile pays six cellular round trips and 45 KB of unused data per screen, and every client change queues behind a backend team.
Mechanism	A thin backend per client experience, client-team-owned, behind the gateway, fanning out in parallel over fast links and returning one shaped response. No domain logic, no BFF-to-BFF calls, one per experience.
Costs	N deployables with real operational weight, intentional duplication, ongoing risk of business rules leaking into presentation.
Skip it when	One client experience, teams too small to own the layers, or GraphQL's flexibility fits better (server cost budgeted).

Rate Limiting

Original lesson: Rate Limiting on DesignGurus ↗

What it is. Count requests **per identity** at the entrance; when an identity exceeds its budget, refuse immediately with a machine-readable "slow down" — HTTP **429 Too Many Requests**, a `Retry-After` header and remaining-budget headers. Fairness becomes arithmetic. (Basics: Module 0 — Rate limiting and throttling.)

> **Analogy.** A turnstile with a personal pass that refills over time; when it's empty the gate shows "try again in 30 seconds" instead of letting you clog the line. (The course diagram calls the gateway a "turnstile".)

The problem: one partner's bug takes everyone down

The public API serves the mobile apps and several hundred partners. One partner ships a bug that retries in a loop **without backoff** (waiting longer between retries): **40 requests/s to 4,000** — 100x normal. Every request is authenticated and well-formed, so the gateway passes them all; shared services overload and one customer's bug becomes everyone's outage.

The logs also show a **scraper** pulling the catalog through search and a **credential-stuffing** bot trying leaked username/password pairs at login. One gap: **the entrance checks *who* you are, but nobody counts *how often*.** Bulkheads contain damage by resource inside; at the boundary callers have identities and tiers, so fairness is enforced there.

Why the obvious fixes fail

- **"Block the partner's IP."** They're a paying customer whose broken endpoint should be *slowed*, not cut off. Bots hop across thousands of addresses, and **NAT (network address translation)** puts many users behind one IP — blocking a university can lock out **5,000** students.
- **"Scale up and soak it up."** Servers for someone else's endless retry loop turn their bug into your bill, and an autoscaler happily automates the waste.
- **"Email the partner."** Their fix may take days; your outage continues. As the circuit-breaker lesson put it, when minutes count the human decision must become automatic.

How it works — four decisions

Every request hits the **gateway turnstile** first. With budget left it takes a token atomically (all-or-nothing, no request slipping in halfway) and forwards; with an empty bucket it refuses at the door with **429 + Retry-After in ~0.1 ms**. Gateway instances share counters in **Redis** via atomic check-and-decrement.

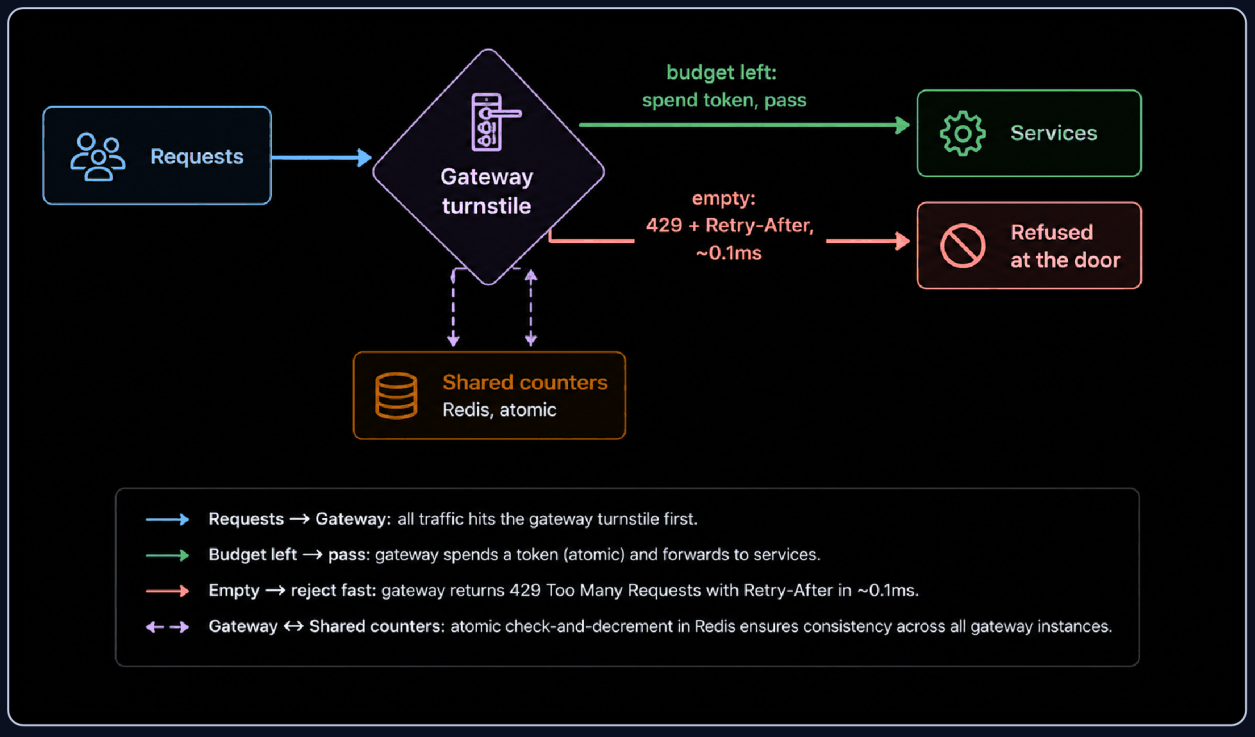


Figure — Rate Limiting (course diagram)

Decision 1 — a refusal is a contract. Return `429` with `Retry-After: 30` and budget headers like `X-RateLimit-Remaining`. Well-behaved SDKs (ready-made client libraries) — the polite client from the retry lesson — back off automatically, so a 4,000/s loop shrinks within seconds.

Decision 2 — the algorithm.

- **Fixed window** ("100 per minute; reset at :00"). Very simple, but has a **boundary burst**: 100 at 11:59:59 + 100 at 12:00:01 = **200 in 2 seconds**, all allowed. Rough protection only.
- **Sliding window** (count the last 60 s continuously). No boundary burst, more bookkeeping; the practical **sliding-window counter** approximates it by blending two adjacent fixed-window counts.
- **Token bucket — the default.** Each identity's bucket holds at most **b** tokens and refills at **r** per second; each request takes one; empty → 429. **r** is the sustainable rate, **b** the forgiven burst. Default because an ordinary page load fires ~**10** requests at once — honest bursts pass while the long-term average stays capped.

Decision 3 — the key: what are you counting per? A trusted **API key or account** as primary identity; IP only as a *generous* backstop for anonymous callers (NAT can make one IP thousands of people).

Refinements: **tiers** (e.g., free 10/min, paid 1,000/min) and **cost weighting** (an expensive search costs 10 tokens, a health check 1).

Decision 4 — where counters live. N gateway instances each enforcing 100/s give a true limit of N x 100. Options:

- **Shared Redis** with atomic `INCR` + `EXPIRE` (increment, auto-expire): about **0.5 ms** per request, exact.
- **Local buckets synced periodically:** faster, approximate. A **tenant** (one customer in a shared system) briefly getting 110 instead of 100 is fine for *protection*; *billing* needs exact counts.

Walkthrough with numbers (Tuesday's incident, replayed: r=100/s, b=200 for the partner tier)

1. **The buggy partner** sends 4,000/s: the first **~200** use the burst, then **100/s** pass and **3,900/s** get a 429 in about **0.1 ms** each, never reaching an app service. Versus **~50 ms** of real work, refusing is **~500x** cheaper.
2. **Nobody else notices.** Every partner and mobile group has its own bucket — the bulkhead idea applied by *identity* at the boundary.
3. **The partner's SDK** honors `Retry-After` and slows itself; the 100/s refill keeps its normal traffic working, and their engineers get an incident graph instead of a shared outage.
4. **Login has its own rule: 5 attempts per minute per account** plus an IP backstop; credential stuffing drops from thousands of attempts to a few dozen.
5. **Search costs 10 tokens**, so a scraper drains its budget **10x** faster.
6. **Operations watches patterns, not totals.** Accepted vs. rejected by identity, endpoint and tier separates a broken integration from a global capacity problem or a too-small burst setting. Alert on *sustained shifts* in a tenant's 429 rate, not single 429s — a working limiter refuses by design.

Trade-offs

You gain: structural fairness (no identity starves others); flipped abuse economics (~0.1 ms per attack request instead of full work); computable worst-case capacity (per-tenant ceilings plus the autoscaler max); monetization (tiers become the pricing page).

You pay:

- **Real bursts refused** if **b** is too small — "random" failures on normal page loads. Measure client bursts before choosing b.
- **A counter hop per request** (~0.5 ms for Redis) or approximate local counting — choose exactness vs. latency deliberately.
- **Shared-IP false positives** whenever IP is the main key.
- **Many knobs** — endpoint costs, tiers, refill rates and bursts need owners and monitoring.
- **Risky config changes** — a mistaken global cut denies *every* client as fast as the intended rule blocks one abuser; ship limit changes through reviewed, staged, quickly reversible releases.

When not to use it

- **As a substitute for capacity.** If ordinary valid traffic exceeds the limit, users pay for your under-provisioning; limits should catch *outliers*.
- **Internal east-west overload, mostly** — use **backpressure** (slowing the sender) and bulkheads; meshes *can* rate-limit internally, but with different identities and goals.
- **Small private APIs with three known callers** — talking to them may beat running limiter infrastructure.

Common misuse: a bare 429 with no `Retry-After`, budget headers or documentation — a well-meaning client can't adapt and may retry *harder*, turning protection into another burst.

The algorithms compared

	Fixed window	Sliding window (counter)	Token bucket
Boundary burst flaw	Yes — 2x at window edges	No	No
Allows natural bursts	Only by accident	Not really — smooth by design	Yes, via b (the feature)
Memory per key	One counter	Two counters	Token count + timestamp
Complexity	Trivial	Low	Low
Reach for it when	Rough protection, simplicity first	Smoothness matters (near billing)	By default — matches real traffic

Three ways to turn work away: rate limiting = ongoing fairness per *identity*, at the entrance; **bulkhead** = ongoing containment per *resource*, inside; **load shedding** = a global *emergency* drop of low-priority work regardless of sender. Large systems use all three.

Real-world examples

- **Stripe and GitHub** — published limits, documented tiers, full header sets (`X-RateLimit-*` , `Retry-After`) so SDKs behave predictably.
- **Cloudflare** — enforces limits at edge locations before abusive traffic heads to your origin.
- **Redis atomic counters** (`INCR` + `EXPIRE`) — the usual shared-state implementation; **Envoy and nginx** ship limiter modules.
- **For AI engineers:** LLM APIs limit **tokens per minute**, not requests, because one call can cost 100x another — cost-based limiting. An internal LLM gateway enforces per-tenant token budgets and model quotas, and is itself a polite client handling providers' 429s (Module 5 covered receiving `Retry-After` ; this lesson covers sending it).

In the interview

"Design a rate limiter" is often a standalone question (algorithms, keys, distributed counters); inside a broader design it keeps one client from consuming shared capacity. Expect the fixed-window boundary question.

30-second answer: "A token bucket per API key at the gateway — r the sustained budget, b the forgiven burst — with plan tiers and cost weights. Refusals are a contract: 429, Retry-After, remaining-budget headers, so good SDKs back off. Counters in Redis with atomic increments, exact enough for protection. IP limits only as a generous anonymous backstop, since NAT makes one IP thousands of people. Refusing costs 0.1 ms versus 50 to serve — the cheapest capacity I own."

Likely follow-ups (with answers):

- *"What's wrong with fixed windows?"* The boundary burst (200 in two seconds under "100/min"); sliding windows remove it, token buckets also model legitimate bursts.
- *"20 gateway instances — where does the count live?"* Shared Redis (exact, sub-ms hop) or synced local buckets (faster, approximate — fine for protection, not billing).

Mistake that fails candidates: banning customers instead of slowing them; IP as the main key for authenticated traffic; silent 429s without `Retry-After` ; hiding missing capacity behind tight limits.

Self-check

- **Q1: In a token bucket, what do r and b control, and which normal behavior depends on b?** → r is the sustainable average; b is how far a short burst may exceed it. Normal users burst (page loads fire many requests; mobile apps sync on wake); without b the limiter makes normal behavior look broken.
- **Q2: Keys and limits for (a) public catalog search, (b) login, (c) a partner's nightly bulk export, (d) free vs. paid tiers on one endpoint.** → (a) Per-API-key bucket with a high search token cost, so scraping drains quickly. (b) Strict per-account limit (e.g., 5/min) — credential stuffing targets accounts — plus a generous anonymous IP backstop. (c) A separate documented schedule or bulk endpoint so batch work doesn't fight interactive traffic. (d) r and b by plan, with the upgrade path in the 429 response.
- **Q3: At a university launch thousands of students get 429s within an hour; traffic is modest and services healthy.** → The campus shares one NAT IP and IP was the main key, so thousands counted as one identity — the limiter caused the outage. Key authenticated requests by account or API key; keep IP as a generous anonymous backstop. Rule: **key on the identity the policy is meant to limit** — an IP is a network location, not a person.

Related patterns

- **API Gateway** — where the policy is enforced.
- **Retry with Exponential Backoff** — the client side honoring `Retry-After`.
- **Bulkhead** — internal resource containment vs. identity limits at the entrance.
- **Auto-Scaling** — known tenant ceilings make maximum capacity computable.
- **Graceful Degradation** — under global pressure, load shedding can prioritize paid or critical work.

TL;DR

Problem	The entrance checks identity, not frequency: one partner's 4,000/s loop, a scraper or a bot, and everyone shares the outage. Blocking IPs bans campuses; scaling up buys servers for a malfunction.
Mechanism	Per-identity token buckets at the gateway (r = sustained rate, b = forgiven burst), weighted by endpoint cost, tiered by plan, counted in shared atomic storage. Refusal takes ~0.1 ms and is a contract: 429, Retry-After, budget headers.
Costs	Too-small bursts punish honest clients, a counter hop (or imprecision), NAT false positives when IP is the key, many knobs needing owners and 429 dashboards.
Skip it when	It would ration your own under-provisioning, the problem is internal east-west overload (backpressure/bulkheads), or three known consumers could just talk.

Cursor-Based Pagination

Original lesson: Cursor-Based Pagination on DesignGurus ↗

What it is. Instead of asking for "page 2", the client says **where it stopped**. A **cursor** is an opaque bookmark (clients can't see inside or depend on its format) — e.g., "everything after order 8412" — returned with each page. Tied to a *row*, not a *count*, it stays fast at any depth and doesn't skip or repeat items when the list changes.

> **Analogy.** A bookmark vs. "page 212": insert pages at the front and "page 212" shows different text; the bookmark stays where you stopped.

The problem: offset pagination fails two ways

Offset pagination requests a numbered page and makes the database skip rows: `?page=2` → `OFFSET 20 LIMIT 20`.

1. **Drift.** A user reads page 1 of the mobile order feed; three new orders arrive at the top, shifting every row down three, so page 2 starts with three already-seen items — **duplicates**. Deletions do the

reverse and **skip** unseen rows. Page numbers are positions in a moving list.

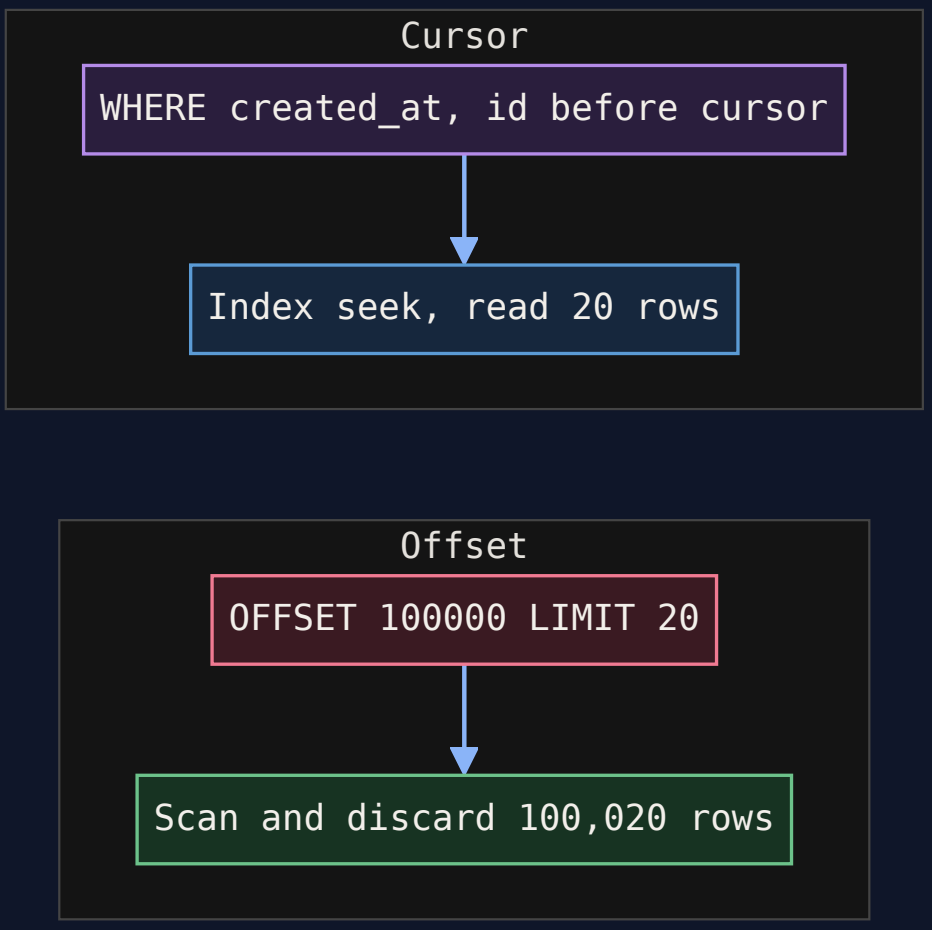
2. **Deep-page cost.** A partner exports with `?page=5000`: `OFFSET 100000` can't jump to row 100,000 — it scans and discards 100,000 rows. Work grows linearly: page 1 **2 ms**, page 5,000 **800 ms**.

How it works

```
Offset:  SELECT ... ORDER BY created_at DESC, id DESC
         OFFSET 100000 LIMIT 20                -- scan & discard 100,020 rows

Cursor:  SELECT ... WHERE (created_at, id) < (:cursor_ts, :cursor_id)
         ORDER BY created_at DESC, id DESC LIMIT 20
                                               -- index seek: reads exactly 20 rows
```

1. **Return a cursor with every page** — `next_cursor`, an opaque token, usually the last row's sort values encoded in **base64**. It records "where I stopped" as values, not a count.
2. **Seek from the cursor via an index** — `WHERE (created_at, id) < (cursor values)` lets the database jump straight there through an index, in the same time at any depth; nothing is scanned and discarded.
3. **Progress is tied to a row** — rows inserted before it can't move it, so there's no drift.

Offset scans and discards; cursor seeks



Three correctness rules:

- **A tiebreaker column is mandatory.** Sorting by `created_at` alone breaks when bulk imports or busy seconds create identical timestamps — "after T" repeats or skips other rows at T. Use `(created_at, id)` and store both in the cursor.
- **Opaque means opaque.** Encode it and document it only as "a token"; if clients parse or build cursors, your internal sort key becomes a permanent public contract (a versioning problem).
- **A stable, indexed sort.** The order must be deterministic and index-backed, or the database still scans despite cursor syntax.

Trade-offs

You gain:

- **Constant cost at any depth** — page 5,000 costs what page 1 does.
- **No drift** — a correctness requirement for infinite scroll.
- **Replica-friendly** — cheap index seeks suit **read replicas**.

You pay:

- **No random access** — next/previous, no "jump to page 47" (though you can jump *by value*, e.g., "start from last Tuesday").
- **Awkward totals** — "page 3 of 812" needs a separate, possibly expensive `COUNT(*)` ; big products use infinite scroll or approximate counts.
- **Cursor lifecycle** — changing the sort breaks existing tokens; every supported sort order needs its own indexed cursor design.

When not to use it

- **Small, shallow UIs where people jump between pages** — a **200-row** admin table with "page 4" works fine with offsets.
- **When the product truly needs numbered pages** over a small dataset — offsets with an explicit **depth cap** (even large search engines cap paging depth).

Common misuse: giving partners offset APIs, which they turn into deep exports that lock you into linear database work forever.

Offset vs. cursor

	Offset (?page=47)	Cursor (?after=eyJ0cy...)
Cost at depth	Linear — scan and throw away	Constant — index seek
List changes while scrolling	Duplicates and skips	Unaffected — tied to a row
Random page jumps	Yes — its single advantage	No — next/prev only (plus seek by value)
"Page 3 of 812"	Natural	Awkward (separate count)
Fits	Small, shallow admin UIs with page jumps	Feeds, APIs, exports, infinite scroll — the default

Real-world examples

The standard for feeds, public and partner APIs, exports, bank transaction histories and infinite scroll (the course uses these as use cases rather than naming products).

In the interview

Infinite-scroll questions test whether you spot drift; "a partner's export times out on page 5,000" tests deep-page cost. Name both failures first.

30-second answer: "Offsets break two ways at scale: OFFSET N makes the database discard N rows, so deep pages slow linearly, and page numbers are positions in a moving list, so inserts during scrolling cause duplicates and skips. Cursors fix both: the response carries an opaque token with the last row's sort values, and the next request seeks with WHERE (created_at, id) < cursor over an index — constant time, tied to a row. Details: a unique tiebreaker because

timestamps collide, opaque tokens so the sort key isn't a public contract, and no jump-to-page-47 — which feeds never needed."

Likely follow-ups (with answers):

- *"Why isn't `created_at` enough?"* Tied timestamps (see the tiebreaker rule); `(created_at, id)` makes every position unique.
- *"The client wants to start from last week."* Seek by value — build the starting cursor from the date; only count-based jumps go away.

Mistake that fails candidates: offsets for a large feed or partner API; timestamp-only cursors; publishing a cursor format clients start parsing.

Self-check

- **Q1: The two offset failures and the one change fixing both.** → **Drift** (inserts/deletes shift rows across pages → duplicates and skips) and **deep-page cost** (OFFSET N scans and discards N rows). A cursor tied to the last row's sort values allows an index seek to a position earlier inserts can't disturb.
- **Q2: Pagination for (a) a bank transaction-history API, (b) a 200-row internal config table, (c) a constantly updated social feed?** → (a) Cursor — a permanent, growing list where correctness matters. (b) Offset — small, with real page navigation. (c) Cursor — constant writes would cause visible drift.
- **Q3: A partner's nightly cursor export misses ~2% of orders, clustered in busy seconds; the cursor holds only `created_at`.** → Timestamp collisions: "strictly after this timestamp" never reaches the other rows sharing it (busy-second clustering is the tell). Query `WHERE (created_at, id) > (:ts, :id)` and put both values in the cursor. Because the token was opaque, its format changes without breaking partners; they rerun the export to recover the missing rows.

Related patterns

- **API Gateway** — the pagination contract documented and applied uniformly at the entrance.
- **API Versioning** — opaque tokens let the internal sort key change without a public format change.
- **Primary-Replica** — cheap indexed cursor reads suit replicas.

TL;DR

Problem	Offsets scan and discard rows to reach deep pages (linearly slower) and point at positions in a moving list (duplicates and skips).
Mechanism	An opaque token with the last row's sort values; the next page is an index seek, `WHERE (sort_key, tiebreaker) < cursor` — constant cost, tied to a row.
Costs	No jump-to-page-N (seek by value instead), awkward totals, a cursor lifecycle tied to a stable, indexed, unique sort key.
Skip it when	Small, shallow UIs with page jumps — offsets with a depth cap are fine.

API Versioning

Original lesson: [API Versioning on DesignGurus ↗]

What it is. Changing a published API without breaking its clients, by three rules: **make additive changes whenever you can; when you can't, create an explicit, coarse-grained version; retire old**

versions through a managed lifecycle.

> **Analogy.** A city renaming a street puts the new sign beside the old one, announces a date, mails letters, posts "old name ending soon" detours — and only then removes the old sign.

The problem: a rename that broke 200 partners

A team renames `order.state` to `order.status` because "state" confused people. Within an hour **200 partner integrations** can't parse responses, and an **18-month-old mobile build** — used by **one-third** of users who can't be forced to update — shows empty order screens.

Once published, an API can't be taken back. Clients you can't see, update or schedule build on every field you release and keep running your old decisions on their own schedules (the pub/sub lesson's public-schema rule; the BFF lesson's "partner APIs are products").

Why the obvious responses fail

- **"Never change anything."** Every early mistake — the confusing `state` — becomes permanent.
- **"Announce a migration deadline."** Assumes you control partners' engineering schedules and users' installed versions. You don't.
- **"Break it and say sorry."** Burns the platform trust integrations depend on; Twitter's v1→v2 move showed the ecosystem cost.

How it works — three rules

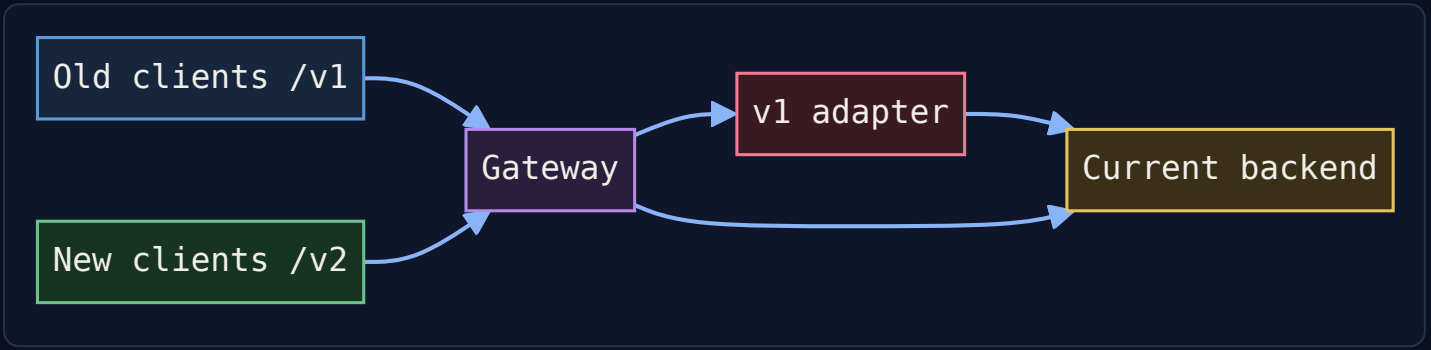
Rule 1 — additive first: most changes need no version.

- Add fields, endpoints and optional parameters with safe defaults; **never remove, rename or retype** existing ones.
- Clients are **tolerant readers**, ignoring unknown fields.
- Additive servers plus tolerant clients make most changes invisible (pub/sub's additive-only rule applied to REST).
- The honest cost is clutter: `status` beside `state`, both documented, one preferred, until a major version cleans up.

Rule 2 — for unavoidable breaks, version explicitly and coarsely.

- The rename ships in `/v2/`; `/v1/` keeps the old shape.
- Version the **whole API or resource**, never single endpoints — `/orders/v3` calling `/items/v5` creates an unmanageable test matrix.
- The gateway routes `/v1` through a thin **adapter** translating old requests and responses onto today's internals: **one backend, two interfaces, no forked implementation.**

One backend, two interfaces



Where the version goes:

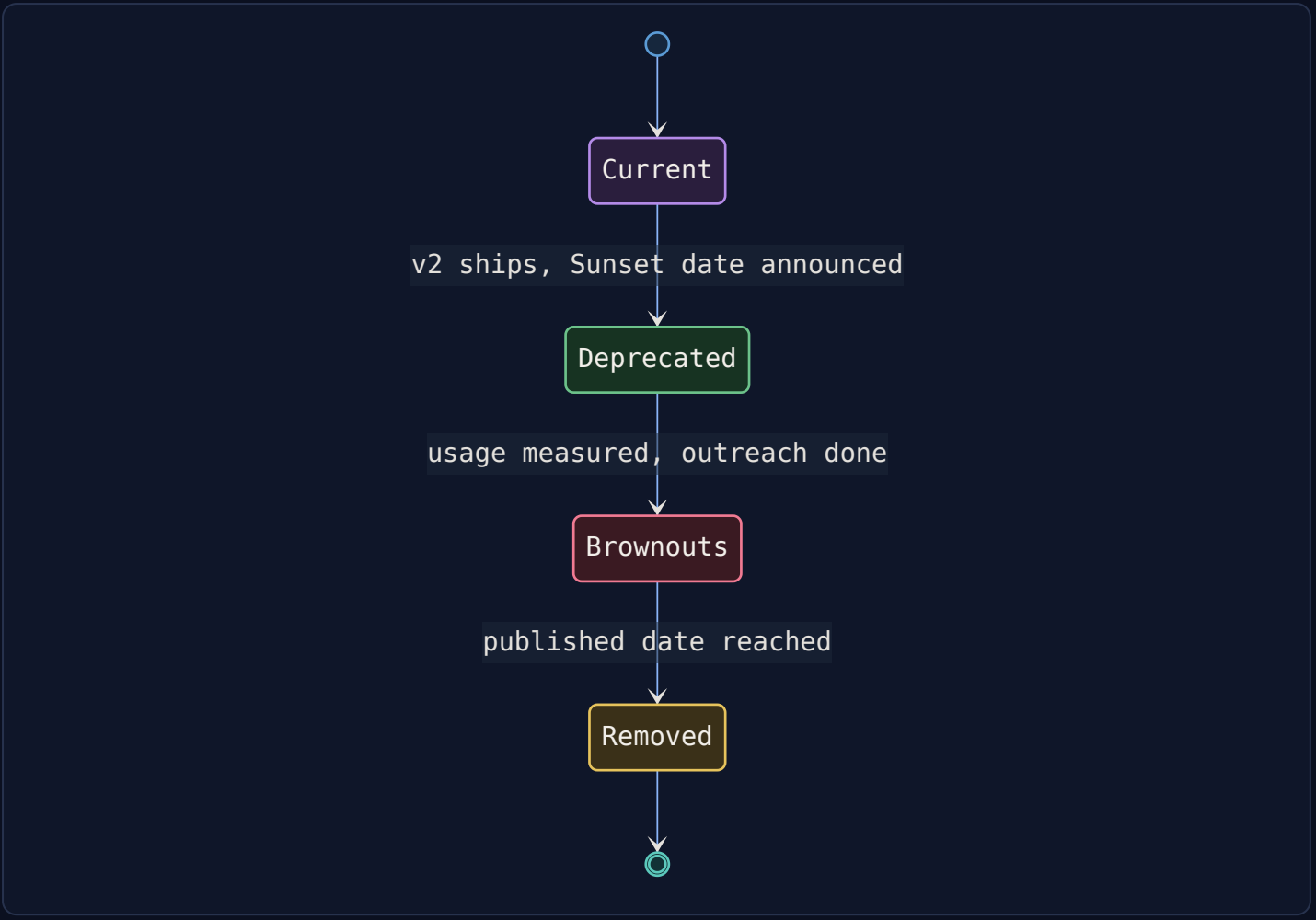
	URL path (/v2/orders)	Header (Accept: ...v2+json)
Visibility	In every log, link and curl command	Hidden — cleaner in theory, harder to debug
Caching/routing	Easy (different URLs)	Needs header-aware caches and routes
Adoption reality	The practical default	Favored by a minority of REST purists

Path is the practical default, but **consistency matters more than placement**.

Rule 3 — versions retire on a lifecycle, or they live forever. Each live version multiplies testing, docs, security and support. To retire one:

1. Announce a deadline with a **specific date**.
2. Send machine-readable `Deprecation` and `Sunset` headers (Sunset carries the removal date).
3. Track usage **per client and per version** on a dashboard.
4. Reach out directly to remaining users.
5. Schedule **brownouts** — announced, temporary outages of the old version (e.g., **15 minutes of HTTP 410 Gone**) that flush out integrations whose owners ignore email.
6. Remove it on the published date. Keep **no more than two live versions**: current and retiring.

Version lifecycle



Worked classification (additive vs. breaking)

- Add `order.estimated_delivery` → **additive** (tolerant readers ignore it).
- Rename `state` → `status` → **breaking**; safe path: add `status` beside `state`, remove `state` only in a major version.
- Make an optional request field required → **breaking** (previously valid requests fail).
- Add an endpoint → **additive** (old clients never call it).

- Change `amount` from integer cents to a decimal string → **breaking** (type and meaning change under the same name).

Trade-offs

You gain:

- Change without breaking the ecosystem — the rename ships, old apps work, trust accumulates.
- Fixable mistakes — design errors get a planned exit.
- Machine-readable retirement — Sunset headers and brownouts drive migrations without endless meetings.

You pay:

- **Maintaining N versions** — the two-version limit is a budget.
- **Adapter decay** — translation code quietly rots.
- **Additive clutter** — duplicate fields between major versions.
- **Sunsetting is organizational work** — dashboards, partner communication, brownout scheduling.

When not to use it

- **Internal APIs with coordinated deploys** — update caller and service together, with **contract tests** (automated checks that both sides honor the shared interface).
- **Pre-launch and beta surfaces** — label instability (e.g., `/beta/`); long-term promises start at **GA** (general availability).
- **Event schemas** — same law, different transport: schema registries, additive rules, versioned topics.

Common misuse: creating `/v3` because editing `/v2` feels risky while `/v1` is never retired — three permanent, half-maintained versions slowing every change.

Real-world examples

- **Twitter's v1→v2** migration — the cautionary tale of breaking changes eroding ecosystem trust.
- Sunset/Deprecation headers, 410 Gone brownouts and gateway adapters are the industry-standard retirement toolkit the course describes.

In the interview

Direct: "How do you change an API without breaking a mobile app you can't update?" Tougher: "How do you remove v1 two years after v2?" — testing whether your design includes retirement.

30-second answer: "Additive first: new fields and endpoints, never renames or removals, tolerant-reader clients — most changes need no version. For a truly breaking change, a coarse /v2 at the gateway, routed through a thin adapter onto current internals — never a forked backend or per-endpoint versions. And versions retire on a lifecycle or live forever: Sunset and Deprecation headers, a per-client usage dashboard, outreach, scheduled brownouts — fifteen announced minutes of failures that wake integrations that don't read email — then removal. At most two live versions, because each extra one multiplies tests, docs and support."

Likely follow-ups (with answers):

- *"URL or header?"* Path, practically (see the table); consistency matters more.
- *"How do you know v1 can really go?"* Per-client, per-version traffic data; headers, outreach and brownouts to move or surface the rest; call the last few; remove on data, not hope.

Mistake that fails candidates: "we'll be careful" instead of a compatibility mechanism; per-endpoint versions; versions with no dated retirement.

Self-check

- **Q1: Additive or breaking? (a) add `order.estimated_delivery` (b) rename `state` → `status` (c) require a previously optional request field (d) add an endpoint (e) change `amount` from integer cents to a decimal string.** → (a) additive; (b) breaking — add `status` first, remove `state` in a major version; (c) breaking; (d) additive; (e) breaking.
- **Q2: Versioning policy for an internal service API, a public partner API and the mobile BFF — and why different?** → Internal: coordinated deploys + contract tests, no public versioning. Partner: coarse versions, Sunset headers, measured usage, brownouts — partners run their own schedules. Mobile BFF: quick additive changes by the client team, major versions tied to a **minimum-supported app build**, because installed builds are uncontrollable consumers. Less deploy control → stronger compatibility process.
- **Q3: `/v1` was deprecated three years ago, still takes 30% of traffic, and nobody knows who uses it. Retirement steps in order, and the original mistake?** → Instrument per-client usage (unknown → named list) → `Deprecation` and `Sunset` headers with a real date → contact consumers, largest first → announced, repeating, escalating 15-minute 410 brownouts to surface dormant integrations → call the final few → remove on the published date. The mistake: "deprecated" with no measurement or sunset date, which made it permanent.

Related patterns

- **API Gateway** — home of version routes and adapters.
- **BFF** — a partner-facing BFF is a public API product under these rules.
- **Publish-Subscribe** — event schemas follow the same permanent-contract rule.
- **Cursor Pagination** — opaque cursors expose less structure, needing fewer versions.

TL;DR

Problem	Published APIs serve clients you can't see, update or schedule; one well-meant rename breaks 200 partners and every old mobile build at once.
Mechanism	Additive-first (add, never rename/remove/retype; tolerant readers); coarse explicit versions via gateway adapters for unavoidable breaks; lifecycle retirement (Sunset headers, usage dashboards, outreach, brownouts, removal) — at most two live versions.
Costs	N versions to maintain, rotting adapters, additive clutter, organizational sunset work.
Skip it when	You control both sides (coordinated deploys + contract tests), the surface is labeled beta, or it's an event schema (pub/sub's playbook).

Sidecar

Original lesson: Sidecar on DesignGurus ↗

What it is. A small helper process running **next to every service instance** on the same machine. The service talks to it in plain text over **localhost**; the sidecar does the outward networking work — TLS, retries, timeouts, metrics, tracing — so the service does only its own job. (Basics: Module 0 — Service discovery, sidecars and service mesh.)

Analogy. A personal assistant beside each executive: the executive says "call inventory"; the assistant dials, does security checks, retries busy lines and logs the call — identically for every executive, whatever language they speak.

The problem: east-west traffic has hundreds of entrances

The module turns to **east-west traffic**, which needs the same consistent handling as public requests:

- **Mutual TLS (mTLS)** — encryption where *both* sides prove identity (normal TLS proves only the server's).
- Timeouts, retries, standard metrics and **trace-header propagation** (passing the trace ID along so tracing links calls).

Doing this with libraries across **11 services in 3 languages** yields 11 retry behaviors — the gateway lesson's policy drift, east-west. A gateway can't help: it centralizes public policy because external traffic shares one entrance, while internal calls rightly go directly between hundreds of instances.

Why the obvious fixes fail

- **"Use a shared library."** One version per language, upgrades waiting on 11 deploy schedules — bound to drift.
- **"Use one central internal proxy."** One policy place, but every internal call gets an extra hop and a SPOF — and the gateway's own rules forbid east-west detours.

The sidecar inverts placement: **if you can't bring all traffic to one proxy, bring a proxy to every instance.**

How it works

A Go Checkout instance and a Python Inventory instance each talk plain text over localhost to their own sidecar ("same binary"). The only off-machine hop is sidecar-to-sidecar mTLS, where timeouts, retries, circuit breaking, metrics and traces happen. Fleet math in the figure: 11 services x 40 instances = 440 sidecars, ~50 MB each, ~22 GB of memory for networking.

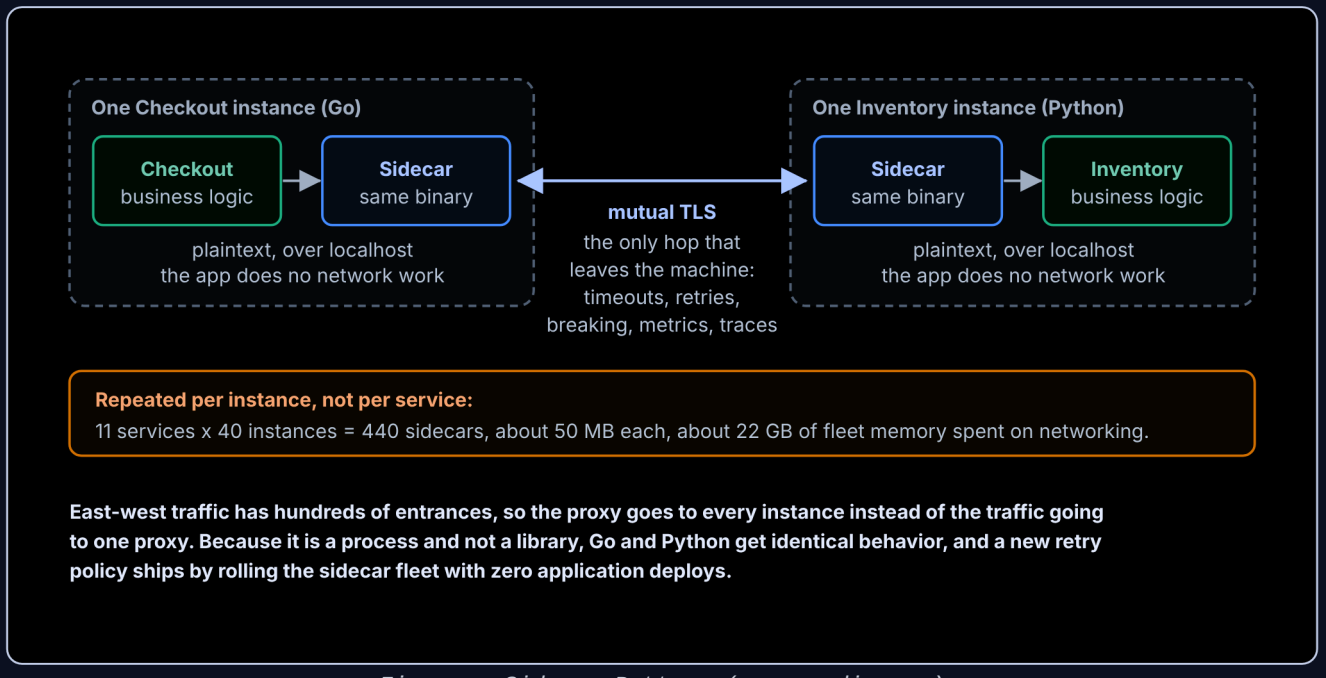


Figure — Sidecar Pattern (course diagram)

1. **One helper process per service instance.** The app sends plain requests to localhost; the sidecar adds TLS, timeouts, retries and telemetry on the network side. Networking logic lives outside the app.
2. **Language-agnostic by design.** A separate process, not a linked library, so Python, Go and Java run the identical binary — no parallel implementations.
3. **Policy upgrades at infrastructure speed.** A new retry policy, timeout or certificate reaches every service by rolling the sidecar fleet — **zero application deploys**. Networking gets its own lifecycle.
4. **Smaller application code.** Timeout, retry and **circuit breaker** mechanics (reliability module) move into infrastructure; services keep only *domain* decisions, like which reduced response to return when a dependency fails.

5. **The wider family.** "Sidecar" means any co-located helper — log collectors, secret fetchers (loading passwords and keys from a secure store), config reloaders — but *proxy* sidecars became the most influential, standardizing service networking. A proxy sidecar is the reverse-proxy primitive, once per instance.
6. **A building block, not the system.** Hundreds of proxies still need certificates, routes and policies from somewhere; the service mesh (next lesson) supplies that. The sidecar does the work; the mesh decides and distributes its configuration.

Roll it out gradually. Start with a non-critical service; measure memory and latency and verify startup/shutdown ordering; expand group by group, separating proxy bugs from app bugs before the fleet depends on it.

Walkthrough with numbers

- **Memory:** **~50 MB** plus a little CPU per sidecar looks cheap until multiplied: **11 services x 40 instances = 440 sidecars ≈ 22 GB**. Always multiply per-instance costs by fleet size.
- **Latency:** **~0.5 ms** per sidecar, two per internal call (sender outbound, receiver inbound) ≈ **1 ms** per call; measured p99 may rise **~2 ms** with TLS. Fine for most services, not microsecond budgets.
- **Lifecycle bug:** if the app is ready before its sidecar, its first outgoing calls fail after every deploy. Fix: a **readiness gate** / "hold application until proxy started". On shutdown, stop the app first and keep the sidecar to finish in-flight requests.
- **Crash blast radius:** a crashed sidecar takes only its instance off the network; it fails health checks, leaves load-balancer rotation and restarts — versus a central proxy taking everything down.

Trade-offs

You gain: identical networking in every language, upgraded fleet-wide without app deploys; mTLS everywhere in one rollout with no code changes; slimmer services.

You pay:

- **Per-instance cost times everything** (the 22 GB).
- **Per-hop latency** (~1 ms per call).
- **A new lifecycle** — startup order, independent sidecar crashes, **version skew** (mixed sidecar versions mid-upgrade).
- **One more debugging suspect** — every network incident must separate app from proxy behavior.

When not to use it

- **Small single-language fleets** — five Go services and one team can keep one good library and skip 50 MB per instance.
- **Microsecond latency budgets** (e.g., trading) — two extra process hops may be too much.
- **Serverless** — no long-running instance to sit beside; the platform must provide equivalent controls.
- **No platform team** — hundreds of injected processes, upgrades and failure modes need an owner. Newer **sidecar-less** options do the same jobs without a process per pod (Kubernetes' smallest deployable unit), using **eBPF** (programs running inside the OS kernel) or **ambient**, node-level (one per machine) proxies. The jobs remain even when the separate process goes.

Common misuse: sidecars for ten services because big companies use them — one team, two languages, little drift: all the cost, none of the organizational-scale benefit. Revisit as teams and languages grow.

Sidecar vs. shared library

	Shared library	Sidecar
Multi-language coverage	One implementation per language, drifting	One binary, identical everywhere
Upgrade path	Waits on N teams' deploys	Fleet-wide in minutes, no app deploys
Latency	Zero — in-process	~1 ms per call (two hops)
Memory	Shared with the app	~50 MB x every instance
Fits	One language, few services, tight budgets	Many languages, many services, drift problems

The deciding factor is organizational: libraries fail when many languages and teams must keep behavior matched on different schedules. Below that point a library is lighter, faster and good enough.

Real-world examples

The course names family members rather than products: proxy sidecars are the Envoy-style proxies meshes use (next lesson); log collectors, secret fetchers and config reloaders are non-proxy sidecars; eBPF and ambient node-level modes are sidecar-less alternatives.

In the interview

Usually the first half of a mesh question, or "how do you roll out mTLS across 11 services in 3 languages without touching every codebase?"

30-second answer: "A proxy process beside every instance: the app speaks plain text over localhost; the sidecar does mTLS, timeouts, retries, metrics and trace propagation. As a process, it behaves identically in every language and upgrades fleet-wide with no app deploys. Costs I size first: ~50 MB and ~0.5 ms per instance, two hops per call, times the fleet, plus startup ordering and crashes. Coordinating hundreds of them is the mesh's control plane."

Likely follow-ups (with answers):

- *"Why not a shared library?"* See the comparison table: drift across languages and team schedules vs. fleet-wide upgrades in minutes; below that tipping point the library wins.
- *"What if the sidecar crashes?"* Only that instance fails health checks and leaves rotation until restart.

Mistake that fails candidates: equating one sidecar with a full mesh; listing benefits without multiplying costs across the fleet.

Self-check

- **Q1: What moves from app code into a sidecar, and which two abilities does a separate process have that a library lacks?** → mTLS certificates and handshakes, timeouts, retries with backoff, circuit-breaking mechanics, metrics, trace propagation. A process is **language-agnostic** and **upgraded independently** of app deploys.
- **Q2: Fleet A: 5 Go services, one team. Fleet B: 30 services, 4 languages, 9 teams. Library or sidecar?** → A: library — no implementation or deploy drift, so sidecars add memory, latency and lifecycle complexity for nothing. B: sidecars — four implementations across nine release schedules would drift. The factor is how many independent language x team combinations must stay consistent.
- **Q3: After adopting sidecars, internal p99 rises ~2 ms and some pods fail their first requests after every deploy. Which is a cost and which a defect?** → The p99 rise is the **documented cost** of two

sidecars plus TLS — budget it. The first-request failures are a **defect**: a startup race; gate app readiness on the sidecar (readiness gates / hold-until-proxy-started), and on shutdown stop the app first and let the sidecar drain.

Related patterns

- **Reverse Proxy** — the primitive at its smallest useful size, one per instance.
- **Service Mesh** — the control plane configuring hundreds of sidecars.
- **API Gateway** — the north-south counterpart: one shared entrance vs. many local proxies.
- **Timeout / Retry** — reliability mechanics moved out of each codebase.

TL;DR

Problem	East-west traffic needs mTLS, retries, timeouts and tracing but has hundreds of entrances and no choke point, so per-language libraries drift at each team's pace.
Mechanism	A proxy process beside every instance, reached over localhost; apps speak plain text locally, sidecars network identically in every language, upgraded fleet-wide with no app deploys.
Costs	~50 MB and ~1 ms per call times the fleet, startup/shutdown ordering, crashes of their own, one more debugging suspect.
Skip it when	Small single-language fleets (library wins), microsecond budgets, serverless, or no owner for hundreds of new processes.

Service Mesh

Original lesson: Service Mesh on DesignGurus ↗

What it is. A central brain for a fleet of sidecars, in two **planes**. The **data plane** is the sidecars, carrying and policing traffic on every hop. The **control plane** is a central coordinator that tracks what's running, computes each sidecar's configuration (routes, certificates, policies) and keeps pushing updates over long-lived streams.

Analogy. Air traffic control: planes (sidecars) fly and follow the rules; the tower carries no passengers but knows where every plane is, issues clearances and changes instructions live. If the tower's radio goes quiet, planes keep flying on their last clearance — they just can't get new ones.

The problem: 440 sidecars, no shared brain

The rollout worked — **440 proxies**, consistent networking. Six months later, coordination hurts:

- **Certificates.** Every sidecar's mTLS certificate expires. Rotating **440 certificates every 90 days** by hand never ends; miss one quarter and at **2 AM** checkout can't call inventory — `certificate expired`. An outage from routine upkeep.
- **Policy drift, relocated.** An engineer tweaks retries during an incident and never reverts; another team keeps an old timeout. Eleven drifting libraries became **440 drifting config files**.
- **Routing keeps changing.** Deploys and autoscaling change instance IPs every few seconds; static files can't keep up.
- **Security's question has no answer.** "Only checkout and refunds may call payments" can't be expressed; network reachability *is* the permission.

Sidecars standardized *how* traffic moves, not the *knowledge* they need: which workloads exist, who may talk to whom, routes, policies, certificates.

Why the obvious fixes fail

- **"Template the files in CI."** Templates handle static settings, but pipelines take minutes while IPs change in seconds. Routing is live state needing continuous delivery.

- **"Use DNS for discovery."** Answers are cached for their **TTL**, so callers hit removed instances for minutes (the load-balancer lesson's caching problem, east-west). DNS gives location only — no health, identity certificates or authorization policy.
- **"Write a cert-rotation runbook."** 440 certificates every 90 days, perfectly, forever; one slip is an overnight outage. Routine decisions need machine-speed automation.

How it works

The control plane (top) watches the registry, issues and rotates certificates and holds policy, pushing config and certificates over long-lived **xDS** streams to the sidecars beside Checkout, Inventory and Payments (x 440), which talk mTLS to each other. If the control plane goes silent, sidecars keep serving with their last config — changes freeze, traffic doesn't.

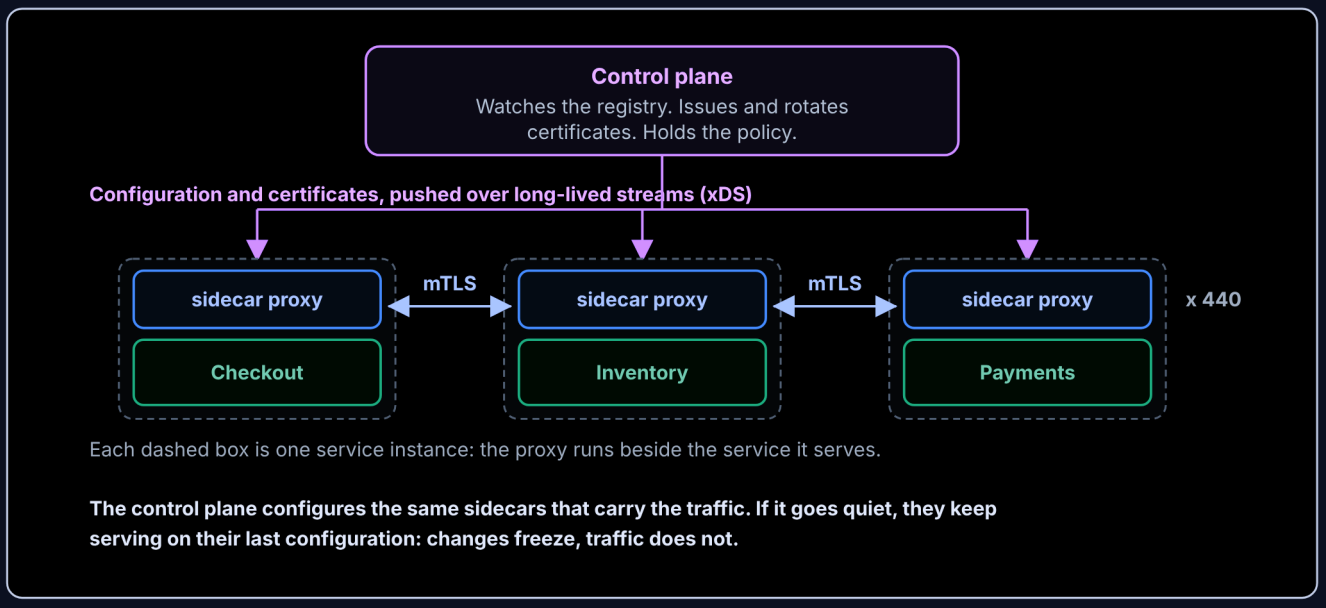


Figure — Service mesh: data plane and control plane (course diagram)

1. **Certificates take care of themselves.** The control plane is a **certificate authority (CA)** issuing each workload a cryptographic **workload identity** (e.g., "checkout, in prod") via **short-lived, automatically rotated certificates** (e.g., hourly). Expired-cert outages become very unlikely, and services trust *identity*, not *network location* — the foundation of **zero-trust networking**, where being inside the network earns no automatic trust.
2. **Routing at platform speed.** The control plane watches the orchestrator's live registry (e.g., the **Kubernetes API server**) and **streams** new routes to affected sidecars within seconds using **Envoy's xDS** — long-lived, typed configuration streams (resolving the forward reference from the bidirectional-streaming lesson).
3. **Declarative, central policy. Declarative** means describing the desired result, not the steps. Retry budgets, timeouts and circuit breaking (often called **outlier detection**) become versioned config objects, global or per service, continuously fed to sidecars.
4. **Authorization has a home.** One policy object — "only checkout and refunds may call payments" — is enforced by the *receiving* payments sidecar against the caller's cryptographic identity, logging every allow and deny. Service-to-service access control becomes explicit and auditable.
5. **Observability for free.** Every proxy sees every request and reports uniform rates, errors, latency and trace spans per service pair with no app instrumentation; the **dependency graph** becomes visible.
6. **Traffic control as configuration.** A small config block sends **5%** of inventory calls to inventory-v2 (a **canary**), **mirrors** real traffic to a test service, or **injects faults** in planned drills — no code changes.
7. **The data plane fails static.** If the control plane is down, sidecars keep their last config: existing traffic flows, but routes, certificates and policies stop updating. The control plane is still not optional — new instances (deploys, autoscaling) may get no routes or identity, and existing certificates keep counting toward expiry. Stable short-term, losing the ability to change; set control-plane recovery targets inside that window.

Walkthrough with numbers (the four incident problems, under a mesh)

- **Certs:** identities rotate **hourly** across all 440 proxies automatically — a risky chore becomes background automation, with zero-trust identity on top.
- **Canary:** one policy sends **5%** to inventory-v2, reversible **within seconds**; previously every caller needed a code change or special balancer (the next module depends on this).
- **Security request:** one authorization policy at the receiving sidecar, logging allowed and denied calls by workload identity.
- **Dependency map:** instant traffic, error and **p99** dashboards for every service pair, often revealing undocumented call paths.
- **The bill:** the sidecar lesson's **~22 GB** and **~1 ms** per call, plus a distributed control plane that must be **HA** (highly available). A platform team runs it as an internal product — safe defaults, migration help, documentation, an exception process — so app teams understand the policies affecting them without knowing every proxy setting.

Trade-offs

You gain: zero-trust mTLS with automatically rotated workload identity (often justifying the project alone); fleet-wide observability without instrumentation; traffic control by settings (canaries, mirroring, fault injection); central policy that can't drift.

You pay:

- **A distributed system to manage your distributed system** — HA, careful upgrades, specialist owners; a platform-team product, not a config switch.
- **All inherited sidecar costs** — memory, CPU, latency, lifecycle complexity.
- **More debugging layers** — app code, sidecar *and* central policy are suspects.
- **A global blast radius** — one bad global policy reaches the whole fleet as fast as config spreads.
- **Config sprawl** — authorization, routing and reliability objects multiply until policy governance is a discipline of its own.

Failure example — the global retry rule. A mesh-wide "retry every failed request 3x" is added for reliability; finance then reports **duplicate orders**, because failed order `POST`s (not **idempotent**) ran several times. The mesh didn't create the unsafe write but spread a bad policy everywhere instantly. Guardrails: policy as code (review, staged rollout, canary on one service first); retries skip non-idempotent methods by default; retrying writes requires audited **idempotency keys**.

When not to use it

- **Fewer than a few dozen services** — 8 services in one language: a library plus a gateway is better.
- **No platform team** — an unowned mesh is hundreds of identical proxies nobody can explain, arguably worse than local drift.
- **Latency-critical paths** — central config doesn't remove per-hop delay.
- **As the public front door** — meshes govern east-west; external traffic still uses an API gateway.

Typical misuse: a mesh because big companies blog about it, when you have two services and one slow query. **Adoption bar (all four):** a multi-language fleet, several dozen services, an explicit zero-trust requirement, and a staffed platform team. Start with **one measurable goal** (usually workload identity or consistent observability) and judge by certificate reliability, policy rollout time, proxy resource use and incident response — is coordination work shrinking, or just moving?

The adoption steps

Step	What it is	What it buys	Right when
1. Shared libraries	A reliability kit inside the code	No infrastructure; in-process speed	One language, few services, one team
2. Ad-hoc sidecars	Proxies beside instances, configured by hand	Same mechanics everywhere, across languages	Several languages, drift starting, config still manageable by hand
3. Full mesh	Sidecars + control plane	Identity, policy, observability and traffic settings — shared knowledge made uniform	Dozens of services, a zero-trust requirement, a staffed platform team

Move up when coordinating the current step becomes the bottleneck: libraries break across languages and team schedules; hand-configured sidecars break when certificates and policies multiply. Sidecar-less meshes (eBPF, ambient node-level) keep the control-plane duties while changing the underlying build.

Real-world examples

- **Istio** — Envoy proxies plus a full-featured control plane, with sizable operating cost. **Linkerd** — deliberately fewer settings, simpler to run.
- **Envoy's xDS** — the streamed-config protocol underneath; the same proxy serves as gateway, sidecar or load balancer.
- **Istio ambient and Cilium (eBPF)** — sidecar-less, keeping the control plane while cutting per-pod proxy cost.
- **For AI engineers:** model canaries (5% of inference to model-v2 while measuring quality), **mirroring** real questions to a candidate model without user impact, and workload identity limiting who can call expensive GPU services.

In the interview

Direct prompt: "fifty services in four languages need consistent mTLS, retries and canary routing." Follow-ups: control-plane failure, difference from the gateway, whether your scale justifies a mesh.

30-second answer: "Data plane: the sidecars moving traffic. Control plane: watches the platform, issues hourly-rotated workload-identity certificates, and pushes routing and policy over Envoy's xDS. You get zero-trust mTLS without app changes, fleet-wide telemetry, and canaries and mirroring as settings. The data plane fails static — a control-plane outage freezes changes, not traffic. Bar: dozens of services, several languages, a zero-trust requirement and a platform team; below that, gateway plus libraries."

Likely follow-ups (with answers):

- *"The control plane goes down at peak — what breaks?"* Fail-static (step 7): current traffic stays up; new workloads get no routes or certificates, so a long outage blocks deploys and autoscaling.
- *"Why not run all of this through the API gateway?"* The gateway is one north-south entrance applying *user* identity and product quotas; a mesh distributes east-west *workload* identity, retries and authorization across hundreds of service relationships. Routing internal calls through the gateway re-centralizes traffic sidecars keep local.

Mistake that fails candidates: a mesh for five services; misunderstanding fail-static; treating the public gateway and internal mesh as interchangeable.

Self-check

- **Q1: The two planes, their jobs, and fail-static precisely.** → **Data plane**: sidecars carrying traffic and enforcing mTLS, timeouts, retries and policy per hop. **Control plane**: watches the workload registry, issues identities, streams computed config. **Fail static**: with the control plane unreachable, sidecars keep their last config — traffic continues, configuration changes stop.
- **Q2: Fleet A: 8 services, one language, no compliance need. Fleet B: 60 services, 5 languages, zero-trust requirement, 6-person platform team. Where on the ladder, and what would move A up?** → A: step 1 (libraries); it moves up if more languages, several dozen services or a security mandate make coordination the bottleneck. B: step 3 — multi-language drift, scale, zero-trust need and an owning team.
- **Q3: A global mesh rule retries every failed request 3x; finance reports duplicate orders. Reconstruct, assign responsibility, design guardrails.** → Non-idempotent order `POST`s were retried, so one purchase ran several times. The unsafe write pre-existed; the mesh *extended the reach* of a wrong policy. Process: review, staged rollout, canary on one service first. Technical: retries skip non-idempotent methods by default; write retries require audited idempotency keys. Central policy fixes drift fast — and spreads mistakes just as fast.

Related patterns

- **Sidecar** — the data plane this pattern configures.
- **API Gateway** — the north-south counterpart.
- **Circuit Breaker** — outlier detection applies breaker behavior via central policy.
- **Distributed Tracing** — proxy-produced spans are its raw material.
- **Bidirectional Streaming** — xDS delivers config over long-lived machine-to-machine streams.

TL;DR

Problem	Hundreds of sidecars gave uniform mechanics but not uniform knowledge: certificates expiring on human schedules, per-pod config drift, routing lagging autoscaling, and nowhere to write "who may call payments".
Mechanism	Control plane + data plane: the coordinator watches the registry, issues hourly-rotated workload identities, holds declarative policy and streams each sidecar's config over xDS; sidecars enforce it per hop and fail static.
Costs	A distributed system to manage your distributed system (platform-team owned), inherited sidecar costs, harder debugging, a global blast radius for bad policy (govern it like code).
Skip it when	Fewer than a few dozen services, one language, no zero-trust mandate, or no owning team — all four are needed to clear the bar.

Authorization Service

Added lesson (not part of the DesignGurus course). Written from primary sources listed under "Sources" at the end of the lesson.

What it is. **Authentication** (AuthN) answers *who is calling?*; **authorization** (AuthZ) answers *may this caller do this action on this particular thing?* An authorization service moves the second question out of every service's private code into one shared system. Services still *ask* at the moment of access ("may user 42 view invoice 8812?"), but the rules and relationship data live in one place, are evaluated identically everywhere, and are logged. The best-known design is Google's **Zanzibar**, which stores permissions as a graph of relationships. (Basics: Module 0 — Authentication and authorization.)

Analogy. Hotel key cards: the desk checks your passport once (authentication); every door asks the central system "may card 7731 open room 512 now?" (authorization). Cancel a card and every door knows.

The problem: thirty services, thirty permission systems

The store's new **seller portal** has 200,000 seller organisations, 2 million staff accounts, teams inside each seller, and 500 million orders and invoices. Each of ~30 services wrote its own checks, usually a SQL join in the request handler. One quarter's findings:

- **The data leak.** `GET /invoices/8812` checked only that the caller was *logged in*; a seller edited the URL and read a competitor's invoices — **broken object-level authorization** (BOLA), number one on the OWASP API Security Top 10.
- **The half-removed employee.** Removed from the "finance" team, he lost payout reports, but search (caching membership for a day) kept showing him invoice titles.
- **The slow "what can I see?"** Nested folders and teams mean recursive queries per request: **2 seconds** for large sellers.
- **The unanswerable audit.** "Who can read our payout data, and why?" — spread across thirty codebases.

The gateway authenticated (the capstone's Stage 2) and deliberately knew nothing of invoices or teams, so object decisions fell to thirty drifting implementations.

Why the obvious fixes fail

- **"Put roles in the JWT."** A JWT (JSON Web Token, a signed token carrying claims about the user) can carry coarse roles ("seller-admin") but not "can view these 40,000 invoices", and a revoked permission lives until the token expires (see the Spring book's token revocation section).
- **"Let the API gateway decide."** It would need to know what an invoice is, who owns it and how folders nest — exactly what a thin gateway must not know. It remains the place for coarse checks: authenticated, token scope (which APIs a token may call), rate limits.
- **"A shared permissions library" / "copy the roles table everywhere."** Consistent *logic*, but each service reads its own copy of the *data*, going stale at thirty speeds — the half-removed employee recurs, and a rule change means thirty redeploys.

How it works

1. Split enforcement from decision. In NIST and XACML (a standard access-control policy language) vocabulary:

- The **policy enforcement point (PEP)** is code at the moment of access — handler, gateway filter, sidecar — that builds the question, asks, and obeys.
- The **policy decision point (PDP)** is the authorization service, holding rules and facts and answering `allow` or `deny` (plus a reason for the logs).

Services keep enforcement (only they know `/invoices/8812` means invoice 8812); the decision moves to one place.

2. Pick a permission model — usually more than one.

Model	Question it answers	Example rule	Breaks down when
ACL (access-control list)	Who is on this object's list?	invoice 8812: {alice: read}	Millions of objects each need their own list; nobody can review them
RBAC (role-based)	Does the user have a role that grants this permission?	"finance-viewer" may read invoices	Roles must be *per organisation, per team, per project* → **role explosion** (thousands of roles)
ABAC (attribute-based)	Do attributes of user, resource and context satisfy a policy?	allow if `user.org == invoice.org` and `user.mfa == true` and time is 08:00–20:00	Answering "who can see X?" means evaluating the policy against every user
ReBAC (relationship-based)	Is there a path of relationships from the user to the object?	user → member of team → viewer of folder → parent of invoice	Pure attribute rules (time of day, IP range) don't fit a graph

Real systems combine them: **RBAC** for coarse staff roles ("support agent"), **ReBAC** for ownership and sharing (organisations, teams, folders), **ABAC conditions** on top (require MFA — multi-factor authentication — for payouts; block unusual countries). The Spring book shows the in-process RBAC end: authorizing requests with roles and authorities and method security.

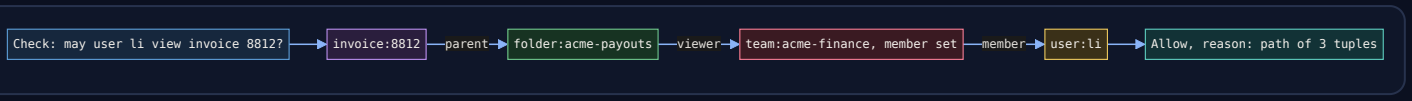
3. Zanzibar-style relationship tuples. Every permission fact is a **relation tuple** `object#relation@subject`, where the subject is a user or a *set* of users described by another tuple:

```
org:acme#member@user:ravi
team:acme-finance#member@user:li
folder:acme-payouts#viewer@team:acme-finance#member   (every member of the team)
invoice:8812#parent@folder:acme-payouts
```

A per-type **schema** says how relations combine — e.g., an invoice's *viewer* is its direct viewer, *or* its editor, *or* a viewer of its parent folder. The PDP answers:

- **Check** — "is `user:li` a `viewer` of `invoice:8812`?" Walk the graph: invoice → parent folder → folder viewers include acme-finance's members → li is a member → **allow**. Three hops.
- **Expand** — the full tree of who views this folder, and why — the audit answer.
- **Read / Write** tuples — the product writes a tuple when someone joins a team or a folder is shared, and deletes it on removal: one place records "li left the finance team".
- **List objects** ("which invoices can li see?") — the expensive reverse question, answered with precomputed indexes (Zanzibar's *Leopard* index flattens deeply nested group membership) or search filtering (step 5).

One check, walked through the relationship graph



The path itself is the audit reason.

4. Consistency: the "new enemy" problem. Stale authorization data can be a security bug. The Zanzibar paper's example, translated: a seller removes Bob from finance at 10:00:00 and uploads a payout report to the finance folder at 10:00:05; a check against a replica 10 seconds behind lets Bob — the *new enemy* — read content created *after* his removal. Fix: a **consistency token** (a *zookie*) saved with the content encodes "permission data as of now", and checks on that content must use a snapshot at least that fresh. Old content can use cached, slightly stale snapshots; new content always sees the removal. Open-source systems offer the same per-request choice ("fully consistent" vs. "at least as fresh as this token").

5. Make it fast enough for every request.

- **Cache decisions and sub-results** ("li is a member of acme-finance" serves thousands of checks), with short lifetimes or tied to consistency tokens.
- **Batch** — one bulk check per 50-invoice page.
- **Filter lists at the source.** *Pre-filter*: index each document with the principals (users, groups, services) allowed to read it, updated from tuple changes — needed when most results would be dropped. *Post-filter*: fetch a page, bulk-check, fetch more — fine for small pages.
- **A local decision point** for pure policy rules: an ABAC engine such as Open Policy Agent (OPA) as a sidecar, answering in microseconds with no network hop.

6. Decide failure behaviour per action. Most requests **fail closed** (deny) when the service is unreachable — a leak is worse than an error; a few low-risk reads may **fail open** from cache briefly. Write it down per endpoint, like the choices in Graceful Degradation.

7. Log every decision — question, answer, policy version and reason (the tuple path) — so "who could read our payout data, and why?" is a query.

Where each check lives, end to end

Layer	Checks	Why there
API Gateway	Is the token valid? Does its scope allow this API?	Cheap, once per request, no domain knowledge
Service mesh (east-west)	May the *checkout service* call *payments*?	Workload identity, not user identity (the Service Mesh lesson)
Service handler + authorization service	May *this user* do *this action* on *this object*?	Only the service knows which object; only the PDP knows the relationships

Walkthrough with numbers (the seller portal)

- **Check volume.** 16,000 req/s reach the gateway (the capstone's figure); about a quarter hit the portal, each making ~3 object checks → **~12,000 checks/s**. An 80% decision-cache hit rate leaves **~2,400 checks/s** — a handful of instances.
- **Latency budget.** Most checks are 2–4 graph hops; with membership sub-results cached, well under **10 ms at p99** against a 200 ms page budget. List pages use one bulk check.
- **Tuple storage.** Memberships and shares: ~2M staff × ~10 tuples = **20M tuples**. A `parent` tuple for all 500M invoices is ~**50 GB** at ~100 bytes each — feasible, but a second copy of ownership data to sync (by Change Data Capture from the orders database). Alternative: pass the invoice's folder in the check as a **contextual fact** and store tuples only for things users actually share. Choose per object type.
- **Incidents, replayed.** Removal deletes one tuple; the later report carries a consistency token, and search's pre-filter updates within seconds. `GET /invoices/8812` now calls `check(user, "view", invoice:8812)`, so an edited URL gets **403**; a shared test suite asserts another organisation's user is denied on every object endpoint.

Trade-offs

You gain: one source of truth (removals everywhere, rules change without thirty redeploys); auditability (decisions logged with reasons); expressive nested sharing without recursive SQL; explicit freshness via consistency tokens.

You pay:

- **A critical dependency on nearly every request** — high availability, caching, deliberate fail-closed.
- **Data duplication** — tuples mirror domain facts; dual-writes drift, so sync via CDC or the outbox pattern (event saved in the same transaction, published afterwards).
- **"What can I see?" stays hard** — list filtering needs extra indexes.
- **A policy language to test** — wrong schemas grant access silently; test "must deny" cases like code.

When not to use it

- **A monolith or a handful of services with a few roles** — framework RBAC plus ownership conditions in queries (`WHERE org_id = :callerOrg`) suffices.
- **As a feature-flag or entitlement system** — "is premium on for this plan?" is product configuration (the Feature Flags lesson's *permission flag*).
- **As a substitute for authentication** — the PDP trusts the identity it's given; token verification stays at the gateway and service boundary.

Typical misuse: a Zanzibar clone for five roles — or thirty services growing their own permission tables until an audit or BOLA report forces change.

Real-world examples

- **Google Zanzibar** — permissions for Drive, YouTube, Calendar, Cloud and more; the 2019 paper reports over two trillion ACL tuples, millions of checks per second, and over three years p95 under 10 ms

and availability above 99.999%.

- **Open-source Zanzibar-style systems:** SpiceDB (AuthZed), OpenFGA (a CNCF project started at Auth0/Okta), Ory Keto. **Airbnb's Himeji** is an in-house relative.
- **ABAC policy engines:** Open Policy Agent (OPA) with Rego, often as a sidecar; AWS's **Cedar** language and Amazon Verified Permissions.
- **For AI engineers:** a RAG system must filter retrieved passages by the *asking user's* permissions before they reach the prompt, or the model summarises documents the user may not open (RAG Pipeline). Agents calling tools for a user should be checked as *that user*, not with the agent's broad service credentials.

In the interview

Prompts: "design sharing for a Google-Drive-like product", "how do 30 microservices enforce permissions consistently?", or in any multi-tenant design, "how do you stop one tenant reading another's data?"

30-second answer: "I separate enforcement from decision. The gateway authenticates and checks coarse token scopes; each service enforces at the object — `check(user, action, object)` — against a central authorization service. RBAC for staff roles, relationship-based rules for ownership and sharing — Zanzibar-style tuples like `folder#viewer@team#member`, so a check is a short graph walk — and attribute conditions like MFA on top. Removals delete one tuple and take effect everywhere; consistency tokens stop a removed user seeing content created after removal. It's on the hot path, so I cache membership sub-results, bulk-check list pages, pre-filter search, fail closed, and log every decision with its reason."

Likely follow-ups (with answers):

- *"How do you show only permitted documents in search?"* — Pre-filter: index each document with its permitted principals, updated from tuple changes, and query "readable by any of my principals." Post-filtering with bulk checks works only when few results are removed.
- *"The authorization service is down?"* — Fail closed for writes and sensitive reads; short-lived cached decisions for low-risk reads only if the product explicitly accepted that. It's tier-0 (the most critical class, everything depends on it): replicated, cached, at least as available as its callers.

Mistake that fails candidates: checking only "is logged in" (BOLA); per-object permissions in JWTs; a domain-aware gateway; ignoring revocation delay.

Self-check

- **Q1: Define PEP and PDP and where each lives.** → PEP: the handler (or gateway filter, for coarse checks) touching the object and obeying the answer — local, since only the service knows the object. PDP: the central authorization service with rules and relationships — central for consistency and audit.
- **Q2: Organisations, nested teams, folders shared with teams — RBAC, ABAC or ReBAC?** → ReBAC for sharing (paths from user through teams and folders), RBAC for a few staff roles, ABAC conditions for context such as MFA. Pure RBAC means a role per organisation per folder (role explosion); pure ABAC can't answer "who can see this folder?" without evaluating every user.
- **Q3: After Bob's removal from a team, he opens a document uploaded 5 seconds later, because checks read a replica 10 seconds behind.** → The "new enemy" problem. Store a consistency token with each content version and evaluate checks at a snapshot at least that fresh; older content may still use cached snapshots.

Related patterns

- **API Gateway** — authentication and coarse scopes; object decisions stay behind it.
- **Service Mesh** — authorizes *service-to-service* calls by workload identity; this lesson authorizes *users* on objects.
- **Sidecar** — a local policy engine (OPA) as sidecar decision point.
- **Change Data Capture** — keeps tuples in sync with domain databases.
- **Feature Flags** — gate features for cohorts; never a substitute for authorization.

TL;DR

Problem	Every service wrote its own permission checks: object-level holes (BOLA), removals not reaching every service, slow recursive sharing queries, no answer to "who can see X?"
Mechanism	Services enforce `check(user, action, object)`; a central decision service holds RBAC roles, ReBAC tuples (Zanzibar-style `object#relation@subject`, checked by short graph walks) and ABAC conditions; consistency tokens for freshness; caches, bulk checks and search pre-filters for speed; every decision logged.
Costs	A critical dependency on nearly every request (replicate, cache, fail closed), tuples to sync with domain data, list filtering needing extra indexes, a policy language to test.
Skip it when	A few services with a few roles — framework RBAC plus ownership conditions in queries is enough.

Sources: R. Pang et al., "Zanzibar: Google's Consistent, Global Authorization System", USENIX ATC 2019; D. Ferraiolo, R. Sandhu et al., "Proposed NIST Standard for Role-Based Access Control", *ACM TISSEC* 2001; NIST SP 800-162, *Guide to Attribute Based Access Control (ABAC) Definition and Considerations* (2014); OASIS XACML 3.0 (PEP/PDP terms); OWASP API Security Top 10 (2023), API1 "Broken Object Level Authorization"; Open Policy Agent documentation; J. Cutler et al., "Cedar: A New Language for Expressive, Fast, Safe, and Analyzable Authorization", OOPSLA 2024; OpenFGA and SpiceDB documentation; A. Yao et al., "Himeji: A Scalable Centralized System for Authorization at Airbnb", Airbnb Tech Blog (2021).

Capstone - The Entry Point, Assembled

Original lesson: Capstone - The Entry Point, Assembled on DesignGurus ↗

What this capstone does. Designs the full public API layer for web, mobile, TV and 200 partners by tracing **one request from a customer's phone in Sydney** inward, layer by layer, deciding which responsibilities belong where. At each "pause and decide," commit to an answer before reading on.

Two rules come with you the whole way:

1. **Every layer is the same proxy primitive playing a different role.**
2. **Handle each request at the earliest layer that can.** The cheapest request goes no deeper.

> **Analogy.** An airport: most questions answered at the entrance kiosk (CDN); one security checkpoint that checks ID but not your destination (gateway); differently shaped lounges (BFFs); a turnstile limiting one badge's passes per minute (rate limits); badge readers between staff rooms (sidecars/mesh).

The brief

- **Clients:** web, iOS and Android, a TV app, **200 partner integrations** (schedules you can't control).
- **Traffic: 40,000 requests/s** at peak, global; about **60%** static or publicly cacheable.
- **Backend:** about **30 services in 3 languages** on the scaled fleet from earlier modules, reliability machinery inside.

- **Standing requirements:** partners must **never break**; **no tenant** may starve others; **east-west encryption** is on the security roadmap.

Stage 1 — The visitor's city (Edge / CDN)

Pause and decide: 60% of traffic is static or publicly cacheable, and Sydney is 200 ms from the origin. What stops at the edge, what's only sped up there, and what must never be cached there?

- **Stops at the edge.** Fingerprinted static assets cached forever; identical-for-everyone API responses like catalog browsing at `s-maxage=60, stale-while-revalidate`. Same-for-everyone bytes never cross the ocean.
- **Sped up at the edge.** TLS terminates over Sydney's **15 ms** path instead of the **200 ms** ocean route; dynamic requests cross on warm connections. Better than a Sydney datacenter: fixes handshakes and most static content without replicating the database.
- **Never cached.** Authenticated responses get `private, no-store` via **reviewed** configuration — a private-data mistake would reach strangers worldwide.

Result: of **40,000 req/s**, around **24,000** stop here, using no application capacity.

Stage 2 — The one entrance (Gateway)

Pause and decide: 30 services and four client types. What does the entrance check, what does it refuse to know, and where does its reliability come from?

The remaining **16,000 req/s** reach one origin hostname.

- **A redundant API gateway** — reliability from load-balancer machinery (12 instances behind the load balancer, with health checks), as befits a deliberate single dependency.
- **Checked once: AuthN** (including token revocation), trusted identity passed inward, rate limiting, request checks (size caps, validation), TLS, path routing (private topology), and the trace **request ID**.
- **Refuses to know:** discounts, order calculations, client-specific shaping. Services handle **AuthZ**, enforced against a shared Authorization Service; shaping goes to BFFs. Thin = rarely changed = stable.

Stage 3 — One entrance, four shapes (BFF)

Pause and decide: the mobile product screen needs six services, TV shows much less, and partners value stability over speed of change. How many BFFs, who owns each, and which "BFF" isn't really one?

- **Three BFFs + one API product:** a **web BFF**, one **shared mobile BFF** (iOS + Android) and a **TV BFF**, each client-team-owned, fanning out in parallel (mobile: six calls x ~5 ms, one ocean crossing) and returning screen-shaped responses with **no business rules** — any number two BFFs could disagree on belongs in a domain service.
- **Partners get a public API product**, not a BFF: **versioned, documented and stable** for consumers you can't update, because partners can't track your release speed.

Stage 4 — The rate limits

Pause and decide: Tuesday's partner bug fires 4,000 authenticated, valid retries per second. Which keys, algorithm and tiers — and where do counters live across 12 gateway instances?

- **Token buckets keyed by API key or account**, not IP (one IP may be a campus); **b** sized for normal page-load bursts.
- **Tiers:** free, paid, partner, with documented rates, plus **cost weighting** — search costs **10x** a health check.
- **The refusal contract:** 429 + `Retry-After` + remaining-budget headers, so partner SDKs back off themselves.
- **Shared counters:** Redis atomic increments across all **12** instances — local limits would allow 12x the budget.
- **Login** gets a per-account limit, since **credential stuffing** targets accounts.

Failure walk — the 4,000 req/s partner bug: the burst is used up, 100/s continue, and **3,900/s are refused in ~0.1 ms each** — about **1/500th** of serving them. Every other tenant is unaffected.

Stage 5 — The contracts that outlive you (Pagination and Versioning)

Pause and decide: partners run full exports, and old mobile builds stay installed for two years. Which pagination, where do versions live, and — the senior question — what retirement plan is written down when a version is announced?

- **Pagination:** opaque cursors over composite `(created_at, id)` positions across the public API — partners may reach page **5,000**, where offsets mean growing database work plus drift; opacity lets the sort key change later.
- **Versioning:** additive changes and tolerant readers by default; unavoidable breaks become one coarse `/v2` via a **gateway adapter** onto one backend. Every version is *born with* its retirement plan: `Sunset` headers, per-client usage reports, a brownout plan. **Two live versions at most.**
- **Mobile BFF middle ground:** quick additive changes, major versions tied to the **oldest supported app build**.

Stage 6 — The turn inward (Sidecar / Mesh)

Pause and decide: 30 services, 3 languages, a security mandate — and a two-person platform team. Which ladder step, honestly?

- Languages, services and security need justify a mesh, but **two engineers can't safely run a control plane every team depends on**.
- **Decision: step 2 now** — sidecars providing mTLS, uniform timeouts and retries via one language-agnostic binary with **templated static config**; **step 3 (mesh control plane) later**, once a platform team is staffed. Adoption thresholds include organizational ownership; the mesh lesson's incident shows hand-configured sidecars will eventually need central coordination — so plan the hiring now.
- The gateway stays the only public entrance; **east-west traffic never detours through it**.

The final architecture

All six stages in one figure: 40,000 req/s → 24,000/s stop at the CDN → 16,000/s through the 12-instance gateway → BFFs and the Partner API product → ~30 services with sidecars (mesh control plane pending); the buggy partner's 3,900/s refused at the entrance.

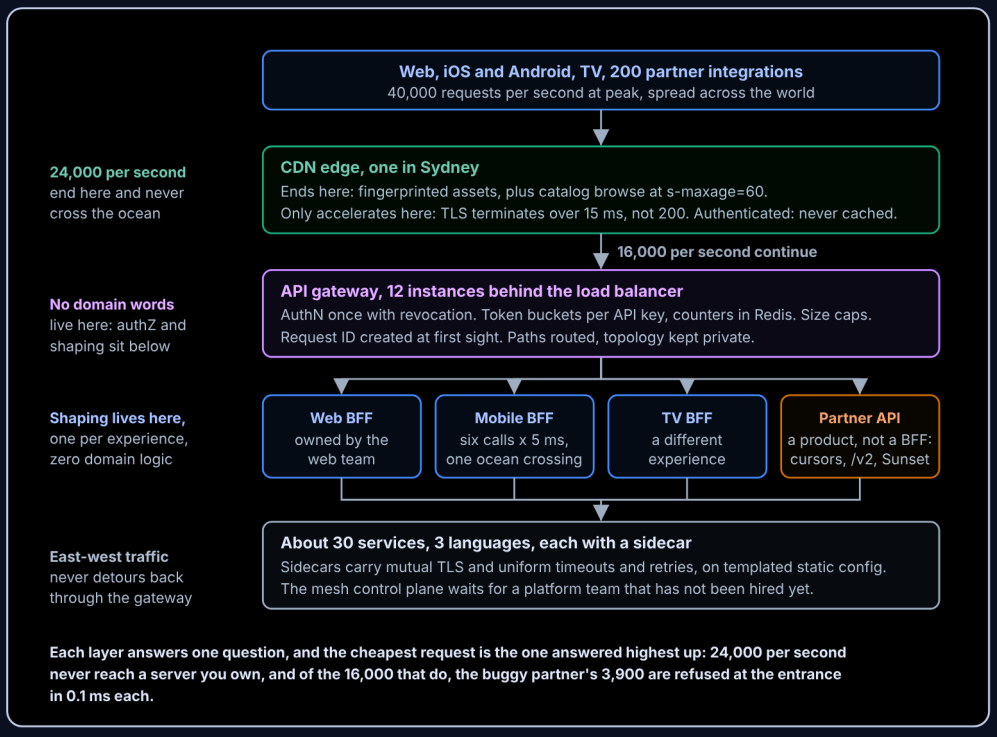


Figure — The entry point, assembled (course diagram)

Debrief — one question per layer

Layer	The question it answers	The decision that mattered
CDN	What never has to arrive at all?	60% stops at the edge; `private, no-store` reviewed like auth code
Gateway	What gets checked exactly once?	AuthN with revocation; no domain words at all; thin = stable
BFF	Who shapes responses, for each experience?	Three BFFs + one API product, owned by client teams
Rate limits	Who gets counted, and how?	Token buckets per real identity; refusal as a contract
Pagination/versioning	What outlasts your intentions?	Cursors with tiebreakers; versions created with retirement dates
Sidecar/mesh	What does the inside need right now?	Step 2 now, honestly; step 3 with a hiring plan
Authorization *(added lesson)*	May *this* user touch *this* object?	Services enforce at the object; one shared decision service holds roles and relationships

Each request stops at the earliest capable layer: **served from the edge cache, turned away by the rate limiter, checked at the gateway, tailored by a BFF, or authenticated service-to-service.** The only deliberate deferral is the full mesh — adoption thresholds include *organizational ownership*. Every layer has a clear owner, a measurable benefit, and a boundary keeping it from absorbing other layers' jobs.

Extend it yourself

- **The partner brownout.** A year after `/v2`, `/v1` still carries **30%** of partner traffic. Design the retirement campaign: dashboards, outreach tiers, brownout schedule, and what the `Retry-After` on a brownout **410** should say.
- **An entry point in multiple regions.** Add a second region. Which layers replicate easily (the CDN already; the gateway mostly)? Where do rate-limit counters live — **global fairness vs. regional latency**? Which BFF assumptions break?
- **For AI engineers.** Add an AI assistant endpoint: streaming responses use **SSE**, so check how each layer buffers; add **token-based** rate limits beside request-based ones; place the internal **LLM gateway** as a second entrance — behind the BFFs or beside the services? Why?

Interview summary

The public API layer is one proxy primitive in six roles, with each request stopping at the earliest layer that can handle it. The CDN answers ~60% at the edge; the thin gateway authenticates once and rate-limits per identity with a 429 contract; client-team BFFs shape responses; cursors and additive versioning with sunset dates keep partner contracts safe; sidecars secure east-west traffic now, with a full mesh deferred until a platform team can own it. Behind it all, each service enforces object-level permissions against one shared authorization service, so "logged in" is never mistaken for "allowed".

Module 9 — Operating in Production (Observability and Delivery)

Module Introduction

Original lesson: Module introduction on DesignGurus ↗

What this module is about. Some outages come from hardware, spikes or attackers, but most come from *changes we make ourselves*: a deploy (putting new code on servers), a config tweak or a database migration (a scripted change to tables or stored data) touches a healthy system and breaks it. The reliability module covered failures from outside; this one turns inward — shipping your own changes safely and *watching* production closely while each goes out.

Analogy. An operating room: the surgery is the deploy, the monitors are observation, and the ability to pause or reverse any step when a monitor turns red is rollback. Monitors first, then careful surgery.

The five patterns (plus two added lessons) and the job each does

The situation	The pattern
Is this one server fit to take traffic at this moment?	Health Check
A single slow request passed through thirty services — which one ate the time?	Distributed Tracing
Replace the old version with the new one in a single move, and keep an instant way back	Blue-Green Deployment
Make the new version earn trust on a small slice of real traffic before everyone gets it	Canary Deployment
Put the code in production "dark" (there, but switched off), then turn the feature on later — step by step, and with an off switch	Feature Flags
Package every service the same way and keep a fleet of them running in the declared state	Containers and Kubernetes *(added lesson)*
Get every change from commit (one saved, recorded code change) to production through one automated, trusted path, with the right rollout strategy	CI/CD Pipelines and Deployment Strategies *(added lesson)*

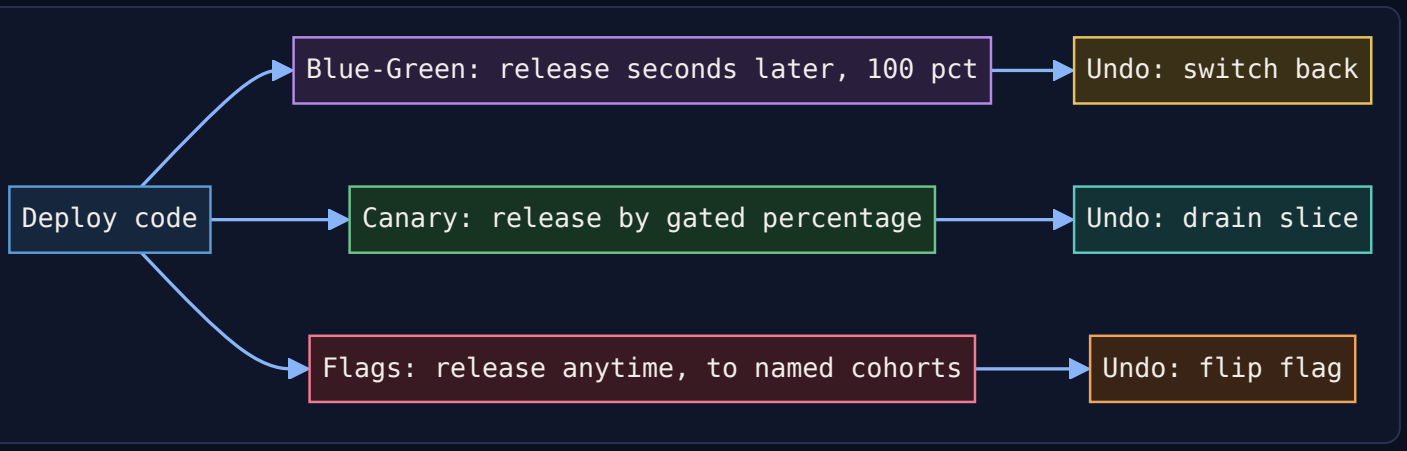
Two patterns **observe** ("is this server okay?", "where did this request spend its time?"); three **change** production without betting every user on each deploy. Observation comes first because every safe delivery technique depends on spotting failure fast — "roll back if the metrics look bad" means nothing without metrics.

Two facts that organize the module

1. **Deploying and releasing are two separate events.** *Deploy* = new code on servers; *release* = users experience it. Traditionally they're fused. The delivery patterns separate them by different amounts:
   - Blue-green readies new code in an idle environment and releases seconds later by moving traffic.
   - Canary releases already-deployed code to a small percentage of users.
   - Feature flags separate them fully — deploy Tuesday, release to a beta group next week, turn it off instantly. The design question for any change: *how far apart should deploy and release be?*

2. **Rolling back should be an everyday move.** Blue-green switches back in seconds, canary drains in seconds, a flag turns off instantly — versus building a revert and pushing it through CI (continuous integration, the automated build-and-test pipeline), which can prolong an outage for tens of minutes. Rule: **every delivery design needs a built-in undo design.** The capstone keeps a safe way back open at every stage of a live migration.

Delivery patterns placed on the deploy-to-release separation spectrum



All three share "deploy code"; they differ in *when, and for whom,* release happens, and each has its own cheap undo.

Four terms used throughout (basics: Module 0 — Deployment strategies and rollback, Observability)

- **Deploy vs. release** — code on servers vs. users feeling it (Fact 1); the module's most important distinction.
- **Exposure scope (blast radius)** — how many users a bad change can hurt before it's stopped: naive release 100%; blue-green 100% but fast to undo; canary ~1%; a flag can shrink it to one named team.
- **Roll back vs. roll forward** — reversing the change vs. pushing a fix on top. Rollback is usually quicker when available; the capstone reshapes a migration to keep it available where the naive plan wouldn't.
- **Golden signals** — the four numbers to watch during any change: **traffic**, **error** rate, **latency** and **saturation** (how close resources are to full). A canary's verdict comes from them — why observation is taught first.

How to read the module. Health Check (extends the load-balancer lesson's endpoints: shallow vs. deep, and a fleet-wide outage from one dependency) → Distributed Tracing (one request across services — the evidence every "watch the canary" step relies on) → delivery from simplest reversal to most flexible: Blue-Green (two environments, one switch) → Canary (widen exposure as measurements pass) → Feature Flags (deploy fully separated from release; "flag debt"). Two added lessons zoom out to the platform: **Containers and Kubernetes** (images, pods, Services, probes, rolling updates, control loops) and **CI/CD Pipelines and Deployment Strategies** (build once and promote, infrastructure as code, GitOps, and a comparison of recreate, rolling, blue-green, canary and shadow). The capstone uses them all in a zero-downtime database migration.

Short on time? Start with **Canary Deployment** — interviewers probe it most. Health Check is brief; don't skip it.

Before you start — three questions about a system you know

- Your most recent bad deploy: how long until someone *noticed*, and until it was *undone*? This module shrinks both numbers.

- When a request is slow, can you tell which service used the time — or do you grep thirty log files and guess?
- Could your newest feature be switched off this minute, with no deploy? Who could do it?

Health Check Endpoint

Original lesson: Health Check Endpoint on DesignGurus ↗

What it is. A tiny HTTP endpoint on each instance (e.g., `/healthz`) answering one question: *"should this instance get traffic right now?"* The load balancer, the container orchestrator (e.g., Kubernetes, which starts, stops and places containerised apps) and the auto-scaler call it every couple of seconds and act drastically on the answer: stop sending traffic, or kill and restart the instance.

Analogy. A manager asks a shop assistant "Are you conscious?" (no → ambulance, i.e., restart) and "Can you serve customers this minute?" (no → often harmless: the till is being set up; send customers to the next counter). Confuse them and you call an ambulance whenever the till is slow.

The problem: naive health checks lie in both directions

From the load-balancer lesson:

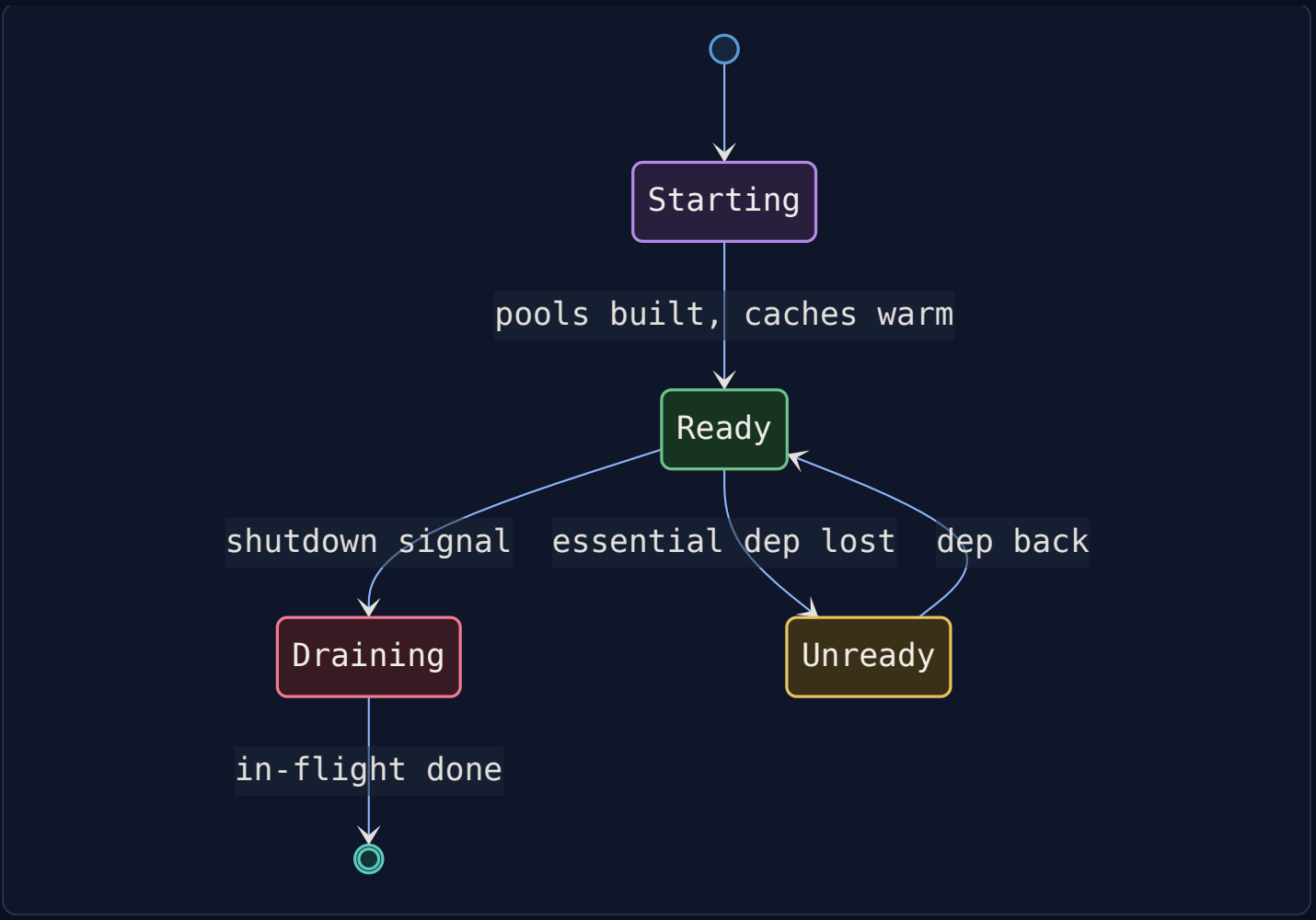
- A **shallow** check ("I'm alive, HTTP 200") stays green while the instance can't reach the databases or services it needs, so users keep being routed to a useless server.
- A **deep** check probes dependencies (e.g., pings the database). When the *shared* database hiccups, **every** instance fails at once and the load balancer pulls *the whole fleet* — a partial slowdown becomes a full outage.

Ask a better question — not "is the architecture healthy?" but "should *this instance* get traffic?" — and, as modern platforms do, split it into two questions with different consequences.

How it works: two verdicts, two consequences

1. **Liveness — "am I broken at a basic level?"** Failure → the platform **restarts** the process. So liveness covers *only* problems a restart cures: unresponsive, deadlocked (threads waiting on each other forever), or permanently stuck. **Dependencies never go here**: restarting your app can't fix a broken database, and a shared outage would fail every instance's liveness, restarting healthy processes over and over — a *restart storm*.
2. **Readiness — "should traffic come to me right now?"** Failure → the platform **de-routes** the instance temporarily — mild and reversible. A perfectly healthy instance can honestly be *not ready*: starting up, filling caches, building connection pools, or draining before shutdown. Each check must fit the consequence it triggers.
3. **Lifecycle honesty.**
   - At **startup**, readiness starts false; the instance builds pools and warms caches, then flips to true — no user hits a cold instance.
   - At **shutdown**, readiness goes false first; in-flight requests finish while the load balancer steers away; then it exits. During a rolling deploy this stops each new instance being slow for its first ~30 seconds.

Instance lifecycle as seen by readiness



Ready only after warm-up; Unready briefly if a truly essential dependency is lost; always Draining on the way out.

Three rules that keep health checks safe

- **Check yourself, not the architecture.** Handle dependency failures *inside* the service with circuit breakers and fallbacks, staying "ready" in a reduced mode. Exception: a service useless without one dependency (a pure database proxy) *may* include it in readiness — accepting, eyes open, that its failure de-routes every instance.
- **Cost almost nothing.** 30 services × 40 instances probed every 2 seconds = **600 checks per second**, forever. A DB query per probe is 600 useless queries per second, and a slow probe can *time out*, making healthy capacity look failed — a self-made outage. Aim for an in-memory check of roughly **1 ms**.
- **Tell the truth at every lifecycle stage** — not ready until warm, not ready while draining.

Routing checks vs. monitoring checks

Routing checks are per instance, frequent and cheap. **Monitoring checks** exercise the whole system with **synthetic transactions** (scripted pretend requests, like a test checkout) and deeper dependency probes, centrally and less often, producing *alerts for people*, never routing decisions. Keep them on separate endpoints; merging them is how deep dependency tests creep into fleet-wide routing and restarts.

Treat every probe condition as a reviewed policy. Don't casually add "one more network call" to readiness for a new dependency. Write down: condition → action on failure → how that action aids recovery. If neither restart nor de-routing fixes it, it doesn't belong in that probe.

Response format. Pass/fail is all the platform needs; diagnostic fields naming the failing local condition are fine, but never secrets or sensitive config, and no costly work to produce them.

Liveness vs. readiness (comparison)

	Liveness	Readiness
Question	Am I broken at a basic level?	Should traffic come to me right now?
Consequence	Restart (drastic)	De-route (mild, reversible)
May include dependencies?	Never	Only the essential ones, picked on purpose
Can honestly say "no" while healthy?	No	Yes — while warming up or draining
What goes wrong if designed badly	Restart storms whenever a dependency fails	The whole fleet removed whenever a shared dependency fails

Trade-offs

You gain: failing instances removed at machine speed (seconds); safely ordered deploys (only warm instances admitted, each drained before termination); self-healing (instances failing liveness are replaced).

You pay:

- **Fleet-wide mistakes.** ~10 lines of code, but dependencies in liveness → restart loops; in readiness → the whole fleet pulled; slow probes → timeouts inventing failures.
- **Probe load** — tiny per probe, never-ending, on every instance; needs discipline as services grow.
- **False confidence** — green means one narrow question was answered, not that a customer can check out; that proof comes from central monitoring.

When not to use it (or how to scope it)

Every service receiving routed traffic needs health checks; keep them narrow:

- **Don't use one endpoint for routing, restarting and alerting** — three consequences, three designs.
- **Don't make readiness depend on optional dependencies** — if recommendations are down, hide the widget (degrade), don't take the fleet offline.
- **Batch workers and queue consumers** get no routed traffic: "health" is liveness plus **backlog age** (age of the oldest unprocessed message); the queue's metrics are the readiness signal.

In the interview

Expect "what does your health endpoint check?" after almost any load-balancer or Kubernetes design. The interviewer listens for the liveness/readiness split and the shared-dependency danger.

30-second answer: "Two endpoints. Liveness: am I broken in a way a restart fixes — responsive, not deadlocked — never dependencies, because a restart won't repair the database and a shared outage becomes a restart storm. Readiness: should I get traffic now — false while warming and draining; dependencies only if the service is genuinely useless without one, since degraded serving beats removal. Both cheap: probes hit the fleet every two seconds, so a DB query means hundreds of wasted queries a second, and a timeout can create an outage. Deep system checks belong to central monitoring feeding alerts, never routing."

Likely follow-ups (with answers):

- *"How does an instance avoid serving while cold?"* — Readiness stays false until pools are built and caches warm; shutdown reverses it: unready, drain in-flight requests, exit.
- *"Your database-proxy service has no degraded mode. Should readiness include the DB?"* — Yes, deliberately: it can't work without it. Accept that a DB failure de-routes every instance and

make *callers* degrade gracefully. The rule follows the consequence, not a blanket "never".

Mistake that fails candidates: dependencies in liveness; one endpoint for routing and alerting; "ping the database" without seeing how it removes the whole fleet.

Self-check:

- **Q1.** The two questions and their consequences; why are dependencies forbidden in one and rationed in the other? → Liveness → restart; no dependencies, since restarting me can't repair another system. Readiness → de-route; only dependencies without which the service is useless — optional ones should cause degradation, not lost capacity.
- **Q2.** A service uses a database (essential), Redis as a cache (misses fall back to the DB) and a recommendations service (optional widget). Where does each check go? → Liveness: in-memory process check only. Readiness: the database plus warm-up/draining state. Redis and recommendations: neither. A synthetic checkout: central monitoring.
- **Q3.** At 3:07 AM the database undergoes a 90-second **failover** (a replica promoted to primary); every instance of six services restarts, some repeatedly; the outage lasts 20 minutes. Why? → Liveness included a DB call: every instance looked "broken" and restarted, discarding in-flight work, returning cold, failing again — and every restart rebuilt its connection pools, hammering the recovering database. Fix: liveness checks only the process; the DB goes in readiness only if truly essential, otherwise breakers handle degradation; deployment review blocks liveness conditions a restart can't fix.

Related patterns:

- **Load Balancing** — the main consumer; readiness drives removal and draining.
- **Graceful Degradation** — where dependency problems belong: degrade and stay ready.
- **Sidecar** — the app reports not-ready until its proxy is ready; the sidecar drains last.
- **Auto-Scaling** — failed liveness triggers replacement; the fleet's self-healing.

TL;DR

- **Problem:** routing and restart machinery wants a per-instance verdict every ~2 s; naive endpoints mislead both ways — shallow stays green while useless, deep turns the fleet red on a shared blip.
- **Mechanism:** liveness covers only restart-fixable brokenness, never dependencies; readiness covers traffic-worthiness — false while warming or draining, only essential dependencies. Both ~1 ms. Deep checks belong to central monitoring.
- **Costs:** tiny code with fleet-wide failure modes (restart storms, mass removal, timeout-invented failures), plus endless probe load.
- **Skip it when:** never — but don't let one endpoint do routing, restarts and alerting, and let queue workers report backlog age.

Distributed Tracing

Original lesson: Distributed Tracing on DesignGurus ↗

What it is. Capturing one request's whole trip across services. At entry it gets a **trace ID**; every service and database call it touches records a timed **span** carrying the trace ID and its triggering call. A collector rebuilds the spans into one timeline — a **waterfall** — showing exactly where the time went.

Analogy. A parcel tracking number: each depot's scan alone says little; the tracking page joins

them and reveals the parcel sat three days at one sorting centre.

The problem: the time hides *between* the logs

Checkout is "slow sometimes": **p99** (latency only the slowest 1% exceed; see percentiles) is **4.2 seconds** against an **800 ms** budget, hitting some requests unpredictably. One checkout crosses **12 services through 40+ calls** — gateway, **BFF**, checkout, inventory, payment, fraud, pricing and more.

Twelve engineers each search one service's logs; every service says "I was fast" (checkout logged 8 ms of its own work). Every log is *correct* and none answers the question, because **the 4.2 seconds was spent between the records**, along a specific call sequence no single log captures. Three engineers then spend two days aligning timestamps from machines whose clocks disagree by milliseconds (the vector-clocks lesson explains why physical timestamps can't order cross-machine events); they ship a fix and p99 doesn't move. Logs answer *"what did this component do?"*; the question was *"which path did this request take, and where did its time go?"* — and nothing recorded that.

Why the obvious fixes fail

- **"Add more logging."** More records, still not *linked*. Relationships were missing, not detail.
- **"Put a correlation ID in every log line."** Helps partly — searching the ID finds every line for the request — but there are no **parent/child relationships** (which call triggered which), clock skew muddles timing, and nothing draws the path.
- **"Send synthetic test requests."** They recreate only known conditions; nobody knows what triggers this one.

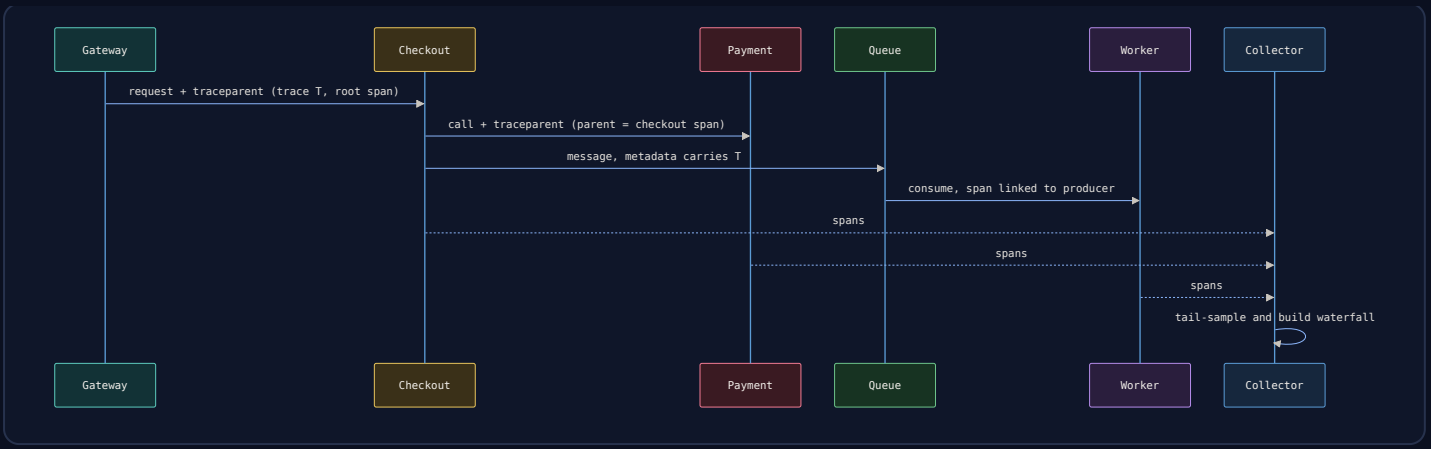
How it works

1. **The entry point creates a trace ID.** The API gateway sees every external request first, so it becomes the *root* of every trace tree (why the gateway lesson gave it request-ID creation).
2. **Each operation records a span** — handling a request, an outbound call, a query — with a name, start and end times, its **parent span**, and attributes (URL, status, error). Spans nest into a tree; drawn on a timeline, that's the waterfall.
3. **Context propagation carries the IDs forward.** On synchronous HTTP calls in a standard header (W3C `traceparent`); on asynchronous queue work in **queue message metadata** (labelled fields travelling with a message, separate from its body). The consumer's span *links* to the producer's, and the gap shows queue wait. **If one service fails to pass the header on, every later span is an orphan** and drops out of the trace.
4. **Spans go to a collector**, which reassembles searchable waterfalls.

The slow checkout's trace (indentation = parent/child):

```
Trace 7f3a...  (checkout, 4,214 ms total)
├── gateway                  2 ms
├── mobile BFF               11 ms
|   ├── checkout svc         4,190 ms   ← there
|   |   ├── inventory        12 ms
|   |   ├── payment svc      4,110 ms   ← in here
|   |   |   ├── fraud check  1,210 ms  (timeout)
|   |   |   ├── fraud check  1,240 ms  (retry 1)
|   |   |   ├── fraud check  1,190 ms  (retry 2)   ← found it
|   |   |   └── charge       180 ms
|   |   └── pricing          9 ms
```

Context propagation through sync and async hops



Instrumentation — three layers, least to most effort:

- **Service-mesh sidecars** capture every inter-service hop with no application change.
- **Auto-instrumentation libraries** such as **OpenTelemetry** wrap HTTP clients, database drivers and queue clients in-process.
- **Manual domain spans** for steps no library can guess ("apply discount rules") — only where the same investigations recur. The two automatic layers already crack many latency puzzles.

Sampling — you can't keep everything. 40,000 requests/s × ~40 spans = **1.6 million spans per second**.

- **Head sampling** decides at the *start* (say keep 1%): cheap and simple, but blind to the outcome — a **99% chance of discarding** the slow or failing trace you want.
- **Tail sampling** decides *after* completion: keep every error, every request missing its latency objective, plus a small random baseline. The price: **buffering every trace** until the decision — real infrastructure, and a favorite interview topic.
- In practice: tail rules for errors and outliers plus a head-sampled baseline.

Walkthrough with numbers: the same incident, with tracing

- **Minute 1:** filter `duration > 4s` — dozens of traces, kept by tail sampling as outliers.
- **Minute 2:** the waterfall shows, inside payment's 4,110 ms, **three sequential fraud-check spans of ~1.2 s each** — a timeout and two retries. Retries behaved as configured; fraud's p99 had crept past payment's timeout, so affected checkouts spend timeout × 3 ≈ **3.9 s**.
- **Minute 4:** root cause: fraud's slowdown, with payment's timeout/retry settings shaping the symptom. The wrong theory had cost six engineer-days; tracing turns **days of reconstruction into minutes of reading**.
- **Bonus:** aggregated traces form a live **dependency map** (who calls whom, how often, how fast), often exposing undocumented paths.
- **The bill:** errors + outliers + a 0.1% baseline ≈ **2,000 spans/s** stored — manageable — but the tail-decision buffering tier belongs in capacity planning.

The three pillars of observability

	Metrics	Traces	Logs
Question	"How much, how fast — overall?"	"Which path did this request take?"	"What exactly happened at this spot?"
Shape	Numbers over time; cheap enough to keep forever	Trees of timed spans; sampled	Text events; very high volume
Alerting	*The* layer that alerts (golden signals)	Seldom alerts; explains the alerts	Sometimes (on error patterns)
Cost	Small	Medium, kept in check by sampling	The largest

Use them in order: **metrics ring the alarm** (p99 over objective), **traces localize it** (fraud retries inside payment), **logs explain it** (the exception and its parameters). Put the trace ID in log lines to jump from a waterfall to its logs. Without traces, teams leap from a global alarm to searching every log — the opening incident.

Trade-offs

You gain:

- Per-request evidence across any number of services — something separate logs can't express.
- Faster latency diagnosis: retry storms (waves of retries piling load on a struggling service), lock contention and **N+1 queries** (one query per item instead of one for all) appear as visible sequences.
- Cross-service error attribution — which component actually returned the 500.
- A free live dependency map.

You pay:

- **Propagation discipline forever.** Every service, client library and queue consumer must pass context; traces are only as good as the *weakest* client.
- **Sampling infrastructure** — the tail buffer and decision layer, plus tuning what counts as "interesting".
- **Instrumentation work** beyond the automatic skeleton, only where puzzles recur.
- **Privacy exposure** — spans can hold URLs, parameters and arbitrary attributes, so **PII scrubbing** (removing personal data) belongs in the collection pipeline (the event-driven-architecture lesson's rule).

When not to use it

- **A monolith** — a profiler explains time inside one process far better; tracing pays off when latency is scattered *between* processes.
- **Three services and a queue** — structured logs (named fields, e.g., JSON) with correlation IDs may suffice; tracing's value grows with hop count.
- **In place of logs** — spans are the skeleton (timing, structure); logs are the story.
- Common misuses: keeping 100% of traces forever (cost explodes with traffic) and hand-instrumenting every function from day one.

Real-world examples

- **Google's Dapper paper** — distributed tracing at planet scale via heavy sampling; most modern ideas descend from it.
- **OpenTelemetry** — vendor-neutral instrumentation, `traceparent` propagation and export; *the* standard to name in interviews.
- **Jaeger, Zipkin, Grafana Tempo** store and show traces; **Honeycomb, Datadog** are all-in-one observability platforms.
- **Service meshes** — produce the service-level skeleton for free.
- **For AI engineers:** observing an agent (an LLM-driven program planning steps and calling tools) *is* tracing: a run is a trace; each model call, tool call and retrieval is a span with token count, cost and model version. A waterfall might show a 40-second, $0.30 run making three mergeable sequential model calls, or the failing tool call. LLM tracing tools like **LangSmith** descend from Dapper, and context must pass through an LLM gateway just as through HTTP headers.

In the interview

Common prompt: "a slow request passes through thirty services — how do you find the delay?" "Search the logs" won't do; follow-ups probe sampling cost and async propagation.

30-second answer: "Distributed tracing: the gateway mints a trace ID, every operation records a timed, parented, tagged span, and context travels in headers across every hop — queue messages included — so any slow request becomes a waterfall and the hidden time shows up in minutes. Two design choices: tail sampling that keeps errors and latency outliers plus a small baseline, because head sampling discards the interesting 1% before knowing it's interesting; and propagation discipline, because one service dropping the header orphans everything after it. The mesh gives the service-level skeleton free; OpenTelemetry adds in-process detail where mysteries recur."

Likely follow-ups (with answers):

- *"Head or tail sampling?"* — Tail for errors and outliers (outcome unknown until the end) plus a head-sampled baseline; tail means buffering every trace until decided. Head-only sampling tends to discard the rare p99 examples you need.
- *"How does a trace cross a queue hop?"* — Trace context in message metadata; the consumer's span *links* to the producer's, and the gap is queue wait, so backlog delay appears in the same waterfall. Non-propagating queues are where traces break most.

Mistake that fails candidates: more logs; 100% retention without a cost model; propagation dying at the first queue.

Self-check:

- **Q1.** Define trace, span and context propagation; where is the trace ID created, and why? → A trace is one request's full path (trace ID); a span is one timed operation with start/end, parent and attributes; propagation carries trace and span IDs across every HTTP call and queue message so new spans attach correctly. The gateway creates the ID as the first component seeing every external request — the natural root.
- **Q2.** Team A is cost-sensitive with ordinary traffic; Team B has rare p99 spikes gone before anyone looks. Design sampling. → A: modest head sample (e.g., 1%) plus always-kept errors — predictable cost. B: tail sampling keeping every error and over-limit trace plus a small baseline, deliberately accepting the buffer cost, because rare unpredictable outliers are exactly what head sampling loses.
- **Q3.** Since Tuesday every checkout trace stops at payment; fraud and charge spans are gone though the calls still happen. → Payment stopped forwarding `traceparent` — likely a dependency update swapped the HTTP client or framework, orphaning fraud's spans. Apply OpenTelemetry auto-instrumentation to the new client; guard the whole class with **trace-completeness monitoring** (alert on rising orphan-span rates or missing expected children) and propagation tests in the service template.

Related patterns:

- **API Gateway** — mints the trace ID and root span.
- **Service Mesh** — sidecars supply the free skeleton.
- **Message Queue** — metadata carries context, keeping async work and queue delay in the waterfall.
- **Retry with Exponential Backoff** — tracing exposes retry storms (as in the walkthrough).
- **Canary Deployment** — canary verdicts rely on this telemetry.

TL;DR

- **Problem:** a slow request's time hides between per-service logs; every service says "I was fast," and stitching timestamps takes days and ends in a guess.
- **Mechanism:** gateway-created trace ID; a timed, parented span per operation; context in headers and queue metadata; collector-built waterfalls; tail sampling selecting errors and outliers from 1.6M spans/s.
- **Costs:** permanent propagation discipline, sampling infrastructure, instrumentation beyond the mesh skeleton, PII scrubbing.
- **Skip it when:** a single process (use a profiler), about 3 services (correlation IDs may do), or you hoped it would replace logs.

Blue-Green Deployment

Original lesson: Blue-Green Deployment on DesignGurus ↗

What it is. Two identical production environments: **blue** is live, **green** idle. Deploy the new version to green, test it on genuine production infrastructure with no users, then flip the router — within seconds all traffic is on green. Rollback is the same flip in reverse.

> **Analogy.** A theatre with two matching stages: the audience watches Blue while the crew builds and rehearses the next scene on Green; the seats swivel when Green is ready, and swivel back if anything goes wrong — Blue's set is still up.

The problems it solves

- **Slow rollback.** In-place rollback means rebuilding and redeploying the old version through CI, artifact storage (where built packages are kept) and rollout — **15–25 minutes** of limping even when engineers know exactly what to do. Rollback becomes one more deployment.
- **The verification gap.** Staging isn't production (data, config, integrations, load all differ); without a second *production* environment, a release's first full test happens with users on it.

How it works

1. **Deploy to green** while blue serves everyone. Green *is* production — same infrastructure, config and database; **infrastructure-as-code** (environments described in version-controlled files) prevents quiet **environment drift**, which would break the idea.
2. **Verify green before any user reaches it**: **smoke tests** (fast basic checks) on real infrastructure, readiness green, caches and pools warm. Deploy and release separated by seconds.
3. **Switch.** The router sends new traffic to green (change the load balancer target, or flip a Kubernetes Service selector). Blue **drains** — in-flight requests finish where they began. Sessions survive because they live in a shared store (the statelessness lessons).
4. **Keep blue on standby** for an undo window, usually hours. Switching back *is* the rollback: about **10 seconds**, no CI, no build. Afterward, blue becomes the next release's green.

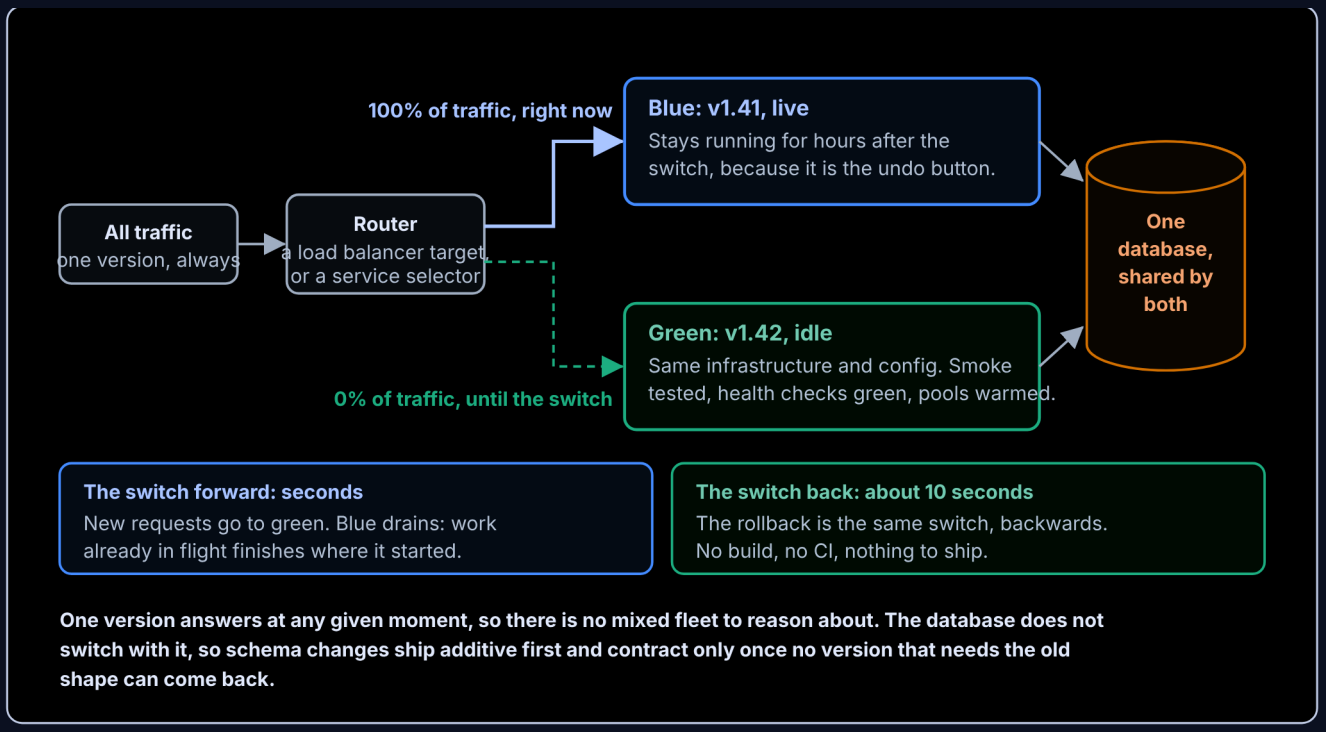


Figure — Blue-Green Deployment (course diagram)

The router sends 100% to blue (v1.41, live) and 0% to green (v1.42, smoke-tested and warm) until the switch; blue then stays up for hours as the undo button. The one shared database in the middle is the component that does not switch.

The database does not blue-green

Both versions use the **same database**, so throughout the rollback window old and new code must both safely read and write the shared schema and data. Schema changes therefore follow **expand-contract**:

- **Expand:** add new structures non-destructively, write both old and new forms, ensure either version reads safely.
- **Contract:** remove the old structure only when *no* version depending on it could return — after the rollback window closes.

This is API versioning's "additive first" applied to data. Blue-green swaps stateless application environments; *state* needs its own compatibility discipline — the subject of the capstone.

Blue-green vs. rolling deploy

A **rolling deploy** replaces instances in place, a few at a time.

	Rolling (replace in place)	Blue-Green
Extra capacity	About 1 instance's worth	2× for the duration of the window
Rollback	Redeploy: minutes	Switch back: seconds
Version mixing	v1 and v2 both serve for the whole rollout	Never — the switch is atomic
Blast radius at cutover	Grows gradually, instance by instance	100% at the moment of the switch
Fits	A cheap default; versions that are compatible with each other	When you need instant undo; changes that can't cope with mixed versions

Rolling is a cheap, universally supported default; blue-green earns its double capacity when reversal speed matters or versions can't coexist. Both choose who gets new code by *infrastructure location*; flags choose by *user*.

Trade-offs

You gain: rollback in seconds (a routing change, not a build); whole-environment verification before any user; never a mixed-version fleet (with rolling, v1.41 and v1.42 responses interleave, so they must coexist safely).

You pay:

- **2× capacity** during the window — cheap in the cloud (create green for hours, then delete), possibly permanently idle hardware on-premises. Weigh it against the outages prevented.
- **All-or-nothing exposure** — the switch moves **100% of users at once**; a bug that slipped verification hits everyone (canary fixes this).
- **Ongoing schema discipline** — every migration must stay expand-contract compatible, or the rollback button stops working.
- **Tricky draining** — long-lived connections (SSE, websockets) keep blue busy until they close or move.

When not to use it

- **You need step-by-step validation.** Blue-green checks the environment, not the variety of user behavior; a bug affecting 3% still reaches 100% — use a canary.
- **2× costs serious money** — large fleets favor canaries on existing capacity; on-prem teams may reserve blue-green for risky releases.
- **State is the main concern** — in database-heavy systems the migration is where risk lives.
- **Common misuse:** flawless app-tier blue-green with a destructive schema rename or dropped column; the router can return to blue, but blue can't read the altered data, so "rollback" makes things worse.

In the interview

Prompt: "zero-downtime deployment with instant rollback." Expect a follow-up on the shared database — it decides whether your rollback really works.

30-second answer: "Two identical environments: deploy to the idle one, verify on real infrastructure with zero users, gated on health and warm-up, then switch the load balancer — all traffic moves in seconds, and rollback is switching back, a reflex rather than a 20-minute redeploy. Blue drains and stays ready through the undo window. The database doesn't switch: both versions share it, so schema changes follow expand-contract — additive first, write both forms, contract only when rollback can't need the old shape — or the rollback button doesn't work. Limits: 2× capacity in the window, and 100% exposure at the switch; gradual exposure is the canary's job."

Likely follow-ups (with answers):

- *"Walk me through the database during one cycle."* — Expand additively (works for both versions) → deploy and switch → stay compatible through the observation/rollback window → contract once blue can never return. The rollback window and schema-compatibility window must be the same length.
- *"Sessions and in-flight requests at the switch?"* — Blue drains its requests while green takes new ones; shared-store sessions mean nobody is logged out.

Mistake that fails candidates: forgetting the database; claiming rollback is safe after green wrote a format blue can't read; choosing blue-green when gradual behavioral validation is needed.

Self-check:

- **Q1.** What switches, what drains, what never switches, and what keeps it safe? → The router target switches; blue drains in-flight requests; the shared database never switches and is protected by expand-contract (additive, dual-version compatible, removal only after the window).
- **Q2.** Team A (cloud, two deploys a day) vs. Team B (on-prem, 60% utilization): what does blue-green cost each? → A creates green for a few hours per release and deletes it — small next to one avoided 20-minute outage. B needs permanently idle hardware, so might reserve blue-green for high-risk releases and use rolling deploys or flags otherwise.
- **Q3.** Green (v1.42) went live at 14:00; at 14:30 you switch back and blue fails to read recent orders. → v1.42's migration renamed, removed or retyped a column, or wrote an incompatible format, so the rollback caused an outage. Now: fix v1.42 and **roll forward** (blue is incompatible), then examine data written 14:00–14:30. Prevention: expand-contract with contraction held until the window closes.

Related patterns:

- **Load Balancing** — the switch is a routing change; draining executes it.
- **Health Check** — readiness says when green is genuinely warm.
- **Canary Deployment** — the gradual alternative to 100% exposure.
- **API Versioning** — expand-contract is additive-first applied to schemas.

TL;DR

- **Problem:** in-place deploys make rollback a ~20-minute redeploy, and new versions meet real infrastructure only once users are on them.
- **Mechanism:** two identical environments: deploy to the idle one and verify, flip the router in seconds, drain the old one, keep it ready through the undo window, protect the shared DB with expand-contract.
- **Costs:** 2× capacity (permanent if on-prem), 100% exposure at the switch, tricky draining, permanent schema discipline.
- **Skip it when:** you need gradual validation across varied traffic, 2× is serious money, or the risk is in the data layer.

Canary Deployment

Original lesson: Canary Deployment on DesignGurus ↗

What it is. Release the new version to a small group of real users (e.g., 1%) and compare its vital signs with the old version's *live*. If it's as healthy, widen in steps (5%, 25%, 100%), each approved by a measured verdict; if worse, drain it within seconds. Only that small group was ever exposed.

Analogy. Miners carried canaries because the bird reacted to bad air first — but *watching* it was the point. A canary release without observation and automatic response is worthless.

The problem: "it runs" is not "it behaves"

v1.43 passes the full blue-green process — health checks, production smoke tests, warm-up — and switches at 14:00. At 14:08 support reports checkout failing for users with **a saved payment method** ***and*** **a non-US locale** — **3% of traffic**. No test covered that pairing (tests with either condition alone passed). The atomic switch exposed every affected user at once, and thousands of checkouts failed before rollback.

Green proved the version *runs*, not that it *behaves* for real users. Production combinations (payment state, locale, account age, device, usage) multiply faster than any test suite. **Real user traffic is the only complete test suite** — the goal is to use it without exposing everyone before a verdict.

Why the obvious fixes fail

- **"Improve the staging data."** Helps with known risks, but staging is always a partial, ageing copy of user behavior; it can't prove there are no unknown interactions.
- **"Mirror (shadow) production traffic to green first."** Shadowing copies live requests to the new version and discards its answers — fine for reads, unsafe for side-effecting writes like payments (a replayed charge bills twice; blocking it skips the path you wanted to test), and discarded responses produce no business outcomes. v1.43 broke in a write flow.
- **"Deploy at 3 AM."** Fewer failures per minute, but the same users hit the bug eventually, the team is exhausted, and light traffic gives *weaker* evidence, so percentage regressions surface later.

How it works

1. **The split.** Load-balancer weights or a mesh traffic policy route a controlled percentage (say 1%) to the new version (flag-based cohort routing is a close alternative).
2. **Sticky slicing.** Each user stays on *one* version for the whole workflow — bouncing between v1.42 and v1.43 could break a multi-step checkout and muddy the comparison. Hashing the **user ID** onto a hash ring (as in consistent hashing) gives a stable 1% cohort; growing it adds users without moving earlier ones, and lets engineers tell which version served a reported session.
3. **Compare against the baseline, not a fixed threshold.** Ask "is the canary *worse than the baseline right now*?" The other 99% — the **concurrent baseline** — sees the same traffic, dependency conditions and cache state at the same instant; yesterday's figures or fixed thresholds mistake daily swings for regressions. **Netflix Kayenta** automates this canary-vs-baseline analysis over many metrics.
4. **Compare the right things.** Golden signals **plus business metrics** (conversion — share of visitors completing a purchase — and add-to-cart rate) **plus traces**. A broken button may raise no errors or latency yet stop purchases; trace comparison can expose a new 200 ms span while overall p99 still looks fine. Without a conversion metric, a commercially broken release sails through.
5. **Automated gates and rollback.** Timed analysis advances each stage (1 → 5 → 25 → 100%) when its evidence window ends, or rejects it — no humans staring at dashboards (they lose focus and apply statistics inconsistently). A failing score **drains the canary automatically in seconds**.
6. **Respect small-group statistics.** At 40,000 req/s a 1% canary gets **400 req/s**; a bug hitting 0.1% of requests yields about **24 bad events per minute** — caught fast. Something affecting 0.01% of users may take hours or need a bigger slice. **Detection time depends on cohort size × effect frequency**, so compute expected event counts *before* choosing percentages and windows, or a "clean" gate may just mean too few samples.
7. **The database rule still holds.** Canary and baseline share one schema (expand-contract required), and unlike blue-green the two versions **coexist during every rollout**, so both must read and write the same in-between data safely — mixed-version compatibility becomes a permanent trait of your delivery system.

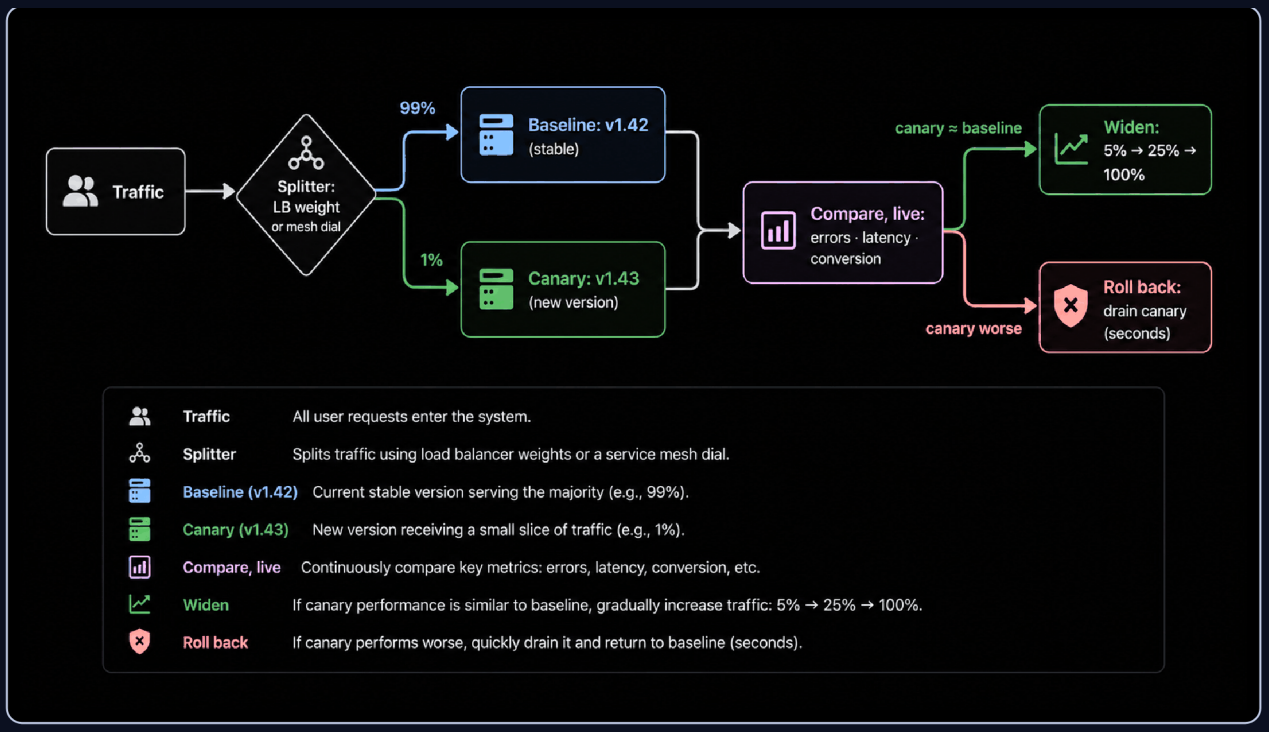


Figure — Canary Deployment (course diagram)

A splitter (LB weight or mesh dial) sends 99% to baseline v1.42 and 1% to canary v1.43; both feed a live comparison of errors, latency and conversion. Canary ≈ baseline → grow 5% → 25% → 100%; worse → drained in seconds.

Walkthrough with numbers

- **v1.43 again, as a canary.** 14:00: a stable 1% cohort gets v1.43. 14:06: analysis sees **2.1%** errors on the canary vs. **0.3%** on the baseline and drains it. About 1% of users were affected for six minutes, and idempotent clients retried failed checkouts harmlessly. A fixed canary starts at 15:30 and clears 1%, 5%, 25%, 100%.
- **A subtler rescue, two weeks later.** v1.44 changes no errors or latency, but canary-cohort checkout **conversion drops 8%**: a CSS bug hid the payment button on small screens, throwing no exceptions. The business-metric gate drained it — technical metrics alone don't tell the whole story.
- **The bill.** Rollouts take **hours** of observation windows instead of seconds; those hours buy evidence, worth it for risky changes.

Canary vs. blue-green

	Blue-Green	Canary
Exposure	100% at the switch	1% → 5% → 25% → 100%, each step gated
Verifies	The environment (it runs, connects and serves)	How it behaves across the variety of real traffic
Rollback	Switch back: seconds	Drain the slice: seconds, with fewer people hurt
Version mixing	Not allowed (atomic)	Required (the mix *is* how it works)
Capacity	2× for the window	About 1% extra
Speed	Seconds	Hours

Mature teams combine them: build green, then shift traffic onto it through a canary ladder — environmental verification *and* behavioral evidence. **A/B testing** uses similar routing for a different question: a canary asks "does this release work?", an A/B test "which product option does better?" over a longer experiment.

Trade-offs

You gain: blast radius as a dial (the current percentage, for minutes); verification by the only complete test suite, with explicit verdicts; compounding confidence — trusted canaries lead to smaller, more frequent, easier-to-reverse deploys.

You pay:

- **Time** — hours per rollout (skipping the ladder can be justified in emergencies).
- **Permanent mixed-version compatibility** for schemas, APIs and messages.
- **Statistical and telemetry discipline** — weak baselines, missing business metrics or unstable cohorts approve bad releases or block good ones.
- **Harder diagnosis** — every metric, log and trace needs a version label, or the canary's regression drowns in baseline totals.
- **Infrastructure** — traffic splitting, stickiness and automated analysis via a mesh, **Argo Rollouts** or **Flagger**.

When not to use it

- **Low traffic** — 1% of 300 req/s is **3 req/s**, barely any signal; use blue-green, solid tests or flags targeting hand-picked users.
- **Versions that can't coexist** — use blue-green's atomic switch.
- **Emergencies** — restore production first; canary the *next* release.
- **Bugs too rare or cohort-specific** for the slice to detect (self-check Q3); a canary is one layer, not a guarantee.
- **"Canary theater":** a cron job stepping up the percentage with no automated analysis or rollback — a staged rollout that never produces a verdict.

Real-world examples

- **Netflix Kayenta** (with Spinnaker) — automated concurrent canary-vs-baseline scoring across metric sets.
- **Argo Rollouts and Flagger** — Kubernetes rollout stages, analysis runs and automatic rollback, with the mesh splitting traffic.
- **Google's percent rollouts** — the same idea at huge scale.
- **For AI engineers:** a *model* canary routes 1% of live requests to the new model and compares, over the same period, answer-quality evals (automated scoring of answers), negative feedback, refusal rate, cost per request and per-token latency; a mesh or LLM gateway splits, automated evals decide.

In the interview

Any "risky release" question expects a canary at the center; follow-ups probe gate metrics, the concurrent baseline, stickiness and statistical limits.

30-second answer: "Release to a sticky 1% cohort hashed by user, so nobody switches version mid-flow. Compare against the concurrent baseline, never yesterday — errors, latency and business metrics like conversion, since the worst bugs throw no errors. Automated analysis gates each step, 1-5-25-100, and a failing score drains the canary in seconds, so the blast radius is the current percentage. Permanent requirement: versions coexist during every rollout, so schemas and APIs stay mixed-version compatible — expand-contract always. Limit: small slices can't see very rare effects, so canaries complement flags and tests."

Likely follow-ups (with answers):

- *"Why baseline instead of a threshold?"* — Errors and latency swing with time of day and traffic mix; only the concurrent 99% shares current conditions.
- *"How long at 1%?"* — Until an effect of the size you care about would have appeared at that cohort's volume: minutes for big failures at high traffic, hours (or a bigger slice) for subtle ones — cohort size × effect frequency.

Mistake that fails candidates: historical comparisons, manual dashboard watching, non-sticky routing, canaries on versions that can't coexist.

Self-check:

- **Q1.** The split, stickiness, verdict basis and gate-failure behavior. → LB or mesh weights send 1%; user-ID hashing keeps cohorts stable; golden signals, business metrics and trace shapes compared with the concurrent baseline; a failed gate drains automatically, exposing only the current slice.
- **Q2.** What can a canary catch that blue-green can't, and vice versa? → Canary: behavior bugs only real traffic reveals (locale combinations, conversion drops). Blue-green: an atomic one-version switch, needed when versions can't coexist. Green plus staged traffic gives both.
- **Q3.** v1.45 clears every gate to 100%; two days later screen-reader users (~0.4% of traffic) can't check out. Why missed, and what catches it? → In a 1% canary they're 0.4% × 1% = **0.004%** of traffic — too few sessions; no server error, and the overall conversion shift is below noise. Catch it with CI accessibility tests (deterministic), a flag-targeted beta deliberately including that cohort, and conversion monitoring broken out for assistive-technology users.

Related patterns:

- **Blue-Green Deployment** — the environmental half; combine as "stand up green, then shift traffic canary-style".
- **Feature Flags** — exposure by *chosen* cohort rather than statistics.
- **Service Mesh** — the traffic split as one line of config.
- **Distributed Tracing** — raw material for the verdict.
- **Consistent Hashing** — the ring behind cohort stickiness.

TL;DR

- **Problem:** environment checks prove a version runs, not that it behaves; real traffic is the only complete test suite, and blue-green runs it on 100% at once.
- **Mechanism:** a sticky hashed 1% cohort; verdicts from live canary-vs-baseline comparison of golden signals and business metrics; automated 1-5-25-100 gates draining on failure; permanent mixed-version compatibility.
- **Costs:** hours per rollout, permanent expand-contract, statistical and telemetry rigor, rare cohorts too small to detect.
- **Skip it when:** traffic is too low, versions can't coexist, or it's an emergency — and never run canary theater.

Feature Flags

Original lesson: [Feature Flags on DesignGurus ↗]

What it is. A switch in running code — `if flags.enabled("wishlist-sharing", user)` — whose rules change *without a deploy*. Code ships "dark"; you **release** by editing the rule (all at once, by percentage, or for chosen groups) and **kill** by flipping it back — seconds, affecting only that feature.

> **Analogy.** A new wing wired during construction with its switches off: on opening day lights go on for staff, then everyone; if a circuit sparks, you cut only that wing.

The problem: deployment is doing release's job

Three teams share one pipeline, and marketing has promised a launch date:

- **Timing.** *Wishlist-sharing* must go live **Tuesday 9:00 AM** on web and both mobile apps, with press and influencer posts scheduled. It's risky, so it should canary — over hours — and app-store review takes days, so mobile code ships early. Deploy time and release time can't be *the same* moment.
- **Integration.** The checkout refactor and search fixes ride the same **release trains** (scheduled deploys bundling every team's changes). Half-built wishlist code can't merge (users would see it), so it lives on a **six-week feature branch** while main changes daily; the final merge takes **two days** of conflict resolution.
- **Reversal.** At **11 AM Tuesday**, at peak, wishlist swamps the database with an **N+1 query** the low-volume canary never surfaced. The only off switch is a rollback that also removes the checkout refactor and search fixes. **Killing one feature kills three.**

Timing, audience and reversal all hinge on which binary is running. Fact 1 says they can be separated; flags finish the job.

Why the obvious fixes fail

- **"Keep half-finished work on branches."** Long-lived branches drift and produce huge merges. **Trunk-based development** (everyone merges to main daily) prevents that — but then unfinished code reaches production and must be deployed dark.
- **"Just deploy at 9 AM on Tuesday."** Turns deploys back into coordinated events, and can't work cross-platform: the iOS build entered app-store review the Thursday before.
- **"Roll back to kill the feature."** Undoes the whole release train. You need an off switch per *feature*, not per *binary*.

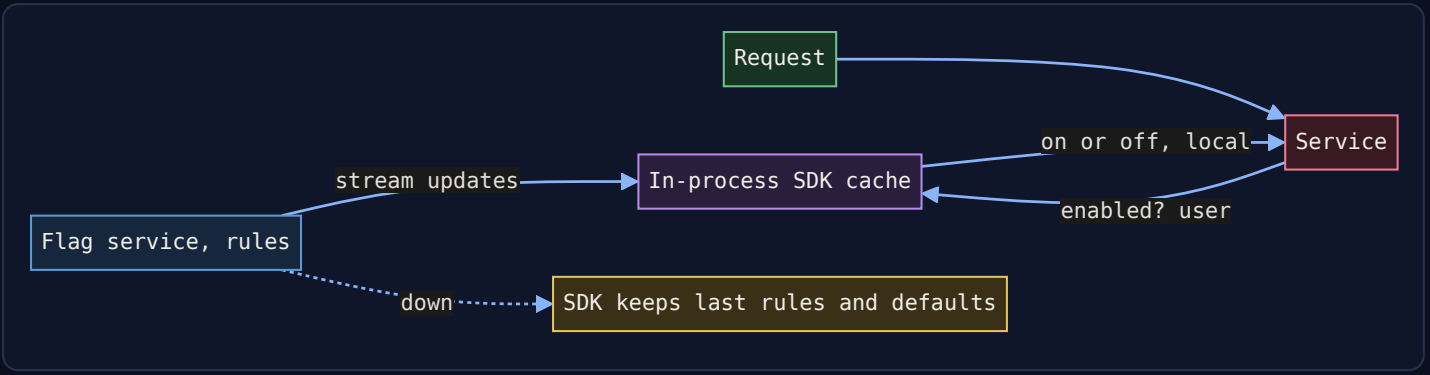
How it works

1. **Evaluate locally and cheaply.** Flag checks sit on **hot paths** (code running on every request), so the **SDK** in each service evaluates *cached* rules in-process in microseconds, no network call per check. Rule changes **stream** to instances over long-lived connections, like a mesh pushing config (xDS).
2. **Fail static.** If the flag service goes down, instances keep the **last known rules**, and every flag has a **safe default**. Users notice nothing; operators just can't edit rules. A flag-service outage must never switch features off or take the site down.
3. **Targeting gives each release a defined audience** — employees → opted-in beta → 1% → 10% → 100%. A **cohort** is a named user group; naming lets you include rare groups deliberately, like screen-reader users lost in the canary's 1% sample. Division of labor: a canary asks "*is the binary safe?*"; flags ask "*who should get this feature, and when?*"
4. **Know which type of flag you're creating — each has its own lifecycle:**

Type	Purpose	Lifecycle
Release flag	Ship dark, ramp up, launch	Temporary (weeks); has to be removed once at 100%
Ops kill-switch	Switch off costly or risky behavior when the system is under stress	Permanent, documented and tested on a regular schedule
Experiment flag	A/B test: which variant wins?	Lasts for the experiment's statistical window, then gets removed
Permission flag	Entitlements: plan tiers, enterprise features	Permanent — really product configuration rather than a flag

5. **Flag debt is the biggest danger.** Every forgotten release flag leaves a dead, untested, unowned `else` branch. **Knight Capital** is the industry's cautionary tale: a deployment reused an old flag name; the dead code tied to it still existed on **one server that missed the rollout**; the revived code traded out of control and lost **$460 million in 45 minutes**. Prevention: an **owner and expiry date** at creation; cleanup as scheduled work — 100% doesn't finish a flag, *deleting the* `if` does; **never reuse a flag name**.
6. **The limit:** flags switch **code paths, not data shapes**. Schema changes still need expand-contract; a flag in front of a destructive migration doesn't make it reversible.

Flag evaluation architecture with fail-static



The service asks its *local* SDK cache; the flag service only streams rule updates into it, and if it goes down the SDK keeps the last rules and defaults.

Walkthrough with numbers: wishlist with flags

- **Weeks before launch:** small pieces merge to main daily behind the dark `wishlist-sharing` flag, spreading integration effort thin; mobile builds clear app-store review with the feature off.
- **Wednesday:** on for **500 employees**.
- **Thursday:** on for the opted-in beta, deliberately including the accessibility cohort — a screen-reader bug affects dozens of beta users instead of generating thousands of tickets.
- **Tuesday 9:00:** exposure reaches 100% over **ten minutes** on web, iOS and Android simultaneously — a rule change, since every binary has carried the code for weeks.
- **11:04 AM:** the N+1 bug hits at peak; an operator flips the kill-switch within **30 seconds**. Wishlist vanishes, checkout and search keep running, the DB recovers. Fix deployed Wednesday, re-ramped Thursday, flag deleted the next sprint.
- **Side by side:** without flags — an impossible cross-platform 9 AM deploy, an 11 AM rollback pulling three features, a six-week merge. With flags — three rule updates and one deleted `if`.

Trade-offs

You gain: deploy and release fully separated (marketing sets timing, cohorts set audience, one rule change reaches every platform); a per-feature kill switch; trunk-based development with dark integration; the most precise control in progressive delivery, complementing canaries.

You pay:

- **Flag debt** — dead branches edging toward another Knight Capital; defend with owners, expiry dates, "done means deleted".
- **State combinations** — N flags yield **2^N** code paths; keep N small; test the combinations actually live in production plus each default.
- **A new critical dependency** — local evaluation and fail-static soften it, but a *bad rule* spreads at config speed; govern rule changes like deployments (the mesh lesson's warning).
- **Runtime indirection** — each check is a branch (microseconds on hot paths), and knowing which code served a request requires its flag state; record evaluated flags as trace attributes.

The progressive delivery stack

Progressive delivery releases in measured stages, observing each before moving on. The three patterns stack:

Layer	Question it answers	Mechanism	Undo
Blue-Green	Is the environment healthy?	Switch from one environment to the other	Switch back: seconds
Canary	Is the binary safe with real traffic?	A statistical slice with measured verdicts	Drain: seconds
Flags	Should *this user* see *this feature* right now?	Rules at runtime, named cohorts	Flip: instant

Mature pipelines create green, canary traffic onto it, then switch features on inside it on their own timetables. In one line: **canaries select traffic statistically; flags select people by name** — so flags can include rare cohorts a sample would miss.

When not to use it

- **Every change** — flagging trivial changes "to be safe" builds debt without protection; canaries cover them.
- **As authorization** — a flag controls *visibility*; **AuthZ** controls *access*. A flag SDK isn't a permission system.
- **Long-running business logic branches** — plan tiers and regional rules are product configuration (the "permission" category), not release flags.
- **Around schema changes** — expand-contract governs data.
- **The classic misuse: the flag nobody deleted** — 10,000 three-year-old flags with departed owners and unknown defaults; the code no longer tells anyone what production does. Keep the count small.

Real-world examples

- **LaunchDarkly, Unleash, OpenFeature** — streamed rules, local evaluation, targeting; OpenFeature is the vendor-neutral flag standard (as OpenTelemetry is for tracing).
- **Facebook's Gatekeeper** — dark launches widened cohort by cohort to billions, before commercial flag products existed.
- **GitHub ships dark** — features can sit in production for months, staff-only, before customers see them.
- **Knight Capital** — the $460M lesson: reused name + dead code + an incomplete deployment, within 45 minutes.

- **For AI engineers:** flags are becoming LLM products' operating controls — prompt versions changed via config, new models to named cohorts before canary verdicts, kill-switches for provider-failure fallbacks, disabling costly features like AI summaries at peak, limiting agent tools to approved cohorts. Module 5's degradation sequence says *what* to shed; flags give on-call engineers the switches.

In the interview

Prompts: "launch at a precise time on web and mobile", "pull one bad feature without rolling back others", "merge unfinished work continuously". Flags solve all three; follow-ups probe flag debt and flag-service outages.

30-second answer: "Flags separate deploy from release entirely: code ships dark with every trunk-based release, and releasing is a runtime rule change — instant for marketing's 9 AM, by percentage, or to named cohorts like employees and beta users, catching what canary statistics miss. Killing is a flip: one feature, thirty seconds, nothing else rolled back. Three disciplines: local cached evaluation with fail-static defaults so the flag service can't take the site down; a taxonomy where release flags are deleted after launch and kill-switches are permanent and tested; and an owner plus expiry on every flag, because dead flags cost Knight Capital $460 million in 45 minutes."

Likely follow-ups (with answers):

- *"The flag service goes down?"* — Nothing user-visible: SDKs evaluate cached rules with declared defaults (fail static, like the mesh); operators can't change rules for a while. Never a network call per evaluation.
- *"How do flags not pile up forever?"* — A lifecycle policy: owner and expiry at creation, deletion in the definition of done, automatic expiry reminders, **lint rules** blocking name reuse, scheduled cleanups, and monitoring flag count and age.

Mistake that fails candidates: flags as authorization, per-evaluation network calls, no deletion plan, claiming a flag makes a destructive migration safe.

Self-check:

- **Q1.** The four flag types, their lifetimes, and the practice preventing Knight Capital failures. → Release (delete at 100%), ops kill-switch (permanent, documented, tested), experiment (statistical window), permission (permanent product config). Prevention: deletion as the definition of done — remove the `if` and its dead branch — plus no name reuse.
- **Q2.** Place the three delivery patterns on the deploy/release split and fit them together. → Blue-green separates by seconds ("is the environment healthy?"); canary by percentage ("is the binary safe?"); flags fully — time, audience, platform ("should this user get it now?"). Pipeline: create green → canary traffic onto it → features stay dark until rules release them to cohorts.
- **Q3.** A new discount engine uses flag `new-pricing`, last used two years ago by a dead experiment whose `else` branch was never fully removed. The deploy lands on 19 of 20 servers (one fails silently), then the flag is turned on. → 19 servers run the new engine; the stale one runs old code where `new-pricing=on` revives the half-deleted experiment, and the load balancer sends it ~5% of pricing requests → intermittent, inconsistent, possibly expensive prices — Knight Capital's recipe. Rules: (1) never reuse names (a registry remembers retired ones); (2) dead branches die with their flags; (3) flips wait for deploy convergence (block or warn while instances report different versions). Flags need code review *and* config governance.

Related patterns:

- **Canary Deployment** — canaries for binaries, flags for features.
- **Graceful Degradation** — kill-switches are the manual levers of Module 5's degradation sequence.
- **Circuit Breaker** — the kill-switch's automatic twin: breakers react to measurements, switches to people; production needs both.

TL;DR

- **Problem:** deploys do release's job — launch timing chained to release trains, half-done work on drifting branches, killing one feature rolls back everyone's.
- **Mechanism:** runtime switches evaluated locally against streamed, cached rules with fail-static defaults: deploy dark; release by flip, percentage or named cohort; kill any feature in seconds; a four-type taxonomy and "done means deleted".
- **Costs:** flag debt (Knight Capital), 2^N state combinations, a config surface whose mistakes reach everyone, code that no longer fully describes production.
- **Skip it when:** the change has no release risk (canary suffices), the "flag" is really auth or product configuration, or the risk is in the data layer (expand-contract).

Containers and Kubernetes

Added lesson (not part of the DesignGurus course). Written from primary sources listed under "Sources" at the end of the lesson.

What it is. A **container** is an ordinary process the Linux kernel isolates and limits: it sees its own filesystem, network and process list and may use only the CPU and memory it was given. It starts from an **image** — a packaged, read-only filesystem with the app and everything it needs. **Kubernetes** runs containers across a fleet and keeps them in the state you declared ("20 copies of checkout, version 4.2, each with 1 CPU and 1 GiB (gibibyte, ≈1.07 billion bytes), reachable as `checkout`"); when reality drifts (a machine dies, a process crashes) it notices and corrects it. (Basics: Module 0 — Virtual machines, containers and Docker, Kubernetes basics.)

Analogy. Shipping containers let any crane, ship or truck move any cargo unseen; Kubernetes is the port authority whose cranes keep making the port match your manifest.

The problem: 30 services, 440 hand-kept servers

~30 services run as **440 instances on 440 virtual machines** (VMs — software-emulated computers with their own OS sharing a physical host). One month:

- **"It works on my machine."** A service passed every test on build servers with one TLS library version and crashed in production with another; every VM is a snowflake built by years of scripts.
- **Waste.** One VM per instance "for isolation"; average fleet CPU **15%**.
- **Slow change.** Baking a VM image takes **40 minutes**, booting minutes more; every team wrote its own deploy script.
- **3 AM restarts.** A dead process pages a human — or stays dead until noticed.

The unit of deployment is a *machine* set up by imperative scripts; it should be the *application*, described once and placed wherever there's room.

Why the obvious fixes fail

- **"Better configuration-management scripts"** (Ansible, Chef, Puppet) reduce drift, but dependencies are still installed *onto* machines rather than travelling *with* the app.
- **"Several services per VM"** — without isolation they fight over library versions and ports, and one memory leak sinks its neighbours (**noisy neighbour**).

- **"Smaller VMs"** — still an OS per instance, still minutes to boot.

How it works

Part 1 — what a container really is. Three Linux features:

1. **Namespaces — what the process can see.** PID (its own process list; it thinks it's process 1), network (own interfaces and ports), mount (own filesystem view), plus hostname, users and IPC.
2. **Control groups (cgroups) — what it may use.** Limits and accounting for CPU, memory and I/O. Over the memory limit → killed (**OOMKilled**); over the CPU limit → **throttled** (paused until the next scheduling period).
3. **Layered images.** A stack of read-only layers (base OS files, runtime such as Java or Python, libraries, app) plus metadata like the start command; a running container adds a thin writable layer (**copy-on-write**). Layers are shared and cached — 20 services on one Java base store it once per machine. Images follow the **OCI** (Open Container Initiative) format, runnable by any compliant runtime (containerd, CRI-O).

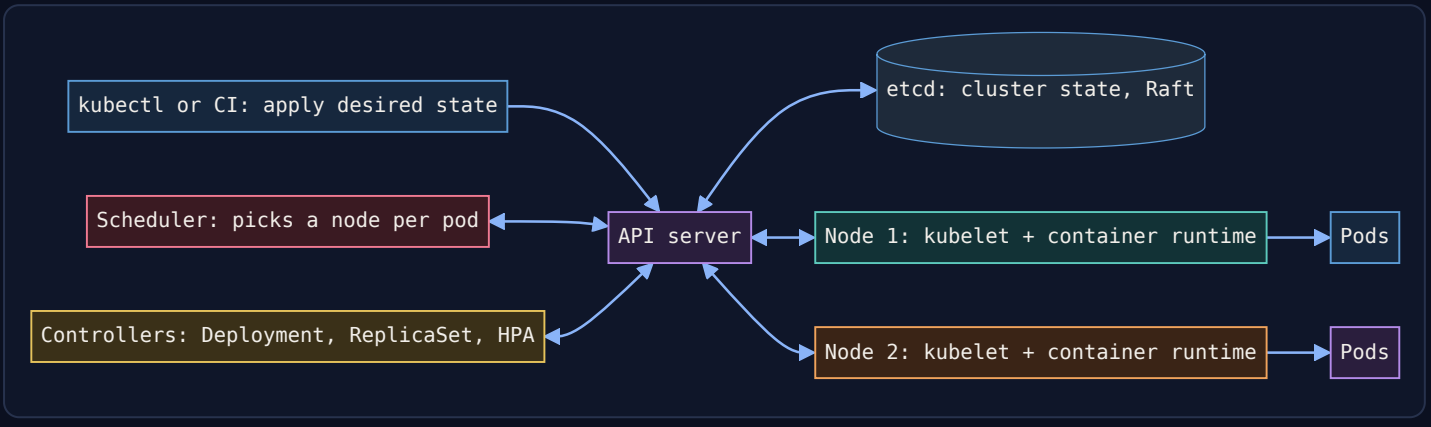
Sharing the host kernel, containers start in well under a second with little memory overhead — far lighter than a VM booting its own kernel — but a kernel bug is shared, so isolation is weaker (why serverless platforms run tenants in microVMs; see the Scaling module's multi-region lesson).

Image practices that matter:

- **Multi-stage builds** — compile in a large builder image, copy only the result into a small runtime image: smaller attack surface, faster pulls.
- **Order layers for the cache** — dependencies first, source last, so code changes rebuild only the top layer.
- **Build once, deploy by digest** — the **digest** is the content hash; tags like `latest` move, digests don't, so the tested image is the deployed image.
- **No secrets in images** (anyone pulling can read every layer); run **non-root**; one main process per container; logs to standard output.
- The Java side (layered jars, buildpacks) is in the Spring book's Faster startup and container images.

Part 2 — Kubernetes: desired state and control loops. You never say "start a container on host 12"; you submit a **desired state**, and **controllers** — control loops like Auto-Scaling's — keep comparing it with actual state and closing the gap.

Kubernetes control plane and nodes



Everything goes through the **API server**, which stores cluster state in **etcd** (a small key-value store kept consistent with Raft). The **scheduler** assigns new pods to nodes with free resources; **controllers** watch for desired/actual differences; each node's **kubelet** starts and stops containers via the runtime and reports back.

The objects you use every day:

Object	What it is	Why it exists
Pod	One or more containers scheduled together, sharing a network address and volumes	The unit of scheduling; usually one app container, sometimes plus helpers
Deployment	"Keep N identical pods of this template running; roll out changes gradually"	Stateless services — the horizontal-scaling fleet
Service	A stable name and virtual IP in front of the pods matching a label	Pods come and go with new IPs; callers use the Service name (built-in discovery)
Ingress / Gateway API	HTTP routing from outside the cluster to Services	The cluster's L7 front door (behind your edge and API gateway)
ConfigMap / Secret	Configuration and credentials injected as files or environment variables (named settings a process reads at startup)	Same image in every environment; config varies
StatefulSet	Pods with stable names and their own persistent volumes (disk storage that survives pod restarts)	Databases, brokers — things that are not interchangeable
DaemonSet	One pod on every node	Log shippers, monitoring agents
Job / CronJob	Run to completion, once or on a schedule	Batch work, the nightly report
HorizontalPodAutoscaler	Changes a Deployment's replica count from a metric	The Auto-Scaling control loop, built in

Service types: **ClusterIP** (in-cluster only, the default), **NodePort** (a port on every node), **LoadBalancer** (requests a cloud load balancer), **headless** (no virtual IP; DNS returns each pod, used by StatefulSets). Why a Kubernetes Service usually replaces a separate discovery server: Service discovery on Kubernetes.

Part 3 — the settings that decide whether it works in production.

- **Requests and limits.** A **request** is what the scheduler reserves ("0.5 CPU, 1 GiB"); a **limit** is the kernel-enforced ceiling. Memory over limit → OOMKilled; CPU at limit → throttled, appearing as mysterious latency spikes. Many teams set memory limit = request and leave CPU unlimited or generous.
- **Probes. Readiness** controls traffic, **liveness** controls restarts, **startup** gives slow starters time before liveness begins — the Health Check Endpoint lesson's verdicts, with the same rule: liveness must never depend on a shared downstream. Spring Boot's mapping: Kubernetes probes.
- **Graceful shutdown.** On rollout or scale-down Kubernetes sends **SIGTERM** (the OS's polite shutdown signal), removes the pod from the Service's endpoints, and waits up to `terminationGracePeriodSeconds` (default 30 s) before killing it. Endpoint removal and SIGTERM happen almost together, so the app should keep serving a few seconds (a short `preStop` sleep), then drain in-flight work — per-pod load-balancer draining. See Graceful shutdown in Kubernetes.
- **Rolling updates.** A Deployment replaces pods in waves bounded by **maxSurge** (extra pods allowed) and **maxUnavailable** (pods allowed missing); new pods count only once ready, so a broken version stalls the rollout instead of replacing everything. (Blue-green and canary on Kubernetes: next lesson.)
- **PodDisruptionBudget** — "never take more than 2 of my 20 pods down at once for maintenance", honored when nodes drain for upgrades.
- **Spread across failure domains** — topology spread constraints place replicas across nodes and zones, so a zone loss removes a third of the pods, not all.

Part 4 — design patterns for containers. Pods can hold several containers sharing a network address and disk, so helper jobs become reusable containers:

- **Sidecar** — a helper beside the app (proxy, log shipper, certificate refresher); the API module's Sidecar lesson.
- **Ambassador** — a local proxy representing a remote service (the app talks to `localhost`; the ambassador handles sharding, retries or TLS).

- **Adapter** — converts the app's output to a standard form (e.g., custom metrics in the monitoring system's format).
- **Init container** — runs to completion *before* the app starts (wait for a dependency, fetch config, run a check).
- Across pods: **leader election** (one replica runs the scheduled job — the horizontal-scaling checklist's distributed lock), **work queue** (a Job's pods pull tasks), **scatter-gather**.

Walkthrough with numbers: the 440 instances, containerised

- **Packing.** **~0.5 CPU and 1 GiB** per instance: 440 × 0.5 = **220 CPUs**. A 16-CPU node has ~14 allocatable → 220 / 14 ≈ **16 nodes**; add a 25% rollout surge and one zone of failure (zone-level N+1) → **~24 nodes** instead of 440 VMs. Utilisation ~15% → ~60%.
- **Deploy time.** Cached image build **~3 minutes** (only the app layer changes); a rolling update of 20 replicas with maxSurge 5, maxUnavailable 0 runs 4 readiness-gated waves (~30 s each) → **~2–3 minutes**, no capacity loss.
- **The 3 AM crash.** The kubelet restarts a crashed container in seconds. A dead node is marked not-ready after ~40–50 s, and its pods are evicted and rescheduled once a default **5-minute** toleration expires (tunable per pod — shorten for stateless services). Nobody is paged unless restarts repeat (**CrashLoopBackOff** alerts).
- **The OOM trap.** A Java service with a 1 GiB limit is OOMKilled every few hours: heap was 1 GiB, but the JVM also needs off-heap memory (threads, metaspace, buffers). Fix: size the heap as a percentage of the container limit (e.g., `-XX:MaxRAMPercentage=75`; modern JVMs read cgroup limits) and alert on restarts.

Trade-offs

You gain: identical artifacts from laptop to production; density (~440 VMs → ~24 nodes); self-healing, declarative operations via control loops; one platform vocabulary for 30 teams.

You pay:

- **A complex platform to run** — upgrades every few months, networking plugins, certificates, etcd backups; managed offerings (EKS, GKE, AKS) remove some, not all.
- **New ways to be wrong** — missing requests (overpacked nodes), tight CPU limits (throttling), deep liveness probes (restart storms), missing PodDisruptionBudgets (maintenance outages).
- **Weaker isolation than VMs** — untrusted multi-tenant code needs sandboxing (gVisor, microVMs) or separate nodes.
- **Harder stateful workloads** — StatefulSets, persistent volumes and usually an **operator** (a controller encoding a DBA's procedures); many teams keep databases on managed services.

When not to use it

- **A few services and a small team** — a managed container service (AWS ECS or Fargate, Google Cloud Run, Azure Container Apps), a PaaS, or serverless functions give most of the benefit without a cluster.
- **"Kubernetes because everyone uses it."** Anti-patterns: lifting fragile stateful "pet" servers into containers unchanged; a microservice per function so each request crosses ten hops (a *distributed monolith*); a cluster for a static site.
- **Workloads needing the whole machine** — very large databases or hardware-tuned latency-critical systems often run better on dedicated VMs.

Real-world examples

- **Google Borg** ran nearly all of Google's work in containers for a decade; Kubernetes, designed partly by the same engineers, applies its lessons (labels, pods, declarative state) per "Borg, Omega, and Kubernetes" (2016).
- **Managed Kubernetes**: Amazon EKS, Google GKE, Azure AKS, usually on **containerd**.

- **The Twelve-Factor App** — config in the environment, logs as event streams, disposable processes: the habits of a container-friendly app.
- **For AI engineers:** GPUs are scheduled as an extended resource (`nvidia.com/gpu: 1`); model weights are too big to bake into images comfortably (pull from object storage or a cache volume at start); slow model loading makes the **startup probe** and warm pools essential — the GPU Auto-Scaling boot-time problem on Kubernetes.

In the interview

Prompts: "how would you deploy these services?", "what happens when a pod dies?", "requests vs. limits?", "why is my pod OOMKilled?" Interviewers want the model (desired state, control loops), not YAML fields.

30-second answer: "A container is a process isolated with namespaces and limited with cgroups, started from a layered immutable image — the artifact we test is the one we run. Kubernetes keeps declared state true: a Deployment keeps N pods running and rolls changes in readiness-gated waves, a Service gives them a stable name, the HPA adjusts N, controllers reschedule when nodes die. Production details: requests and limits — memory over limit is OOMKilled, CPU over limit throttled; readiness vs. liveness with liveness shallow; graceful SIGTERM with a short pre-stop delay; replicas spread across zones. For a handful of services I'd consider a managed container service before running a cluster."

Likely follow-ups (with answers):

- *"Container vs VM?"* — A VM runs its own kernel (strong isolation, slow boot, GBs of overhead); a container shares the host kernel (sub-second start, little overhead, weaker isolation). MicroVMs (Firecracker) sit between.
- *"A pod restarts every few minutes with no error in the app log."* — `kubectl describe pod` for the last state: OOMKilled (raise the limit or cap the heap), liveness failures (probe too strict or checking a dependency), or crash-on-start exit codes (CrashLoopBackOff).

Mistake that fails candidates: Kubernetes as a deploy script rather than a reconciliation system; liveness probes calling the database; no resource requests; "lightweight VMs" without mentioning the shared kernel.

Self-check:

- **Q1.** Which kernel features make a container, and what does each do? → Namespaces (what it sees), cgroups (what it may use, with OOM kills and throttling), and a layered copy-on-write image as its filesystem.
- **Q2.** 20 replicas, maxSurge 5, maxUnavailable 0, and the new version's readiness never passes. → 5 new pods start, none become ready, no old pods are removed: the rollout stalls with all 20 old pods serving and reports no progress. Roll back or fix — readiness-gated rollouts turn a bad version into a stuck deploy, not an outage.
- **Q3.** After a database blip all 60 checkout pods restart at once; the site is down 4 minutes. → Liveness checked the database, so every pod failed and restarted together, cold. Liveness tests only the process; dependency status belongs in readiness (or degraded responses).

Related patterns:

- **Health Check Endpoint** — readiness and liveness probes.
- **Auto-Scaling** — the HPA and cluster autoscaler as control loops on pods and nodes.
- **Sidecar / Service Mesh** — pod-level helpers and the mesh configuring them.

- **Blue-Green and Canary Deployment** — built from Deployments, Services and traffic weights (next lesson).

TL;DR

- **Problem:** machines as the deployment unit — snowflake servers, 15% utilisation, 40-minute image bakes, humans restarting crashed processes.
- **Mechanism:** containers (namespaces + cgroups + layered immutable images) as the artifact; Kubernetes control loops keep declared state true — Deployments, Services, probes, rolling updates, autoscaling, rescheduling.
- **Costs:** a platform to operate, new misconfiguration traps (limits, probes, disruption budgets), shared-kernel isolation, harder stateful workloads.
- **Skip it when:** a handful of services fits a managed container service or serverless, or the workload needs a whole tuned machine.

Sources: Linux man pages `namespaces(7)` and `cgroups(7)`; Open Container Initiative image and runtime specifications; Kubernetes documentation (concepts: Pods, Deployments, Services, probes, resource management, disruptions); A. Verma et al., "Large-scale cluster management at Google with Borg", EuroSys 2015; B. Burns, B. Grant, D. Oppenheimer, E. Brewer, J. Wilkes, "Borg, Omega, and Kubernetes", *ACM Queue* 2016; B. Burns and D. Oppenheimer, "Design Patterns for Container-based Distributed Systems", HotCloud 2016; Docker documentation, "Building best practices"; A. Wiggins, *The Twelve-Factor App*.

CI/CD Pipelines and Deployment Strategies

Added lesson (not part of the DesignGurus course). Written from primary sources listed under "Sources" at the end of the lesson.

What it is. **Continuous integration (CI)**: everyone merges small changes into main at least daily, and every merge is automatically built and tested. **Continuous delivery (CD)**: every passing change *could* go to production at the push of a button; **continuous deployment** sends it automatically. The **pipeline** is the automated, code-defined path from commit to production; a **deployment strategy** is how the new version replaces the old — recreate, rolling, blue-green, canary or shadow. This lesson ties the module's delivery patterns into one path. (Basics: Module 0 — CI/CD, Infrastructure as code.)

Analogy. An assembly line: each station does one check, a failing car never reaches the next, every car is built the same way, and the final road test on a quiet road before joining traffic is the canary.

The problem: the monthly release

40 platform engineers ship once a month:

- **Big batches. ~400 commits** per release; finding the guilty change means bisecting (testing halfway points) 400 commits under incident pressure.
- **Merge week.** Long-lived branches take **two days** to merge; bugs appear in the merge itself.
- **Different builds per environment.** Staging and production build separately from the same tag, resolving dependencies on different days — "passed in staging" proves little.
- **Hand-made infrastructure.** Load balancers and databases clicked together in the console; nobody knows production's configuration, and staging doesn't match.
- **Slow fixes.** A one-line hotfix takes **6 hours**: manual approvals, a manual QA pass, a manual deploy runbook.

Each safeguard (more QA, fewer releases) made batches *bigger* and releases riskier.

Why the obvious fixes fail

- **"Release less often to be safer."** More changes per release, harder diagnosis, longer rollbacks — risk grows with batch size.
- **"More manual QA."** Slows every change, still misses what only real traffic reveals, and doesn't scale to 120 merges a day.
- **"A code freeze before big events."** Fine for a few days around Black Friday; as a habit it just moves the batch problem past the freeze.

How it works

1. The pipeline — build once, promote the same artifact.

A pipeline from commit to production, one artifact all the way

- **Commit stage:** compile, unit tests, linters, static security analysis, dependency vulnerability scans — **under ~10 minutes**, so engineers wait instead of context-switching.
- **Package once:** an immutable artifact — a container image by **digest** — plus a **software bill of materials** (SBOM, every component inside) and a signature. Every later stage deploys *this exact artifact*; environments differ only in configuration (Twelve-Factor's "build, release, run").
- **Test environments:** integration tests against real dependencies, and **contract tests** checking each service still honours its consumers' API (the Spring book's contract testing section).
- **Production rollout:** a progressive strategy with automatic golden-signal verdicts and post-deploy checks; a failed verdict rolls back without a human.
- **Test pyramid:** many fast unit tests, fewer integration tests, very few slow end-to-end tests. **Flaky tests** (intermittently failing on unchanged code) are quarantined and fixed — an ignored pipeline protects nothing.

2. Choose the deployment strategy per change.

Strategy	How the new version replaces the old	Downtime	Rollback	Extra capacity	Users exposed to a bad version
Recreate	Stop all old, start all new	Yes	Redeploy old (minutes)	None	100%
Rolling	Replace instances in waves (Kubernetes default)	No	Roll back in waves (minutes)	A small surge	Grows wave by wave
Blue-green	Start a full new environment, switch traffic at once	No	Switch back (seconds)	2× during the switch	100%, briefly
Canary	Send a small slice of traffic to the new version, widen on good verdicts	No	Drain the slice (seconds)	Small	1–10% until trusted
Shadow (mirroring)	Copy real requests to the new version; discard its answers	No	Nothing to undo	Up to 2×	0% — users only see the old version
Feature flag	Code deployed dark; behaviour released by rule	No	Flip (instant)	None	Any chosen cohort

A/B testing answers a product question ("which variant converts better?"), not a safety one, and usually runs on experiment flags. Details: Blue-Green Deployment, Canary Deployment, Feature Flags. On Kubernetes, rolling is built in; blue-green is two Deployments with a Service switched between them; canary uses replica ratios or weighted routing in an ingress, gateway or mesh, with Argo Rollouts or Flagger automating verdicts.

Every strategy shares one hidden requirement: **old and new versions must work with the same database and message formats simultaneously**, since both run during any rollout — hence expand-contract (the

capstone). Shadow traffic must cause no side effects: a mirrored "place order" must never charge a card, so writes are stubbed or sandboxed.

3. Infrastructure as code (IaC). Servers, networks, load balancers and databases described in version-controlled files, reviewed like code, applied by the pipeline.

- **Terraform** (and similar) is *declarative*: describe desired resources; `plan` shows the diff from reality; `apply` makes it so. It tracks what it manages in a **state file**, stored remotely with locking so two runs can't collide.
- **Drift** (someone changed production by hand) appears as an unexpected diff in the next plan.
- Reviewing the *plan* in the pull request shows "this will delete a database", not just a code diff.

4. GitOps — the cluster pulls its desired state. Each environment's desired state (image digest, config) lives in a Git repository; an in-cluster agent (Argo CD, Flux) continuously reconciles the cluster to it. Deploying is a reviewed commit changing a digest; rollback is reverting it. CI never needs production credentials — the cluster pulls rather than being pushed to, removing a powerful secret from the pipeline.

5. Secrets in pipelines. Avoid long-lived cloud keys in CI: modern CI proves its identity to the cloud with short-lived **OIDC** (OpenID Connect, signed-token identity) tokens ("this workflow in this repository on this branch"), and apps read secrets from a secrets manager at runtime (see Storing keys and secrets).

6. Measure with the four DORA metrics. The *Accelerate* research (Forsgren, Humble, Kim) found four measures separating strong from weak delivery teams: **deployment frequency**, **lead time for changes** (commit to production), **change failure rate** (share of deploys causing failure) and **time to restore service**. Key finding: speed and stability rise *together* — small, frequent changes are easier to test, diagnose and undo.

Vocabulary interviewers mix in (one line each)

- **DevOps** — a culture where the team that builds a service runs it, with shared tooling and automation. **SRE** (Site Reliability Engineering) — Google's practice of running operations as software engineering, with SLOs (e.g., "99.9% of requests succeed") and error budgets (the failure the SLO still allows; once spent, releases slow) setting the shipping pace. **Platform engineering** — a team building the internal "paved road" (pipelines, clusters, templates) as a self-service product.
- **Monorepo vs polyrepo** — one repository for many services (atomic cross-service changes, one toolchain, needs build tooling that rebuilds only what changed) vs. one per service (clear ownership, independent pipelines, but coordinated releases for cross-service changes).
- **How Git stores history** — every file version (blob), directory (tree) and commit is stored under its content hash; a branch is a movable pointer to a commit. **Merge** joins histories with a merge commit, preserving them; **rebase** replays your commits onto the target for a straight history but rewrites commit IDs — never rebase commits others have pulled.

Walkthrough with numbers: the monthly release, replaced

- **Flow.** **~120 small changes a day** merge to main, each running an **8-minute** commit stage and a **12-minute** integration stage.
- **Production.** Small batches (often 1–3 changes per deploy); canary 1% → 10% → 50% → 100% with a **10-minute bake** per step and automatic verdicts → **~45 minutes** from merge to full traffic, no human for routine changes.
- **Hotfix.** Same path, shorter bakes: **~25 minutes** instead of 6 hours — trusted because always exercised.
- **A bad change.** Fails its verdict at 1%: about **1% of requests for ~10 minutes**, auto-rolled back, culprit one of ~2 commits instead of 400.
- **Infrastructure.** Importing the console-made load balancers into Terraform, the first `plan` shows **17 differences** between staging and production — the drift that made "passed in staging" meaningless.

Trade-offs

You gain: small, understandable, undoable batches with failures pointing at a few commits; one trusted path for features, fixes and infrastructure (the emergency path is the everyday path); matching environments from one artifact and one codebase; speed and stability together (DORA).

You pay:

- **Automation investment** — test suites, pipelines and rollout tooling are real software.
- **Discipline on every change** — trunk-based development, backward compatibility, expand-contract, flag hygiene.
- **Pipeline security** — it can change production, so it's a prime target: protect credentials, pin and verify dependencies, sign artifacts.
- **Flaky tests and slow stages** erode trust; they need owners and budgets.

When not to use it

- **Continuous *deployment* where releases can't be undone quickly** — mobile apps (store review, old versions in use), firmware, some regulated systems use continuous *delivery* (always releasable, human-approved) plus flags.
- **Shadow traffic for writes with real side effects**, unless stubbed.
- **Canary at tiny traffic** — a few requests per minute give no statistical verdict; use blue-green with smoke tests, or flags for named cohorts.

Real-world examples

- **Amazon** reported (Velocity 2011) a mean of 11.6 seconds between weekday production deployments in May 2011 — many small, automated, independent deployments.
- **Google's SRE book** describes release engineering with hermetic builds (same inputs, same output) and progressive rollouts.
- **Netflix's Spinnaker** with **Kayenta** automated canary analysis (built with Google) statistically compares canary and baseline and decides automatically.
- **Pipelines:** GitHub Actions, GitLab CI, Jenkins; **GitOps:** Argo CD, Flux; **IaC:** Terraform/OpenTofu, Pulumi, AWS CloudFormation.
- **For AI engineers:** model or prompt changes take the same pipeline with an extra gate — an offline **evaluation suite** (golden questions, LLM-judge scores) before the canary, whose verdict includes quality and cost, not only errors and latency.

In the interview

Prompts: "how does code get to production?", "zero-downtime deploy?", "how do you roll back a bad release?", "blue-green vs. canary?"

30-second answer: "Trunk-based CI on every merge: a sub-ten-minute commit stage, then one immutable signed image built once and promoted unchanged through test, staging and production — only configuration differs. Infrastructure is Terraform with plans reviewed in pull requests, reconciled GitOps-style so the cluster pulls its desired state. In production, canaries with automatic golden-signal verdicts and auto-rollback; blue-green where I need an instant full switch; flags to separate release from deploy. Every rollout assumes old and new run together, so schema changes are expand-contract. The four DORA metrics tell me whether it's working."

Likely follow-ups (with answers):

- *"Why build once instead of per environment?"* — So the tested thing is bit-for-bit the running thing; rebuilds can pull different dependencies or compiler output. Environment differences belong in configuration.

- *"Rollback is impossible because the new version wrote data in a new format."* — The change wasn't backward compatible: expand first (both formats readable), deploy, contract only once the old version is gone — the capstone's sequence.

Mistake that fails candidates: rebuilding per environment; `latest` tags in production; a strategy with no rollback story; ignoring database compatibility during rollouts; "we test in staging" as the only net.

Self-check:

- **Q1.** Define continuous integration, delivery and deployment. → CI: small changes merged to main at least daily, each automatically built and tested. Delivery: every passing change is releasable at a button. Deployment: every passing change goes to production automatically.
- **Q2.** Strategy for (a) a high-traffic payment service, (b) an internal tool at 20 requests an hour, (c) a new search ranking to compare without affecting users. → (a) Canary with automatic verdicts (plus flags for risky features). (b) Blue-green or rolling with smoke tests — a 1% canary sees almost nothing. (c) Shadow traffic compared offline, then canary or A/B test.
- **Q3.** Two engineers ran `terraform apply` simultaneously and a load balancer was deleted. What was missing, and how does GitOps change who can deploy? → Remote state with locking, and applies only by the pipeline from reviewed plans. With GitOps the in-cluster agent applies what's in Git, so the path to production is a reviewed commit, not someone's credentials.

Related patterns:

- **Blue-Green, Canary, Feature Flags** — the strategies this pipeline chooses among.
- **Health Check Endpoint** — readiness gates every rollout wave.
- **Distributed Tracing** — evidence behind a canary verdict when something is slow.
- **The capstone (next)** — expand-contract keeps rollback possible when data changes.

TL;DR

- **Problem:** monthly ~400-commit releases, per-environment builds, hand-made infrastructure, 6-hour hotfixes — each safeguard made batches bigger and riskier.
- **Mechanism:** trunk-based CI; one immutable signed artifact promoted through environments; IaC and GitOps reconciliation; a deployment strategy per change (rolling, blue-green, canary, shadow, flags) with automatic verdicts and rollback; DORA metrics.
- **Costs:** automation to build and maintain, backward-compatibility discipline, a pipeline to secure, flaky tests to police.
- **Skip it when:** continuous *deployment* can't be undone quickly (mobile, firmware, some regulated systems) — use continuous delivery plus flags.

Sources: J. Humble and D. Farley, *Continuous Delivery* (2010); N. Forsgren, J. Humble, G. Kim, *Accelerate* (2018) and the DORA *State of DevOps* reports; B. Beyer et al. (eds.), *Site Reliability Engineering*, O'Reilly 2016, ch. 8 "Release Engineering"; J. Jenkins, "Velocity Culture", O'Reilly Velocity 2011 (Amazon's deployment rate); Netflix Tech Blog and Google Cloud Blog, "Introducing Kayenta" (2018); HashiCorp Terraform documentation (plan/apply, state and locking); OpenGitOps principles (CNCF); Kubernetes documentation (Deployment strategies); *The Twelve-Factor App* (build, release, run); *Pro Git* (S. Chacon, B. Straub), ch. 10 "Git Internals".

Capstone - The Zero-Downtime Migration

Original lesson: [Capstone - The Zero-Downtime Migration on DesignGurus ↗]

What this capstone is. Changing how a busy production database stores money, with zero downtime and a safe way back at every stage. Migrations are often riskier than feature releases because they alter *stored data*, which no router can switch back. The module's two rules apply — keep deploy apart from release, keep rollback everyday — and where a stage seems impossible to roll back, you redesign the stage.

> **Analogy.** Moving a busy restaurant's kitchen without stopping service: build the new kitchen unused, cook every dish in both and taste side by side, gradually serve from the new one, and tear out the old one only weeks later.

At each decision, ask: **what is the undo button right now, and what does pressing it cost?**

The brief

- An audit finds orders store money as `total FLOAT` (binary fractions can't hold cents exactly); rounding drift has diverged totals from the payment provider's records. Finance wants **`total_cents` as an integer**.
- **400 writes/second**, nonstop; **no maintenance window**.
- **12 consumers** must keep working, including **CQRS projections** (query-shaped read copies) and reporting.
- Money: the new format needs **proof** before anything relies on it, and every stage stays reversible until then.
- **The database does not blue-green** — the whole project is careful expand-contract.

Stage 1 — Expand: the change that changes nothing

Decision: add `total_cents BIGINT NULL` — nullable, unwritten, unread; the application behaves exactly as before. **Undo:** drop the column, free, since no code refers to it.

Why the full delivery process anyway? Even this "trivial" migration goes through a canary, watched on readiness and golden signals: an `ALTER TABLE` can take unexpected **locks** (blocking reads or writes) or exceed a migration tool's timeout. Harmless-looking schema changes still cause 3 AM incidents. Acceptance: healthy signals, nobody notices.

Stage 2 — Dual-write, behind a flag, plus a backfill

Decision 1 — dual-write ships dark; a flag releases it. Code writing both `total` and `total_cents` deploys switched off and is canaried as a *binary*; a separate flag change turns the *behavior* on. Dual-writing might add latency, clash with a database trigger, or expose a peak-only N+1, so undo should be a **30-second** flip, not a rollback. Users can't see it, so no cohort ramp — the point is independent control.

Decision 2 — backfill 200 million historical rows with `total_cents = round(total × 100)`, made safe to run and rerun by Module 5's reliability patterns:

- **Idempotent**: each write sets a deterministic value per row, so failed batches just rerun.
- **Rate-limited as a bulkhead**: throttled and isolated so production requests win contention.
- **Checkpointed**: durable progress markers (the write-ahead-log habit) let it resume.

Undo: flag off, stop the job. Nothing is lost; `total` is still the source of truth.

Stage 3 — Verify: the stage everyone skips

Both columns are populated and code *could* read `total_cents` tomorrow. It doesn't, until there's proof across 200 million rows plus 400 writes/s:

- **A comparison job** continuously counts rows where `total_cents ≠ round(total × 100)`. It finds **several thousand** mismatches from an old code path that never dual-wrote (legacy paths linger). Fix the path, re-backfill, and wait for **zero, sustained for days**.
- **Shadow reads**: a separately flagged read path computes from *both* columns, returns the old value, and logs each mismatch with its **trace ID** for investigation — production as a never-ending test, the canary idea applied to data.

Undo: nothing to undo; users see no change. This stage produces the *evidence* the next one depends on; skipping it bases a money migration on an untested assumption.

Stage 4 — Switch reads with a flag ramp and verdicts

Decision: move reads from `total` to `total_cents` via a flag ramp — internal users → 1% → 10% → 100% — with canary-style verdicts on golden signals **plus business metrics**: order-total disputes, odd refund patterns, checkout conversion (customers may complain about a one-cent difference before any error rate moves).

Why is rollback instant and safe here, unlike blue-green's failure? Dual-writing continues, so **both columns stay current**; pointing reads back at `total` targets an up-to-date source. In the blue-green failure the old version couldn't read data written after the switch; here neither format falls behind, so the rollback window can stay open as long as verification needs. **Undo:** flip the flag — seconds.

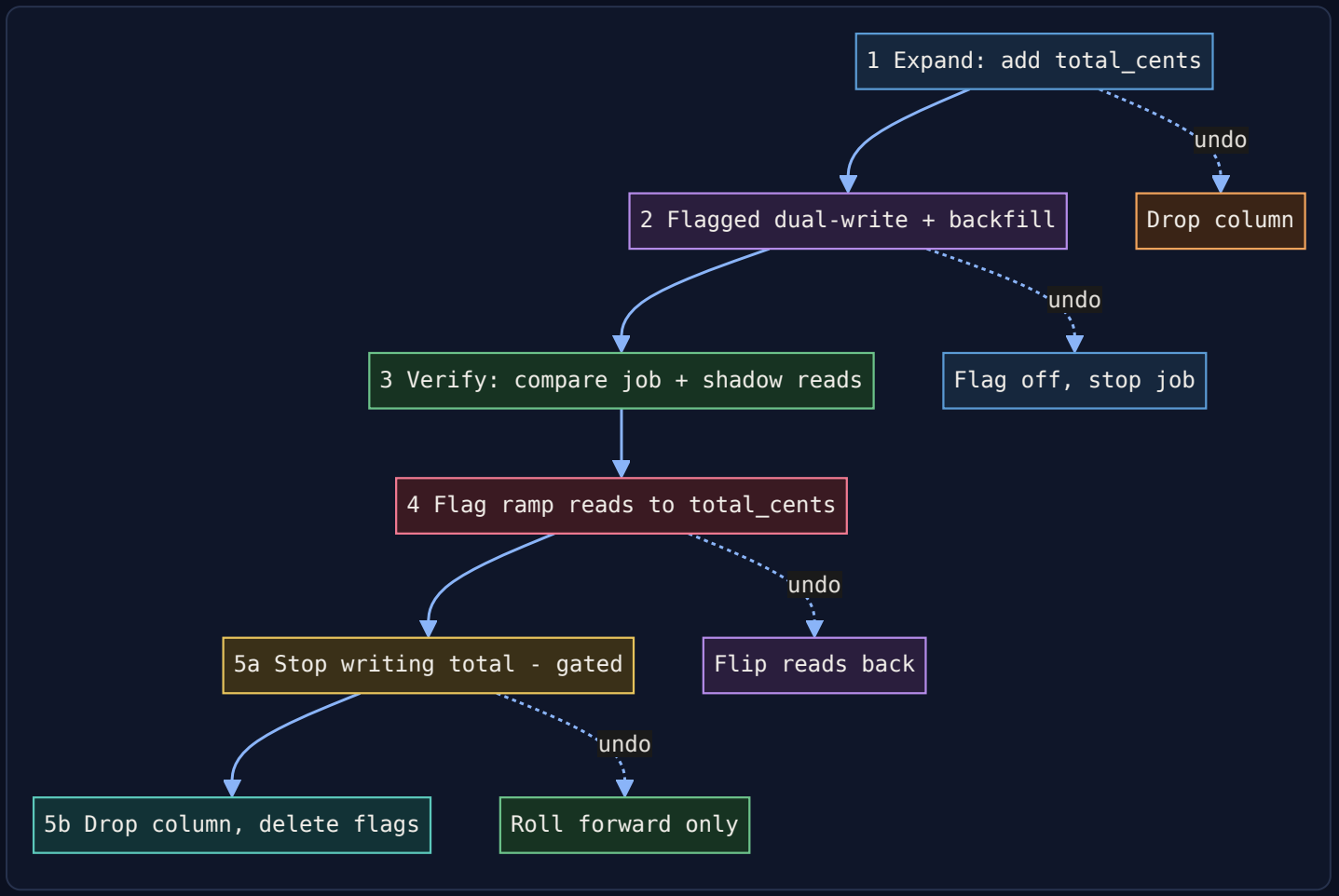
Stage 5 — Contract: the one-way door, entered deliberately

Reads have used `total_cents` for weeks. Two sub-steps, weeks apart:

- **5a. Stop writing `total`** (write flag → cents-only). `total` now goes stale and read rollback is no longer safe, so three **gates** must pass: zero mismatches **for weeks**; every consumer confirmed migrated (the tracing lesson's dependency map finds forgotten reports still reading `total`); written **sign-off from finance**. Afterward, recovery is **roll forward only** — losing reversibility is why the gates and waiting exist.
- **5b. Drop the column** after another safe interval and a documented **backup snapshot** of the old schema. Then delete both migration flags and their dead branches. **Dropping the column doesn't finish the migration; deleting the temporary two-column code does.**

The irreversible step comes last, backed by all the evidence, by conscious decision — the saga lesson's **pivot** (point of no return) applied to operations.

Expand-contract migration pipeline with undo at each stage



Undo gets costlier down the stages, until 5a has only "roll forward".

Debrief: the undo column

Stage	The change	The undo button	Its cost
1. Expand	Add a nullable column	Drop it	Free
2. Dual-write + backfill	Write to both, backfill old rows	Flag off, stop the job	Free
3. Verify	Comparison job, shadow reads	n/a — nothing changes	—
4. Switch reads	Flag ramp with canary verdicts	Flip back (safe because dual-write kept running)	Seconds
5a. Stop old writes	Cents only from now on	Roll forward only — which is why the gates exist	—
5b. Drop + cleanup	Column and flags removed	Backup snapshot (for disasters only)	—

The undo column reads free, free, not needed, seconds — then, deliberately, gone after gates and approval: Fact 2 at work. **Sequencing the undo options is the migration design** — cheap reversals first, the one-way door last, backed by evidence. Every pattern plays a part: canaries check binaries, flags release behavior, health checks gate deploys, traces tie mismatches to requests, expand-contract governs the schema; underneath, the pipeline builds each image once and promotes it unchanged, and Kubernetes' readiness-gated rolling updates carry every stage's deploy. With Module 5's reliability patterns, a high-risk change becomes routine (the scaling capstone's aim too).

Interview one-liner

Move state in stages: expand with an unused column; flagged dual-write plus an idempotent, throttled, checkpointed backfill; verify with a comparison job and shadow reads; ramp reads with a flag and canary verdicts; contract in two gated steps. Each stage keeps a cheap undo until the last, deliberate one-way door.

Extend it yourself (practice questions)

- **Queue-format version:** the same process for a breaking message-format change consumed by five teams — publish both formats, shadow-consume the new one, contract later. What should a **schema registry** (central store of message formats rejecting incompatible changes) reject?
- **Multi-region:** with cross-region primary-replica replication, backfill traffic raises **replication lag** (delay before a primary's write appears on a replica). Design the comparison job so stale replicas don't trigger false mismatch alerts.
- **For AI engineers:** migrate prompt-v1 → prompt-v2 or model A → model B. "Dual-write" means running both and comparing outputs; the comparison job runs **LLM-judge evals** (a model grading answers) on sampled pairs. Define the flag ramp and the evidence needed before contract retires prompt-v1.

Module 10 — Data-Intensive Systems (Batch and Streaming)

Module Introduction

Original lesson: Module introduction on DesignGurus ↗

What this module is about. Earlier modules mostly handled one request at a time in milliseconds. This one asks questions about *lots* of data at once — revenue per hour, fraud patterns, features built from long history — over input that never stops: a **continuous flow** of clicks, order events and table changes. Data engineers usually build these systems, but every engineer should know how a dashboard's numbers are produced.

Analogy. Counting votes: wait until polls close and count every ballot (batch — complete but late), or keep a running tally as boxes arrive (streaming — fresh but provisional, since some boxes are late). Same question, different balance of completeness and freshness.

New constraints: input may exceed one machine; records arrive late or more than once; consumers read at different speeds. So every data design answers four things: how is work **partitioned** across machines? Where does **intermediate state** live? How does processing **resume after a failure**? Which delays are tolerable, and which data losses are not?

Two facts that organize the module

1. **Batch and streaming are the same computation with different staleness budgets** (how old an answer may be). "Revenue per product" can be computed nightly from yesterday's full data (batch: simpler, thorough, hours stale) or updated with each order (streaming: seconds fresh, costlier to run). As in the caching module, the allowed age picks the mechanism. Batch works on **bounded** data (a finished dataset with a known end); streaming on **unbounded** data, so it can't wait for "all the input" and must decide when a partial answer is complete enough to publish.
2. **The log is the foundation.** Module 3's **write-ahead log (WAL)** — every change written to a sequential log before being applied — grows here into a data platform: Kafka-style logs (**Kafka**: an open-source system storing event streams as a durable, replayable log) carry event flows; **CDC** (Change Data Capture) turns a database's WAL into a public stream; **replay** (rereading from an earlier position) enables reprocessing; exactly-once combines log positions with **idempotency** (doing an operation twice has the effect of doing it once). Basics: Module 0 — Batch vs stream processing, Event streaming, Delivery guarantees.

The seven patterns (plus one added lesson)

The situation	The pattern
One big question asked over a huge dataset	MapReduce
Getting answers while data is still coming in	Stream Processing
Bringing a slow-but-thorough result and a fast-but-rough result into agreement	Lambda & Kappa
Broadcasting every change the database makes	Change Data Capture
Counting each thing once, even with retries and replays	Exactly-Once Semantics
A producer that sends faster than its consumer can keep up	Backpressure
Running consumers in parallel while keeping the order for each key	Partitioned Consumption
Knowing what the log underneath really guarantees, and which settings decide it	Inside Kafka *(added lesson)*

How to read it. MapReduce shows batch's shape (split, compute locally, merge) with deliberately hours-old answers — the reason Stream Processing (windows, event vs. processing time, answering over unfinished input) exists. Lambda & Kappa weighs separate batch and stream pipelines against one

replayable stream pipeline. The rest is supporting infrastructure: CDC makes streams from database changes, Exactly-Once builds on idempotency, Backpressure handles traffic beyond a consumer's capacity, Partitioned Consumption keeps per-key order under parallelism. **Inside Kafka** (added lesson) opens the log: segments and retention, why it's fast, replication with the ISR and high watermark (the fully caught-up copies, and how far all of them have got), every message-loss path and the setting preventing it, consumer-group internals (a **consumer group** is a set of readers sharing one stream's work), and how idempotent producers and transactions implement exactly-once. The capstone assembles a clickstream dashboard. Pick batch when completeness beats freshness; streaming only for short deadlines.

Short on time? Data engineers: Stream Processing and CDC (most tested in data-platform interviews). Everyone else: CDC, which fixes many everyday integration problems.

Four terms used throughout

- **Bounded vs. unbounded** — a known end (yesterday's orders; batch) vs. a never-ending flow (streaming, answering without all the input).
- **Event time vs. processing time** — when something happened vs. when the system received it; a mobile order placed in a tunnel at 9:02 may arrive at 9:14.
- **Watermark** — the processor's claim "I've probably seen every event up to time T," letting computation over endless data publish time-based answers.
- **Throughput vs. latency** — batch maximizes rows per hour; streaming minimizes per-result delay. Say which matters; it shapes design and cost.

Before you start

- How stale is a number on your main dashboard — and does anyone *know*, or assume it's live?
- How does analytics get database data: a nightly dump, API scraping, a read replica? What does that route miss?
- If event volume doubled for a day, would your pipeline slow, fall behind or break — and did anyone decide which?

MapReduce

Original lesson: MapReduce on DesignGurus ↗

What it is. One big question over a huge dataset, spread across hundreds of machines, written as two small functions: **map** turns each record into key/value pairs; **reduce** combines all values sharing a key. The framework splits the data, runs your code near where the data sits, groups results by key, and re-runs failed tasks.

Analogy. A national election count: each polling station tallies its ballots per candidate (map + local tally), sends subtotals sorted by candidate to regional centres (shuffle), where each centre sums its candidate (reduce). If one counter falls ill, only that station is recounted.

The problem

Finance wants **"total revenue by product category, all time, by region"** over **5 billion order rows, ~2 TB**, in **object storage** (cheap cloud storage such as S3 holding whole files).

An analyst's single-machine script reads about **100 MB/s**: 2 TB ÷ 100 MB/s = **20,000 seconds ≈ 5.5 hours just to read**. Started Monday 9 AM, it dies at 12:40 PM when the provider reclaims its **spot instance** (cheap, reclaimable capacity), losing three hours of in-memory progress. Wednesday's retry dies after four hours on a network failure. By Thursday the analyst has hand-built a checkpoint file and resume flag — and still no answer.

Two limits: **one machine is too slow**, and **at multi-hour runtimes failure is likely** (even 1% per hour makes a five-hour job risky). Long jobs need parallelism *and* automatic recovery.

Why the obvious fixes fail

- **"A bigger machine."** Read bandwidth still has a ceiling, and it's still one **failure domain** — one crash loses everything in memory.
- **"Split it by hand"** over 20 machines and merge with a script. Works once, but a person must notice machine 14 dying at minute 50 and rerun its share; uneven chunks leave 19 machines idle while one grinds; the merge script is fragile; every new question needs a new manual procedure.
- **"Run it on the production database shards."** Scatter-gathering (sending the query to every shard and combining answers) over the **sharded** orders DB makes analytics scans fight checkout for production **I/O**. The storage module separated these workloads deliberately — why analytics has its own copy in object storage, and needs its own compute.

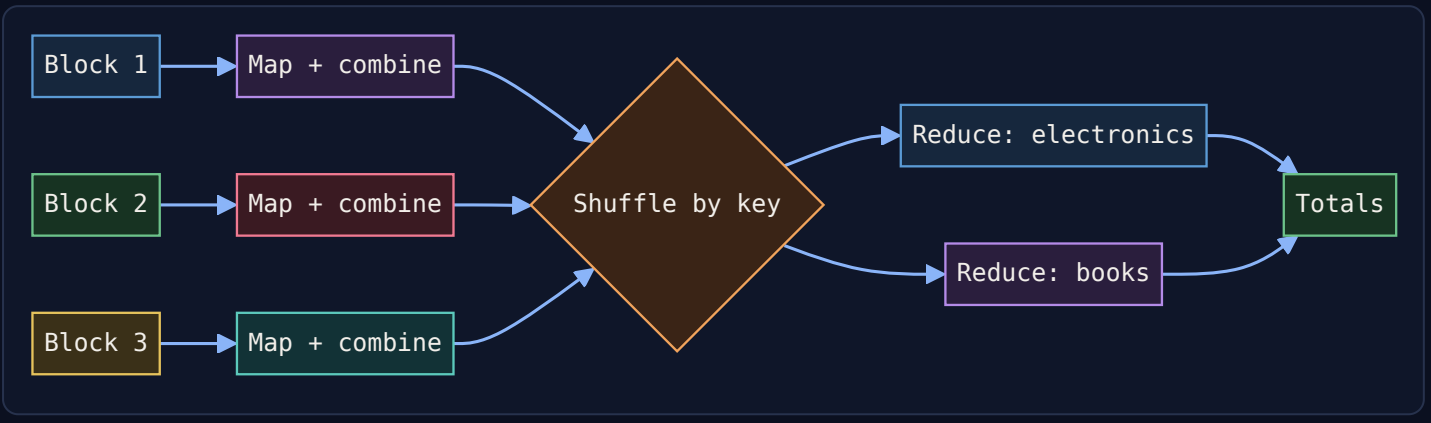
The hand-split had the right *algorithm* (divide, count locally, merge); missing was everything around it — scheduling, failure handling, **stragglers** (tasks running far slower than the rest), reliable merging. MapReduce is that execution framework.

How it works

```
map(order)             -> emit (order.category, order.amount)
       ... the shuffle groups: all "electronics" amounts to one place ...
reduce(category, [amounts]) -> emit (category, sum(amounts))
```

1. **Split and map where the data lives.** Input is cut into blocks and map tasks run on or near the machines storing them — ship a small program to the data instead of 2 TB to a program. This **data locality** makes distributed batch affordable, since the network is the scarcest resource.
2. **Combine (optional but important).** A **combiner** pre-aggregates each mapper's output locally — "electronics: $4.2M" once instead of a million amounts — shrinking the next step by orders of magnitude.
3. **Shuffle — the expensive middle.** The framework regroups every mapper's output *by key* across the cluster (an all-to-all sort plus network transfer), so every "electronics" value reaches one reducer.
4. **Reduce.** Each reducer merges its keys' values (e.g., sums) and writes results.
5. **Fault tolerance from the model's constraints.** Map and reduce must be **pure functions** (no side effects; same input, same output) over **immutable** input, so any task can rerun anywhere with the same result — idempotency for free. A dead machine's tasks are rescheduled (minutes lost, not the job); a straggler gets a **speculative duplicate**, and the first to finish wins. The framework automates the checkpoint-and-resume the analyst was hand-building.

Map-combine-shuffle-reduce for revenue by category



Walkthrough with numbers (200-machine cluster)

- **Read:** 2 TB ÷ 200 = 10 GB per machine, about **100 seconds** in parallel — 5.5 hours becomes under two minutes.

- **Map + combine:** one partial sum per category per machine — dozens of rows, so the shuffle moves **kilobytes** per machine.
- **Reduce:** summing 200 partials per category takes seconds.
- **End to end: ~10 minutes**, mostly startup and reading — versus a babysat week of restarts.
- **Failure drill:** machine 141 dies at minute 4; its ~30 map tasks rerun elsewhere, adding **90 seconds**, logged.
- **Straggler drill:** machine 77's failing disk slows it at minute 8; speculative copies finish first and 77 is flagged for ops. Nobody waits.

Trade-offs

You gain: scale by addition (200 machines ≈ 200× read bandwidth, coordination handled); fault tolerance for free (pure functions + immutable input make reruns safe); a tiny programming model — the question *is* the program.

You pay:

- **A latency floor by construction** — cluster startup, a full data pass and the shuffle: minutes at best, hours at scale, and the answer reflects data as of job start. Freshness is out of reach (next lesson).
- **A rigid shape** — every job is map → shuffle → reduce; real multi-stage analyses chained jobs, and classic MapReduce wrote each stage to disk, multiplying I/O.
- **Shuffle-dominated cost**, worsened by **skew** (one key with far more records; self-check Q3).
- **A scanner, not an index** — reading 2 TB to find "revenue for order #8412" is the wrong tool.

When not to use it

- **Interactive questions** — analysts iterating need seconds: use a **data warehouse** (an analytics database over huge tables; *columnar* — each column's values stored together, so scanning one column over billions of rows is fast). Under their SQL, warehouses still run map-shuffle-reduce shapes.
- **Small data — and "small" has grown.** 2 TB fits on a laptop SSD; one strong machine running **DuckDB** or **Polars** (an embedded SQL engine and a fast DataFrame library) scans it in minutes. Rule: **exhaust one machine before renting two hundred** (the vertical-scaling lesson's restraint). Cluster infrastructure for laptop-sized data is a common modern mistake.
- **Low-latency needs** — unbounded streams: next lesson.
- **Iterative algorithms** — ML training reads the same data hundreds of times; classic MapReduce's full disk I/O per pass is too slow, which drove in-memory successors and specialized training systems.

MapReduce vs. Spark (the idea vs. the implementation)

Apache Spark is an open-source cluster engine that keeps split-locally-then-merge but runs whole pipelines in memory.

	Classic MapReduce	Spark (and successors)
Between stages	Saved to disk after every stage	Held in memory through the whole pipeline
Program shape	Strictly map, then shuffle, then reduce	Any **DAGs** (graphs of steps) made of transforms: map, filter, join…
Multi-stage / iterative work	A chain of complete jobs, so the I/O multiplies	A single job with cached intermediate results: 10–100× faster
Fault tolerance	Run the tasks again from the input	Same idea: rebuild lost partitions from their **lineage** (the recorded chain of transforms)
You write	Java boilerplate, two functions at a time	SQL / DataFrames (tables you manipulate in code), and the shape is generated for you

Interview takeaway: **the execution model outlived the original implementation.** Nobody hand-writes MapReduce, yet Spark plans, BigQuery and Snowflake (cloud data warehouses) and distributed ML data prep

all split locally, shuffle by key, and merge.

Real-world examples

- **Google's 2004 paper** — web indexing needed this shape; it started the "big data" era.
- **Hadoop** — the open-source MapReduce (plus a distributed file system), dominant for about a decade. **Spark** — began at Berkeley to skip inter-stage disk writes and speed iterative work; it displaced Hadoop for many workloads and, via **Databricks** (hosted Spark), runs much production batch.
- **BigQuery, Snowflake and peers** — SQL on top, map-shuffle-reduce underneath; **dbt** — declarative pipelines on those engines.
- **For AI engineers:** training-data preparation is today's largest MapReduce workload: deduplicating, filtering and **tokenizing** (splitting text into model-readable tokens) a web crawl — map processes each document independently, shuffle groups near-duplicates by hash key, reduce picks survivors; serious labs run it on Spark. **Embedding** a corpus (turning each document into a meaning vector) is a map with no shuffle. Distributed training's **all-reduce**, merging **gradients** (per-step weight adjustments) across **GPUs** every step, belongs to the same compute-locally-then-merge family.

In the interview

Classic: "count word frequencies across 10 TB"; newer: "compute X over everything" or "the analytics job takes 9 hours and fails at hour 7 — fix it." Follow-ups probe the shuffle and failure handling.

30-second answer: "A batch aggregation shaped map-shuffle-reduce. Map runs where the data lives — code to the terabytes — emitting key-value pairs; combiners pre-aggregate locally so the shuffle carries partial sums; reducers merge per key. Fault tolerance comes from the structure: pure functions over immutable input mean any task reruns anywhere, a dead machine costs minutes, stragglers get speculative duplicates. In practice I'd write it in Spark or SQL, but this shape runs underneath. The shuffle is always the costly part, and the answer is stale by design — which is where streaming comes in."

Likely follow-ups (with answers):

- *"Where does the time go?"* — The shuffle (all-to-all regrouping over the network) and the input scan. Levers: combiners shrink what's shuffled; partitioning and skew management spread it.
- *"A machine dies at minute 40 of 60?"* — Its tasks rerun elsewhere on the same immutable inputs; minutes lost, nothing corrupted or restarted — a direct consequence of banning side effects and mutation. Say that link out loud.

Mistake that fails candidates: batch for interactive or low-latency needs; ignoring the shuffle; hand-building distributed counting instead of naming the shape and modern tools.

Self-check:

- **Q1.** "Average order value per country" as map and reduce — mind the averaging trap. → No average of averages. Map emits `(country, (amount, 1))`; reduce (and combiner) sums pairwise to `(sum_amounts, sum_counts)`; divide only at the end. The pair keeps the operation **associative** (partials merge in any grouping), which combiners and parallel reducers require. Rule: build aggregations from mergeable pieces.
- **Q2.** Three hand-split failures the framework avoids, and the property behind each fix. → (1) Dead machine: tasks rerun — safe because pure over immutable input. (2) Stragglers: speculative duplicates — harmless because reruns give the same answer. (3) Uneven chunks and custom merges: fixed-size blocks, and the shuffle *is* a standard merge. All from restricting the model so reruns are free.

- **Q3.** On 200 machines, 199 reducers finish in 2 minutes and one runs 6 hours; all healthy. → **Skew**: one hot key (say Electronics = 40% of orders) sends 40% of the work to one reducer — the sharding lesson's hot key in batch form (hashing spreads keys, not weight). Fixes: combiners (200 partial sums instead of 2 billion rows); for still-heavy keys, **salting** (`electronics-1..16` over 16 reducers, merged in a second stage); or process extreme keys separately.

Related patterns:

- **Stream Processing** — the same computations over unbounded input, answering the latency floor.
- **Sharding** — same partitioning math and skew, one layer down.
- **Lambda & Kappa** — how batch and streaming share responsibility for truth.

TL;DR

- **Problem:** one machine needs hours just to read, and at multi-hour runtimes failure is routine; hand-splitting reinvents scheduling, straggler handling, merging and recovery.
- **Mechanism:** two pure functions — map (per record, where the data lives) and reduce (merge per key); the framework owns splitting, the combiner-shrunk shuffle, speculative copies and re-execution.
- **Costs:** minutes-to-hours latency, a rigid shape (classically with disk I/O between stages), shuffle-dominated cost, skew.
- **Skip it when:** interactive (warehouse), fits one machine, needs fresh answers (streaming), or iterative (ML).

Stream Processing

Original lesson: [Stream Processing on DesignGurus ↗]

What it is. A job that **never ends**: it reads a growing event log, holds running **state** per tracked thing (counts, sums, windows) and emits answers continuously. The central hard question: when input never ends, when is an answer "final"?

Analogy. A teller watching live card swipes with a 10-minute tally per card, flagging a card at its sixth swipe — versus reading yesterday's statements next morning.

The problem: correct, but eight hours too late

The nightly fraud batch spots coordinated stolen-card use with **99% precision**. Tuesday it flags **30 fraudulent orders** from one ring, all placed **3:40–3:55 PM** and shipped by 6 PM; the job ran at **2 AM**, eight hours after the goods left. **Right but useless.** "Revenue by category, all time" can be hours stale; "is this card being abused *right now*" needs seconds, which no batch tuning reaches. Likewise an ops dashboard showing yesterday's "orders per minute". Fraud detection, live metrics and fresh ML features need ongoing answers about the last few seconds of endless input.

Why the obvious fixes fail

- **"Run the batch every five minutes"** (micro-batching). Real (it inspired Spark Streaming), but job startup sets a floor on the interval (a cluster job every 30 s may mostly be starting up), and "orders on this card in the last hour" spans many runs, so each run needs **running state from previous runs**. The true batch/stream difference is *persistent state*, not just speed.
- **"Query the database on every order"** — `SELECT COUNT(*) WHERE card=? AND time > now()-1h` is **400 expensive queries/s** on production: analytics-on-production again, with indexes not built for it. Databases store history; they aren't built to *watch* arrivals.
- **"Count in application memory."** With checkout scaled horizontally, **40 servers each see ~1/40** of a card's orders — the per-server-counter problem. Every event for a card must reach *one* place holding its state: **key-based routing**.

How it works

1. **State is the core.** Each operator (processing step) keeps its running answer locally (memory or local disk), **keyed**: all events for card X go to the instance owning card X — the shard-key idea for compute, formalized by Partitioned Consumption. Most of a streaming framework's job is managing this state.
2. **Recovery = checkpoint + rewind.** The framework periodically saves a durable **snapshot of all state together with the matching log position**. After a crash it loads the snapshot, **rewinds the log** to that position and reprocesses — possible only because the source is replayable. Events near the crash are processed **twice**, so outputs must tolerate duplicates (→ Exactly-Once).
3. **Windows slice endless input into finite pieces**, since an infinite stream has no "final total":
   - **Tumbling** — fixed, back-to-back intervals; each event in exactly one (orders per minute).
   - **Sliding** — overlapping intervals (orders in the last 10 minutes, re-evaluated every 30 s).
   - **Session** — open until an **inactivity gap** (a browsing session ends after 30 idle minutes); length set by the data.
4. **Event time vs. processing time — the hardest, most consequential idea.** Grouping by *processing time* (arrival) counts the 9:02 tunnel order at 9:14: simple, but wrong — retries, mobile batching and network delay reorder arrivals and create fake spikes. Grouping by *event time* counts it at 9:02, but at 9:03 the system can't know it exists, so windows must stay open for late arrivals.
5. **Watermarks decide when a window may close.** An estimate like "I've probably seen everything up to 9:05"; when it passes a window's end, the window closes and emits. The watermark delay *is* the staleness budget: longer → more complete, later; shorter → faster, more "late" events. So every consumer needs a **late-data policy**: drop late events, emit corrections consumers agree to accept, or route them to a side channel for reconciliation.

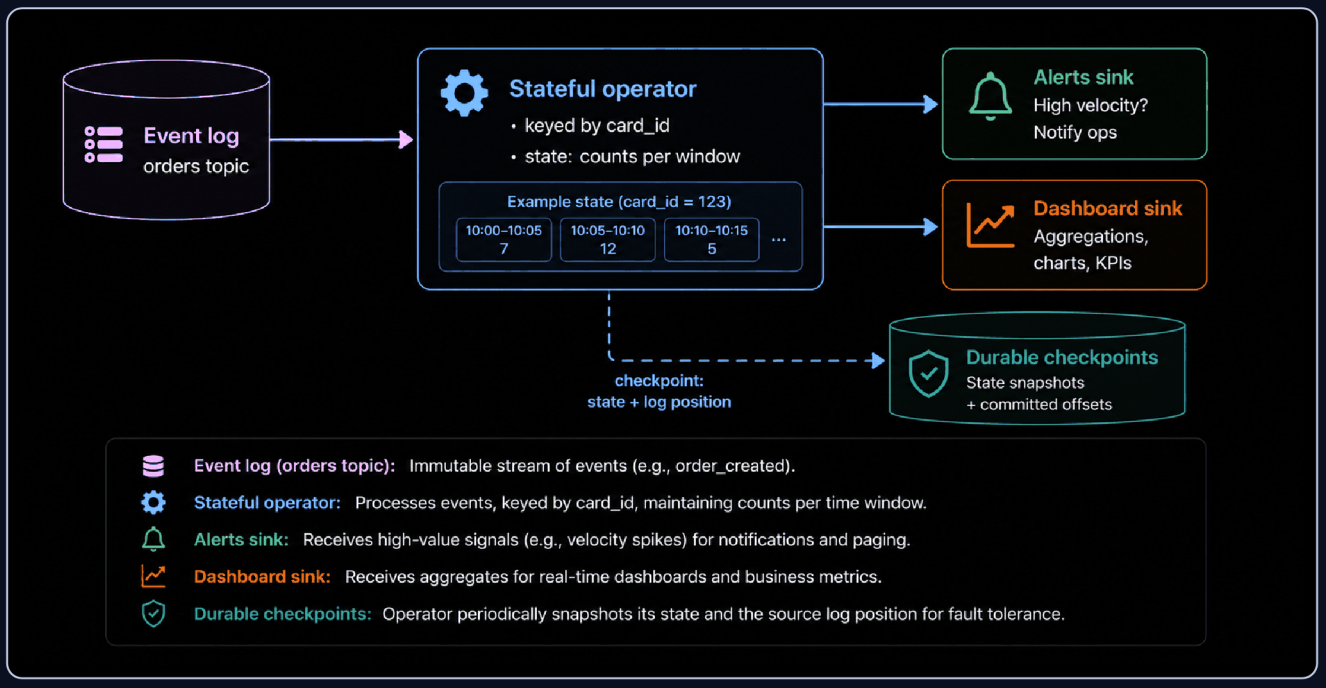


Figure — Stream Processing (course diagram)

An orders **topic** (named event stream) feeds a stateful operator keyed by `card_id`, whose state holds per-window counts (card 123: 7, 12, 5 across three 5-minute windows); it emits to an alerts **sink** and a dashboard sink (where a pipeline writes output) and regularly checkpoints state + log position.

Walkthrough with numbers (rule: more than 5 orders on one card within 10 minutes)

- **The path:** order event → log → the operator holding the card's state → count updated → threshold checked → alert: **~200 ms**. Tuesday's ring is caught at **order #6 at 3:44 PM**, goods still on the shelf.
- **State size by arithmetic:** **~200,000** active cards per 10-minute window × a few bytes = **megabytes**. Compute this before reaching for heavy state storage.

- **Crash drill:** load the **30-second-old** checkpoint, rewind 30 s, reprocess — **under a minute**, no orders missed. Replayed alerts go out twice, so the alert sink **dedupes on** `(card, window)` (idempotency).
- **Tunnel drill:** fraud's watermark lags only **30 s**, so the 9:02 order arriving at 9:14 is late data. Fraud's policy: process it, update the count, alert late if the threshold is crossed. The dashboard on the same stream chose **drop** (a per-minute chart won't redraw for one straggler). **Same stream, two policies — the policy belongs to the question, not the pipeline.**

True streaming vs. micro-batch

	True streaming (per event)	Micro-batch (tiny frequent jobs)
Latency floor	Milliseconds	The length of the batch interval (seconds up to a minute)
State between "runs"	Built in — the whole operator is designed around it	Carried from run to run by the framework
Complexity	Everything described above	Clearly easier to reason about
Fits	Fraud, alerting, features needed in under a second	Dashboards, feeds, pipelines with a one-minute budget

The budget decides: a one-minute budget → micro-batch is simpler and correct (**Spark Structured Streaming**); a one-second budget → only true streaming (**Flink**, Apache's stateful stream engine). "The budget chooses" beats tool loyalty.

Trade-offs

You gain: seconds-fresh answers over endless input (making fraud checks, live metrics and fresh features possible); incremental economics (each event updates state once instead of rescanning); solid correctness machinery (checkpoints + replayable log).

You pay:

- **The course's steepest concept stack** — windows, two kinds of time, watermarks, late policies, state; routinely underestimated.
- **Stateful operations** — checkpointing, restoring, monitoring and **rescaling** state (re-partitioning when the job grows or shrinks — hard).
- **Provisional answers** — event-time results change when stragglers arrive, so consumers must accept corrections (**eventual consistency** in analytics).
- **Operational weight** — a forever-running stateful job is a *service* needing **on-call**, upgrades and capacity planning.

When not to use it

- **No deadline** — nightly rollups and weekly cohorts are simpler, cheaper and easy to reprocess in batch.
- **Small scale** — a **cron job** with a SQL query counts a startup's daily actives without a Flink cluster.
- **Ad-hoc exploration** — analysts need a warehouse, not a streaming job per idea.
- **No owning team** — an unowned stateful platform becomes the main outage source between log and dashboard.
- Misuse: rebuilding stable nightly jobs as streams without a business need for freshness.

Real-world examples

- **Apache Flink** — the full model (event time, watermarks, exactly-once state); most of this lesson's vocabulary.
- **Google's Dataflow model (2015 paper)** — unified windows, watermarks and late-data triggers; most frameworks implement it.
- **Kafka Streams** — the pattern as a library inside your service, no separate cluster.

- **Spark Structured Streaming** — micro-batch at industrial scale.
- **For AI engineers:** feature freshness affects model quality — a fraud model on hour-old velocity features misses coordinated attacks. A **feature store** (computing and serving ML input values for training and live prediction) uses such jobs on its **online path** for millisecond-fresh keyed aggregates; token metering, prediction monitoring and **evals** over sampled traffic rely on it too. Use **event time**: processing-time features teach the model *network lag*, not user behavior.

In the interview

Direct prompts: "design real-time fraud detection", "a live orders-per-minute dashboard". Predictable follow-ups: late events, crash recovery, counting across a server fleet.

30-second answer: "A never-ending job reading the order log with state keyed by card, so every event for a card reaches the instance holding its counts; it checks a sliding window per event and alerts within a couple hundred milliseconds. Windows use event time, because retries and mobile lag scramble arrival order. A watermark thirty seconds behind closes windows; late stragglers still update the count, since a late fraud alert beats none. Recovery is checkpoint plus rewind — state snapshots tied to log positions, with the replayable log reprocessing the gap — and because some events process twice, the sink dedupes on card-and-window. With a one-minute budget I'd micro-batch and keep it simple."

Likely follow-ups (with answers):

- *"An event arrives 12 minutes late?"* — Its window has closed, so the late policy applies — chosen *per consumer*: fraud updates and alerts late, the dashboard drops, a reconciliation stream side-channels it. Per-consumer policy is the senior detail.
- *"The operator crashes mid-afternoon — what's lost?"* — Nothing durable: last checkpoint, rewind, reprocess. Sinks see a few seconds replayed — the real cost of a crash is *duplicate emissions*, hence idempotent sinks.

Mistake that fails candidates: processing-time windows for money or fraud; state in application memory across a fleet; a database query per event — each a failed "obvious fix" dressed up as a design.

Self-check:

- **Q1.** Windows for (a) revenue per calendar minute for a chart, (b) "more than 5 orders in any 10-minute span", (c) average items per shopping session. → (a) Tumbling — clean non-overlapping buckets. (b) Sliding — must catch *any* 10-minute span. (c) Session — the boundary is the activity gap.
- **Q2.** A carrier outage holds back 20 minutes of app events, then they arrive in one burst. Processing-time pipeline vs. event-time with a 2-minute watermark? → Processing time: a fake quiet dip during the outage, then a huge fake spike "now" as 20 minutes land at once — triggering rate alerts and permanently recording traffic that never happened. Event time: delayed events keep real timestamps; outage windows close ~2 minutes after ending with provisional *undercounts*, then late events trigger corrections per policy and numbers settle on reality. Processing time measures the network; event time measures users.
- **Q3.** After a routine deploy, fraud gets duplicate alerts (same card, same window) for about a minute of traffic, with no real spike. → The deploy restarted the operator via normal recovery (restore checkpoint, rewind, reprocess ~1 minute); the computation was right both times but the sink emitted twice. Make alerts idempotent on `(card_id, window_id)` or use a transactional exactly-once sink. **Every deploy rehearses the crash.**

Related patterns:

- **Message Queue** — the replayable log source enabling rewind recovery.
- **Partitioned Consumption** — keyed routing of each card's events to its state.
- **Exactly-Once Semantics** — handling what replays do to sinks.
- **Write-Ahead Log** — checkpoints borrow it: durable state plus position, recovery by replay.
- **Lambda & Kappa** — when batch and streaming both claim the truth.

TL;DR

- **Problem:** deadline questions (fraud, metrics, features — now) concern the last seconds of endless input; batch answers arrive hours late.
- **Mechanism:** a forever-running job with keyed state over a replayable log; tumbling, sliding and session windows; event time with watermarks; per-consumer late policies; checkpoint + rewind recovery.
- **Costs:** the steepest concept stack, a stateful service to run, provisional answers that get amended, replays requiring idempotent or transactional sinks.
- **Skip it when:** the staleness budget is generous, a cron job suffices, the work is ad-hoc exploration, or nobody can own a forever-running service.

Lambda and Kappa

Original lesson: [Lambda and Kappa on DesignGurus ↗]

What it is. Two architectures for needing *both* fresh streaming answers and complete, correct batch answers. **Lambda** runs batch and streaming pipelines side by side, with batch results periodically **overwriting** streaming ones. **Kappa** runs **one** streaming pipeline over a log that retains history, and corrects by fixing the code and **replaying** the log.

Analogy. Lambda: a newspaper's quick online updates replaced by a fact-checked overnight print edition. Kappa: one newsroom that reruns its stories from the archive with corrected facts.

The problem: two pipelines, two truths

A nightly batch pipeline feeds official reports and a streaming pipeline feeds the live dashboard; both compute revenue. Wednesday, finance sees Tuesday at **$412,880** on the dashboard and **$413,205** in the batch report. Both are "right" by their own rules: the stream dropped late events per its watermark policy and had a **one-minute dedupe gap** during Tuesday's deploy (the stream lesson's Q3, for real); batch read everything and is correct as of 2 AM. Nobody ever decided which is *the truth*.

Three weeks later a discount-rule change ships only in the batch code. Now they differ by a **coding error** that grows daily, and a monthly "reconciliation meeting" appears where people referee their own computers.

Named precisely: two codebases implementing the same business logic drift apart over time — a problem anyone running separate batch and streaming must handle on purpose.

Why the obvious fixes fail

- **"Perfect the stream and retire batch."** Exactly-once reduces processing errors, but batch's main strength is **reprocessing history with current code** (a changed fraud rule, a found bug). A stream can revisit events only if a replayable log retains them — which becomes Kappa's key idea.
- **"Run batch hourly."** Fresher, but still two independently changing codebases; the inconsistency window shrinks, its cause remains.
- **"Label the dashboard 'approximate' and declare batch the truth."** A valid policy if stakeholders agree, but it doesn't stop forgotten-change drift and keeps two codebases.

How it works — Lambda

Proposed by **Nathan Marz** (Twitter / Storm era; **Storm** was an early open-source stream processor). Three layers:

1. **Speed layer** — streaming; fast, provisional.
2. **Batch layer** — slow, authoritative; recomputed on schedule from raw history.
3. **Serving layer** — stitches them: recent hours from speed, settled history from batch.

The central mechanism is **the overwrite**: each batch run *replaces* the speed layer's results for its period, so streaming bugs — the deploy gap, dropped stragglers, even the forgotten discount rule — are wiped at the next run. Lambda doesn't prevent speed-layer errors; it **bounds their lifetime to one batch cycle**. The permanent cost: **every metric is implemented twice**, so logic drift is built into the architecture.

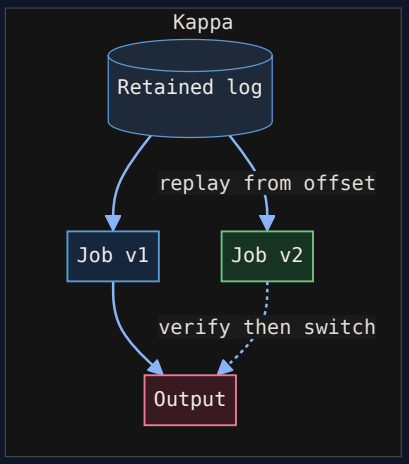
How it works — Kappa

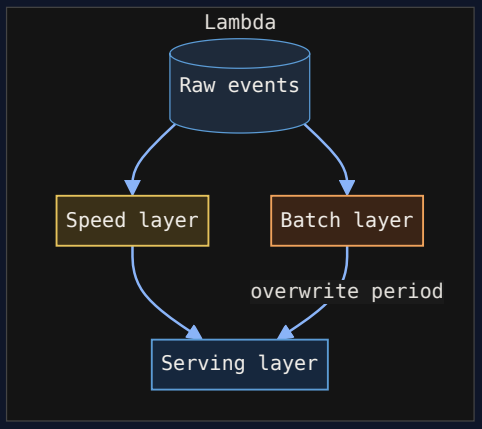
Proposed by **Jay Kreps** in "Questioning the Lambda Architecture". If the event log retains history for weeks, months or effectively forever via **tiered storage** (old segments moved to cheap object storage), batch is just *streaming a bounded prefix of the log, fast*, and reprocessing becomes a streaming operation:

1. Deploy job **v2** with the new logic, reading from **offset** (log position) zero or, say, 90 days back.
2. v2 races through history at full read speed — hours, not weeks.
3. v2 catches up to "now" and runs beside v1; outputs are compared.
4. Point consumers at v2's output; retire v1.

The zero-downtime migration playbook (run both, check, switch, retire) applied to pipelines. One codebase makes the discount-rule drift impossible. Costs: long retention, replay compute far above the live rate (90 days in an afternoon), and a streaming engine mature enough (exactly-once, event time) to be the *only* truth.

Lambda vs Kappa data flow





The pragmatic synthesis

Most organizations run a practical **dual path** — streaming for operational freshness, a warehouse for deep analysis — often without Lambda's formal overwrite, but they should keep **scheduled backfills** (recomputing a past period and replacing existing results). The engines are converging: **Flink** runs bounded inputs in batch mode, **Spark** processes streams, and **lakehouse tables (Delta, Iceberg)** (open table formats giving database-like tables over object-storage files) accept streaming writes while serving batch queries. Increasingly: **one logic, two execution tempos** — Kappa's aim.

Walkthrough with numbers: the deploy-gap bug under each design

- **Lambda:** Tuesday's deploy silently loses **~300 orders** from dashboard revenue; at 2 AM Wednesday batch recomputes Tuesday and overwrites the speed layer — self-healing, which is what the second codebase buys.
- **Kappa:** fix the bug in the one codebase and replay from **Monday's offset**; at **~50× the live rate**, **two days reprocess in under an hour**. Once v2 catches up, verify against finance's **$413,205**, move consumers, retire v1. Repeatable, but a person must start it.
- **The discount-rule drift:** under Lambda only one implementation got the update, surfacing when Wednesday's overwrite changed Tuesday's dashboard numbers; Kappa rules it out entirely. Summary: Lambda auto-fixes operational errors but risks two disagreeing implementations; Kappa removes duplication but needs someone to replay.

Comparison

	Lambda	Kappa	Pragmatic dual path
Codebases per metric	Two, on purpose	One	Two, by accident (fix: shared definitions)
Streaming-bug recovery	Wiped out automatically by the next batch overwrite	Replay from an offset — a routine operation	A manual backfill (if someone notices)
Logic drift	Part of the design	Impossible by structure	What happens by default
History economics	Raw data kept in cheap storage	Log retention (tiered)	Warehouse tables
Demands	Stitching in the serving layer	Mature streaming plus capacity to replay	Discipline that nobody enforces

Interview verdict: Lambda's overwrite survives as scheduled backfills; engines increasingly support Kappa's single codebase; many teams run **accidental Lambda** unknowingly and should fix it with automatic overwrites or unified logic.

Trade-offs

- **Lambda gains:** automatic scheduled correction bounds streaming errors to one cycle; each layer is simpler alone; truth is always recomputable from raw history. **Costs:** every metric twice (constant drift risk, duplicate infrastructure); the serving layer must define where "recent" ends and "settled" begins.
- **Kappa gains:** one codebase, no implementation drift; reprocessing as a tested migration routine; the log as the only history. **Costs:** retention (reduced by tiered storage), burst replay capacity, total dependence on the streaming engine — an unreliable pipeline compromises your only truth.

When not to use each

- **Don't Lambda with a small team** — you can't keep two pipelines agreeing. Two pipelines *without* the automatic overwrite is **accidental Lambda**: all the duplication, none of the recovery — the opening incident, and very common.
- **Don't Kappa when history is huge and reprocessing rare** — years of log for one yearly replay can cost more than rerunning a warehouse job; batch-shaped work (large multi-way joins, ML training passes) fits batch better.
- **Don't be dogmatic** — for most teams, a dual path with disciplined backfills and **one shared metric-definition library** (written once, run by both engines) beats either pure form.

Real-world examples

- **The founding documents:** Marz's Lambda (Storm era) and Kreps' "Questioning the Lambda Architecture" (Kafka side) — read as a debate, they *are* this lesson.
- **Uber, Netflix, LinkedIn** describe journeys along this spectrum: Lambda first (growth), drifting toward Kappa where log-centred culture took hold, a dual path where warehouses dominated.
- **Tiered storage** in Kafka removed Kappa's retention objection; Flink batch mode, Spark streaming and Delta/Iceberg lakehouse tables are where tools converge.
- **For AI engineers:** this is **training/serving skew** in infrastructure: batch warehouse code computes *offline training* features, streaming code computes *online serving* features, and if they define "user's 7-day order velocity" differently the model degrades. **Feature stores** define each feature once and run it in both modes.

In the interview

Prompt: "analytics with a live dashboard *and* accurate finance reports." The trap is building two pipelines unknowingly. Typical follow-up: "your streaming pipeline had a bug for three hours — recovery?"

30-second answer: "Two needs, one truth: the dashboard needs seconds-fresh numbers, the reports complete ones, and the risk is implementing metrics twice and drifting — the accidental Lambda most companies run. I'd build around the replayable log and lean Kappa: one streaming codebase where recovery is reprocessing — fix the logic, replay from an earlier offset at ~50× real time, verify against the old output, switch. Truly batch-shaped work (huge joins, training passes) reads the same log with shared metric definitions. I'd keep Lambda's best idea as scheduled backfills, bounding any streaming error's lifetime. Either way: one definition, one owner, one logic source per metric."

Likely follow-ups (with answers):

- *"The streaming job was wrong for three hours — recovery?"* — Kappa: replay from before the bug with fixed code; sinks accept restated numbers (the stream lesson's amended answers at architecture scale) or the output swaps after verification. Lambda: nothing — the next overwrite wipes the period. Pragmatic path: a backfill, if the habit exists. Your answer reveals which architecture you really have.
- *"Why not always run both to be safe?"* — Without an automatic overwrite it's double cost plus drift; with one, still two implementations per metric. Recoverability comes from *replayable history*; a second pipeline is just one way to use it.

Mistakes that fail candidates: two pipelines without a reconciliation plan (unnoticed accidental Lambda); Kappa without retention or replay-capacity math; treating the choice as taste rather than budget and team.

Self-check:

- **Q1.** Lambda's three layers, and the mechanism justifying building everything twice. → Speed (fast, provisional), batch (slow, authoritative, from raw history), serving (stitches recent + settled). The justification is the **overwrite**: each batch run replaces the speed layer's period, so streaming errors (gaps, drops, even logic mistakes) last at most one cycle. Without it, Lambda is just two disagreeing pipelines.
- **Q2.** Case A: log-centred organization, metrics changing monthly, 90 days of history enough for all reprocessing. Case B: 5 years of history, reprocessed about once a year, heavy multi-way joins. → A: **Kappa** — retention covers the range, frequent changes favor one codebase, log experience supplies maturity. B: a **batch-anchored dual path** — 5 years of log for one yearly replay isn't worth it and big joins suit batch; warehouse as authority, stream only operational views, shared metric definitions.
- **Q3.** You inherit the opening incident (disagreeing numbers, drifted discount logic, no overwrite, a monthly meeting). Name it, give both exits with first steps, state the invariant. → **Accidental Lambda.** Path 1 — finish Lambda: add the overwrite; first step: write nightly batch results into the dashboard store keyed by period. Path 2 — Kappa: consolidate into the streaming codebase; first step: extend retention beyond the reprocessing horizon and rehearse one replay-verify-switch for a single metric. Invariant: **one definition, one logic source, one owner per metric**.

Related patterns:

- **MapReduce** and **Stream Processing** — the two layers whose disagreement this settles.
- **Message Queue** — the retained, replayable log that makes Kappa possible.
- **Exactly-Once Semantics** — the correctness bar a sole pipeline must meet.
- **Feature Store** — training/serving skew is this lesson in ML form.

TL;DR

- **Problem:** batch and streaming compute the same metrics in two codebases that drift (policy differences, forgotten changes) until people reconcile monthly.
- **Mechanism:** Lambda — run both, with a scheduled batch overwrite of the speed layer's period (errors last at most one cycle). Kappa — one streaming codebase over a retained log; reprocess via replay-verify-switch.
- **Costs:** Lambda — every metric twice, drift built in, stitching seams. Kappa — retention, burst replay capacity, total dependence on a mature stream engine.
- **Skip / adapt when:** the team is small (don't run two of anything), history is huge and rarely reprocessed (warehouse wins), or compute is batch-shaped — and never run accidental Lambda.

Change Data Capture

Original lesson: Change Data Capture on DesignGurus ↗

What it is. CDC turns the change log a database keeps for itself into a public event stream. A **connector** reads the database's **write-ahead log (WAL)** — the ordered record of every committed change — as a **replica** would, but *publishes* changes instead of applying them. Every committed insert, update and delete, from any code path, becomes an event on a topic in commit order, consumed by search, caches, warehouses and streams.

Analogy. Instead of asking every clerk to remember to call other departments after changing a file (they'll forget), tap the office's official logbook — where every change must be written to count — and broadcast each entry.

The problem: four incidents, one cause

Four earlier incidents share a root cause:

- **Search (CQRS lesson):** the app wrote the DB, then separately **Elasticsearch**; during a deploy the Elasticsearch calls timed out, and orders existed in the DB but never in search.
- **Cache (cache-aside lesson):** a bulk import wrote directly to the DB and never **invalidated** the cache; products stayed stale for an hour.
- **Analytics (pub/sub lesson):** analytics may not read the orders tables directly, yet needs the data.
- **Warehouse:** a full nightly dump — a 2 AM window, day-old data, reading *everything* to find the 1% that changed.

Derived systems need a *complete* change record, but ad hoc announcements always leak: app code forgets, bulk imports and hand-written SQL bypass the app. Meanwhile the database *already* records every committed change from every path, in order, in its WAL (Module 3's durability mechanism). **What if that log stopped being internal?**

Why the obvious fixes fail

- **Dual-write** — update the DB, then notify others. No single **transaction** covers both, so a crash between them leaves systems **permanently and silently inconsistent** — *loss*, not *lag* (lag is visible and recoverable; a missed write leaves no error).
- **Polling** — `SELECT * WHERE updated_at > :last_check` on a timer has three built-in gaps: (1) **deletes are invisible** (a deleted row matches no query, leaving a **ghost row** in the index); (2) **intermediate states collapse** (three updates between polls look like one snapshot — fine for a cache, not for audit or event processing); (3) it relies on **every writer** maintaining `updated_at`.
- **Database triggers writing to a change table.** A **trigger** (code the database runs on every insert, update or delete) does see every write, including bypasses — the right idea — but adds synchronous work to every transaction and per-table upkeep, while the database already keeps an ordered change record *below* the trigger layer.

How it works

A CDC connector **tails** the replication log (keeps reading as it grows). Each property fixes one problem above:

1. **It cannot miss.** Every committed change passes through the WAL — that's what "committed" means — so admin tools, bulk imports and manual `UPDATE`s are captured with no application discipline.
2. **Deletes arrive as events** — published as a **tombstone** ("this key no longer exists"). Deletion is a recorded fact (the vector-clocks rule); no ghost rows.
3. **Ordered per key, for free** — commit order.
4. **Zero application changes** — a full change history without rewriting the app or adopting **event sourcing** (storing every change as an event and deriving current state from them).
5. **At-least-once delivery** — never lost, occasionally duplicated, because after a crash the connector resumes from its saved log position. Consumers must be **idempotent**; row-image events ("set name to X") naturally are.
6. **Snapshot bootstrap.** On first connect the connector takes a consistent table snapshot, then switches to tailing with no gap, so new consumers get history plus live stream in order.

What CDC events are, and aren't. They say *what* changed ("row 8412: status active → cancelled"), not *why* ("customer cancelled due to delay") — **row diffs, not business intent**. Perfect for data infrastructure (indexes, caches, warehouses); for **domain events** (business facts like "OrderCancelled"), use the outbox.

The outbox pattern — the standard replacement for dual-writes. The app writes its business event ("OrderCancelled, reason: delay") into an **outbox table in the same transaction** as the data change — both or neither. CDC reads the outbox and publishes the event with its meaning. **The transaction provides atomicity, the log provides delivery, the app provides intent** — the "reliable publisher" the pub/sub lesson promised.

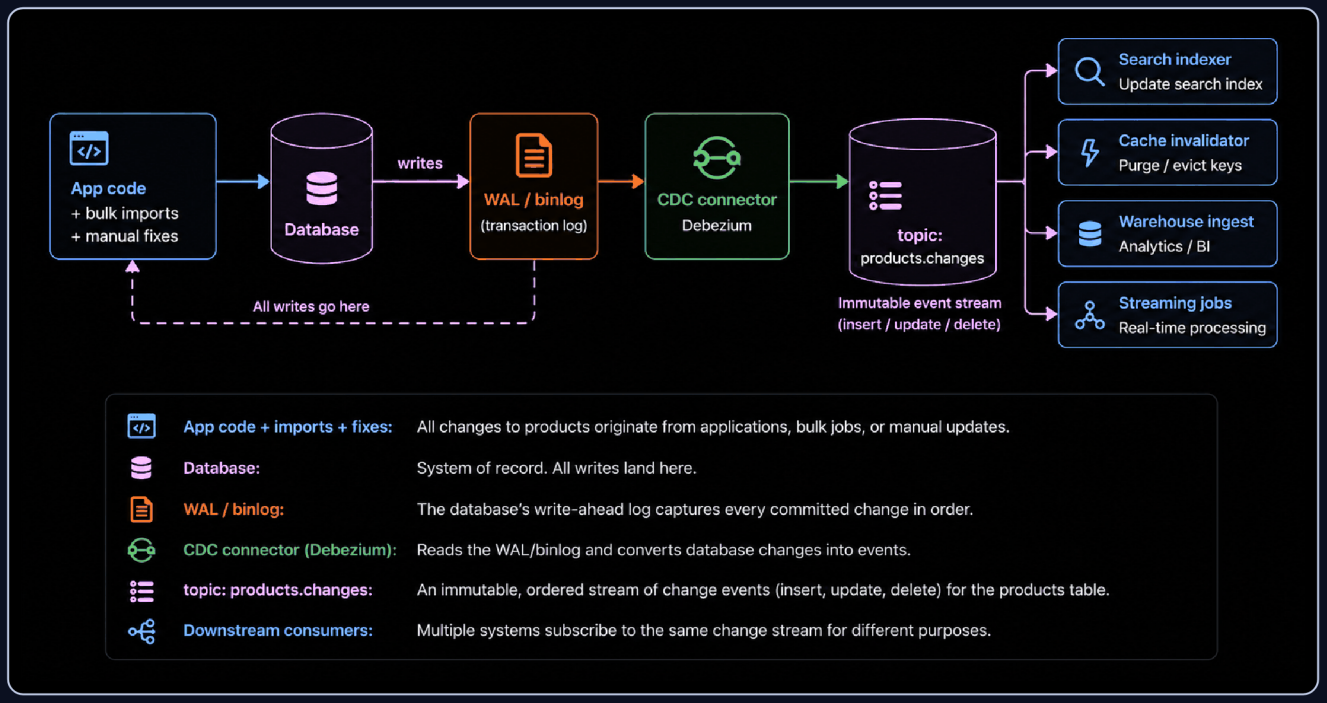


Figure — Change Data Capture (course diagram)

All write paths — app code, bulk imports, manual fixes — land in the database's WAL/binlog (*binlog* is MySQL's change log). The **Debezium** connector (popular open-source CDC) reads it and publishes ordered insert/update/delete events to a `products.changes` topic, consumed independently by a search indexer, cache invalidator, warehouse ingest and streaming jobs.

Walkthrough with numbers: the four problems, fixed

- **Search:** Debezium reads the products table; the indexer applies changes idempotently. Deploy timeouts can no longer cause permanent mismatch — an interrupted connector means measurable, recoverable **lag**. Freshness **under a second**; bulk imports update the index with no import code.

- **Cache:** a consumer deletes cache keys on each change event; forgotten invalidations disappear, since every committing path produces an event.
- **Warehouse:** the nightly dump read **2 TB to find 20 GB** of changes; streaming changes is **400 writes/s × ~500-byte events ≈ 200 KB/s**. "Yesterday" becomes "seconds ago".
- **Failure drill:** the connector stops Friday at **6 PM**, unnoticed until **8 PM**. The database retains unread WAL segments because the connector's **replication slot** (a bookmark of one log reader's position) hasn't confirmed them; restarted, it catches up in minutes — lag, not loss. A connector that *never* returns, though, makes retained log grow without limit (self-check Q3).

Three ways to publish a change

	Naked dual-write	Raw CDC (table streams)	Outbox + CDC
Atomicity	None — this is the torn-write bug	Perfect (the log *is* the commit)	Perfect (a single transaction)
Carries intent	It could, but writes can tear	No — only row diffs	Yes — whatever event you choose
App changes	Yes, and every path needs discipline	None	One outbox write for each event
Consumer coupling	Tied to your API	Tied to your table schemas (column names and types)	Tied to a contract you designed
Fits	Nothing — retire it	Your own infrastructure: caches, indexes, warehouse	Domain events that other teams consume

Rule to memorize: raw CDC for derived infrastructure you own; outbox for events other teams consume; never dual-writes. Together they solve Module 2's reliable-publishing problem.

Trade-offs

You gain: cannot-miss completeness (every path, bypass writes and deletes included); one integration, N consumers (pub/sub economics); seconds of freshness instead of nightly, with failures causing **lag, not loss**.

You pay:

- **Schema coupling — the big one.** Raw events copy table schemas, so a column rename hits every consumer and your internal schema becomes public API. Soften it with an outbox for domain events, a **schema registry** (stores and checks event schemas, rejecting incompatible changes), or a transformation layer between table and event schemas.
- **WAL retention pressure.** A slow or dead connector blocks log trimming (the WAL lesson's failure mode); replication-slot lag must be a top-priority alert, or it becomes a full disk on the **primary**.
- **Diffs, not intent** — "why" needs the outbox.
- **Another critical component** — connector **high availability** (usually a standby), snapshot management, topic hygiene; small, but on every derived system's path.

When not to use it

- **Other teams want meaning** — don't ship them your table structure; use the outbox (CDC underneath, publishing a designed contract).
- **Tiny scale** — a nightly dump may suffice; CDC's operating cost needs a real freshness or completeness payoff.
- **As a transaction mechanism** — CDC is **asynchronous**, consumers lag by design; it spreads facts *eventually*, never making two systems atomically consistent (Module 7).
- **No log access** on a managed database — increasingly rare (Postgres logical replication, MySQL binlog and cloud equivalents are nearly universal), but check.

- **Common misuse:** raw CDC topics as the company-wide domain-event bus — every consumer depends on table schemas and every migration becomes a multi-team negotiation (the EDA lesson's governance problem).

Real-world examples

- **Debezium** — open-source connectors reading Postgres, MySQL, MongoDB and others into Kafka; interviewers often expect it.
- **Postgres logical replication and replication slots** — the mechanism underneath (and Q3's disk failure); **MySQL's binlog** is the equivalent.
- **Managed ingest: Fivetran, AWS DMS, Snowpipe Streaming**.
- **Netflix's DBLog** and similar in-house systems — snapshot plus tail at streaming-giant scale.
- **For AI engineers:** CDC keeps feature stores and retrieval indexes fresh: "user's current plan" or "orders this week" update within seconds without app teams publishing anything. For **RAG**, content-table changes trigger **re-embedding only changed rows** (the CQRS lesson's vector-index read model; a **vector index** finds items whose embeddings are near a query's), so stale embeddings don't hurt relevance.

In the interview

Prompt: "keep the search index / cache / warehouse in sync with the database." Dual-writes aren't atomic; polling misses deletes and intermediate states; the complete answer is log-based CDC plus an outbox for domain events.

30-second answer: "A Debezium-style connector tails the database's WAL and publishes every committed insert, update and delete as ordered events. It can't miss anything — bulk imports and manual fixes go through the same log — and deletes arrive as tombstones, which polling can never see. Consumers (search indexer, cache invalidator, warehouse) apply events idempotently because delivery is at-least-once; a down connector means lag, not loss. Two rules: raw table streams only for my own infrastructure, since their schema is my table's schema; domain events for other teams go through an outbox written in the same transaction, eliminating dual-writes. And I alert on replication-slot lag, because a dead connector silently stops WAL trimming until the primary's disk fills."

Likely follow-ups (with answers):

- *"Why not have the app write to both places?"* — The torn write: no transaction spans DB and index, so a crash between writes splits them silently and forever — loss, not lag. CDC reads *after* commit from the commit's own record, so a split is structurally impossible; only visible lag remains.
- *"A row is updated then deleted between two polls — polling vs. CDC?"* — Polling sees nothing (the deleted row matches no query, its intermediate state was never sampled) and leaves a ghost row; CDC emits the update then the delete tombstone, in order.

Mistakes that fail candidates: confident dual-writes; polling presented as complete with no answer for deletes; raw CDC as the company event bus; at-least-once without idempotency.

Self-check:

- **Q1.** Why can log-tailing never miss a change, and what two things does timestamp polling miss by design? → Every committed change passes through the WAL, so a WAL reader sees every change from every path in commit order. Polling misses (1) deletes and (2) intermediate states, and depends on every writer maintaining `updated_at`.

- **Q2.** Raw CDC or outbox for (a) your team's search index, (b) an "order shipped" notification to partners, (c) the warehouse? → (a) Raw CDC — your index wants row diffs, no app changes. (b) Outbox — partners need a versioned, intent-carrying "OrderShipped" contract written in the same transaction as the status change; never expose internal tables in hard-to-retract contracts. (c) Raw CDC — internal copying, with a downstream transformation layer for analytics models and a schema registry catching breaking migrations.
- **Q3.** Saturday 4 AM: the primary's disk is at **94%** and rising, writes normal, WAL segments not trimmed; a replication slot shows a consumer **30 hours** behind — a connector that crashed Thursday. → The slot stops the database deleting unconfirmed WAL, so a dead connector for a *derived* system threatens the *authoritative* database. Triage: restart the connector if it can catch up before the disk fills (30 hours may take tens of minutes); otherwise drop the slot to free space, then re-snapshot and rebuild consumers. Guards: **alert on replication-slot lag** in bytes or time (after hours, not at 94% disk), a highly available connector, and a written maximum-retention policy saying when operators may drop a slot to protect the primary.

Related patterns:

- **Write-Ahead Log** — CDC publishes Module 3's internal log.
- **Publish-Subscribe** — one feed, N consumers, schema-contract rules in force.
- **CQRS** — read models fed correctly after dual-writes failed.
- **Event Sourcing** — contrast: CDC for change history (diffs), event sourcing when events *are* the product (intent), the outbox between.
- **Cache-Aside** and **Idempotency** — complete invalidation events; consumers must dedupe.

TL;DR

- **Problem:** derived systems drift because app-level change announcements leak: dual-writes tear, polling misses deletes, bypass writes tell no one.
- **Mechanism:** tail the WAL like a replica and publish every committed change (inserts, updates, delete tombstones) in commit order; snapshot bootstrap; at-least-once to idempotent consumers; an outbox for intent-carrying domain events.
- **Costs:** table schemas become contracts, replication-slot retention can fill the primary's disk, diffs carry no intent, one more critical connector.
- **Skip it when:** consumers deserve a designed contract (outbox), the scale fits a nightly dump, or someone expects atomicity.

Exactly-Once Semantics

Original lesson: *Exactly-Once Semantics on DesignGurus ↗*

What it is. Techniques that make a streaming pipeline's *results* count each event exactly once, even though messages underneath are delivered and processed more than once. The key insight: **exactly-once *delivery* is impossible** over a failing network, but an **exactly-once *effect*** can be built — keep at-least-once delivery and make duplicates harmless.

Analogy. Counting stadium entrants with a clicker when a fire alarm makes you lose track: from an earlier crowd photo you must reset the clicker to the photo's number *and* count only people who entered after it. Reset only one and the total is wrong — count and bookmark are restored together.

The problem: wrong in both directions

A streaming job counts every partner's API calls per day for invoicing — the company's most money-sensitive pipeline.

- **Month one — overbilling.** A routine **crash-and-rewind** reprocessed **40 minutes** of events, and the running counts added them twice. Partner A was billed **1,041 calls**; their logs said **987**. The refund was cheap; "your metering can't be trusted" was not.
- **Month two — underbilling.** To stop duplicates, operations committed the **offset** (reading position) *before* processing; now a crash discards in-flight events — an undercount that can go unnoticed for months.

The two classic failure modes:

- **At-least-once:** never lost, sometimes counted twice.
- **At-most-once:** never twice, sometimes lost.

The idempotency lesson said exactly-once delivery is impossible, yet Kafka advertises "exactly-once semantics." Both are true: exactly-once here describes an **effect**, not delivery.

Why the obvious fixes fail

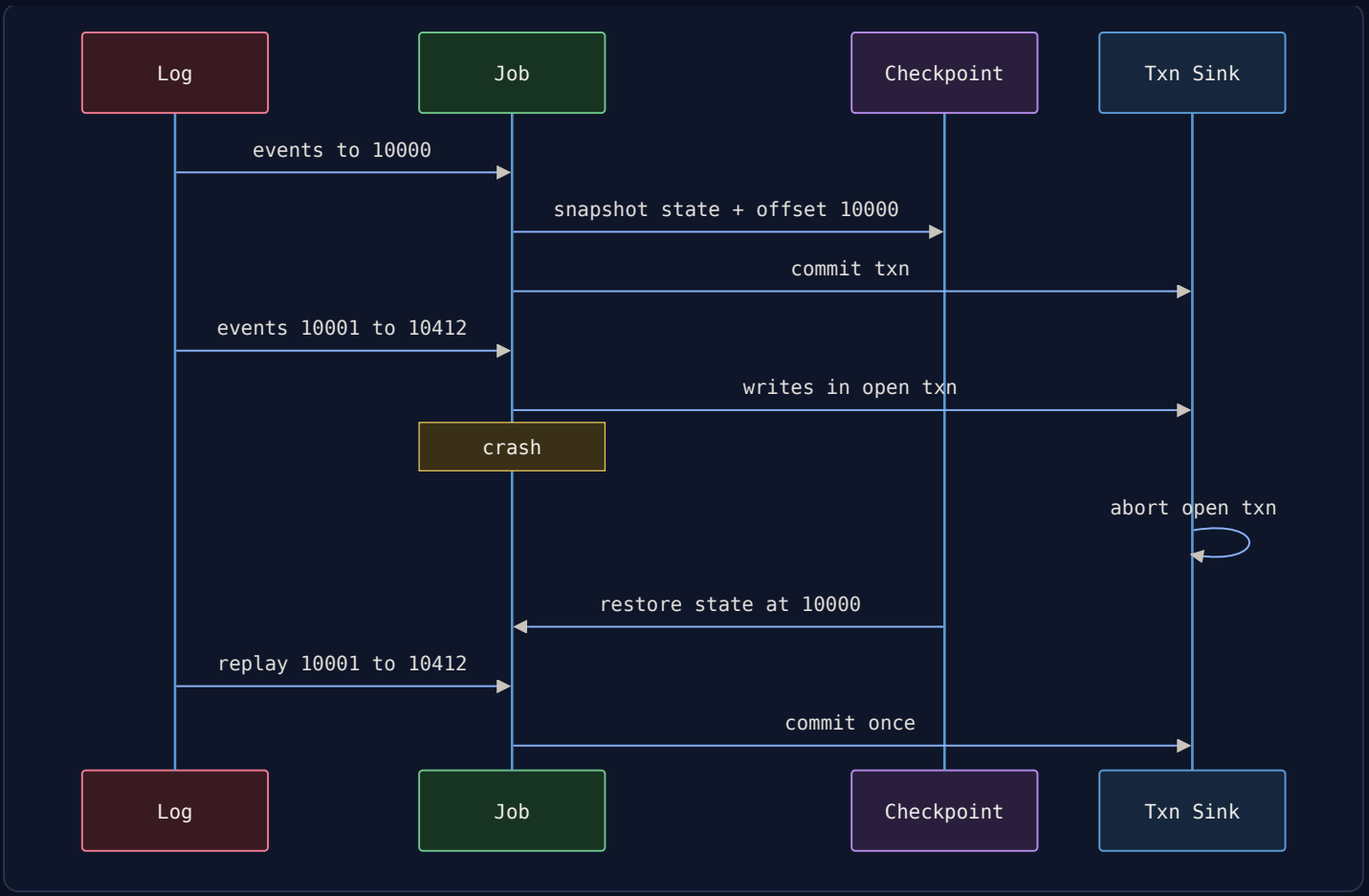
- **"Commit the offset after processing."** At-least-once: a crash between processing and commit replays the batch — month one. The right default, but not enough.
- **"Commit before processing."** At-most-once: a crash between commit and processing loses the batch — month two. Strictly worse for money: loss is silent, while duplication at least triggers audits.
- **"Dedupe in the sink with idempotency keys."** (The **sink** receives results; an **idempotency key** names a write so repeats are ignored.) Module 5's technique covers ~80% of cases — keyed, whole-value writes — but breaks for **aggregates**: on replay the sink can drop the repeated *output*, but the operator's *internal* running count has already incremented again. Correctness needs state, position and outputs tied together; output dedupe doesn't stop state double-counting, and state snapshots don't stop duplicate outputs.

How it works

Exactly-once effect = at-least-once delivery + atomic coupling at two points — the idempotency-key idea stretched across a pipeline.

1. **Coupling point 1 — state with position.** The stream lesson's **checkpoint** saves running counts *atomically with* their log offset, so restore-and-rewind returns both to the same logical moment, and replayed events update a state that doesn't yet include them. **Flink** snapshots a whole multi-operator pipeline consistently using **checkpoint barriers** — markers flowing with the data; each operator snapshots when the barrier arrives, so all snapshots describe the same point.
2. **Coupling point 2 — outputs with position.** Replays reach the sink; two ways to make them harmless:
   - **Transactional sink** (e.g., Kafka). A **transaction** makes writes visible all together or not at all: the job writes outputs *and* commits consumed offsets in one transaction, and downstream readers with `read_committed` see both or neither. A crash mid-window **aborts** the open transaction — partial outputs hidden, offsets unmoved — and the replay commits once. Underneath it's **two-phase commit** inside the log system, with replicated participants and recoverable coordinators.
   - **Idempotent sink.** Deterministic keyed writes: `UPSERT (partner, day) = count` (an **upsert** inserts or overwrites). A replay writes identical values — no transactions, no read-committed delay. Enough for most production pipelines; try it first.
3. **Kafka's two enablers:** the **idempotent producer** tags each append with a per-producer, per-**partition** sequence number so the **broker** drops duplicate appends (an idempotency key by another name); **transactions** make multi-partition writes plus the offset commit atomic.
4. **The boundary.** The guarantee holds **only inside the transactional log ecosystem**. External side effects — an email, a partner API call, a **webhook** — can't be rolled back by a Kafka transaction; Module 5's idempotency keys and receiver-side dedupe still apply.

Crash recovery with checkpoint and transactional sink



Walkthrough with numbers: billing rebuilt with a transactional sink

- **Normal flow:** counts accumulate in checkpointed state; every **2 seconds** a checkpoint completes and the open transaction (outputs + offsets) commits. Invoice tables read only committed data, advancing in 2-second steps.
- **Crash drill:** the job dies at event **10,412**; the last checkpoint covers **10,000**. The open transaction with outputs for 10,001–10,412 aborts, unseen by committed readers; state is restored, the log rewinds to 10,000, the same 412 events replay from the immutable log and commit once. Partner A's count is exactly **987**: at-least-once prevents loss; paired state/offset recovery plus transactional outputs prevent double-counting.
- **The bill:** about **3–5% throughput**, plus commit-interval latency — outputs are as fresh as the checkpoint cadence (2 s). Worth it for invoices. The live dashboard on the *same* stream uses plain at-least-once with idempotent upserts, avoiding the lag. **Two consumers, two guarantees, one stream** — the guarantee belongs to the question (like the late-data policy).

Idempotent sink vs. transactional sink

	Idempotent sink (keyed upserts)	Transactional sink (Kafka txns / 2PC)
Mechanism	A deterministic key; a replay overwrites with identical values	Outputs + offsets commit together atomically; readers only see committed data
Running aggregates	Yes, when the output is "current total per key"	Yes, including outputs that aren't keyed or that span several topics
Freshness	Right away	Moves in steps set by the checkpoint/commit cadence
External systems	Any store that can upsert	Only inside the transactional ecosystem
Complexity	Picking a good key	All of the machinery and its fine points
Reach for it	First — the ~80% default	When being atomic with offsets is a must: money, ledgers

Senior summary: idempotent upserts where outputs are keyed; transactions where outputs must move atomically with offsets; neither reaches external side effects.

Trade-offs

You gain: money-grade correctness through routine crashes, rewinds and deploys — billing, ledgers and audited numbers can run on a stream — and an end to oscillating between at-least-once and at-most-once.

You pay:

- **Throughput and freshness tax** — transaction overhead; read-committed outputs only as fresh as the checkpoint cadence.
- **A bounded guarantee** — it ends at the log system's edge; assuming external calls are covered is a common misuse.
- **Operational subtlety** — debugging aborted transactions, tuning checkpoints, and **barrier alignment stalls under backpressure** (a slow stage slows the barriers too).
- **No protection from wrong logic** — exactly-once runs *incorrect* code exactly once; fixing logic still needs Lambda/Kappa-style fix-and-replay.

When not to use it

- **Approximately-once is honestly fine** — dashboards, trends and ops metrics tolerating ±0.1%: at-least-once with idempotent upserts (or nothing). Save the machinery for auditor-checked numbers.
- **Side-effect-heavy pipelines** — if the main output is emails or partner API calls, transactions can't cover what matters; make external effects idempotent with receiver-side dedupe.
- **A deterministic keyed upsert suffices** — most of the time; transactions are for unkeyed, multi-destination, or offset-atomic outputs.
- **Common misuse:** flipping on "exactly-once" for a pipeline whose sink calls several external APIs — duplicates still reach them.

Real-world examples

- **Kafka EOS** — idempotent producer + transactions + read-committed consumers; made "exactly-once semantics" a product phrase and permanent interview topic.
- **Flink checkpoint barriers + two-phase-commit sinks** — prepare at checkpoint, commit at checkpoint completion (the 2PC lesson's promised example).
- **Spark Structured Streaming** — checkpoints plus idempotent sinks stamped with transaction IDs, at micro-batch cadence.
- **Google Dataflow** — exactly-once processing by default on a managed platform.

- **For AI engineers:** token-usage metering (billing per million tokens) is this incident's pipeline — counter-shaped, high-volume, money — so it runs on this machinery: usage events on the log, checkpointed aggregation, transactional or keyed-upsert sinks feeding billing, while a live dashboard reads the same stream loosely. Eval pass-rate counts and per-**tenant** cost attribution deserve the same care. This closes the write-behind lesson's caveat: anything an invoice or leaderboard depends on needs state, position and output tied together.

In the interview

Trap version: "Can you guarantee exactly-once delivery?" Explain the distinction first (delivery: no; effect: yes, if built), then the machinery. Applied version: "design usage-based billing on a stream."

30-second answer: "Exactly-once delivery is impossible — acknowledgments can be lost, so a sender must choose between resending and risking loss. What you can build is an exactly-once *effect*: at-least-once delivery with duplicates made harmless by two couplings. Operator state is snapshotted with its log offset, so a rewind restores counts and bookmark to the same moment. Outputs move with offsets — one Kafka-style transaction with read-committed consumers, or deterministic keyed upserts a replay overwrites identically; I try upserts first. The boundary: it lives inside the log ecosystem, so an email sent from the pipeline needs its own idempotency key and receiver dedupe. And for a dashboard I'd skip all this — the machinery is for audited numbers."

Likely follow-ups (with answers):

- *"Crash mid-window — recovery?"* — The open transaction aborts unseen; state restores from the checkpoint with its offset; the log replays the gap and outputs commit once. Exact, because restored state and replay offset describe the same moment.
- *"The pipeline also emails a usage alert at 80% of quota — covered?"* — No; an external effect no transaction can undo. Give it an idempotency key like `(partner, quota-threshold, period)` so replays can retry and the sender drops duplicates.

Mistakes that fail candidates: claiming exactly-once delivery exists; sink dedupe for a running aggregate (state still double-counts); transactions on a dashboard; no answer for external side effects.

Self-check:

- **Q1.** The two coupling points, and what breaks with only one? → State ↔ position: without it, a rewind replays events into state already containing them — internal double-counting no sink can repair. Outputs ↔ position (transactional commit or deterministic upserts): without it, replays re-emit results downstream already consumed. You need both.
- **Q2.** Machinery for (a) partner billing, (b) an orders-per-minute ops dashboard, (c) a job whose output is webhook calls to partners. → (a) Full treatment: checkpointed state, transactional sink (or keyed upserts if the output shape allows), read-committed consumers — money justifies it. (b) At-least-once with idempotent upserts on `(metric, minute)`, or accepted approximation — no auditor, no tax. (c) The machinery can't cover external calls: at-least-once processing with idempotent webhooks (event IDs the receiver dedupes, Module 5 retries). The guarantee depends on where output lands.
- **Q3.** Full exactly-once is in place, yet partner B's count is **~2% high** monthly; their mobile SDK re-sends usage events on timeout with a **freshly generated event ID** each time. → The duplicate is created *upstream* of the log: the pipeline correctly counted two differently-identified events describing one API call, indistinguishable downstream. Fix at the source (the idempotency lesson's first rule): the client creates the event ID *before* the first attempt and reuses it on

retries; ingestion (or the first keyed stage) dedupes on it. **End-to-end exactly-once is a chain** — every hop from SDK to invoice must avoid creating duplicates or carry the key exposing them.

Related patterns:

- **Idempotency** — the foundation; its client-generated-key rule covers everything upstream of the log.
- **Stream Processing** — checkpoints provide state-position coupling; this lesson seals the sinks.
- **Two-Phase Commit** — the protocol behind transactional sinks, in a controlled setting.
- **Message Queue** — the immutable, replayable log enabling recounts.
- **Write-Behind** — Module 4's billing caveat resolved: dashboards approximate, invoices use this.

TL;DR

- **Problem:** money-touching streaming aggregates swing between failure modes: at-least-once double-counts on every rewind, at-most-once silently loses events, sink dedupe can't fix re-incremented state.
- **Mechanism:** at-least-once underneath plus two atomic couplings — state snapshotted with log position; outputs committed with offsets (Kafka transactions + read-committed) or written as deterministic keyed upserts (try first).
- **Costs:** a few percent of throughput, freshness tied to checkpoint cadence, subtle operations, a firm boundary at external side effects.
- **Skip it when:** no auditor (dashboards), outputs are external calls (make them idempotent), or a keyed upsert already suffices.

Backpressure

Original lesson: Backpressure on DesignGurus ↗

What it is. When a producer sends faster than its consumer handles, something gives: a buffer fills, data is dropped, or the producer slows. **Backpressure** is *choosing deliberately* which — and, when slowing is the choice, carrying the consumer's "slow down" signal upstream. With no deliberate choice, the default is an unbounded buffer that eventually dies out-of-memory.

Analogy. A chef plating faster than waiters can carry: a heated shelf of limited size (bounded buffer), discarding dishes gone cold (drop), or telling the chef to wait for a free waiter (slow the producer) — never plates piling on the floor forever.

The problem: rate mismatch is normal

You've met it three times: a slow phone filling a gateway's memory (bidirectional streaming), growing queue depth revealing a mismatch (queues), and the rate limiter's **429** at the edge. The rule: **producers outrunning consumers is a normal operating condition, not an exception.** Consumers redeploy, pause for **garbage collection**, hit slow **sinks**, and change speed, so every fast-to-slow link must decide where excess goes.

The default's math: a producer at **5,000 events/s** feeding a consumer at **3,000/s** accumulates **2,000 buffered events every second**; an unbounded buffer *will* run out of memory unless the mismatch stops — a crash on a timetable set by the mismatch.

How it works: exactly three responses

Every system picks at least one per stream:

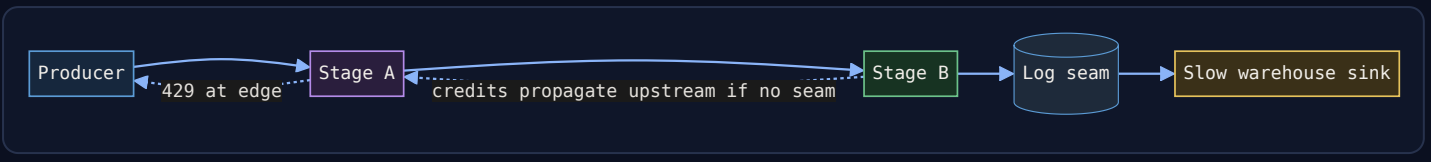
1. **Buffer it (bounded)** — for a *temporary* mismatch: a burst in front of a consumer that can catch up, in a buffer with a fixed maximum. An unbounded buffer isn't a strategy; it's a predictable failure.

2. **Drop it (by explicit policy)** — when old data is worthless: a video call doesn't need 3-second-old frames, metrics can sample, a "current location" replaces the last (**drop-oldest, keep-latest** = a one-slot buffer). Dropping fails only when *accidental*.
3. **Slow the producer** — send the consumer's capacity upstream so the source produces sustainably:
   - **TCP flow control** — the receiver's *window* tells the sender how much it may send.
   - **Kafka's pull model** — consumers fetch each batch when ready, so the consumer's pace *is* the flow rate: backpressure for free, no return signal.
   - **Reactive streams** (a standard async-stream API with built-in backpressure) `request(n)` — the consumer permits *n* more items.
   - **Flink credit-based channels** — a slowing operator's reduced credits propagate stage by stage to the source.

A log between stages. A common design answer: a Kafka topic between producer and consumer — a huge disk-bounded buffer with **retention** — decouples the sides and turns mismatch into **consumer lag**, measurable, alertable and usable to **autoscale** consumers on backlog age. The limit remains (lag beyond retention loses data), but a sudden full-buffer crash becomes a visible, recoverable backlog.

Inside a streaming topology (a stream's chain of stages), backpressure makes the *whole pipeline* run at its slowest stage's speed, and since checkpoint barriers share the data channels, they slow too — so **checkpoint timeouts are often the first visible symptom, even when the cause is stages away** (self-check Q3).

Backpressure propagation and the log seam



Without the log seam, the slow sink's pressure would travel upstream as shrinking credits until it hit the producer as a 429; with it, the sink falls behind alone.

The three responses side by side

	Bounded buffer	Drop (by policy)	Slow the producer
Where the excess goes	Into memory or disk, for a while	Thrown away, on purpose	It is never created
Fits	Bursts in front of a consumer that can keep up	Data where only freshness matters	A mismatch that lasts
Failure mode	A full buffer still needs a backup plan	Dropping the wrong data	Pressure reaches the edge (users notice)
The log's version	The retention window	Lag beyond retention	Pull-based consumption

Real pipelines mix all three — bounded channels inside, logs between stages, drop policies for freshness-only streams, 429s at the boundary — each an explicit policy.

Trade-offs

You gain: predictable memory (every buffer bounded); visible mismatches (lag and buffer fill as operational signals); controlled bursts (buffers absorb short ones, propagation handles long ones).

You pay:

- **Someone upstream feels it.** The slowdown spreads to the system's edge — a throttled upload, a 429. There backpressure *becomes* rate limiting, so choose deliberately where clients meet the signal.
- **Latency under pressure** — with buffers full, every event waits; alive but slow.

- **Whole-pipeline coupling** — one slow sink paces everything and stalls checkpoints unless log seams isolate it.

When dropping is right (and when it never is)

- **Freshness-only data** — live positions, video frames, sensor readings: drop-oldest beats slowing the producer; nobody needs a driver's full history.
- **Sampled observability** — metrics and **traces** are built to shed; **tail sampling** is principled dropping.
- **Never for money or facts** — billing events, orders, anything under exactly-once must stay in a log or slow the source; dropping them is data loss.
- **Common misuse: hidden unbounded buffers** — in-memory queues, unbounded **thread-pool** queues, default async channels: out-of-memory crashes at unknown times. Audit every "unbounded" setting — each is an unmade capacity decision.

In the interview

Direct prompt: "your producer emits 5,000/s but the consumer handles 3,000 — what happens?" (the three responses and the unbounded-buffer failure). Debugging version: "why have our checkpoints started timing out?"

30-second answer: "Something gives: buffer, drop or slow down. The one wrong answer is an unbounded buffer, turning a visible mismatch into a timed out-of-memory crash. Bounded buffers for bursts; deliberate keep-latest drops for freshness-only data; for lasting mismatch, push pressure upstream — pull-based consumers do it for free. That's why a log between stages is the go-to: mismatch becomes lag you can see, alert on and autoscale from. Inside a topology a slow sink paces the whole pipeline and stalls checkpoints. At the edge, backpressure becomes a 429 with Retry-After. The signal must end somewhere; the design choice is where."

Likely follow-ups (with answers):

- *"Why do pull systems get backpressure free?"* — The consumer initiates each exchange, asking only when ready, so its speed is the flow rate and silence is the signal. Push systems need a return channel (credits, windows, 429s).
- *"If every stage slows, where does pressure end up?"* — At the original producer at the edge — a user, partner or device — as a throttled upload, 429 or "retry later." It never vanishes, so choose that boundary deliberately and shed by priority tier when capacity runs out.

Mistakes that fail candidates: "we'd buffer it" with no limit; dropping billing events; not linking checkpoint stalls to a slow sink.

Self-check:

- **Q1.** The three responses, where each fits, and the classic mistake. → Bounded buffering for bursts before a consumer that can keep up; deliberate dropping for freshness-only data and sampled observability; slowing the producer (credits, windows, pull, 429s) for lasting mismatch. The classic mistake: an unbounded buffer, a silent commitment to running out of memory.
- **Q2.** Responses for (a) billing-feeding order events, (b) live driver locations on a map, (c) a 10× burst for 90 s at a consumer sized for steady traffic. → (a) Buffer in the log, never drop (exactly-once territory); scale the consumer on backlog age if the mismatch lasts. (b) Drop-oldest, keep-latest — one slot per driver. (c) A bounded buffer sized for 90 seconds of 10× traffic, with log retention as the deeper version.
- **Q3.** Checkpoints start timing out at **2 PM**; by **2:20** end-to-end lag rises and alerts fire; no deploys; at **1:55** the warehouse sink slowed sharply due to a table lock. → The slowed sink's

bounded input channels filled; credit-based flow control throttled operators one by one upstream; by 2:00 the topology ran at sink speed; barriers sharing those channels slowed and timed out — the first alarm, stages from the cause, with lag alerts ~25 minutes after the cause. Backpressure protected memory as designed; symptoms just surfaced far away. Diagnose with Flink's per-operator backpressure metrics (find the busy stage whose downstream is idle). Contain with a **log between pipeline and warehouse sink**: the sink lags on its own topic (retention-bounded, age-monitored) while the main pipeline and checkpoints run at full speed.

Related patterns:

- **Message Queue** — a log separates stages, turning mismatch into measurable lag.
- **Stream Processing** — propagation inside topologies and the telltale checkpoint stalls.
- **Rate Limiting** — backpressure at the boundary: 429 + Retry-After is "slow the producer" for outside callers.
- **Bidirectional Streaming** — the single-connection version: bounded send buffers, drop-and-resync per socket.

TL;DR

- **Problem:** producers outrunning consumers is normal, and the default (an unbounded buffer) schedules an out-of-memory crash instead of making the mismatch visible.
- **Mechanism:** choose per stream — bounded buffers for bursts, deliberate drops for freshness-only data, slow-down signals upstream (credits, windows, pull, 429s) for lasting mismatch; log seams turn crises into visible lag.
- **Costs:** someone upstream always feels it (design where), latency rises under pressure, the slowest stage paces the pipeline unless seams isolate it.
- **Drop when:** the newest value supersedes older ones (positions, frames) or the stream is designed to be sampled — never for audited facts.

Partitioned Consumption

Original lesson: Partitioned Consumption on DesignGurus ↗

What it is. Processing a stream in parallel *without* losing per-entity order. The stream is split into **partitions** by key (e.g., customer ID): every event with a given key goes to the same partition, and each partition is read by exactly one consumer at a time. The contract: **order per key, parallelism across keys.**

Analogy. Checkout lanes where each family always goes to the same lane: items scan in order within a lane, lanes run in parallel, more cashiers than lanes doesn't help, and a family with twenty trolleys holds up its lane however many cashiers are idle.

The problem

- **Parallel consumers break ordering** — the queue lesson's Q3 refund processed before its charge, because two workers picked up the same customer's events independently.
- **Global order kills parallelism** — one in-order consumer can't scale.
- **Keyed streaming state** (the stream lesson's per-card counts) *needs* every event for a key at one operator. Kafka-style **consumer groups** are the one mechanism behind all three.

How it works

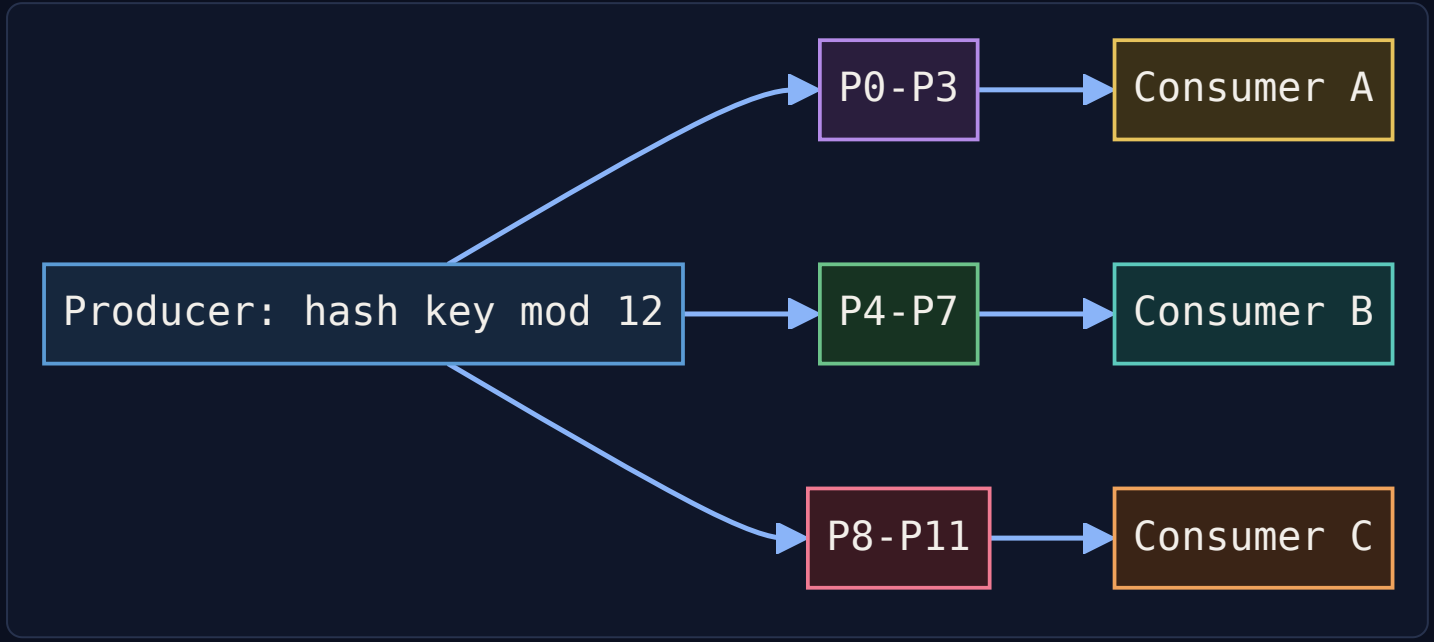
```
topic: orders (12 partitions, keyed by customer_id)

producer:  partition = hash(customer_id) % 12
           → customer 8412's events ALWAYS land in partition 7, in order

consumer group (3 consumers):
   consumer A ← partitions 0-3      each partition has exactly ONE
   consumer B ← partitions 4-7      consumer; each consumer owns its
   consumer C ← partitions 8-11     partitions' offsets (bookmarks)
```

1. **The producer routes by key hash.** A **hash** maps a key to the same number every time, and `% N` keeps the remainder, so `partition = hash(key) % N` puts every event for a key in one ordered partition, in production order. Choose the key like a shard key: keep related events together *and* spread load evenly.
2. **The exclusive lease.** Within a group, exactly one consumer owns each partition at a time — only one reader processes it and advances its position, which preserves order. *Different* groups each receive all partitions independently (pub/sub fan-out).
3. **Offsets per partition.** An **offset** records how far a consumer has read in one partition; committing *after* processing gives at-least-once, so handlers must be **idempotent**.
4. **Rebalancing.** When a consumer joins or dies, partitions are reassigned; processing pauses while ownership, offsets and any local state move (stateful consumers must move state too). Frequent **rebalance storms** usually mean unstable consumers or processing longer than the session timeout (the group declares the consumer dead).
5. **Partition count = maximum parallelism.** 12 partitions support at most 12 active consumers; a 13th idles. Raising the count later (`% 12` → `% 24`) reroutes keys mid-stream, breaking per-key order and splitting keyed state — the streaming **modulo trap**. Choose plenty up front; unused partitions are cheap.

Two familiar problems, in new form:

- **The hot partition.** A **hot key** sends all its volume to one partition — maybe 40% of all work on one consumer. Hashing spreads *keys*, not *weight* (the skew of shards, cache keys and reducers). Remedies: cache reads, split or **salt** the heavy key (a composite key, only if its ordering needs allow), pre-aggregate, or isolate extreme keys (e.g., their own topic).
- **Per-lane head-of-line blocking.** A **poison message** (fails every time) blocks only its partition, not the topic. But in strictly ordered lanes, moving it to a **dead-letter queue** (DLQ) changes the order, so those consumers must **pause and fix** instead.

Key-hashed partitions with exclusive consumer ownership



Partitioned log vs. competing-consumer queue

	Partitioned log (Kafka-style)	Competing consumers (SQS-style; Amazon's simple cloud queue)
Ordering	Strict for each key (each lane)	None — any worker can take any message
Scaling granularity	By partition; the count is fixed at the start	By worker; add one whenever you like
Replay / history	Built in — rewind the offsets	Gone as soon as it's acknowledged
Poison message	Blocks its lane (pause or DLQ, by policy)	Blocks nothing (DLQ it freely)
Fits	Keyed facts: orders, events, state feeds	Independent jobs: renders, emails, resizes

Ruling: **independent jobs → competing consumers; ordered facts → partitioned logs.** Within one key you can't have strict order and unlimited parallelism; the partition count sets available parallelism.

Trade-offs

You gain: strict per-key order *and* horizontal parallelism (impossible with plain competing consumers); free locality (each key's events at one consumer, enabling keyed state); a limited blast radius (a poison message blocks one lane).

You pay: a parallelism ceiling chosen in advance (repartitioning is disruptive); rebalance pauses and storms (liveness tuning, state movement, deploy hiccups); skew exposure (a hot-key lane runs hot while others idle); order only per key — none across keys (rarely needed, often wrongly assumed).

When not to use it

- **Order doesn't matter** — independent jobs (image resizes) fit competing consumers: easier scaling, no lanes, no rebalances.
- **You need global order** — one partition, one consumer, capped throughput. First ask what *really* needs ordering; usually it's per entity.
- **Tiny throughput** — one consumer, one lane.
- **Common misuse:** random or **round-robin** keys while assuming per-entity order — refund-before-charge returns. The key *defines* the ordering scope.

In the interview

Prompt: "process events in parallel but keep per-user order." Follow-ups: partition count, rebalancing, hot keys.

30-second answer: "Partition the topic by the entity key, like customer ID, so each key's events form one ordered lane, and give each partition exactly one consumer in the group — exclusivity guarantees order, lane count gives parallelism. I'd size the partition count up front with headroom, since it's the parallelism ceiling and resizing reroutes keys mid-stream. Two classic pains: rebalance pauses as consumers come and go, and hot partitions when one key dominates — hashing spreads keys, not weight. A poison message blocks only its lane; strictly ordered lanes need pause-and-fix, not dead-lettering. Order-agnostic jobs skip all this and use competing consumers."

Likely follow-ups (with answers):

- *"How many partitions?"* — Work back from consumer throughput: 50,000 events/s ÷ 2,000 per consumer = **25 minimum**, then a generous margin (e.g., 48–96, more if growth is fast) covering 3–5 years of parallelism growth — partitions are cheap, resizing isn't.

- *"You added consumers and throughput stopped improving?"* — (1) Consumers already equal partitions, so extras idle (more partitions means disruptive repartitioning). (2) A hot key in one partition — consumers can't split a partition; manage the skew.

Mistakes that fail candidates: assuming cross-partition order; random keys when order matters; casually planning to "add partitions later" without addressing key rerouting.

Self-check:

- **Q1.** State the contract and which component enforces each half. → "Order per key, parallelism across keys." Deterministic key-hash routing puts each key in one lane; the group's exclusive one-consumer-per-partition lease prevents interleaved reads. Lanes × one consumer each gives the parallelism.
- **Q2.** 50,000 events/s, 2,000/s per consumer, per-customer order, volume doubling every 18 months. Partition count? → Minimum 25; with growth ~50 consumers in 2 years, ~100 in 4. Choose **96–128**: partitions are cheap, resizing is surgery that reroutes keys and breaks per-key order during the switch, and the count caps *future* parallelism. Run 25 consumers with 4–5 partitions each and scale by adding consumers for years.
- **Q3.** Payments keyed by `customer_id`; throughput plateaued going from 12 to 20 consumers, and partition 7's lag is **40×** every other partition's. → A hot partition: one huge enterprise customer's whole volume lands on partition 7 while other consumers idle — **extra consumers could never fix it**, since each partition has one reader. Isolate that tenant (own topic), use a composite key like `customer_id + region` *if* its ordering allows, or pre-aggregate/batch upstream. (Consumers beyond 12 also had nothing to read.) Check lag per partition before total lag.

Related patterns:

- **Message Queue** — per-key ordering on top of the queue's delivery guarantees.
- **Stream Processing** — keyed state works because each key arrives in one place.
- **Sharding** — the same key discipline and skew, applied to stream compute.
- **Dead Letter Queue** — per-lane poison policy: DLQ where order is loose, pause-and-fix where strict.

TL;DR

- **Problem:** parallel consumers wreck ordering, global ordering wrecks parallelism, and keyed state needs each key's events in one place.
- **Mechanism:** hash keys to partitions (one ordered lane per key), one exclusive consumer per partition within a group, offsets per lane; the partition count is a ceiling chosen ahead of time.
- **Costs:** repartitioning surgery (choose with years of headroom), rebalance pauses, hot-partition skew, order only within a key.
- **Skip it when:** order doesn't matter (competing consumers) or throughput is tiny — and never assume an order your keys don't promise.

Inside Kafka

Added lesson (not part of the DesignGurus course). Written from primary sources listed under "Sources" at the end of the lesson.

What it is. A look inside the log that Stream Processing, CDC, Exactly-Once, Backpressure and Partitioned Consumption all assume: how a broker stores a partition, why that's fast, how replicas agree on what's committed, how consumer groups share work, and which settings each guarantee depends

on. (Designing such a system from scratch with capacity numbers is Case 4 in Module 13; Spring client code is in the Spring book's Kafka in five minutes and Producer guarantees.)

> **Analogy.** A paper ledger copied by clerks: the head clerk only writes at the end, two assistants copy new lines, an entry is "on the record" once every keeping-up clerk has it, readers keep their own bookmarks, and old pages are archived after a set time.

The problem: three beliefs, three incidents

The team's Kafka knowledge was three slogans, each costing an incident:

- **"Kafka never loses messages."** Producers used `acks=1` (the leader confirms once *it* has the write). A broker's disk controller failed before followers copied the last ~40 ms: **~800 confirmed orders vanished** (≈20,000 orders/s × 0.04 s).
- **"Replication factor 3 means three copies."** (The **replication factor** is how many copies of each partition exist: a **leader** taking writes and **followers** copying it.) Both followers fell out of sync; the topic kept accepting writes with one in-sync copy; that broker died, an out-of-sync follower became leader, and offsets **went backwards** with a gap of missing events.
- **"Just add consumers."** A consumer taking 6 minutes per batch (limit 5) was declared dead, its partitions moved, it returned, the group rebalanced again — a **rebalance storm** stalling processing for an hour.

None was a Kafka bug; each was a setting nobody had checked.

Why the obvious fixes fail

- **"Set replication factor 3 and relax."** RF is how many copies *can* exist; what protects a write is how many exist *before the producer hears "ok"* — `acks` together with `min.insync.replicas`.
- **"Force every write to disk (fsync)."** Kafka deliberately relies on copies on *other machines* (ideally other **zones** — separate data centres in one region); per-message **fsync** would cut throughput heavily. Durability comes from replication, not one disk.
- **"Turn on exactly-once."** Kafka's exactly-once covers *read from Kafka, process, write back to Kafka*; it won't stop your consumer sending an email twice. External effects need the Idempotency and Exactly-Once Semantics machinery.

How it works

1. Storage: a partition is a folder of segment files. A **topic** splits into **partitions**, each an ordered append-only log with its own offsets (0, 1, 2, …), stored on a broker as **segments**:

- `00000000000000500000.log` — records in **record batches** (base offset, timestamps, a **checksum** for corruption detection, producer ID and sequence numbers for idempotence); the filename is the segment's first offset.
- `.index` — a **sparse offset index**, an entry roughly every 4 KB mapping offset → byte position. To find offset 500,123 Kafka **binary-searches** the index, jumps near it and scans a few KB.
- `.timeindex` — timestamp → offset, so a consumer can "rewind to 09:00".
- Only the newest (**active**) segment is written; at its size limit (1 GiB default) or age it closes and a new one starts.

Retention works on whole segments. `cleanup.policy=delete` deletes closed segments past the retention time (7 days default) or a size cap — a cheap file delete, no per-message work. `cleanup.policy=compact` has a background cleaner keep only the **latest record per key**, with an empty-value **tombstone** eventually removing the key — how Kafka stores consumer offsets and keeps CDC reference tables small (Log compaction in DDIA). **Tiered storage** (production-ready since Kafka 3.9) moves closed segments to object storage, so long retention no longer needs big broker disks.

2. Why it is fast. Five ways of avoiding work:

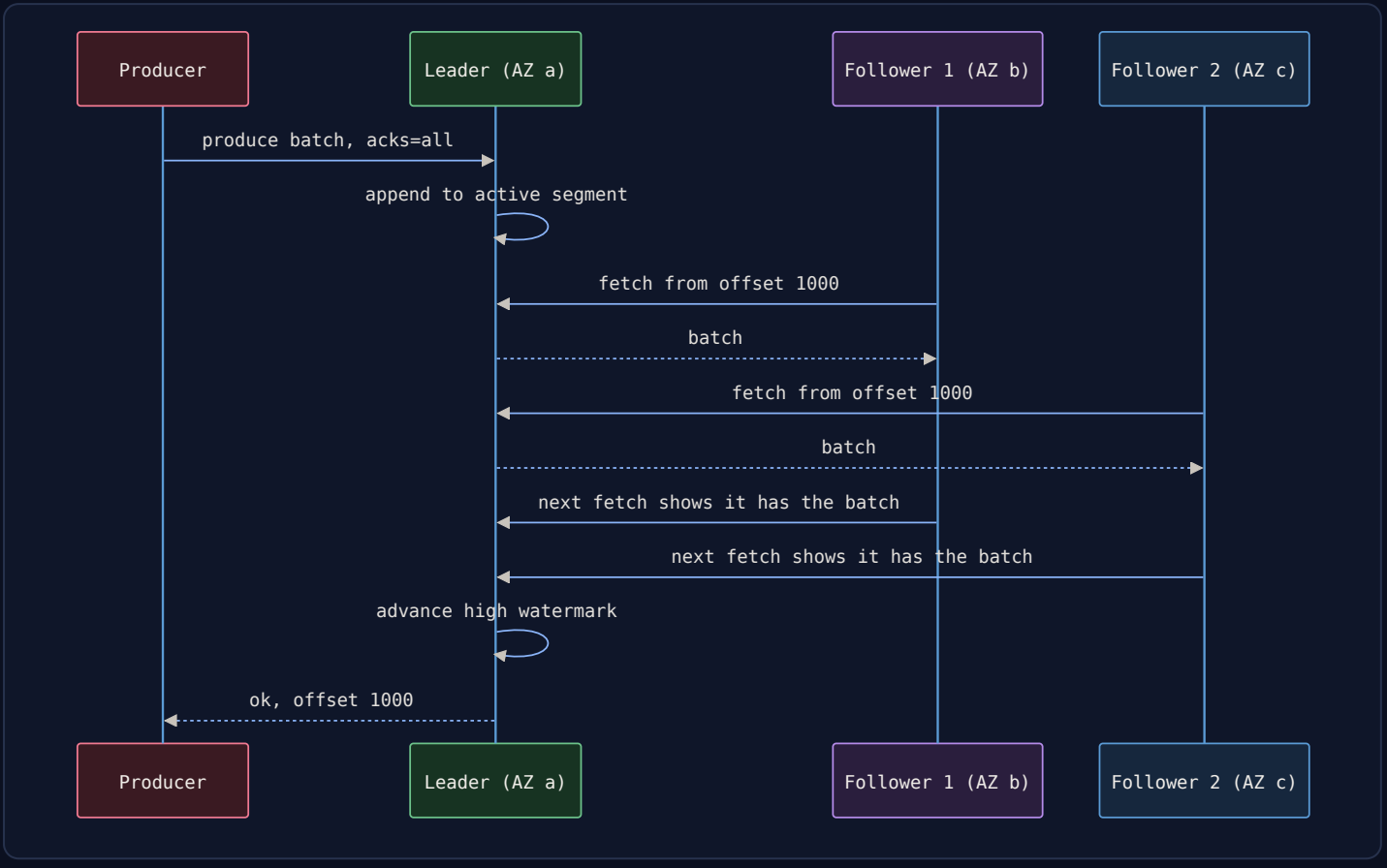
- **Sequential I/O** — appends and forward reads are a disk's fastest operations (hundreds of MB/s even on spinning disks).

- **The OS page cache** — no big in-process cache; up-to-date consumers read recent data from kernel memory.
- **Zero-copy** — bytes are never modified, so `sendfile` moves them from page cache to **socket** without passing through the application (**TLS** needs them in user space, so encrypted clusters lose this and need more CPU).
- **Batching and compression end to end** — producers fill per-partition batches (waiting `linger.ms`) and compress; brokers store as-is; consumers decompress. Fewer requests, 3–5× fewer bytes for text-like data.
- **Pull with long polling** (the server holds a request until data arrives or a timeout) — "everything after offset X, up to 1 MB, wait up to 500 ms"; progress is one number per partition.

3. Replication: leader, followers, ISR and the high watermark. Each partition has one **leader** (serving all writes and, by default, reads) and followers on other brokers — in different **availability zones** (AZs) when brokers are rack/zone-labelled. Followers *fetch* from the leader like consumers.

- **LEO (log end offset)** — the next offset a replica will write; per replica.
- **ISR (in-sync replica set)** — the leader plus followers caught up within `replica.lag.time.max.ms` (30 s default). A slow follower leaves the ISR, so it can't hold back commits, and rejoins after catching up.
- **High watermark (HW)** — the highest offset every ISR member has; records below it are **committed**. Consumers read only up to the HW, so they never see a record a leader **failover** could erase.
- **Leader epochs** — each new leader increments an epoch; a returning old leader uses it to find where its log diverged and **truncates** its uncommitted tail.
- **The controller** — a small **quorum** of controller nodes (KRaft mode, using the **Raft consensus algorithm**; **ZooKeeper**, the coordination service Kafka once depended on, was removed in Kafka 4.0) stores metadata, detects dead brokers and elects partition leaders from the ISR.

A produce with acks=all: the producer is answered only after every in-sync replica has the batch



Each follower's *next* fetch tells the leader how far it got; once every in-sync replica has the batch, the HW moves past it — committed, visible, acknowledged.

4. "Can Kafka lose messages?" — every path, and its setting.

Where	How data is lost	The setting that prevents it
Producer	`acks=0` or `acks=1` : "ok" before followers have it; leader dies	`acks=all` (the default since Kafka 3.0)
Producer	Send failed after retries, but the app ignored the error callback	Handle send failures: retry, alert, or stop
Producer	Retries create duplicates or reorder batches	`enable.idempotence=true` (default since 3.0)
Broker	ISR shrinks to 1; that one copy is lost	`min.insync.replicas=2` with replication factor 3 — writes are *refused* instead of accepted with one copy
Broker	An out-of-sync follower becomes leader, discarding committed records	`unclean.leader.election.enable=false` (the default)
Broker	All replicas in one zone; a power loss hits before the page cache is flushed	Spread replicas across zones (rack awareness)
Consumer	Offset committed (automatically or by hand) *before* processing; crash in between	Commit *after* processing; idempotent handlers
Retention	A consumer falls behind by more than the retention time; old segments are deleted unread	Alert on **lag age** well before retention (e.g. at 50%)

The safe default set: **replication factor 3 across 3 zones, `min.insync.replicas=2`, `acks=all`, idempotence on, unclean election off, commit after processing, alert on lag age** — losing one broker means no loss *and* no downtime; losing two makes writes fail loudly rather than silently.

5. Consumer groups, inside. Each group has a **group coordinator** — the broker leading the partition of the internal `__consumer_offsets` topic its name hashes to. Members send **heartbeats** to it; committed offsets go to that compacted topic.

- **Two timers decide "dead".** `session.timeout.ms` (45 s default): no heartbeat → the process is gone. `max.poll.interval.ms` (5 minutes default): no `poll()` call → the consumer is stuck and removed even though its heartbeat thread lives. That timer caused the rebalance storm: smaller batches (`max.poll.records`) or slow work on a thread pool.
- **Rebalancing styles.** The original *eager* protocol made every member surrender every partition on each change — stop-the-world. **Cooperative (incremental) rebalancing** moves only partitions that must move. **Static membership** (`group.instance.id`) lets a restarting **pod** keep its partitions if it returns within the session timeout, so **rolling deploys** don't rebalance. The **new consumer group protocol** (KIP-848 — a **KIP** is a Kafka Improvement Proposal; GA in Kafka 4.0; opt in with `group.protocol=consumer`) moves assignment to the broker and makes every rebalance incremental.
- Per-key ordering, partitions as the parallelism ceiling and hot partitions are the Partitioned Consumption lesson.

6. Exactly-once, inside. Two layered mechanisms:

- **Idempotent producer.** Each producer gets a **producer ID** and numbers its batches per partition (**sequence numbers**); the leader remembers recent sequence numbers per producer, dropping retried batches already written and rejecting out-of-order ones — why idempotence allows at most 5 in-flight requests per connection.
- **Transactions.** A producer with a `transactional.id` (a stable name surviving restarts) can write to several partitions *and* commit a consumer group's offsets atomically. A **transaction coordinator** tracks progress in the internal `__transaction_state` topic and on commit writes **commit markers** into every touched partition. `isolation.level=read_committed` consumers read only up to the **last stable offset**, never seeing aborted or open transactions' records. When a restarted instance registers the same `transactional.id`, its **epoch** is bumped and the old "zombie" instance is **fenced** — its writes rejected.

Together: **read-process-write exactly-once within Kafka** — what Kafka Streams enables with `processing.guarantee=exactly_once_v2`. The Exactly-Once Semantics boundary applies: database writes and emails are outside the transaction and must be idempotent or transactional themselves.

7. How message brokers evolved (why Kafka looks the way it does).

Generation	Examples	Model	Strength	Weak spot
Enterprise queues (1990s)	IBM MQ (MQSeries, 1993), JMS (Java Message Service) brokers	Queue manager; per-message acknowledgment; transactions	Reliable delivery of individual business messages	Vertical scaling; messages gone once consumed
Smart brokers (2007 →)	RabbitMQ (AMQP, an open messaging protocol)	Exchanges route messages to queues; per-message acks; priorities, delays	Flexible routing, task queues	Throughput per queue; no replay (streams added later)
The distributed log (2011 →)	Apache Kafka (from LinkedIn)	Partitioned, replicated append-only log; consumer offsets	Huge throughput, replay, many independent readers	Per-message routing, priorities and delays need extra topics
Separated storage (2016 →)	Apache Pulsar (from Yahoo)	Stateless brokers over a separate storage layer (BookKeeper)	Independent scaling of serving and storage; multi-tenancy; built-in geo-replication	More components to run

Newer designs keep Kafka's protocol but change engine or storage (Redpanda in C++; systems writing segments straight to object storage for cheap retention at some latency cost). Underneath all: **the log is the foundation.**

Walkthrough with numbers: the order topic, hardened

- **Setup.** `orders`: 48 partitions, RF 3, one replica per zone, `min.insync.replicas=2`, `acks=all`, idempotence on, 7-day retention. Peak 20,000 orders/s × 1 KB = **20 MB/s** in, **60 MB/s** written cluster-wide with replication.
- **Latency cost of `acks=all`.** Followers ~1–2 ms away across zones, producers lingering 5 ms: **p99** produce latency ≈ **10–15 ms** vs. ~5 ms with `acks=1` — the price of not losing the 800 orders.
- **A broker dies.** The controller elects new leaders from the ISR within **seconds**; producers retry (idempotence drops duplicates); nothing committed is lost; affected ISRs are now 2 — at `min.insync.replicas`, so writes continue.
- **A second broker dies.** ISR-1 partitions **reject writes** (`NotEnoughReplicas`) — a visible outage instead of silent single-copy data.
- **The rebalance storm, replayed.** `max.poll.records` 500 → 50 makes a poll loop ~40 s instead of 6 minutes; static membership keeps partitions through deploys. Rebalances per deploy: **~20 → 0**.
- **The retention trap.** Alert when the oldest unconsumed record exceeds **3.5 days** (half of retention), long before unread data is deleted.

Trade-offs

You gain: very high throughput (sequential I/O, batching, zero-copy); durability without single-disk dependence; replay and many independent consumer groups; precise, documented guarantees — once set.

You pay: guarantees are **opt-in combinations** (`acks`, min ISR, zone placement *and* consumer commits — one wrong setting silently weakens them); **latency for safety** (`acks=all` across zones adds ms, transactions more); **operational weight** (partitions, rebalances, retention, upgrades — or a managed bill); **a log, not a smart queue** (per-message routing, priorities, delays and acks are awkward).

When not to use it

- **A modest task queue** needing priorities, delays or per-message retries — RabbitMQ or SQS is simpler.
- **Request-reply** between services — a direct call; a log adds latency and bookkeeping for nothing.

- **Tiny volume, no replay need** — a table plus polling, or a managed queue.
- **Common misuse:** querying a topic like a database ("find order 123") — logs are read sequentially; build a keyed store from the log.

Real-world examples

- **LinkedIn** built Kafka for activity data and metrics; by 2019 it reported about **7 trillion messages a day**.
- **Managed services:** Confluent Cloud, Amazon MSK, Azure Event Hubs (Kafka-compatible endpoint), Aiven; **Kafka-compatible engines:** Redpanda.
- **Kafka Streams and Apache Flink** build keyed state and exactly-once pipelines on these internals.
- **For AI engineers:** Kafka carries feature-update streams into online feature stores and logs every model request and response for evaluation; compacted topics hold "latest embedding version per document", so a vector index can be rebuilt by replaying one topic.

In the interview

Prompts: "can Kafka lose messages?", "why is Kafka fast?", "what is the ISR?", "how does Kafka do exactly-once?" — answer with mechanisms and settings, not slogans.

30-second answer: "A partition is an append-only log in segment files with a sparse offset index; retention deletes whole segments or compacts to the latest value per key. It's fast thanks to sequential I/O, the page cache, zero-copy sends and compressed batches end to end. Each partition has a leader and fetching followers; the ISR is the caught-up set, and the high watermark — what every ISR member has — bounds what consumers see. Kafka loses data if you let it: `acks=1`, `min.insync.replicas=1`, unclean leader election, single-zone replicas, or committing before processing. Safe set: RF 3 across zones, min ISR 2, `acks=all`, idempotence on, commit after processing. Exactly-once is idempotent producers plus transactions with read-committed consumers — inside Kafka only."

Likely follow-ups (with answers):

- *"Why no fsync per message?"* — Durability comes from cross-zone copies before the "ok"; a zone power loss loses only that zone's unflushed data. Per-message fsync costs throughput for little extra safety (forceable per topic).
- *"Consumers keep rebalancing — debug it."* — Find which timer fires: session timeout (crashes, long GC) or `max.poll.interval.ms` (slow processing per poll). Smaller batches or off-thread work, cooperative rebalancing, static membership.

Mistakes that fail candidates: "RF 3 means no data loss" without `acks` and min ISR; claiming Kafka exactly-once covers database writes; assuming cross-partition order; not knowing consumers read only up to the high watermark.

Self-check:

- **Q1.** Define LEO, ISR and high watermark; why only read up to the HW? → LEO: a replica's next write offset. ISR: leader plus followers within the lag limit. HW: the highest offset every ISR member has (committed). Anything above it could be truncated after a leader change, so reading it could show events that later vanish.
- **Q2.** RF 3, `min.insync.replicas=2`, `acks=all`: one broker down, then a second. → One down: ISR 2, writes continue, nothing lost. Two down: ISR 1 < 2, writes rejected — nothing is acknowledged with one copy, and all acknowledged data is on the survivor. Availability traded for durability deliberately.

- **Q3.** A billing consumer auto-commits every 5 s and writes to a database; after a crash some invoices are missing and some duplicated. → Offsets committed before the DB write → skipped (missing); rows written whose offsets weren't committed → re-read (duplicates). Commit after the write and make it idempotent (upsert by event ID), or store offsets in the same DB transaction.

Related patterns:

- **Partitioned Consumption** — the consumer-side contract these internals implement.
- **Exactly-Once Semantics** — the end-to-end boundary around Kafka's idempotence and transactions.
- **Write-Ahead Log** — the same append-first idea as a distributed product.
- **Change Data Capture** — compacted topics carry table snapshots; CDC streams are Kafka topics.
- **Case 4 in Module 13** — the same system designed from scratch, with capacity numbers.

TL;DR

- **Problem:** teams run Kafka on slogans — "never loses messages", "RF 3 is safe", "exactly-once" — each hiding a setting that decides the real guarantee.
- **Mechanism:** segmented append-only partitions with sparse indexes; speed from sequential I/O, page cache, zero-copy and compressed batches; leader/follower replication with ISR, high watermark and leader epochs; consumer groups with a coordinator and two liveness timers; idempotent producers and transactions for exactly-once inside Kafka.
- **Costs:** safety is a settings combination (RF 3 across zones, min ISR 2, `acks=all`, idempotence, commit after processing, lag-age alerts), extra latency for durability, real operational work.
- **Skip it when:** a small task queue with priorities and delays, request-reply, or tiny volume with no replay need.

Sources: J. Kreps, N. Narkhede, J. Rao, "Kafka: a Distributed Messaging System for Log Processing", NetDB 2011; J. Kreps, "The Log: What every software engineer should know about real-time data's unifying abstraction" (2013); Apache Kafka documentation — Design (persistence, efficiency, replication), Configuration, and KIP-98 (exactly-once delivery and transactions), KIP-101 (leader epochs), KIP-345 (static membership), KIP-405 (tiered storage), KIP-429 (cooperative rebalancing), KIP-500 (KRaft), KIP-848 (new consumer group protocol); Apache Pulsar documentation (architecture overview); RabbitMQ documentation (AMQP 0-9-1 model); LinkedIn Engineering, "How LinkedIn customizes Apache Kafka for 7 trillion messages per day" (2019); Kleppmann, *DDIA* ch. 11 (Partitioned Logs).

Capstone - Clickstream to Dashboard

Original lesson: Capstone - Clickstream to Dashboard on DesignGurus ↗

What this capstone is. A step-by-step build of the store's analytics platform — from ingesting clickstream and order events to a live dashboard, fresh fraud features and audited finance reports — using all seven patterns, including failure recovery and peak behavior.

Analogy. One water source for three customers: a fire hydrant (now, a little dirt is fine), a hospital (certified pure, can wait for treatment), a garden sprinkler (quick, roughly clean). You don't pay hospital-grade treatment for the sprinkler.

The design rule for every stage: match each implementation to a named consumer's **staleness budget**, each correctness guarantee to its question, and each partition key to the ordering scope actually needed.

The brief

- **In:** the **clickstream** (page views, searches, cart events) at about **30,000 events/s at peak** from web and mobile, plus order events (since Module 2).
- **Out — three customers, three budgets:**

    - **Live ops dashboard** — orders per minute, conversion, top products. Staleness: *seconds*. Accuracy: honest but forgiving.
    - **Daily business reports** — revenue by category for finance. Staleness: *daily*. Accuracy: *audited*.
    - **Fresh fraud features** — card velocity (the stream lesson's example). Staleness: *sub-second*.
- **Incidents to prevent** (all from earlier lessons): dashboard disagreeing with finance; double-billing after a rewind; refund before charge; one slow warehouse sink stalling everything.

Stage 1 — Ingestion: keys and event IDs

Decision: two topics, each keyed by its ordering scope.

- **Clickstream keyed by session ID** — funnel and journey analysis needs a session's events in order, in one partition.
- **Orders keyed by customer ID** — a customer's charge is always handled before their refund.
- **Overprovision partitions** (e.g., **96** for clickstream), since resizing reroutes keys; the count caps future parallelism (Stage 6 depends on it).

Decision: client-generated event IDs, created before the first send and reused on every retry. Mobile **SDKs** resend on flaky networks; a fresh ID per retry makes duplicates indistinguishable from real events downstream (Exactly-Once Q3). Duplicate prevention starts in the SDK.

Stage 2 — Enrichment: getting reference data to the stream

Clicks carry `product_id`; the dashboard wants names and categories from the catalog database, so the job joins the stream with **reference data** (slowly changing lookup data). *How does the catalog reach the stream and stay fresh?*

Decision: CDC into local keyed state. **Debezium** publishes catalog changes to a **compacted topic** (latest event per key only); the job loads it into a per-partition lookup keyed by product ID, so enrichment is a local-state lookup.

- *Why not query the catalog per event?* The stream lesson's rejected per-event query (hundreds of expensive queries/s on production).
- *Why not a nightly dump?* A day stale, and another drifting copy. With CDC, renaming "Gadgets" to "Electronics" reaches the stream **within seconds**, and the same feed serves cache invalidation and the search index — one integration, many readers.

Stage 3 — Processing: time, windows, and three late policies

Mobile events can arrive minutes late; the dashboard counts per minute, fraud per card per 10 minutes.

Decision: event time with watermarks. Arrival-time counting measures the network: after a carrier outage, delayed events land at once as a fake spike triggering rate alerts. Analytics jobs use event time with a watermark about **60 seconds** behind.

Decision: a late-data policy per consumer (one stream, three policies):

- **Dashboard: drop** late events — its per-minute chart doesn't revise past minutes, and a seconds budget tolerates small inaccuracy.
- **Reports: accept amendments** — finance's numbers become complete eventually, with same-day restatements accepted.
- **Fraud: process late events** — a late alert is still worth having.

Stage 4 — Correctness tiers: the guarantee belongs to the question

Dashboard counts and finance revenue come from the same events. *Which sink machinery does each get?*

- **Dashboard → idempotent keyed upserts** `(metric, minute) = value`. Replays (every deploy's restore-and-rewind — "every deploy rehearses the crash") overwrite identical values harmlessly; no transactions or read-committed delay, matching its forgiving budget.
- **Finance revenue → full exactly-once:** checkpointed state paired with offsets, a transactional sink, read-committed readers.
- **Fraud features → idempotent upserts**, with the fraud decision still double-checking the authoritative source.

- *Why not transactions everywhere?* Cost and latency without improving the two consumers that don't need it.

Stage 5 — The reports: refusing accidental Lambda

The usual move — a separate nightly warehouse batch for finance — means two implementations of the same metrics: the Lambda & Kappa lesson's accidental Lambda, sure to drift.

Decision: one logic source. Metric definitions (revenue calculation, return adjustments) live in **one shared library**; the streaming job computes them continuously, and the "daily report" is a view of those aggregates **finalized at day close**.

Where does the warehouse fit? Fed by CDC and the same topics, it serves ad-hoc analysis, **ML training** and batch-shaped work (multi-year joins, **cohort studies** comparing groups such as same-month signups). It stores the stream's outputs and raw history but **never re-implements** the dashboard metrics.

When a definition changes: the Kappa move — replay from an earlier offset with new logic, verify, switch (the migration playbook). **Scheduled backfills**, borrowed from Lambda, bound any streaming error's lifetime. Dashboard and finance agree because each metric has one implementation.

Stage 6 — Black Friday: backpressure by design

Peak triples input to **90,000 events/s**; mid-morning the warehouse sink (the table-lock incident's) slows again. *Follow the pressure.*

1. **The log absorbs the burst.** Producers keep writing at 90k/s, consumers process sustainably, and the difference becomes monitored **lag** by age.
2. **Autoscaling on backlog age** adds consumers within minutes, up to Stage 1's partition count — why those partitions were bought.
3. **The warehouse sink reads its own topic** — a **log seam**. When its table lock hits at **10:40**, only the warehouse consumer falls behind; dashboard, fraud and finance run at full speed and checkpoints aren't held up.
4. **Who feels what:** one consumer's backlog grows on a dashboard; nothing else changes. No facts dropped, bounded buffers prevent memory exhaustion, and on-call investigates one lagging sink, not a dead pipeline.

The final architecture

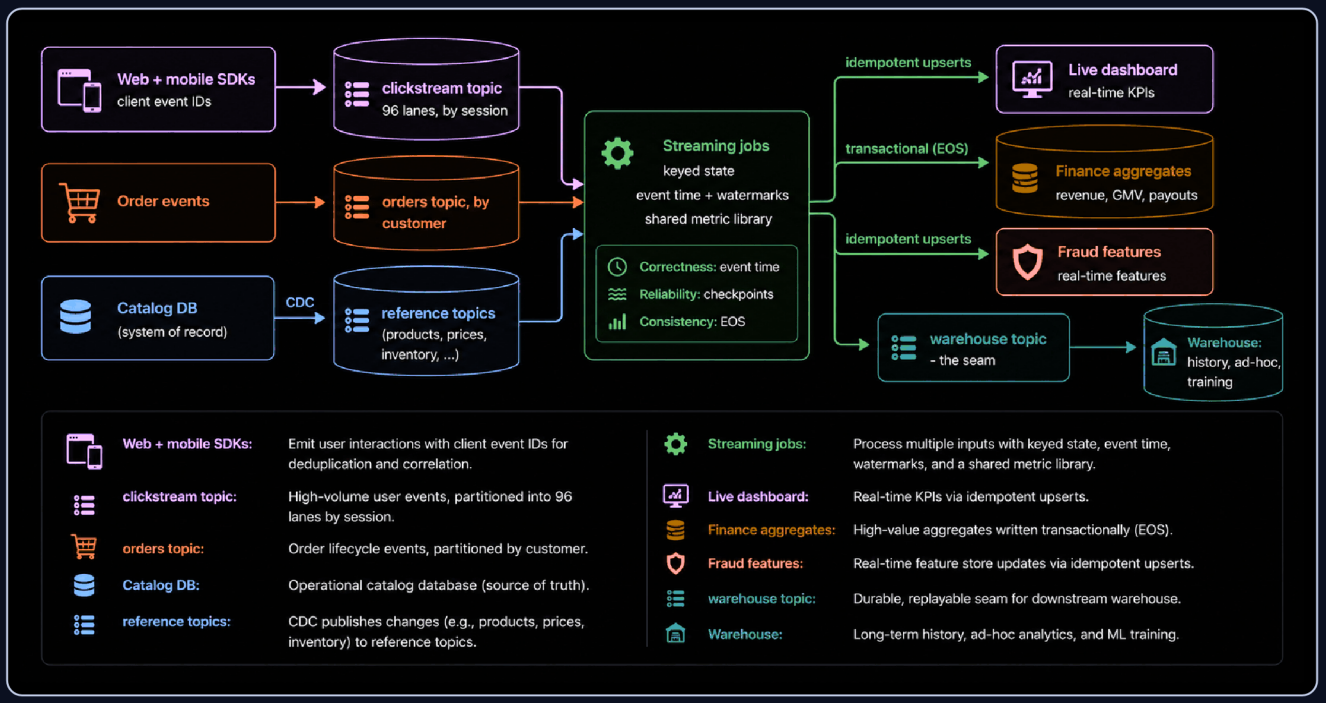


Figure — Clickstream: final architecture (course diagram)

Web and mobile SDKs (stamping client event IDs) write to a 96-lane session-keyed clickstream topic; orders go to a customer-keyed topic; the catalog DB feeds reference topics via CDC. Streaming jobs

(keyed state, event time + watermarks, the shared metric library, checkpoints, exactly-once) write idempotent upserts to the dashboard and fraud features and transactional (EOS) writes to finance aggregates; a separate warehouse topic — the seam — feeds the warehouse for history, ad-hoc queries and training.

Debrief: the module, applied

Decision	Lesson used	Incident it prevents
A key for each topic, and extra partitions	Partitioned Consumption	Refund-before-charge; resize surgery
RF 3 across zones, min ISR 2, `acks=all`, commit after processing, lag-age alerts	Inside Kafka *(added lesson)*	Confirmed events lost on a broker failure; unread data deleted by retention
Client event IDs created before the first send	Exactly-Once (Q3)	Duplicates created upstream of every guarantee
Reference data brought in via CDC into keyed state	CDC	Dumps, drift, a DB query per event
Event time; three late policies	Stream Processing	Counting the network instead of the users
Upserts for forgiving consumers, transactions for audited ones	Exactly-Once	Over-billing and over-engineering
One logic source; the warehouse stores but never recomputes	Lambda & Kappa	The same question answered by two authors
Log seams; lag instead of loss; scaling on backlog age	Backpressure	One slow sink stalling everything

What the design deliberately **didn't buy** matters as much: no dashboard transactions, no streaming for cohort studies, no global ordering — MapReduce's restraint rule, generalized. **Every guarantee, freshness tier and ordering promise was purchased for a named customer.**

Interview one-liner

Key each topic by its ordering scope with plenty of partitions, stamp event IDs on the client, enrich from CDC-fed local state, count in event time with per-consumer late policies, spend exactly-once only on audited outputs, share one metric definition between stream and warehouse, and put log seams before slow sinks so bursts become lag, not outages.

Extend it yourself (practice questions)

- **Sessionization:** product wants funnel analysis (browse → cart → checkout). Design session windows: what defines the inactivity gap, what per-session state grows, and what happens to a session spanning a deploy?
- **Multi-region:** clickstream from two continents, regional logs, one global dashboard. Where do you merge, and what happens to event-time watermarks when one region's stream falls far behind?
- **For AI engineers:** Stage 4's fraud features now also feed training (the feature store's offline path). Using Lambda & Kappa's skew diagnosis: what guarantees "card velocity" at training time matches serving time? (Hint: Stage 5's single shared metric library — one definition run in both modes prevents training/serving skew.)

Module 11 — AI-Era Systems (LLM and ML Infrastructure)

Module Introduction

Original lesson: Module introduction on DesignGurus ↗

What this module is about. The plumbing under AI products: feature stores for classic ML, fleets of **GPUs** (graphics processors that do the heavy maths of running models) serving large language models (LLMs), gateways in front of model vendors, caches that match on meaning, vector indexes, and retrieval-augmented generation (RAG). The message: **nothing here is architecturally new**. Every pattern appeared in earlier modules; what changed is the "physics" they run under.

- **LLM gateway** = Module 8's **API gateway** (the single front door for auth, routing and rate limits) turned outward, toward model vendors.
- **Semantic caching** = Module 4's caching, with "keys are equal" replaced by "meanings are similar."
- **Vector database sharding** = Module 3's sharding, applied to a different kind of index.
- **GPU auto-scaling** = Module 6's auto-scaling, under far tougher limits.
- **Feature store** = CQRS (separate write and read models) plus the data module's "one question, two implementations" lesson, applied to ML.

The course claims roughly **60 recurring shapes** underlie all of system design; this module tests that claim on the newest stack.

The three constants that changed. Every lesson is driven by at least one:

1. **Work is counted in tokens, and requests differ hugely in size.** A *token* is a piece of text (~3/4 of an English word). One LLM request may hold 10 tokens and the next 2,000, so one request can cost **100x** its neighbour. A **load balancer**, **rate limiter**, or capacity plan that counts *requests* is silently wrong (building blocks: System Design Fundamentals). The true unit of work is the token.
2. **Workers are costly and slow to boot.** A GPU machine costs ~**100x** a web server. A new **replica** must load tens of gigabytes of **weights** (the numbers holding what the model learned) before serving, so cold starts last *minutes*. Reactive scaling stops working, and "idle vs. cold" becomes a money decision. The **KV cache** (below) lives in GPU memory, so later turns of a conversation prefer the *same* replica: **session affinity**, which the **stateless** web spent a decade removing, is back.
3. **Equality turned into similarity.** Classic caches and indexes answer only on exact key match; AI systems ask "what is *near* this?" Fuzzy lookups change what a cache hit means, how indexes behave, and what a "correct" answer is. This is the one truly new area, yet once the key is redefined, the old rules still apply.

Four terms used throughout (defined once, here):

- **Token:** the unit of text a model works in (~3/4 of a word). Cost, compute and rate limits are measured in tokens.
- **KV cache:** the per-conversation working memory a model builds as it reads and writes. It sits in GPU memory, can be as large as the weights, is costly to rebuild, and lives only for the request or conversation.
- **Embedding:** a vector of numbers capturing the *meaning* of text as a position in space; nearby points mean similar things. Semantic caching, vector search and RAG all rest on embeddings, which turn "how similar?" into maths.
- **TTFT (time to first token):** the wait before the first word of the reply appears; in a streaming UI it is the delay users notice, so it is the main SLO (service level objective, the committed latency target) of the **inference** tier (running a trained model to produce an answer).

The seven patterns

The situation	The pattern
Training and live predictions must get exactly the same inputs	Feature Store
Workers are expensive, but each request is small	Model Serving & Batching
A new worker needs minutes before it can serve	GPU Auto-Scaling
A single entry point for many model vendors	LLM Gateway
The same question asked in many different wordings	Semantic Caching
An index built on meaning grows too big for one server	Vector DB Sharding
Answers should come from documents, not from memory	RAG Pipeline

How to read the module. Feature Store continues from the data module's capstone (training/serving skew). Model Serving lays out the inference economics the later lessons assume. GPU Auto-Scaling handles the hard case Module 6 deferred. LLM Gateway is where provider calls are routed, metered and failed over (the most-requested lesson after CDC). Semantic Caching and Vector DB Sharding are the similarity versions of Modules 4 and 3. RAG Pipeline ties everything together, and the capstone builds a production RAG chatbot drawing on every module.

Short on time? LLM Gateway and RAG Pipeline first (probed hardest in AI-infra interviews), then Semantic Caching.

Before you start, pick an AI product you know and ask:

1. What does one request cost? Do rate limits, capacity plans and prices allow for the next request being 100x more expensive?
2. When traffic surges, how many minutes until new capacity truly serves users, and who pays during that window?
3. If two people ask the same question in different words, does the second pay full cost again?

Feature Store

Original lesson: Feature Store on DesignGurus ↗

What it is. Each model input (a "feature", e.g. "orders this card made in the past hour") is **written once, as code**. The platform runs that one definition over historical data to build training sets and over live data to feed production. One definition means production values equal training values.

Analogy. A restaurant chain where the test kitchen and every branch cook from one master recipe card, instead of each writing its own version.

The problem (the incident). A fraud model catches **40% more fraud** than the old rules offline, yet three weeks into production it does *worse* than those rules, with no change to model or weights. The cause is one feature, `card_velocity_1h`:

- **Training side:** warehouse SQL with *event-time* windows (grouped by when orders happened), full history, late events counted, NULLs (SQL's missing value) converted to 0.
- **Serving side:** a hand-coded Java service on **Redis** (a fast in-memory key-value store) with near *processing-time* windows (grouped by arrival), late events dropped, NULLs passed through.

The model trained on one version and was served the other: **training/serving skew**. It fails *silently*: nothing errors, decisions just get worse. It is the Lambda architecture's "one question, two implementations" problem, now hurting model quality. Side effects: each new feature took **3 weeks** (scientist writes SQL, engineers rewrite it), and the growth team was coding a *third* copy because nobody could find the second.

Why the obvious fixes fail

- *"One team owns both implementations and reviews closely."* Two codebases, two languages, two release schedules drift by construction; review slows it, but discipline never beats structure.
- *"Serve features straight from the warehouse."* One implementation, wrong speed: warehouse queries take hundreds of ms to seconds; inference has **~10 ms** for features.
- *"Log what serving computed and train on that" (log-and-wait).* Removes skew for any logged period, but a new feature has no logs (months of waiting), and logs faithfully record serving bugs. Keep it as *verification*, not the core fix.

How it works

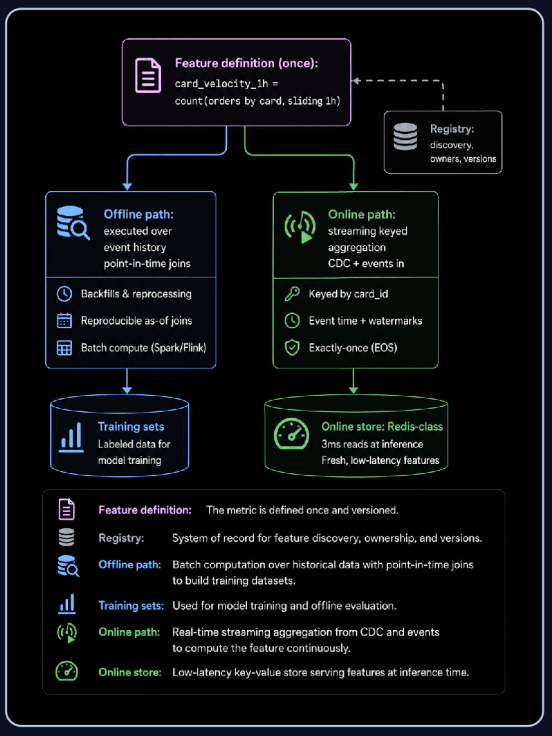


Figure — Feature Store (course diagram)

One feature definition feeds an offline (training) path and an online (serving) path, with a registry between them.

1. **Write every feature once.** `card_velocity_1h` is ~a dozen lines of the platform's DSL (domain-specific language), compiled into both batch SQL and a streaming job. *Why:* one author means the paths can't disagree — the Kappa-style fix of removing duplicates rather than syncing them.
2. **Online path.** Event topics plus change data capture (CDC, streaming the database's change log) feed a keyed sliding-window aggregation that writes to a Redis-class store via idempotent upserts (insert-or-update, safe to retry). Values trail reality by **~200 ms**; reads take **~3 ms**, well inside the ~10 ms budget. This is Module 10's streaming machinery as a product.
3. **Offline path with point-in-time correctness.** A training example **labeled** (tagged with the known answer) "fraud on Tuesday 3 PM" must be joined with feature values *as of* that moment, not today's. Joining today's values onto old labels teaches the model facts that didn't exist at decision time — **leakage**, the classic "amazing offline, useless live." A point-in-time join is **event sourcing**'s "as-of" query applied to training data.
4. **Registry.** Every feature is discoverable, owned and **versioned**. Discoverability lets the growth team reuse velocity (one producer, many consumers); versions matter because a model was trained on v1's meaning, so a definition is a contract like a public API.
5. **Log and verify.** Serving logs what it computed; a background job recomputes the same moments offline and asserts **zero difference**. This catches drift between the two compiled paths (e.g. a compiler bug).

Walkthrough with numbers (the fraud feature, rebuilt)

- *Time to ship:* **3 weeks → ~2 days**, mostly validation.
- *Serving:* **3 ms** reads, values ~**200 ms** old.
- *Training:* the same definition over **6 months** of history with point-in-time joins; after retraining, live results land *within noise* of offline scores — the real product the platform delivers.
- *Reuse:* the growth team's ranking model pulls the same feature; one definition, three consumers.
- *The bill:* **10M cards × 40 features** ≈ a few GB of RAM. Each new feature needs a **backfill** (replaying history so it exists for training) — an ordinary Kappa replay.

Trade-offs

- *You gain:* skew removed by design, so offline evaluation predicts production; leakage prevention guaranteed by the platform; features shared across models; shipping in days.
- *You pay:*
  - **A platform to run** (streaming jobs, online store, registry, backfill tooling) — run it as a product with ML teams as customers.
  - **DSL limits:** every escape hatch is a spot where skew can return.
  - **Freshness tiers:** streamed features (velocity, seconds) vs. nightly batch (90-day aggregates, ~**1/10 the cost**); a model is only as fresh as its stalest feature.
  - **Backfill compute** per new feature.

When not to use it

- *One team, one model, ~a dozen features:* a shared code module called by both paths, plus log-and-verify.
- *Features computed purely from the request* (text length, time of day): stateless transforms inside the model server.
- *Batch-only inference* (nightly scoring): careful point-in-time joins in the warehouse suffice.
- *Classic misuse:* adopting the store but letting teams hand-code "just this one" serving path — the third implementation again, hidden from the registry.

Comparison: three consistency strategies

	Two implementations + review	Log-and-wait	Feature store (define once)
Skew	Built in; review only slows it	Removed for logged periods	Removed by design
Adding a feature	Written twice; weeks	Months waiting for logs	Definition + backfill; days
Leakage protection	Each scientist's care	Good (logs are as-of by nature)	Point-in-time joins enforced by platform
Cost	Seems cheap; grows every quarter	Cheap	A whole platform
Verdict	Avoid	Keep, as verification only	Use, with log-and-verify

Real-world examples

- **Uber's Michelangelo** named the category; **Airbnb's Zipline** appeared around the same time.
- **Feast** (open source), **Tecton**, and cloud offerings (**SageMaker, Vertex, Databricks**) package definitions, dual materialization (computing values into both offline and online stores) and registries.

- Large ranking, fraud, pricing and recommendation systems use this design whether or not they call it a feature store.
- *For LLM engineers:* the "user context" injected into **prompts** (plan, recent activity, preferences) is online feature serving. A template pulling from three ad-hoc services is three chances to drift from what you evaluated.

In the interview. "Design a fraud detector / recommender" eventually becomes: how do serving features stay identical to training features? "Same logic, careful testing" states the goal without a structural fix.

30-second answer: "Each feature is defined once, as code, in a registry, and the platform runs that definition on two paths. Offline runs over event history with point-in-time joins, so each training row sees only what was known at its timestamp; today's values on old labels would leak the future. Online, a streaming aggregation keeps the same feature current in a Redis-class store answering in milliseconds. One author means no skew; log-and-verify catches compiler drift; offline evaluation finally predicts production."

Likely follow-ups:

- **"Why do point-in-time joins matter so much?"** Leakage: current values on training rows contain facts from after the labeled event. Offline scores jump, live performance collapses because production never sees the future. The costliest silent bug in ML, often misdiagnosed as "drift."
- **"Which features stream and which batch?"** Per feature, by tolerable staleness: velocity and session activity stream (seconds); 90-day aggregates batch nightly at ~a tenth of the cost. Each feature declares its tier.

Mistakes that fail candidates: accepting two implementations; building training sets from current-state tables (silent leakage); querying the warehouse on the inference path.

Self-check:

- **Q1:** Define training/serving skew, the engineering problem behind it, and the structural fix. → Serving features differ from training features (definition, windowing, nulls, freshness), so live inputs come from a different distribution and quality quietly drops. Underneath: one question, two implementations. Fix: one definition compiled into both paths, plus log-and-verify.
- **Q2:** A teammate joins March fraud labels onto the *current* features table and gets offline AUC 0.97 (AUC scores ranking quality; 1.0 is perfect). Production? → Much worse. The 0.97 is leakage: today's velocity may be high *because of* the March fraud. Rebuild with point-in-time joins.
- **Q3:** After six months a model slowly degrades; no deploys, log-and-verify clean. Upstream changed `order_total` to exclude shipping. Why didn't the store notice? → It guarantees the paths agree, not that *meaning* stays fixed; both computed the new meaning identically (temporal skew). Add upstream schema/semantic contracts (outbox, schema registry), monitor feature distributions against the training profile (mean, **percentiles** — the value below which a given share of observations fall — null rate), and version features (`order_value_v2`) that models pin.

Related patterns: CQRS (offline writes, online reads; features are read models); Lambda & Kappa (skew is their duplicate-implementation problem); Stream Processing and CDC (the online path); Event Sourcing (point-in-time joins are as-of queries).

TL;DR

- *Problem:* a feature coded twice (SQL for training, a service for serving) drifts; the model looks great offline and quietly fails live. Weeks per feature; third copies appear.

- *Mechanism:* define once in a registry; compile to an offline path (history + point-in-time joins) and an online path (streaming into a millisecond store); log-and-verify as the drift alarm; backfill by replay.
- *Costs:* a platform, DSL limits, per-feature freshness tiers, replay compute per feature.
- *Skip it when:* one team with ~a dozen features, request-only transforms, or batch-only inference.

Model Serving (and Batching)

Original lesson: Model Serving on DesignGurus ↗

What it is. The layer that runs a model to answer live requests. Its key trick is **batching**: hold requests a few milliseconds and run a group through the GPU together. The LLM version is **continuous batching**, where requests join and leave the batch at every generated token.

Analogy. A bus costs nearly the same to run with 1 rider or 32; batching fills the bus. Continuous batching lets riders get off at their own stop and new riders board at every stop.

The problem (the incident). The store self-hosts a 7-billion-parameter ("7B"; parameters are learned weights) support model for privacy and serves it one request at a time. Latency is fine; capacity maths is not:

- An A100-class GPU (popular NVIDIA data-centre GPU) costs ~**$35K/year** and produces ~**50 tokens/s** for a single request.
- A typical reply is **400 tokens** → **8 s** of GPU time → **0.125 req/s** per GPU.
- Peak needs **20 req/s** → **160 GPUs ≈ $5.6M/year**, ~10x the budget.

Profiling shows each GPU at ~**8%** compute. With a batch of one, every token step streams tens of GB of weights from GPU memory through the chip for a tiny amount of maths: the work is **memory-bandwidth-bound**, limited by data movement, not calculation.

Why the obvious fixes fail

- *"Buy more GPUs."* 160 GPUs each still 8% busy; the bottleneck is work per pass over the weights.
- *"Use a smaller model."* Distillation (training a small model to copy a big one) and quantization (fewer bits per weight) cut data per step, but one-at-a-time serving still idles. Consider it *after* batching, if quality allows.
- *"Queue everything into hourly batch jobs."* Fine offline (providers offer batch at ~**50% off**), but a chat customer is waiting; batches must form in *milliseconds*.

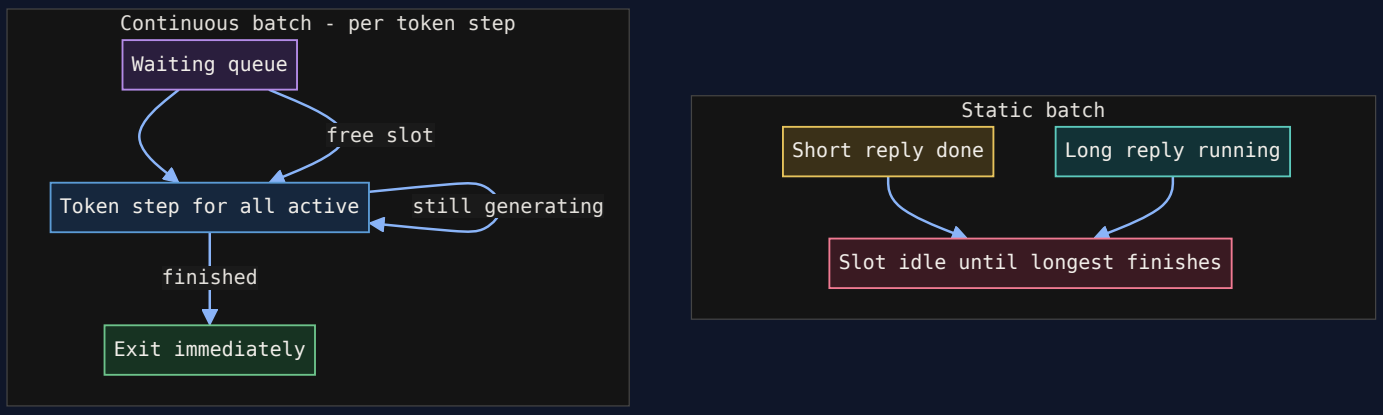
How it works

1. **Dynamic batching.** Wait ~**5–10 ms** or until a max batch size, then run the group. Each step streams the weights once regardless of batch size, so **32** requests take barely longer than one and deliver ~**20–30x throughput**, for a few ms of wait. For same-shaped jobs (embeddings, image classification) this is the whole solution.
2. **The LLM twist: variance (Constant 1).** In a *static* batch nobody leaves until the slowest generation ends; a 10-token reply waits for a 2,000-token essay and finished slots sit empty — the **convoy effect**.
3. **Continuous (in-flight) batching.** The scheduler works in *token steps*: each step emits one token per active request; finished requests leave immediately, waiting ones join mid-flight. Capacity is recovered every token, so utilization holds under 100x length variance.
4. **Paged KV-cache memory (PagedAttention).** Naive allocation fragments GPU memory (free space in unusable scraps), capping concurrency. Borrowing OS **virtual memory** paging (small fixed-size, non-contiguous pages) removes **fragmentation** and fits **2–4x more concurrent requests**.

5. **The surrounding stack.** Load-balance on **queue depth and pending tokens**, not connection counts (misleading under 100x variance). Keep conversations **sticky** to the replica holding their KV cache. At scale, **disaggregate prefill from decode**: *prefill* (reading the prompt) is **compute-bound**, *decode* (writing the reply) is bandwidth-bound, so separate pools are tuned separately.

In the static batch below, a finished short reply idles until the longest finishes; in the continuous batch, finished requests exit each token step and queued ones fill the freed slots.

Static batching vs continuous batching



Walkthrough with numbers (same assistant on a vLLM-class engine) — **vLLM** is a popular open-source LLM serving engine.

- *Per GPU:* batch ~32 → **2,500–3,000 tokens/s** vs. 50; one 400-token reply still takes a few seconds, but the GPU serves **6–7 req/s**.
- *Fleet:* 20 ÷ ~6 ≈ 3.3 → **4 GPUs + 1 headroom = 5 instead of 160**; **$5.6M → ~$175K/year**, a **30x** cut from scheduling alone, same model and quality.
- *Latency cost:* TTFT +~**20–30 ms**, invisible when streaming; per-token pace barely changes.
- *Utilization:* **8%** → **~70%**.
- *Bulk work:* the nightly **10M product descriptions** job goes to the half-price batch API, **bulkheaded** from the interactive fleet.
- Batching stays nearly free until combined compute outlasts weight streaming or KV memory fills; then you need another GPU.

Trade-offs

- *You gain:* **10–30x throughput per dollar** (what makes self-hosting affordable), variance under control, fewer and fuller GPUs to autoscale.
- *You pay:*
  - **A small latency floor** (batch window + scheduler) on TTFT — matters for hard real-time.
  - **Shared fate:** one huge prompt squeezes the rest; an engine crash kills the whole batch.
  - **A complex engine** (token schedulers, paged memory, **preemption** — pausing/evicting a running request). **Buy it** (vLLM, TGI, Triton); a home-built scheduler is a mistake, not an accomplishment.
  - **Per-workload tuning** of batch caps, windows, memory budgets.
- *Giant-prompt spikes:* a pasted contract can push everyone's **p99** TTFT (the value 99% of requests come in under) from **200 ms to 4+ s** — a huge compute-heavy **prefill** hogs the token steps. Fixes: **chunked prefill** (spread the prompt read over many steps); **input tiering** (huge inputs to a long-context pool or async flow — a bulkhead); **product caps with honest UX** ("large document: may take longer"); **prefill/decode disaggregation** at scale.

When not to use it

- *Inference that must finish in a few ms* (voice frames, some ranking): no room for a window; use small-batch or specialized runtimes.
- *Trivial traffic* (dev, 0.1 req/s): a batch of one is correct.
- *Offline bulk jobs:* batch APIs at half price, off the interactive fleet.
- *Classic misuse:* interactive and bulk on one fleet, so the nightly 10M-item job starves chats — a bulkhead failure, GPU edition.

Comparison: dynamic vs. continuous batching

	Dynamic (request-level)	Continuous (token-level)
Batch membership	Fixed at start; all wait for the last	Changes every step; leave when done, join midway
100x output variance	Convoy effect	Handled; capacity recovered each token
Good for	Same-sized jobs: embeddings, vision, rankers	LLM text generation
Implementation	Window + max size (Triton)	Full scheduler (buy vLLM/TGI/TensorRT-LLM/SGLang)

Rule of thumb: **same-sized jobs → batch by request; varying sizes → batch by token.** The embedding endpoint (used by the semantic cache) runs dynamic batching beside the continuous-batching LLM fleet.

Real-world examples

- **vLLM:** open-source continuous batching + paged KV; made self-hosted LLM serving practical.
- **NVIDIA Triton:** dynamic batching as general infrastructure for any model type.
- **TGI, TensorRT-LLM, SGLang:** variants of the same ideas.
- **Provider batch APIs (Anthropic, OpenAI):** ~50% off for work that can wait, slotted into spare fleet capacity.

In the interview. LLM inference-serving questions are increasingly common; "why does batching help GPUs so much?" separates people who have run inference from those who only called an API.

30-second answer: "One-at-a-time LLM inference is memory-bandwidth-bound: every token step streams all the weights for barely any maths, so GPUs idle near 8% and the fleet costs millions. Batching shares that stream across ~32 requests for 20–30x throughput at a few ms of TTFT, hidden by streaming. LLMs need continuous, token-level batching because output lengths vary 100x; request-level batching would trap short answers behind long ones. Paged KV fits 2–4x more requests. Balance on queue length and pending tokens, keep conversations sticky, buy a vLLM-class engine, and send bulk work to half-price batch APIs."

Likely follow-ups:

- **"How can 32 requests take barely longer than one?"** Each step's cost is dominated by streaming weights, which is the same at any batch size; per-request compute rides along nearly free until the batch saturates the compute units.
- **"Someone pastes a 50-page file and everyone's latency spikes."** The huge prefill is a compute burst hogging token steps (shared fate). Chunked prefill, input caps and size tiers, and at scale separate prefill/decode pools.

Mistakes that fail candidates: pricing one-request-per-pass serving without spotting the waste; request-level batching for chat (convoy effect); hand-writing the token scheduler.

Self-check:

- **Q1:** Why does a batch of 32 take about as long per step as one request, and when does that stop? → The fixed cost of streaming weights dominates; per-request maths is small. It stops when total compute outlasts streaming time or KV memory runs out — the GPU is now compute-bound and you need another one (the autoscaling signal).
- **Q2:** Match strategies: (a) embeddings (~50 ms each), (b) customer chat (100x output variance), (c) nightly summaries of 10M pages. → (a) dynamic batching (Triton-style); (b) continuous batching + paged KV (vLLM-class); (c) neither fleet: provider batch API or off-peak offline run, bulkheaded from the interactive pool.
- **Q3:** p99 TTFT, usually 200 ms, jumps to 4+ s for everyone when users paste contracts. → Giant compute-bound prefills compete with every decode (Constant 1 applies to inputs too). Chunked prefill, input tiering, product caps with honest UX, prefill/decode disaggregation.

Related patterns: Load Balancing (by queue depth / outstanding tokens); GPU Auto-Scaling (adding GPUs once full); Message Queue (bulk work via half-price batch queues); Semantic Caching (the cheapest request never reaches the GPU).

TL;DR

- *Problem:* one-at-a-time LLM serving leaves GPUs ~8% busy (bandwidth-bound); 20 req/s costs $5.6M/year.
- *Mechanism:* batch — continuous, token-level for 100x variance, plus paged KV (2–4x). Buy a vLLM-class engine.
- *Costs:* small TTFT floor, shared fate (giant prompts), a complex engine, per-workload tuning.
- *Skip it when:* hard real-time ms budgets, trivial traffic, or offline bulk (batch APIs, bulkheaded).

GPU Auto-Scaling

Original lesson: [GPU Auto-Scaling on DesignGurus ↗]

What it is. Adding and removing GPU serving replicas with demand — web auto-scaling, except idle GPUs cost ~100x more and new ones take minutes to start. So instead of one reactive loop you **layer** defenses: the right signal, warm spares, scheduled pre-warming, faster boots, a degradation plan for the gap, and careful scale-down.

Analogy. A restaurant can't hire and train a chef in ten minutes after a tour bus arrives. It keeps chefs on call (warm pool), staffs up from the bookings calendar (scheduled scaling), trains hires faster (boot engineering), simplifies the menu when swamped (smaller model), and never sends a chef home mid-dish (drain before scale-down).

The problem (the incident). The five-GPU support fleet runs at 70%. A morning-show mention quadruples traffic within **15 minutes**, and the web-style autoscaler fails three ways:

1. **Wrong signal.** It watches host CPU, which shows **12%** while requests pile up on the GPU. GPU utilization misleads too: under continuous batching the chip always looks busy while the queue grows. Only **queue depth** (requests waiting for a GPU) showed reality, and nobody watched it.
2. **Minutes to boot.** Getting a GPU node takes **3–8 minutes** (if the **region** has one), then pulling a **15 GB container**, loading **14 GB** of weights and warming the runtime: **8–12 minutes** from decision to first token. The spike is over first.
3. **The maximum isn't yours.** At minute 20, four more A100s return "insufficient capacity in this region": the real maximum was the provider's **quota**.

At night, five GPUs sit **10%** busy at **~$20/hour**, yet scale-to-zero means the next customer waits ~10 minutes. Versus web servers: idle costs **100x** more, cold starts last **60x** longer.

Why the obvious fixes fail

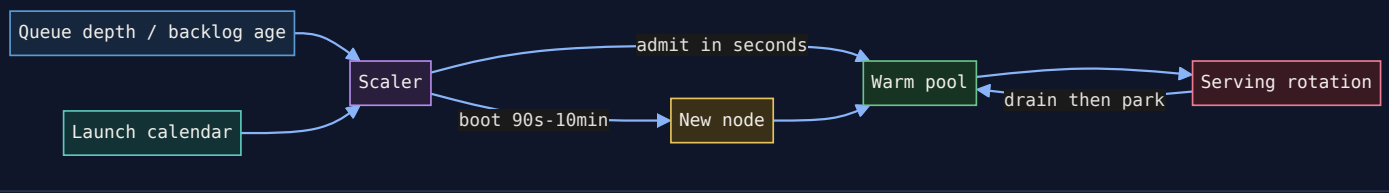
- *"Tune the reactive loop harder."* ~1 min to notice + ~10 min to boot vs. a 15-minute spike: no setting wins. Reactive-only is a well-tuned way to arrive late.
- *"Provision for peak."* At **~$35K/GPU-year**, 4x spare capacity wastes the 30x batching savings.
- *"Serverless GPUs (Modal, Baseten, RunPod)."* (*Serverless*: pay per use, platform manages machines.) Often the right choice — they hide cold starts behind warm pools shared across customers, so you pay someone to solve this lesson. At large scale or under strict privacy, the problem returns to you, so know the mechanics.

How it works: layered defenses

1. **Right metrics: queue depth and backlog age per replica.** *Backlog age* (how long the oldest request has waited) triggers alarms. Replicas needed ≈ token demand ÷ token throughput per GPU. CPU and connection counts don't reflect token work.
2. **Warm pool.** Keep *k* instances fully booted (weights loaded) *outside* rotation; scale-up moves one in within **seconds** while slow boots refill the pool in the background. Size: **pool ≥ spike arrival rate × boot time**. The idle pool is the explicit price of instant capacity (Module 6's headroom at GPU prices).
3. **Scheduled, ahead-of-time scaling.** Pre-warm before the daily curve (e.g. the 9 AM ramp); make marketing file pre-warm requests for launches. Predictable demand sees zero delay; reactive scaling handles only surprises.
4. **Faster boots.** Stream container images (start while downloading); keep weights on local NVMe (fast directly attached SSDs) or pre-attached snapshot volumes; restore from memory snapshots where supported; co-locate models on nodes that keep weights loaded. Boot drops from **8–12 min to 60–90 s**, making every other layer cheaper.
5. **Degrade gracefully during the gap.** Queue with an honest wait time → shed free tier before paid (rate-limit tiers) → **switch to a smaller model and say so** (e.g. quantized 7B when the 70B pool is full). Users prefer a labeled weaker answer to a spinner; quality becomes one more thing you can shed.
6. **Scale down without cutting anyone off.** Stop new admissions, let in-flight generations finish, park the replica in the warm pool, then release. Keep an overnight floor; allow scale-to-zero only if breakeven works (idle hours × **~$4/GPU-hour** vs. a multi-minute wait for the next user — dev usually passes, production rarely). Immediate termination cuts a paying user off mid-sentence.
7. **Quota is procured capacity.** Reservations and committed-use contracts (discounts for promising spend over a year or more), plus an overflow target (another region, cloud, or hosted API), routed by the LLM gateway (next lesson).

Queue depth drives the scaler, which admits pooled instances in seconds while new nodes refill the pool; the launch calendar feeds the scaler, and scale-down drains replicas back into the pool.

Warm pool: fast admission, slow refill off the critical path



Walkthrough with numbers (the morning show, replayed)

- *T−1 day:* the launch calendar grows the pool **2** → **5** — the cheapest fix in the lesson is a meeting.

- *T+0–60 s:* traffic 4x; queue-depth alarms fire; 5 pooled instances join in seconds, roughly doubling capacity in the first minute.
- *T+90 s onward:* fast boots add capacity every **90 s** and refill the pool.
- *Worst minute:* p95 TTFT (95% of requests under it) **3.8 s**; free tier waits briefly behind paid; **12%** of traffic served by the labeled smaller model; no error page.
- *T+25 min:* demand falls; scale-down drains, parks, holds the new baseline.
- *3 AM:* floor of **2**, with **preemptible** nightly summarization (evicted instantly if real traffic arrives) filling quiet hours: **70% utilization by day, 60% at night**.

Trade-offs

- *You gain:* spikes survived despite 10-minute boots; a bill that tracks demand; conversations that complete through scale-down.
- *You pay:* warm-pool cost (insurance that looks like waste, defended every quarter); layered ops (calendars, pools, boots, ladders, preemption); preemption machinery (priorities, instant eviction, checkpointing batch jobs); ongoing quota work (reservations, cross-region overflow, provider relationships).

When not to use it

- *Steady traffic:* fixed fleet, spare time filled with preemptible bulk jobs.
- *Tiny scale:* with one or two GPUs, plain queueing with honest wait times beats warm-pool machinery.
- *Pure batch:* schedule the fleet like batch work.
- *You shouldn't own GPUs yet:* start with a hosted API and let the provider own cold starts and quota until traffic, latency or privacy forces self-hosting.
- *Classic misuse:* web-style GPU scaling — CPU signal, reactive only, scale-to-zero in prod, terminate on scale-down.

Comparison: own the fleet vs. rent the problem

	Provider APIs	Serverless GPU	Own fleet + this lesson
Cold starts	Provider's problem	Eased by platform warm pools	Yours: everything in this lesson
Cost at high volume	Highest per token	Middle	Lowest, if GPUs stay busy
Privacy / control / custom models	Least	Middle	Full
Ops burden	Almost none	Low	A whole team
Fits	Starting point for most	Bursty, specialized workloads, experiments	Large steady volume, strict privacy, custom models

Move one column right only when traffic or constraints force it; this lesson is the cost of each move.

Real-world examples

- **KEDA** (Kubernetes event-driven autoscaling; **Kubernetes** runs containers across a cluster) scales on queue depth (the signal layer); **Ray Serve** and vLLM's autoscaling handle model-aware parts.
- **Cloud warm pools** (e.g. AWS Auto Scaling Group warm pools), GPU and **capacity reservations**: readiness and quota as products.
- **Modal, Baseten, RunPod:** rented warm pools.
- **SkyPilot-class multi-cloud spillover:** finds capacity across regions/providers when quota runs out.

In the interview. "Traffic to your LLM service goes up 5x — handle capacity." Each follow-up maps to a layer (signal, cold start, scale-to-zero, quota), each with a tempting one-line wrong answer.

30-second answer: "GPU scaling is web scaling with idle cost and cold starts ~100x worse, so reactive-only is late by design. I scale on queue length and oldest-request age, never CPU. A warm pool sized at spike rate × boot time admits capacity in seconds while new nodes refill it; a calendar pre-warms known events; local weights and streamed images cut boots to ~90 s. In the gap, a ladder queues, sheds lower tiers, and falls back to a labeled smaller model. Scale-down drains in-flight generations and keeps an overnight floor. Quota is negotiated, with spillover via the gateway. And I'd start API-first until volume or privacy justifies owning GPUs."

Likely follow-ups:

- **"Why is CPU a poor signal?"** The model runs on the GPU, so host CPU looks idle while users wait; GPU utilization looks busy under continuous batching while the queue grows. Queue length and backlog age show directly whether arrivals outpace completions.
- **"Should production scale to zero?"** Breakeven: overnight idle at ~$4/GPU-hour vs. the first morning users eating a multi-minute cold start. Dev: zero. Production: a small floor nearly always wins. Show the maths.

Mistakes that fail candidates: CPU-triggered GPU scaling; reaction alone with minute-long boots; prod scale-to-zero without breakeven maths; scale-down that kills generations mid-stream.

Self-check:

- **Q1:** Which three delays make reactive GPU scaling late, and which layer handles each? → Detection lag (~1 min) → queue-depth/backlog-age signals. Boot lag (minutes) → warm pools + boot engineering. Spike duration (often 10–20 min, about the first two combined) → scheduled/predictive pre-warming. The degradation ladder covers the remainder.
- **Q2:** Spikes add up to 2 GPUs of demand per minute; boots take 90 s. Pool size? → ≥ 2 × 1.5 = **3 GPUs** (+1 for overlapping spikes): ~3 × $35K ≈ **$105K/year** of deliberate idleness turning a 10-minute outage into 10-second admission. Recompute when boot time or spike shape changes.
- **Q3:** Around 2 AM answers are cut mid-sentence, and first morning users wait "forever." Config: min=0, scale down after 15 min low utilization, immediate termination. → Termination kills live generations: drain (stop admission, finish up to max output length, park, release). min=0 means full cold starts: keep a breakeven floor of 1–2 and pre-warm before the morning ramp. **With GPUs, scaling down hits users directly.**

Related patterns: Auto-Scaling (the parent, with idle cost and boot time ~100x worse); Model Serving (fuller batches = fewer GPUs); Graceful Degradation (the gap ladder, including the smaller model); LLM Gateway (spillover routing); Message Queue (preemptible bulk work in quiet hours).

TL;DR

- *Problem:* minute-long cold starts make reactive scaling late; $4/h idle makes overprovisioning costly; the real ceiling is provider quota.
- *Mechanism:* queue-depth signals; warm pool = spike rate × boot time; calendar pre-warming; 90-second boots; a ladder ending in a labeled smaller model; drain-then-park; negotiated quota with gateway spillover.
- *Costs:* paid idleness, layered ops, preemption machinery, capacity procurement.
- *Skip it when:* flat traffic, tiny scale, pure batch, or you shouldn't own GPUs (go API-first).

LLM Gateway

Original lesson: [LLM Gateway on DesignGurus ↗]

What it is. One internal service that *every* model-provider call passes through. Services authenticate with their internal identity; the gateway holds provider **API keys**, counts every token, maps "logical" model names to real providers, applies per-provider timeouts, retries and **circuit breakers**, and follows a pre-planned fallback sequence on failure. It is Module 8's API gateway applied to **outbound** (egress) traffic.

Analogy. A single purchasing office that holds the company cards, logs who bought what, enforces budgets, and knows which backup supplier to call — instead of every department carrying its own card.

The problem (the incident). After 18 months of AI enthusiasm, **14 services** call providers directly (the assistant, search, summarization, three agent experiments, a fraud explainer, and eight uncatalogued). Three bills arrive together:

- **A key leaks.** A team shipped a provider key in a client bundle (code sent to browsers/apps); it appears on a public paste site. To **rotate** it you must find every copy across 14 services, 3 languages and unknown configs, then redeploy them all.
- **Nobody can attribute spend.** "Who spent the **$80K** last month?" Invoices group by informally shared keys. An experimental agent retried in a loop for two days and burned **$40K** unnoticed; there was no place to enforce a cap.
- **A provider brownout** (partial outage: slow or failing replies) at peak gets fourteen reactions: **6 fail with no fallback, 5 hang with no timeout, 3 run untested custom degradation.**

The root cause is Module 8's: a **cross-cutting** policy (auth, logging, budgets) implemented per service drifts. The difference is that every request has a direct dollar cost, so drift becomes large unplanned charges within days.

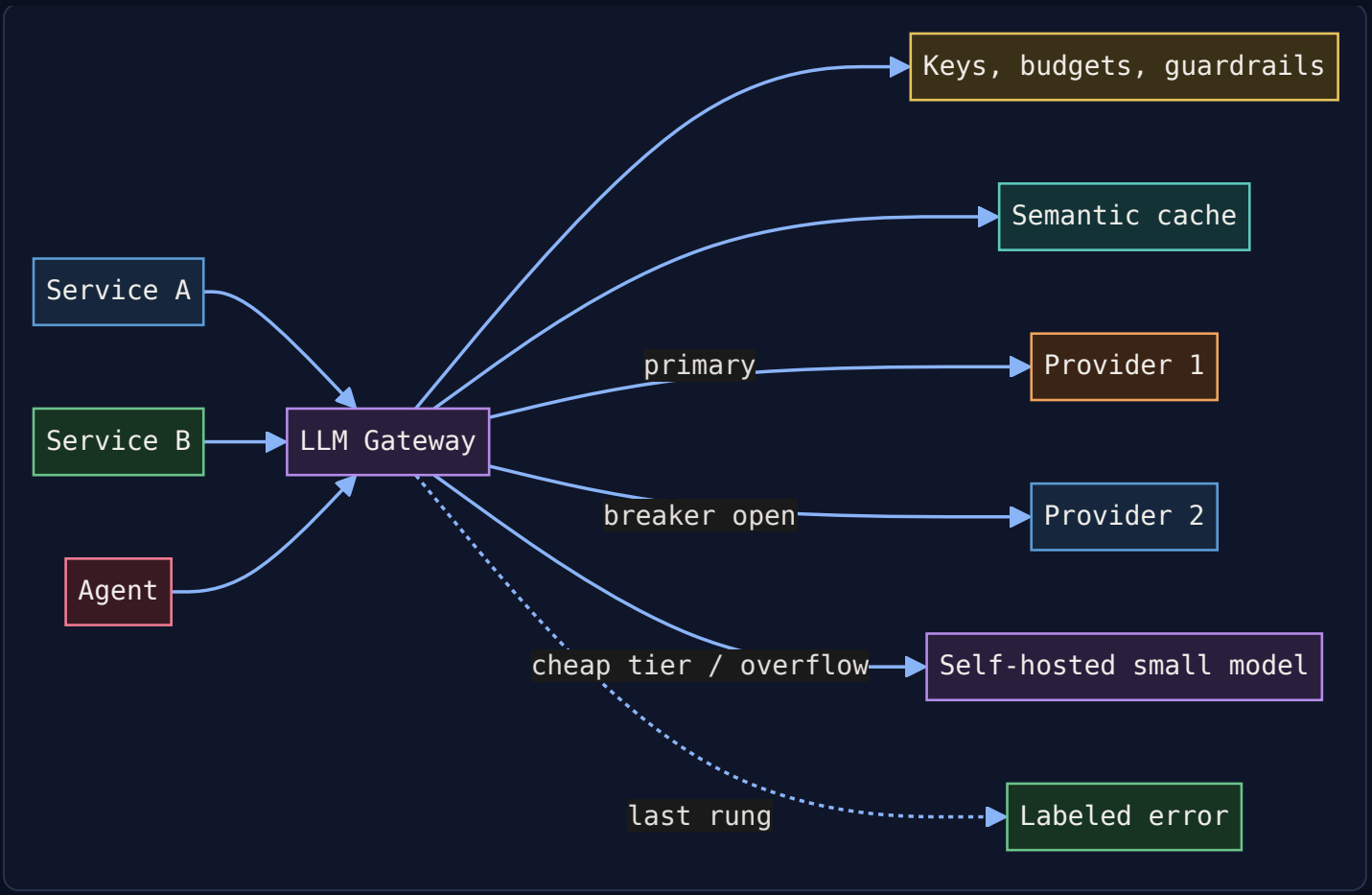
Why the obvious fixes fail

- *"A shared SDK"* (client library). Rollout rides 14 deploy schedules, 3 languages mean 3 SDKs, and keys still live in 14 configs. A library centralizes *code*; custody, accounting and live policy need one *runtime* enforcement point.
- *"A key registry and a spending policy document."* The runaway agent didn't read the document, and a document can't cap spend at 2 AM Saturday.
- *"Each team hardens its own integration."* 14 half-gateways that drift — the incident made official.

How it works: each job is a Module 8 job, pointed outward

Services and agents call one gateway, which applies keys, budgets and guardrails, checks the semantic cache, and routes to the primary provider; an open breaker shifts traffic to a second provider; cheap tiers and overflow go to a self-hosted small model; the last rung is a labeled error.

One egress gateway in front of all model providers



1. **Key custody (was: authentication).** Services authenticate with **service mesh** workload identity (the mesh carries inter-service traffic and gives each workload a verifiable identity); only the gateway holds provider keys. A leak means rotating **one** key with **zero** redeploys, and every call is attributed automatically.
2. **Token accounting and budgets (was: rate limiting).** Count tokens per team, feature and user, with soft alerts and **hard caps** — rate limiting in tokens per minute. Outward, handle provider `429 Too Many Requests` by retrying per the `Retry-After` header. Metering feeds billing, which needs exactly-once processing when invoices depend on it. One component manages both your budgets and the provider's limits, and a runaway **agent** (an LLM program acting in a loop) hits a cap instead of causing a **postmortem**.
3. **Logical model routing (was: path routing).** Services request a capability (`summarize-fast`, `chat-premium`); the gateway maps it to provider, model and version. Upgrades, price changes and provider switches become one routing change, not 14 deploys. Roll out as a **canary behind a feature flag** (e.g. 5% of `chat-premium` to the new model — a canary is a small first share of traffic; a flag changes behaviour without a deploy), judged by eval scores and negative-feedback rates. Model choice becomes an operational setting.
4. **Per-provider reliability kit.** Timeouts tuned to each provider's p99; retries honouring `Retry-After` within a per-provider retry budget; a **circuit breaker per provider-model pair**. In a brownout the breaker opens within seconds and traffic steps down the **fallback ladder**: primary → secondary provider → smaller self-hosted model → semantic cache → honest error. Module 5's graceful degradation, built and tested once.
5. **Streaming passthrough.** LLM replies stream over SSE (server-sent events); the gateway must forward them *unbuffered* or TTFT is wrecked. Test it in **CI** (automated tests on every change) like a feature.
6. **Safety point.** PII (personally identifiable information) scrubbing, **prompt-injection** screening (hidden instructions in input trying to hijack the model), and audit logging with **trace IDs** happen once, here.
7. **Capacity federation.** When the self-hosted GPU pool is full, overflow goes to provider APIs via the same chains (where the GPU lesson's spillover lands).

8. **Keep it thin.** The gateway routes, meters and protects; it does **not** own prompts, templates or business logic, which are product code in the services. Otherwise every prompt tweak becomes a platform ticket (the "monolith in disguise"). Placement: the API gateway handles inbound **north-south** traffic (outside clients ↔ your system); the LLM gateway handles outbound calls and usually sits behind the API gateway, beside the services.

Walkthrough with numbers (the three bills, replayed)

- *The leak:* rotate one key in one **vault** (secret store): minutes, zero deploys, every caller already on the dashboard.
- *The brownout:* the primary's error rate crosses the breaker threshold in **~20 s**. **80%** of traffic shifts to the secondary (**+300 ms**, quality within tolerance); `classify-fast` moves to the self-hosted **7B**; **15%** of chat is served from the semantic cache; the UI shows a "responses may be slower" banner. **Zero services down** (vs. six), because the response was prepared configuration.
- *The weekend agent:* stopped at **$500**, not $40K.
- *Gateway cost:* **+3–8 ms** proxy overhead per call (noise against multi-second generations); one more component needing high availability — run it redundant and thin.

Trade-offs

- *You gain:* custody, budgets and safety enforced once at the only point that sees every call; failover built and tested once; model swaps, canaries and price-based provider choice become routing; per-feature cost visibility; runaway spend capped at machine speed.
- *You pay:* **a critical-path dependency** for every AI feature (run it redundant; the least-changing component is the most stable); **streaming passthrough as a standing obligation** (one buffering regression silently triples TTFT); **adapter churn** as provider APIs change (a reason to buy the core); **constant pressure toward scope creep** (prompts, "just one transformation").

When not to use it

- *One service, one provider:* a well-configured client with timeout, retry and breaker suffices.
- *Fully self-hosted single model:* the serving load balancer already routes; an extra hop adds latency without new policy.
- *Tiny latency-critical local models* (in-process rankers): no door needed.
- *Build vs. buy:* start from **LiteLLM** (open source), **OpenRouter** (gateway as a service), **Portkey**, or cloud AI gateways. Build only where custody, safety or routing policy is truly yours, on an open-source core; hand-writing every adapter repeats the "hand-built scheduler" mistake.
- *Classic misuse:* the gateway as a **prompt CMS** — templates, **few-shot examples** (sample Q/A pairs in the prompt) and product logic pile up in config until every feature change needs a platform ticket (the ESB, enterprise service bus, centralization problem).

Comparison: API gateway vs. LLM gateway, job by job

Job	API Gateway (inbound)	LLM Gateway (egress)
Identity	AuthN (authentication): who sent this?	Key custody: our identity toward providers; each caller attributed
Fairness	Per-client rate limits	Per-team/feature token budgets with hard caps
Routing	Paths → services	Logical models → providers/versions
Change safety	API versioning	Model flags + canaries judged by evals
Reliability	Timeout/retry per upstream	Full kit per provider + fallback ladder
Observability	Request IDs, trace root	Token/cost accounting, trace propagation
Discipline	Thin: no domain logic	Thin: no prompts

Same pattern, opposite direction; mature stacks run both, one behind the other.

Real-world examples

- **LiteLLM:** the open-source standard (one interface to all providers, budgets, fallbacks); the default core to build on.
- **OpenRouter:** the pattern as a business — a public gateway over dozens of providers, billed per token.
- **Portkey, Kong AI Gateway, Cloudflare AI Gateway:** enterprise and edge variants with guardrails and caching at the same chokepoint.
- Every serious AI platform team builds this within a year of its "14 services" incident, usually right after.

In the interview. "A dozen teams call providers directly. What breaks, and what do you build?" Follow-ups probe the outage chain, safe model swaps, and streaming.

30-second answer: "One internal egress gateway for all model traffic — Module 8's gateway turned outward. Services authenticate with internal identity and the gateway holds provider keys, so a leak is one rotation, not 14 deploys. It meters tokens per team and feature with hard caps. Logical model names map to providers, so upgrades are canaried routing changes judged by evals. Each provider gets tuned timeouts, Retry-After-aware retries, and a breaker per provider-model feeding a fallback ladder: secondary provider, smaller self-hosted model, semantic cache, labeled error. Streams pass through unbuffered, and the gateway stays thin with no prompts. Start from a LiteLLM-class core."

Likely follow-ups:

- **"Walk through the provider outage."** The provider-model breaker trips on error rate within seconds; traffic falls down the configured chain (secondary provider for quality tiers, self-hosted small model for cheap tiers, semantic cache where hits exist, honest error last). Budgets stop failover from doubling spend. "The response was configuration, not a war room."
- **"How do you move chat to a new model safely?"** Logical names decouple callers from models: flag 5% to the new model, compare against the concurrent baseline on live evals, negative feedback, cost and per-token latency, then ramp or roll back via config.

Mistakes that fail candidates: prompts in the gateway; no hard budget caps (the $40K weekend remains possible); buffered streams; hand-written provider adapters.

Self-check:

- **Q1:** Map five API-gateway jobs to LLM-gateway counterparts and name the two disciplines. → AuthN → key custody + caller attribution; rate limits → token budgets with hard caps; path routing → logical-model routing; API versioning → model flags + eval-judged canaries; request-ID observability → token/cost accounting + trace propagation. Disciplines: **unbuffered streaming passthrough** and **thinness** (no prompts or product logic).
- **Q2:** Fallback chains for (a) customer chat and (b) internal ticket classification. → (a) premium model → comparable model at the secondary provider → semantic cache for high-similarity hits → **honest labeled error with retry guidance**; it stops short of a much smaller model because poor open-ended answers are worse than waiting (**fail closed** on quality). (b) cheap model → self-hosted 7B → keyword rules → almost never an error, since availability matters more than quality. Chain length is a per-tier product decision priced by the cost of a wrong answer.
- **Q3:** The gateway team is a bottleneck: 400 prompt templates in config, a ticket per prompt change. → A monolith in disguise (prompt CMS, the ESB mistake). Move prompts, templates and output formats to owning services or a product-owned prompt registry (like the **BFF**, backend-for-frontend, split). The gateway keeps custody, metering/caps, routing, reliability kit, fallback ladder, safety and passthrough. Rule: product vocabulary doesn't belong in the gateway.

Related patterns: API Gateway (inbound parent; run both); Circuit Breaker and Retry (per-provider kit); Graceful Degradation (the fallback ladder); Semantic Caching (shared response cache at the gateway); Canary Deployment + Feature Flags (shipping model swaps).

TL;DR

- *Problem:* 14 services call providers directly: scattered keys (one leak = 14 deploys), unattributable spend ($40K runaway), 14 homemade outage responses.
- *Mechanism:* one thin egress door — key custody, token metering with hard caps, logical-model routing (canaried config), per-provider reliability kit feeding a fallback ladder, guardrails once, unbuffered streaming, no prompts.
- *Costs:* a highly available critical-path component, adapter churn (buy the core), passthrough testing, a permanent fight against prompt creep.
- *Skip it when:* one service + one provider, single-model self-hosting, or in-process models; never write adapters from scratch.

Semantic Caching

Original lesson: *Semantic Caching on DesignGurus ↗*

What it is. A cache keyed on the *meaning* of a question rather than its wording. Each query is embedded; if a stored question's embedding is close enough (above a similarity **threshold**), the saved answer returns in milliseconds. Otherwise a fresh answer is generated and saved for the next person asking in other words.

Analogy. A good librarian sends "WWII tanks" and "armored vehicles in the Second World War" to the same shelf — but double-checks before handing over something that merely *sounds* similar.

The problem (the incident). The support assistant costs ~**$2,000/day**. A week of logs shows **40% of questions are rephrasings of ~200 questions** ("order status pls", "when will my package arrive"), each paying full price (~**$0.002**, **2.5 s**). Module 4's cache-aside keyed on the query string gets a **hit ratio** of **4%**: exact keys only catch copy-pastes. That captures a tenth of the opportunity and wastes ~**$800/day**

re-answering answered questions. The answers are cacheable; the *keys* can't recognize them. Module 4's laws still hold — what broke is the equality operator (Constant 3).

Why the obvious fixes fail

- *"Normalize the text"* (lowercase, strip punctuation, **stem** — cut words to roots, "shipped" → "ship"). Hit ratio rises only to **7%**; "where is my package" and "order status?" share no stems.
- *"Keyword-match a canonical FAQ."* The brittle pre-LLM chatbot again: endless upkeep, misses unforeseen paraphrases.
- *"Per-user session caches."* The 40% is repetition *across* users, which per-user caches never see.

Two sentences that mean the same thing need a shared key — a geometry problem, not string manipulation.

How it works

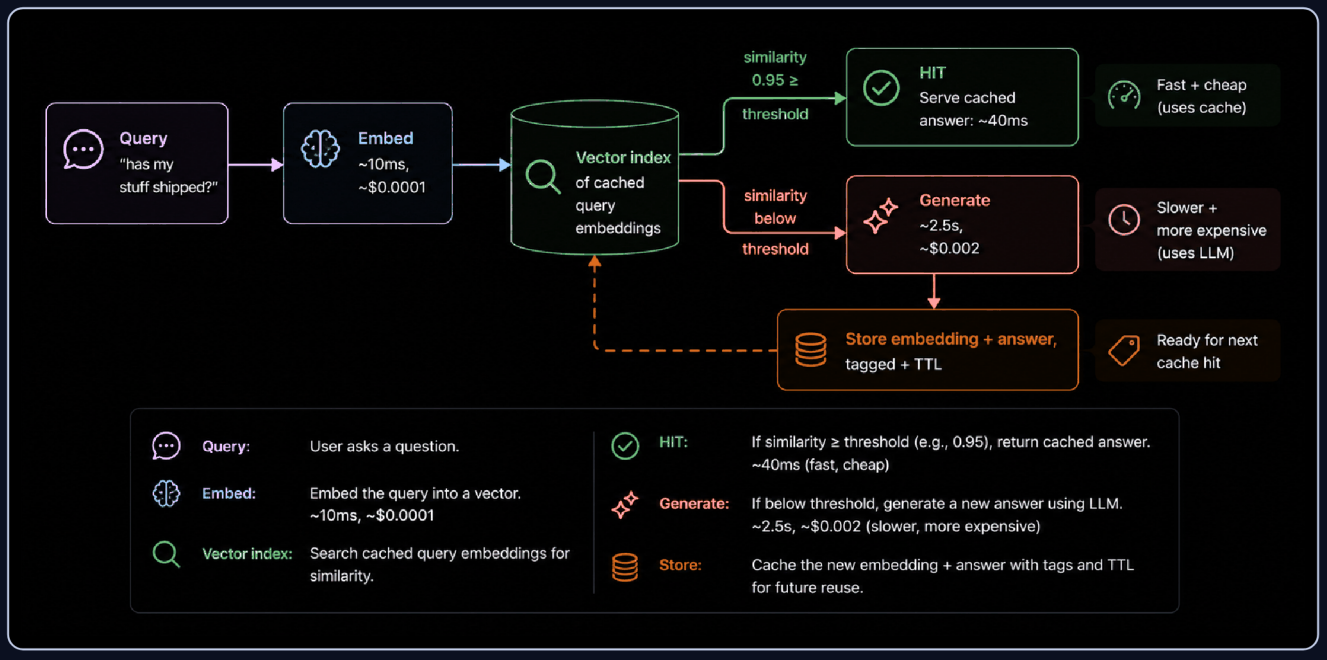


Figure — Semantic Caching (course diagram)

The query is embedded and compared with cached entries, then answered from cache (above threshold and passing guards) or sent to generation with the answer written back.

1. **Embed the query** with a small model: **~10 ms**, a tiny fraction of a generation's cost. The embedding is the new key.
2. **Nearest-neighbor search** (finding stored vectors closest to the query vector) over cached questions' embeddings.
3. **Threshold check.** Above threshold and guards pass → return cached answer; otherwise generate and store `(embedding, query, answer, tags)`.
4. **Set the threshold cautiously.** Besides hit and miss there is now a **false hit**: serving an answer to a question that seems similar but means something else ("cancel my order" vs. "cancel my subscription"). Costs are lopsided: a false miss costs $0.002; a false hit gives wrong instructions and costs trust.
5. **Guard every hit.** (a) **Entity check:** a query about order #8412 never gets an answer naming #7311. (b) **No personalized content** in a shared cache (the CDN lesson's `private, no-store` rule in embedding space). (c) **TTLs** (time-to-live), e.g. a return-policy answer expires because policy changes. (d) **Sampled grading** of real hits — the false-hit rate must be *measured*, not assumed from config.
6. **Invalidate by tag.** Fuzzy keys can't be deleted by name, so tag entries with topics at write time and "purge tag: shipping-policy"; TTL is the safety net.
7. **Place it at the LLM gateway** so all teams share one cache. Read path: **exact cache → semantic cache → generation**.
8. **Decide what is cacheable.** Informational, impersonal, stable answers: yes. Account state: no (depends on asker). Creative requests: no (the same poem twice is a bug). Time-sensitive: short

TTLs only.

Walkthrough with numbers

- *Threshold tuning:* on an offline set of same/different-meaning pairs, **0.92** gives **31%** hits (vs. 4% exact) with **0.3%** false hits in audits. Moving to **0.94** plus the entity guard gives **27% hits, 0.05% false hits**: 4 points of hit ratio for **6x fewer wrong answers**, documented as a product decision.
- *Money:* 27% of $2,000/day ≈ **$540/day, ~$200K/year**, for an embedder, an index and a threshold.
- *Latency:* hits in ~**40 ms** vs. 2.5 s generation.
- *Viral questions:* during an incident, variants of "is the site down?" are served from one generation **10,000 times an hour** — **stampede** protection (**single-flight**: one request does the work, concurrent identical ones share its result), matched by meaning.

Trade-offs

- *You gain:* **25–40%** lower inference cost on repetitive workloads (often the single most valuable change in an LLM stack); hits in a faster latency class; free stampede protection.
- *You pay:*
  - **"Confidently wrong":** exact caches serve stale data; semantic caches can serve *incorrect* data. Thresholds and entity guards limit it, sampling monitors it, it's never zero, and someone must own it.
  - **Hot-path infrastructure** (embedder + vector index). If they fail, **fail open**: bypass and generate — the miss path is always correct.
  - **Fuzzy invalidation:** tags and TTLs replace precise deletes, so correctness leans on expiry.
  - **Geometry upkeep:** keys live in one embedder's coordinate space; upgrading the embedder needs the dual-run migration (Q3).

When not to use it

- *Personal or stateful queries* ("where's my order?", balances): never in the shared cache; entity checks are a backstop, not the gate.
- *When variety is the product* (creative writing, brainstorming).
- *Long-tail workloads:* at ~**5%** repetition the infrastructure costs more than it saves. **Measure the repetition rate first** — it is the whole business case.
- *Zero-false-hit domains* (medical, legal, financial advice): the threshold creeps to exact match, so use exact matching.
- *Classic misuse:* reporting savings without sampled correctness audits, so false hits surface via customer complaints.

Comparison: the caching stack

	Exact cache	Semantic cache	Provider prompt caching
Key	Byte-identical string	Meaning (embedding proximity)	Identical prefix (system prompt, i.e. fixed instructions; docs)
What is reused	Whole answer	Whole answer, across wordings	Model's internal KV state for the prefix
Layer	In front of the gateway	At the gateway	Inside the provider/model
False hits	Impossible	The managed risk	Impossible (exact prefix)
Wins	Free, safe; copy-pastes	The 40% repetition	50–90% off input tokens for shared long prompts

Use a three-tier read path (exact → guarded semantic → generation). **Provider prompt caching is a separate mechanism** — reusing model state for identical prefixes with no false-hit risk — and runs alongside response caching, not instead of it.

Real-world examples

- **GPTCache** (open source): the reference design (embed, index, threshold, serve).
- **Portkey, Cloudflare AI Gateway:** semantic caching as a gateway tier.
- **Anthropic / OpenAI prompt caching:** the *other* kind — prefix/KV reuse for 50–90% input discounts.
- **Support-deflection platforms:** FAQ recognition was semantic caching before it had a name.

In the interview. "Your LLM bill is huge; cut it" leads here; follow-ups probe threshold, false hits and invalidation.

30-second answer: "40% of queries repeat a meaning but only 4% repeat the string, so I key the cache by meaning: embed, nearest-neighbor search over cached entries, serve above a similarity threshold. Costs are lopsided — a false miss is a fifth of a cent, a false hit costs trust — so I tune conservatively on labeled pairs, check entities on hits, never cache personal answers, and audit sampled hits. Invalidation is topic tags plus TTL. It sits at the gateway behind an exact-match tier, and it's distinct from provider prompt caching, which reuses KV state for identical prefixes; I'd use both."

Likely follow-ups:

- **"How do you pick the threshold?"** Offline: sweep thresholds over labeled same/different pairs and plot hit ratio vs. false-hit rate. Online: keep grading sampled hits. Choose tight (wrong answers cost trust, misses cost fractions of a cent) and document it.
- **"What about 'where's my order?', your top question?"** Most common *and* uncacheable: it's per-user state and the entity guard blocks it. Use a tool/API call for real status, with the LLM formatting the result; cache the template, never the content.

Mistakes that fail candidates: caching personalized answers ("a privacy incident with a savings dashboard"); no false-hit monitoring; confusing semantic with prompt/prefix caching; not measuring repetition rate.

Self-check:

- **Q1:** Why does exact-match fail on natural language, what does the key become, and what new outcome appears? → People phrase differently, so byte keys only match copy-pastes. The key becomes the query embedding, matched by nearest-neighbor similarity. New outcome: the **false hit** (confidently wrong) — hence cautious thresholds and guards on every hit.
- **Q2:** Classify: (a) "what's your return policy?" (b) "where's my order?" (c) "write a haiku about autumn" (d) "how do I reset my password?" → (a) cache with a `returns-policy` tag and TTL; (b) never — personal state, use a tool call; (c) no — variety is the product; (d) cache the shared instructions, with the entity guard ensuring no one else's reset link or account data leaks. Instructions are cacheable; artifacts aren't.
- **Q3:** After an embedder upgrade, hits fall 27% → 2% and odd false hits appear (shipping question gets a password-reset answer). → Different models produce **different coordinate systems**, so cosine similarity (how closely two vectors point the same way) across them is meaningless: new queries land far from old keys (misses), and occasional nonsense scores clear the threshold (false hits). Tag embeddings with model version and **never compare across versions**; re-embed stored queries in batch or run two indexes (old serves, new fills), then cut over: expand → dual-run → verify → contract. **An embedding model is a schema**; upgrading it is a migration.

Related patterns: Cache-Aside (all laws carry over; only equality changes); Cache Stampede Prevention (single-flight by meaning); LLM Gateway (where it plugs in); Vector Database Sharding (the index past one node); CDN (private/no-store in embedding space).

TL;DR

- *Problem:* 40% of questions reword known ones, but exact keys catch 4%, so you pay to regenerate answers you have.
- *Mechanism:* embed, nearest-neighbor search, serve above a cautious threshold; entity guards, topic tags, TTLs, sampled audits; at the gateway behind an exact tier.
- *Costs:* false hits (measured, not configured), embedder + index on the hot path (fail open), fuzzy invalidation, keys tied to one embedder.
- *Skip it when:* answers are personal, variety is the product, repetition is low, or no false hits are tolerable.

Vector Database Sharding

Original lesson: Vector Database Sharding on DesignGurus ↗

What it is. A vector database stores embeddings and answers "which vectors are closest to this one?" Past one machine's RAM, the index must be sharded. The difference from ordinary sharding: a nearest-neighbor query carries **no shard key**, so by default it asks *every* shard and merges (scatter-gather) — unless the domain supplies a filter, such as tenant ID (the customer account owning the data), that can act as a key.

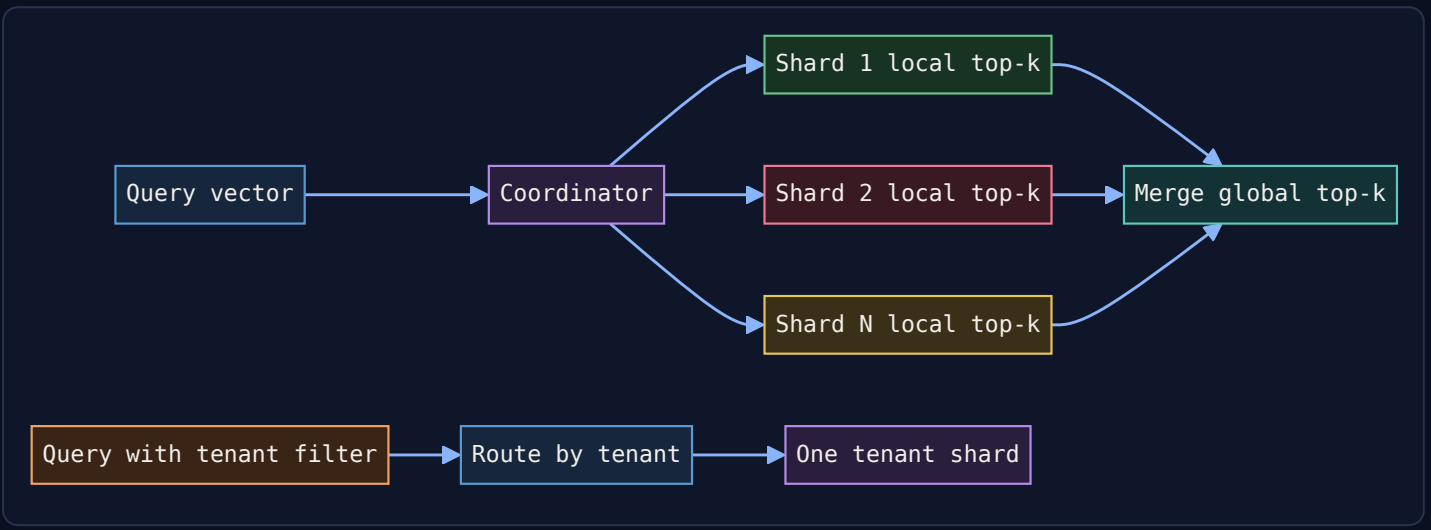
Analogy. Books spread over 12 buildings by author: "books by Tolkien" goes to one building, but "books *similar to* The Hobbit" means asking all 12 librarians and merging — unless you only search your own company's private library, which sits in one building.

The problem. Each number in an embedding is a *dimension*. **500M embeddings × 1,536 dimensions × 4 bytes ≈ 3 TB** of raw vectors, plus ANN index overhead in RAM (*ANN*, approximate nearest neighbor: an index such as **HNSW** that finds nearby vectors without checking each one). No single machine holds it. In Module 3 every query carried a key (`customer_id`) routing it to one shard; "find the 10 nearest vectors" names none. **Proximity is a result, not an address**, so neighbors may live on any shard.

How it works: three ways to split

Top: a coordinator fans the query to all N shards, each returns a local top-k, and a merge picks the global top-k. Bottom: a tenant-filtered query routes straight to one shard.

Default scatter-gather vs tenant-keyed routing



1. **Even shards + full scatter-gather (default).** Spread vectors randomly over N shards; each query hits all N; each returns its local *top-k* (k nearest it holds); a coordinator merges N×k candidates into the global top-k. Correct and predictable, but every query costs N shard queries and the slowest shard sets latency — mitigate with **hedged requests** (also query a slow shard's replica, take the first reply).
2. **Partition by metadata / tenant.** In a multi-tenant product (one system, many customer *tenants*), "search your documents" is always scoped to one tenant. **A filter present in every query can be the shard key**, restoring single-shard routing and all of Module 3, including **dedicated shards for giant tenants**. The most common production design.
3. **Partition by vector geometry (centroid routing, IVF-style).** Cluster the space around *centroids* and send each query only to the nearest clusters, probing **2–3 shards instead of 12**. Costs: **recall** (fraction of true nearest neighbors returned) becomes probabilistic, since a true neighbor may sit in an unprobed cluster; popular topics form **hot clusters** (hot shards).

Three more Module 3 ideas carry over:

- **Replication:** each shard runs **primary-replica**, since RAG retrieval is overwhelmingly reads.
- **The index is a read model** (a rebuildable copy shaped for reading); the source of truth is the documents plus the embedding-model version. An embedder upgrade is a **dual-index rebuild and cutover**, never an in-place edit (an embedding model is a schema).
- **Filtered search** ("similar AND in stock AND under $50") is awkward for ANN. *Pre-filtering* narrows candidates before the geometric search; *post-filtering* removes results after and may return fewer than k. Engines mix both.

Trade-offs

- *You gain:* indexes beyond one machine's RAM (the only route to 500M+ vectors); tenant isolation with single-shard routing where the domain allows; read-model freedom (rebuilds, re-embeddings and experiments on a disposable copy).
- *You pay:* scatter-gather as the default cost model (query cost × shards; p99 set by the slowest shard; hedging and replicas as permanent mitigations); a recall trade-off that must become a *measured* SLO; merge machinery (normalizing scores across shards) and rebalancing as shards fill (**consistent hashing**: adding a server moves only a small share of keys); skew from hot clusters or giant tenants.

When not to use it

- *Under one machine's RAM:* tens of millions of vectors fit on one big node with replicas — scale up before out.
- *When quantization fixes it:* fewer bytes per vector (int8, i.e. 1-byte integers; binary; product quantization/PQ, compact codes for groups of dimensions) cuts memory **4–32x** for a small recall loss and can delay sharding for years. **Compress before adding machines.**
- *When the real problem is one giant tenant:* give it its own shard.
- *Classic misuse:* hash-sharding by document ID like an orders table — every similarity query still hits all 12 shards.

Comparison: the three splits

	Even + scatter-gather	Tenant/metadata-keyed	Centroid-routed (IVF)
Query fan-out	All N shards	One shard	2–3 nearest clusters
Recall	Exact (setting aside per-shard ANN)	Exact within tenant	Probabilistic: a tunable dial
Skew risk	None (random spread)	Giant tenants	Hot clusters
Fits	Global search, no natural filter	Multi-tenant products (usual case)	One enormous latency-critical corpus

Routing rule: if every query carries a filter, shard on it; otherwise scatter-gather with hedging; route by geometry only for one enormous corpus, with the recall trade-off explicit.

In the interview. "Your RAG index outgrew one machine" follows every RAG design. The test: do you see that similarity queries have no shard key?

30-second answer: "Half a billion embeddings is terabytes of RAM, so the index must split. A nearest-neighbor query has no key, so the default is scatter-gather: even shards return local top-k, a coordinator merges, hedging handles slow shards, replicas carry the read load. But first I check whether every query is tenant-scoped; if so, I shard by tenant to get single-shard routing back, with dedicated shards for the biggest tenants. For one enormous corpus, centroid routing probes the nearest clusters, trading exact recall for less fan-out. The index is a read model, so re-embedding means building a second index and cutting over."

Likely follow-ups:

- **"Why not route like the orders table?"** `customer_id = 8412` is a key; "nearest to this point" is a property only computable by comparing candidates, which are everywhere. No key, no routing — unless the domain supplies a filter.
- **"How do you upgrade the embedder on a 12-shard index?"** Build v2 beside v1 by re-embedding from source documents, run both and compare retrieval quality on live queries, cut over per tenant or all at once, retire v1. Version tags prevent cross-coordinate-system comparisons.

Mistakes that fail candidates: hash-sharding while assuming keyed routing; omitting the merge; ignoring the RAM maths; re-embedding in place.

Self-check:

- **Q1:** Why does similarity search break keyed routing; what are the three strategies and fan-outs? → The query names no key; proximity is a result, so neighbors may be on any shard. Even + scatter-gather: fan-out N, exact, default. Tenant/metadata-keyed: fan-out 1 when every query has the filter. Centroid-routed: fan-out 2–3, probabilistic recall.
- **Q2:** A B2B SaaS (software sold online to businesses) runs RAG over each customer's private documents: 4,000 tenants, 3 enormous. → Shard by tenant; pack small tenants many per shard by volume; give the 3 giants **dedicated shards**; replicas per shard. Per-tenant indexes also make deletion/**compliance** (legal and contractual data rules) one clean purge.
- **Q3:** A centroid-routed index (probing 2 of 16 clusters) has poor recall on niche technical topics and two shards at 6x load. → One cause: clusters follow *semantic density*, not load. Niche topics form small sparse clusters whose true neighbors fall just outside the probed set; mainstream topics form huge hot ones. Dials: raise probe count (per query, for low-confidence matches); split hot centroids and merge sparse ones (routine re-clustering); more replicas for hot shards.

Related patterns: Sharding (keyless queries invert it; tenant filters restore it); Primary-Replica (per-shard read scaling); CQRS (index as disposable read model); Consistent Hashing (placement and rebalancing).

TL;DR

- *Problem:* 500M embeddings need terabytes of RAM, and nearest-neighbor queries have no shard key.
- *Mechanism:* even shards with scatter-gather, global top-k merge, hedging, per-shard replicas; escape via tenant/metadata keys (common) or centroid routing (a recall dial). The index is a rebuildable read model.
- *Costs:* query cost × shards, slowest-shard p99, probabilistic recall, merge machinery, hot clusters and giant tenants.
- *Skip it when:* it fits one machine (quantize first) or the real issue is one giant tenant.

RAG Pipeline

Original lesson: RAG Pipeline on DesignGurus ↗

What it is. RAG (retrieval-augmented generation) answers from *your current documents* instead of what the model memorized in training. Per question, it fetches relevant document pieces the user may see, hands them to the model, and asks for a cited answer grounded in them. It is **two pipelines**: continuous *ingestion* keeping an index of chunks current, and a per-request *query* pipeline.

Analogy. An open-book exam: instead of trusting year-old memory (weights), give the student today's relevant textbook pages — only ones they're cleared to read — and require citations.

The problem (the incident). The assistant tells a customer they have **60 days** to return an item; last quarter the policy became **30 days**. The customer returns on day **45**, the dispute escalates, and the company pays. The model worked; its knowledge was stale. Knowledge in **weights** has three built-in limits no prompt fixes:

- **No updates:** the policy changed; the weights didn't.
- **No citations:** it can claim but not point to a source.
- **No access control:** what it knows, it knows for everyone.

The fix: **look up current documents at question time instead of relying on weights.**

Why the obvious fixes fail

- *"Fine-tune on our documents."* Fine-tuning (further training on your data) runs on a training schedule, so the new policy waits weeks; it gives no citations, weights can't check permissions, and training doesn't store facts reliably. Use **fine-tuning for behavior** (tone, format, domain reasoning style), retrieval for knowledge.
- *"Paste every document into the context"* (everything sent in one request). A **500K-token** knowledge base costs dollars per question even with discounts, adds latency, and models miss facts buried mid-context. A small, stable corpus (a 50-page handbook) *can* use long context with prompt caching.
- *"Let the model search the wiki/web itself"* (agentic retrieval: the model decides what and when to search). Flexible but harder to control: each search adds latency, quality depends on the model's queries, and governance must decide what it may read, cite or reveal. Agents should call the controlled pipeline below.

How it works: two pipelines

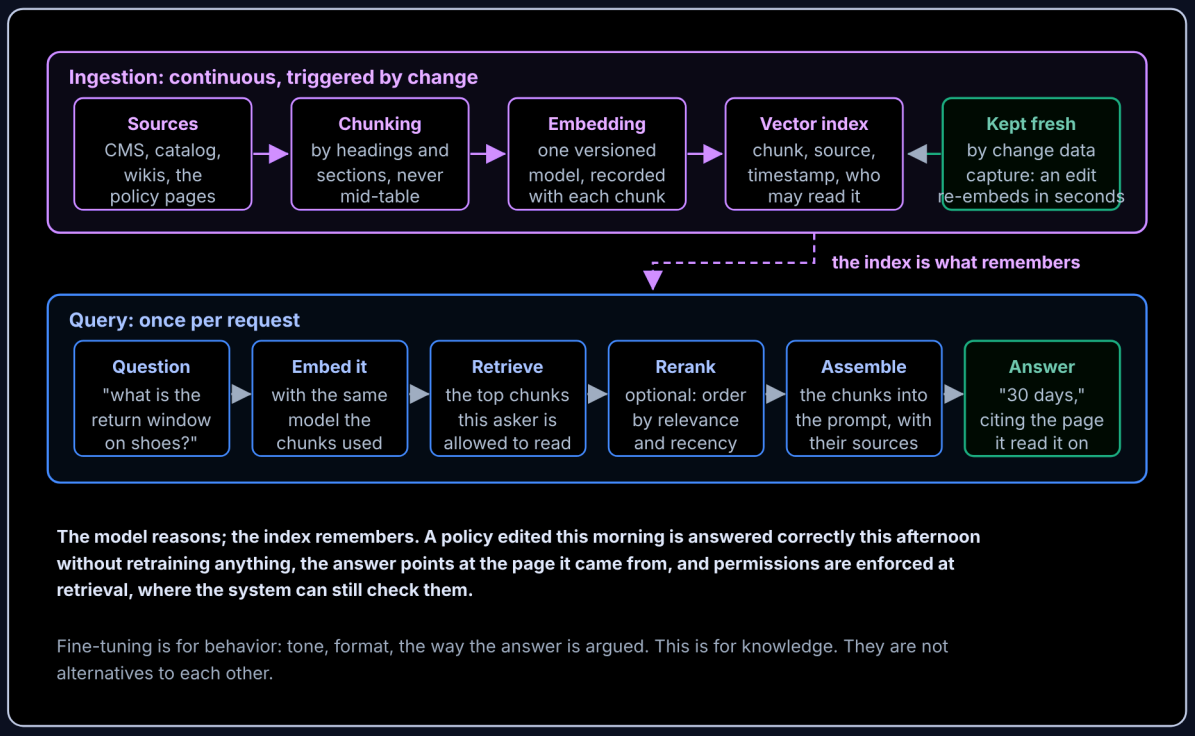


Figure — RAG: two pipelines (course diagram)

Ingestion: sources → chunking → embedding → ACL-tagged vector index, kept current by CDC. Query: cache → embed → permitted-chunk retrieval → rerank → generation.

Ingestion (always running):

1. **Chunk the documents** into retrievable pieces. Too big drags unrelated text into prompts; too small loses context; splitting mid-table makes nonsense. Use **structure-aware splitting** (headers, sections, semantic boundaries, with overlap). Chunking **sets the quality ceiling**: no reranker rescues bad chunks.
2. **Embed every chunk.**
3. **Index** vectors in the sharded vector database with metadata and **ACL tags** (an access control list records who may read each document).
4. **Stay fresh with CDC.** A connector watches the CMS and catalog tables; on edit, *only changed pieces* are re-chunked and re-embedded within seconds. No nightly recrawl, no accumulating staleness, bulk imports flow through too — another 60-day incident is prevented by design.

Query (per request):

1. **Check the semantic cache.**
2. **Embed the question** with the *same* model that embedded the chunks (different models, different coordinate systems).
3. **Retrieve with access control.** Search the sharded index (tenant-routed where applicable) for top-k candidates (e.g. top-50), **filtered by the requester's ACL tags at retrieval time**. Telling the model "don't reveal confidential info" is **not** access control; authorization must happen before content enters the context.
4. **Rerank.** Vector search is a fast, broad recall stage; a **cross-encoder reranker** reads query and chunk *side by side* — slower, more accurate — and picks the best **5** in **~40 ms**. Cheap filter then costly judge beats one stage.
5. **Build the prompt** from chunks, sources and question; instruct the model to answer only from context, cite sources, and say "I don't know" when uncovered. Citations are the **audit trail** for disputes.
6. **Generate** via the LLM gateway (fallbacks, budgets) and stream over SSE.

Eval discipline: measure stages separately. **Retrieval quality** — did we fetch the right chunk? — via **recall@k** (fraction of correct chunks in the top k). **Generation faithfulness** — did the answer really come from those chunks? Separate failures, separate fixes. Most "the bot was wrong" tickets are **retrieval failures dressed up as generation failures**.

Every part is an earlier pattern: ingestion = CDC + stream processing + index as read model; query = semantic cache, sharded retrieval, gateway, batched serving on autoscaled GPUs, SSE.

Walkthrough with numbers

- *Incident replayed:* edit hits the CMS at **14:02:11**, CDC emits at **14:02:15**, three chunks re-indexed by **14:02:19**: **~8 seconds** edit-to-truth vs. weeks for fine-tuning.
- *Latency budget:* cache check **10 ms** (answers **27%**). On a miss: embed **10 ms** + sharded retrieval **25 ms** + rerank **40 ms** + generation **~2 s** — retrieval adds **~75 ms to a 2,000 ms answer (<4%)**.
- *Economics:* retrieval costs **~1/10** of generation, and external knowledge can let a *smaller* model match a bigger one, spending capacity on reasoning rather than memorized facts.
- *Eval dashboard:* **recall@5 = 91%**, **faithfulness = 96%**, alarmed separately, so a drop answers "which stage?"

Trade-offs

- *You gain:* updatable knowledge (edit the doc, the assistant knows in seconds); citations for trust, audits and disputes ("section 3, updated March 4th"); real per-asker access control at retrieval; the option of smaller models.
- *You pay:* **retrieval caps quality** (garbage in, faithful garbage out; chunking, embedding and reranking are permanent skills); **two pipelines to run** (CDC health, re-embedding lag, replication-slot disk on ingestion; a four-stage latency budget on query); **hallucination reduced, not removed** (a *hallucination* is a claim unsupported by context; the model can still ignore it); **permanent two-stage evals**.

When not to use it

- *Behavior problems* (tone, format, style, reasoning): fine-tuning.
- *Small, stable corpora:* a 50-page handbook fits in context; prompt caching makes the repeated prefix cheap.
- *General knowledge* ("capital of France?"): parametric memory (knowledge in the weights) handles stable public facts.
- *Latency budgets under ~100 ms:* no time to embed, retrieve, rerank and generate.
- *Classic misuse:* reflexive RAG when 12 PDFs fit in one prompt. Rule: **RAG for knowledge, fine-tuning for behavior, long context for small stable corpora**, combined when needs overlap (a fine-tuned model fed RAG context is a common enterprise design).

Comparison: RAG vs. fine-tuning vs. long context

	RAG	Fine-tuning	Long context (paste it all)
Knowledge freshness	Seconds (CDC-fed)	Training schedule: weeks	Fresh (re-paste on change)
Citations	Built in: chunks carry sources	None	Possible, unstructured
Access control	Real: ACLs at retrieval	Impossible inside weights	All or nothing per prompt
Cost profile	Index + ~10% per query	Training runs + hosting	Tokens × corpus × every query
What it's for	Knowledge: large, changing, access-controlled	Behavior: tone, format, reasoning style	Small stable corpora (with prompt caching)

Layers, not competitors: fine-tuning shapes the voice, RAG supplies current facts, prompt caching makes a stable preamble cheap.

Real-world examples

- **Every enterprise assistant** (support bots, internal knowledge tools, **Glean**-class products): RAG is the standard shape of "LLM + your data."
- **Code assistants** use the repository as the **corpus** (body of documents searched), chunked along function and file structure.
- **Agentic retrieval** adds query rewriting, multi-hop retrieval (find a policy, then its exceptions), and model-decided retrieval, calling the controlled pipeline via a **planner** (the agent component choosing the next step).

In the interview. "Design a chatbot over company documents" is *the* signature AI design question; follow-ups are freshness, permissions, and debugging bad answers.

30-second answer: "RAG is two pipelines. Ingestion watches sources via CDC, so an edit re-chunks and re-embeds only changed pieces within seconds; structure-aware chunking, because bad chunks cap everything downstream; output to a sharded vector index with metadata and ACL tags. The query path checks the semantic cache, embeds the question, retrieves top-k filtered by the requester's permissions so the model never sees what the user can't, reranks with a cross-encoder, and generates a cited answer through the gateway. I evaluate recall@k and faithfulness separately and debug retrieval first. RAG for knowledge, fine-tuning for behavior, long context for small stable corpora."

Likely follow-ups:

- **"A user asks about a document they can't access."** The retrieval filter excludes it; its chunks never reach the candidate set or the model, and the answer is an honest "I don't have information on that." Enforcing this by prompt instruction fails: instructions to a text generator aren't access control.
- **"Answers got worse last week. Debug it."** Separate stages and check **retrieval first**: recall@k on a fixed eval set with known correct chunks. If fine, check faithfulness (ignoring context?). Then chunking changes, embedder version drift, and ACL config emptying candidate sets.

Mistakes that fail candidates: fine-tuning for freshness; ACLs by prompt; no citation plan; dismissing chunking as detail; one quality metric for both stages.

Self-check:

- **Q1:** Name the two pipelines and stages; what keeps ingestion fresh without recrawling? → Ingestion: sources → structure-aware chunking → embedding → sharded index with metadata + ACL tags, kept fresh by CDC reprocessing only changed chunks within seconds. Query: semantic cache → embed → ACL-filtered top-k → cross-encoder rerank → prompt with citation rules → generation via gateway, streamed.
- **Q2:** Choose: (a) a 50-page handbook updated yearly; (b) a changing 500K-document knowledge base with per-team permissions; (c) "answer in our brand voice." → (a) long context + prompt caching; (b) full RAG (CDC freshness, retrieval-time ACLs, citations); (c) fine-tuning (voice is behavior). Combinable: a fine-tuned model fed by RAG.
- **Q3:** Confidently wrong warranty answers, yet faithfulness is 96%. → The model faithfully uses the *wrong* chunks: a retrieval failure looking like a generation failure. Run recall@k on a warranty eval set; check chunking shredding warranty tables, an embedder upgrade without re-indexing (cross-coordinate retrieval), CDC lag or stall (stale index), misconfigured ACL tags filtering out the right chunks. **Generation is downstream of retrieval, so grade retrieval first.**

Related patterns: Change Data Capture (fresh ingestion); Vector Database Sharding (the index at scale, tenant-keyed); Semantic Caching (front tier; 27% never reach retrieval); LLM Gateway (generation's door).

TL;DR

- *Problem:* knowledge in weights can't be updated, cited or access-controlled, and fine-tuning fixes none quickly.
- *Mechanism:* CDC-fed ingestion (structure-aware chunking → embedding → sharded ACL-tagged index) plus a query funnel (cache → ACL-filtered retrieval → rerank → cited generation via gateway), evaluated per stage (recall@k, then faithfulness).
- *Costs:* retrieval sets the ceiling, two pipelines, hallucination reduced not removed, two-stage evals.
- *Skip it when:* the problem is behavior, the corpus is small and stable, or the knowledge is public and general.

Capstone — A Production RAG Chatbot at Scale

Original lesson: Capstone - A Production RAG Chatbot at Scale on DesignGurus ↗

What this capstone is. Design the store's production assistant for **~5 million users** plus B2B partners, where every request costs money — and test the module's thesis by naming **which earlier module supplies each mechanism**. (The course asks you to pause at each "diagnose" prompt and decide before reading on.)

The brief (requirements)

- **Scale: 2,000** concurrent conversations at peak, **30 requests/second** to models; multi-tenant: the store's customers plus **200 B2B partners** embedding the assistant under their brands.
- **Knowledge:** catalog + policies (RAG), plus live order status (personal state — a different problem).
- **Experience:** streamed, cited answers; conversations survive reconnects.
- **Money:** partners billed by usage, internal budgets, and the **$40K** runaway loop must never recur.
- **Compound requirement:** stay useful through a provider brownout *during* a traffic spike.

Stage 1 — Knowledge plane. *Challenge:* content changes constantly; each partner's knowledge base is private.

- **Decision:** CDC on the CMS and catalog tables; edits re-chunked (structure-aware) and re-embedded in seconds — no recrawls (Module 10).
- **Decision:** index **sharded by tenant**; every partner query carries a tenant filter, so retrieval hits one shard, and the **2 largest partners get dedicated shards**. Not uniform scatter-gather, because the filter is a natural shard key (Module 3 sharding).
- **Decision: ACL tags** on every chunk; retrieval-time filtering is the security boundary — prompts aren't access control.
- **Decision:** the index is a rebuildable read model (CQRS, Module 3); the embedder is versioned like a schema with **dual-index migrations** (zero-downtime playbook, Module 9). Four modules before the first question.

Stage 2 — Conversation plane. *Challenge:* token-by-token streaming, reconnects after network drops, follow-ups needing history.

- **Decision: SSE** (one-way server→client over plain HTTP, auto-reconnect with `Last-Event-ID`). Not WebSocket (a persistent two-way connection): the client sends nothing mid-answer (Module 2).

- **Decision: sticky routing by conversation ID on the consistent-hash ring**, so later turns reach the warm KV cache — Constant 2's stickiness via Module 3's ring (only ~1/N of conversations move when replicas change).
- **Decision: the stream is transport; the history store is truth.** Finished messages are persisted; after a disconnect the server replays from history, not leftover stream state (Module 2's resume design). No new patterns.

Stage 3 — Serving plane. *Challenge:* 30 req/s, 100x output variance, the $40K incident, a marketing calendar. One request, front to back:

1. **Semantic cache** answers **>25%** in **40 ms** (cautious threshold, entity guards, sampled audits — Module 4 with fuzzy keys).
2. Misses go to the **LLM gateway**: provider keys, **per-tenant token budgets with hard caps**, logical model routing, fallback ladder (Module 8 gateway + rate limiting; Module 5 degradation).
3. The **self-hosted fleet** runs continuous batching (the measured **30x** saving), overflowing to provider APIs via the gateway.
4. The fleet scales on **queue depth** with **warm pools sized at spike rate × boot time**, pre-warmed from marketing's calendar (Module 6 at 100x prices). Six modules in one request path.

Stage 4 — Money plane. *Challenge:* partners pay per token — where does metering live, and what guarantee does each consumer need?

- **Decision:** the gateway (the one point every call crosses) meters each call (tokens in/out, model, tenant, feature) and publishes usage events to a **log partitioned by tenant** (Module 2).
- **Decision:** live spend **dashboards** aggregate with **idempotent upserts** — forgiving and fresh.
- **Decision: invoices** use **exactly-once** machinery (checkpointed aggregation, transactional sinks) because partners audit bills (Module 10: pick the guarantee per consumer).
- **Decision:** event IDs are minted **on the client before first send**, an unbroken idempotency chain from SDK to invoice (Module 5).

Stage 5 — Personal-state boundary. *Challenge:* "Where's MY order?" is the top question, uncacheable, and must never enter the RAG index; and what about "cancel that order"?

- **Decision:** personal state goes through a **tool call** (`get_order_status(order_id)`): an ordinary request-response API reads authoritative state from the orders system; the model only formats it. The index holds shared knowledge; source systems hold personal state (Module 2).
- **Decision:** "cancel it" is a consequential write, so the tool call carries an **idempotency key derived from the tool-call ID** (frameworks retry; agents change their minds) and enters the orders domain's own **saga** (multi-step flow with compensations): the assistant starts it, the domain orchestrates. Rule (the BFF rule for agents): **presentation composes reads; domains own writes** (Modules 5, 7).

Stage 6 — The bad day (failure walk). Tuesday 11 AM: marketing's segment airs, traffic triples (**3x**); at 11:04 the primary provider browns out.

1. **11:00:** the calendar already pre-warmed the pool; the surge lands on ready GPUs while queue-depth scaling boots replacements.
2. **11:04:** per-provider-model breakers open after **~20 s** of errors; overflow goes to the **secondary provider**, `classify` -tier work to the **self-hosted small model**.
3. **The semantic cache carries the most:** already ~25%, it absorbs the trending questions every spike brings ("is the sale still on?") — one generation serves **10,000** askers.
4. **Per-tenant budgets** stop failover from doubling anyone's spend.
5. **Users see** slightly slower answers and one labeled banner.

6. **No human decisions:** every branch was configuration written on a quiet afternoon (degradation tiers, gateway chains, autoscaler calendar); on-call watches dashboards confirm the plan.

Final architecture

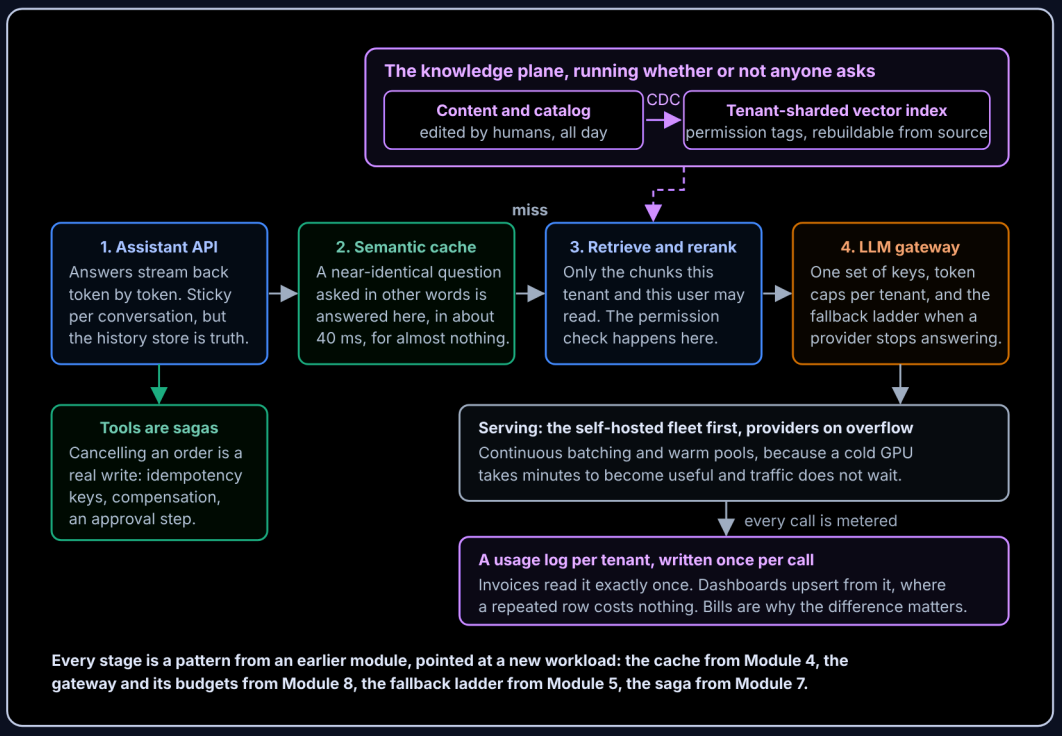


Figure — RAG chatbot at scale: final architecture (course diagram)

The planes together: CDC-fed, tenant-sharded knowledge index; SSE conversation layer with ring stickiness and a history store; serving path semantic cache → gateway → batched GPU fleet with API overflow; metering log feeding dashboards and invoices; tool calls into the orders domain.

Debrief: the tally

Plane	Decisions	Modules doing the work
Knowledge	CDC freshness, tenant shards, ACL-filtered retrieval, dual-index migrations	10, 3, 9
Conversation	SSE + resume, ring stickiness, history as truth	2, 3
Serving	Cache → gateway → ladder → batching → warm pools	4, 8, 5, 6, 11
Money	Metering on a partitioned log; upserts or exactly-once per audience	2, 10, 5
Personal state	Tool calls, idempotency keys, saga boundaries	2, 5, 7
The bad day	Breakers, tiers, calendars: configuration instead of war rooms	5, 6, 8, 9

The right column covers **every module except the capstones**. The truly AI-specific pieces are few — embeddings, similarity thresholds, token meters — supporting the thesis: AI systems reuse known patterns under three changed constants.

Extend it yourself (follow-up questions)

- **Agents:** multi-step tasks ("find a cheaper alternative and set a price alert"). Which boundaries do tool calls strain most (saga pivots? budget caps?), and what would **durable execution** look like (a workflow engine persisting each step so long tasks resume after a crash)?
- **Voice:** what transport replaces SSE (bidirectional streaming)? What happens to the latency budget when TTFT becomes time-to-first-*syllable*? Which plane is unchanged?

- **Multi-region:** partners demand EU **data residency** (EU data stored and processed in the EU). Which planes replicate without coordination? Where does the tenant-sharded index's compliance design help most? Which component (the billing log) needs the strongest guarantees from the consistency module?

Interview one-liner: "Knowledge enters via CDC into a tenant-sharded, ACL-tagged index. Conversations stream over SSE with history as truth and ring stickiness for KV caches. Requests flow semantic cache → gateway with hard budgets and a fallback ladder → a batched, warm-pooled GPU fleet overflowing to APIs. Metering rides a log, exactly-once for invoices. Personal state and actions go through idempotent tool calls into domain sagas. Every bad-day response is configuration written in advance."